

SEMA SIMAI Springer series 30

Rubén Sevilla
Simona Perotto
Kenneth Morgan *Eds.*

Mesh Generation and Adaptation

Cutting-Edge Techniques

SEMA

SIMAI
SOCIETÀ ITALIANA DI MATEMATICA
APPLICATA E INDUSTRIALE



Springer

SEMA SIMAI Springer Series

Volume 30

Editors-in-Chief

José M. Arrieta, Departamento de Análisis Matemático y Matemática Aplicada, Facultad de Matemáticas, Universidad Complutense de Madrid, Madrid, Spain

Luca Formaggia , MOX–Department of Mathematics, Politecnico di Milano, Milano, Italy

Series Editors

Mats G. Larson, Department of Mathematics, Umeå University, Umeå, Sweden

Tere Martínez-Seara Alonso, Departament de Matemàtiques, Universitat Politècnica de Catalunya, Barcelona, Spain

Carlos Parés, Facultad de Ciencias, Universidad de Málaga, Málaga, Spain

Lorenzo Pareschi, Dipartimento di Matematica e Informatica, Università degli Studi di Ferrara, Ferrara, Italy

Andrea Tosin, Dipartimento di Scienze Matematiche “G. L. Lagrange”, Politecnico di Torino, Torino, Italy

Elena Vázquez-Cendón, Departamento de Matemática Aplicada, Universidade de Santiago de Compostela, A Coruña, Spain

Paolo Zunino, Dipartimento di Matematica, Politecnico di Milano, Milano, Italy

As of 2013, the SIMAI Springer Series opens to SEMA in order to publish a joint series aiming to publish advanced textbooks, research-level monographs and collected works that focus on applications of mathematics to social and industrial problems, including biology, medicine, engineering, environment and finance. Mathematical and numerical modeling is playing a crucial role in the solution of the complex and interrelated problems faced nowadays not only by researchers operating in the field of basic sciences, but also in more directly applied and industrial sectors. This series is meant to host selected contributions focusing on the relevance of mathematics in real life applications and to provide useful reference material to students, academic and industrial researchers at an international level. Interdisciplinary contributions, showing a fruitful collaboration of mathematicians with researchers of other fields to address complex applications, are welcomed in this series.

THE SERIES IS INDEXED IN SCOPUS

Rubén Sevilla • Simona Perotto • Kenneth Morgan
Editors

Mesh Generation and Adaptation

Cutting-Edge Techniques

 Springer

Editors

Rubén Sevilla
College of Engineering
Swansea University
Swansea, UK

Simona Perotto
Dipartimento di Matematica
Politecnico di Milano
Milan, Italy

Kenneth Morgan
College of Engineering
Swansea University
Swansea, UK

ISSN 2199-3041

ISSN 2199-305X (electronic)

SEMA SIMAI Springer Series

ISBN 978-3-030-92539-0

ISBN 978-3-030-92540-6 (eBook)

<https://doi.org/10.1007/978-3-030-92540-6>

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2022

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

Foreword

This volume in the SEMA SIMAI Springer Series, entitled *Mesh Generation and Adaptation: Cutting-Edge Techniques*, is dedicated to Professor Oubay Hassan, on the occasion of his 60th birthday.

Oubay Hassan was born in Damascus, Syria, on May 3, 1960. He obtained his first degree in civil engineering from the University of Damascus in 1983. In 1985, he enrolled in the MSc course on the finite element method in the Department of Civil Engineering at Swansea University. He has remained at Swansea ever since.

As a practicing civil engineer, he was naturally attracted initially to the area of structural mechanics, and for his MSc thesis, he worked on the solution of non-linear problems involving reinforced concrete plates and shells [1]. However, he moved to a different area for his PhD studies, as he became interested in the unstructured mesh CFD research which was being carried out in the Department. Oubay addressed the problem of compressible viscous high-speed flow simulations, and this, with the accompanying difficulties of creating suitable meshes, initiated his interest in mesh generation and adaptivity. During his PhD studies, he combined the development of original ideas with skillful computer implementations. He became adept at manipulating unstructured meshes and devised a novel algorithm for creating continuous lines, made up of element sides, which pass once through each node of a general unstructured mesh. He was then able to use these lines as the basis for an implicit solution procedure in which the solution was achieved by line relaxation [2].

Although his initial work used the advancing front method, he made a major contribution to mesh generation with his research on the application of Delaunay



triangulation procedures to general three-dimensional configurations. This work resulted in a robust technique for creating a valid boundary conforming mesh of unstructured tetrahedra, with automatic point creation, for domains of arbitrary geometric complexity [3].

Following his appointment to the academic staff at Swansea, Oubay has continued to provide leadership, within what is now the Zienkiewicz Centre for Computational Engineering, on the development of CFD schemes able to operate effectively on unstructured grids. During this time, he has developed and enhanced his mesh-generation procedures [4] to such an extent that his advice and assistance is now regularly sought by companies and organizations in Europe, the USA, and the Far East.

The flexibility and generality of the mesh-generation tools that he has developed provided him with the possibility of making additional important contributions in the field of computational electromagnetics [5], as well as addressing various complex problems in CFD.



At his PhD award ceremony at Swansea with some distinguished colleagues.



Enjoying a coffee during a break at a conference in South Africa.

Oubay's research has been recognized in a number of different ways. In the 1990s, he used his CFD techniques to assist in the aerodynamic design process for the Thrust supersonic car. This car eventually took the World Land Speed Record beyond the speed of sound in October 1997. For his contribution to this project, Oubay was appointed a Member of the Most Excellent Order of the British Empire (MBE) by Queen Elizabeth. In 2012, he was appointed to the Fellowship of the UK Royal Academy of Engineering (FREng) and also to the Fellowship of the Learned Society of Wales (FLSW).

This volume represents a compilation of invited papers in the general area of mesh generation and adaptation, the research field in which Oubay has made the most profound and enduring contributions. The quality of the work and the range of material presented in these papers make this volume a fitting tribute to Oubay on the occasion of his 60th birthday.

MOX, Dipartimento di Matematica, Politecnico di Milano
Faculty of Science and Engineering, Swansea University

Luca Formaggia
Kenneth Morgan

References

1. Cervera, M., Hinton, E., Hassan, O.: Nonlinear analysis of reinforced concrete plate and shell structures using 20-noded isoparametric elements. *Comput. Struct.* **25**, 845–869 (1987)
2. Hassan, O., Morgan, K., Peraire, J.: An implicit finite–element method for high speed flows. *Int. J. Numer. Methods Eng.* **32**, 183–205 (1991)
3. Weatherill, N.P., Hassan, O.: Efficient three–dimensional Delaunay triangulation with automatic point creation and imposed boundary constraints. *Int. J. Numer. Methods Eng.* **37**, 2005–2039 (1994).
4. Xie, Z.Q., Sevilla, R., Hassan, O., Morgan, K.: The generation of arbitrary order curved meshes for 3D finite element analysis. *Comput. Mech.* **51**, 361–374 (2013)
5. Xie, Z.Q., Hassan, O., Morgan, K.: Tailoring unstructured meshes for use with a 3D time domain co–volume algorithm for computational electromagnetics. *Int. J. Numer. Methods Eng.* **87**, 48–65 (2011)

Contents

Mixed Order Mesh Curving	1
Steve L. Karman, Kristen Karman-Shoemake, and Carolyn D. Woeber	
A R&D Software Platform for Shape and Topology Optimization Using Body-Fitted Meshes	23
C. Nardoni, D. Danan, C. Mang, F. Bordeu, and J. Cortial	
Investigating Singularities in Hex Meshing	41
Dimitrios Papadimitrakis, Cecil G. Armstrong, Trevor T. Robinson, Alan Le Moigne, and Shahrokh Shahpar	
Intercode Hexahedral Meshing from Eulerian to Lagrangian Simulations	69
Nicolas Le Goff, Franck Ledoux, and Jean-Christophe Janodet	
Gmsh’s Approach to Robust Mesh Generation of Surfaces with Irregular Parametrizations	95
Jean-François Remacle and Christophe Geuzaine	
Adaptive Single- and Multilevel Stochastic Collocation Methods for Uncertain Gas Transport in Large-Scale Networks	113
Jens Lang, Pia Domschke, and Elisa Strauch	
HexDom: Polycube-Based Hexahedral-Dominant Mesh Generation	137
Yuxuan Yu, Jialei Ginny Liu, and Yongjie Jessica Zhang	
Mesh Adaptivity in the Framework of the Cartesian Grid Finite Element Method, cgFEM	157
Juan José Ródenas, Enrique Nadal, José Albelda, and Manuel Tur	
<i>h</i>- and <i>r</i>-Adaptation on Simplicial Meshes Using MMG Tools	183
Luca Arpaia, Héloïse Beaugendre, Luca Cirrottola, Algiane Froehly, Marco Lorini, Léo Nouveau, and Mario Ricchiuto	

Geometry and Adaptive Mesh Update Procedures for Ballistics Simulations	209
Saurabh Tendulkar, Fan Yang, Rocco Nastasia, Mark W. Beall, Assad A. Oberai, Mark S. Shephard, and Onkar Sahni	
High-Order Implicit Shock Tracking (HOIST)	233
Andrew Shi, Per-Olof Persson, and Matthew J. Zahr	
Breakthrough ‘Workarounds’ in Unstructured Mesh Generation	261
Rainald Löhner	
An Adaptive Conservative Moving Mesh Method	277
Simone Appella, Chris Budd, and Tristan Pryer	
A Global Optimization and Adaptivity-Based Algorithm for Automated Edge Grid Generation	301
Suzanne M. Shontz and David McLaurin	

Mixed Order Mesh Curving



Steve L. Karman, Kristen Karman-Shoemake, and Carolyn D. Woeber

Abstract Linear hybrid unstructured meshes are elevated to mixed-order meshes in response to geometry curvature. The linear meshes are elevated to the required degree on an element-by-element basis in regions of high geometry curvature. Weighted condition number mesh smoothing is used to untangle and improve the quality of the current mixed-order mesh. Periodically the mesh is tested for additional element elevation using a deviation criterion. Once the mesh smoothing is complete the mesh can be exported as a mixed order mesh or uniformly elevated to the desired degree. Details of the mesh elevation and smoothing process are described. Two three-dimensional examples are included that demonstrate the effectiveness of the method to produce high quality mixed-order meshes.

1 Introduction

High order mesh curving is an emerging technology that will greatly benefit those that utilize Finite-Element Methods (FEM) within the Computational Fluid Dynamics (CFD) solver community. Finite-element techniques offer increased accuracy with lower element counts over traditional CFD methods such as finite-volume and finite-difference methods. The increased accuracy is achieved by introducing additional vertices (new degrees of freedom) to edges, faces and interiors of linear elements. For elements adjacent to curved geometry these new degrees of freedom must lie on the geometry, thereby altering the shape of the original linear element. This process is more difficult when the mesh contains clustering of elements toward viscous boundaries. The edges and faces of interior elements must also be curved in response to the boundary element curvature to prevent element inversion.

S. L. Karman
Oak Ridge National Laboratory, Oak Ridge, TN, USA
e-mail: karmanst@ornl.gov

K. Karman-Shoemake · C. D. Woeber (✉)
Cadence Design Systems, Fort Worth, TX, USA
e-mail: kshoema@cadence.com; cwoeber@cadence.com

Research into mesh curving is taking place at a number of institutions. Radial basis function interpolation was investigated at Imperial College [1] and the University of Kansas [2, 3]. The more prominent mesh curving approaches tend to use solid mechanics analogies where the mesh is treated as an elastic solid that deforms due to forces acting on the boundaries [4, 5]. Other efforts focus on the solution to the Winslow equations to perform the interior mesh curving [6]. This approach is a natural application of Winslow smoothing techniques in the sense that a copy of the unperturbed, elevated mesh serves as the computational mesh. The solution to the Winslow equations then forces the interior of the physical mesh to take on the same character of the computational mesh. A novel mesh optimization approach with edge and face flips for moving P2 meshes and two-dimensional quadratic mesh adaptation to a Riemannian metric were developed by INRIA [7] and Gmsh [8]. Researchers at the Barcelona Supercomputing Center developed a mesh optimization method that attempts to minimize distortion [9]. Pointwise collaborated with researchers from the University of Tennessee Knoxville on an alternate approach for viscous mesh curving using weighted condition number (WCN) smoothing [10]. Subsequent updates to the technique were developed in 2018 and form the basis of the work presented in this chapter [11, 12].

Mesh generation applications with curving capabilities are now available on a limited basis to the CFD community. MeshCurve, developed as part of a Master's degree research project, is available for download [3]. Gmsh is a full featured mesh generation and visualization tool with curving capability to very high order [13]. Nektar++ has a meshing component, NekMesh, that has curving capabilities [14]. Pointwise recently released a version of their mesh generation software with an elevate-on-export capability [15].

Research on the WCN approach used by Pointwise has continued and permits mixed order meshes to resolve geometry curvature. The elements can be elevated to a maximum polynomial degree 4 (quartic) near highly curved geometry while far away from curved geometry, the elements remain linear. The mesh smoothing method uses a cost function to enforce desired element shapes and positive Jacobians across each element. Viscous mesh spacing is maintained as the elements are curved near the geometry. At completion, the mixed order mesh can be exported or a uniformly elevated mesh of the desired degree can be created. Results are shown for complex 3D configurations.

2 Mixed-Order Curving Framework

Elevating linear meshes and curving them in response to surface curvature requires easy access to the geometry and a robust initialization and smoothing process. Surface queries of the geometry are necessary to ensure the high-order nodes are accurately placed on the geometry during initialization and remain on the surface during mesh smoothing. The mesh smoothing process must be robust to ensure a valid mesh is produced that maintains the character of the input linear mesh with

respect to the distribution of nodes normal to the surface in the boundary layer region. Geometry access will be briefly described followed by a detailed description of the mixed order mesh curving process.

2.1 Geometry

Geometry access for elevating and smoothing is provided through the MeshLink API [16]. MeshLink is a library for managing geometry¹ and mesh data and provides a simple interface to query functions pertinent to mesh generation and mesh adaptation applications. Associations between geometric entities within a geometry data file and the mesh elements within a related mesh data file are stored in a MeshLink file. The complete geometry and mesh configuration can then be represented with the MeshLink file (XML), the geometry data file (CAD), and the mesh data file (NMB).

A key benefit of the MeshLink library that is integral to the elevation and smoothing process is the ability to define and use geometry groups. Geometry groups enable a mesh entity to be associated with multiple geometry entities that should be considered for a projection process. For instance, a surface mesh edge may be associated with a geometry face or a specific geometry curve. Alternatively, a surface mesh edge may be associated with several geometry curves. The mesh curving application program's efficiency depends on rapidly being able to query the correct geometry entity for a given mesh node projection without having to keep track of the details of the multiple geometry associations.

As an example, when the mesh curving program starts, the MeshLink API imports the CAD and XML file to create a database in memory that associates the surface elements of the mesh with the CAD entities. During element elevation and mesh smoothing the curving program makes node projection requests from the MeshLink API for nodes at surface mesh edges and faces. The queries include the end nodes of the edges and the corner nodes of the faces. The appropriate geometric entity is used by the library to project the requested query node. All of this is hidden from the mesh curving application program. All that was provided to the MeshLink function was the forming nodes for the mesh entity, edge or face, and the input query physical location. The process is more efficient than projecting to all geometry surfaces and more robust. If the linear mesh topology does not change, which is the case for this implementation of mesh curving, then the nodes are projected to the proper geometry entity. There is no ambiguity about projecting to the wrong surface, such as the lower wing surface from a node on the upper surface near a thin trailing edge.

¹ The intended application of this technique is mesh curving on NURB geometric surfaces but does not preclude use of discrete surfaces.

For the results presented within this chapter, the Pointwise meshing software was used to create the initial linear meshes. At completion the three files required by MeshLink were exported from Pointwise: the linear mesh in CGNS format, the geometry CAD file in NMB format, and the XML MeshLink file.

2.2 Mixed Order Curving Process

Mixed order mesh curving uses a process that begins with a valid linear mesh. The major components of the process are provided in the flowchart in Fig. 1. Within the flowchart and this chapter, note that the order or polynomial degree of an element is indicated using Q1 through Q4 nomenclature. Linear, quadratic, cubic, and quartic elements are Q1, Q2, Q3, and Q4 respectively. The high-order elements use Lagrangian basis functions to evenly distribute high-order nodes across the element's edges, faces and interior. These physical nodes are an integral part of the WCN method to enforce sub-element and element shapes.

The Initialization process, seen in the shaded box on the left in Fig. 1, uses the input linear mesh to begin walking through the element elevation process based on the user requested final degree, Q_{final} . The initialization process elevates elements in the mesh to the next higher degree depending on the deviation metric evaluation performed in Deviation Metric Testing process (see shaded box in the middle in Fig. 1). The process first elevates surface elements (2.2.1) and volume elements

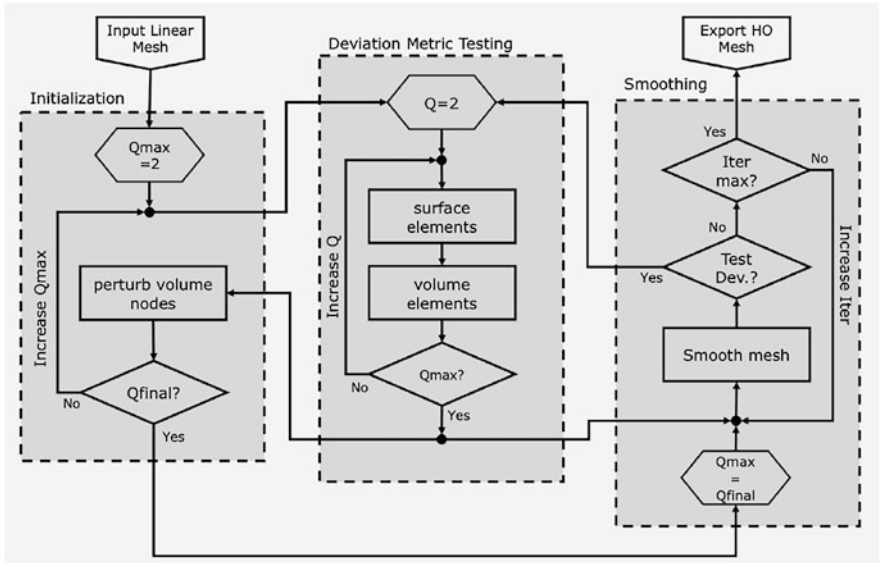


Fig. 1 Flowchart of the mixed order mesh curving process

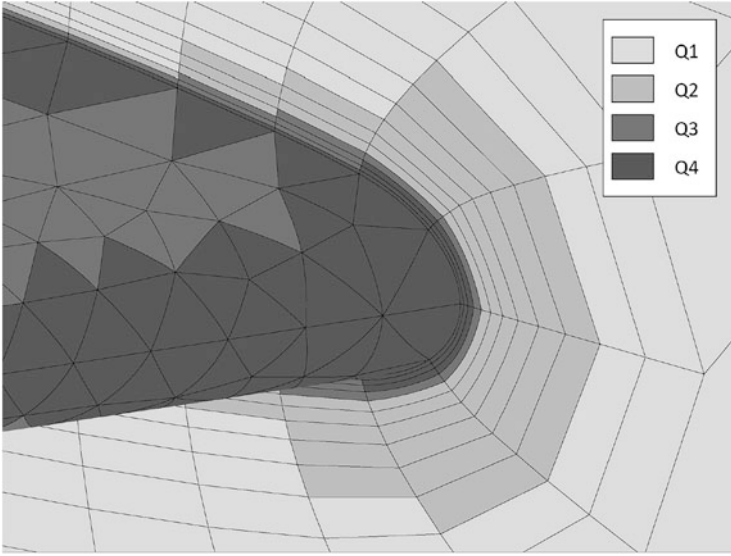


Fig. 2 Leading edge of the Onera M6 wing at the symmetry plane

(2.2.3) to Q2. The new boundary nodes are placed on the geometry surface and the perturbations of these nodes from the initial linear surface are propagated into the interior using a simple transfer process. The initialization continues to Q3 and possibly Q4,² if requested, using the same deviation metric testing process and interior node perturbation process. Once the bootstrapping process is complete a mixed order mesh is produced that may include invalid elements near highly curved geometry.

The Smoothing process, seen in the shaded box on the right in Fig. 1, uses the WCN mesh smoothing method to correct any element inversions and improve the quality of the elements produced by the initialization process. Periodically each volume element is measured using the deviation metric testing in the middle box to determine whether additional elevation is warranted, not to exceed the specified maximum polynomial degree. The mesh smoothing phase is completed when all elements meet the deviation criterion and the mesh smoothing process converges.

The final output from the elevation and smoothing process is a mesh that contains high-order nodes that are shared between elements of the same order. Faces and edges shared between elements of different order will not share the same interface nodes. Shape conformity at these interfaces is imposed before export. At this point the mesh is exported in the appropriate high order mesh file format. An example of a mixed-order mesh created with this process is shown in Fig. 2 for a Q1–Q4

² The highest surface and volume element degree will be Q4 even if the surface polynomial degree is higher.

mesh on the Onera M6 wing leading edge: the light gray elements are Q1 and the dark gray elements are Q4. The elements at the leading edge, where the curvature is highest, are quartic and the element order decreases away from the leading edge as the curvature decreases. A quality constraint ensures the degree jump between elements is limited to one.

2.2.1 Surface Element Deviation Metric

A deviation metric is used to control the p-refinement (element elevation) process during initialization and as part of the mesh smoothing process. The deviation metric measures the displacement of test nodes on the edges and faces of an element adjacent to either a curved boundary or an adjacent volume element. If the element is on a curved boundary test nodes are computed at quadrature integration points of the surface element and projected to the geometry. The deviation amount is demonstrated in Fig. 3 where a test node at the centroid of a linear triangle (dark gray) is projected to a curved geometry surface (light gray). If the displacement of this test node exceeds a threshold distance for the adjacent volume element, then elevation is indicated. The threshold amount triggering elevation is the minimum linear edge length within the element multiplied by an input deviation threshold parameter, typically 1–5%.

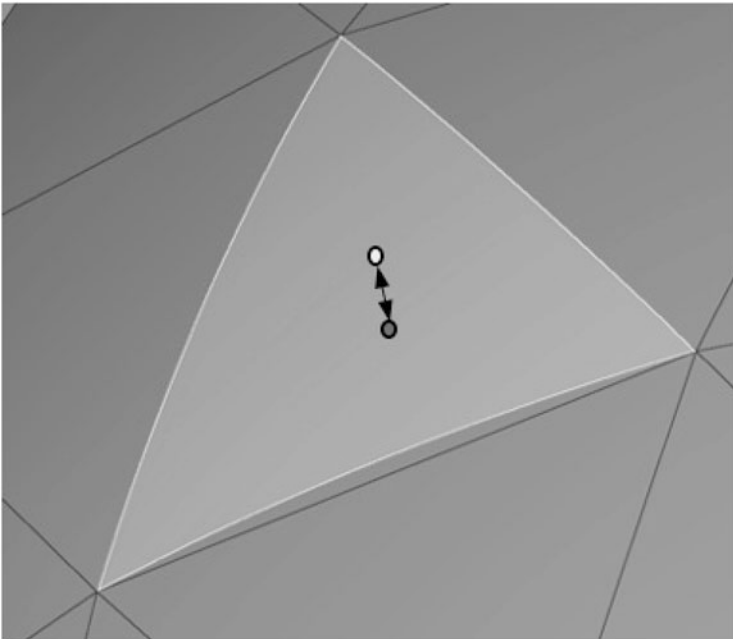


Fig. 3 Test node at centroid of surface element projected to the geometry

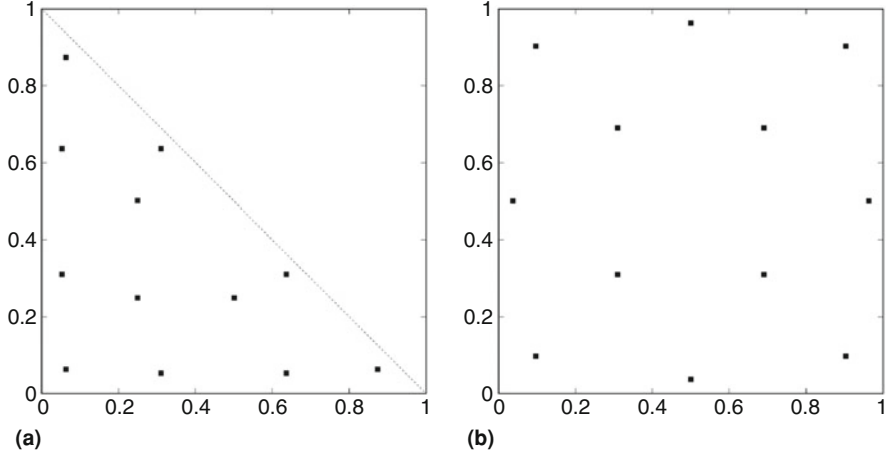


Fig. 4 Sixth order Gauss points for reference (a) triangles and (b) quadrilaterals

Each surface element is examined at 6th order quadrature Gauss point locations, shown in Fig. 4a for a reference triangle and Fig. 4b for a reference quadrilateral. The physical coordinates at these Gauss points are computed using the 2D basis functions for the current element order and physical nodes in the surface element. If the deviation of any Gauss point from the surface geometry is farther than the threshold, then the surface element (and adjacent volume element) is marked for elevation to the next higher order.

2.2.2 Shape Conformity Metric

The surface element deviation metric described in Sect. 2.2.1 is also used to define the shape conformity metric, which measures how well the discrete curved surface matches the underlying geometry. It is defined as the integration of the difference between the mesh surface and the geometry surface over the surface triangular or quadrilateral element. Equation (1) integrates the distance from a mesh node to the geometry over the surface of an element using numerical integration. The numerator results in the volume of the space between the mesh surface and the geometry. The denominator is the surface area. Combined the quantity is the average distance between the mesh and the surface.

$$SC = \frac{\iint |\vec{dr}| ds}{\iint ds} \quad (1)$$

The distance given by $|\vec{dr}|$ is the surface element deviation, as described in Sect. 2.2.1 and illustrated in Fig. 3. Numerical integration is performed using

Gaussian quadrature over each surface element using the 6th order quadrature points shown in Fig. 4a,b for surface triangles and quadrilaterals.

The shape conformity metric produces a dimensional quantity in the units of the mesh length scale. For flat planar surfaces all mesh orders should produce machine zero values, indicating the mesh is on the planar surface. For curved boundaries the linear mesh should exhibit the largest error and increasing the mesh order should produce smaller error values.

2.2.3 Volume Element Deviation Metric

Volume elements with neighbors of a different order need deviation testing to ensure the shapes at the interface are similar. This process will also propagate high curvature regions of the geometry into the volume. At these interfaces, the deviation test is performed in one of two ways: either testing the lower order nodes against the higher order shape or vice versa. When performing mesh smoothing the nodes on the faces and edges of the lower order element are projected onto the adjacent, higher order shape. Figure 5a is used to illustrate. The high-order nodes on the face common to elements of different order are not shared by each element. The small dots along edges and in faces are the high-order nodes. At the edge between a quartic and cubic element there are two high-order nodes from the cubic element and three high-order nodes from the quartic element. Nodes on the cubic element must be forced to adhere to the quartic shape. Otherwise, the mesh smoothing process will drive the cubic nodes towards the original linear shape, as shown in Fig. 5b. For highly clustered meshes this would cross over the geometry, resulting in an invalid mesh. The same is true for the nodes at the interface of the Q3 and Q2 elements. Linear (Q1) elements have no face interior or edge interior nodes, so no shape enforcement is required.

Periodically, during mesh smoothing, the deviation test is performed where the nodes on the higher order side are tested against the lower order shape. The parametric coordinates of the higher order nodes are used to compute the

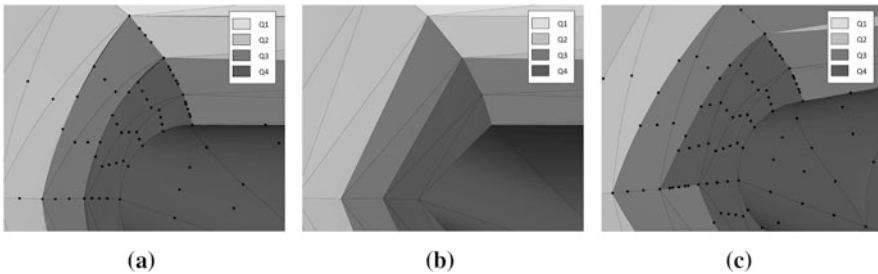


Fig. 5 The deviation test is used to ensure both that the presence of the curved surface is felt on the interior and that interfaces between lower order elements and higher order elements match. (a) Transition gaps. (b) Linear shape. (c) Lower order shape enforced

physical coordinates on the lower order shape, using the lower order basis function and physical nodes. If the distance from the current location to the lower order shape location exceeds the deviation threshold then the lower order element at that interface is marked for elevation. When the two shapes at element interfaces converge to within the tolerance then p-refinement stops. At the completion of mesh smoothing, before export, the higher order nodes at these mixed order interfaces are projected (not just measured) to the lower order shapes. The final example mesh, after additional p-refinement and mesh smoothing, is shown in Fig. 5c. All nodes on the interfaces between elements of different order appear to lie on the same shape.

2.2.4 Geometry Driven Mesh Perturbations

During the initialization process the elements near curved geometry are tested using the deviation criterion and the current maximum allowed degree. When an element is elevated the perturbations produced by the geometry are spread to other high-order nodes in the element using a simple transfer process with a linearly decaying rate. An iterative process of spreading these perturbations takes place where the deviation test is performed on adjacent elements. Neighboring elements use the volume deviation metric to sense the change in the shape of these newly elevated elements. This may trigger additional elements to elevate and the spreading continues. The requirement that the difference in degree between adjacent elements is limited to one continues to be enforced. These deviation and order difference tests quickly transfer the geometry perturbation into the volume. This initialization process may still result in mesh crossing near highly curved geometry, so mesh smoothing is required to ensure a valid mesh is produced.

2.2.5 Iterative Perturbation-Based Smoothing

Recent modifications to the smoothing method have resulted in a more robust technique for ensuring a valid computational mesh [12]. The basic smoothing method attempts to enforce shapes, derived from the original linear mesh, on the elevated high-order mesh. This is the weighted condition number (WCN) component of the cost function. The smoothing also imposes element size control through a normalized Jacobian-based component of the cost function. The overall cost function divides the normalized Jacobian by the weighted condition number, shown in (2) [17]. This function is computed on sub-triangles of each surface element and sub-tetrahedra of each volume element.

$$C = \frac{\min \left[1, \frac{J_p}{J_c} \right]}{WCN} \quad (2)$$

The numerator is the normalized Jacobian, a ratio of the determinants of the Jacobian matrices. The Jacobian matrix, given in (3), is computed at survey locations across the element using the appropriate Lagrangian basis functions. Subscript c refers to the computational mesh. This is the original mesh elevated but with no perturbations applied to any node, i.e. a copy of the physical mesh with straight-sides and no displacements. This mesh is assumed to be always valid with positive volumes and Jacobians. Subscript p refers to the physical mesh. When the physical Jacobian at a given survey point is less than the computational Jacobian the ratio will be less than one and the smoothing scheme will strive to increase it. This strongly influences node movement in tight viscous boundary layer regions of the mesh. The ratio is capped at 1, so larger ratios are permitted.

$$J = \begin{bmatrix} \sum \frac{\partial N_i}{\partial \xi} x_i & \sum \frac{\partial N_i}{\partial \xi} y_i & \sum \frac{\partial N_i}{\partial \xi} z_i \\ \sum \frac{\partial N_i}{\partial \eta} x_i & \sum \frac{\partial N_i}{\partial \eta} y_i & \sum \frac{\partial N_i}{\partial \eta} z_i \\ \sum \frac{\partial N_i}{\partial \zeta} x_i & \sum \frac{\partial N_i}{\partial \zeta} y_i & \sum \frac{\partial N_i}{\partial \zeta} z_i \end{bmatrix} \quad (3)$$

Mesh smoothing is applied to all nodes in the mesh. Surface nodal values of the cost function are computed using a biased average of sub-triangle cost value surrounding each surface node. Interior volume nodal values of the cost function are computed using a biased average of the sub-tetrahedra cost value surrounding each node. The surface (2D) cost function is defined first, followed by the volume (3D) cost function. Modifications to this smoothing method involved splitting the surface and volume mesh smoothing by leveraging separate weighted condition number and normalized Jacobian calculations for surface and volume elements.

Surface Weighted Condition Number

The surface weighted condition number enforces the shape of the surface element. The WCN component seen in the denominator of (2) serves as the weighted condition number for triangles given in (4). This quantity is computed for a sub-triangle of the high-order elements. All high-order surface elements, triangles and quadrilaterals, can be decomposed into sub-triangles.

$$WCN = \frac{\|AW^{-1}\| \|WA^{-1}\|}{2} \quad (4)$$

The bracketed quantities in (4) are the Frobenius norms of the matrix products. The W matrix is the weight matrix derived from computational coordinates for the sub-triangle. This defines the desired shape of the sub-triangle. Figure 6 shows the weight matrix where the entries of W are computed using the lengths of the three edges of a general triangle. The A matrix is the same as the W matrix using the physical coordinates. Even though these are two-dimensional elements, the matrices are defined using three-dimensional lengths of surface triangle edges.

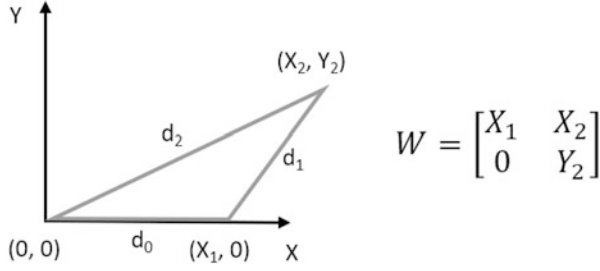


Fig. 6 The weight matrix can be formed from the computational edge lengths of the triangle

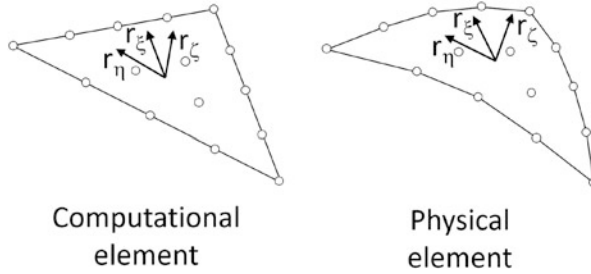


Fig. 7 Quartic triangle. Computational element on the left. Physical element on the right

Surface Normalized Jacobian

The surface normalized Jacobian imposes size control for the surface element. For surface elements, it represents a ratio of surface normal vectors from computational and physical space at the same parametric coordinates of the element. To illustrate, a quartic triangle is shown in Fig. 7. The computational mesh is shown on the left. The physical, curved mesh is shown on the right. All 15 nodes of the elements are shown at the corners, edges and face interior. The rows in the Jacobian matrix from (3) represent the directions of the parametric coordinates in computational or physical space. For surface elements r_ξ (first row of the matrix) and r_η (second row of the matrix) are computed from the Lagrangian basis functions. This can be computed at any location in the element. The r_ζ vector for surface elements is computed as the cross product of the r_ξ and r_η vectors and represents the surface normal direction. The normalized Jacobian value used in (2) for surface elements is the ratio of magnitudes of these two vectors. The sign is taken from the dot product of the vectors to indicate a surface element inversion when the value is negative.

The normalized Jacobian part of the cost function can be computed in a number of ways. The simplest and least expensive computes the area ratio of the sub-triangles, shown in Fig. 8a. Only the nodes in the sub-triangle are involved in the calculation. This is consistent with the weighted condition number calculation. The assumption is that the sub-triangle area will provide enough influence to ensure the actual Jacobians remain positive throughout the entire element.

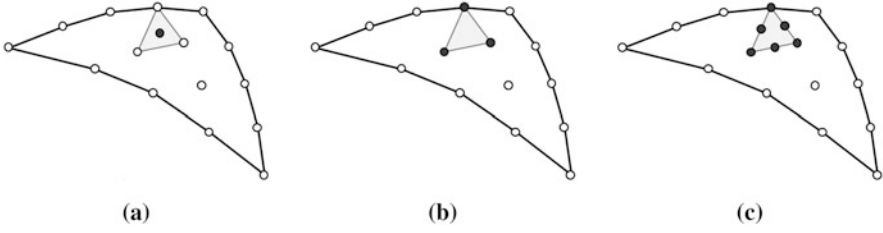


Fig. 8 The normalized Jacobian is computed using the area of sub-triangle or a biased weighting of the Jacobian evaluated at the quadrature points shown for 1 and 2 subdivisions. (a) Linear area. (b) One subdivision. (c) Two subdivisions

An alternate approach uses subdivision levels in the sub-triangle to create a grid of quadrature points. The normalized Jacobian is computed using all nodes of the element. Figure 8b,c show the quadrature point locations for 1 and 2 subdivision levels, respectively. Higher subdivision levels are possible. The normalized Jacobian value reported for the subdivision level is a biased average of the values at the quadrature points within the sub-triangle. The biased weighting is given by (5), where C and F are the normalized Jacobian values assuming the range is from 0 to 1.

$$F = F_{min} (1 - C_{min}) + F_{avg} (C_{min}) \quad (5)$$

The weighting biases the minimum value over the simple averaged value. If a negative value is detected the minimum value is returned as the cost function without computing the WCN component. In these cases, the mesh smoothing scheme is forced to correct element inversions first before enforcing element shape.

Volume Weighted Condition Number

The volume weighted condition number enforces the shape of the volume element. The WCN component seen in the denominator of (2) serves as the weighted condition number given in (6). This quantity is computed for a sub-tetrahedron of the high-order element. All high-order elements can be decomposed into smaller hexes, prisms, pyramids and tetrahedra. For sub-elements other than tetrahedra, the corners of the sub-element are used to form the tetrahedra in the cost function calculation.

$$WCN = \frac{\|AW^{-1}\| \|WA^{-1}\|}{3} \quad (6)$$

The bracketed quantities in (6) are the Frobenius norms of the matrix products. The W matrix is the weight matrix derived from computational coordinates for the same sub-tetrahedron. This defines the desired shape of the tetrahedron. Figure 9 shows the weight matrix where the entries of W are computed using the lengths of

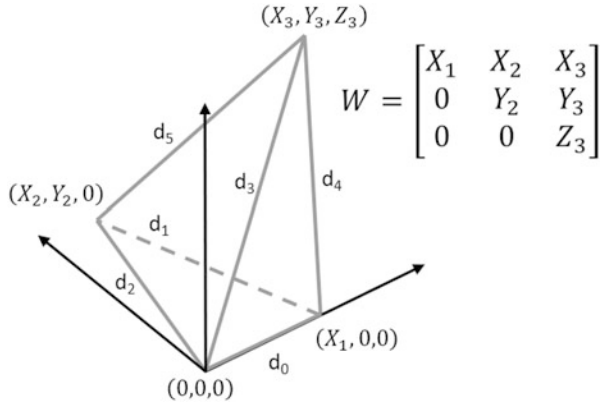


Fig. 9 The weight matrix can be formed from the computational edge lengths of the tetrahedron

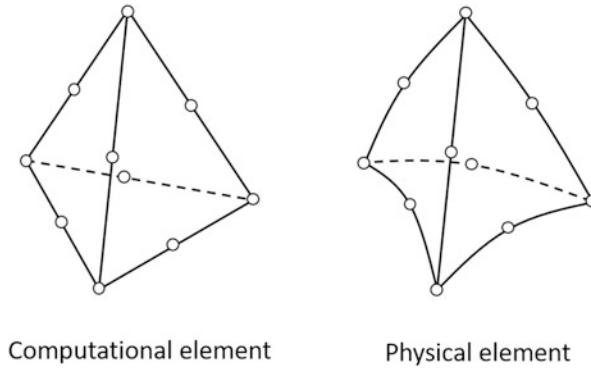


Fig. 10 Quadratic tetrahedron. Computational element on the left. Physical element on the right

the six edges of a general tetrahedron. The A matrix is the same as the W matrix using the physical coordinates.

Volume Normalized Jacobian

The volume normalized Jacobian imposes size control for the volume element. For volume elements, it represents the ratio of the determinant of the Jacobian matrices from the computational and physical space. To illustrate, a quadratic tetrahedron is shown in Fig. 10. The computational mesh is shown on the left. The physical, curved mesh is shown on the right. All 10 nodes of the elements are shown at the corners and mid-edges.

The normalized Jacobian part of the cost function can be computed in a number of ways. The simplest and least expensive computes the volume ratio of the sub-tetrahedrons, shown in Fig. 11a. Only the nodes in the sub-tetrahedron are involved

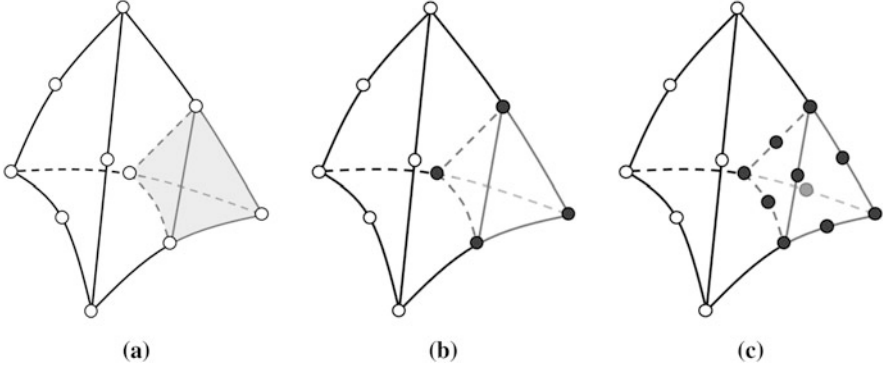


Fig. 11 The normalized Jacobian is computed using the volume of sub-tetrahedra or a biased weighting of the Jacobian evaluated at the quadrature points shown for 1 and 2 subdivisions. (a) Linear volume. (b) One subdivision. (c) Two subdivisions

in the calculation. Again, this is consistent with the weighted condition number calculation. The assumption is that the sub-tetrahedron volume will provide enough influence to ensure the actual Jacobians remain positive throughout the entire element.

An alternate approach uses subdivision levels in the sub-tetrahedron to create a grid of quadrature points. The normalized Jacobian is, again, computed using all nodes of the element. Figure 11b,c show the quadrature point locations for 1 and 2 subdivision levels, respectively. The value reported for the subdivision level approach is a biased average of the values at quadrature points within the sub-tetrahedron. The biased weighting is given by the same formula shown earlier (5), where C and F are the normalized Jacobian values assuming the range is from 0 to 1.

The first smoothing pass uses the simplest computational method. If negative Jacobians are detected after that pass is complete, a second smoothing pass is initiated with one subdivision level. Additional smoothing passes are possible but are rarely required.

Marching Direction

The smoothing is a perturbation method and requires a marching direction that will improve the value of the cost function locally. The marching direction for the mesh nodes is computed using the sensitivity of the cost function with respect to the X , Y and Z directions. The sensitivity of the corner nodes of the sub-tetrahedron is determined using C++ operator overloading of the math functions in a dual number framework [18]. This is essentially the numerical chain-rule differentiation of the cost function. Then the mesh nodal values of these derivatives are computed using the biased averaging formula in (5) where C is the cost value and F is the derivative

vector. The cost function routine for a specific sub-tetrahedron will return the cost value and 4 vectors at the corners comprised of 3 doubles for a total of 13 doubles. The biased averaging will produce directions that focus on improving the worst cost value of the surrounding sub-tetrahedra but will blend smoothly with the average cost as the minimum cost improves.

Marching Step Size

For the perturbation method, a meaningful distance is needed in addition to the marching direction already described. To determine this distance, the computed displacement of a given node is computed first using the minimum inscribing radius of the surrounding sub-tetrahedra. The inscribing radius of a sub-tetrahedron is shown in Fig. 12 for one corner of a Q2 tetrahedron. The minimum radius is computed for each moving node using the computational mesh. In collapsed elements the physical inscribing radius approaches zero which would be a poor selection for the step size. Conversely, the computational mesh is always valid and unchanging. Using the computational radius ensures a non-zero step size is used. The minimum radius is then multiplied by a user relaxation parameter, typically 0.05. This is further reduced by the minimum of one and the current difference between the nodal cost function value, which will approach a zero displacement as the node approaches the ideal position.

During mesh smoothing only nodes whose cost value is below a user specified convergence threshold, such as 0.95, are moved. This greatly reduces the overall computational expense, especially as the mesh smoothing converges. Most of the nodes in the mesh become inactive. Only nodes associated with lower cost values remain active towards the end of mesh smoothing.

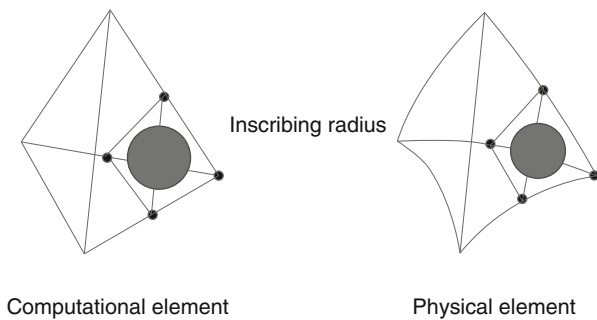


Fig. 12 Step size determined by inscribing radius of sub-tetrahedral elements

3 Results

Two realistic, complex cases are included that demonstrate the ability of the WCN-based mesh curving approach. These cases start with linear meshes generated by Pointwise. The linear mesh, geometry file and MeshLink XML file are exported and used by the curving code to produce mixed order meshes containing linear, quadratic, cubic and quartic elements.

3.1 *Juncture Flow Model*

The Juncture Flow Model (JFM) is a popular case for validating CFD methods on wing root viscous separation. It has been the focus of numerous studies and workshops, including the 3rd American Institute of Aeronautics and Astronautics (AIAA) Geometry and Mesh Generation Workshop (GMGW-3) [19]. Several mesh families were generated for this model for the purposes of the workshop. These include linear, mixed order and uniform order meshes up to quartic. Shown below in Fig. 13 is a representative mesh for the coarsest mesh in the sequence. Cubic elements are shown on the highest curvature regions of the body while some elements on the flat portion of the fuselage remain linear. Quadratic elements transition linear to cubic, enforcing the one order difference constraint between

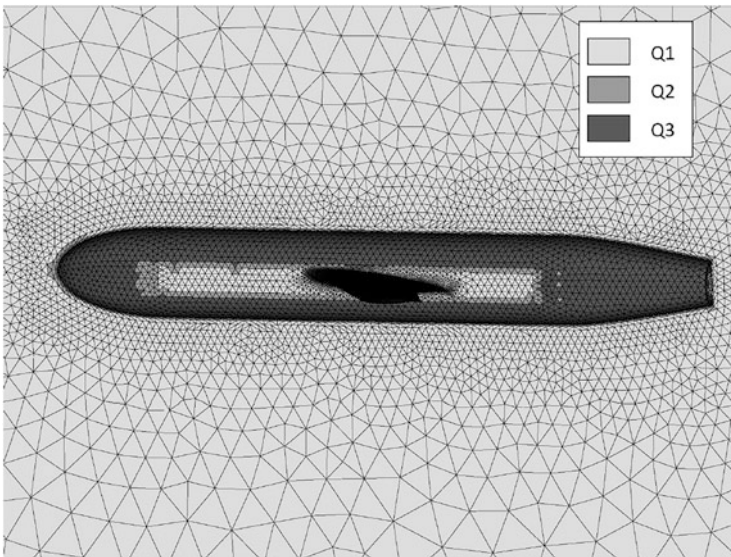


Fig. 13 Mixed order mesh of the Juncture Flow Model using Linear, Quadratic, and Cubic elements

neighboring elements. The highest curvature in the geometry occurs at the wing tip trailing edge, shown in Fig. 14. The wing tip is rounded and the coarse grid shown here has only 4 elements wrapped around 180° turn near the trailing edge. The wall normal spacing is large enough to allow viewing of the highly curved tetrahedral elements at the surface. Finer meshes in the series have more elements spanning the wing tip, but also have finer wall normal spacing which challenges the curving process.

At the completion of each mesh smoothing phase the shape conformity metric is evaluated for all boundaries except planar boundaries, such as symmetry planes. The error values achieved for the shape conformity metric for the JFM fuselage is listed in Table 1. The left, middle, and right columns are the element order, average error, and maximum error respectively. As expected, as the element order is increased the errors reduce significantly. Notice that the maximum error for the mixed order Q1–Q3 and uniform order Q3 meshes are equal. The same is true for the Q1–Q4

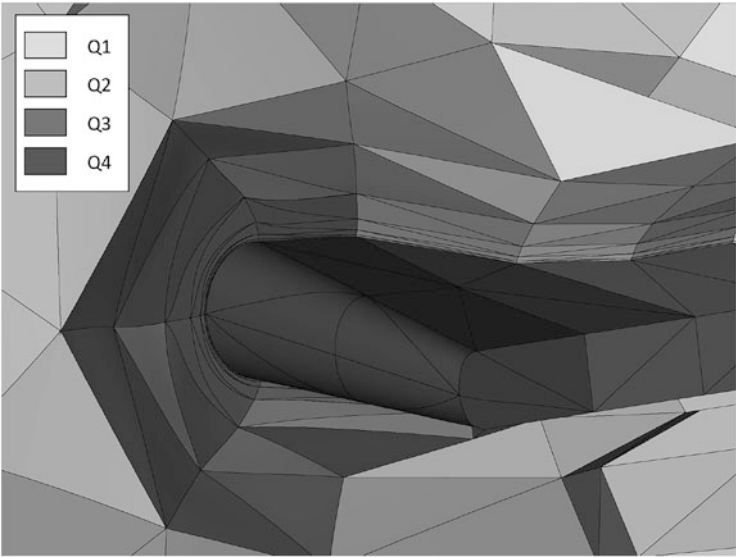


Fig. 14 Cut at the wing tip trailing edge of the Juncture Flow Model showing mixed order Q1–Q4 elements

Table 1 Shape conformity for juncture flow model fuselage

Elevation order	Average error	Maximum error
Q1	0.471557	3.45525
Q1–Q2	0.00454984	0.180504
Q2	0.00454117	0.180398
Q1–Q3	0.000556685	0.0413742
Q3	0.000553584	0.0413742
Q1–Q4	0.000207317	0.0190914
Q4	0.000183944	0.0190914

Table 2 Shape conformity for juncture flow model wing

Elevation order	Average error	Maximum error
Q1	0.0293635	0.785052
Q1–Q2	0.000796288	0.0753876
Q2	0.000794145	0.0757585
Q1–Q3	0.00013674	0.0145939
Q3	0.0000920631	0.0145939
Q1–Q4	0.0000744936	0.0056608
Q4	0.0000252808	0.0056608

and Q4 meshes. This indicates that the maximum error is occurring on an element of maximum order, Q3 and Q4 respectively. The average error difference between the mixed order and the fully elevated order for the same maximum order mesh only vary slightly. This is expected with the user specified deviation metric of 0.01. Smaller values of the deviation metric will reduce this difference further at the cost of elevating more elements in the mesh to higher order.

The shape conformity metric for the wing surface is displayed in Table 2. Similar trends can be seen for the wing surface. The error levels are smaller than those reported for the fuselage due to the finer resolution mesh on the wing.

3.2 NASA High Lift Common Research Model

Another case studied at GMGW-3 was the NASA High Lift Common Research Model (CRM-HL) configuration. This case was the focus of the 4th AIAA High Lift Prediction Workshop (HLPW-4) [20] co-located with GMGW-3. Several mesh families were generated for this configuration also, including linear, mixed order and uniform order up to quartic. The meshes shown below represent the coarsest level meshes from the sequence. The symmetry plane mesh is mostly linear, as shown in Fig. 15a. The center section of the fuselage is quadratic. Cubic and quartic elements are shown for the forward and aft fuselage regions. Most of the underside of the wing, shown in Fig. 15b, is quadratic. Cubic and quartic elements exist at the leading edges of the wing and nacelle. This is also true for the topside view of the slats and nacelle pylon shown in Fig. 15c.

An axial cut at the wing tip trailing edge is shown in Fig. 16a. This very coarse mesh has only two triangle elements spanning the 180° turn of the rounded wing tip. The wall normal spacing, equivalent to an approximate Y^+ value of 100, is extremely coarse. Much finer wall normal spacing was used for other meshes in the series, but those are more difficult to visualize. Also shown in the figure are the nodes of the mesh. Notice the quartic elements contain 5 points (four segments) along each edge. When the adjacent element is cubic these mid-edge nodes are not shared. The adjacent elements have a different set of edge internal points. The

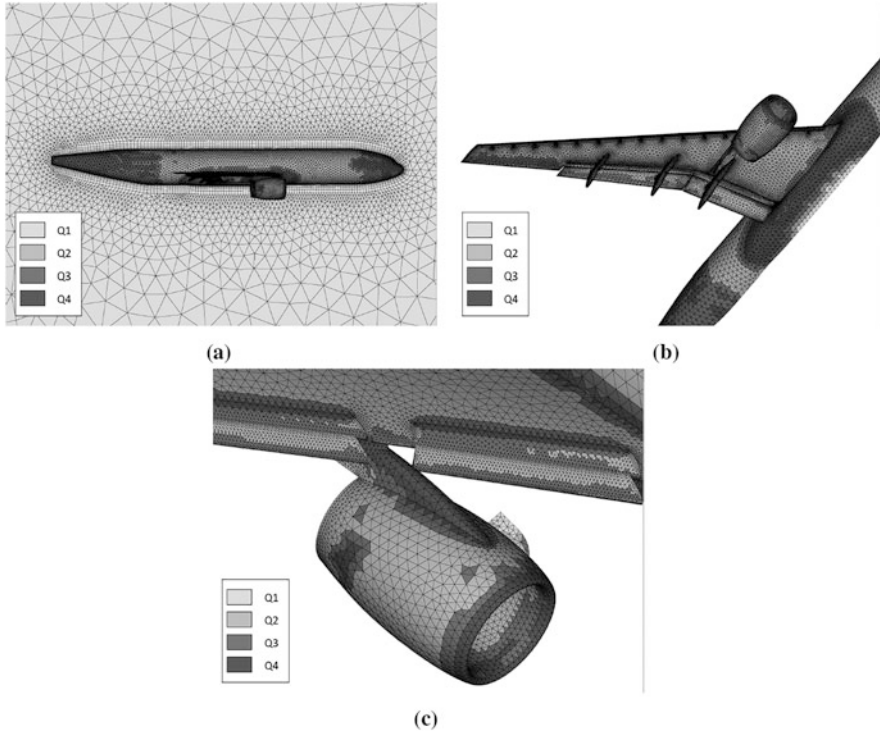


Fig. 15 Mixed order mesh of the Common Research Model using Linear through Quartic elements. (a) Symmetry plane. (b) Underside of wing. (c) Nacelle

enforcement of shape conformity at this interface ensures the curves represented on each side are the same, eliminating gaps in the mesh.

A cut through the volume mesh at the engine nacelle/pylon is shown in Fig. 16b. The high curvature of the nacelle leading edge is resolved with quartic elements. Quadratic elements cover most of the nacelle. The majority of the elements away from the curved geometry remain linear.

4 Conclusions

A method for generating curved, mixed order meshes has been presented. Geometry access is provided through the MeshLink API. A deviation metric is used to indicate when surface and volume elements need elevation. Elements up to 4th order are possible. Iterative perturbation-based smoothing is used to ensure a valid, high quality mesh is produced. The cost function for the smoothing is comprised of a normalized Jacobian component that ensures positive Jacobians and a Weighted

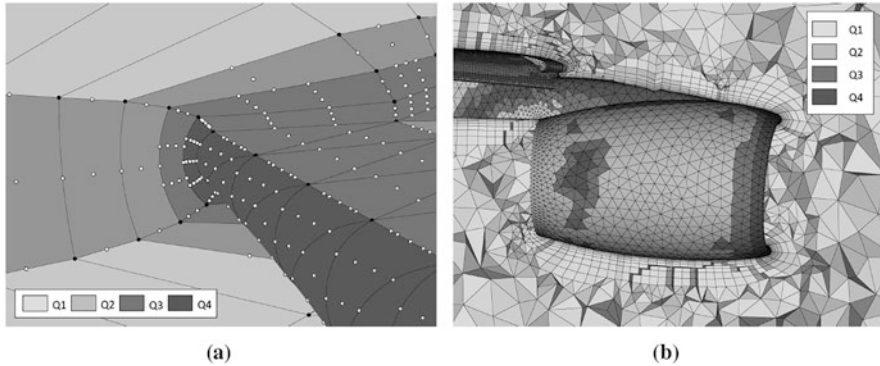


Fig. 16 Cutting planes showing Q1–Q4 elements on the CRM. (a) Axial cut at wing tip trailing edge. (b) Cut at engine nacelle and pylon

Condition Number component that enforces element shape. The combination allows for elevation and smoothing of meshes that include clustering to viscous boundaries. Shape conformity is imposed between elements of different order and used to evaluate the error between the elevated surface mesh and the underlying geometry. Two examples of realistic geometries were presented for configurations studied in the 4th AIAA High Lift Prediction Workshop and 3rd AIAA Geometry and Mesh Generation Workshop.

References

1. Chen, C.H.: A Radial Basis Functions Approach to Generating High-Order Curved Element Meshes for Computational Fluid Dynamics. Master's thesis, Imperial College, London (2013)
2. Stees, M., Shontz, S.M.: Spectral and High Order Methods for Partial Differential Equations, p. 229 (2018)
3. Ims, J., Duan, Z., Wang, Z.J.: In: 22nd AIAA Computational Fluid Dynamics Conference (AIAA 2015-2293)
4. Persson, P.O., Peraire, J.: In: 47th AIAA Aerospace Sciences Meeting Including The New Horizons Forum and Aerospace Exposition (AIAA 2009-0949). <https://doi.org/10.2514/6.2009-949>
5. Moxey, D., Ekelschot, D., Keskin, Ü., Sherwin, S.J., Peiró, J.: Comput. Aided Des. **72**, 130 (2016). <https://doi.org/10.1016/j.cad.2015.09.007>
6. Fortunato, M., Persson, P.O.: J. Comput. Phys. **307**, 1 (2016)
7. Feuillet, R., Loseille, A., Alauzet, F.: In: International Meshing Roundtable, pp. 3–21 Springer, Berlin (2018)
8. Zhang, R., Johnen, A., Remacle, J.F.: In: International Meshing Roundtable, pp. 57–69. Springer, Berlin (2018)
9. Ruiz-Gironés, E., Sarrate, J., Roca, X.: Procedia Eng. **163**, 315 (2016)
10. Karman, S.L., Erwin, J.T., Glasby, R.S., Stefanski, D.: In: 46th AIAA fluid dynamics conference (AIAA 2016-3178), p. 3178. <https://doi.org/10.2514/6.2016-3178>
11. Karman, S.L.: In: International Conference on Spectral and High Order Methods (2018)

12. Karman, S.L.: In: International Meshing Roundtable, pp. 303–325. Springer, Berlin (2018)
13. Geuzaine, C., Remacle, J.F.: Gmsh. <http://www.gmsh.info>
14. Nektar++: Nekmesh. <https://www.nektar.info>
15. Pointwise Inc.: Pointwise. <https://www.pointwise.com>
16. Computational geometry kernel support. U. S. Air Force contract FA9101-18-P-0042, Topic AF181-015
17. Karman, S.L.: In: AIAA Aviation 2019 Forum (AIAA 2019-3317), p. 3317
18. Aubert, P., Di Césaré, N., Pironneau, O.: Comput. Vis. Sci. **3**(4), 197 (2001)
19. 3rd AIAA Geometry and Mesh Generation Workshop. <https://www.gmgworkshop.com>
20. 4th AIAA High Lift Prediction Workshop. <https://hiliftpw.larc.nasa.gov/index.html>

A R&D Software Platform for Shape and Topology Optimization Using Body-Fitted Meshes



C. Nardoni, D. Danan, C. Mang, F. Bordeu, and J. Cortial

Abstract Topology optimization is devoted to the optimal design of structures: It aims at finding the best material distribution inside a working domain while fulfilling mechanical, geometrical and manufacturing specifications. Conceptually different from parametric or size optimization, topology optimization relies on a freeform approach enabling to search for the optimal design in a larger space of configurations and promoting disruptive design. The need for lighter and efficient structural solutions has made topology optimization a vigorous research field in both academic and industrial structural engineering communities. This contribution presents a Research and Development software platform for shape and topology optimization where the computational process is carried out in a level set framework combined with a body-fitted approach.

1 Introduction

Several shape and topology optimization methods have been proposed and are currently employed for structural design in commercial solutions (SIMP method, BESO method, level set method among them). Density-based optimization methods, such as the widespread SIMP method, use as the design variable a density field which takes intermediate values between the material and the void densities. The fictitious material densities are eventually penalized in order to enforce a binary material/void optimized design. In the present work we opt for level-set-based structural optimization in order to avoid the introduction and the treatment of fictitious material densities. The level set method relies on the classical sensitivity analysis from the shape optimization framework to compute a descent direction

C. Nardoni (✉) · D. Danan · C. Mang

Irt Systemx, Palaiseau, France

e-mail: chiara.nardoni@irt-systemx.fr; david.danan@irt-systemx.fr; chetra.mang@irt-systemx.fr

F. Bordeu · J. Cortial

Safran Tech, M&S, Châteaufort, Magny-Les-Hameaux, France

e-mail: felipe.bordeu@safrangroup.com; julien.cortial@safrangroup.com

and advect the structural interface. The overall optimization process is driven by a gradient-type algorithm. In the present setting the level set method is coupled with a remeshing routine which enables the reconstruction of a body-fitted mesh at each step of the underlying optimization process, as proposed in [4, 8]. Since the structural interface is known explicitly at each step of the iterative procedure, the body-fitted approach simplifies the evaluation of the mechanical quantities of interest. Moreover, the computational mesh of the optimized design can be readily exported together with its finite element model for further validation analysis using a dedicated external software application.

In this work we handle two classical problems in structural optimization. First, in the static linear elasticity setting, we consider stress minimization problems. Avoiding stress concentration plays a paramount role in the design of reliable mechanical structures [3, 6, 13, 15, 17, 19, 21, 23, 24]. In the present context we focus on the von Mises stress which is a key ingredient of most failure criteria. Second, we consider the problem of maximizing the first eigenfrequency of an elastic structure under a volume constraint. Vibration analysis is also a crucial assessment to avoid structural failure [18].

The proposed numerical examples are realized using PISCO, a Research and Development software platform devoted to topology optimization that is in active development at IRT SystemX.¹ Isovalue discretization, mesh adaptation and mesh displacement are performed by the remeshing tool mmg3d.² The finite element analyses are carried out using the industrial-grade solver Code_Aster.³

2 Level Set Method for Shape and Topology Optimization

This section introduces some basic notions about the level set method for shape and topology optimization. For more detailed surveys we refer to [1, 5, 22].

2.1 Shape Sensitivity Analysis

Shape optimization aims at minimizing an objective function $J(\Omega)$ over a set $\mathcal{O}$ of admissible shapes. Typically the admissible shapes are constrained into a given design space D . In order to differentiate with respect to the domain and enforce optimality conditions, we refer here to the Hadamard's boundary variation method (see e.g. [1, 16]). Thus, variations of a given shape Ω are considered under the

¹ <https://www.irt-systemx.fr/project/top>.

² <https://www.mmgtools.org/>.

³ <https://www.code-aster.org>.

form:

$$\Omega_\theta = (I + \theta)(\Omega),$$

where $\theta : \mathbb{R}^d \mapsto \mathbb{R}^d$ is a ‘small’ diffeomorphism. Indeed, each admissible variation Ω_θ of Ω is parametrized in terms of a transformation of the form $I + \theta$, which remains ‘close’ to the identity. The admissible vector field θ is sought among the Banach space $W^{1,\infty}(\mathbb{R}^d, \mathbb{R}^d)$ of bounded and Lipschitz functions endowed with the norm:

$$\|\theta\|_{W^{1,\infty}(\mathbb{R}^d, \mathbb{R}^d)} := \|\theta\|_{L^\infty(\mathbb{R}^d)^d} + \|\nabla \theta\|_{L^\infty(\mathbb{R}^d)^{d \times d}}, \forall \theta \in W^{1,\infty}(\mathbb{R}^d, \mathbb{R}^d).$$

The shape derivative is then defined as follows.

Definition 1 A function $F(\Omega)$ of the domain is said to be *shape differentiable* at Ω if the mapping $\theta \mapsto F(\Omega_\theta)$, from $W^{1,\infty}(\mathbb{R}^d, \mathbb{R}^d)$ into $\mathbb{R}$, is Fréchet differentiable at $\theta = 0$. The associated Fréchet differential is denoted as $\theta \mapsto F'(\Omega)(\theta)$ and called the shape derivative of F ; the following expansion then holds:

$$F(\Omega_\theta) = F(\Omega) + F'(\Omega)(\theta) + o(\theta), \text{ where } \frac{|o(\theta)|}{\|\theta\|_{W^{1,\infty}(\mathbb{R}^d, \mathbb{R}^d)}} \xrightarrow{\theta \rightarrow 0} 0.$$

We recall that for a large class of functions of the domain the shape derivative admits the following structure [1]:

$$\forall \theta \in W^{1,\infty}(\mathbb{R}^d, \mathbb{R}^d), \quad F'(\Omega)(\theta) = \int_{\partial\Omega} v_\Omega(s) \theta n_{\partial\Omega} ds, \quad (1)$$

where $n_{\partial\Omega}$ is the outward normal to $\partial\Omega$ and v_Ω is a scalar field depending on F typically through a direct state and an adjoint state, both solutions of PDEs modeling the physical system of interest.

2.2 Level Set Method

2.2.1 Implicit Parametrization of Shapes

In the level set approach, the structural interface is represented as the 0 isovalue of a scalar function—the *level set function*—defined over the whole design space D . The implicit description allows to easily track the interface evolution and naturally handles topology changes, as for example the merging of two interfaces. More precisely, a level set function of a shape $\Omega \subset D \subset \mathbb{R}^3$ is a scalar function

$\phi : D \rightarrow \mathbb{R}$ enjoying the following properties

$$\begin{cases} \phi(x) < 0 & \text{if } x \in \Omega, \\ \phi(x) = 0 & \text{if } x \in \partial\Omega, \\ \phi(x) > 0 & \text{if } x \in {}^c\overline{\Omega}. \end{cases}$$

The level set function is typically initialized with the *signed distance function* d_Ω owing to the unitary gradient property:

$$|\nabla d_\Omega(x)| = 1,$$

which holds for all x where d_Ω is differentiable.

2.2.2 Optimization Procedure

Starting from a given admissible shape Ω_0 and a function of the domain J , the shape derivative allows to select a descent direction for J . This procedure enables to produce a sequence of shapes $\Omega_{k\{k=0,\dots\}}$ with decreasing values of J . At each iteration the domain is updated using the following advection equation

$$\frac{\partial \phi}{\partial t} + \theta_k |\nabla \phi| = 0 \text{ in } D, \quad (2)$$

where the vector field θ_k is a descent direction for J , set as

$$\theta_k = -w_{\Omega_k} n_{\partial\Omega_k}, \quad (3)$$

where w_{Ω_k} and $n_{\partial\Omega_k}$ are respectively a velocity field and the normal defined in (1). Thus, the new shape is defined implicitly by $\Omega_{k+1} = \{x \in D : \phi_{k+1}(x) < 0\}$.

2.2.3 Regularization of the Descent Direction

The field (1) is *only* rigorously defined on the interface $\partial\Omega$; it has to be extended to the whole domain to move the interface further than an infinitesimal distance. Moreover, the choice $\theta = -vn$ on $\partial\Omega$ can generate an irregular descent direction, unsuitable for numerical practice. To circumvent these difficulties, the literature suggests to *extend* and *regularize* the descent direction [9]. The general idea is to replace the optimal scalar product over $L^2(\partial\Omega)$ by a more regular one. In the present context the extended and regularized scalar field is defined as the unique solution $z \in H^1(\Omega)$ of the following variational problem

$$\forall w \in H^1(D), \quad \alpha \int_\Omega \nabla z \cdot \nabla w \, dx + \int_{\partial\Omega} zw \, dx = \int_{\partial\Omega} v_{\partial\Omega} w \, ds, \quad (4)$$

where $\alpha > 0$ is a parameter tuning the intensity of the regularization.

3 Shape Evolution Using Body-Fitted Meshes

At each step of the optimization procedure an unstructured mesh of the current shape is obtained by the explicit discretization of the 0 isovalue of the implicit domain ϕ_{k+1} [4, 8]. This routine allows to generate a tetrahedral mesh whose boundary fits the structural interface. This goal is achieved thanks to the following steps:

- The 0 isosurface of the level set function is explicitly discretized;
- The quality of the underlying mesh is improved by means of local remeshing operations driven by both geometrical and user requirements.

See Fig. 1 for an example of such a procedure. This method allows to dynamically track the evolution of the interface even when topology changes occur. Note that the isovalue discretization can be combined with classical metric-based adaptation routines, allowing the user to prescribe a spatially-varying desired mesh size.

The above procedure permits the evaluation of the mechanical performances of the structure without falling back on the ersatz material approximation, which is currently used for level-set-based topology optimization in a fixed background mesh setting. This approximation can impact the accuracy of the finite element computation in some sensitive cases. For example, in the context of stress evaluation, in particular when the volume fraction of the material part inside the design space is small, the residual stresses stored in the soft material can affect the measurement of global stress indicators as well as the local stresses near the interface [11]. A special attention must also be paid in the context of eigenfrequency optimization since the presence of a soft material [7] or a density-based approach [10] can modify the eigenfrequencies of the structure.

An alternative to conformal remeshing consists in keeping the computational support unchanged while enriching the finite element space in the vicinity of the interface with ad-hoc chosen basis functions (X-FEM-type methods [12, 20, 23]).

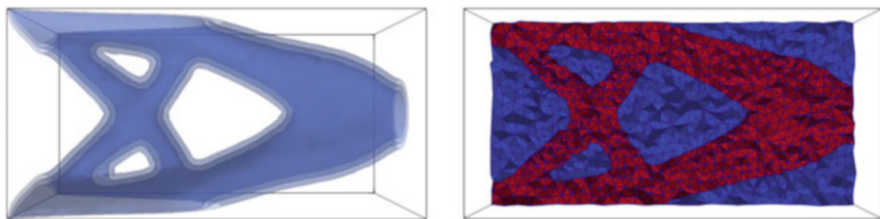


Fig. 1 Isovalues of the level-set function (left) and body-fitted mesh (right) of a given shape. The interior part of the shape is represented in red

The main drawback of these methods is their intrusiveness making them difficult to couple with existing physical solvers.

4 Stress-Based Optimization

Let $\Omega \subset \mathbb{R}^3$ be a shape such that $\partial\Omega = \Gamma_D \cup \Gamma_N \cup \Gamma$. Let u_Ω be the displacement field solution of the following linear elasticity problem

$$\begin{cases} -\operatorname{div}(\sigma(u_\Omega)) = 0 & \text{in } \Omega, \\ \sigma(u_\Omega) \cdot n = g & \text{on } \Gamma_N, \\ \sigma(u_\Omega) \cdot n = 0 & \text{on } \Gamma, \\ u_\Omega = 0 & \text{on } \Gamma_D, \end{cases} \quad (5)$$

where $\varepsilon(u) = \frac{1}{2}(\nabla u + \nabla u^T)$ is the linearized strain tensor and σ is the stress tensor obeying to the following Hooke's law with Lamé parameters λ, μ :

$$\sigma(u) = 2\mu\varepsilon(u) + \lambda \operatorname{tr}(\varepsilon(u))I.$$

Let us denote by $\sigma_{vm}(u)$ the Von Mises equivalent stress associated to $\sigma(u)$. Since stress measurements are intrinsically of local nature, a typical stress constraint over a region Ω translates into the following non differentiable form

$$\max_{x \in \Omega(x)} \sigma_{vm}(x) \leq \bar{\sigma}_{vm}, \forall x \in \Omega. \quad (6)$$

From a numerical point of view, incorporating such a condition at each stress evaluation point leads to an unacceptably large number of constraints that further increases with the refinement of the underlying computational mesh. In order to overcome these difficulties constraints aggregation techniques can be considered. A popular choice [3, 21] consists in regularizing the criterion (6) by penalizing the following integral functional:

$$J(\Omega) = \left(\int_{\Omega} j_{\alpha}(\sigma(u_{\Omega})) dx \right)^{\frac{1}{\alpha}} = \left(\int_{\Omega} \sigma_{vm}^{\alpha}(u_{\Omega}) dx \right)^{\frac{1}{\alpha}}, \quad (7)$$

where $j_{\alpha} = \sigma_{vm}^{\alpha}$ and $\alpha \geq 1$ is a scalar parameter. By a classical calculation (see for example [3]) the functional (7) is shape differentiable. In the present context the descent direction θ is sought in the space

$$\Theta_{ad} = \{\theta = 0 \text{ on } \Gamma_N \cup \Gamma_D\}.$$

Thus the shape derivative of (7) reads

$$\forall \theta \in \Theta_{ad}, J'(\Omega)(\theta) = \frac{1}{\alpha} J(\Omega)^{1-\alpha} \int_{\Gamma} (\sigma_{vm}^{\alpha}(u_{\Omega}) + \sigma(u_{\Omega}) : \varepsilon(p_{\Omega})) \theta \cdot n \, ds. \quad (8)$$

The *adjoint state* p_{Ω} is solution of the following problem

$$\begin{cases} -\operatorname{div}(\sigma(p_{\Omega})) = \operatorname{div}(A j'_{\alpha}(\sigma(u_{\Omega}))) & \text{in } \Omega, \\ \sigma(p_{\Omega}) \cdot n = A j'_{\alpha}(\sigma(u_{\Omega})) \cdot n & \text{on } \Gamma \cup \Gamma_N, \\ p_{\Omega} = 0 & \text{on } \Gamma_D, \end{cases} \quad (9)$$

where j'_{α} denotes the derivative of j_{α} with respect to σ .

Remark 1 As noted in [17], increasing the exponent α can lead to overly large values of the integrand in (8). To avoid degrading the numerical accuracy, one can consider an alternative, mathematically equivalent formulation such that the integrand is defined as:

$$j_{\alpha} = \left(\frac{\sigma_{vm}}{\bar{\sigma}} \right)^{\alpha}, \quad (10)$$

where $\bar{\sigma}$ is a normalization parameter.

5 k -th Eigenfrequency Maximization

In this section we focus on a criterion that requires a modal search analysis. Let $\Omega \subset \mathbb{R}^3$ be a shape such that $\partial\Omega = \Gamma_D \cup \Gamma$. The eigenmodes and eigenfrequencies of Ω are determined by solving the following problem

$$\begin{cases} -\operatorname{div}(\sigma(u_{\Omega})) = \omega^2 \rho u_{\Omega} & \text{in } \Omega, \\ u_{\Omega} = 0 & \text{on } \Gamma_D, \\ \sigma(u_{\Omega})n = 0 & \text{on } \Gamma, \end{cases} \quad (11)$$

where ρ denotes the material density. Note that (11) admits a countable set of solutions $(\omega_k, u_k)_{k \in \mathbb{N}}$. When the positive values ω_k are sorted such that $\omega_k < \omega_{k+1}$, $\forall k$, then u_k is called the k -th eigenvector or eigenmode. The quantity

$$f_k = \frac{\omega_k}{2\pi}$$

is called the k -th eigenfrequency of the structure. Moreover each eigenmode is normalized as follows

$$\int_{\Omega} \rho |u_k|^2 dx = 1, \quad \forall k. \quad (12)$$

In order to maximize the k -th eigenfrequency we consider the minimization of following functional of the domain

$$J_k(\Omega) = -\omega_k^2. \quad (13)$$

A classical computation (see for example [2]) shows that if the eigenvalue associated to the k -th eigenmode is simple, then (13) is shape differentiable and the shape derivative reads

$$\forall \theta \in \Theta_{ad}, \quad J'(\Omega)(\theta) = \int_{\Gamma} (\omega_k^2 \rho |u_{k\Omega}|^2 - \sigma(u_{k\Omega}) : \varepsilon(u_{k\Omega})) \theta \cdot n ds, \quad (14)$$

where Θ_{ad} is the set

$$\Theta_{ad} = \{\theta = 0 \text{ on } \Gamma_D\}.$$

Note that the above problem is self-adjoint, meaning that the evaluation of the shape derivative does not require the computation of an adjoint state. Note also that functional (13) extends without difficulty to the optimization of a continuously differentiable function of eigenfrequencies.

6 Numerical Implementation

PISCO includes the following components:

- An algorithmic toolbox specialized in the treatment of level sets
- A generic interface to finite element solvers
- Algorithms for the resolution of constrained optimization problems
- Physical and geometrical optimization criteria
- An interface to the remeshing tool `mmg3d`

The components devoted to the physical analysis computations and the constrained optimization algorithms are implemented in a generic fashion in dedicated modules. These components are linked to the topology optimization problems and criteria in a non-intrusive way. The non-intrusiveness of the implementation is proved by the coupling with several external physical solvers such as `Code_Aster` and `FreeFem++`. In the present context, all physical evaluations are performed using the finite element solver `Code_Aster`, developed at EDF France. The choice of `Code_Aster` is motivated by the large range of available physics and by the richness of the available post-processing routines.

The numerical optimization algorithm handles the balancing between the minimization of the objective and the non-violation of the constraints. Popular penalization methods such as the Augmented Lagrangian reformulate a constrained optimization problem into a sequence of unconstrained optimization problems by incorporating the constraints as penalizations of the objective function. In the present context, we rely on a gradient-flow algorithm designed to decrease both the value of the objective function and the violation of the constraints [14].

6.1 Overview of the Numerical Algorithm

As far as the numerical setting is concerned, the initial shape Ω_0 is supplied through its signed distance function ϕ_0 , e.g. as a $\mathbb{P}^1$ piecewise affine function on the mesh of the design space D . A remeshing procedure is employed to compute a new mesh $\mathcal{T}_0$ of the design space in which the structural interface is explicitly discretized. The mesh $\mathcal{T}_0$ contains naturally a computational mesh of the shape Ω_0 as a subdomain. The complete numerical procedure allows to generate a sequence of meshes $\mathcal{T}_k$ $\{k=0, \dots\}$. Each mesh $\mathcal{T}_k$ contains a submesh corresponding to a body-fitted discretization of the shape Ω_k . At each iteration k the domain evolution is achieved numerically by the following steps:

1. Computation of the signed distance function to the shape Ω_k on the vertices of the mesh $\mathcal{T}_k$;
2. Evaluation of the direct and the adjoint physical states on the computational mesh of the shape Ω_k ;
3. Evaluation of the objective function and constraints values on the shape Ω_k ;
4. Evaluation of physical and geometrical sensitivities on the vertices of the structural interface;
5. Combination of physical and geometrical sensitivities and resolution of (4) on $\mathcal{T}_k$ in order to select a descent direction θ_k ;
6. Selection of a pseudo time t_k and resolution of the advection equation (2) on the mesh $\mathcal{T}_k$ to get a $\mathbb{P}^1$ piecewise affine function of the domain ϕ_{k+1} ;
7. Check geometrical conditions for the acceptance of the implicit shape ϕ_{k+1} ;
8. When all the geometrical requirements are fulfilled, remesh $\mathcal{T}_k$ and generate a new mesh $\mathcal{T}_{k+1}$ to fit the structural interface of the implicit domain ϕ_{k+1} .

The algorithm ends whenever a maximum number of design steps is reached or when the merit function measuring jointly the decrease of the objective function and the non-violation of the constraints fails to decrease.

Remark 2 The variational formulation (4) is solved using $\mathbb{P}^1$ finite elements. The level set transport Eq. (2) is solved by a spatial first-order numerical scheme based on the method of characteristics. The variational formulations associated to the evaluation of values and sensitivities of each optimization criterion are discretized using linear or high-order elements, depending on user requirements.

6.2 Stress Sensitivity Evaluation on the Structural Interface

In stress-based optimization problems the integrand appearing in (8) has to be evaluated on the structural interface. To achieve this goal the sensitivity field in (8) is extrapolated from the stress evaluation points to the structural interface nodes. The nodal extrapolation is achieved by a least-squares approach based on a finite element interpolation. Eventually, for a given node, the nodal value is computed by weighting the values over the elements sharing the node. We consider the following smoothing function inside each finite element

$$\hat{s}(x) = \sum_{i=1}^n N_i(x) \hat{s}_i(x), \quad (15)$$

where $\hat{s}_i$ are the nodal unknown values, N_i the shape function at node i in the considered finite element. For each finite element, these unknowns are defined as the minimizer of a discrete functional

$$\chi(\tilde{\sigma}) \equiv \sum_{k=1}^{n_{GP}} (s(\xi_k) - \tilde{s}(\xi_k))^2 = \sum_{k=1}^{n_{GP}} (s(\xi_k) - \sum_{i=1}^n \tilde{s}_i N_i(\xi_k))^2, \quad (16)$$

where n_{GP} and $\xi_{k \{k=1, \dots, n_{GP}\}}$ are respectively the number and the coordinates of the evaluation points (i.e. the Gauss integration points) inside the element. Under the assumption $n < n_{GP}$, the minimization of (16) implies the resolution of the following linear system (hereafter written in matrix form)

$$\hat{s} = M^{-1} P s. \quad (17)$$

Note that the matrices $P \in \mathbb{R}^{n \times n_{GP}}$, $P_{ik} = N_i(\xi_k)$ and $M = P P^T \in \mathbb{R}^{n \times n}$ can be pre-calculated in the reference finite element, resulting in a very efficient extrapolation procedure. Note that the above general procedure can be used for the extrapolation of stress-based quantities (equivalent stresses, elastic density energy for example) from Gauss integration points to mesh nodes. In stress-based optimization problems, the described procedure is used to extrapolate the stress-based integrand appearing (8) to the structural interface nodes.

Remark 3 An alternative approach consists in replacing the discrete functional (16) by the following functional

$$\bar{\chi}(\tilde{s}) \equiv \int_e (s - \tilde{s})^2 dx = \int_e (s - \sum_i^n N_i \tilde{s}_i)^2 dx. \quad (18)$$

where e denotes any element. In this case the matrices M et P need to be evaluated on each finite element making the extrapolation procedure more expensive.

7 Numerical Results

This section presents the numerical examples.

7.1 Minimum Stress Design of an L-Shaped Beam

Let us consider an L-shaped design domain D with a bounding box of size $2\text{ m} \times 1\text{ m} \times 2\text{ m}$. The beam is clamped on the plane $z = 2\text{ m}$ and submitted to a vertical load $g = (0, 0, 10)\text{ kN}$ on a small circular region of radius $r = 0.1\text{ m}$ on the plane $x = 2\text{ m}$, as represented in Fig. 2–left. The Young modulus is equal to 210 GPa and the Poisson ratio equals 0.3 . The Dirichlet and Neumann boundaries are surrounded by two non-optimisable regions, as represented in Fig. 2–right. Here the linear elastic system (5) is solved using $\mathbb{P}^2$ finite elements. A 5-point Gauss integration rule is used in each tetrahedral finite element.

The goal is to minimise the global Von Mises indicator (7) under a volume constraint. The target volume is set to 0.7 m^3 . For the value $\alpha = 2$ in the objective function (7), the optimized design is represented in Fig. 3 and for $\alpha = 12$ the optimized design is represented in Fig. 4. Note that for a small value of the parameter α ($\alpha = 2$ here) the optimized design is reminiscent of one obtained when minimizing compliance. As the parameter α increases ($\alpha = 12$ here) the obtained design is modified in order to avoid stress concentration regions (in the vicinity of the sharp angle here), which are not captured using a compliance-type criterion.

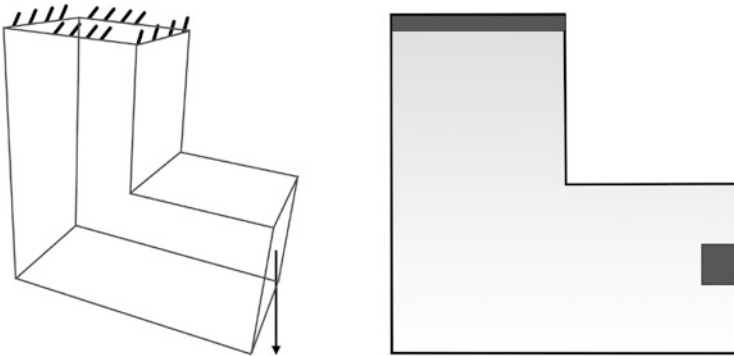


Fig. 2 Boundary conditions for the L-beam test case (left). Design space in light grey and non-optimizable regions in dark grey (right)

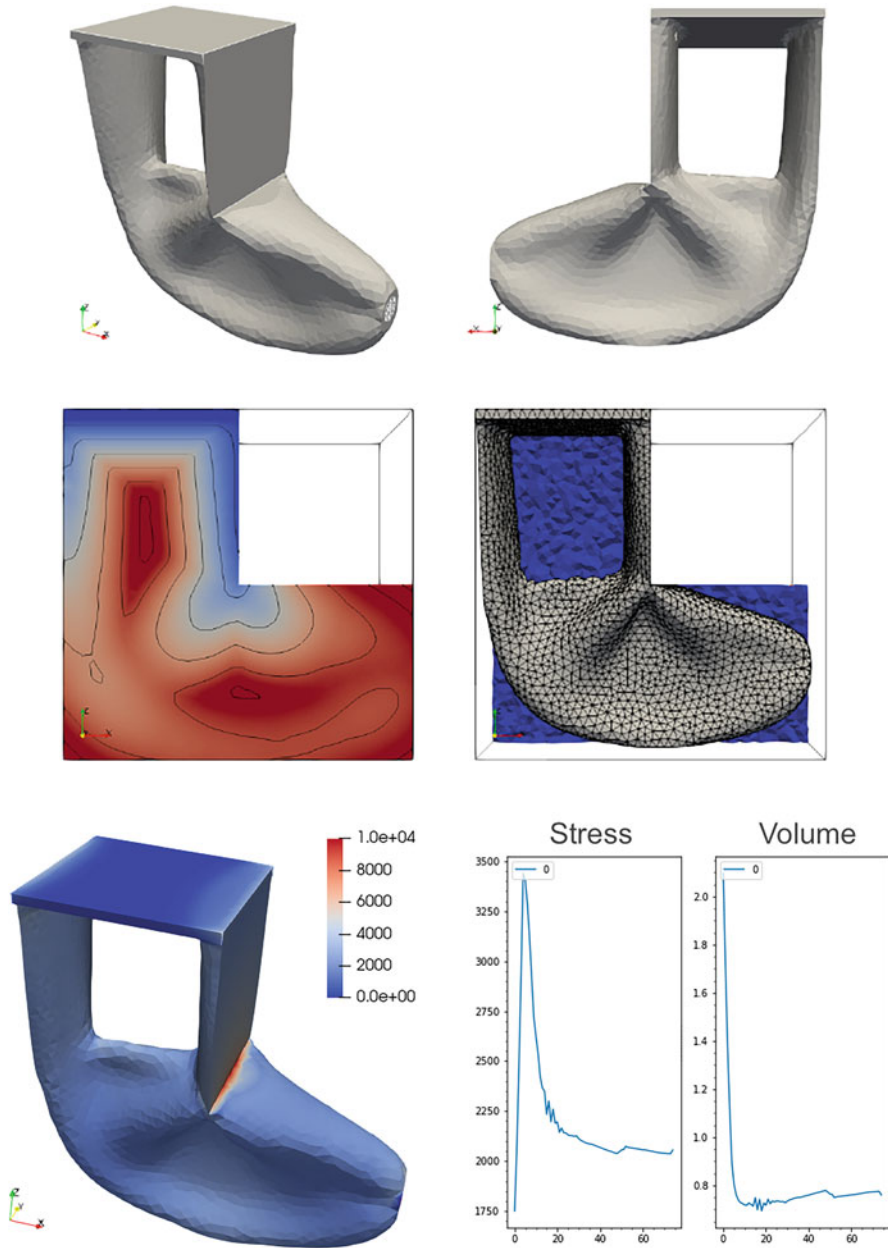


Fig. 3 Two views of the optimized design with $\alpha = 2$ (top). Level set function and body-fitted mesh of the optimized design (middle). Von Mises stress and convergence history for the L-beam test case with $\alpha = 2$ (bottom)

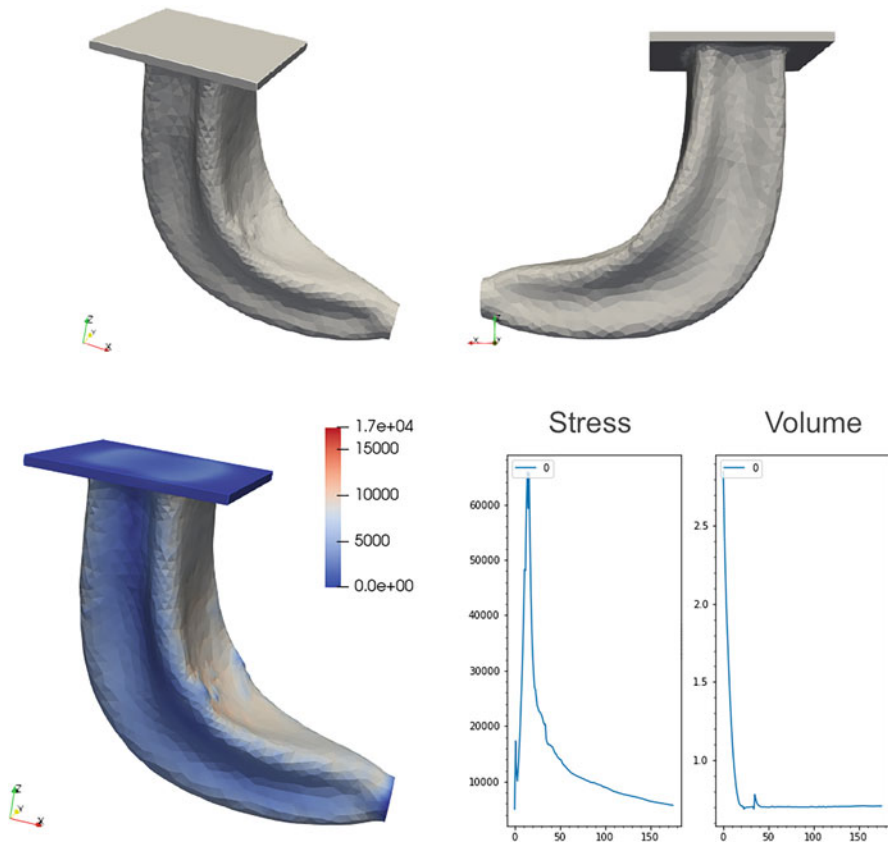


Fig. 4 Two views of the optimized design with $\alpha = 12$ (top). Von Mises stress and convergence history for the L-beam test case with $\alpha = 12$ (bottom)

7.2 Maximum Eigenfrequency Design of a Cantilever Beam

Let us consider a design space of size $2 \text{ m} \times 0.5 \text{ m} \times 1 \text{ m}$. The structure is clamped on the right plane $x = 0 \text{ m}$ and includes a concentrated tip mass localized at point $(2 \text{ m}, 0 \text{ m}, 0.5 \text{ m})$, as represented in Fig. 5.

The material density of the structure is set to 0.42 kg.m^{-3} . The point mass is set to 420 kg . The Young modulus is fixed to $32,000 \text{ Pa}$ and the Poisson coefficient

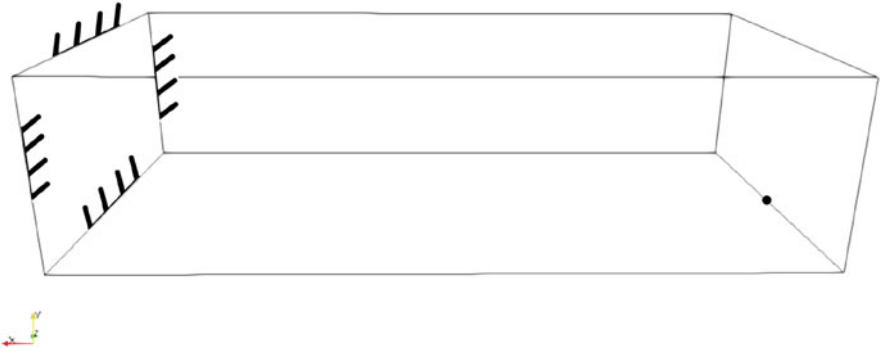


Fig. 5 Boundary conditions for the cantilever test case: clamped face (on the left) and point mass (on the right)

to 0.3. The goal is to maximize the first eigenfrequency of the structure under a volume constraint. The target volume equals $\frac{1}{2}V_0$ with V_0 the volume of the full design space. The optimized design achieved after 90 iterations is represented in Fig. 6.

Remark 4 Since the level set approach enables the generation of arbitrary topologies, some intermediate shapes can exhibit several disconnected components. In this case, all components connecting the supports of boundary conditions constitute the actual, useful shape. The others are spurious components that are detected and removed before remeshing to avoid the existence of artificial rigid body modes during the eigenvalues analysis.

8 Conclusions

A computational solution for shape and topology optimization using level sets and body-fitted meshes has been discussed and illustrated on some classical albeit challenging optimization problems.

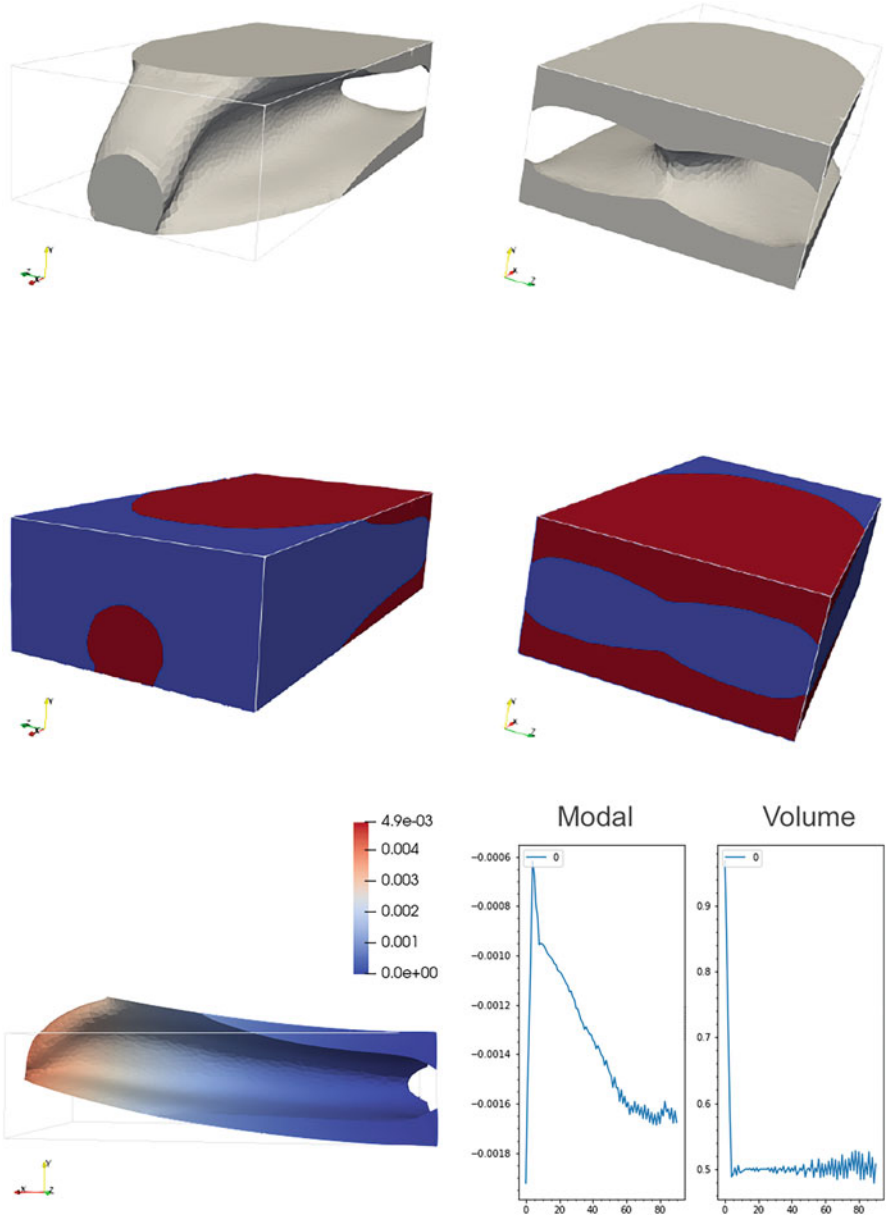


Fig. 6 Two views of the optimized design (top). Optimized design inside the design space. First eigenmode amplified by a factor of 70 and convergence history for the test case described in Sect. 7.2 (bottom)

Acknowledgments This research work has been carried out in the framework of IRT SystemX, Paris-Saclay, France, and therefore granted with public funds within the scope of “Programme d’Investissements d’Avenir”. The authors would like to sincerely thank the industrial and academic partners of TOP project and the mmg team for the ongoing development of the mmg software package.

References

1. Allaire, G.: *Conception Optimale de Structures*. Springer, Heidelberg (2006)
2. Allaire, G., Jouve, F.: A level-set method for vibration and multiple loads structural optimization. *Comput. Methods Appl. Mech. Eng.* **194**(30–33), 3269–3290 (2005). Elsevier
3. Allaire, G., Jouve, F.: Minimum stress optimal design with the level set method. *Eng. Anal. Bound. Elem.* **32**(11), 909–918 (2008). Elsevier
4. Allaire, G., Dapogny, Ch., Frey, P.: A mesh evolution algorithm based on the level set method for geometry and topology optimization, *Struct. Multidiscip. Optim.* **48**(4), 711–715 (2013). Springer
5. Allaire, G., Dapogny, Ch., Jouve, F.: Shape and Topology Optimization, to appear in *Handbook of Numerical Analysis 22, Geometric PDES* (2020)
6. Amstutz, S., Novotny, A.: Topological optimization of structures subject to von Mises stress constraints. *Struct. Multidiscip. Optim.* **41**(3), 407–420 (2010). Springer
7. Dalkint, A., Wallin, M., Tortorelli, D.: Eigenfrequency constrained topology optimization of finite strain hyperelastic structures. *Struct. Multidiscip. Optim.* **61**(6), 1–18 (2020). Springer
8. Dapogny, Ch., Dobrzynski, C., Frey, P.: Three-dimensional adaptive domain remeshing, implicit domain meshing, and applications to free and moving boundary problems. *J. Comput. Phys.* **262**, 358–378 (2014)
9. De Gournay, F.: Velocity extension for the level-set method and multiple eigenvalues in shape optimization. *SIAM J. Control. Optim.* **45**(1), 343–367 (2006). SIAM
10. Du, J., Olhoff, N.: Topological design of freely vibrating continuum structures for maximum values of simple and multiple eigenfrequencies and frequency gaps. *Struct. Multidiscip. Optim.* **34**(2), 91–110 (2007). Springer
11. Dunning, P., Kim, A., Mullineux, G.: Investigation and improvement of sensitivity computation using the area-fraction weighted fixed grid FEM and structural optimization. *Finite Elem. Anal. Des.* **47**(8), 933–941 (2011). Elsevier
12. Duysinx, P., Van Miegroet, L., Jacobs, T., Fleury, C.: Generalized shape optimization using X-FEM and level set methods. In: *IUTAM Symposium on Topological Design Optimization of Structures, Machines and Materials*, pp. 23–32. Springer, Berlin (2006)
13. Duysinx, P., Van Miegroet, L., Lemaire, E., Brûls, O., Bruyneel, M.: Topology and generalized shape optimization: Why stress constraints are so important?. *Int. J. Simul. Multidiscip. Des. Optim.* **2**(4), 253–258 (2008). EDP Sciences
14. Feppon, F., Allaire, G., Dapogny, C.: Null space gradient flows for constrained optimization with applications to shape optimization. *ESAIM Control Optim. Calc. Var.* **26**, 90 (2020). EDP Sciences
15. Giraldo-Londoño, O., Paulino, G.H.: A unified approach for topology optimization with local stress constraints considering various failure criteria: Von Mises, Drucker–Prager, Tresca, Mohr–Coulomb, Bresler–Pister and Willam–Warnke. *Proc. R. Soc. A* **476**(2238), 20190861 (2020). The Royal Society Publishing
16. Henrot, A., Pierre, M.: *Variation et Optimisation de Formes, une analyse géométrique*. Springer, Heidelberg (2005)
17. Holmberg, E., Torstenfelt, B., Klarbring, A.: Stress constrained topology optimization. *Struct. Multidiscip. Optim.* **48**(1), 33–47 (2013). Springer

18. Kang, Z., He, J., Shi, L., Miao, Z.: A method using successive iteration of analysis and design for large-scale topology optimization considering eigenfrequencies. *Comput. Methods Appl. Mech. Eng.* **362**, 112847 (2020). Elsevier
19. Le, C., Norato, J., Bruns, T., Ha, C., Tortorelli, D.: Stress-based topology optimization for continua. *Struct. Multidiscip. Optim.* **41**(4), 605–620 (2010). Springer
20. Moës, N., Dolbow, J., Belytschko, T.: A finite element method for crack growth without remeshing. *Int. J. Numer. Methods Eng.* **46**(1), 131–150 (1999). Wiley Online Library
21. Picelli, R., Townsend, S., Brampton, C., Norato, J., Kim, A.: Stress-based shape and topology optimization with the level set method. *Comput. Methods Appl. Mech. Eng.* **329**, 1–23 (2018). Elsevier
22. van Dijk, N., Maute, K., Langelaar, M., Van Keulen, F.: Level-set methods for structural topology optimization: A review. *Struct. Multidiscip. Optim.* **48**(3), 437–472 (2013). Springer
23. Van Miegroet, L., Duysinx, P.: Stress concentration minimization of 2D filets using X-FEM and level set description. *Struct. Multidiscip. Optim.* **33**(4–5), 425–438 (2007). Springer
24. Xia, Q., Shi, T., Wang, M.: A level set based method for topology optimization of continuum structures with stress constraint. In: 6th China-Japan-Korea Joint Symposium on Optimization of Structural and Mechanical Systems, Kyoto, Japan (2010)

Investigating Singularities in Hex Meshing



Dimitrios Papadimitrakis, Cecil G. Armstrong, Trevor T. Robinson,
Alan Le Moigne, and Shahrokh Shahpar

Abstract Hexahedral meshing of complex domains is a long-standing problem in engineering simulation. One strategy to achieve it is through multi-block decomposition. Recent efforts have focussed on deriving the block topology using frame-fields which aim to capture the desired mesh orientation throughout the domain. This reduces to determining the approximate position, orientation and connectivity of lines of mesh edges where the number of elements is different from what it would be in a regular mesh. These are known as mesh singularity lines and they form the framework of a block decomposition. However, frame fields often produce singularity lines which are connected in invalid configurations and cannot support a valid block topology. The contribution in this paper is to demonstrate how information encapsulated in the Medial Axis of a $3D$ domain can provide rational solutions to a number of meshing problems that have been identified in the literature as having no satisfactory automated solution. The approach is not yet a formal algorithm but provides extra insights that should assist in the development of one.

1 Introduction

In the recent years, a lot of effort has been expended on tackling the problem of automatically generating a hexahedral mesh for an arbitrary $3D$ domain. To achieve this, in [1], the authors propose generating multi-block decompositions for $2D$ domains based on an evolutionary algorithm and suggest an extension to $3D$ could

D. Papadimitrakis (✉) · C. G. Armstrong · T. T. Robinson
The Ashby Building, Queen's University Belfast, Belfast, UK
e-mail: dpapadimitrakis01@qub.ac.uk; t.robinson@qub.ac.uk

A. Le Moigne
Rolls-Royce plc, Group Business Services—IT, Product Development System, Derby, UK

S. Shahpar
Rolls-Royce plc, Innovation Hub, Future Methods, Derby, UK

be possible. The most promising recent techniques have relied on the generation of a smooth, boundary-aligned frame field [2]. A frame consists of three mutually perpendicular unit vectors suggesting the optimum orientation of a cubical element at that position. A frame field is a collection of such frames defined on each node (or element) of a tetrahedral mesh of the domain. The orientation of the frames evokes a collection of critical lines in the domain (called singularity lines) where the mesh connectivity is different from what it would be in a structured mesh. These lines form a network of lines (called a singularity line network) which defines a wire frame of a block decomposition of the domain. The existence of a valid singularity network in a general 3D domain ensures that the resulting hexahedral mesh conforms to the boundary, and optionally controls the transitioning of mesh density.

Tackling the problem of producing automatically, robustly and efficiently a frame field that can be used for hex-mesh generation is a topic of extensive research [3–8]. Based on these frame fields, parameterizations are built [9] or partition surfaces are created [8, 10, 11] which are then used to support the construction of a hexahedral mesh. Efforts have also been made to use the topological and geometrical properties of the medial object along with its proximity information [12, 13].

There are constraints on how the irregular edges can be connected, which means that in some cases the singularity networks derived from frame fields do not correspond to a valid block structure, requiring the generation of hex-dominant meshes [14–17]. This led to the concept of manually providing a valid singularity line network (or correcting manually an existing one) and then generating automatically a frame field that adjusts both to the boundary constraints and to the existing singularity line network [18, 19]. In a later work [20], the authors propose ways to automatically correct frame-fields to remove common invalid network features. However, in most cases this results in pushing singularity lines and element distortion close to the boundary, thus reducing the element and solution quality there.

A further problem in hex mesh generation is that mesh singularities have a non-local effect. This means that the mesh round a local feature can affect the mesh at a remote location, or equivalently, that a regular mesh on the large scale, in for example an external aerodynamic domain, cannot interface properly with small local features.

In this work, the focus is on invalid features in singularity line networks and how they can be corrected to admit a valid hexahedral mesh. Based on element connectivity constraints expressed by hex-mesh primitives that were first presented in [21], techniques to correct invalid singularity line networks are presented. Finally, the medial axis of the domain provides proximity and orientation information, which is utilised to correctly place the required additional singularity lines.

This paper is structured through a number of case studies rather than a fully automatic algorithm, but the insights obtained are generally useful. It is organised as follows. First, some preliminary information about the medial axis of a 3D domain is given. Important characteristics of a hexahedral mesh that conforms to specific boundary constraints are described next. After that, hex mesh primitives

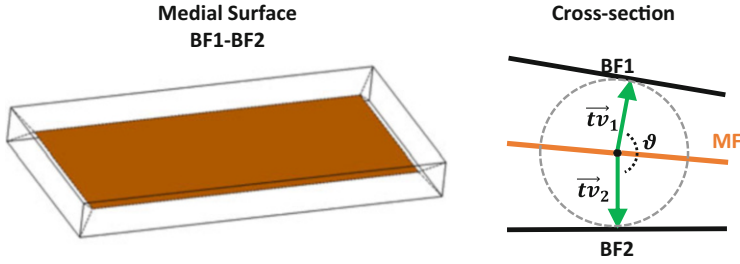


Fig. 1 Medial object of a thin rectangular plate

are used to describe important properties of singularity line networks. Based on these primitives, a strategy to correct invalid singularity line networks is presented through examples that state-of-the-art methods fail to resolve. Finally, conclusions are drawn, and possible paths of future research are discussed.

2 Medial Axis

The medial object (MO), medial axis (MA), or skeleton of a subset D of R^3 , is the locus of points which are centres of spheres which are maximal in D , together with the limit points of this locus [22]. It has its own structure consisting of medial surfaces, edges and vertices. In general, a point on a medial surface is equidistant from two points on the boundary, a point on a medial edge to three and a medial vertex to four. Degenerate medial edges and vertices may be equidistant from more than this number of entities, and curvature or finite contact between the maximal sphere and the boundary is possible. The defining entities of a given medial entity identify parts of the object boundary in geometric proximity. The vectors that connect a point on the medial axis with the corresponding points on the boundary (touching vectors) provide directional information which can be used to locate singularity lines in the interior of the domain [13]. An example of a medial object with a highlighted medial surface is depicted in Fig. 1. The touching vectors highlighted in green connect the point on the medial surface MF with the closest points on the boundary faces BF1 and BF2, which are in proximity. BF1 and BF2 are roughly parallel to each other $\theta \approx 180^\circ$, implying that a regular structured mesh should lie between them.

3 Hexahedral Mesh

Let D be a 3D domain with closed boundary ∂D . A hexahedral mesh that approximates this domain is defined as a graph $G = V, E, F, H$ consisting of a set of vertices $V = v$, a set of edges $E = e$, a set of faces $F = f$ and a set

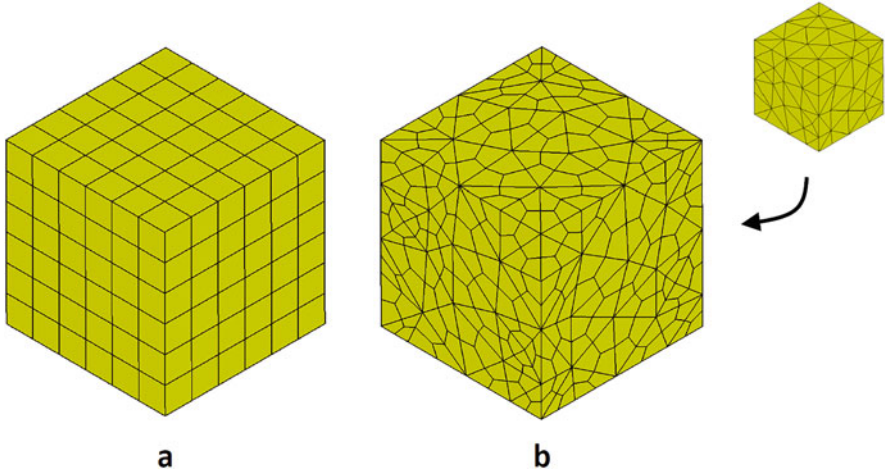


Fig. 2 Structured (a) and unstructured (b) hexahedral mesh

of hexahedral blocks $H = h$. It is a volumetric mesh whose cells/elements are all hexahedra. Such a mesh forms a discrete representation of the domain D and its boundary ∂D .

A hexahedral mesh can be structured or unstructured. Two examples of hexahedral meshes that approximate a cube can be seen in Fig. 2. In (a), a structured hexahedral mesh is depicted. In (b), a tetrahedral mesh has been converted into an unstructured hexahedral mesh by dividing each tetrahedron into four hexahedra. The difference between the two meshes lies in the fact that the one in (a) has a regular block structure while for the one in (b) there is no perceptible structure. Almost all the elements in (b) are significantly distorted.

3.1 Singularity Lines

A number can be assigned to each interior mesh edge $e \in E$ of a hexahedral mesh H , which describes the number of elements that are incident to it, Fig. 3. This number is called the valence of the edge and can be any positive integer number; $val(e) \in \mathbb{N}_{\geq 1}$. Based on the valence, a mesh edge can be described as regular or singular. To do that, the index of an interior edge is defined as:

$$index(e) = val(e) - 4.$$

If $index(e) < 0$ it is a negative singular edge, if $index(e) > 0$ it is a positive singular edge and if $index(e) = 0$ the edge is regular. In this paper, positive singularity lines are always coloured blue whilst negative singularity lines are coloured red.

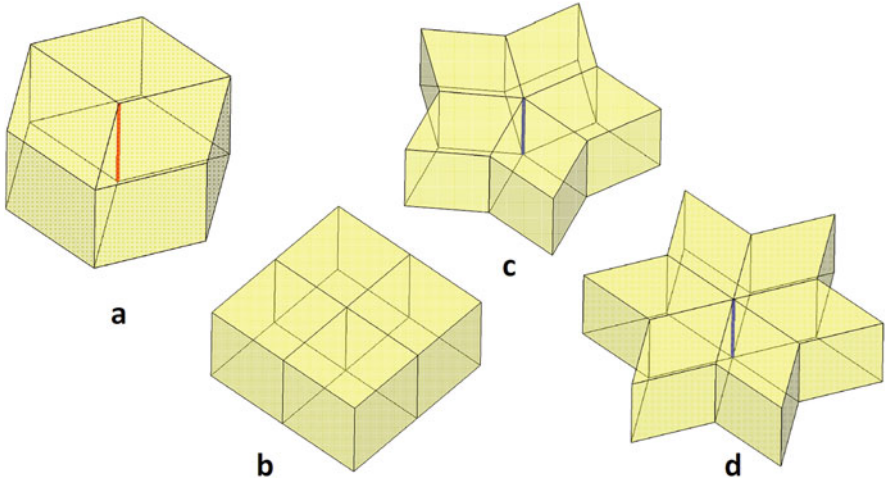


Fig. 3 The number of elements attached to a mesh edge can vary. Three elements (a). Four elements (b). Five elements (c). Six elements (d)

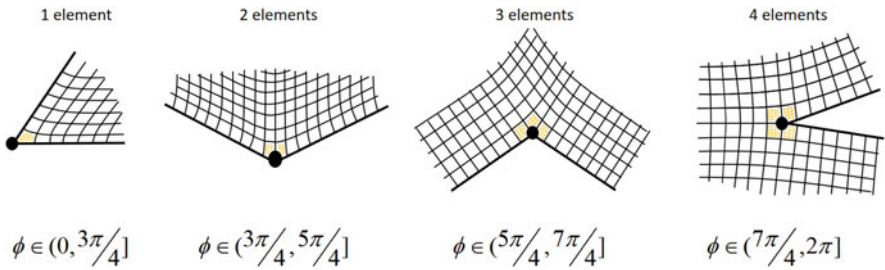


Fig. 4 Index based on edge dihedral angle (based on [23])

For mesh edges that lie on the boundary of the domain this definition changes. To decide whether a boundary mesh edge is singular or not, the number of elements attached to it has to be compared to the ideal number based on the local dihedral angle. For example, for a mesh edge on a boundary edge with a dihedral angle of 270° , the ideal number is 3 and thus:

$$index(e) = val(e) - 3.$$

For dihedral angles which are not exact multiples of 90° , the index is usually chosen based on the limits shown in Fig. 4.

In Fig. 5 (right) two hexahedra are attached to the highlighted boundary mesh edge instead of three, Fig. 5 (left), which implies it is a negative boundary singular edge with $index = -1$. If one of the boundary mesh edges connecting to a boundary vertex is singular, then the boundary vertex is also singular. It has to be noted that the definition used here is different than that given in [18] where both boundary

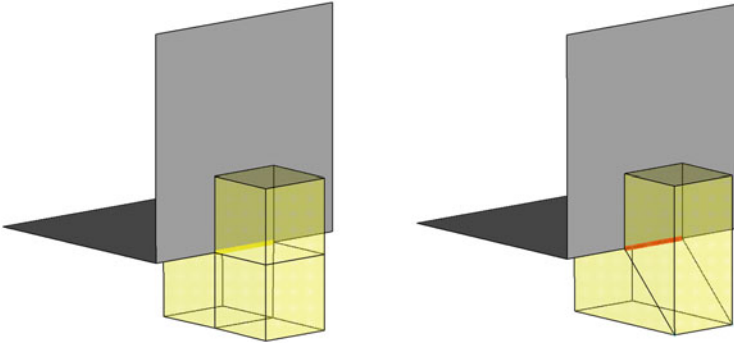


Fig. 5 Mesh configuration on a boundary edge with dihedral angle 270° . Three elements for a regular mesh edge (left). Two elements for a singular mesh edge (right)

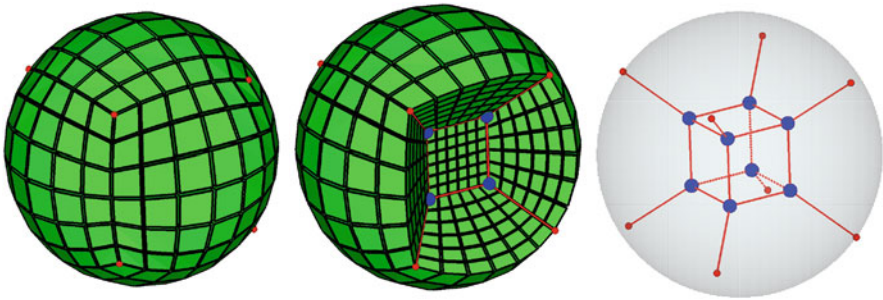


Fig. 6 Quad mesh on the boundary of a sphere (left). Internal hex-mesh (middle). Singularity line network (right)

and interior mesh edges connected to other than four elements are considered to be singular.

In a hexahedral mesh, singular mesh edges connect to each other to form complete lines, called singularity lines, which form loops or connect to singular nodes. These nodes either lie on the boundary or in the interior of the domain. In the latter case, more than one singularity line joins at the singular nodes forming the outlines of what are called hex meshing primitives. To illustrate how singularity lines behave in a three-dimensional domain, the example of a hex-meshed sphere is depicted in Fig. 6. Singularity lines join to form a singularity line network which gives a wire frame of a block decomposition of the domain. To better understand the structure of the block decomposition described by the singularity line network, partition surfaces must be described.

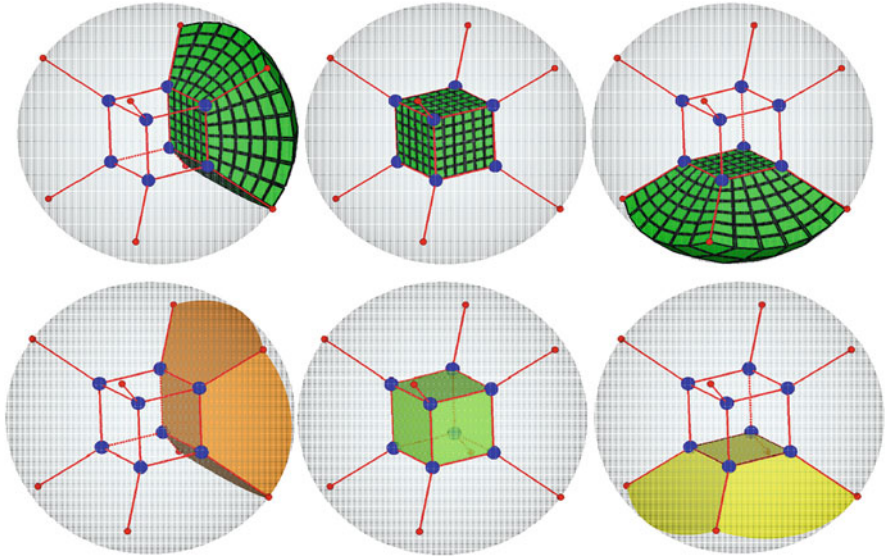


Fig. 7 Three hex-meshed block-regions of the sphere in Fig. 6 (top). Corresponding partition surfaces (bottom)

3.2 Partition Surfaces

A hexahedral mesh is, by definition, a collection of hexahedral elements. These elements can be grouped in regions or blocks, each of which has a regular mesh structure, with no singular mesh edges and, thus, no singularity lines. For the mesh of the sphere shown in Fig. 6, three such regions can be seen in Fig. 7. All singularity lines of the mesh lie either outside or on the boundary of those regions, but not in the interior. These meshes correspond to the block regions of the sphere. The outer faces of the outer hex elements of these regions define surfaces which either correspond to part of the external boundary faces of the domain or separate them from all other regions. The internal partition surfaces can be seen in the bottom row. They are called partition surfaces since they partition the domain into regions where a regular mesh can be constructed. Partition surfaces also emanate from concave boundary features of the domain (Fig. 8 middle right).

3.3 2D Boundary Singularities

Similar to a hexahedral mesh, a quad mesh can be thought of as a graph $Q = \{V, E, F\}$, where $V = \{v\}$ is the set of quadrilateral vertices v , $E = \{e\}$ is the set of quadrilateral edges e and $F = \{f\}$ is the set of quadrilateral faces f . For every

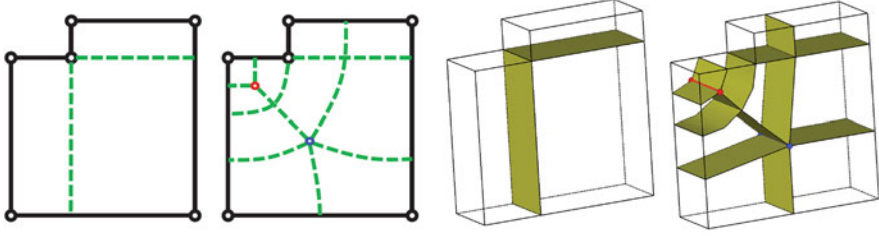


Fig. 8 Decomposition for the same surface with zero singularities (left) or a pair of positive and negative singularities (middle left). In this 2D model $\#V = 6$, $\#E = 5$ and $\#F = 1$, therefore $\chi = 0$. The equivalent 3D partition surfaces are given on the right

internal vertex v of the quad mesh, $val(v)$ is the number of quad faces adjacent to it and $index(v) = val(v) - 4$ is a quantity that describes the vertex. For a regular vertex, $index(v) = 0$. A vertex with $index(v) < 0$ is called negative singular vertex and a vertex with $index(v) > 0$ a positive singular vertex. Similar to singular edges, singular vertices with $index(v) = -1$ or $index(v) = +1$ are sufficient to generate a valid mesh. As with the hex mesh case, to decide whether a boundary mesh vertex is singular or not, the number of elements attached to it has to be compared to the ideal number based on the local geometric characteristics of the boundary using the same equation as was used for hex mesh edges in Fig. 4. For example, for a mesh vertex at the boundary which has an included angle of 80° , the ideal number of incident elements is 1 and thus:

$$index(e) = val(e) - 1.$$

Let $S_V = \{v | index(v) \neq 0\}$ be the set of all internal singular vertices of a quadrilateral mesh on a surface R bounded by a set of curves C . Let also $B_V = \{v \in C\}$ be the set of all boundary vertices. Following the work in [22], given n^+ the number of positive singular vertices and n^- the number of negative singular vertices, the net sum $N = n^+ - n^-$ of singularities is calculated by

$$N = \sum_{i=1}^{\#S_V} index(v_i) = -4\chi(R) + \sum_{j=1}^{\#B_V} (2 - n_{v_j})$$

where, $\chi(R)$ is the Euler characteristic of the surface and n_{v_j} is the vertex classification of a boundary vertex. The Euler characteristic of a surface R with $\#V$, $\#E$, $\#F$, the number of vertices, edges and faces is given by

$$\chi(R) = \#V - \#E + \#F.$$

This equation relates the number of singularities of a quad mesh with pre-defined characteristics of the surface. It can be used to identify the net sum of singularities

on the surface without even having generated the quad mesh. Note that while this equation identifies the net sum of singularities, it does not locate their position nor provide the exact number of each. If, however, a quad mesh that conforms to the vertex classification of the boundary of the surface is created, then the net sum of its singularities will equal that given by (1). In Fig. 8 (left), a planar surface with $N = 0$ is given. In the left diagram no singularities are used to decompose the model. In the middle left diagram, one negative and one positive singularity are placed, resulting in a different decomposition. This example indicates that the net sum of the singularities on a surface does not uniquely define how this surface will be decomposed.

Since the quadrilateral mesh on the boundary of the domain forms the outer boundary of the hexahedral mesh, singularity lines ending on the boundary must connect to the singular vertices of the quadrilateral surface mesh. According to equation (1), singularities on surfaces must respect certain topological constraints which define their number and type. These constraints, along with their position on the surface, restrict the volume singularity line network and force it to have a certain number and type of singularity lines [23]. This can sometimes have a negative effect on the ability of the singularity line network to induce a valid block decomposition for hexahedral meshing. In Fig. 8 (middle right), no singularity lines are required for a block decomposition to be created. The partition surfaces that emanate from the concave boundary edge are enough for that task. However, if a pair of positive and negative singularities is placed on the front and back boundary faces, then a pair of positive and negative singularity lines that connects them must be introduced to the domain. This changes the final structure of the block decomposition (right). In both cases, the net sum of singularities on the boundary faces equals zero and equation (1) is satisfied.

4 Singularity Line Network

Identifying a singularity line network whose partition surfaces, together with the partition surfaces that emanate from concave features of the domain, are sufficient to produce a block decomposition of the domain seems to be the way forward in generating automatically a hexahedral mesh for an arbitrary 3D domain. A lot of focus have been given recently in the problem of identifying such networks based on a frame field constructed on top of a tetrahedral mesh and using it to guide the generation of a hexahedral mesh either based on parameterizations [5] or by first generating partition surfaces that emanate from them [8, 10, 11]. Even though a lot of progress has been made, problems still exist and singularity lines that cannot support a valid block decomposition of the domain are commonly generated even for simple models [24]. To bypass these problems the authors in [18] suggest manually correcting invalid singularity line networks. However, this is not an easy task for a non-expert and the associated challenge increases considerably for more complex models. More recently, the authors in [20] propose steps to automatically correct

invalid singularity line networks. However, they suggest their approach only works for invalid configurations that appear close to the boundary of the domain.

To better understand the behaviour of singularity lines and consequently, the structure of a singularity line network, it is important to study the possible ways they can connect to each other in the interior of the domain. This connectivity can be described based on the hex-mesh primitives first proposed in [21, 25] and then rediscovered more recently in [18].

4.1 Hex-Mesh Primitives

Hex mesh primitives describe the ways $+1$, -1 singularity lines can connect to each other so that a hexahedral mesh can be constructed around them. In Fig. 9, all the primitives are shown, together with their corresponding singularity line network and the structure of the partition surfaces there.

In [21, 25], these primitives were used together with the medial axis of the domain to construct a decomposition into regions that can be easily hex-meshed with midpoint subdivision [26]. More precisely, primitives were placed around all medial entities. The process started by placing them along medial edges based on the topological connectivity between the three defining entities of a medial edge. Then, primitives were placed on medial vertices and medial faces to fill the entire domain. The singularity lines implied by the primitives formed the entire singularity line network in the interior of the domain.

There are 11 hex mesh primitives including the simple hexahedron with no singularity lines. This means that there are 10 different ways that singularity lines can connect to each other. Thus, the number and the type of singularity lines that can connect at a singular vertex is highly constrained. Below each primitive a set of three numbers (a,b,c) is given. (a) represents the number of the three-sided faces on the primitive indicating that a negative singularity exists, (b) represents the number of four-sided faces on the primitive indicating no singularities and (c) the number of five-sided faces on the primitive with a positive singularity. For example, the primitive (2, 3, 0) has two three-sided, three four-sided and zero five-sided faces. Thus, two negative singularity lines connect at the centre of the primitive or one single singularity line connects the two opposite three-sided faces. Similarly, at the (1, 3, 3) primitive, one negative and three positive singularity lines are connected at an interior point.

4.2 Fundamental Primitives

Of all the primitives that have singularity lines, three are the fundamental ones while all the others can be converted to a combination of those three. These are the triangular prism (2, 3, 0), the pentagonal prism (0, 5, 2) and the “cube-with-

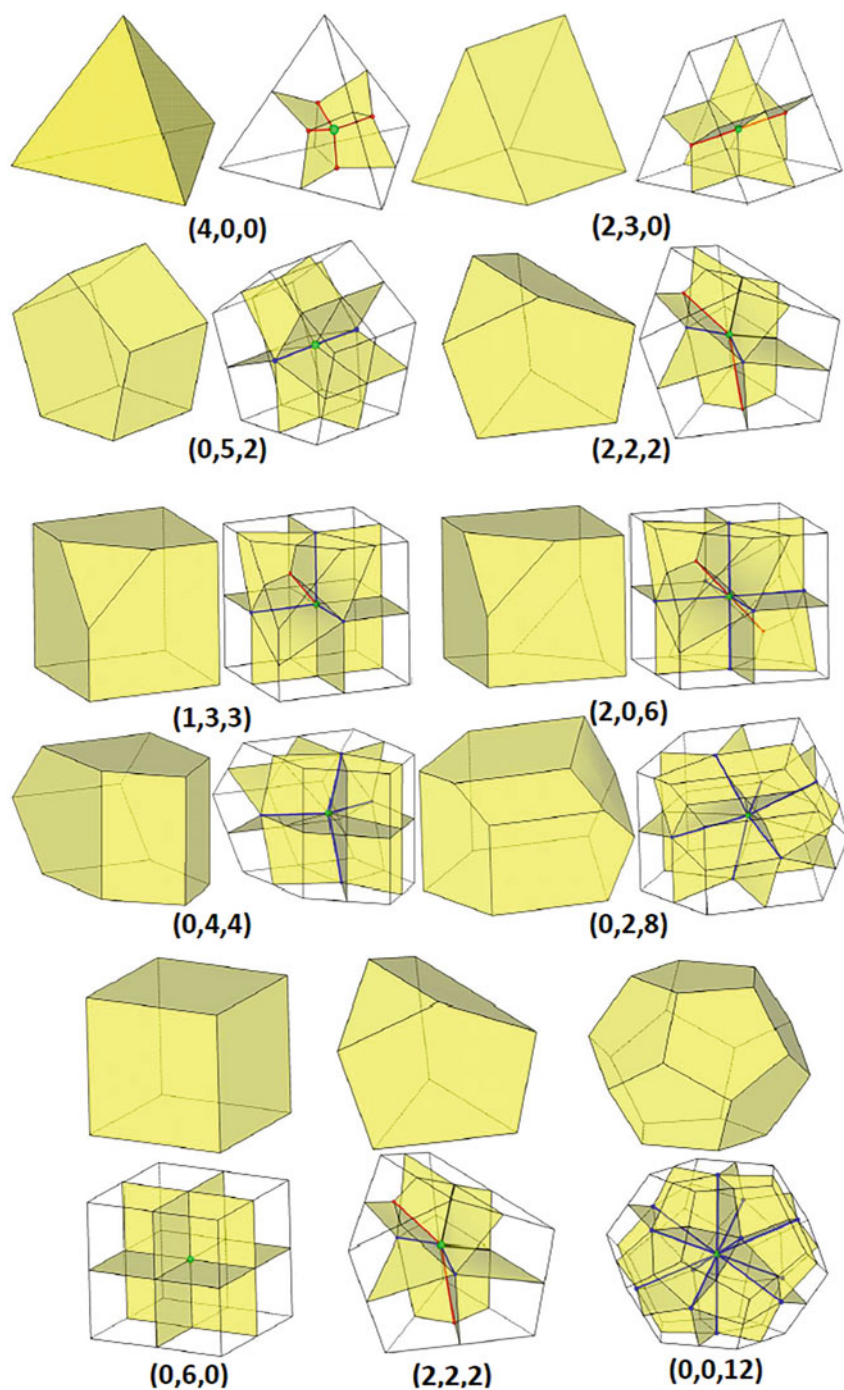


Fig. 9 Hex mesh primitives with their singularity line network and partition surfaces [21, 25]

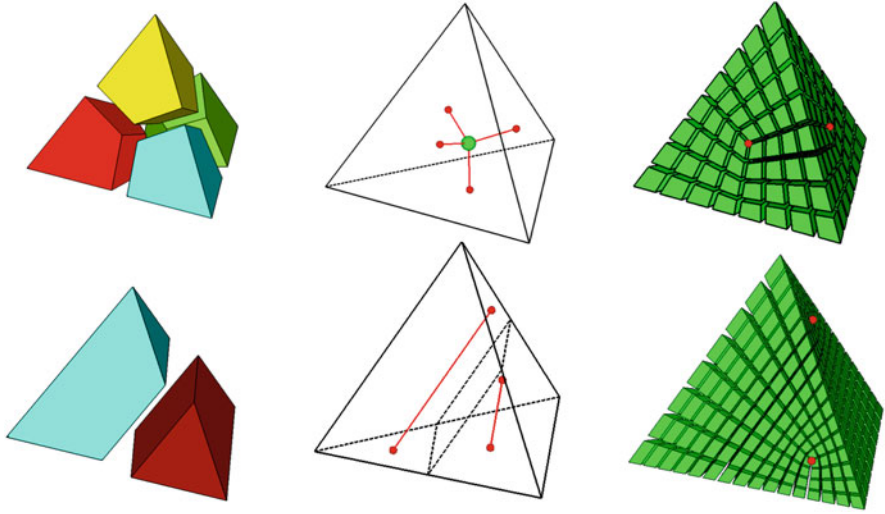


Fig. 10 Breaking a $(4, 0, 0)$ primitive in two $(2, 3, 0)$ fundamental primitives

a-corner-chip-off” $(1, 3, 3)$ primitives. For example, in Fig. 10, it is shown how a tetrahedron, a $(4, 0, 0)$ primitive can be represented by two $(2, 3, 0)$ fundamental primitives if the four singularity lines connecting at a singular vertex in the interior are separated into two negative singularity lines passing close to each other. The corresponding block decomposition and their hexahedral meshes are different. In a similar way, other primitives can be separated into combinations of the fundamental primitives.

4.3 $(1, 3, 3)$ Fundamental Primitive

Although mesh distortion is higher around singularity lines, they are necessary in order to generate a block decomposition of a 3D domain of arbitrary shape, e.g. Fig. 6. Understanding how singularity lines behave is thus crucial to creating a valid hex mesh or multi-block decomposition.

Based on the fundamental primitives, the only way singularity lines can join is through the $(1, 3, 3)$ fundamental primitive. In this case, a negative singularity line enters the domain through the three-sided face and disappears at the singular vertex in the interior where it connects with three positive singularity lines. This primitive can be thought of as generated by removing a tetrahedron or one octant of a sphere from one of the corners of a cube as can be seen in Fig. 11.

With this primitive, singularity lines can be redirected in the interior of the domain to keep mesh distortion local and ensure that a block decomposition can be maintained. To illustrate the range of mesh patterns that can be produced, four possible decompositions of the cube will be described.

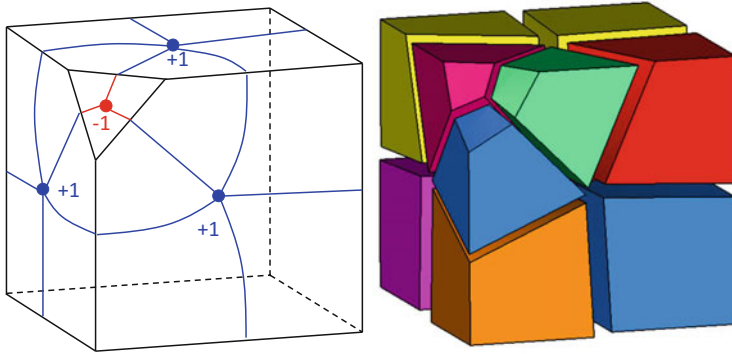


Fig. 11 $(1, 3, 3)$ primitive generation by removing a tetrahedron from a corner of a cube

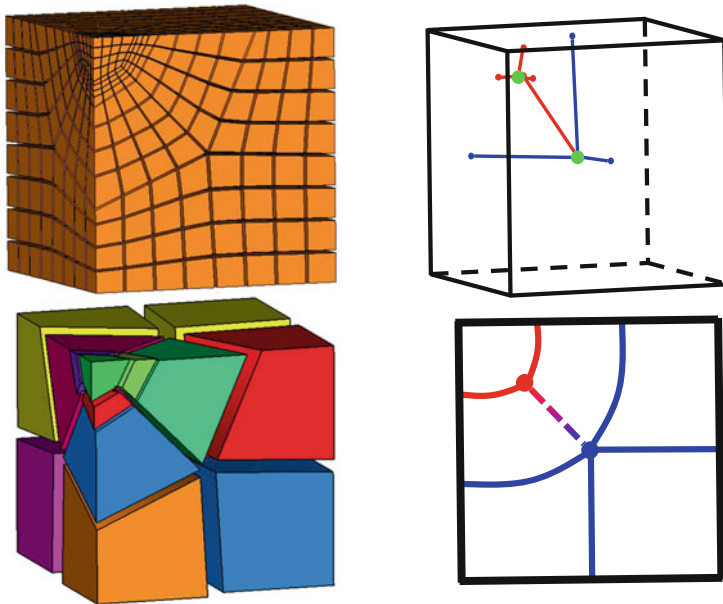


Fig. 12 Isolated singularity line network on the interior of a cube generated by using one $(1, 3, 3)$ fundamental primitive

A cube by itself is a primitive of type $(0, 6, 0)$. It can be hex meshed with no singularity lines present since all faces are four-sided. However, by splitting the cube into $(1, 3, 3)$ and $(4, 0, 0)$ primitives we can selectively introduce pairs of positive and negative singular vertices on three faces of the boundary of the domain, which provides opportunity for controllable mesh transition [27]. This is shown in Fig. 12. The net sum of singularities at each boundary face is zero. The singularity line network consists of 4 negative and 3 positive singularity lines. By placing eight

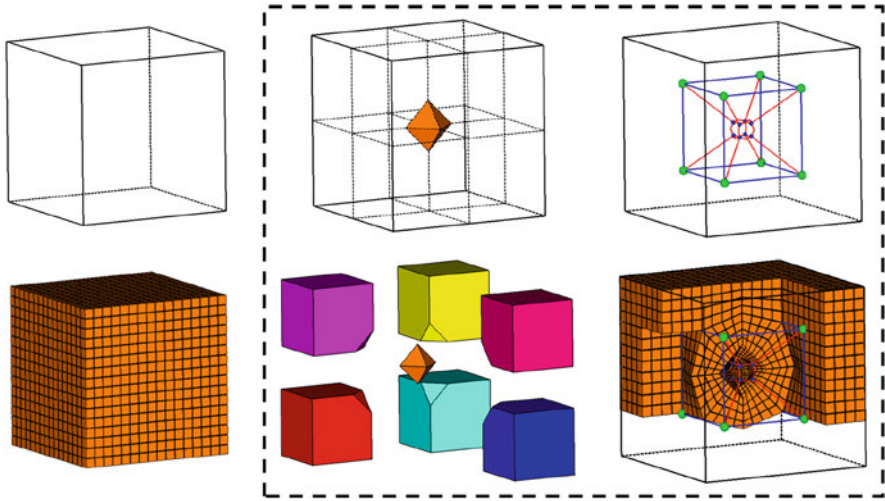


Fig. 13 Isolated singularity line network in the interior of a cube generated by using eight $(1, 3, 3)$ and $(4, 0, 0)$ primitives

$(1, 3, 3)$ fundamental primitives on the corners of the cube and eight tetrahedra at the centre of the domain a fully isolated network of positive and negative singularity lines can be generated in the interior of the domain, as can be seen in Fig. 13. The interior octahedron is equivalent to a spherical cell with increased mesh density in the interior of the domain. The singularity line network consists of 20 negative and 12 positive singularity lines that have no interaction with the boundary of the cube. This implies that any mesh mating to the faces of the cube is undisturbed.

If fewer primitives of type $(1, 3, 3)$ are used, then the decomposition of the domain changes and the singularity line network emerges on the boundary. For example, in Fig. 14 four such primitives are used and only one of the boundary faces of the cube is intersected by four positive and four negative singularity lines. On that face the net sum of singularities remains zero. In total, the singularity line network consists of 12 negative and 8 positive singularity lines and is half of that shown in Fig. 13.

In a similar way, by using two of these primitives then one quarter of the singularity line network is obtained and two boundary faces are now connected to singularity lines. The corresponding hexahedral mesh, the decomposition of the domain into primitives and the singularity line network are all depicted in Fig. 15. Two possible connectivity patterns are highlighted. By using fewer primitives the singularity lines are redirected and connect to another boundary face, as opposed to the case of Fig. 14. Again, the net sum of singularities at all boundary faces remains zero.

All of the above networks can be realised by the relative position between the cube and a sphere, where the interior cell of Fig. 13 represents the decomposition when a spherical cell is inside the cube. Depending on whether the sphere intersects

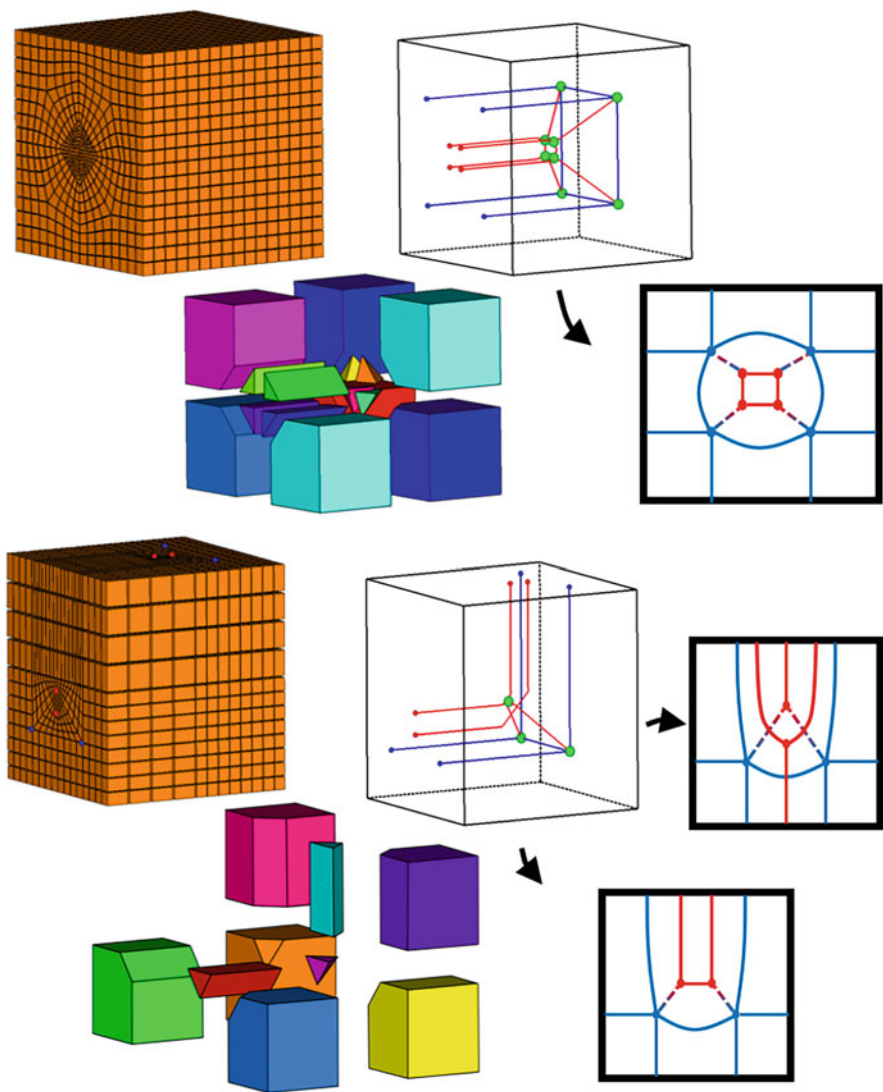


Fig. 14 Decompositions of a cube by using 4 (top) and 2 (bottom) (1, 3, 3) fundamental primitives

with a face, an edge or a vertex of the cube the three networks shown in Fig. 12 and Fig. 14 are obtained. This is illustrated in Fig. 15. Furthermore, by closely observing the block topology and the hexahedral meshes produced, it can be seen that, with the aid of the (1, 3, 3)/(4, 0, 0) primitives, mesh constraints can be redirected and fine meshes can be localised from coarser ones away from the singularity lines.

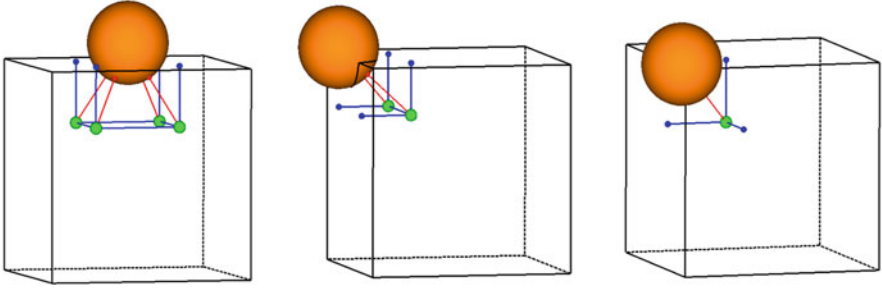


Fig. 15 Cube—sphere interaction and the singularity line network it induces in the cube

5 Correcting Singularity Line Networks

Herein, instead of using primitives as building blocks and placing them directly on the medial object [21], it is proposed to use them to correct invalid singularity line networks that are frequently created by frame-field methods. The most common invalid singularity line feature is that where positive and negative singularity lines appear to connect directly. It is invalid because this arrangement would indicate three elements directly interfacing with five which is not possible.

5.1 Notch Model

In many cases, existing state-of-the-art methods (e.g. [5]) fail to identify all necessary singularity lines to construct a valid singularity line network. The simplest example is that of the notch model shown in Fig. 16. As can be seen in Fig. 16 (left), frame-field methods tend to give rise to a singularity line network that consists of a positive and a negative singularity line (here represented by the corresponding singular tet faces with blue and red colour respectively) connecting in the interior of the domain close to the concave boundary edge. These singularity lines start from the corresponding point singularities on the 5-sided and 3-sided boundary faces.

The medial faces highlighted in orange in Fig. 16 (right) define the region which is close to the concave boundary edge and the 6-sided boundary faces. The 3- and 5-sided faces of the model will contain a negative and positive singularity vertex, respectively. As these faces are offset inwards, the offset surface vertices follow medial edges, Fig. 16 (right), and the offset surfaces shrink to zero area close to the concavity. At this point the singularity lines propagated from the singularity vertices in the interior of the 3- and 5-sided faces must interact with other singularity lines—they cannot just terminate in the interior of the volume.

This is the area where the corresponding singularity lines from the frame fields connect. Figure 16 (bottom), the singularity lines together with the medial object can be seen from two views. The singularity lines end up at the medial vertices

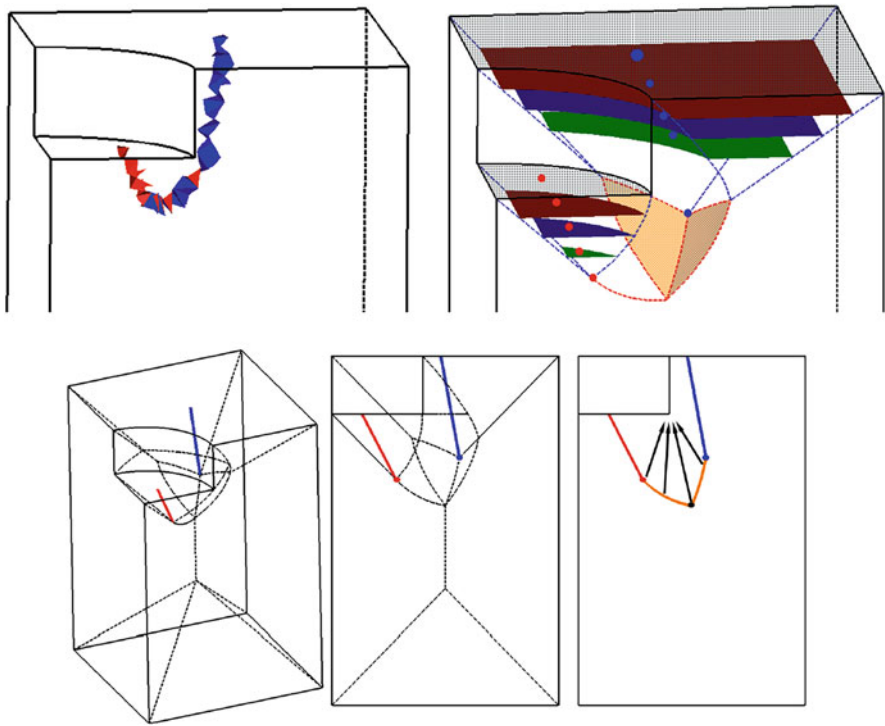


Fig. 16 Frame field analysis of notch model (top left). A positive and a negative singularity line join around the concave boundary edge where the boundary offsets shrink (top right). Singularity lines together with various medial entities (bottom)

where the offset surfaces shrink to zero area. From a side view the association of the medial entities with the concave boundary edge is shown. It is around these medial entities where the singularity lines identified from the frame-fields connect in an incompatible way. It is not possible to connect a triangular prism (a negative singularity) with a pentagonal one (a positive singularity).

So, based on the hex mesh primitives, such a configuration is not possible. One solution to generate a valid singularity line network would be to separate the negative and the positive singularity lines and extend them until they connect to the bottom boundary face. Such a solution is equivalent to propagating the concave edge constraint to the bottom (Fig. 17). However, the longer the model becomes the greater the distance these constraints must be propagated, and this local feature will have an effect on the block topology in a remote region. However, propagating singularity lines (or their equivalent boundary constraints) might generate new problems in other areas of the model, as was illustrated in [20].

What is desired is a more localised solution that does not allow singularity lines to propagate away from where they are needed. In [20], the solution to this problem was to snap singularity lines to the boundary in order to remove the problematic

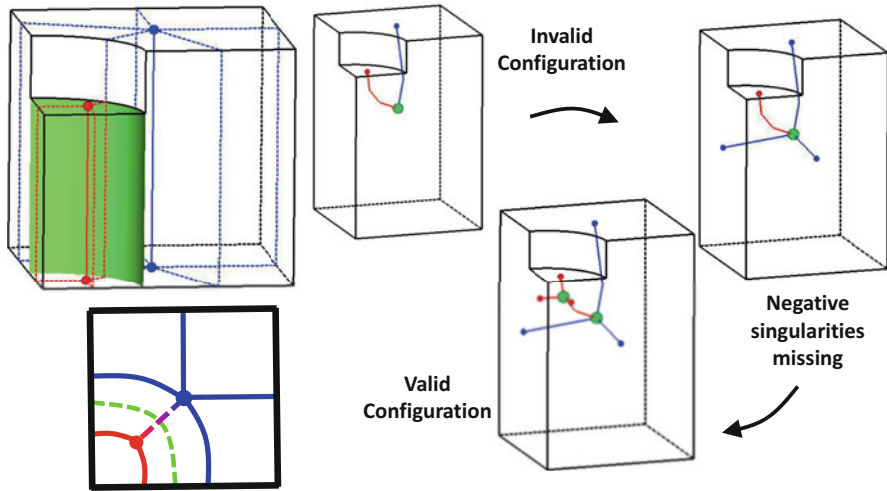


Fig. 17 Possible solutions to the notch problem: propagating singularity lines (or the concavity) to the bottom boundary face (left) or adding additional local singularity lines (right)

connection. By doing so, singularity lines were removed from the interior of the domain and were pushed to the boundary, thus reducing the element quality there. Furthermore, this technique does not work in cases where such singularity line patterns occur far from the boundary (like those in the rocker arm model shown in Fig. 10 of [18]).

Here, a different solution to this problem is proposed. It does not rely on pushing the problematic singularity lines to the boundary, but rather inserting new ones in order to comply with the singularity line connectivity expressed by the hex mesh primitives and more precisely the (1, 3, 3) fundamental primitive. This will ensure a valid block topology with high mesh quality on the boundary, whilst constraining the singularities to have only local effect.

By using this primitive, two extra positive singularity lines are introduced in the domain interior (Fig. 17 top right). Based on the proximity information of the medial object, they are connected to the 6-sided boundary faces. However, these boundary faces must have a net sum of singularities equal to zero and, as was shown in Fig. 8, if one positive singularity on the surface is introduced one more negative singularity is required to keep the sum equal to zero. By introducing two more negative singularity lines that end up on the same two boundary faces, a singularity line network which respects both the hex mesh primitives and the boundary constraints is created (Fig. 17 bottom right).

The hexahedral mesh that corresponds to this singularity line network is shown in Fig. 18. No singularity lines have been propagated to the bottom boundary face. On the other hand, extra singularity lines have been introduced to the closer side boundary faces. As a result, this solution can be considered to have a local effect on the domain.

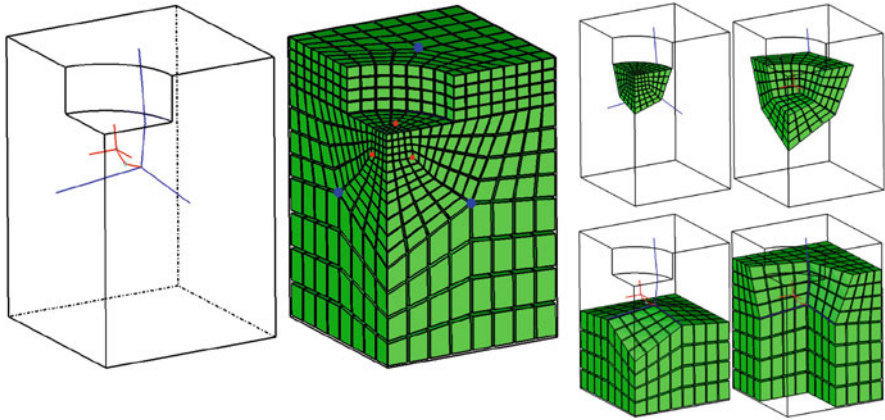


Fig. 18 Singularity line network and hexahedral mesh for the notch model

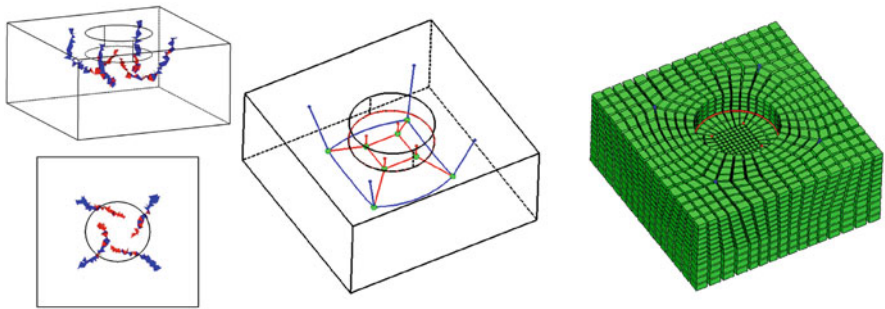


Fig. 19 Problematic frame-field, singularity line network and hexahedral mesh for the complete model

Note that, if the model was sufficiently short in the vertical direction, there would be a three-sided medial face between the notch face and the bottom of the object and a five-sided medial face between the top and bottom of the model. This implies a negative singularity running from the notch face to the bottom and a positive singularity running from the top face to the bottom, i.e. the solution of Fig. 17 (left).

By connecting four notch models together side by side, the model shown in Fig. 19 is obtained. By following the same procedure as before a correct singularity line network is obtained. In this case however, no singularity line is connected to the side boundary faces of the domain. The extra positive and negative singularity lines form a loop in the interior of the domain following the structure of the medial object and more precisely that of the medial edges associated with the concavity. This singularity line network and the corresponding hexahedral mesh are shown in Fig. 19 (middle and right).

It can also be seen that the two singularity line networks presented for the notch and the complete model are similar to those shown in Fig. 15 (or those of the

cube shown in Figs. 14 and 12). Here, however, extra negative singularity lines are needed below the concavity which, in the case where the sphere intersects with the boundary, lie inside the sphere. Note that the mesh on the bottom of this object is regular with no singularities, which would be desirable for meshing a small protuberance in a global regular mesh.

5.2 Pipe-Cylinder with No Concavities

In Fig. 20 (top left) the singularities on the boundary faces are also shown together with the block decomposition they induce. Two negative singularities are required for the semi-circular front face, two positive for the front and a pair of positive and negative for the back boundary faces. This is a half version of the model that was discussed in [24]. There, it was shown that generating a singularity line network which satisfies the boundary constraints and has a valid internal structure is not possible with current frame-field methods. More precisely, singularity lines tend to diverge from the directions of the frame field and negative singularity lines connect to positive ones.

In Fig. 21, all singular tetrahedral faces of the corresponding frame-field are highlighted. A detail of the frame field around the hole and a view from the top are also given on the right. The two negative singularity lines from the front face are clearly formed and propagated through the solid cylindrical section. One positive singularity line is formed above the bend and one more around the hole. One more negative singularity line is formed behind the hole. Although singularity lines satisfy the constraints, the internal structure of the network is not correct. The negative singularity lines connect with the positive ones at areas where singularity lines are

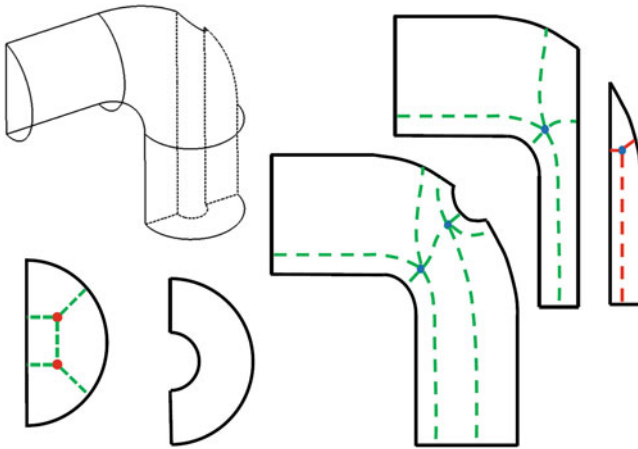


Fig. 20 Model and corresponding internal and external singularity constraints

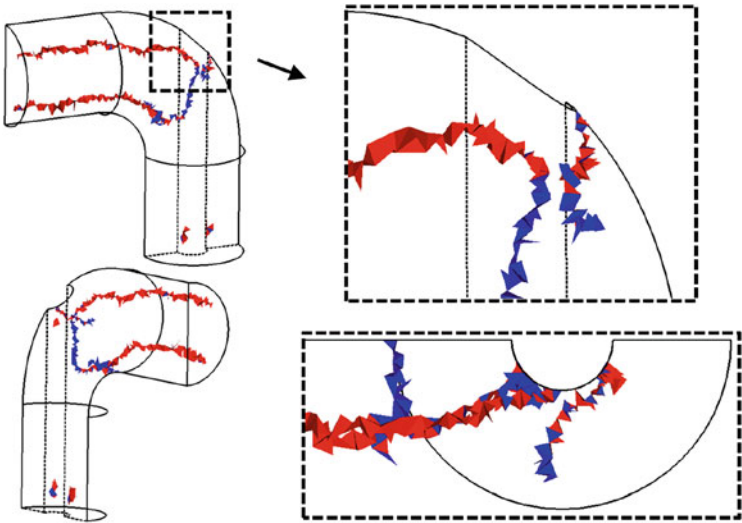


Fig. 21 Singularity line network from frame-field analysis of the half model

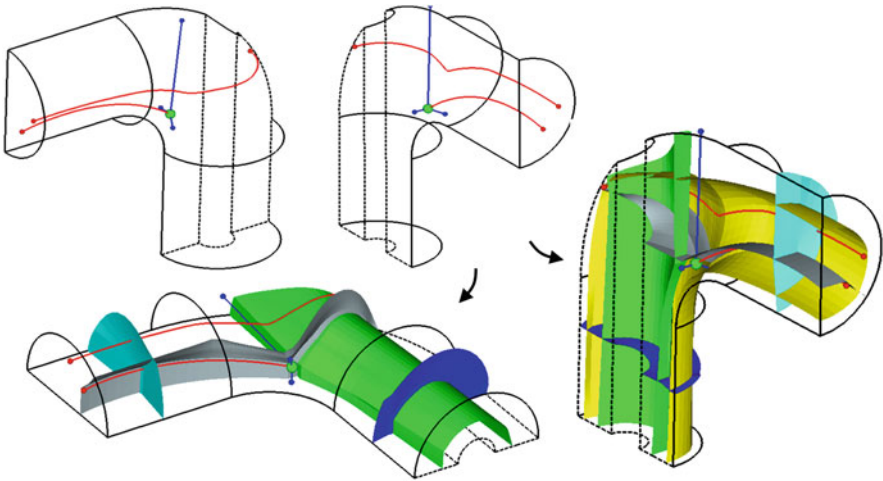


Fig. 22 Correct singularity line network and various partition surfaces

not aligned with the frame field [24]. Although the frame-field is smooth and the boundary constraints are satisfied, as singularity lines propagate in the interior they join in incompatible configurations.

By correctly placing a (1, 3, 3) primitive in areas where positive and negative singularity lines connect to each other, a valid singularity line network is generated. Such a singularity line network is shown together with various partition surfaces in Fig. 22. Here, one negative and three positive singularity lines connect in a

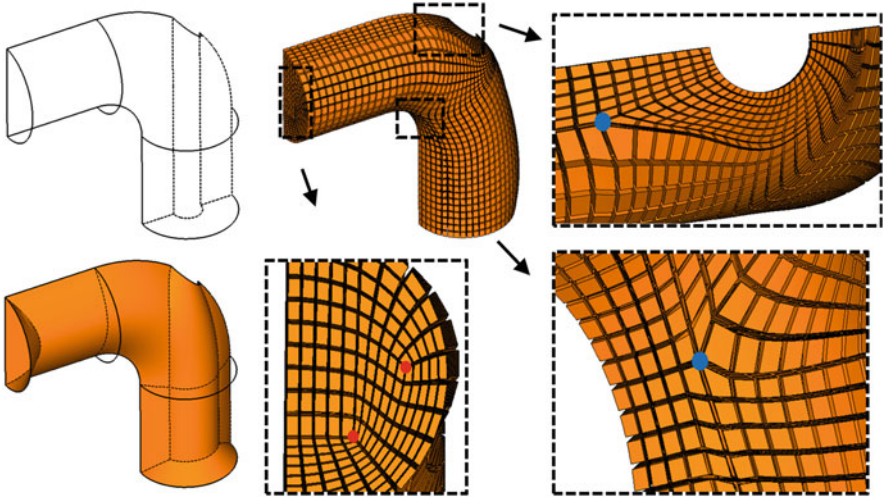


Fig. 23 Half model (top left). Corresponding medial object (bottom left). Hexahedral mesh (right)

(1, 3, 3) primitive. One more negative singularity line travels from the front to the back face curving around one of the positive singularity line and the hole of the object. The (1, 3, 3) primitive allows a valid block structure to be generated and restricts singularity lines from propagating to regions where they are not required. Furthermore, no invalid configurations such as the sudden change of a negative into a positive singularity line occur. The corresponding hexahedral mesh is shown in Fig. 23.

5.3 *Hole Close to a Convex Edge*

Another model that indicates the importance of the (1, 3, 3) primitive and the proximity information of the medial object in constructing a singularity line network is shown in Fig. 24. A valid singularity line network can be constructed following the analysis of the medial object described in [13], or any frame field method. This is illustrated in Fig. 24 (middle) together with the partition surfaces (right) that divide the domain into blocks. The three- and five-sided medial surfaces imply negative and positive singularities running through the thickness of the model. The hole through the model is surrounded by four positive singularity lines.

As the model becomes longer, the hole becomes more localised and its proximity to the side faces is lost. This change is captured by the medial object, which, far from the boundary, is similar to the medial object of a long square prism with no hole (Fig. 25 left and middle). The medial surface highlighted in Fig. 25 (right) has a gap through which the cylindrical hole passes. The medial edge that bounds this gap also bounds the two medial surfaces shown in Fig. 27 (top left). These medial

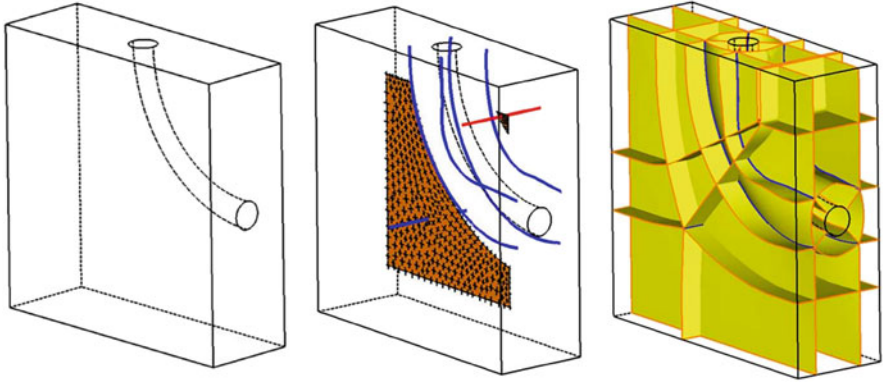


Fig. 24 Cylindrical hole turning 90° together with the singularity line network and the partition surfaces

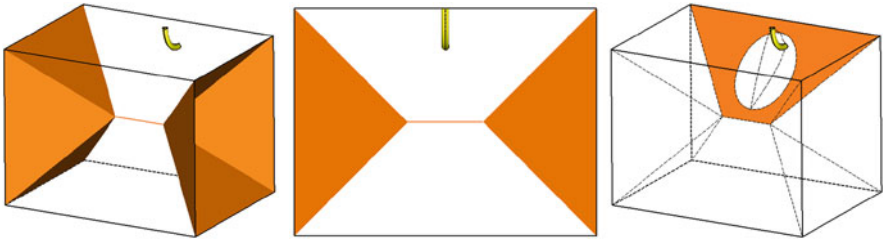


Fig. 25 Medial object for a longer version of the model in Fig. 24 (left)

entities identify the region close to the cylindrical hole. A local singularity line network should be contained in this region to prevent the mesh disruption from propagating far from the hole.

By using two $(1, 3, 3)$ primitives on the sides of the hole, such a local singularity line network can be constructed around the hole which results in valid block decomposition. Again, the proximity information and the structure of the medial object provide sufficient information to construct the singularity line network. More precisely, the pair of positive and negative singularity lines that, in Fig. 24, were connected to the front and back faces of the model are now connected with two of the positive singularity lines around the hole to form two $(1, 3, 3)$ primitives. As a result, a positive and a negative singularity line connect and create a loop around the hole, similar to the medial edge. Two more positive singularity lines pass inside this loop parallel to the cylindrical hole. These can all be seen in Fig. 27 (bottom left). This singularity line network remains isolated in the area suggested by the highlighted medial surfaces in Fig. 27 (top left). The singularity line network is similar to that in Fig. 14 bottom. The difference is that, because of the hole, two positive and not two negative singularity lines pass below the negative singularity line that connects the two $(1, 3, 3)$ primitives.

An alternative explanation is that the medial face with a central hole in Fig. 25 requires 4 positive singularities around it. However, this medial face is a “flap”, which terminates in convex edge, so the mesh cannot flow from one defining face of the flap to the other—it needs a negative singularity near the convex edge to re-orient the mesh flow, Fig. 26. This negative singularity joins to the two (1, 3, 3) primitives, each of which also contributes one positive singularity around the holes on the boundary face and the medial face. The remaining two positive singularities surrounding the hole just run directly from the top to the side face.

The complete singularity line network and the hexahedral mesh can be seen in Fig. 27. Four out of the six outer boundary faces are not affected by the singularity lines and have a regular mesh structure. Therefore, for this configuration, the presence of the small feature (which is representative of a configuration that

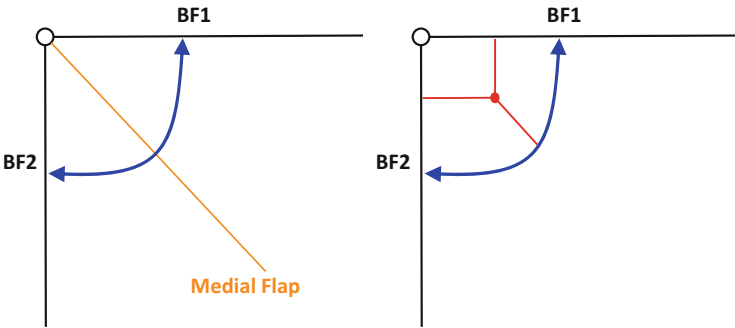


Fig. 26 Negative singularity needed near edge of flap medial face with interior singularities

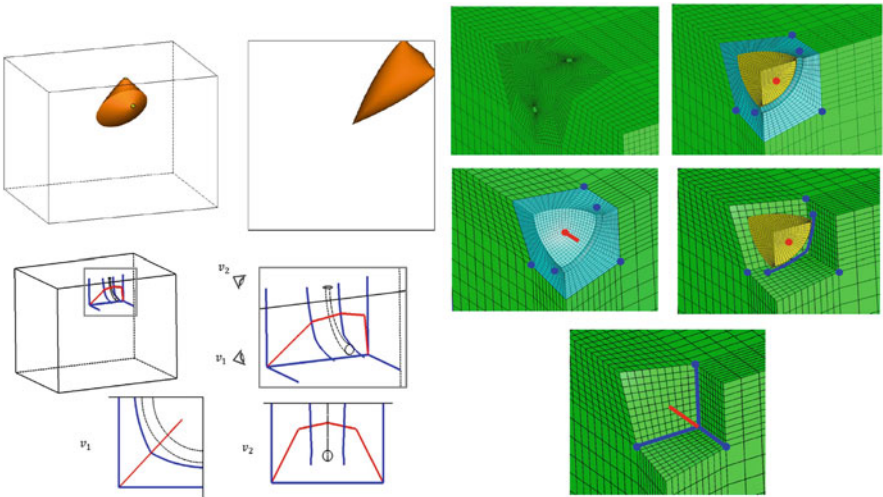


Fig. 27 Long model (top left). Local singularity line network (bottom left). Decomposition (top right). Hexahedral mesh (bottom right)

occurs around cooling holes in gas turbine components) does not affect the global mesh. By considering only the symmetric half, five positive and one negative singularities emerge on the boundary. The singularity line network can be realised by removing portions of the mesh. The (1, 3, 3) primitive that localises the effect of the singularity lines is depicted in the bottom right where the mesh close to the hole has been removed.

6 Conclusions

To generate a singularity line network that can support the block decomposition of a 3D domain, problematic configurations where positive and negative singularity lines in frame fields connect in an invalid way have to be corrected. Currently, this can be done either manually or based on the methods discussed in [20]. These solutions are often problematic and, thus, a more comprehensive exploration of the possible configurations of singularity line networks is needed.

Here, based on the connectivity rules suggested by the hexahedral primitives a new solution path is suggested. Whenever a positive singularity line attempts to connect to a negative one, two more positive singularity lines can be introduced in the network, in effect placing a (1, 3, 3) primitive at that location. Based on the proximity information of the medial object, two extra positive singularity lines can be projected to the boundary of the domain, or the extra singularity lines can form loops following the structure of local medial edges. Extra negative singularity lines are also introduced and projected to the same boundary faces (as is required to maintain the desired net sum of singularities on the boundary) or form loops parallel to the positive ones. These result in more localised singularity line networks. Furthermore, a regular grid structure is maintained far from the singularity lines.

The above approach has been used to generate singularity line networks, block decompositions and hexahedral meshes of the examples shown as an extension to the method described in [13]. A more thorough investigation of the properties of the medial object and its connection to the singularity line network of a block decomposition is still required to derive specific steps for correcting invalid singularity line networks or constructing valid ones immediately. Although the suggested steps worked on the models presented here, no mathematical proof is provided that they will work for all other possible scenarios.

However, based on the current results it looks promising that the known constraints on the connectivity of singularity lines, plus the information on the proximity, geometry and topology of the domain provided by the medial object, can help generate singularity line networks with correct topology, and support a block decomposition of general domains for all hex meshing.

Acknowledgments The authors wish to acknowledge the financial support provided by Innovate UK via the GEMinIDS (project 113088), a UK Centre for Aerodynamics project and Rolls Royce, who supported the PhD studentship of Dimitrios Papadimitrakis and granted permission to publish this work.

References

1. Lim, C.W., Yin, X., Zhang, T., Su, Y., Goh, C.K., Moreno, A., Shahpar, S.: Automatic blocking of shapes using evolutionary algorithm. In: Proceedings of the 27th international meshing roundtable (2019). https://doi.org/10.1007/978-3-030-13992-6_10
2. Huang, J., Tong, Y., Wei, H., Bao, H.: Boundary aligned smooth 3D cross-frame field. *ACM Trans. Graph.* **30**(6), 1–8 (2011) <https://doi.org/10.1145/2070781.2024177>
3. Huang, J., Jiang, T., Wang, Y., Tong, Y., Bao, H.: Automatic frame field guided hexahedral mesh generation (2012). <http://www.cad.zju.edu.cn/home/hj/12/hex/techreport/hex-techreport.pdf>. Cited 25 Nov 2020
4. Li, Y., Liu, Y., Xu, W., Wang, W., Guo, B.: All-hex meshing using singularity-restricted field. *ACM Trans. Graph.* **31**(6), 1–11 (2012). <https://doi.org/10.1145/2366145.2366196>
5. Ray, N., Sokolov, D., Lévy, B.: Practical 3D frame field generation. *ACM Trans. Graph.* **35**(6), 1–19 (2016). <https://doi.org/10.1145/2980179.2982408>
6. Solomon, J., Vaxman, A., Bommes, D.: Boundary element octahedral fields in volumes. *ACM Trans. Graph.* **36**(4), 1 (2017). <https://doi.org/10.1145/3072959.3065254>
7. Palmer, D., Bommes, D., Solomon, J.: Algebraic representations for volumetric frame fields. *ACM Trans. Graph.* **39**(2), 1–17 (2020). <https://doi.org/10.1145/3366786>
8. Kowalski, N., Ledoux, F., Frey, P.: Smoothness driven frame field generation for hexahedral meshing. *CAD Comput. Aided Des.* **72** 65–77 (2016). <https://doi.org/10.1016/j.cad.2015.06.009>
9. Nieser, M., Reitebuch, U., Polthier, K.: CUBECOVER—Parameterization of 3D volumes. *Comput. Graphics Forum* **30**(5), 1397–1406 (2014). <https://doi.org/10.1111/j.1467-8659.2011.02014.x>
10. Zheng, Z., Wang, R., Gao, S., Liao, Y., Ding, M.: Automatic block decomposition based on dual surfaces. *CAD Comput. Aided Des.* **127**, 102883 (2020) <https://doi.org/10.1016/j.cad.2020.102883>
11. Calderan, S., Hutzler, G., Ledoux, F.: Dual-based user-guided hexahedral block generation using frame fields. In: 28th International Meshing Roundtable (2020). <https://doi.org/10.5281/ZENODO.3653430>
12. Papadimitrakis, D., Armstrong, C.G., Trevor, T.R., Le Moigne, A., Shahpar, S.: (2018) A Combined Medial Object and Frame Approach to Compute Mesh Singularity Lines. 27th International Meshing Roundtable. <https://project.inria.fr/imr27/files/2018/09/2000.pdf>
13. Papadimitrakis, D., Armstrong, C.G., Trevor, T.R., Le Moigne, A., Shahpar, S.: Building direction fields on the medial object to generate 3D domain decompositions for hexahedral meshing. In: 28th International Meshing Roundtable (2020). <https://doi.org/10.5281/ZENODO.3653428>
14. Bernard, P.E., Remacle, J.F., Kowalski, N., Geuzaine, C.: Hex-dominant meshing approach based on frame field smoothness. In: 23rd International Meshing Roundtable (2014). <https://doi.org/10.1016/j.proeng.2014.10.382>
15. Baudouin, T.C., Remacle, J.F., Marchandise, E., Henrotte, F., Geuzaine, C.: A frontal approach to hex-dominant mesh generation. *Adv. Model. Simul. Eng. Sci.* **1**(1), 1–30 (2014). <https://doi.org/10.1186/2213-7467-1-8>
16. Sokolov, D., Ray, N., Untereiner, L., Levy, B.: Hexahedral-Dominant Meshing. *ACM Trans. Graph.* **35**(5), 1–23 (2016). <https://doi.org/10.1145/2930662>
17. Gao, X., Jakob, W., Tarini, M., Panozzo, D.: Robust hex-dominant mesh generation using field-guided polyhedral agglomeration. *ACM Trans. Graph.* **36**(4), 1–13 (2017). <https://doi.org/10.1145/3072959.3073676>
18. Liu, Z., Zhang, P., Chien, E., Solomon, J., Bommes, D.: Singularity-constrained octahedral fields for hexahedral meshing. *ACM Trans. Graph.* **37**(4), 93–101 (2018). <https://doi.org/10.1145/3197517.3201344>
19. Corman, E., Crane, K.: Symmetric Moving Frames. *ACM Trans. Graph.* **38**(4), 1–16 (2019). <https://doi.org/10.1145/3306346.3323029>

20. Reberol, M., Chemin, A., Remacle, J.F.: Multiple approaches to frame field correction for CAD models. In: 28th International Meshing Roundtable (2020). <https://doi.org/10.5281/ZENODO.3653414>
21. Price, M.A., Armstrong, C.G., Sabin, M.A.: Hexahedral mesh generation by medial surface subdivision: Part I. Solids with convex edges. *Int. J. Numer. Methods Eng.* **38**(19), 3335–3359 (1995). <https://doi.org/10.1002/nme.1620381910>
22. Sherbrooke, E.C., Patrikalakis, N.M., Wolter, F.E.: Differential and topological properties of medial axis transforms. *Graph. Models Image Process.* **58**(6), 574–592 (1996). <https://doi.org/10.1006/gmip.1996.0047>
23. Fogg, H.J., Sun, L., Makem, J.E., Armstrong, C.G., Robinson, T.T.: Singularities in structured meshes and cross-fields. *CAD Comput. Aided Des.* **105**, 11–25 (2018). <https://doi.org/10.1016/j.cad.2018.06.002>
24. Viertel, R., Staten, M.L., Ledoux, F.: Analysis of non-meshable automatically generated frame fields. 25th International Meshing Roundtable (2016). <https://www.osti.gov/servlets/purl/1375569>
25. Price, M.A., Armstrong, C.G.: Hexahedral mesh generation by medial surface subdivision: Part II. Solids with flat and concave edges. *Int. J. Numer. Methods Eng.* **40**(1), 111–136 (1997). [https://doi.org/10.1002/\(SICI\)1097-0207\(19970115\)40:1<111::AID-NME56>3.0.CO;2-K](https://doi.org/10.1002/(SICI)1097-0207(19970115)40:1<111::AID-NME56>3.0.CO;2-K)
26. Li, T.S., McKeag, R.M., Armstrong, C.G.: Hexahedral meshing using midpoint subdivision and integer programming. *Comput. Methods Appl. Mech. Eng.* **124**(1–2), 171–193 (1995) [https://doi.org/10.1016/0045-7825\(94\)00758-F](https://doi.org/10.1016/0045-7825(94)00758-F)
27. Armstrong, C.G., Li, T.S., Tierney, C., Robinson, T.T.: Multiblock mesh refinement by adding mesh singularities. In: 27th International Meshing Roundtable, Lecture Notes in Computational Science and Engineering, vol. 127 (2019). <https://doi.org/10.1007/978-3-030-13992-6>

Intercode Hexahedral Meshing from Eulerian to Lagrangian Simulations



Nicolas Le Goff, Franck Ledoux, and Jean-Christophe Janodet

Abstract In this chapter, we deal with the problem of mesh conversion for coupling lagrangian and eulerian simulation codes. More specifically, we focus on hexahedral meshes, which are known as pretty difficult to generate and handle. Starting from an eulerian hexahedral mesh, i.e. a hexahedral mesh where each cell contains several materials, we provide a full-automatic process that generates a lagrangian hexahedral mesh, i.e. a hexahedral mesh where each cell contains a single material. This process is simulation-driven in the meaning that the we guarantee that the generated mesh can be used by a simulation code (minimal quality for individual cells), and we try and preserve the volume and location of each material as best as possible. In other words, the obtained lagrangian mesh fits the input eulerian mesh with high-fidelity. To do it, we interleave several advanced meshing treatments—mesh smoothing, mesh refinement, sheet insertion, discrete material reconstruction, discrepancy computation, in a fully integrated pipeline. Our solution is evaluated on 2D and 3D examples representative of CFD simulation (Computational Fluid Dynamics).

1 Introduction

Many numerical simulation codes require to discretize a study domain Ω by a *mesh* that partitions Ω into a set of basic connected elements, called *cells*, that will

N. Le Goff · F. Ledoux (✉)

Laboratoire en Informatique Haute Performance pour le Calcul et la simulation,
Université Paris-Saclay, CEA, Bruyères-le-châtel, France

CEA, DAM, DIF, Arpajon, France

e-mail: nicolas.le-goff@cea.fr; franck.ledoux@cea.fr

J.-C. Janodet

IBISC, University d'Evry, Evry, France

Université Paris-Saclay, Gif-sur-Yvette, France

e-mail: jeanchristophe.janodet@univ-evry.fr

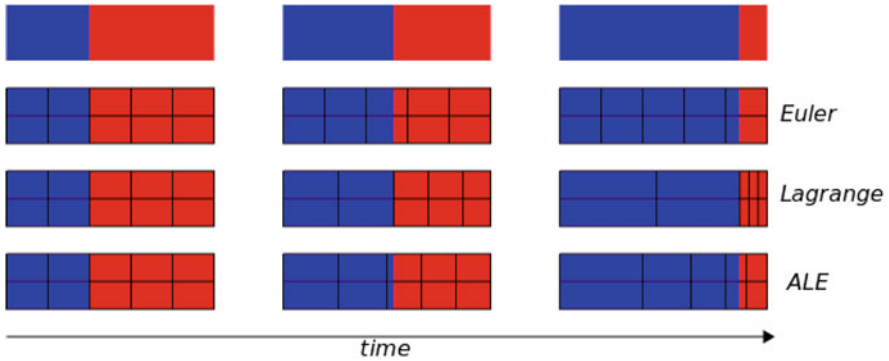


Fig. 1 A 2D domain made of two materials (top) discretized in eulerian, lagrangian and ALE ways

carry physical data (pressure, temperature, material description, etc.). Cells define a support to apply traditional numerical methods, like Finite Element Methods (FEM) or Finite Volumes Methods (FVM), which rely on basic functions that are defined onto finite elements or finite volumes. Depending on the numerical methods, many types of meshes can be used and they can differ in the way they match simulation materials. Let us consider a study domain made of two materials *A* and *B* that are in movement, and depicted in various ways on Fig. 1. The first line shows the “physical” evolution where *A* and *B* are respectively colored in blue and red. Materials are moving accordingly to the effect of a simulated physical phenomenon, leading to an expansion of *A* that tries to fill the whole domain, while *B* is contracted. Meshes are shown on the three remaining lines. At the initial time, the meshes are identical: a *pure* quadrilateral mesh, that is a mesh where each 2D cell is a quad and contains a single material (*A* or *B*). Three approaches are then possible:

- **Euler** On the second line, the mesh remains fixed while the materials move through cell boundaries; when several materials are present inside one cell, the cell is called *mixed* and we denote such a mesh as being *eulerian*. In this case the interface between *A* and *B* is lost.
- **Lagrange** On the third line, the mesh moves at the same speed as materials do. Cells remain pure during the whole simulation but their geometry can drastically change. We note such a mesh as *lagrangian*, and the interface between *A* and *B* is totally defined by a set of mesh nodes and edges.
- **ALE** Eventually, on the fourth line, the mesh moves but not at the same speed as materials do. Cells can become mixed but their geometry remains controlled by the simulation code. We qualify such a mesh as being *ALE*, for *Arbitrary Lagrangian-Eulerian*.

Conversion of data between simulation codes are usual in real-case studies where different codes are used to solve complex multi-physics problems. It can be done

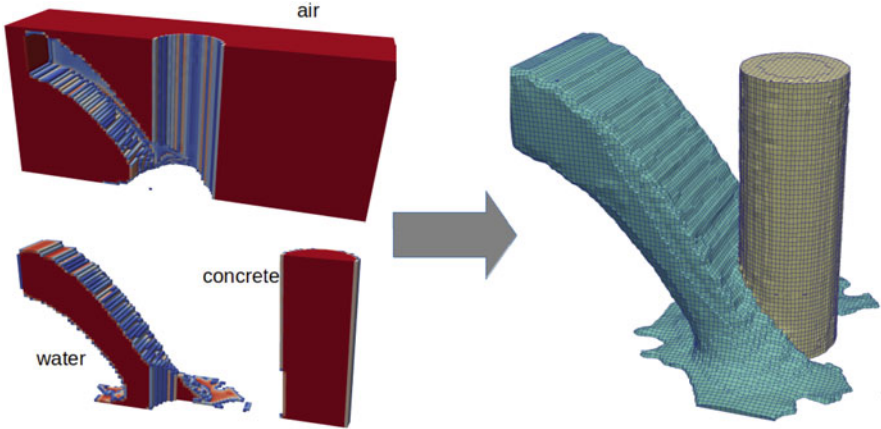


Fig. 2 A 3D Example of what we want to achieve after having run the simulation code Gridfluid [13] with water being poured against a concrete pillar. On the left, our input, a grid mesh carrying the volume fractions of respectively the water, the concrete pillar and the air. On the right the output lagrangian mesh where we only display the water and the concrete. The air is also discretized by a pure hexahedral mesh but not shown here for a sake of clarity. Obtained mesh quality is controlled for the three materials (water, concrete, air)

in many manners going from loosely-coupled codes, where codes are assembled in a pipeline and communicate by reading and writing files, to tightly-coupled codes, that are interleaved in a simulation loop and share in-memory data. We focus in this work on loosely-coupled codes. As each code has its own input and output requirements, an intercode tool is required to convert the output of one code into the input of the next one. In the case of converting the output of an eulerian code into the input of a lagrangian code, this task is not trivial at all, especially when we aim to generate full hexahedral lagrangian meshes (see Fig. 2). Eulerian meshes are in most cases easy to generate. They are typically a grid, where mixed cells are specified by the *volume fractions* of the materials they contain. Some of those eulerian codes have Adaptive Mesh Refinement (AMR) capabilities, meaning that, usually by use of an octree-like data structure, the mesh is locally refined or coarsened to respectively track a phenomenon of interest or to reduce the execution time and the memory footprint of the simulation.¹ On the opposite, in the case of lagrangian codes, quadrilateral and hexahedral meshes are quite more challenging to generate especially in 3D.

This issue is not directly handled by many research works that try either to create volume-preserving material interfaces without satisfying some meshing constraints, or to generate a full hexahedral mesh without preserving material volumes. First are the interface reconstruction methods [19]. These methods take an eulerian mesh as an input and reconstruct the interfaces between materials inside each cell; they are

¹ We do not consider AMR meshes in this work.

classically used in ALE simulation codes and in associated scientific visualization software [1, 8]. While the volume fractions preservation is actually enforced by design, a limitation of those methods is that the obtained interfaces do not fit our purpose, as they are jagged, not continuous and potentially with small slivers of materials. It seems impossible to use such interfaces for projecting mesh nodes associated to those surfaces without any significant modification, which would in turn render the volume preservation property null and void. Secondly figure among the variety of hexahedral meshing techniques the overlay-grid methods [30], or octree-based isocontouring methods [34], where a shape—an explicit geometrical CAD model for instance—that needs to be meshed is embedded into a mesh that discretizes its bounding box. Said mesh will usually be a grid, easy to generate and possibly locally refined [31]; its cells are assigned to the components of the geometrical model and those outside discarded, and the mesh is then deformed or a padding layer of hexahedral cells is inserted in order to capture the geometrical features of the model. Based on this pioneer idea of embedding a shape into an existing mesh, several works were proposed during the two past decades to improve the process. For instance, [34, 37] consider single-material domains and use local refinement patterns to adapt the mesh around the material interface; sharp features are tried to be preserved in [28]; multi-material are considered with hybrid meshes mixing tetrahedral and hexahedral elements in [36]; mesh quality is improved in [29]. We can also cite geometric flow-based methods that preserve volume for each material region and that have been approved mathematically and employed in mesh quality improvement [22, 35]. Contrary to the interfaces reconstruction methods, extracting a geometrical model from that mesh will give a relatively smooth model with a clean topology, but as a drawback one does not guarantee to preserve the volume of materials. There are additional incompatibilities with our aim: first the expected inputs of those methods are explicitly-defined models, likd CAD ones, not meshes carrying volume fractions; secondly, most of those methods are designed to mesh only one component and cannot be used to mesh a CAD model that is the assembly of several pieces, while we have several materials. Thirdly, those methods heavily rely on the ability to generate an adequate initial mesh, possibly with local refinements to offer more robustness, to better capture the CAD or to provide a modicum of volume preservation [11]. This is again in direct conflict with what we need, as in our case the input mesh is fixed as part of our input.

In order to get a valid mesh for numerical simulation while preserving materials, we propose to adopt an overlay-grid approach, that meets our concerns, and more specifically, we extend the SCULPT algorithm [25, 26], which implements an overlay-grid approach considering volume fractions data as an input. Starting from a 3D eulerian mesh M_E with a set of materials $\mathcal{M}$, we aim to generate both an interface geometrical model $\mathcal{G}$, and a full hexahedral mesh M_L such that:

1. $\mathcal{G}$ is usable for mesh generation, that is its surfaces are smooth and “cleanly”² connected along curves and vertices;
2. Cells of M_L fit minimum quality requirements to be used by a FEM or FVM simulation codes;
3. Every material $m \in \mathcal{M}$ is preserved as best as possible, in the meaning that the overall volume of m is similar in M_E and M_L and it is located at the same spatial location.

To do it, we designed a fully-integrated pipeline that interleaves several advanced meshing treatments—mesh smoothing, mesh refinement, sheet insertion, discrete material reconstruction, discrepancy computation. In this chapter, we focus on the generation of the geometrical model $\mathcal{G}$, which is a key component of our pipeline. It is described in Sect. 3. Our solution is evaluated on 2D and 3D examples representative of CFD simulations (Computational Fluid Dynamics) in Sect. 4. But beforehand, we give an overview of the full pipeline in Sect. 2.

2 Eulerian to Lagrangian Hexahedral Remeshing Pipeline

In order to generate a lagrangian hexahedral mesh M_L that fits the materials carried by the input eulerian mesh M_E , we follow the process depicted on Fig. 3, where three main stages are identified:

1. **Geometry Extraction** The first stage consists in extracting a valid geometrical model $\mathcal{G}$ while assigning each mixed cell of M_E to a single material. A refinement stage is interleaved into it when we encounter topologic inconsistency between $\mathcal{G}$ and the pure mesh M_L deduced from M_E . Figure 4a–d show such a refinement.
2. **Quality-driven mesh projection** Then, the mesh M_L is modified and its nodes moved in order to obtain both a minimum cell quality (which is required by simulation codes) and a mesh that fits the topology of $\mathcal{G}$. Even if what defines a “good” mesh varies between simulation codes and the cases run, some geometrical cell quality criteria are common among the simulation codes, which cannot operate when even one single cell becomes too distorted [17].
3. **Discrepancy-driven mesh deformation** Eventually, we apply a smoothing stage to measure and improve the volume preservation compared to the eulerian mesh M_E they originated from.

In this chapter, we focus on the first stage of this process. But let us give a few words abouts the second and third stages. The quality-driven mesh projection strongly relies on three key ingredients: first the existence of the geometrical model

² When three material meet, we need to get a clear curve that is adjacent to the three of them and not a series of small curves and surfaces that would be adjacent to only a pair of them.

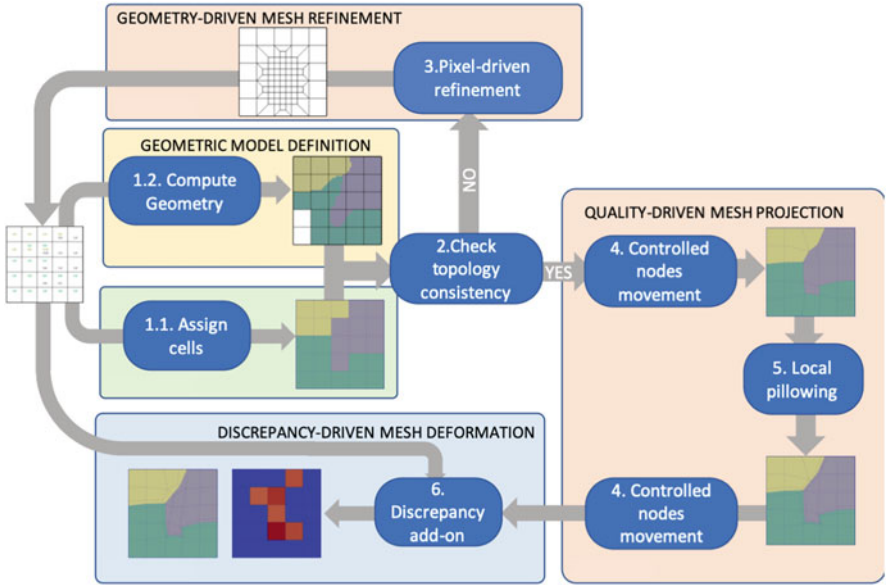


Fig. 3 Main stages of ELG, an Euler to Lagrangian hexahedral remeshing pipeline

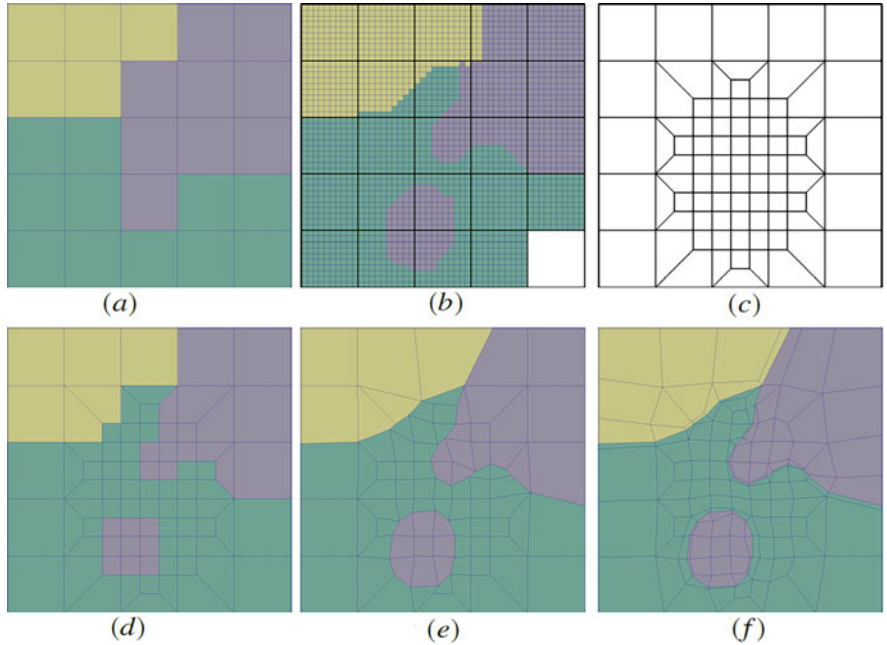


Fig. 4 Example of input mesh refinement in order to fit the built geometrical model. (a) Initial assignment. (b) Voxelated geometry. (c) Refined mesh. (d) New assignment. (e) node movement. (f) Pillowing and smoothing

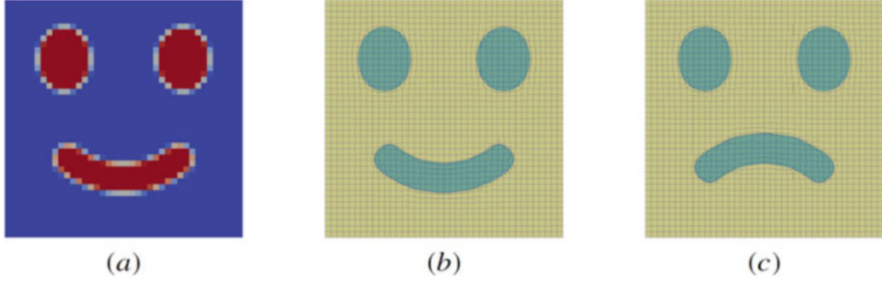


Fig. 5 Given a grid mesh with the material volume fractions represented in (a) ranging from 0 in blue to 1 in red, we can see that despite being of the same total volume for each material, the material mesh (b) fits better the volume fraction grid than (c) does

built at stage 1; second, the idea to deform the mesh to fit the geometry but only if the cell quality meets lagrangian code quality requirements, i.e. material interface nodes are moved towards the geometrical model but they reach it only if the quality of adjacent cells is good enough; third, some topological local padding operations are performed to give more degree of freedom to nodes that do not reach the expected location. A full description of this stage can be found in [21].

The discrepancy-driven mesh deformation process is a smoothing stage that aims to measure and improve the volume preservation compared to the eulerian mesh M_E they originated from. This stage is done as a post-process to ensure the overall constraint, which is to avoid to deviate too much from the input data, namely the material volume fractions. This constraint is both strong and weak: strong because as this process is used in a physical simulation pipeline, it is mandatory to preserve physical quantities as best as possible to get “high-fidelity” results; weak since this problem is over constrained and so approximations are unavoidable. Getting both high-fidelity material preservation and a smooth clean geometrical definition of the material interfaces is quite difficult in many cases. In order to control the material preservation, we use the notion of *discrepancy* as introduced in [20]. Let A and B be two meshes of a domain Ω and $\mathcal{M}$ be the set of materials that disjointly fills Ω , the global difference of material volumes between A and B is

$$\Delta V = \sum_{m \in \mathcal{M}} |\Delta V_m| = \sum_{m \in \mathcal{M}} |V_m^A - V_m^B| \quad (1)$$

where V_m^A and V_m^B are respectively the volume of a material m in meshes A and B . Minimizing ΔV only ensures a global volume preservation, which proves not to be sufficient as it can lead to unexpected results (see Fig. 5). To get a local volume control, it is mandatory to consider the notion of *local discrepancy*. Let us consider the meshes A and B again with A the input mesh and B the output mesh. Let c_j^A be a

cell of M^A and m be a material of $\mathcal{M}$, we note $d_{j,m}$ the discrepancy of c_j^A relatively to material m and mesh B and we define it as

$$d_{j,m} = d(c_j^A, m) = V(c_j^A \cap B|_m) - f_{j,m} V(c_j^A) \quad (2)$$

where $V(X)$ is the volume of any geometric space X , $B|_m$ is the output mesh restricted to the pure cells of material m and $c_j^A \cap B|_m$ is the geometrical intersection of c_j^A with the cells of $B|_m$. Let us note that in practice, we compute geometrical intersections using³ [15]. In order to compare the whole meshes A and B , we finally use the *global discrepancy* of a cell $c_j^A \in A$ defined as $d_j = d(c_j^A) = \sum_{m \in \mathcal{M}} |d_{j,m}|$, and the global discrepancy of A is defined as $d = \sum_{c_j^A \in A} d_j$. The global discrepancy gives us a way to compare material locations between two meshes both globally and locally to each cell and so each subpart of Ω . In [20], this quantity is used to move node interfaces in the lagrangian mesh B in order to improve the material preservation. We will use it in Sect. 4 to evaluate our solution.

3 Geometry Extraction

Geometry extraction and cell material assignment are performed pairwise in order to obtain a consistent correspondence between the geometrical model and the mesh. Cell assignment is performed as the SCULPT algorithm [25, 32] does. The mesh M_L is first created as a copy of M_E , then cells of M_E are assigned to materials on a majority basis: a cell is assigned to the material of highest volume fraction in the corresponding input cell in M_E (see Fig. 6). A correction step can be optionally activated to avoid some topological issues for the incoming lagrangian simulation code (as the inability to handle non-manifold interfaces for instance). When changing materials, cells are reassigned around each problematic node to the second best (in the sense that it is closest in terms of volume fractions) correct assignment and as it could lead to non-manifold configurations appearing in the neighborhood of changed nodes this phase is executed again.

Extracting the geometrical model $\mathcal{G}$ raises two main issues: the desired interfaces have to be smooth, typically when the goal is to visualize them or to use them to generate a mesh, but at the same time they must also fit the input data as best as possible, namely the volume fractions; those two objectives can conflict with one another. To address these issues we consider that several methods are relevant. A scientific field where material interface reconstruction is extensively studied and applied is Arbitrary Lagrangian-Eulerian (ALE) CFD simulations. Obtained reconstructed interfaces have a built-in volume fractions preservation but are not smooth and discontinuous across cells [19]. Most of these methods

³ The interested reader can directly use the open-source library *portage* [15] based on *R3D* [27].

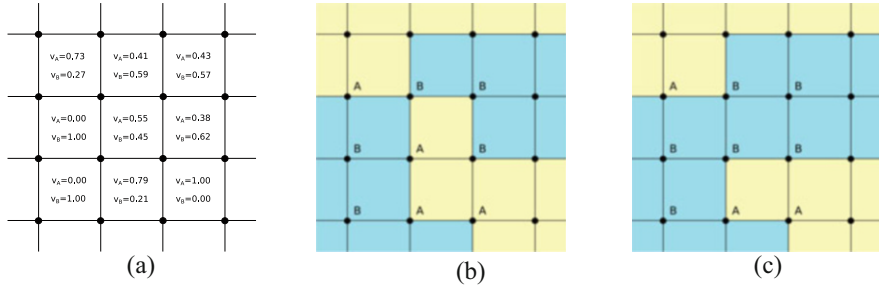


Fig. 6 Starting from the volume fractions given in (a), grid cells are assigned to each material in two stages (b) and (c). (a) volume fractions. (b) majority assignment. (c) assignment correction

also have the additional drawback of being material order-dependent. In order to visualize material interfaces for eulerian meshes, several approaches relies on a voxel decomposition of the mixed cells. Each mixed cells is split into sub-elements—typically an hexahedron will be refined into a grid of voxels—on which a partitioning strategy is applied with respect to the volume fractions inside each cell. The interfaces at the sub-elements level are aliased, and since these methods originated from visualization purposes the interfaces are usually simplified into smooth triangular surfaces. Finally, in the Sculpt algorithm [25], interface nodes’ locations are computed using the volume fractions (and their gradient) and the material assignment but it fails to capture at all the materials that have no majority volume fractions in any of the mesh cells;

Considering that the Sculpt strategy—that we have extensively evaluated—can become limited in some cases, and that getting smooth surface is of first interest for our purpose, we propose to rely on a voxel-based approach.⁴

3.1 Interface Reconstruction Via Voxel Assignment

The discrete voxel-based interface reconstruction technique stems from the need to visualize the location of materials in the case where some of the cells are mixed and where the number of materials is greater than two. In the case where the number of materials equals two, classic iso-contouring methods provide a “clean” solution but with more materials small gaps or artifacts can appear that are non-desirable. In [14], the authors introduced the decomposition of mixed cells into subcells (or voxels) which are in turn assigned to the materials present in the mixed cells they

⁴ We can note that as we handle both 2D and 3D cases in structured and unstructured cases, throughout this chapter we will use the term “voxel” as a misnomer in place of pixel (in 2D), sub-cell or sub-element.

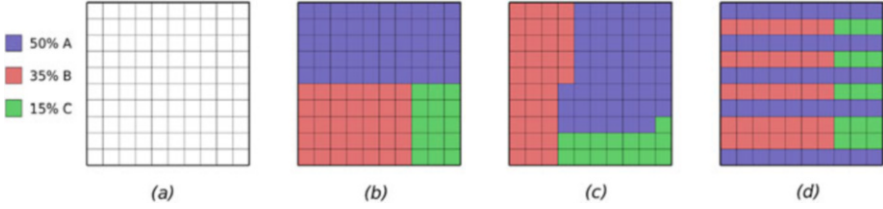


Fig. 7 Example of the voxel-assignment problem and some unexpected valid results

were spawned from; the work in [2, 3] extends it to cases with more than three materials per cell.

The voxel-assignment problem can be stated as follows. Considering a coarse mixed cell c containing materials $m \in \mathcal{M}$, with volume fractions denoted $f_{c,m}$ such that $\sum_{m \in \mathcal{M}} f_{c,m} = 1$, and c discretized as a set $\mathcal{V}_c$ of n_v voxels, assign one single material m to each voxel of $\mathcal{V}_c$ while ensuring material volume preservation. Figure 7 illustrates such a situation where in (a), a coarse mixed cell, made of 50% of material A, 35% of material B and 15% of material C, is split into 100 voxels. Results given in (b), (c) and (d) are valid solutions for the voxel-assignment problem, as they all respect the volume fractions, but they are wildly different from one to the another; it probably means that our problem could receive some additional description in order to be appropriately formalized. The voxel-assignment should aim towards several objectives:

- First, to enforce the volume preservation of each material, we favor solutions having a low *discrepancy*, defined as the sum over each coarse cell c of the absolute difference between the volume of each material present in c (for material m it is $f_{c,m}V(c)$) and the sum of the volumes of the voxels of c assigned to m . It expresses whether the voxels material assignment fits the volume fractions;
- Secondly, as usual in partitioning algorithms, we favor connected components for each material. It translates into minimizing the *edgcut* function, defined as the sum of the number of pairs of adjacent voxels assigned to different materials;
- Thirdly, surrounding pure cells of c provides the initialization to our problem. If a mixed cell is bounded by pure cells, then we extend the voxelization process to the vicinity of c , i.e. all the mixed cells and their adjacent pure cells are subdivided into voxels, and voxels spawned from pure cells are already assigned to the material of their corresponding pure coarse cell, leaving those spawned from mixed coarse cells as being “free” (see Fig. 8).

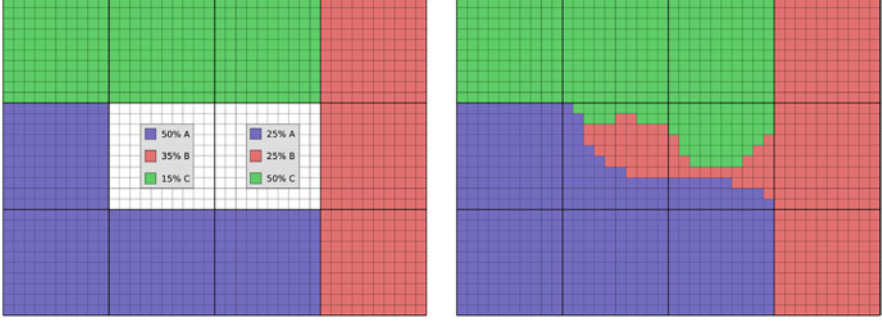


Fig. 8 Two mixed cells surrounded by pure cells where voxels must be partitioned into materials (left). Considering adjacent pure cells and the objective of minimizing the number of connex components for each material could lead to the result shown on the right

Such a problem can be formally described by the following mixed-integer linear program (MILP):

$$\left\{ \begin{array}{l} \min \sum_{v \in \mathcal{V}, m \in \mathcal{M}} |ma_{v,m} - \frac{1}{|N(v)|} \sum_{w \in N(v)} ma_{w,m}| \\ \text{constrained to} \\ \left. \begin{array}{l} ma_{v,m} \in \{0, 1\} \\ \sum_{m \in \mathcal{M}} ma_{v,m} = 1 \\ \sum_{v \in \mathcal{V}_c} ma_{v,m} = nbsub * f_{c,m} \end{array} \right\} \begin{array}{l} \forall v, m \\ \forall v \\ \forall m, \forall c \end{array} \end{array} \right.$$

where $\mathcal{V}$ is the total set of voxels of the whole domain, $ma_{v,m} = 1$ if voxel v is assigned to material m and $ma_{v,m} = 0$ otherwise; $N(v)$ is the set of voxels adjacent to v . The first two constraints indicate that every voxel has one assignment and only one. The third constraint expresses that we want to have a *discrepancy* equal to zero ($nbsub$ being the number of voxels in a coarse cell c). The objective function that we want to minimize reflects the aim for voxels assigned to the same material to be clustered together, i.e. having a low *edgcut*. Since our variables are integers, we in fact have a mixed-integer linear problem. This type of problem can be solved using various solving libraries [5, 9, 12, 23]. But it is in practice too computationally expensive to be used in our pipeline. That is why we propose a greedy heuristic that is designed to fit our specific requirements.

3.1.1 A Greedy Heuristic to Assign Voxels

We follow the Algorithm 1 to assign voxels. It iteratively assigns a material value to free voxels that are surrounded by enough already material-assigned voxels (see the evolution at several iterations in Fig. 9). Some 3D results are shown on Fig. 12. The

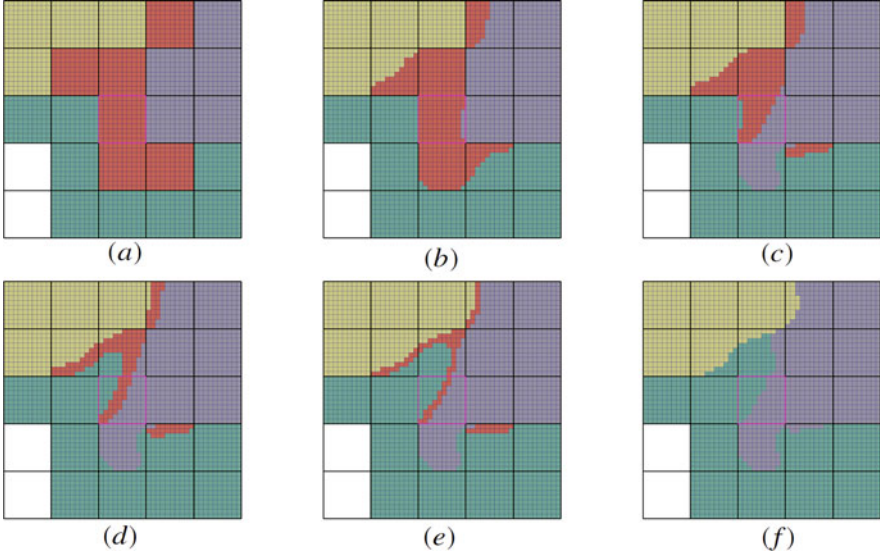


Fig. 9 A 5×5 example where the evolution of the volume fractions (see Algorithm 1:13) assigned to the free voxels of the central coarse cell are given below each figure. The wireframe black grid is the coarse mesh and the voxels colored in orange are those not yet assigned. (a) $vf=0.40$, $vf=0.60$. (b) $vf=0.43$, $vf=0.57$. (c) $vf=0.62$, $vf=0.38$. (d) $vf=0.58$, $vf=0.43$. (e) $vf=0.53$, $vf=0.47$

underlying idea of this algorithm is to assign a material to each voxel following an advancing-front strategy. We consider a set S of connected mixed cells as a starting point (orange cells on Fig. 9a). Each cell $c \in S$ is split into voxels that we have to assign to a specific material. The material each voxel will be assigned to depends on the volume fractions of materials that compose its parent cell in S . For example, on Fig. 9, volume fractions of the central cell are given; the central cell should be filled by 40% of green and 60% of grey voxels at the end.

In order to assign a material to a voxel v , we consider the materials that are already assigned in its vicinity (the 8 surrounding pixels in 2D when the case is structured) and we diffuse those materials into the voxel v . Voxel v is assigned to a material m if its newly computed volume fraction is higher than a minimum threshold. The threshold value is iteratively decreased in order to avoid blocking situations where the algorithm is unable to assign a material to any voxel during an iteration. At the end of each iteration, the volume fractions to reach for each material in a cell are updated (see Fig. 9a–e). With this strategy voxels on the boundary of S tend to be assigned first and we get the expected advancing front assignment.

To evaluate our solution, it was compared on several cases with three other approaches: the mixed-integer linear program previously given, that provides us an optimal solution to the problem; simulated annealing as used in [2, 8]; as partitioning a graph via techniques like graphcut [6, 7, 18]. Comparisons between those four approaches were fully described in in [21]. Examples of results are given in 2D on

Algorithm 1: Voxels assignment greedy heuristic

Data: volume fraction VF , voxelated mesh
Result: Voxels assignment

```

1  $threshold \leftarrow 1.$ 
2  $freeVoxels \leftarrow allVoxels$ 
3  $fixedVoxels \leftarrow \emptyset$ 
4  $vf \leftarrow (VF, freeVoxels)$ 
5 for  $freeVoxels \neq \emptyset$  do
6   /* get the free voxels with a vf higher than the threshold for one material */
7    $fixedVoxelsToAdd \leftarrow extractVoxelsAbove(freeVoxels, threshold)$ 
8    $fix(fixedVoxelsToAdd)$ 
9   if  $fixedVoxelsToAdd \neq \emptyset$  then
10     $reduce\ threshold$ 
11  end
12  /* update the vf while subtracting the voxels already assigned */
13   $vf \leftarrow update(VF, freeVoxels)$ 
14  for  $iter \leq maxNbIter || convergence$  do
15    /* kind of a vf smoothing */
16     $vf \leftarrow average(vf) \text{ for voxels where } < threshold$ 
17     $normalize(vf)$ 
18  end
19 end

```

Fig. 10. The MILP implementation is impractical, as it does not return a solution in an acceptable time. For this small example, we stopped the optimization process after 5 min.⁵ Let us note that the result is valid, in the meaning meaning that it fits the constraints, but not optimal, which is the case in Fig. 10a. The graphcut approach tends to return straight interfaces, resulting in a good edgecut, but is quite bad when considering the *discrepancy*. That leaves us with the simulated annealing, which is a little better than our greedy heuristic regarding the edgecut in the case of a grid, but fares badly concerning the *discrepancy* in unstructured cases, as shown in Fig. 11. All of those methods have the same memory limitation, as the submesh, i.e. the set of voxels, can be quite large. In practice, we use our heuristic to build the voxelated interfaces as it is a good compromise between structured and unstructured cases and does not rely on tuning parameters depending on the case (Fig. 12).

3.1.2 Voxel Assignment Improvement

Our voxel assignment procedure can produce a few isolated voxels, which leads us to believe that the edgecut could be improved. Indeed our greedy approach tends to clump together voxels assigned to the same material but we do not make it mandatory for a free voxel to be assigned a material one of its neighbors is already

⁵ We use the open-source GLPK software [12].

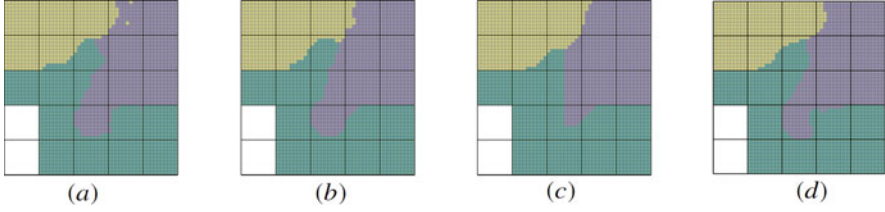


Fig. 10 Comparison of the interfaces reconstruction methods. (a) MIP $d=0$, $\text{edgecut}=592$. (b) simulated annealing $d=0$, $\text{edgecut}=536$. (c) graphcut $d=2.18$, $\text{edgecut}=440$. (d) greedy heuristic $d=0$, $\text{edgecut}=568$

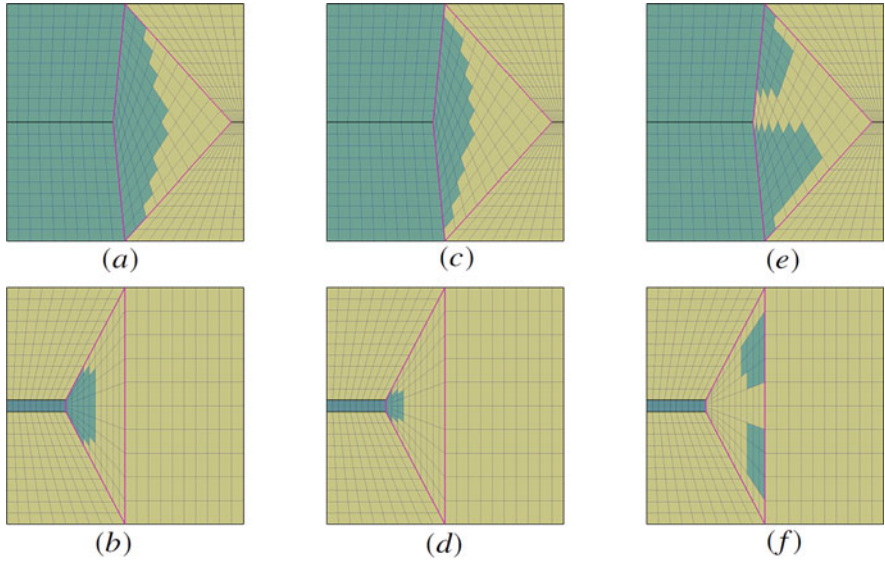


Fig. 11 Greedy heuristic (first column) vs. simulated annealing (second and third columns) on unstructured cases. Only the highlighted cell is mixed, and respective volume fractions are $(0.5, 0.5)$ on the top case, $(0.2, 0.8)$ on the bottom. Two different results (c) and (e), (d) and (f) are shown for the simulated annealing method since clusters of voxels can appear due to the randomness of the initial voxel assignment and the swaps. (a) $d=0.0768$. (b) $d=0.024$. (c) $d=0.264$. (d) $d=0.1444$. (e) $d=0.1344$. (f) $d=0.1254$

assigned to. We have so devised a correction procedure that spawns from a simple consideration: as the voxels assignment can be seen as a graph partitioning problem, adjusting the obtained partitions can be considered a repartitioning problem. We have thus experimented with two well-known repartitioning algorithms: the Kernighan-Lin and the Fiduccia-Mattheyes algorithm. Interested readers can find some more up-to-date references on the subject of graph partitioning in [4, 24].

The Kernighan-Lin [16] graph bi-repartitioning algorithm takes as an input a graph with its vertices split into two sets and proceeds to improve upon the initial partitioning by exchanging vertices between the sets, two by two. The algorithm

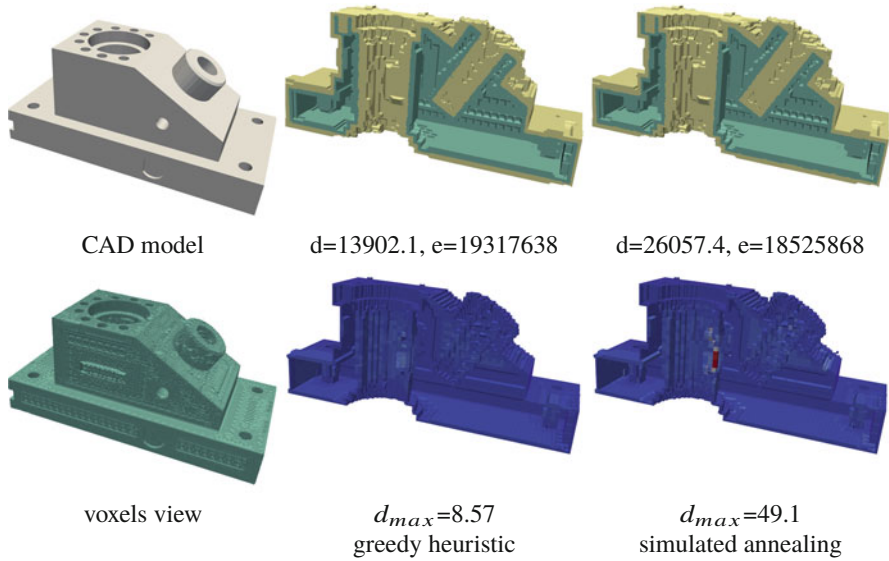


Fig. 12 Example in a real-life unstructured case. The results show that while our greedy heuristic (middle) is a little worse edgcut-wise than the simulated annealing (right), it fares better by a factor of 2 and 5 in terms of discrepancy, the total sum and its cell maximum respectively

drives the swaps by determining the sequence of swaps that maximizes the gain;⁶ incidentally it allows for “bad” moves, or negative gain moves as long as they are compensated. We modified the traditional implementation to fit some of our concerns. First we handle more than two partitions and secondly we restrict the possible exchanges to the swaps between voxels spawned from the same source cell.

Results in Fig. 13 show that it is quite effective in reducing the *edgcut* when there are isolated assignment artifacts. As the Kernighan-Lin method atomic operation is the swap, it has the exact same issue as the simulated annealing when the coarse mesh is unstructured and the voxels do not all have the same volume; it will preserve the number of voxels assigned to each material, and while it may improve on the edgcut it may lead to an increase of the discrepancy (see Fig. 14). In order to address this issue we applied to our problem the Fiduccia-Mattheyses repartitioning algorithm. Unlike the Kernighan-Lin algorithm, the Fiduccia-Mattheyses [10] method only changes the part to which a vertex is assigned instead of swapping two vertices at a time. In particular, it means that in the case where the initial partitions are perfectly balanced the algorithm will need to be allowed some wiggle room, i.e. the possibility to increase the imbalance between partitions, to be able to operate. In our problem it translates into our adaptation seen in Algorithm 2 line 6 where a

⁶ “gain” in terms of improving the *edgcut*.

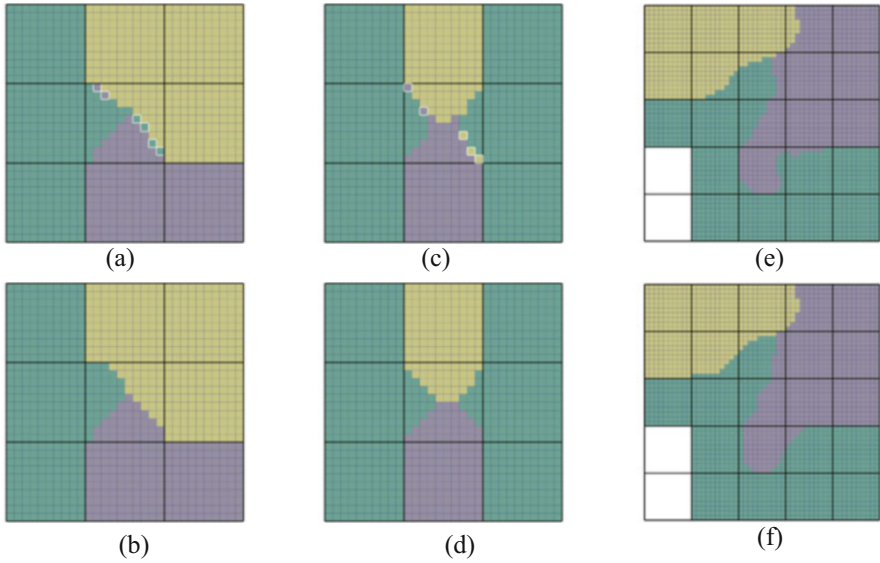


Fig. 13 KL algorithm (bottom) applied after the greedy heuristic (top). (a) $e=186$. (b) $e=144$. (c) $e=222$. (d) $e=178$. (e) $e=568$. (f) $e=532$

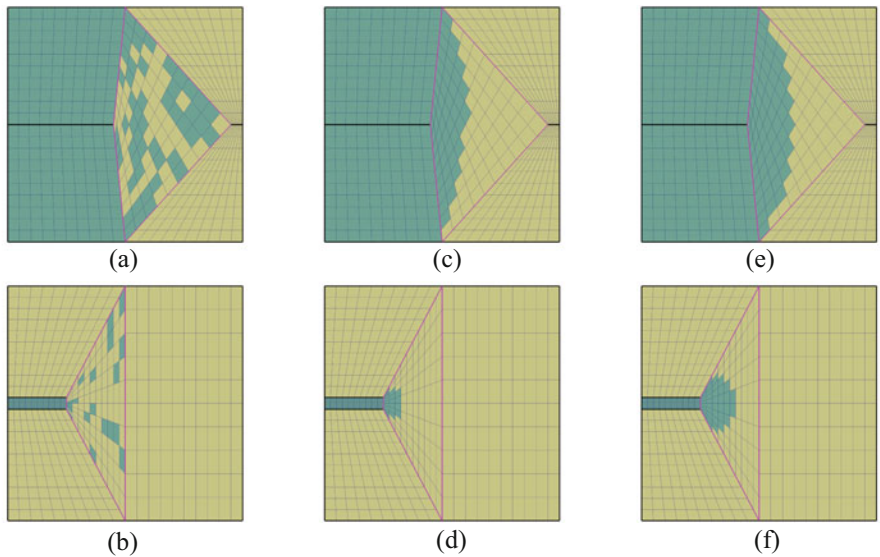


Fig. 14 Example of re-partitioning applied on two unstructured cases that shows the limits of only swapping the assignments (Kernighan-Lin) instead of changing the assignment (Fiduccia-Mattheyses). (a) and (b) an initial random assignment; (c) and (d) after applying the Kernighan-Lin algorithm, where we can see that while the edgecut is reduced, the discrepancy increases; (e) and (f) after applying the Fiduccia-Mattheyses, which reduces both. (a) $d=0.048$, $e=444$. (b) $d=0.0038$, $e=276$. (c) $d=0.264$, $e=84$. (d) $d=0.1444$, $e=76$. (e) $d=0.0128$, $e=86$. (f) $d=0.001$, $e=96$

Algorithm 2: Fiduccia-Mattheyes

```

1 while gain_cumul > 0 do
2   matAssign_tmp  $\leftarrow$  matAssign
3   compute costs for all vertices  /* one cost per material */
4   free all vertices
5   while  $\exists$  possible material change do
6     v, m  $\leftarrow$  find best material change /* change has to be allowed under volume
        fractions constraints */
7     store best material change in sequence
8     lock(v)
9     matAssign_tmp(v)  $\leftarrow$  m
10    update costs for v and neighbors
11  end
12  gain_cumul  $\leftarrow$  find sequence of material changes with maximal cumulative gain
13  if gain_cumul > 0 then
14    | matAssign  $\leftarrow$  execute material changes
15  end
16 end

```

material change for a voxel will only be considered if it does not degrade too much the discrepancy of the coarse cell this voxel is issued from. And again, negative gain moves can be performed, as long as they are compensated afterwards.

In Fig. 14 is shown the benefits of being able to change the material assignment of the voxels—as is done in the Fiduccia-Mattheyes algorithm—instead of only proceeding by swapping in unstructured cases. The Kernighan-Lin implementation greatly reduces the edgecut at the cost of increasing the discrepancy, while the Fiduccia-Mattheyes manages to reduce both, albeit a little less concerning the edgecut.

3.2 Geometrical Model Definition

Starting from our input mesh carrying volume fractions, we have built a finer submesh of “voxels” and assigned those to materials; we now get the opportunity to extract the interfaces between materials from which we can build an explicit geometrical model. This model can be built at two different levels of discretization (see Fig. 15): either on the fine mesh made of voxels (top row) or on the coarse mesh (bottom row). By construction, the first one provides high-fidelity to the reconstructed interfaces and preserves material volumes, while the second one is coarser, making it easier to handle (smaller memory footprint, easier to visualize and to use for mesh-to-geometry projection and smoothing). Considering our goal

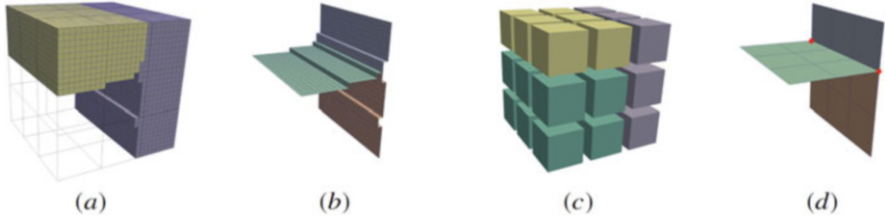


Fig. 15 Example of explicit geometrical models built from a 3 materials case in a $3 \times 3 \times 3$ grid. (a) and (b) the voxels assignment and its corresponding geometrical model; (c) and (d) the same after the assignment of the coarse cells

of getting a pure full-hexahedral mesh starting from an Eulerian mesh, we decided to build the coarser model. Moreover, it is a straightforward manner of ensuring to get a clean topology for the geometrical model $\mathcal{G}$.

Both models (the finest and the coarsest) can be built by first extracting the faces—the edges in 2D—between cells assigned to different materials (see Fig. 15b,d), then building a geometrical model $\mathcal{G} = (S, C, V)$. Starting from an hexahedral mesh $M = (H, Q, E, N)$, where H are hexahedral cells, Q are quadrilateral faces, E are edges and N are nodes, $\mathcal{G}$ can be extracted using the following rules:

- A **multi-surface** of S is a set of faces of Q that are adjacent to the same 2 materials. We get 3 such distinct multi-surfaces in the example on Fig. 15;
- A **multi-curve** of C is defined as a set of edges bounding the quad of a surface $s \in S$. Considering all the faces forming s , we get the set of edges $E_s \subseteq E$ that bounds those faces. This set of edges is then partitioned into multi-curves as follows: two edges of E_s are assigned to the same multi-curve if they are adjacent to the same set of materials assigned to the cells. For instance, let us consider the green surface on Fig. 15b,d; this surface is bounded by 2 curves: the first one corresponds to the intersection between the three surfaces—all the edges are then adjacent to exactly the 3 same materials; the second one is made of the remaining edges, which are adjacent to only 2 materials and located on the boundary of the domain—here the bounding box of the input grid;
- A **multi-vertex** of V corresponds to all the nodes of N that are adjacent to the same multi-curves in C , or in other words, to the same set of materials assigned to the cells of H . Considering the example of Fig. 15, the two nodes highlighted in (d) define a single multi-vertex.

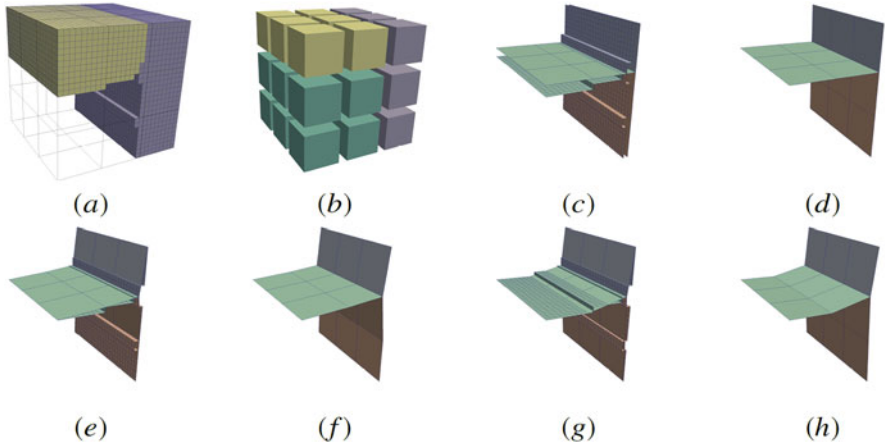


Fig. 16 Illustration of how the finer geometrical model is used as a support to project and smooth the coarser geometrical model. (c) and (d) the initial models; (e) and (f) the coarser model is projected onto the finer one; (g) and (h) the coarser model is smoothed while being constrained

Characterizing entities of the geometrical models using the material assignment of adjacent cells leads to form **multi**-entities that are potentially non-connected—in particular a vertex can have several spatial locations. For our purpose, this definition is sufficient and we do not further differentiate by splitting those entities into connected parts. In the example of Fig. 15, the geometrical models extracted are both made of 3 multi-surfaces, 4 multi-curves and 1 multi-vertex with two positions.

Now that we have those two models, we draw the correspondence between them and adapt the coarser representation by constraining it onto the finer model as seen in Fig. 16. Considering a fine model G_f and a coarse model G_c extracted from compatible data (see Fig. 16a and b), multi-entities of G_f and G_c are defined from the same set of materials and so can be identified and associated through this material correspondence. In Fig. 16, both models are superposed on (c), (e) and (g). To adapt G_c to fit G_f as best as possible, we first project each node of the coarse mesh that corresponds to a multi-entity of G_c onto the corresponding multi-entity of G_f (see Fig. 16e and f), then we smooth the node positions while keeping them projected onto the corresponding multi-entity of G_f (see Fig. 16g and h). The proposed solution has been widely used on several examples including realistic data, such as the result of a CFD simulation case that is shown in Fig. 17 where our input is a grid mesh carrying the volume fractions at $t = 1$ s and $t = 2$ s of the simulation.

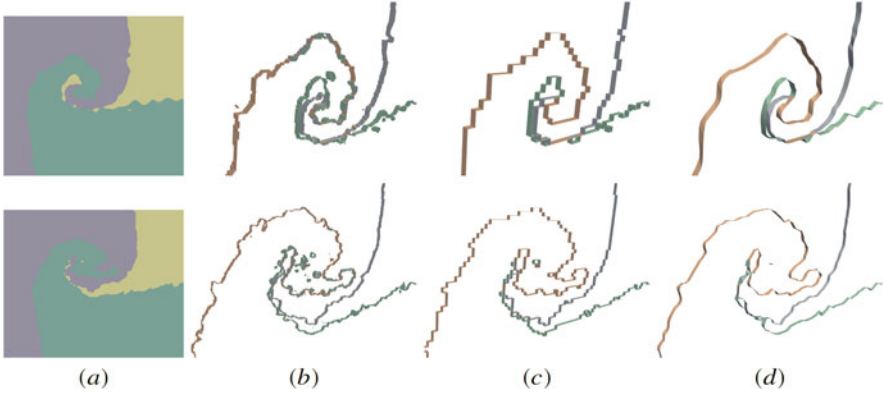


Fig. 17 Coarse geometrical model (c) projected and smoothed (d) onto the voxelated one (b) in the triple point problem at $t = 1$ s (top) and $t = 2$ s (bottom)

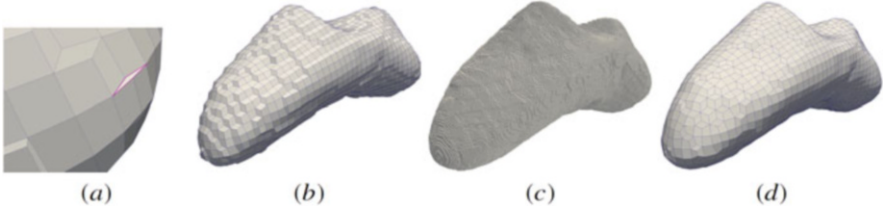


Fig. 18 Motivation of the geometrical model extraction and projection smoothing. (a) close-up of the expected interface mesh where the marked quad has a low quality of 0.068; (b) the mesh after our quality-driven mesh projection step; (c) the voxelated interface G_f between the asteroid and the exterior; (d) the coarse geometrical model G_c paired to G_f ; it replaces (a) as the expected positions of the interface nodes and no longer has low quality quads

4 Results

In the previous section, we built an explicit geometrical model $\mathcal{G}$ from a finer but dirty voxel-based geometry representation. We make use of $\mathcal{G}$ in our pipeline so as to avoid situations that can be encountered when working with implicit geometry representation only. For instance, if one tries to move the nodes towards a location computed using only the input volume fractions and the cells material assignment in the eulerian mesh, and considering each node independently and not the interfaces as a whole; in particular with no care taken for the expected interfaces quality (see Fig. 18a). It becomes especially relevant in 3D where the mesh entities forming the interfaces are no longer edges but faces. Moving the nodes to their computed ideal location can by-design lead to bad quality faces, hence severely limiting nodes movement during the quality-driven mesh projection. This causes our algorithm to be stuck with a mesh still fairly stair-shaped (see Fig. 18b). Such a resulting mesh could be considered satisfactory, quality-wise, still we want the interface nodes to

be located as close as possible to where the material interfaces were determined to be. Such issues are avoided when working with the coarser model $\mathcal{G}$ (see Fig. 18c and d).

In the remainder of this section, we give results obtained with the implementation of our pipeline, named *ELG*, in comparison with our own implementation of the SCULPT algorithm [25, 26], named *base algorithm*. Experiments were done on 2D and 3D CFD cases and results are gathered in Table 1. Cell quality is measured by the minimum scaled jacobian and a minimum threshold of 0.3 is chosen. Let us first begin with the triple point and double bar problems for which input grids of different resolutions were available. Taking the triple point case at 1s, we can see that for one grid resolution the base algorithm (where the mesh projection step does not strictly enforce a minimum quality) returns with a mesh containing no inverted cells (but still lower than the 0.3 minimum scaled jacobian threshold). That is not the case for the other resolution, making it unreliable. It is unrealistic to ask users to rerun their simulations with different resolutions at random, assuming it is even feasible, hence the need for our quality-driven mesh projection method that consistently works. Our method was applied in 2D on two additional hydrodynamics simulations issued from [33] (see Fig. 19); in all those cases it improves the distance by at least an order of magnitude.

The cases from Fig. 20 were extruded⁷ and run in 3D. While our method does indeed result in meshes meeting the quality requirements the ratio of $dist_{final}$ over $dist_{init}$ remains much higher than in the purely 2D cases. Fully 3D cases were also studied, one of which input is a grid where the volume fractions data were computed by imprinting an asteroid model onto the grid (see the example in Fig. 18). Other examples are eulerian meshes from hydrodynamic simulations run using [13], a ball of liquid that drops in a box taken at several time steps, a dam that breaks in Fig. 21 and a three material case where a liquid is poured onto a concrete pillar in Fig. 22.

While the measured distance illustrates the efficiency of our quality-driven mesh projection step, and the use it makes of the geometric model extraction, it remains a metric that relies on data that we extrapolated from the input mesh M_E and its material volume fractions. In Table 2, we measure the proximity of M_L to M_E at several stages of our pipeline, using the discrepancy criterion computed by imprinting M_L onto M_E . In those examples minimal values for the scaled jacobian of 0.2 and 0.15 were chosen for respectively the quality-driven mesh projection and the discrepancy-driven mesh deformation steps. Figure 22 shows those results for the in_out_flow case.

⁷ The 3D mesh is created from a 2D quad mesh, lying in the XY plane, by creating successive layers of hexahedral cells along the Z direction. Volume fractions are simply derived for each hexahedral cell from their origin quadrilateral cell.

Table 1 Quality and distance metrics for the examples. $dist_{init}$ and $dist_{final}$ are the sums of the distance between the interface nodes and their computed destination at respectively the beginning and the end of the mesh projection phase

Case name	$minJS$ <i>base algo</i>	$minJS$ <i>ELG</i>	$dist_{init}$	$dist_{final}$	$\frac{dist_{final}}{dist_{init}}$
2D					
triplepoint 1s 420×180	0.215	0.322	0.0676	0.0071	0.105
triplepoint 1s 518×222	−0.071	0.310	0.0856	0.0061	0.071
triplepoint 2s 420×180	−0.031	0.311	0.165	0.0138	0.084
triplepoint 2s 518×222	0.097	0.308	0.186	0.0163	0.088
doublebar 0.5s 200×100	0.074	0.306	0.5915	0.0163	0.027
doublebar 0.5s 214×107	0.091	0.301	0.5950	0.0121	0.020
doublebar 1s 200×100	−0.177	0.300	0.5768	0.0319	0.055
doublebar 1s 214×107	−0.109	0.301	0.6146	0.0411	0.067
hydro_toro_a	−0.104	0.300	116.70	6.0177	0.051
hydro_toro_b	−0.994	0.300	1902.3	104.34	0.055
3D					
triplepoint 1s $420 \times 180 \times 3$	0.067	0.300	34.048	21.587	0.634
triplepoint 2s $420 \times 180 \times 3$	−0.157	0.300	74.5847	44.388	0.595
doublebar 0.5s $200 \times 100 \times 3$	0.043	0.300	134.47	26.025	0.193
doublebar 1s $200 \times 100 \times 3$	−0.159	0.300	122.24	44.576	0.365
asteroid	−0.13	0.200	319.874	31.148	0.097
balldrop_10		0.274	41.426	5.5029	0.133
balldrop_15		0.209	35.243	6.3432	0.18
balldrop_20		0.221	35.824	18.149	0.506
balldrop_25		0.200	75.346	39.089	0.519
dambreak_10		0.200	34.444	17.638	0.512
dambreak_20		0.200	51.669	27.745	0.537
dambreak_30		0.200	46.866	24.097	0.514
dambreak_40		0.200	112.73	67.012	0.594
in_out_flow		0.200	132.75	75.079	0.565

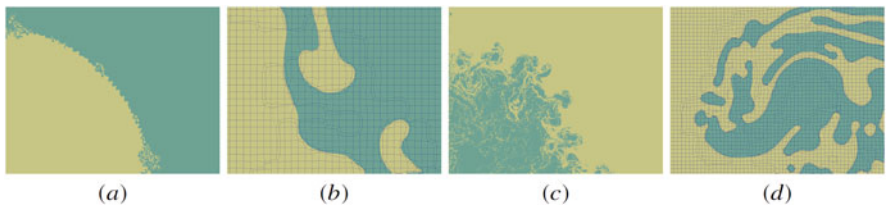


Fig. 19 Other examples of hydrodynamics simulations in 2D [33]. (a) and (c) the two cases; (b) and (d) close-ups on our resulting meshes shown respectively

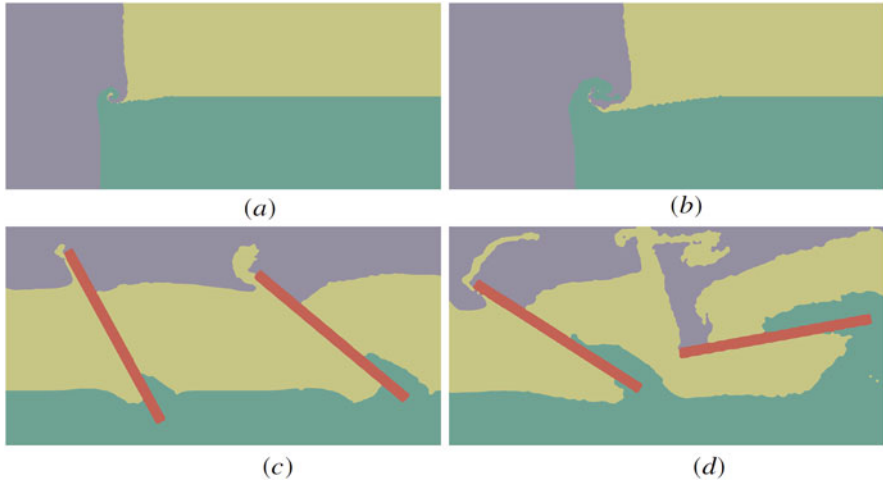


Fig. 20 Examples of CFD simulations in 2D. (a) and (b) triple point problem where three fluids of different densities lead to the formation of a vertex; (c) and (d) double bar problem where three fluids of different densities are stirred by two rotating blades. (a) $t = 1$ s. (b) $t = 2$ s. (c) $t = 0.5$ s. (d) $t = 1$ s

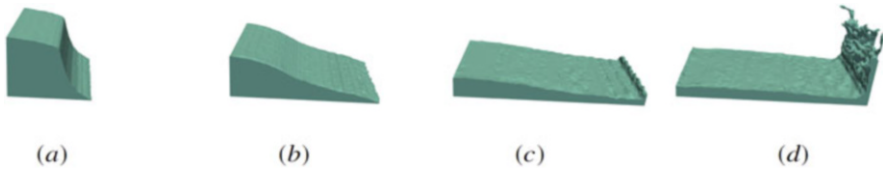


Fig. 21 Resulting meshes from our algorithm applied to the dambreak case. (a) time step 10. (b) time step 20. (c) time step 30. (d) time step 40

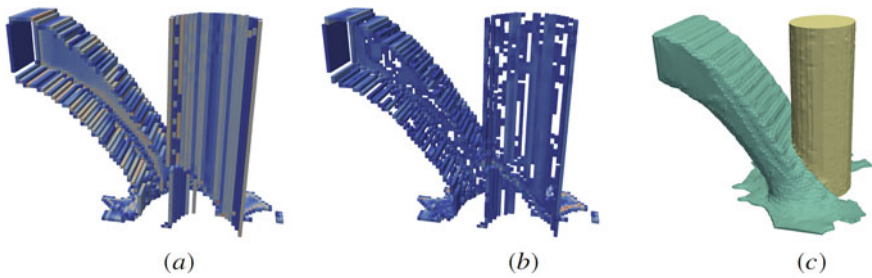


Fig. 22 Discrepancy displayed after the assignment step and the mesh projection step in the in_out_flow case (a) and (b). Red indicates a higher discrepancy per cell while blue is lower. (a) d_{assign} . (b) d_{proj} . (c) obtained mesh M_L

Table 2 Discrepancy across the full pipeline. d_{assign} is the discrepancy measured after the assignment step (see Fig. 6), d_{proj} is measured after our quality-driven mesh projection method and d_{deform} at the end after our discrepancy-driven final step

Case name	d_{assign}	d_{proj}	d_{deform}
dambreak_10	4.65907	1.3016	0.713
dambreak_20	6.2756	2.35647	1.370
dambreak_30	7.63199	3.48279	1.653
dambreak_40	12.3486	7.14286	4.486
in_out_flow	12.7552	4.70129	3.415

5 Conclusion

With the pipeline described in this chapter, we have got the ability to transform the output of an eulerian simulation code into an acceptable input for a lagrangian simulation code. This input consists in a mesh, which is full-quad in 2D and full-hex in 3D. Materials are preserved as best as possible in terms of locality and global volume using the notion of global discrepancy. We also ensure the strong constraint of providing lagrangian cells whose minimal quality reaches a user-parameter threshold. Ensuring it can be incompatible with the volume preservation. In this case, the priority is given to the cell quality, which is mandatory to run the lagrangian simulation code.

Let us note that the techniques depicted in this work are not limited to eulerian to lagrangian intercode problems: they are relevant as long as one is able to provide a mesh carrying volume fractions, which is typically the case for the examples issued from CAD models that we have used. Similarly, while the majority of the inputs that we have shown are grid meshes, we are not limited to those meshes and can handle any unstructured conformal hexahedral meshes as an input of the proposed pipeline. We do not handle non-conformal meshes, because as our method consists in using the input mesh as a base for our overlay-grid algorithm this base mesh should meet the requirements, first of all being conformal. Finally, most of the steps are not restricted to hexahedra and can directly accommodate other types of cells, such as tetrahedra and prisms, with the caveat of course that our output would not be an hexahedral mesh, but it was not the focus of this process.

References

1. Ahrens, J., Geveci, B., Law, C.: ParaView: An end-user tool for large data visualization. In: Visualization Handbook (2005)
2. Anderson, J.C., Garth, C., Duchaineau, M.A., Joy, K.I.: Discrete multi-material interface reconstruction for volume fraction data. Comput. Graphics Forum **27**(3), 1015–1022 (2008)
3. Anderson, J.C., Garth, C., Duchaineau, M.A., Joy, K.I.: Smooth, volume-accurate material interface reconstruction. IEEE Trans. Vis. Comput. Graph. **16**(5), 802–814 (2010)

4. Barat, R.: Load balancing of multi-physics simulation by multi-criteria graph partitioning. PhD Thesis, Bordeaux (2017)
5. Berkelaar, M., Eikland, K., Notebaert, P.: *Ip_solve* (2004)
6. Boykov, Y., Kolmogorov, V.: An experimental comparison of min-cut/max- flow algorithms for energy minimization in vision. *IEEE Trans. Pattern Anal. Mach. Intell.* **26**(9), 1124–1137 (2004)
7. Boykov, Y., Veksler, O., Zabih, R.: Fast approximate energy minimization via graph cuts. *IEEE Trans. Pattern Anal. Mach. Intell.* **23**(11), 1222–1239 (2001)
8. Childs, H., Brugger, E., Whitlock, B., Meredith, J., Ahern, S., Pugmire, D., Biagas, K., Miller, M., Harrison, C., Weber, G.H., Krishnan, H., Fogal, T., Sanderson, A., Garth, C., Wes Bethel, E., Camp, D., Rübél, O., Durant, M., Favre, J.M., Návrátil, P.: VisIt: An End-User Tool For Visualizing and Analyzing Very Large Data. In: *High Performance Visualization—Enabling Extreme-Scale Scientific Insight*, pp. 357–372 (2012)
9. Cplex, I.I.: V12. 1: User’s Manual for CPLEX. International Business Machines Corporation, **46**(53), 157 (2009)
10. Fiduccia, C.M., Mattheyses, R.M.: A linear-time heuristic for improving network partitions. In: *Proceedings of the 19th Design Automation Conference, DAC ’82*, pp. 175–181. IEEE Press, New York (1982)
11. Gao, X., Shen, H., Panozzo, D.: Feature Preserving Octree-Based Hexahedral Meshing. *Comput. Graphics Forum* **38**(5), 135–149 (2019)
12. GLPK: GLPK—GNU Project—Free Software Foundation (FSF)
13. Guy, R.: A PIC/FLIP fluid simulation based on the methods found in Robert Bridson’s “Fluid Simulation for Computer Graphics”: *rlguy/GridFluidSim3D* (2019)
14. Hege, H.-C., Seebass, M., Stalling, D., Zöckler, M.: A Generalized Marching Cubes Algorithm Based on Non-Binary Classifications (1997)
15. Herring, A., Certik, O., Ferenbaugh, C., Garimella, R., Jean, B., Malone, C., Sewell, C.: (U) Introduction to Portage. Technical report, Los Alamos National Laboratory (LANL) (2017)
16. Kernighan, B.W., Lin, S.: An efficient heuristic procedure for partitioning graphs. *Bell Syst. Tech. J.* **49**(2), 291–307 (1970). Conference Name: The Bell System Technical Journal
17. Knupp, P.M.: Algebraic mesh quality metrics. *SIAM J. Sci. Comput.* **23**(1), 193–218 (2001)
18. Kolmogorov, V., Zabin, R.: What energy functions can be minimized via graph cuts? *IEEE Trans. Pattern Anal. Mach. Intell.* **26**(2), 147–159 (2004)
19. Kucharik, M., Garimella, R.V., Schofield, S.P., Shashkov, M.J.: A comparative study of interface reconstruction methods for multi-material ALE simulations. *J. Comput. Phys.* **229**(7), 2432–2452 (2010)
20. Le Goff, N., Ledoux, F., Owen, S.J.: Hexahedral mesh modification to preserve volume. *Comput. Aided Des.* **105**, 42–54 (2018)
21. Le Goff, N., Ledoux, F., Janodet, J.-C., Owen, S.J.: Guaranteed quality-driven hexahedral overlay grid method. In: *Proceedings of the 28th International Meshing Roundtable* (2019)
22. Leng, J., Xu, G., Zhang, Y., Qian, J.: Quality improvement of segmented hexahedral meshes using geometric flows. In: *Image-Based Geometric Modeling and Mesh Generation. Lecture Notes in Computational Vision and Biomechanics* (2013)
23. LLC Gurobi Optimization: Gurobi Optimizer Reference Manual (2020)
24. Morais, S.: Study and obtention of exact, and approximation, algorithms and heuristics for a mesh partitioning problem under memory constraints. PhD Thesis. University of Paris-Saclay, France (2016)
25. Owen, S.J., Staten, M.L., Sorensen, M.C.: Parallel Hex Meshing from Volume Fractions. In: *Quadros, W.R. (ed.) Proceedings of the 20th International Meshing Roundtable*, pp. 161–178. Springer, Berlin (2012)
26. Owen, S.J., Brown, J.A., Ernst, C.D., Lim, H., Long, K.N.: Hexahedral mesh generation for computational materials modeling. *Procedia Eng.* **203**, 167–179 (2017)
27. Powell, D., Abel, T.: An exact general remeshing scheme applied to physically conservative voxelization. *J. Comput. Phys.* **297**, 340–356 (2015)

28. Qian, J., Zhang, Y.: Automatic unstructured all-hexahedral mesh generation from b-reps for non-manifold cad assemblies. *Eng. Comput.* **28**, 345–359 (2012)
29. Qian, J., Zhang, Y., Wang, W., Lewis, A.C., Siddiq Qidwai, M.A., Geltmacher, A.B.: Quality improvement of non-manifold hexahedral meshes for critical feature determination of microstructure materials. *Int. J. Numer. Methods Eng.* **82**(11), 1406–1423 (2010)
30. Schneiders, R.: A grid-based algorithm for the generation of hexahedral element meshes. *Eng. Comput.* **12**(3), 168–177 (1996)
31. Schneiders, R., Schindler, R., Weiler, F.: Octree-based Generation of Hexahedral Element Meshes. In: *Proceedings of the 5th International Meshing Roundtable* (1999)
32. Staten, M.L., Owen, S.J.: Parallel octree-based hexahedral mesh generation for eulerian to lagrangian conversion. Report (2010). Library Catalog: digital.library.unt.edu Number: SAND2010-6400 Publisher: Sandia National Laboratories
33. Toro, E.: Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction. In: *Riemann Solvers and Numerical Methods for Fluid Dynamics* (2009)
34. Zhang, Y., Bajaj, C.: Adaptive and quality quadrilateral/hexahedral meshing from volumetric data. *Comput. Methods Appl. Mech. Eng.* **195**(6), 942–960 (2006)
35. Zhang, Y., Bajaj, C., Xu, G.: Surface smoothing and quality improvement of quadrilateral/hexahedral meshes with geometric flow. *Commun. Numer. Methods Eng.* **25**(1), 1–18 (2009)
36. Zhang, Y., Hughes, T.J.R., Bajaj, C.: An automatic 3d mesh generation method for domains with multiple materials. *Comput. Methods Appl. Mech. Eng.* **199**(5), 405–415 (2010). *Computational Geometry and Analysis*
37. Zhang, Y., Liang, X., Xu, G.: A robust 2-refinement algorithm in octree and rhombic dodecahedral tree based all-hexahedral mesh generation. *Comput. Methods Appl. Mech. Eng.* **256**, 88–100 (2013)

Gmsh's Approach to Robust Mesh Generation of Surfaces with Irregular Parametrizations



Jean-François Remacle and Christophe Geuzaine

Abstract This paper proposes a robust and effective approach to overcome a major difficulty associated to surface finite element mesh generation: the handling surfaces with irregular (singular) parametrizations such as spheres, cones or other surfaces of revolution produced by common Computer Aided Design tools. The main idea is to represent triangles incident to irregular points as trapezoids with one degenerated edge. This new approach has been implemented in Gmsh and examples containing thousands of surfaces with irregular points are presented at the end of the paper.

1 Introduction

Computer Aided Design (CAD) systems are used extensively for industrial design in many domains, including automotive, shipbuilding, and aerospace industries, industrial and architectural design, prosthetics, and many more. Engineering designs are encapsulated in such CAD models, which up to manufacturing tolerances exactly represent their geometry. While the engineering analysis process begins with such CAD models, the predominant method of analysis (the finite element method) requires an alternative, discrete, representation of the geometry: a finite element mesh. In such a mesh, the CAD model is subdivided into a (large) collection of simple geometrical shapes such as triangles, quadrangles, tetrahedra and hexahedra, arranged in such a way that if two of them intersect, they do so along a face, an edge or a node, and never otherwise.

J.-F. Remacle (✉)

Institute of Mechanics, Materials and Civil Engineering (iMMC), Université catholique de Louvain, Louvain-la-Neuve, Belgium

e-mail: jean-francois.remacle@uclouvain.be

C. Geuzaine

Department of Electrical Engineering and Computer Science, Montefiore Institute B28, Université de Liège, Liège, Belgium

e-mail: c.geuzaine@uliege.be

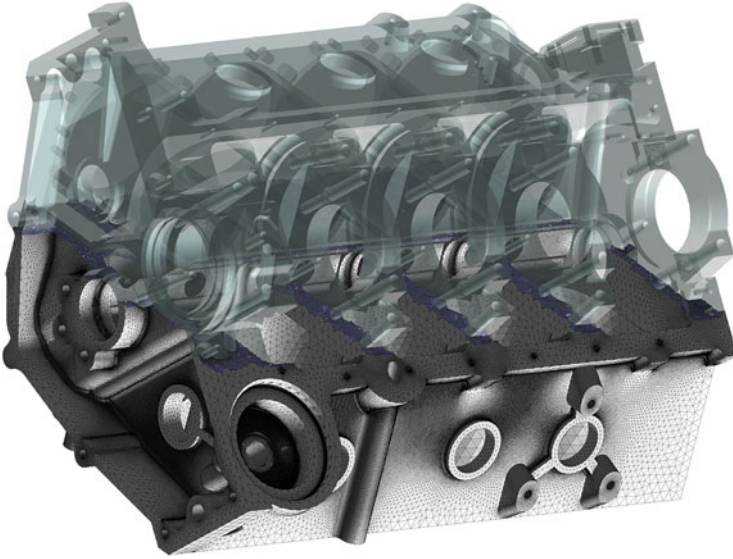


Fig. 1 An engine block

Three-dimensional CAD models are represented on a computer using a “Boundary Representation” (BRep) [1]: a volume is bounded by a set of faces, a face is bounded by a serie of curves and a curve is bounded by two end points. The BREP is a discrete object: it is a graph that contains model entities together with all their topological adjacencies. Then a geometry is associated to each model entity. Figure 1 presents a moderately complex CAD model together with its 3D mesh generated using Gmsh [2].

As an example, consider a model face F with its boundary

$$\partial F = \{C_1, \dots, C_n\}.$$

Face F is topologically closed, i.e. $\partial(\partial F) = \emptyset$: each endpoint of the bounding curves C_j is considered twice in F , one time positively and one time negatively. The geometry of a model face F is its underlying surface S with its parametrization

$$\mathbf{x} : A \mapsto \mathbb{R}^3, \quad (u, v) \mapsto \mathbf{x}(u, v)$$

where $A \subset \mathbb{R}^2$ is a rectangular region $[u_0, u_1] \times [v_0, v_1]$. A parametrization is said to be regular if $\partial_u \mathbf{x}$ and $\partial_v \mathbf{x}$ are linearly independent:

$$\partial_u \mathbf{x} \times \partial_v \mathbf{x} \neq \mathbf{0}$$

for any $u, v \in A$. Points where $\partial_u \mathbf{x} \times \partial_v \mathbf{x} = \mathbf{0}$ are called irregular or singular points of the parametrization. We assume here that irregular points are isolated. Irregular

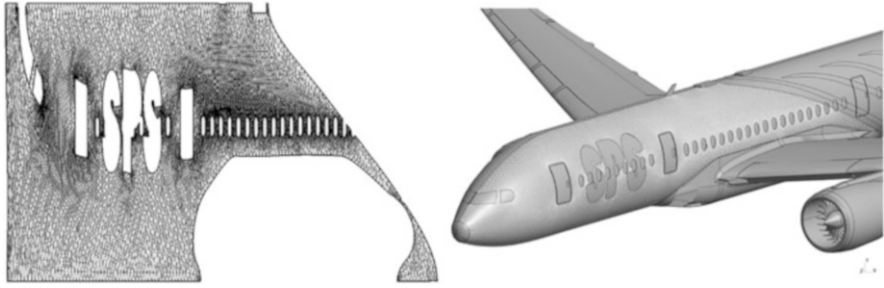


Fig. 2 Surface mesh of a model face. View of the mesh in the parameter plane (left) and on $\mathbb{R}^3$

points can occur for two possible reasons: (i) one of the partial derivatives $\partial_u \mathbf{x}$ or $\partial_v \mathbf{x}$ is equal to $\mathbf{0}$ or (ii) partial derivatives are non zero and are parallel. The second case (ii) where partial derivatives are non zero yet are parallel is not common in practice while case (i) appears quite often.

The underlying geometry of a face F is thus a parametric surface $\mathbf{x}(u, v)$. Yet, its domain is often smaller than A : A is usually trimmed by boundaries C_j and the geometry of the trimming curves are algebraic curves $\mathbf{c}_j(u, v) = 0$ defined in the (u, v) plane of F . Figure 2 shows an example of a trimmed surface.

Generating a triangular surface mesh of F consists in generating a planar triangular mesh in its parameter plane whose map through $\mathbf{x}(u, v)$ is a valid mesh in $\mathbb{R}^3$ with triangles of controlled shapes and sizes.

A triangle is valid in the (u, v) plane when it is properly oriented, i.e. when its area is strictly positive. It is indeed more complicated to assess that a triangle is valid in $\mathbb{R}^3$. Assume a triangle $(\mathbf{a}, \mathbf{b}, \mathbf{c})$ with its non unit normal $\mathbf{n} = (\mathbf{b} - \mathbf{a}) \times (\mathbf{c} - \mathbf{a})$ and the normal to the CAD surface at the centroid

$$(u_t, v_t) = \frac{1}{3}(u_a + u_b + u_c, v_a + v_b + v_c)$$

of the triangle:

$$\mathbf{n}_{\text{CAD}} = \partial_u \mathbf{x}(u_t, v_t) \times \partial_v \mathbf{x}(u_t, v_t).$$

We say that triangle (a, b, c) is valid if $\mathbf{n}_{\text{CAD}} \cdot \mathbf{n} > 0$.

In the example of Fig. 2, the depicted trimmed surface has no irregular points and the mesh generation procedure is usually straightforward. In this specific example, the anisotropic frontal-Delaunay approach that is implemented in Gmsh [2] was used based on the metric tensor

$$\mathcal{M} = \begin{pmatrix} \|\partial_u \mathbf{x}\|^2 & \partial_u \mathbf{x} \cdot \partial_v \mathbf{x} \\ \partial_u \mathbf{x} \cdot \partial_v \mathbf{x} & \|\partial_v \mathbf{x}\|^2 \end{pmatrix} \quad (1)$$

that is of full rank everywhere.

Surfaces with isolated irregular points are however very common in CAD systems: spheres, cones and other surfaces of revolution may contain one or two irregular points. Mesh generation procedures are known to be prone to failure close to irregularities. Consider for example the parametrization of a sphere as it is used to our best knowledge in every CAD system. A sphere of radius R centered at the origin is parametrized as

$$x(u, v) = R \sin u \cos v$$

$$y(u, v) = R \sin u \sin v$$

$$z(u, v) = R \cos u$$

where $u \in [0, \pi]$ is the inclination and $v \in [0, 2\pi[$ is the azimuth. At the poles, i.e. when $u = 0$ or $u = \pi$,

$$\partial_v \mathbf{x} = R(-\sin u \sin v, \sin u \cos v, 0) = (0, 0, 0)$$

vanishes and this parametrization is irregular at the two poles of the sphere.

In this paper, a new approach is proposed that allows to generate meshes of surfaces with irregularities in an efficient and robust fashion. At first, we explain in Sect. 2 why indirect surface mesh generation procedures become fragile at the vicinity of irregular points. Then in Sects. 3 and 5, we present the critical modifications to standard meshing procedures that allow to address issues related to irregular parametrizations. Examples of CAD models with thousand of spheres and cones are finally be presented in Sect. 7.

2 The Issue of Meshing Surfaces with Irregular Parametrizations

Two main approaches exist for surface meshing. The first approach, usually called the “direct approach” [3], consists in generating the mesh directly in $\mathbb{R}^3$. Different direct approaches have been proposed in the literature: advancing front methods [4, 5], octree based methods [6, 7], methods based on local mesh modifications [8, 9], methods based on restricted Voronoi diagrams [10], ... Octree- and Voronoi-based methods have in common the need to intersect a 3D object (an octree or a Voronoi Diagram) with the surface that is to be meshed. When an octree is used, the intersection of the octree with the surface is usually irregular and local mesh modifications have to be performed in order to obtain a quality mesh. On the other hand, when the Voronoi diagram of the points is used, recovering edges (sharp features) of the surface is an issue. Other direct methods generate triangles on the surface without using any kind of 3D object. Advancing front methods and paving methods [11] add points and triangle on the surface using a frontal approach. Those methods handle sharp features without difficulties and allow to generate quality

meshes. Yet, such methods are plagued with robustness issues (front colliding and 3D intersections of 2D objects). Some direct approaches [8, 9] start from a “CAD” mesh and modify it to produce a “computational” mesh with elements of controlled shapes and sizes. The main disadvantage of such an approach is that it requires an initial mesh. One may use STL triangulations provided by CAD modelers but those are not guaranteed to be watertight on a whole CAD model and a complex preprocessing step is usually required to fix holes and T-junctions. Another issue is related to what could be called an “isogeometric” argument: the final “computational” mesh and the initial “CAD” mesh are piecewise linear complexes that do not necessarily cover the same geometry. Modifying an existing surface mesh using local mesh modification like vertex repositioning leads to vertices located outside of the input geometry i.e. the “CAD” mesh. While meshing procedures of this kind that actually ensure that the distance between the “CAD” and the “computational” mesh is bounded, those are based on complex datastructures and require to compute Hausdorff distances between triangulations [12].

When mesh generation procedures have access to parameterizations of surfaces, one can generate a planar mesh in the parametric domain and map it in 3D. This surface meshing approach is called “indirect”. In Gmsh, surface meshes are generated in the parameter plane (u, v) and standard “off the shelf” anisotropic 2D meshers are used for generating surface meshes. This is of course the main advantage of the indirect approach: a priori, no major coding effort is required to go from planar meshing to surface meshing. This last statement is of course a little bit too optimistic. Ensuring that a planar mesh is valid is trivial: all triangles should be positively oriented. Now, if the surface parametrization $\mathbf{x}(u, v) \in \mathbb{R}^3$ is regular, then the mapping of the (u, v) mesh onto the surface is itself valid because the composition of two regular mappings is regular. For example, the very simple mesh of the parameter plane of the whole sphere presented in Fig. 3 maps exactly the sphere as depicted in the bottom part of Fig. 3.

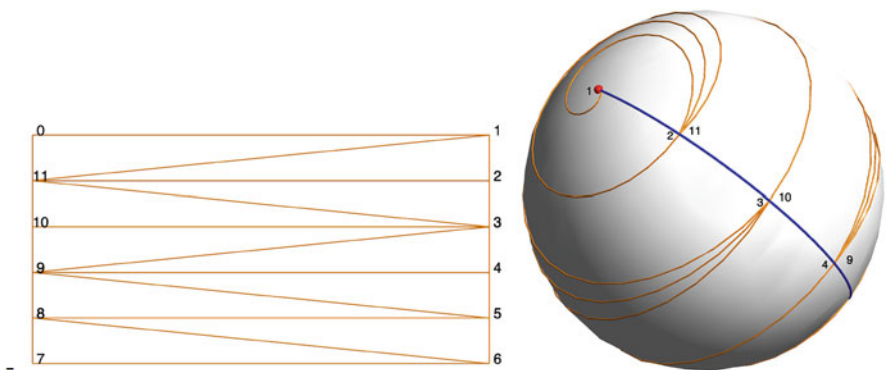


Fig. 3 A very simple mesh (left) of the parameter plane of a sphere and (right) its mapping through spherical coordinates

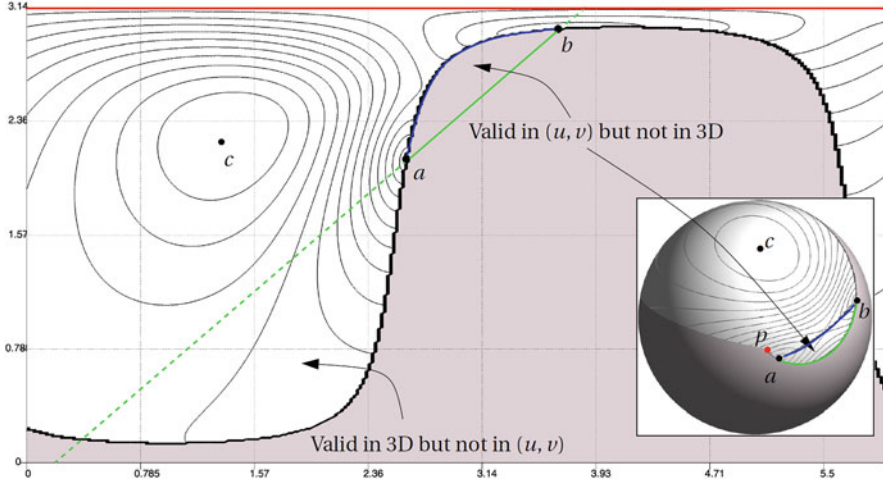


Fig. 4 Parameter plane of a sphere. We consider an edge (a, b) . Figures depict the quality of a triangle (a, b, c) with c positioned anywhere in the parameter plane. The grey zone corresponds to invalid triangles in 3D

Here, the main issue is that we do not actually map entire triangles onto the surface but only their corners. The topology of the 2D mesh is simply “translated” in 3D: straight sided triangles in the (u, v) plane become straight sided triangles in 3D. Another “isogeometric” issue thus appears in the indirect approach: the mapping $\mathbf{x}(u, v)$ of a triangle in the (u, v) plane is not equal to the straight sided triangle in $\mathbb{R}^3$. So, a valid 2D triangle in the parameter plane does not necessary produce a valid 3D triangle on the surface. For example all the triangles in the parameter plane in Fig. 3 are mapped onto zero-area triangles in $\mathbb{R}^3$. On the other hand, an invalid 2D triangle (i.e. with a negative area) may be perfectly valid in 3D.

In order to illustrate those issues, Fig. 4 shows the example of the parameter space of a complete sphere. An edge (a, b) where b is close to the north pole p (in red) is considered. Edge (a, b) is used to form a triangle (a, b, c) where $c \in A = [0, 2\pi[\times [0, \pi]$. The iso-lines that are presented are iso-values of triangle qualities:¹ the particular point c drawn on the Figure is the only one in the parameter plane leading to a valid equilateral triangle in 3D. The grey zone in the Figure corresponds to the locations of points c that form invalid elements in 3D. Invalid elements in the parameter plane (u, v) correspond to points above the green line that passes through (a, b) . It can be seen that there exists a zone where triangles are valid in 2D but not in 3D, and another zone where elements are valid in 3D but not in 2D. Some interesting comments can be made with respect to Fig. 4:

¹ We use as quality metric the ratio $2\frac{r}{R}$ between the inner-radius r and the circumradius R multiplied by 2 in order to have a quality equal to one for the equilateral triangle.

- The blue line is the 3D geodesic between a and b . This geodesic is far from being a straight line in the parameter plane, especially when the edge (a, b) is close to the pole (in red). Geodesics are straight lines in the parameter plane when the metric tensor $\mathcal{M}$ is constant (this is a sufficient condition). We will see in the next section that geodesics that are incident to an irregular point are also straight lines, even though the metric $\mathcal{M}$ has strong variations close to singularities.
- The point c in Fig. 4 that corresponds to an equilateral triangle (a, b, c) is always in the valid zone i.e. the zone where triangles are both valid in 2D and 3D. More generally, good quality triangles can always be formed in the parameter plane, even when the metric is very distorted.
- The zone that is valid in 2D but not in 3D is the most problematic for mesh generation algorithm that work in the parametric plane. Hopefully, this zone only contains points c for which triangles (a, b, c) are of bad quality.

It is difficult to generalize those three comments to general surfaces but our experience (through numerical experiments) shows that they do indeed hold.

The main question can thus be formulated as follows: *assuming a surface with a parametrization that may contain isolated irregular points, can we always find a valid 2D mesh that corresponds to a valid 3D mesh?*

When we started to think about version 4 of Gmsh, our answer to that question was tending to be *no*, at least using the current implementation of the surface mesh generators. The typical issue that was encountered at the time is illustrated on Fig. 5. The right part of the Figure represents the mesh in the parametric plane (u, v) while the right part of the Figure represents the surface mesh in $\mathbb{R}^3$.

In Fig. 5, the surface $\mathcal{S}$ is a sphere. Points like c or g (in green) are classified on model face F . Points like d or f (in pink) are classified on regular model edges that bound F while points like b and e are classified on the seam of F (in order to have $\partial(\partial F) = \emptyset$ some CAD systems like OpenCASCADE close periodic surfaces with a seam). Point b is a pole of the sphere: it is an irregular point. The parametric mesh is perfectly valid i.e. triangles cover exactly A without overlap. Yet, even though

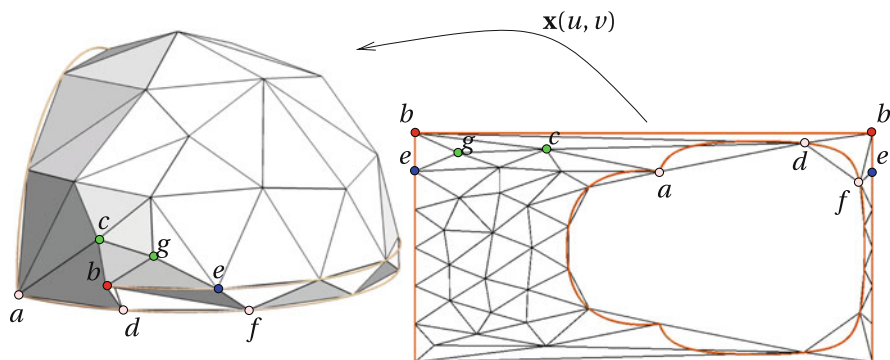


Fig. 5 A valid mesh in the parameter space that is invalid in the real space

triangle (b, c, d) is correctly oriented in the parametric plane, it is invalid in $\mathbb{R}^3$. We are here in the situation of Fig. 4 where point a is above the geodesic between c and d in the parameter plane. One single edge flip could potentially make the 3D mesh valid and of better quality: exchanging edges (c, d) and (a, b) fixes all issues. Yet, doing so makes the parametric mesh invalid. With the set of points that is depicted in Fig. 5, we found it impossible to build a quality mesh in $\mathbb{R}^3$ that is valid in the (u, v) plane.

Contrary to what one might think, the main issue here is not the fact that the metric tensor (1) is of rank 1 at irregular points and very distorted around it. In the context of mesh generation, geometrical queries like the evaluation of the metric tensor $\mathcal{M}$ are never done at irregular points; and anisotropic mesh generators are able to generate meshes for smooth metric fields even though they are very distorted. The mesh generation issue that arises here is essentially related to triangles (e.g. (b, c, d) in the Figure) and edges that have one vertex like b that corresponds to an irregular point of the parametrization.

Another minor issue will be fixed by our new approach. The existence of one degenerated mesh edge connecting points b implies the existence of an irregular triangle (d, b, b) that has one degenerated edge. This triangle can be eliminated in a post processing stage but its presence is quite annoying in the mesh generation process: computation of circumcircles, edge flips (flipping edge (b, d) does not change the mesh), ...

3 Geodesics of Surfaces of Revolution

Most of the CAD surfaces that have irregular points are surfaces of revolution. Consider a surface of revolution with respect to the z -axis and suppose that the generating curve is

$$\mathbf{c}(v) = (f(v), 0, g(v)) \quad , \quad v \in [0, T].$$

The parametrization of the surface is given by

$$\mathbf{x}(u, v) = (f(v) \cos(u), f(v) \sin(u), g(v)), \quad (2)$$

$(u, v) \in [0, 2\pi[\times [0, T]$. Geodesics of surface of revolution, even though their forms are not trivial (see for example the blue line of Fig. 4), have specific properties [13]. One interesting property of surfaces of revolution is that meridian curves $u = \text{cste}$ are geodesics.

Surfaces of revolution may have irregular points: if $f(0) = g(0) = 0$ in (2), then $\mathbf{x}(u, 0) = (0, 0, 0)$ for every u . The origin of the axis belongs to the surface and is thus an irregular point as defined above. Let us now look at the parameter plane (u, v) corresponding to a surface of revolution with a irregular point at $u = 0$.

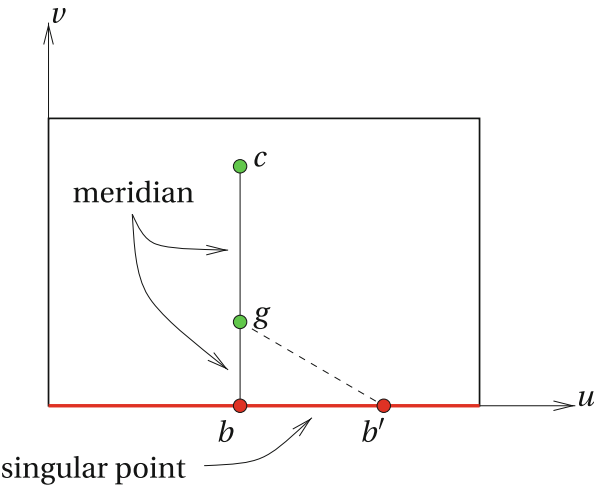


Fig. 6 Parameter space corresponding to a surface of revolution with an irregular point at $u = 0$

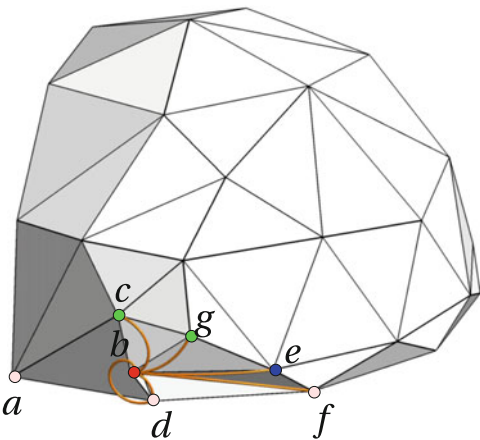


Fig. 7 True mapping of straight lines in the parameter space onto $\mathbb{R}^3$ close to an irregular point

Figure 6 gives an illustration of that situation. The thick red line $u = 0$ is mapped onto one single point $\mathbf{x} = (0, 0, 0)$. Thus, edges (g, b) and (g, b') have the same end-points in $\mathbb{R}^3$ but the only geodesic between those two points is the meridian (g, b) .

This simple result allows us to critically examine Fig. 5: edges like (c, b) , (g, b) or (d, b) are far from being geodesics and are thus far from their corresponding straight edges in $\mathbb{R}^3$, as depicted in Fig. 7. On the other hand, edge (b, f) is close to be a geodesic and its 3D representation is close to the corresponding straight line.

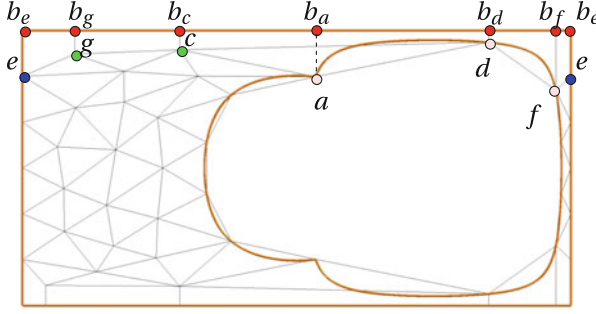


Fig. 8 New representation in the parameter space where every edge connected to an irregular point is a meridian. Point b_i 's all belong to edge (b_i, i) even though all b_i 's 3D locations are equal

Coming back to the mesh generation problem, it should be interesting to replace all the edges that are incident to irregular points by meridians. The new representation of the mesh in the (u, v) plane is depicted in Fig. 8.

With this representation, the unique edge flip that allows to have a valid mesh in $\mathbb{R}^3$ is permitted. Edge (a, b_a) (in dashed lines) can replace edge (c, d) without creating invalid triangles in the parameter plane (edge (c, g) could be flipped as well even though it is not required). Note here that triangles incident to irregular points are now right trapezoids with one degenerated edge, which means that no degenerated triangles exist in that new representation.

4 Modifications of the Initial Mesh

Our surface mesh generation procedure starts with an initial “empty mesh” i.e. a mesh in the parameter space that contains only vertices of the surface boundaries. Then, in this new procedure, edges that are adjacent to singularities are transformed onto geodesics. The question that is addressed in this section is the validity of this initial transformation.

Consider the surface presented in Fig. 9 together with a mesh generated using the new version of Gmsh's *MeshAdapt* surface mesher (see Sect. 5 below). The initial mesh that contains all boundary points is presented in Fig. 9. Again, a seam and two irregular points a and b are present in the surface plus a trimming curve that contains points c and d .

In our new procedure, all edges that are adjacent to irregular points are transformed onto geodesics. Figure 9 shows the result of that transformation: points like b_c are added to the parameter space to generate a geodesic cb_c .

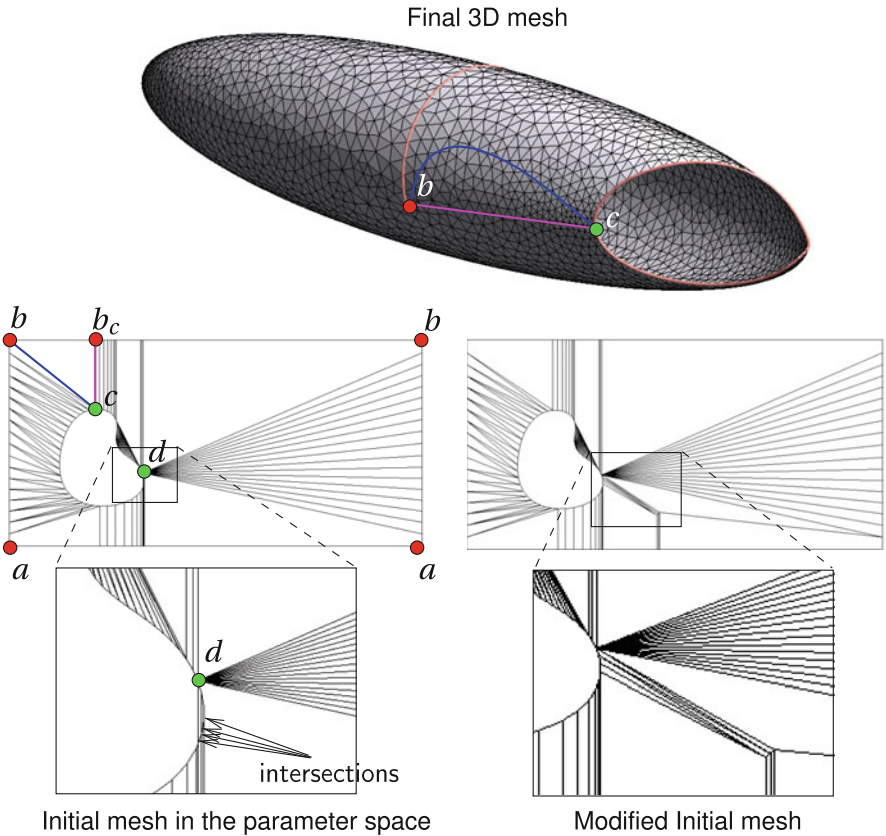


Fig. 9 Modification of the initial mesh in the parameter space in order to avoid edge intersections

It is actually easy to figure out that the initial mesh of Fig. 9 is actually wrong (inverted) in the parameter space. Figure 9 shows a zoom of the three problematic edges that make intersect other edges of the mesh and makes it invalid. The problem comes from the fact that, in Fig. 9, four edges of the internal boundary are initially connected to the point on the bottom right of the external rectangle. Yet, those edge normals are pointing upward so the proposed correction creates edges that intersect other edges of the internal boundary.

Addressing this problem is indeed quite simple. All problematic geodesic edges are split along their original path (not along the geodesic of course) up to the point when no intersection occurs. The resulting initial mesh is presented in Fig. 9.

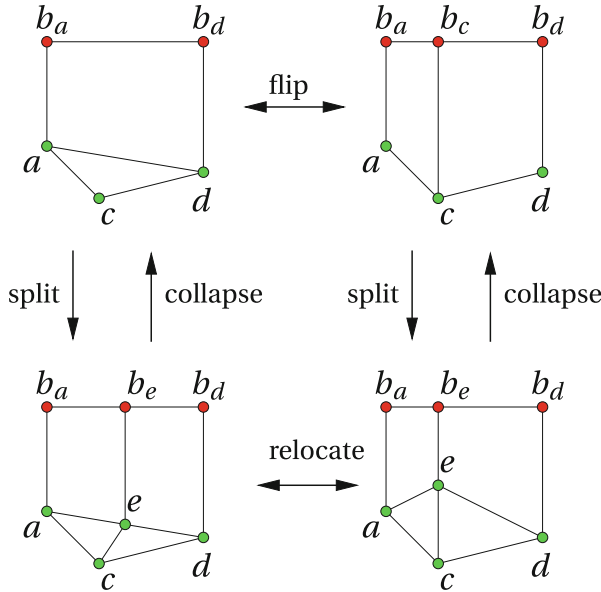


Fig. 10 Local mesh modifications with irregular points

5 Local Mesh Modifications: Gmsh's MeshAdapt Algorithm Revisited

Gmsh's most basic surface mesher is called *MeshAdapt*.² MeshAdapt's surface meshing strategy is based on the concept of local mesh modifications [14–16]. The algorithm works as follows. First, an initial mesh containing all the mesh points and edges of the model edges that bound a face is built in the parametric space (u, v) (see Sect. 4). Then, local mesh modifications are applied to the mesh in the parameter plane:

1. Each edge that is too long is split;
2. Each edge that is too short is collapsed;
3. Edge flips are performed in order to increase mesh quality;
4. Vertices are re-located optimally after steps 1, 2, and 3.

Figure 10 illustrates local mesh modifications applied to edges that are in the vicinity of a irregular point b . When edge (a, c) is flipped, a new instance of point b_c is created on the degenerated edge and point c becomes connected to b_c . The operation can be reversed as depicted in Fig. 10. When an edge like (a, d) is split at point e , a new point b_e is created on the degenerated line. When an edge like (c, b_c) that is

² `gmsh -algo meshadapt` is the commandline that forces gmsh to use that algorithm.

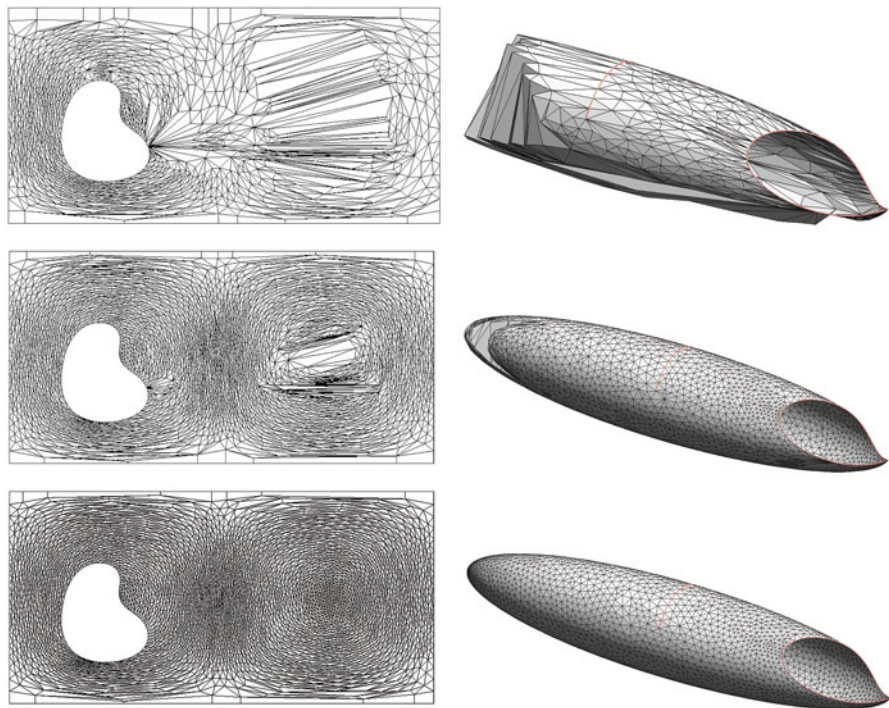


Fig. 11 Different stages of the *MeshAdapt* algorithm

connected to the irregular point is split, b_c is replaced by b_e . Note that when a point like e is relocated, point b_e is relocated as well.

All four local mesh modifications of our algorithm involve details of implementation that are too specific to be described in a paper but that are critical for robustness. Interested readers can download the source code of Gmsh 4.3.0 that implements the algorithm that exactly corresponds to the examples of the paper. Nevertheless, the most critical part of the *MeshAdapt* surface mesher is the vertex relocation, both in term of the final mesh quality and of CPU time (it actually takes about 60% of the total mesh generation time). When parameterizations are very distorted, simple smoothing strategies do not actually produce improvements of the mesh, especially close to singularities. In this new version of the algorithm, advanced optimization procedures have been used [17] for vertex relocation. Figure 11 show the mesh of the surface of Fig. 9 at different stages of the *MeshAdapt* algorithm.

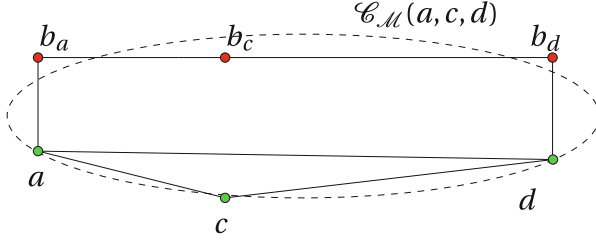


Fig. 12 Illustration of the application of the circle criterion close to an irregular point

6 Delaunay Mesh Generation Close to Irregular Points

Gmsh's frontal-Delaunay algorithm is an extension to surface meshing of the planar frontal-Delaunay mesher described in [18]. Points are inserted in the domain in a frontal fashion while always keeping a valid mesh during the process. The mesh is generated in the (u, v) plane which means that an anisotropic Delaunay criterion is required to produce isotropic meshes in 3D.

The most critical operation involved in that algorithm is the edge flip. Consider Fig. 12: we would like to figure out whether edge (a, d) should be flipped or not. In order to apply Delaunay's empty circle criterion, we actually work in the tangent plane and compute a unique metric tensor $\mathcal{M}$ at location $(a + b_c + c + d)/4$ that is symmetrical with respect to points a , b_c , c , and d . This allows to avoid "unstable flips". In this tangent plane, circle $C_{\mathcal{M}}(a, c, d)$ is an ellipsis. The new representation that is proposed here allows to provide a robust way of computing Delaunay flips.

In the example of Fig. 12, edge (a, d) should be flipped to (c, b_c) because b_c is inside $C_{\mathcal{M}}(a, c, d)$. Other occurrences of b like b_a or b_d may be located outside $C_{\mathcal{M}}(a, c, d)$ but the only edge that should be considered in the circle test is the geodesic (c, b_c) .

7 Examples

We have chosen two examples that were invariably creating invalid elements in all previous versions of Gmsh.

7.1 Many Spheres

One of the nastier parametrization that is constantly used in CAD systems is the sphere, with its two irregular points at the poles. As a first example, we have generated a CAD model that consist in a unit cube B containing 5000 spheres with (pseudo-)random centers and radii S_i , $i = 1, \dots, 5000$. The final CAD model C is computed as

$$C = B \setminus S_1 \setminus S_2 \cdots \setminus S_{5000}.$$

The final model C is depicted in Fig. 14. It consist in 3 volumes, 4971 surfaces (mainly trimmed spheres resulting from the boolean operations) and 18112 curves. Gmsh's actual script that was used to generate that model is given by

```
// Gmsh's script to generate
// a CAD model with 5000 spheres
SetFactory("OpenCASCADE");

DefineConstant[
  rmin = {0.002, Name "Min radius"}
  rmax = {0.03, Name "Max radius"}
  n = {5000, Name "Number of spheres"}
];

For i In {1:n}
  r = rmin + Rand(rmax - rmin);
  x = -0.5 + Rand(1);
  y = -0.5 + Rand(1);
  z = -0.5 + Rand(1);
  Sphere(i) = {x, y, z, r};
EndFor

Block(n + 1) = {-0.5, -0.5, -0.5, 1, 1, 1};

BooleanDifference{ Volume{n + 1}; Delete; }
               { Volume{1:n}; Delete; }
```

Some of the surfaces of the 5000 spheres model are really complex to mesh, especially when a trimmed curve is very close to an irregular point. Figure 13 shows a complicated situation.

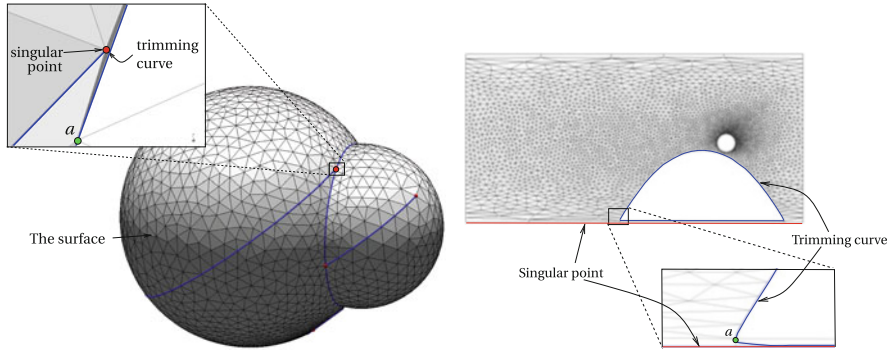


Fig. 13 One surface of the 5000 sphere model that exhibit a complex configuration close to one of the poles of the sphere

7.2 Many Ellipsoids

Generating an ellipsoid can be done by applying an affine transformation to a sphere followed by a rotation. Using Gmsh's built-in scripting language, this can be achieved as follows:

```
R = 2;
Sphere(1) = {0, 0, 0, R};
Affine{ 1,0,0,0, 0,10,0,0, 0,0,1,0 }
{ Volume{i}; }
Rotate {{Sqrt(2), Sqrt(2), 0},
{0, 0, 0}, Pi/3} Volume{i};}
```

Ellipsoids have parametrizations that are even more distorted than spheres. In the following example, 450 ellipsoids E_i , $i = 1, \dots, 450$ have been inserted into a unit cube, with random orientations and random sizes. The final CAD model is again built as the unit cube “minus” all ellipsoids. In OpenCASCADE, ellipsoids are encoded as B-spline surfaces and their intersections takes way more effort than intersecting spheres: it actually took about 7 min to generate the CAD model while only 3 min were required to generate the surface mesh (295 surfaces for a total of 220,523 triangles) and only 14 s were required to generate the 3D mesh (55 volumes and 735 million tetrahedra). Figure 14 show a picture of the resulting mesh.

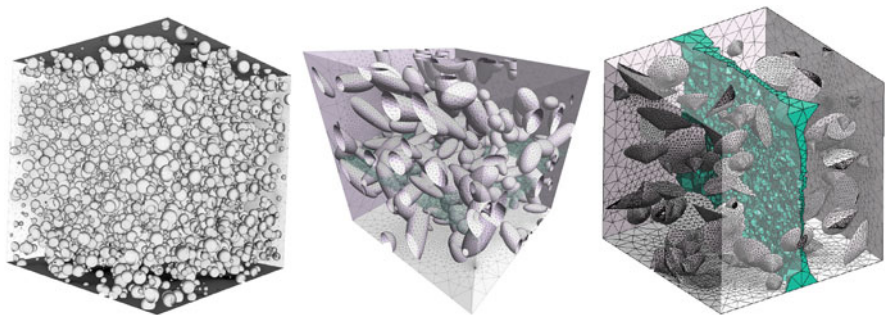


Fig. 14 A complex model made of 5000 spheres (left), another model made of 450 ellipsoids (center) and a third model made of 100 intersecting cones (right)

7.3 *Many Cones*

Apex of cones are singular points of their parametrizations. We have generated a geometry with 100 intersecting cones in a box. Figure 14 show an image of the mesh.

Conclusions

Generating in a reliable manner a quality surface mesh for arbitrary CAD models entails dealing with various CAD systems idiosyncrasies. In this paper, we have presented a crucial modification of Gmsh's surface meshing algorithms that is an important step forward towards this goal, by handling surfaces with a finite number of irregular points. The two test cases that are presented are only indicative: hundred of other examples were successfully tested during the writing of this paper, all with surfaces that have singularities.

The lack of a structure of proof for surface meshing that is briefly explained in the introduction is one of the curses mesh generation people have to live with. Surface meshers that are reasonably reliable are all based on heuristics and their disfunctions and bugs can only be found through extensive testing. For example, the issue that has been explained in Sect. 4 has only been encountered twice in all our test cases. Yet, it has to be addressed because the rare conditions of apparition of the bug will definitively happen at some point in a software like Gmsh that is used by a large community.

In conclusion, we are aware that other issues will show up in the long term (maybe impossible) goal of 100% reliability. Yet, the improvements that are presented in this paper definitively make Gmsh's surface meshers more reliable on a large number of test cases that were failing in previous versions. The method that is proposed does not require deep modifications of existing surface meshing algorithms. Yet, it allows to produce quality meshes for all test cases that we encountered.

References

1. Weiler, K.: Geometric Modeling for CAD Applications. Springer, Berlin (1988)
2. Geuzaine, C., Remacle, J.F.: *Int. J. Numer. Methods Eng.* **79**(11), 1309 (2009)
3. Borouchaki, H., Laug, P., George, P.L.: *Int. J. Numer. Methods Eng.* **49**(1–2), 233 (2000)
4. Hartmann, E.: *Vis. Comput.* **14**(3), 95 (1998)
5. Owen, S.J., Staten, M.L., Canann, S.A., Saigal, S.: *Int. J. Numer. Methods Eng.* **44**(9), 1317 (1999)
6. Maréchal, L.: In: Proceedings of the 18th international meshing roundtable, pp. 65–84. Springer, Berlin (2009)
7. Shephard, M.S., Georges, M.K.: *Int. J. Numer. Methods Eng.* **32**(4), 709 (1991)
8. Frey, P.: Yams a fully automatic adaptive isotropic surface remeshing procedure. Ph.D. thesis, Inria (2001)
9. Béchet, E., Cuillière, J.C., Trochu, F.: *Comput. Aided Des.* **34**(1), 1 (2002)
10. Yan, D.M., Lévy, B., Liu, Y., Sun, F., Wang, W.: In: Computer Graphics Forum, vol. 28, pp. 1445–1454. Wiley Online Library, New York (2009)
11. Blacker, T.D., Stephenson, M.B.: *Int. J. Numer. Methods Eng.* **32**(4), 811 (1991)
12. Borouchaki, H., Frey, P.: *Comput. Methods Appl. Mech. Eng.* **194**(48–49), 4864 (2005)
13. Do Carmo, M.P.: *Differential Geometry of Curves and Surfaces: Revised and Updated Second Edition*. Courier Dover Publications, New York (2016)
14. Remacle, J.F., Li, X., Chevaugéon, N., Shephard, M.S.: In: IMR, pp. 261–272 (2002)
15. Li, X., Shephard, M.S., Beall, M.W.: *Comput. Methods Appl. Mech. Eng.* **194**(48–49), 4915 (2005)
16. Remacle, J.F., Li, X., Shephard, M.S., Flaherty, J.E.: *Int. J. Numer. Methods Eng.* **62**(7), 899 (2005)
17. Knupp, P.M.: *Eng. Comput.* **15**(3), 263 (1999)
18. Rebay, S.: *J. Comput. Phys.* **106**(1), 125 (1993)

Adaptive Single- and Multilevel Stochastic Collocation Methods for Uncertain Gas Transport in Large-Scale Networks



Jens Lang, Pia Domschke, and Elisa Strauch

Abstract In this paper, we are concerned with the quantification of uncertainties that arise from intra-day oscillations in the demand for natural gas transported through large-scale networks. The short-term transient dynamics of the gas flow is modelled by a hierarchy of hyperbolic systems of balance laws based on the isentropic Euler equations. We extend a novel adaptive strategy for solving elliptic PDEs with random data, recently proposed and analysed by Lang, Scheichl, and Silvester [J. Comput. Phys., 419:109692, 2020], to uncertain gas transport problems. Sample-dependent adaptive meshes and a model refinement in the physical space is combined with adaptive anisotropic sparse Smolyak grids in the stochastic space. A single-level approach which balances the discretization errors of the physical and stochastic approximations and a multilevel approach which additionally minimizes the computational costs are considered. Two examples taken from a public gas library demonstrate the reliability of the error control of expectations calculated from random quantities of interest, and the further use of stochastic interpolants to, e.g., approximate probability density functions of minimum and maximum pressure values at the exits of the network.

1 Introduction

The role of natural gas transport through large-scale networks has been rapidly increased through the ongoing replacement of traditional energy production by coal fired and nuclear plants with gas consuming facilities. The safekeeping of energy security and the development of clean energy to meet environmental demands have

J. Lang (✉) · E. Strauch

Department of Mathematics, Technical University of Darmstadt, Darmstadt, Germany
e-mail: lang@mathematik.tu-darmstadt.de; strauch@mathematik.tu-darmstadt.de

P. Domschke

Frankfurt School of Finance and Management, Finance Department, Frankfurt am Main, Germany

e-mail: p.domschke@fs.de

generated a significant increase in gas consumption for electric power stations in the last decade. The future energy mix will mainly be based on low-carbon and regenerative energy and natural gas is considered as a bridging combustible resource to achieve this goal. The seasonally fluctuating disposability of wind and solar resources causes a growing variability in electricity production and hence also in the demands of gas transportation by pipelines. The resulting intra-day uncertain oscillations in demand for natural gas leads to new challenges for computer based modelling and control of gas pipeline operations. Here, an increasing focus lies on the short-term transient dynamics of gas flow. Operators have to responsively control varying loads to realize a reliable operational management for both gas and electricity delivery systems. These challenging new conditions demand advanced decision tools based on reliable transient simulation and uncertainty quantification taking into account serious operating restrictions.

Although significant progress has been made to tackle these challenging problems in recent years, they still remain unsolved for real-life applications with sudden changes in gas supply and demand. In such cases, the gas flow is characterized by multiple scales in time and space across the whole network, which limits the use of the full compressible Euler equations of fluid dynamics. However, there exists a hierarchy of models with decreasing fidelity that allows to predict the system behaviour with varying levels of accuracy. In order to make real-time decisions, an appropriate trade-off between accuracy and computational complexity is mandatory and places the demand for an adaptive strategy to automatically steer the simulation by changing the models and the discretization meshes.

In this paper, we propose a novel computational approach for the reliable quantification of the transport of uncertainties through a network of gas pipelines. It extends an adaptive multilevel stochastic collocation method recently developed in [22] for elliptic partial differential equations with random data to systems of hyperbolic balance laws with uncertain initial and boundary conditions. We have been developing in-house software tools for fast and reliable transient simulation and continuous optimization of large-scale gas networks over the last decade [7–11]. Exemplarily, here we will investigate the important task of safely driving a stationary running system into a newly desired system defined by uncertain gas nominations at delivery points of the network. To be usable in a real-time application of risk analysis and reliability assessment of gas delivery, we have designed our method to meet user-defined accuracies while keeping the computing time for large-scale gas networks at a moderate level. It offers also the opportunity to be integrated in a probabilistic constrained optimization approach [30].

We will consider the following one-dimensional parameterized hyperbolic system of balance laws on a set of gas pipes Ω_j , $j = 1, \dots, M$, with random initial and boundary data:

$$\partial_t U^{(j)}(x, t, y) + \partial_x F_{m_j}(U^{(j)}(x, t, y)) = G_{m_j}(x, t, U^{(j)}(x, t, y)), \quad (1)$$

$$U^{(j)}(x, 0, y) = U_0^{(j)}(x, y), \quad (2)$$

$$B(U^{(j)}(x_b, y)) = H(x_b, t, y), \quad b \in \mathcal{B}, \quad (3)$$

$$\Phi(U^{(1)}(x_i, t, y), \dots, U^{(M)}(x_i, t, y)) = \Pi(x_i, t), \quad i \in \mathcal{C}, \quad (4)$$

where the solutions are represented as $U^{(j)}(x, t, y) : D^{(j)} \times \Gamma \rightarrow \mathbb{R}^2$ with the deterministic physical domain $D^{(j)} := \Omega_j \times \mathbb{R}^+$ and $\Gamma = \Gamma_1 \times \Gamma_2 \times \dots \times \Gamma_N$ being a stochastic parameter space of finite dimension N (finite noise assumption). The component parameters $y_1, \dots, y_N$ will be associated with independent random variables that have a joint probability density function $\hat{d}(y) = \prod_{n=1}^N \hat{d}_n(y_n) \in L^\infty(\Gamma)$ such that $\hat{d}_n : [-1, 1] \rightarrow \mathbb{R}$. Typically, gas pipeline systems are buried underground and hence temperature differences between a pipe segment and the ground can be neglected in practice. It is therefore standard to consider an isothermal process without a conservation law for the energy, i.e., $U^{(j)}$ is the vector of density ρ and momentum ρv for each pipe with v being the velocity.

The index sets $\mathcal{B}$ and $\mathcal{C}$ in (3) and (4) describe the indices of the boundary and the coupling nodes, respectively. Boundaries in gas networks are sources, where gas is injected into the pipeline system, and exits, where it is taken out by consumers. The modelling of connected pipes, flow at junctions, and the pressure increase caused by a compressor leads to certain coupling conditions in (4) at inner nodes. We ensure conservation of mass and claim the equality of pressure, except for compressors, where the time-dependent term $\Pi(\cdot, t)$ represents the pressure jump that is realised by the compression process. The pressure is calculated from the equation of state for real gases, $p = \rho z(p)RT$, with compressibility factor $z(p) \in (0, 1)$. An exemplary gas network is described in Fig. 1.

We also allow for different gas transport models in each pipe. They are identified by the parameters $m_j \in \mathcal{M} := \{\mathcal{M}_1, \mathcal{M}_2, \mathcal{M}_3\}$ in (1) representing a whole hierarchy of models with decreasing fidelity. In our applications, we use the nonlinear isothermal Euler equations as $\mathcal{M}_1$, its semilinear approximation as $\mathcal{M}_2$, and a quasi-stationary model as $\mathcal{M}_3$. They will be described in more detail later on.

Let $U = (U^{(1)}, \dots, U^{(M)})$ and $X = C([0, T]; L^1(\Omega_1)) \times \dots \times C([0, T]; L^1(\Omega_M))$. Throughout this paper, we assume that there is a unique weak entropy solution $U(\cdot, \cdot, y) \in X$ of the gas flow problem (1)–(4) for all $y \in \Gamma$. For uncertainty quantification in gas network applications, it is more natural to consider a functional $\psi(U)$ of the solution U instead of the solution itself. Thus, suppose a possibly nonlinear functional (or quantity of interest) $\psi : X \rightarrow \mathbb{R}$ with $\psi(0) = 0$ is given. The standard collocation method is based on a set of deterministic sample points $\{y^{(q)}\}_{q=1, \dots, Q}$ in Γ , chosen to compute independent, finite dimensional space-time approximations $U_h(y^{(q)}) \approx U(y^{(q)})$. These approximations are used to construct a single-level interpolant

$$\Psi_{Q,h}^{(SL)}(y) = \mathcal{I}_Q[\psi(U_h)](y) = \sum_{q=1}^Q \psi_q \phi_q(y) \quad (5)$$

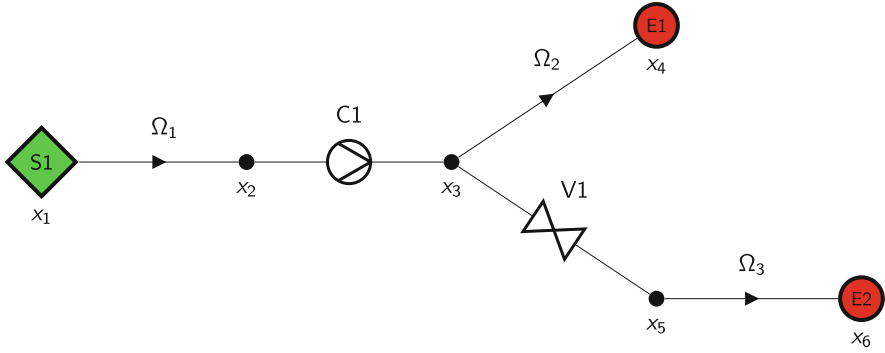


Fig. 1 Exemplary gas network with three pipes $\Omega_1 = (x_1, x_2)$, $\Omega_2 = (x_3, x_4)$, $\Omega_3 = (x_5, x_6)$, one source S1 at node x_1 , two exits E1 at node x_4 and E2 at node x_6 , one compressor C1, and one valve V1. Thus, in system (1)–(4), we have $M = 3$, boundary conditions with $\mathcal{B} = \{1, 4, 6\}$ and coupling conditions with $\mathcal{C} = \{2, 3, 5\}$. Compressors and valves are usually modelled as edges of length zero. The valve V1 can be open or closed, uniquely determining the flow variables on both sides as input for the inner nodes x_3 and x_5 . When compressor C1 is not active, then the flow variables are unchanged. Otherwise, the pressure is increased and taken as input for the inner node x_3 . We ensure conservation of mass and the equality of pressure at inner nodes, see also the discussion before (9). A typical scenario for uncertainty quantification could be to study the impact of uncertain withdrawal of gas at exits E1 and E2 modelled by two random parameters y_1 and y_2 on the work load of the compressor and the operation of the valve. In this case, we have $N = 2$ for the dimension of the stochastic parameter space

for the function $\psi(U)$ in the polynomial space with basis functions ϕ_q ,

$$\mathcal{P}_Q = \text{span}\{\phi_q\}_{q=1,\dots,Q} \subset L^2_d(\Gamma) := \{\phi : \Gamma \rightarrow \mathbb{R} \text{ s.t. } \int_{\Gamma} \phi^2(y) \hat{d}(y) dy < \infty\}. \quad (6)$$

The interpolation conditions $\mathcal{I}[\psi(U_h)](y^{(q)}) = \psi(U_h(y^{(q)}))$ for $q = 1, \dots, Q$ determine the coefficients ψ_q . The quality of the interpolation process depends on the accuracy of the space-time approximations $U_h(y^{(q)})$, the regularity of the solution with respect to the stochastic parameters y , and on the number of collocation points Q , which grows rapidly with increasing stochastic dimension N . The interpolant $\Psi_{Q,h}^{(SL)}(y)$, also called *response surface approximation*, can be used to directly calculate moments such as expectation and variance. Since its evaluation is extremely cheap, it also forms the basis for approximating its probability density function by a kernel density estimator and determining the practically relevant probability that $\Psi_{Q,h}^{(SL)}(y)$ lies in a certain interval over the whole time horizon. We will apply this approach to check the validity that gas is delivered in a pressure range stipulated in a contract between gas company and consumer.

The uncertainties in the initial and boundary data in (2) and (3) result in a propagation of uncertainties in the functional $\psi(U)$. It is essential in nowadays natural

gas transport through large networks that operators apply a reliable operational management to guarantee a *sufficiently smooth* gas flow, respecting at the same time operating limits of compressors and pressure constraints inside the pipes in a safe manner. There is always a safety factor that prevents the whole transport system to really hit the limits. Therefore, we may assume appropriate regularity of the solution in the random space in order to ensure a fast convergence of the global approximation polynomials $\phi_q(y)$ in (5). Exemplarily, we will investigate the influence of uncertain gas demand when safely driving a stationary running system into a newly desired system defined by shifted gas nominations at the delivery points of the network.

There are two main alternative approaches to stochastic collocation: Monte Carlo sampling and stochastic Galerkin methods. A detailed discussion of comparative advantages and disadvantages in the context of hyperbolic systems of conservation laws is given in [2], see also [5, 14] for a general overview on uncertainty quantification in solutions of more general partial differential equations. Monte Carlo methods and its variants are the most commonly used sampling methods. They are non-intrusive and robust with respect to lack of regularity, have a dimension-independent convergence rate and offer a trivial parallelization. However, they are not able to exploit any smoothness or special structure in the parameter dependence and their convergence rate is rather low even when Multilevel Monte Carlo methods are applied. Combined with finite volume discretizations for the physical space, such methods are extensively investigated in [25–27]. Stochastic Galerkin methods based on generalized polynomial chaos are intrusive and request the solution of heavily extended systems of conservation laws [28, 33]. Although sparse grids and efficient solvers for block-structured linear systems are used, the computational costs in general are formidable. Recently, an intrusive polynomial moment method which is competitive with non-intrusive collocation methods has been proposed in [21]. In the presence of discontinuities in the random space, promising semi-intrusive approaches are provided by the stochastic finite volume method [1] and a novel hierarchical basis weighted essentially non-oscillatory interpolation method [19].

The paper is organised as follows. In Sect. 2, we describe the single-level approach and especially focus on the main ingredients for the adaptive solvers in the physical and parameter space. The extension to the multilevel approach is explained in Sect. 3, where we also give asymptotic rates for the complexity of the algorithm. In Sect. 4, two examples based on networks from a public gas library are investigated to demonstrate the efficiency and potential of the fully adaptive collocation method. We conclude with a summary and outlook in Sect. 5.

2 Adaptive Single-Level Approach

The main advantage of sampling methods is the reuse of an efficient solver for the transient gas flow through a network in the range of parameters defined by the stochastic space Γ . Since the gas transport through a complex network may be

very dynamic and thus changes both in space and time, an automatic control of the accuracy of the simulation is mandatory. In order to further reduce computational costs, adjusting the transport model in each pipe according to the time-dependent dynamics has proven to be very attractive. As a rule of thumb, the most complex nonlinear Euler equations ($\mathcal{M}_1$) should be used when needed and the simplest algebraic model ($\mathcal{M}_3$) should be taken whenever possible without losing too much accuracy. In a series of papers, we have developed a posteriori error estimates and an overall control strategy to reduce model and discretization errors up to a user-given tolerance [7–11]. A brief introduction will be given next.

Let a parameter $y \in \Gamma$ be fixed and an initial distribution of gas transport models $\{m_1, \dots, m_M\}$ be given. Then, we solve the gas network equations (1)–(4) by means of an adaptive implicit finite volume discretization [20] applied for each pipe until the estimate of the error in the functional $\psi(U_h(y))$ is less than a prescribed tolerance $\eta_h > 0$. Here, h refers to resolution in space, time and model hierarchy. To raise efficiency, the simulation time $[0, T]$ is divided into subintervals $[t_i, t_{i+1}]$, $i = 0, \dots, N_t - 1$, of the same size. We then successively process the classical adaption loop

$$\text{SOLVE} \Rightarrow \text{ESTIMATE} \Rightarrow \text{MARK} \Rightarrow \text{REFINE} \Rightarrow \text{SOLVE} \quad (7)$$

for each of the subintervals such that eventually

$$|\psi(U(y)) - \psi(U_h(y))| \leq c_h(y) \left| \sum_{j=1}^M (\eta_{x,j} + \eta_{t,j} + \eta_{m,j}) \right| < c_h(y) \cdot \eta_h \quad (8)$$

in the second step with a sample-dependent constant $c_h(y)$ that is usually close to one. The a posteriori error estimators $\eta_{x,j}$, $\eta_{t,j}$, and $\eta_{m,j}$ for the j -th pipe determine the error distribution along the network for the spatial, temporal and model discretizations. They measure the influence of the model and the discretization on the output functional ψ and can be calculated by using the solutions of adjoint equations. A detailed description which would go beyond the scope of our paper is given in [7, Sect. 2.2], see also [9, 10]. Polynomial reconstructions in space and time of appropriate orders are used to compute $\eta_{x,j}$ and $\eta_{t,j}$, respectively. The model error estimator $\eta_{m,j}$ is derived from the product of differential terms, representing the difference between models, and the sensitivities calculated from the adjoint equations.

In our calculations, we use the following model hierarchy:

- $\mathcal{M}_1$: Nonlinear isothermal Euler equations

$$\begin{aligned} \partial_t \rho + \partial_x(\rho v) &= 0, \\ \partial_t(\rho v) + \partial_x(p + \rho v^2) &= g(\rho, \rho v), \end{aligned}$$

- $\mathcal{M}_2$: Semilinear isothermal Euler equations

$$\begin{aligned}\partial_t \rho + \partial_x(\rho v) &= 0, \\ \partial_t(\rho v) + \partial_x p &= g(\rho, \rho v),\end{aligned}$$

- $\mathcal{M}_3$: Algebraic isothermal Euler equations

$$\begin{aligned}\partial_x(\rho v) &= 0 \\ \partial_x p &= g(\rho, \rho v)\end{aligned}$$

with the joint source term $g(\rho, \rho v) = -\lambda \rho v |v| / (2D)$, where D is the pipe diameter and λ the Darcy friction coefficient. We note that the algebraic model can be analytically solved in the variables ρv and p . The models are connected at inner nodes, where we ensure conservation of mass and equality of pressure. The latter one is often used in engineering software, but can be also replaced by the equality of total enthalpy. The interested reader is referred to the discussion in [24]. Pipes can also be connected by valves and compressors. Valves are used to regulate the flow in gas networks. A valve is modelled as edge of length zero, whereas conservation of mass and equality of pressure hold for an open valve and $q := \rho v = 0$ is required at both sides of a closed valve. Compressors compensate for the pressure loss due to friction in the pipes. The power of a compressor $c \in \mathcal{J}$ that is needed for the compression process is given by

$$G_c(U(t)) = c_F q_{in}(t) z(p_{in}(t)) \left(\left(\frac{p_{out}(t)}{p_{in}(t)} \right)^{\frac{\gamma-1}{\gamma}} - 1 \right) \quad (9)$$

with in- and outgoing pressure p_{in} , p_{out} , and ingoing flow rate q_{in} [23]. The parameter c_F is a compressor specific constant, γ the isentropic coefficient of the gas, and $z \in (0, 1)$ the compressibility factor from the equation of state for real gases. In our application, we use the specific energy consumption needed by the electric motors to realize all desired compressions as quantity of interest that drives the adaptation process. It can be estimated by a quadratic polynomial in G_c , i.e., we set

$$\psi(U(y)) = \alpha \sum_{c \in \mathcal{J}} \int_0^T g_{c,0} + g_{c,1} G_c(U(y)) + g_{c,2} G_c^2(U(y)) dt \quad (10)$$

with given compressor-dependent constants $g_{c,i} \in \mathbb{R}$ and a scaling factor $\alpha > 0$.

The complex task in the step MARK (for refinement) of finding an optimal refinement strategy that combines the three types of adaptivity is a generalisation of the unbounded knapsack problem, which is NP-hard. A good approximation can be found by a greedy-like refinement strategy as investigated in [7]. It leads

to considerable computational savings without compromising on the simulation accuracy. Eventually, we have an adaptive black box solver ANet—our working horse—at hand that, once a random parameter $y \in \Gamma$ and a specific tolerance η_h have been chosen, delivers a numerical approximation $U_h(y)$ such that the accuracy requirement (8) is satisfied for

$$\psi(U_h(y)) = \text{ANet}(y, \eta_h). \quad (11)$$

Working close to the asymptotic regime, we can assume that the adaptive algorithm converges for fixed $y \in \Gamma$ and $\eta_h \rightarrow 0$.

Starting from the pointwise error estimate (8) and supposing bounded first moments of $c_h(y)$, we directly get the following error bound:

$$|\mathbb{E}[\psi(U(y)) - \psi(U_h(y))]| := \left| \int_{\Gamma} (\psi(U(y)) - \psi(U_h(y))) \hat{d}(y) dy \right| \leq C_h \cdot \eta_h \quad (12)$$

with a constant

$$C_h := \int_{\Gamma} c_h(y) \hat{d}(y) dy \quad (13)$$

that does not depend on y .

We will now discuss the control of the error for the adaptive stochastic collocation method. Let us assume $\psi(U) \in C^0(\Gamma, \mathbb{R})$ and consider the interpolation operator $\mathcal{I}_Q : C^0(\Gamma) \rightarrow L^2_{\hat{d}}(\Gamma)$ from (5). This operator is constructed by a hierarchical sequence of one-dimensional Lagrange interpolation operators, using the anisotropic Smolyak algorithm as introduced in [13]. It reads

$$\begin{aligned} \mathcal{I}_Q[\psi(U_h)](y) &= \sum_{\mathbf{i} \in I} \Delta^{m(\mathbf{i})}[\psi(U_h)](y) \\ &:= \sum_{\mathbf{i} \in I} \bigotimes_{n=1}^N \left(\mathcal{I}_n^{m(i_n)}[\psi(U_h)](y) - \mathcal{I}_n^{m(i_n)-1}[\psi(U_h)](y) \right) \end{aligned} \quad (14)$$

with multi-indices $\mathbf{i} = (i_1, \dots, i_N) \in I \subset \mathbb{N}_+^N$, $m(\mathbf{i}) = (m(i_1), \dots, m(i_N))$, and univariate polynomial interpolation operators $\mathcal{I}_n^{m(i_n)} : C^0(\Gamma_n) \rightarrow \mathbb{P}_{m(i_n)-1}$. These operators use $m(i_n)$ collocation points to construct a polynomial interpolant in $y_n \in \Gamma_n$ of degree at most $m(i_n) - 1$. The operators $\Delta^{m(\mathbf{i})}$ are often referred to as hierarchical surplus operators. The function m has to satisfy $m(0) = 0$, $m(1) = 1$, and $m(i) < m(i+1)$. We formally set $\mathcal{I}_n^0 = 0$ for all $n = 1, \dots, N$ and use the nested sequence of univariate Clenshaw–Curtis nodes with $m(i) = 2^{i-1} + 1$ if $i > 1$. The index Q in (14) is then the number of all explored quadrature points in Γ determined by $m(\mathbf{i})$.

The value of the hierarchical surplus operator $\Delta^{m(\mathbf{i})}$ in (14) can be interpreted as profit and therefore used as error indicator for already computed approximations. Applying once again the classical adaption loop from (7), the adaptive anisotropic Smolyak algorithm computes profits in each step, adds the index of the highest profit to the index set $m(\mathbf{i})$ and explores admissible neighbouring indices next. The algorithm stops if the absolute value of the highest profit is less than a prescribed tolerance, say $\eta_s > 0$. Obviously, the method is dimension adaptive. There is a MATLAB implementation *Sparse Grid Kit* available, which can be downloaded from the CSQI website [4]. Its numerical performance is discussed in the review paper [31].

Following this adaptive methodology, we get an error estimate

$$|\mathbb{E}[\psi(U_h(y)) - \mathcal{I}_Q[\psi(U_h)](y)]| \leq C_s \cdot \eta_s \quad (15)$$

with a constant $C_s > 0$. We assume that C_s does not depend on h . If we now split the overall error into the sum of a physical error resulting from the chosen resolution in space, time and model hierarchy, and a stochastic interpolation error, then using the inequalities (12), (15) and the triangle inequality yields the final estimate

$$\begin{aligned} & |\mathbb{E}[\psi(U(y)) - \mathcal{I}_Q[\psi(U_h)](y)]| \\ & \leq |\mathbb{E}[\psi(U(y)) - \psi(U_h(y))]| + |\mathbb{E}[\psi(U_h(y)) - \mathcal{I}_Q[\psi(U_h)](y)]| \\ & \leq C_h \cdot \eta_h + C_s \cdot \eta_s. \end{aligned} \quad (16)$$

Let $\varepsilon > 0$ be a user-prescribed tolerance for the error on the left-hand side. Then the usual strategy to balance both the physical and the stochastic error on the right-hand side is to choose the individual tolerances as $\eta_h = \varepsilon/(2C_h)$ and $\eta_s = \varepsilon/(2C_s)$. Finally, the adaptive Smolyak algorithm is called with the tolerance η_s , where for each chosen sample point $y \in \Gamma$, the black box solver in (11) runs with $\text{ANet}(y, \eta_h)$, resulting in a sample-adaptive resolution in the physical space. The algorithm is illustrated in Table 1.

Table 1 Algorithm to approximate solution functionals $\psi(U)$ by an adaptive single-level stochastic collocation method

Algorithm: Adaptive single-level stochastic collocation method	
1.	Given ε , estimate C_h, C_s , and set $\eta_h := \varepsilon/(2C_h), \eta_s := \varepsilon/(2C_s)$.
2.	Compute $\mathbb{E}[\mathcal{I}_Q[\psi(U_h)]] := \text{ASmol}(\text{ANet}(y, \eta_h), \eta_s)$.

3 Adaptive Multilevel Approach

Next, we consider an adaptive multilevel approach in order to enhance the efficiency of the uncertainty quantification further. First multilevel strategies in the context of Monte Carlo methods were independently proposed as an abstract variance reduction technique in [15, 17]. Extensions to uncertainty quantification were developed in [3, 6]. Later on, they also entered the field of stochastic collocation methods [22, 32, 34]. The methodology in this paper can be viewed as an extension of the adaptive multilevel stochastic collocation method developed for elliptic PDEs with random data in [22] to the hyperbolic case, where a sample-dependent hierarchy of spatial approximations is replaced by a more sophisticated space-time-model hierarchy.

Let a sequence $\{\eta_{h_k}\}_{k=0,\dots,K}$ of tolerances with

$$1 \geq \eta_{h_0} > \eta_{h_1} > \dots > \eta_{h_K} > 0 \quad (17)$$

be given. Each h_k refers to a certain resolution in space, time and model hierarchy such that for any solution $U_{h_k}(y)$ with $y \in \Gamma$ it holds

$$|\mathbb{E}[\psi(U(y)) - \psi(U_{h_k}(y))]| \leq C_H \cdot \eta_{h_k}, \quad k = 0, \dots, K, \quad (18)$$

with a constant $C_H := \max_{k=0,\dots,K} C_{h_k}$ that does not depend on y . The constants C_{h_k} are defined in (13) with $h = h_k$. We consider now a second family of (stochastic) tolerances $\{\eta_{s_k}\}_{k=0,\dots,K}$ and assume that there exists numbers Q_k , $k = 0, 1, \dots, K$ and a positive constant C_Y not depending on k such that

$$|\mathbb{E}[(\psi(U_{h_k}) - \psi(U_{h_{k-1}})) - \mathcal{I}_{Q_{K-k}}[\psi(U_{h_k}) - \psi(U_{h_{k-1}})]]| \leq C_Y \cdot \eta_{s_{K-k}} \quad (19)$$

for $k = 0, 1, \dots, K$. Here, we formally set $\psi(U_{h_{-1}}) := 0$. Observe that with increasing index k , the differences $|\psi(U_{h_k})(y) - \psi(U_{h_{k-1}})(y)|$ decrease and hence the number of collocation points Q_{K-k} necessary to achieve the tolerance $\eta_{s_{K-k}}$ gets smaller. Consequently, less samples on fine meshes and with high fidelity models are needed to achieve the overall tolerance, which is the main motivation for the use of a multilevel approach.

Using a telescopic sum of single-level interpolants, we construct a multilevel interpolant for the functional $\psi(U)$ through

$$\begin{aligned} \Psi_K^{(ML)}(y) &:= \sum_{k=0}^K \left(\Psi_{Q_{K-k}, h_k}^{(SL)}(y) - \Psi_{Q_{K-k}, h_{k-1}}^{(SL)}(y) \right) \\ &= \sum_{k=0}^K \mathcal{I}_{Q_{K-k}} [\psi(U_{h_k}) - \psi(U_{h_{k-1}})](y). \end{aligned} \quad (20)$$

Its error can be estimated by

$$\begin{aligned}
 & \left| \mathbb{E}[\psi(U(y)) - \Psi_K^{(ML)}(y)] \right| \\
 & \leq \left| \mathbb{E}[\psi(U(y)) - \psi(U_{h_K}(y))] \right| + \left| \mathbb{E}[\psi(U_{h_K}(y)) - \Psi_K^{(ML)}(y)] \right| \\
 & \leq C_H \cdot \eta_{h_K} + C_Y \cdot \sum_{k=0}^K \eta_{s_{K-k}},
 \end{aligned} \tag{21}$$

where we have used the identity $\psi(U_{h_K}) = \sum_{k=0, \dots, K} (\psi(U_{h_k}) - \psi(U_{h_{k-1}}))$ and the inequalities (18) and (19). There are two different ways to balance the errors on the right-hand side: (1) set $\eta_{s_k} = C_H \cdot \eta_{h_K} / ((K+1)C_Y)$ for all $k = 0, \dots, K$, which yields $2C_H \cdot \eta_{h_K}$ as upper bound in (16), and (2) choose η_{s_k} in such a way that the computational cost is minimized. We will go for the second option and follow the suggestions given in [22]. Let W_k denote the work (computational cost) that must be invested to solve the gas network equations for a sample point $y \in \Gamma$ with accuracy η_{h_k} . Then we make the following assumptions:

$$\begin{aligned}
 (A1) \quad & W_k \leq C_W \cdot \eta_{h_k}^{-s}, \\
 (A2) \quad & C_Y \cdot \eta_{s_{K-k}} = C_I(N) Q_{K-k}^{-\mu} \eta_{h_{k-1}},
 \end{aligned} \tag{22}$$

for all $k = 0, \dots, K$. Here, we fix $\eta_{h_{-1}} := |\mathbb{E}[I_{Q_0}[\psi(U_{h_0})]]|$. The constants $C_W > 0$, $C_I(N) > 0$ are independent of k , y , and the rates s and μ are strictly positive. Recall that N is the dimension of the stochastic space.

To achieve an accuracy $\varepsilon > 0$ for the multilevel interpolant, i.e.,

$$\left| \mathbb{E}[\psi(U(y)) - \Psi_K^{(ML)}(y)] \right| \leq \varepsilon, \tag{23}$$

at minimal cost $C_\varepsilon^{(ML)} := \sum_{k=0, \dots, K} Q_{K-k}(W_k + W_{k-1})$, the optimal choice of the stochastic tolerances is given in [22, Theorem 2.1]. They are

$$\eta_{s_{K-k}} = (2C_Y G_K(\mu))^{-1} (F_k(s))^{\frac{\mu}{\mu+1}} \eta_{h_{k-1}} \varepsilon, \quad k = 0, \dots, K, \tag{24}$$

where

$$\begin{aligned}
 F_0(s) &= \eta_{h_0}^{-s} \eta_{h_{-1}}^{-1}, \\
 F_k(s) &= \left(\eta_{h_k}^{-s} + \eta_{h_{k-1}}^{-s} \right) \eta_{h_{k-1}}^{-1}, \quad k = 1, \dots, K, \\
 G_K(\mu) &= \sum_{k=0}^K (F_k(s))^{\frac{\mu}{\mu+1}} \eta_{h_{k-1}}.
 \end{aligned} \tag{25}$$

Table 2 Algorithm to approximate solution functionals $\psi(U)$ by an adaptive multilevel stochastic collocation method

Algorithm: Adaptive multilevel stochastic collocation method	
1.	Given ε , q , and K , estimate C_H , C_Y , s , μ and set $\eta_{h_K} := \varepsilon / (2C_H)$.
2.	Set $\eta_{h_k} := q^{k-K} \eta_{h_K}$ for $k = 0, \dots, K-1$.
3.	Compute $\eta_{h_{-1}} := \mathbb{E}[\mathcal{I}_{Q_0}[\psi(U_{h_0})]] := \text{ASmol}(\text{ANet}(y, \eta_{h_0}), \eta_{h_0})$.
4.	Set $\eta_{s_{K-k}} := (2C_Y G_K(\mu))^{-1} (F_k(s))^{\frac{\mu}{\mu+1}} \eta_{h_{k-1}} \varepsilon$ for $k = 0, \dots, K$.
5.	Compute $E_0 := \text{ASmol}(\text{ANet}(y, \eta_{h_0}), \eta_{s_K})$.
6.	Compute $E_k := \text{ASmol}(\text{ANet}(y, \eta_{h_k}) - \text{ANet}(y, \eta_{h_{k-1}}), \eta_{s_{K-k}})$ for $k = 1, \dots, K$.
7.	Compute $\mathbb{E}[\Psi_K^{(ML)}] := \sum_{k=0, \dots, K} E_k$.

Typically, in practical calculations, a decreasing sequence of tolerances $\eta_{h_k} = q^k \eta_{h_0}$ with a positive reduction factor $q < 1$ is used. In this case, we can estimate the overall multilevel costs using a standard construction [22, Theorem 2.2],

$$C_\varepsilon^{(ML)} \lesssim \begin{cases} \varepsilon^{-\frac{1}{\mu}} & \text{if } s\mu < 1 \\ \varepsilon^{-\frac{1}{\mu}} |\log \varepsilon|^{1+\frac{1}{\mu}} & \text{if } s\mu = 1 \\ \varepsilon^{-s} & \text{if } s\mu > 1. \end{cases} \quad (26)$$

For the convenience of the reader, we summarize the multilevel algorithm in Table 2. Once the parameters are set, the approach is self-adaptive in nature. Observe that already computed samples at level $k-1$ can be reused to compute E_k in step 6. In general, sufficient estimates for the constants C_H , C_Y and the rates μ , s can be derived from the study of a few samples with relatively coarse resolutions. In any case, these samples can be reused later on.

4 Practical Examples from a Gas Library

Exemplarily, we will consider two gas network configurations: *GasLib-11* and *GasLib-40* from the public gas library gaslib.zib.de [29]. They are parts of real gas networks in Germany. We have implemented the adaptive approach described above for the deterministic black box solver $\text{ANet}(y, \eta_h)$ from (11) in our in-house software package ANACONDA. More details of the implementation can be found in [18]. The adaptive stochastic collocation method $\text{ASmol}(\cdot, \eta_s)$ was realized by means of the *Sparse Grid Kit* developed in MATLAB [31]. All calculations have been done with MATLAB version R2020a on a Intel(R) Xeon(R) Gold 6130 CPU running at 2.1 GHz.

A common daily operation of gas networks is the smooth transformation of a stationary state U_A , which has worked well for the given demands so far, into a new stationary state U_B , which realizes a change in the gas demand over a couple

of hours. This scenario is best treated by appropriate optimization tools which determine the operating range of all compressor stations and valves in such a way that, e.g., lower and upper bounds of pressures are satisfied during the whole time-dependent conversion process. In what follows, we will assume that a feasible, optimized control, i.e., the operation modes for compressors (pressure jump) and valves (open or closed), is already known for this so-called nomination change. Then, we will fix these controls and focus on the influence of uncertainties in the consumers' demands around state U_B on the compressor costs and the feasibility of the pressure at which the gas is delivered to the consumers. Typically, corresponding pressure requirements are regulated in contracts.

4.1 An Example with 11 Pipes

The first example is taken from the *GasLib-11*, which consists of 11 pipes, 2 compressors, 1 valve, 3 sources, and 3 exits, see Fig. 2. The stationary initial state U_A and the final state U_B are determined by the boundary conditions and controls given in Table 3. The simulation is started with $U_0 = U_A$. After 4 h, the boundary values and controls are linearly changed to reach the new conditions defined for U_B .

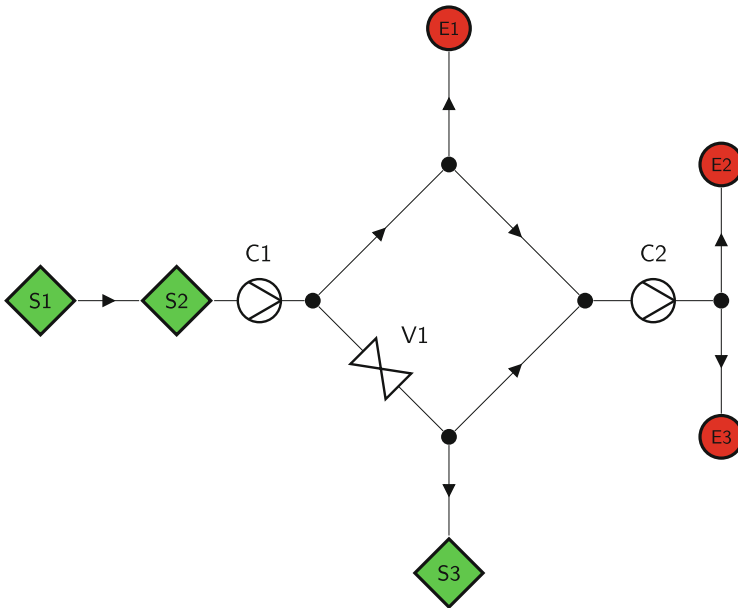


Fig. 2 Schematic description of the network *GasLib-11* with 11 pipes, 2 compressors (C1,C2), 1 valve (V1), 3 sources (green diamonds: S1, S2, S3) and 3 exits (red circles: E1, E2, E3). The arrows determine the orientation of the pipes to identify the flow direction by the sign of the velocity

Table 3 *GasLib-11*: Boundary data for sources (S1–S3), exits (E1–E3), and controls for compressors (C1–C2) and valves (V1) for initial state U_A and final state U_B

Source	State U_A			State U_B		
	S1	S2	S3	S1	S2	S3
Pressure [bar]	70.00	65.00	70.00	48.00	46.00	54.00
Exit	E1	E2	E3	E1	E2	E3
Volume flow [$\text{m}^3 \cdot \text{s}^{-1}$]	38.22	38.22	38.22	25.48	25.48	25.48
Compressor	C1	C2		C1	C2	
Pressure jump [bar]	0	0		5	15	
Valve	V1			V1		
Operation	Open			Closed		

at $t = 6$ h. The valve is closed at $t = 4.5$ h. The simulation time of 24 h is split into subintervals of 4 h, for which the classical adaption loop (7) is processed. For the state U_B , the volume flows q_E at the three exits $E = E1, E2, E3$, are *uncertain* due to an individual behaviour of the consumers and are parameterised by three variables $y = (y_1, y_2, y_3)$, representing the image of a triple of independent random variables with $y_i \in \mathcal{U}[-1, 1]$. We set

$$q_{Ei}(y_i) = 25.48 + 10 \cdot y_i, \quad i = 1, 2, 3. \quad (27)$$

According to (10), the quantity of interest ψ is defined by the specific energy consumption of the compressors,

$$\psi(U(y)) = \alpha \sum_{c=C1, C2} \int_{0h}^{24h} g_{c,0} + g_{c,1} G_c(U(y)) + g_{c,2} G_c^2(U(y)) dt \quad (28)$$

with $g_{c,0} = 5000$, $g_{c,1} = 2.5$, $g_{c,2} = 0$ for both compressors and G_c defined in (9). We set the weighting factor $\alpha = 10^{-10}$ to bring the expected value of $\psi(U(y))$ in the order of 0.1.

In order to start the adaptive stochastic collocation method, we have performed a few calculations for low tolerances to estimate the parameters in Table 2. We found $C_H = C_Y = 0.1$, $s = 1$, and $\mu = 2$ by a least squares fit and appropriate rounding. Let us now consider a single-, two, and three-level approach with a reduction factor $q = 0.5$. The overall accuracy requirements are $\varepsilon = 10^{-6}$, 5×10^{-7} , 2.5×10^{-7} , where a reference solution $\mathbb{E}[\psi(U)] = 0.123765671196008$ is calculated with $\varepsilon = 5 \times 10^{-8}$. The results are shown in Fig. 3. The differences of the methods are not very high. This is due to the fact that only 25 collocation points are sufficient to reach the highest accuracy.

We have also tested an adaptive multilevel Monte Carlo method (see, [15, 17]) on this problem. For the tolerance $\varepsilon = 10^{-4}$ and an average over 10 independent realizations, the three-level algorithm achieves an accuracy of 2.1×10^{-4} in 4070 s. The numbers of samples for each level are $Q_0 = 10223$, $Q_1 = 359$, and $Q_2 = 158$.

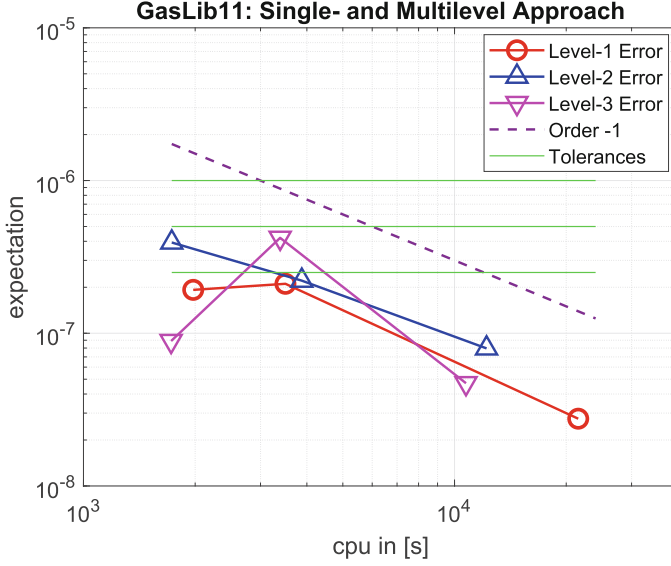


Fig. 3 *GasLib-11*: Errors for the expected values $\mathbb{E}[\psi_K^{(ML)}]$ and $\mathbb{E}[\psi^{(SL)}]$ for the three-level (magenta triangles down, $K = 2$), two-level (blue triangles up, $K = 1$), and one-level (red circles) approach with adaptive space-time-model discretizations for $\varepsilon = 10^{-6}, 5 \times 10^{-7}, 2.5 \times 10^{-7}$ (green lines). The accuracy achieved is almost always better than the tolerance. The single-level and the two-level approach perform quite similar. The three-level approach shows an irregular behaviour, but also delivers very good results

Obviously, the slow Monte Carlo convergence rate of 0.5 is prohibitive for higher tolerances here.

We can also use the anisotropic Smolyak decomposition in (14) to study the validity of the pressure bounds at the three exits $E1$, $E2$, and $E3$. Replacing $\psi(U_h)$ by the time-dependent pressure yields

$$\mathcal{I}_Q[p(U_h)](y) = \sum_{i \in I} \Delta^{m(i)}[p(U_h)](y). \quad (29)$$

Exemplarily, in Fig. 4, we show the pressure curves at the exits for 15 collocation points $y^{(j)} \in \Gamma$ adaptively chosen by the Smolyak algorithm for $\varepsilon = 10^{-5}$. Supposing a feasible range [43 bar, 63 bar] for the pressure p_{exit} at which the gas should be delivered to the consumers, we are now interested in the probabilities

$$\mathbb{P}(p_{min} < 43 \text{ bar}) \quad \text{and} \quad \mathbb{P}(63 \text{ bar} < p_{max}), \quad (30)$$

with $p_{min}(y) = \min_{t \in [0, T]} p_{exit}(t, y)$ and $p_{max}(y) = \max_{t \in [0, T]} p_{exit}(t, y)$. The surrogate model (29) allows a fast evaluation over a sufficiently fine uniform mesh

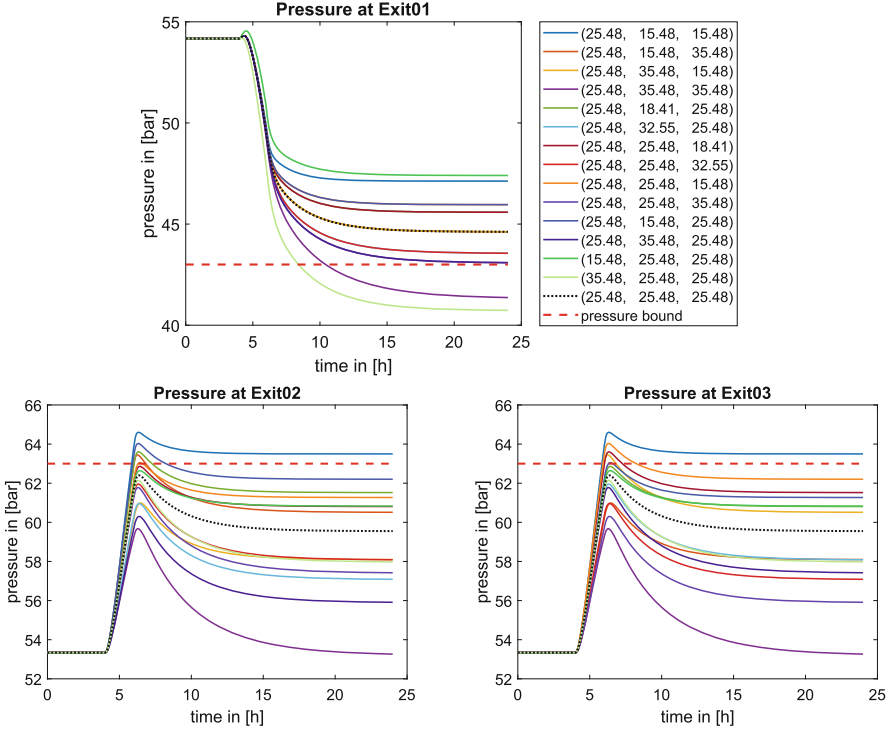


Fig. 4 *GasLib-11*: Pressure evolution at exits E1, E2, E3, for 15 collocation points $y^{(j)} \in \Gamma$ chosen by the adaptive collocation method. The point (25.48, 25.48, 25.48) (dotted black line) corresponds to the original final state U_B with no uncertainties. The predefined pressure bounds $p_{min}^* = 43$ bar and $p_{max}^* = 63$ bar are also plotted (red dotted lines). Obviously, these bounds are violated by a few samples

in the stochastic parameter space $\Gamma \subset \mathbb{R}^3$, thus giving enough information to approximate the probability density functions of the random variables p_{min} and p_{max} by a one-dimensional kernel density estimator (KDS)

$$KDS(x) = \frac{1}{N_s H} \sum_{i=1}^{N_s} \frac{1}{\sqrt{2\pi}} \exp \left(-\frac{1}{2} \left(\frac{x - p(y^{(i)})}{H} \right)^2 \right), \quad (31)$$

for $p = p_{min}, p_{max}$, where $H = 1.06 \sigma_{N_s} / N_s^{0.2}$ and the random vector $y \in \Gamma$ is uniformly sampled with $N_s = 51^3$. Observe that the bandwidth H depends on the standard deviation σ_{N_s} of the samples as, e.g., stated and explained in [16, Chap. 4.2]. The corresponding KDSs are plotted in Fig. 5.

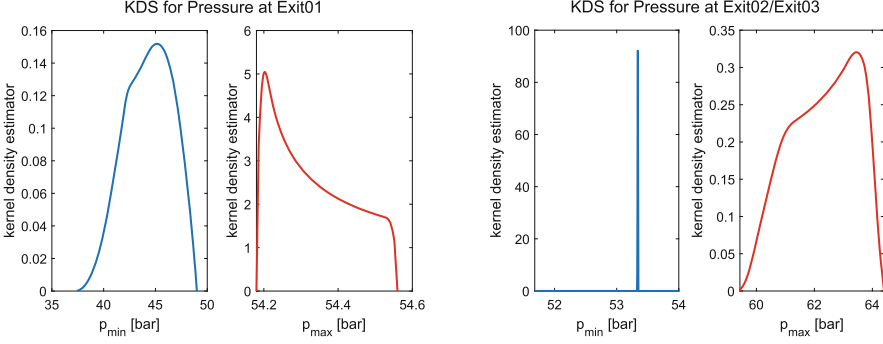


Fig. 5 *GasLib-11*: Kernel density estimators (KDS) as approximation of the probability density functions for the minimum (left) and maximum (right) pressure at $E1$, $E2$ and $E3$. Due to an inherent symmetry, the KDS for $E2$ and $E3$ are equal

From the KDSs, we calculate

$$\begin{aligned} \mathbb{P}(p_{min} < 43 \text{ bar}) &= \begin{cases} 0.30, & \text{for } E1, \\ 0.00, & \text{for } E2, \\ 0.00, & \text{for } E3, \end{cases} \\ \mathbb{P}(63 \text{ bar} < p_{max}) &= \begin{cases} 0.00, & \text{for } E1, \\ 0.33, & \text{for } E2, \\ 0.33, & \text{for } E3. \end{cases} \end{aligned} \quad (32)$$

With such information at hand, a managing operator is prepared to react on sudden changes in the gas network with an appropriate adaptation of the controls. It also forms the basis for probabilistic constrained optimization, see [30] for more details.

4.2 An Example with 40 Pipes

Our second example is *GasLib-40*, a simplified real part of the German Gas Network, and consists of 40 pipes, 6 compressor stations, 3 sources, and 29 exits. Its structure is shown in Fig. 6. The exits will be clustered in 8 different local regions (REs) with equal uptake rates and uncertainties:

$$\begin{aligned} RE1 &= E1, \quad RE2 = E2 - E11, \quad RE3 = E12 - E13, \quad RE4 = E14 - E18, \\ RE5 &= E19 - E20, \quad RE6 = E21 - E24, \quad RE7 = E25 - E26, \quad RE8 = E27 - E29. \end{aligned} \quad (33)$$

The stationary initial state U_A and the final state U_B are determined by the boundary conditions and controls given in Table 4. The temporal evolution of these values is

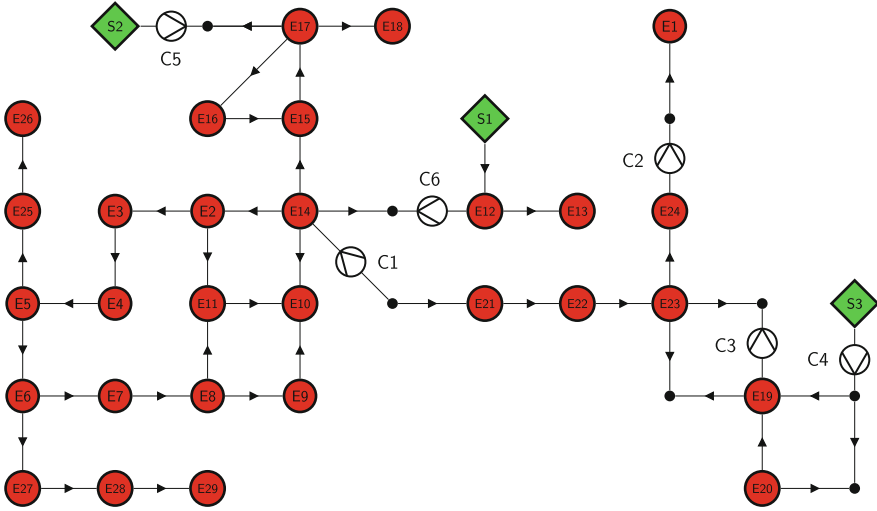


Fig. 6 Schematic description of the network *GasLib-40* with 40 pipes, 6 compressors (C1–C6), 3 sources (green diamonds: S2, S2, S3) and 29 exits (red circles: E1–E29). The arrows determine the orientation of the pipes to identify the flow direction by the sign of the velocity

Table 4 *GasLib-40*: Boundary data for sources (S1–S3), exit regions (RE1–RE8), and controls for compressors (C1–C6) for initial state U_A and final state U_B

	State U_A						State U_B					
Source	S1	S2	S3				S1	S2	S3			
Pressure [bar]	60.0						60.0					
Volume flow [$\text{m}^3 \cdot \text{s}^{-1}$]		53.2	53.2					58.0	53.2			
Exit	RE1	RE2	RE3	RE4	RE5	RE6	RE1	RE2	RE3	RE4	RE5	RE6
Volume flow [$\text{m}^3 \cdot \text{s}^{-1}$]	5.5	5.5	5.5	5.5	5.5	5.5	7.5	8.0	6.5	6.0	7.0	4.0
Exit	RE7	RE8					RE7	RE8				
Volume flow [$\text{m}^3 \cdot \text{s}^{-1}$]	5.5	5.5					8.5	6.0				
Compressor	C1	C2	C3	C4	C5	C6	C1	C2	C3	C4	C5	C6
Pressure jump [bar]	0	0	5	0	0	0	5	15	7	12	5	12

shown in Fig. 7. The computational time interval [0 h, 12 h] is split into 4 equal subintervals.

The quantity of interest $\psi(U)$ is again defined by the specific energy consumption of the compressors,

$$\psi(U(y)) = \alpha \sum_{c=C1, \dots, C6} \int_{0h}^{12h} g_{c,0} + g_{c,1} G_c(U(y)) + g_{c,2} G_c^2(U(y)) dt \quad (34)$$

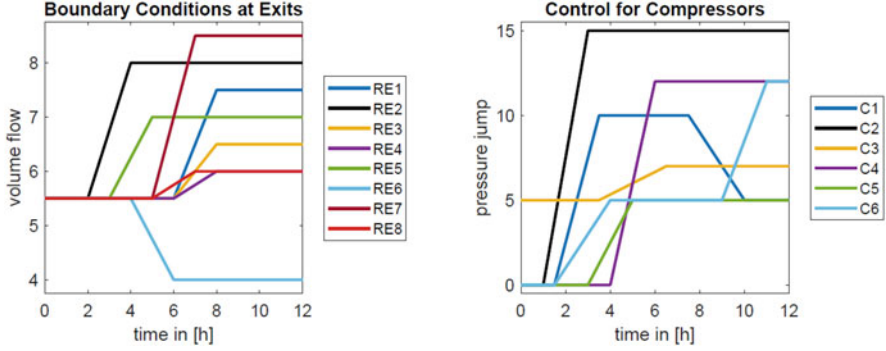


Fig. 7 *GasLib-40*: Time-resolved boundary conditions at the exit regions RE1–RE8 and control for the compressors C1–C6 for a smooth transition from state U_A to state U_B defined in Table 4

Table 5 *GasLib-40*: Errors (ERR) computed from the approximate quantity of interest $\psi = 0.1207377$ for $\eta_h = 10^{-5}$, absolute value of the sum of error estimators (EST) used in (8), minimum and maximum time steps Δt and mesh resolution Δx , distribution of models over the pipes and computing time (CPU) for different tolerances η_h

η_h	ERR	EST	max/min Δt [s]	max/min Δx [m]	$\mathcal{M}_1 : \mathcal{M}_2 : \mathcal{M}_3$ [%]	CPU[s]
10^{-1}	$1.1 \cdot 10^{-2}$	$2.2 \cdot 10^{-2}$	3600/1800	7915/767	0:0:100	3.1
10^{-2}	$1.3 \cdot 10^{-3}$	$2.9 \cdot 10^{-3}$	3600/1800	7915/767	05:20:75	4.2
10^{-3}	$3.9 \cdot 10^{-4}$	$1.5 \cdot 10^{-4}$	1800/1800	7915/767	17:51:32	5.3
10^{-4}	$1.2 \cdot 10^{-5}$	$1.5 \cdot 10^{-5}$	112/112	7915/767	31:56:13	47.9
10^{-5}	–	$3.4 \cdot 10^{-6}$	28/7	7807/767	46:47:07	511.5

with $g_{c,0} = 2629$, $g_{c,1} = 2.47428571429$, $g_{c,2} = 1.37142857143 \times 10^{-5}$ for all compressors and G_c defined in (9). The weighting factor is chosen as $\alpha = 10^{-10}$ to get values of moderate size, i.e., around 0.1.

First, we would like to demonstrate the performance of the adaptive black box solver $\text{ANet}(\cdot, \eta_h)$ for this larger network. Given the boundary conditions and controls defined in Fig. 7, we always start with the initial time step $\Delta t_0 = 1800$ s, the mesh width $\Delta x_0 = 1000$ m, and the simplest algebraic model $\mathcal{M}_3$. The statistics of the runs for tolerances $\eta_h = 10^{-i}$, $i = 1, \dots, 5$ are summarized in Table 5. The observed estimation process is quite reliable and the tolerances are always satisfied. It is nicely seen that the portion of the most detailed physical model $\mathcal{M}_1$ is increasing with higher tolerances. For the last three tolerances, we can detect $\text{CPU} \sim \eta_h^{-1}$. This was also reported for even more complex networks in [12].

Next, we model uncertainties in the exit regions RE1–RE8 by eight independent, uniformly distributed parameters $y = (y_1, \dots, y_8)$, $y_i \in \mathcal{U}[-1, 1]$, to describe random volume flows for the state U_B through

$$q_{REi}(y_i) = (1 + 0.3 \cdot y_i)q_{REi}(U_B), \quad i = 1, \dots, 8, \quad (35)$$

where $q_{REi}(U_B)$ is the corresponding volume flow for the stationary state U_B defined in Table 4. The parameters necessary to run the adaptive stochastic collocation methods were determined by a few samples for low tolerances as follows: $C_H = 0.25$, $C_Y = 0.1$, $s = 1$, and $\mu = 2$.

Let us now consider a single- and two-level approach with a reduction factor $q = 0.5$ and tolerances $\varepsilon = 5 \times 10^{-4}$, 2.5×10^{-4} , 10^{-4} , 5×10^{-5} , 2.5×10^{-5} , 10^{-5} . We computed a reference solution $\mathbb{E}[\psi(U)] = 0.120729561141951$ with $\varepsilon = 5 \times 10^{-7}$. The results are shown in Fig. 8. Both methods deliver equal values for the expectation of $\psi(U_h)$. A closer inspection shows that only 17 collocation points on the finest level are sufficient to reach the desired accuracy in all runs. This also explains the observation that the two-level approach takes slightly larger computing times since the method additionally calculates values on the coarse level. As also seen in the last example, the numbers of samples necessary to reach the tolerances are extremely small such that the single-level approach works already very efficient. Once again, the adaptive multilevel Monte Carlo method is not competitive here. The fast increasing numbers for Q_0 are too challenging, especially for higher tolerances.

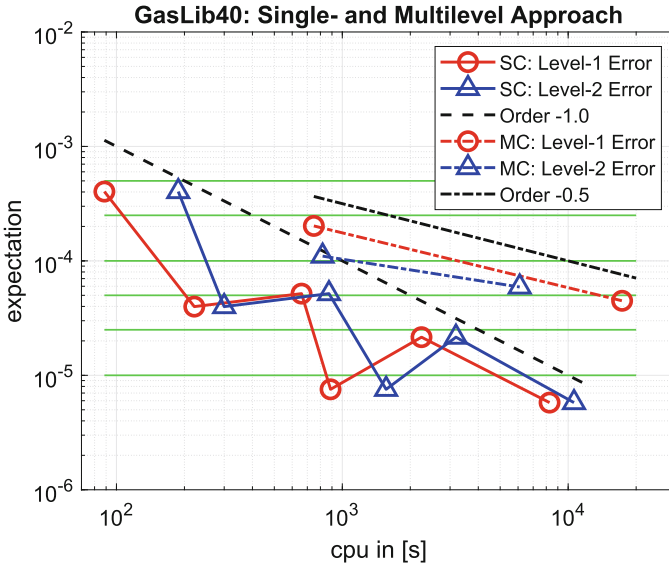


Fig. 8 *GasLib-40*: Errors for the expected values $\mathbb{E}[\psi_1^{(ML)}]$ and $\mathbb{E}[\psi^{(SL)}]$ for the two-level (blue triangles, solid line), and one-level (red circles, solid line) stochastic collocation approach with adaptive space-time-model discretizations for $\varepsilon = 5 \times 10^{-4}$, 2.5×10^{-4} , 10^{-4} , 5×10^{-5} , 2.5×10^{-5} , 10^{-5} (green lines). The accuracy achieved is always better than the tolerance. Both methods deliver equal expectations since exactly the same collocation points in the stochastic space are used. For comparison, the results of two runs of a one-level (blue triangles, dash-dot line) and two-level (red circles, dash-dot line) adaptive multilevel Monte Carlo method for $\varepsilon = 2.5 \times 10^{-4}$, 10^{-4} , calculated from an average over 10 independent realizations, are also shown. The reduced order of convergence is clearly seen

5 Conclusion and Outlook

In this study, we have applied a combination of two state-of-the-art adaptive methods to quantify smooth uncertainties in gas transport pipelines governed by systems of hyperbolic balance laws of Euler type. Our in-house software tool ANACONDA and the open-source MATLAB package *Sparse Grid Kit* provide a posteriori error estimates that can be exploited to drastically reduce the number of degrees of freedom by using a sample-dependent strategy so that the computational effort at each stochastic collocation point can be optimised individually. A single-level as well as a multilevel approach have been discussed and applied to two practical examples from the public gas library *gaslib.zib.de*. Both strategies perform similar and quite reliable even for very high levels of accuracy. However, we expect to see a greater potential of the multilevel approach when facing more challenging problems in future case studies.

In contrast to Monte Carlo methods, stochastic collocation schemes provide an access to a global interpolant over the parameter space, which can be interpreted as response surface approximation and used to easily calculate statistical moments and approximate probability density functions in a postprocessing. We are planning to incorporate these techniques into our continuous optimization framework and thus aiming for solving nonlinear probabilistic constrained optimization problems.

Acknowledgments The authors are supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) within the collaborative research center TRR154 “Mathematical modeling, simulation and optimisation using the example of gas networks” (Project-ID239904186, TRR154/2-2018, TP B01). We would like to thank Oliver Harbeck for making his drawing software available to us. We have enjoyed using it to create the Figs. 2 and 6.

References

1. Abgrall, R., Congedo, P.: A semi-intrusive deterministic approach to uncertainty quantification in non-linear fluid flow problems. *J. Comput. Phys.* **235**, 828–845 (2013)
2. Abgrall, R., Mishra, S.: Chapter 19—uncertainty quantification for hyperbolic systems of conservation laws. In: Abgrall, R., Shu, C.-W. (eds.) *Handbook of Numerical Methods for Hyperbolic Problems. Handbook of Numerical Analysis*, vol. 18, pp. 507–544. Elsevier, Amsterdam (2017)
3. Barth, A., Schwab, C., Zollinger, N.: Multi-level Monte Carlo finite element method for elliptic PDEs with stochastic coefficients. *Numer. Math.* **119**, 123–161 (2011)
4. Beck, J., Nobile, F., Tamellini, L., Tempone, R.: Stochastic spectral Galerkin and collocation methods for PDEs with random coefficients: a numerical comparison. In: Hesthaven, J.S., Ronquist, E.M. (eds.) *Spectral and High Order Methods for PDEs. Lecture Notes Computer Science Engineering*, vol. 76, pp. 43–62. Springer, Berlin (2011)
5. Bijl, H., Lucor, D., Mishra, S., Schwab, Ch. (eds.) *Uncertainty Quantification in Computational Fluid Dynamics. Lecture Notes in Computational Science and Engineering*, vol. 92. Springer, Berlin (2014)
6. Cliffe, K.A., Giles, M.B., Scheichl, R., Teckentrup, A.L.: Multilevel Monte Carlo methods and applications to elliptic PDEs with random coefficients. *Comput. Visual. Sci.* **14**, 3–15 (2011)

7. Domschke, P., Dua, A., Stolwijk, J.J., Lang, J., Mehrmann, V.: Adaptive refinement strategies for the simulation of gas flow in networks using a model hierarchy. *Electron. Trans. Numer. Anal.* **48**, 97–113 (2018)
8. Domschke, P., Kolb, O., Lang, J.: An adaptive model switching and discretization algorithm for gas flow on networks. *Procedia Comput. Sci.* **1**, 1325–1334 (2010). ICCS 2010
9. Domschke, P., Kolb, O., Lang, J.: Adjoint-based control of model and discretisation errors for gas flow in networks. *Int. J. Math. Model. Numer. Optim.* **2**, 175–193 (2011)
10. Domschke, P., Kolb, O., Lang, J.: Adjoint-based control of model and discretization errors for gas and water supply networks. In: Koziel, S., Yang, X.-S. (eds.) *Computational Optimization and Applications in Engineering and Industry. Studies in Computational Intelligence*, vol. 359, pp. 1–18. Springer, Berlin (2011)
11. Domschke, P., Kolb, O., Lang, J.: Adjoint-based error control for the simulation and optimization of gas and water supply networks. *Appl. Math. Comput.* **259**, 1612–1634 (2015)
12. Domschke, P., Kolb, O., Lang, J.: Fast and reliable transient simulation and continuous optimization of large-scale gas networks. *Math. Meth. Oper. Res.* (2022). <https://doi.org/10.1007/s00186-021-00765-7>
13. Gerstner, T., Griebel, M.: Dimension-adaptive tensor-product quadrature. *Computing* **71**, 65–87 (2003)
14. Ghanem, R., Higdon, D., Owhadi, H. (eds.) *Handbook of Uncertainty Quantification*. Springer, Berlin (2016)
15. Giles, M.: Multilevel Monte Carlo path simulation. *Operations Res.* **56**, 607–617 (2008)
16. Gramacki, A. (ed.) *Nonparametric Kernel Density Estimation and Its Computational Aspects*. Springer, Berlin (2018)
17. Heinrich, S.: Multilevel Monte Carlo methods. In: Margenov, S., Waniowski, J., Yalamov, P. (eds.) *Large-Scale Scientific Computing. Lecture Notes Computer Science*, vol. 2179, pp. 58–67. Springer, Berlin (2001)
18. Kolb, O.: *Simulation and Optimization of Gas and Water Supply Networks*. PhD thesis, Technische Universität Darmstadt, Darmstadt (2011)
19. Kolb, O.: A third order hierarchical basis WENO interpolation for sparse grids with application to conservation laws with uncertain data. *J. Sci. Comput.* **74**, 1480–1503 (2018)
20. Kolb, O., Lang, J., Bales, P.: An implicit box scheme for subsonic compressible flow with dissipative source term. *Numer. Algorithms* **53**, 293–307 (2010)
21. Kusch, J., Wolters, J., Frank, M.: Intrusive acceleration strategies for uncertainty quantification for hyperbolic systems of conservation laws. *J. Comput. Phys.* **419**, 109698 (2020)
22. Lang, J., Scheichl, R., Silvester, D.: A fully adaptive multilevel stochastic collocation strategy for solving elliptic PDEs with random data. *J. Comput. Phys.* **419**, 109692 (2020)
23. Menon, E.S. (ed.) *Gas Pipeline Hydraulics*. Taylor & Francis, London (2005)
24. Mindt, P., Lang, J., Domschke, P.: Entropy-preserving coupling of hierarchical gas models. *SIAM J. Math. Anal.* **51**, 4754–4775 (2019)
25. Mishra, S., Schwab, Ch.: Sparse tensor multi-level Monte Carlo finite volume methods for hyperbolic conservation laws with random initial data. *Math. Comp.* **81**, 1979–2018 (2012)
26. Mishra, S., Schwab, Ch., Šukys, J.: Multi-level monte carlo finite volume methods for nonlinear systems of conservation laws in multi-dimensions. *J. Comput. Phys.* **231**, 3365–3388 (2012)
27. Mishra, S., Risebro, N.H., Schwab, Ch., Tokareva, S.: Numerical solution of scalar conservation laws with random flux functions. *SIAM/ASA J. Uncertain.* **4**, 552–591 (2016)
28. Poette, G., Despreś, B., Lucor, D.: Uncertainty quantification for systems of conservation laws. *J. Comput. Phys.* **228**, 2443–2467 (2009)
29. Schmidt, M., Aßmann, D., Burlacu, R., Humpola, J., Joormann, I., Kanelakis, N., Koch, T., Oucherif, D., Pfetsch, M.E., Schewe, L., Schwarz, R., Sirvent, M.: GasLib—A library of gas network instances. *Data* **2**, article 40 (2017)
30. Schuster, M., Strauch, E., Gugat, M., Lang, J.: Probabilistic constrained optimization on flow networks. *Optimization and Engineering* (2021). <https://doi.org/10.1007/s11081-021-09619-x>
31. Tamellini, L., Nobile, F., Guignard, D., Tesei, F., Sprungk, B.: Sparse grid matlab kit, version 17-5, software available at <https://csqi.epfl.ch>

32. Teckentrup, A.L., Jantsch, P., Webster, C.G., Gunzburger, M.: A multilevel stochastic collocation method for partial differential equations with random input data. *SIAM/ASA J. Uncertain.* **3**, 1046–1074 (2015)
33. Tryon, J., Le Maitre, O., Ndjinga, M., Ern, A.: Intrusive projection methods with upwinding for uncertain non-linear hyperbolic systems. *J. Comput. Phys.* **235**, 491–506 (2010)
34. Zhu, X., Linebarger, E.M., Xiu, D.: Multi-fidelity stochastic collocation method for computation of statistical moments. *J. Comput. Phys.* **341**, 386–396 (2017)

HexDom: Polycube-Based Hexahedral-Dominant Mesh Generation



Yuxuan Yu, Jialei Ginny Liu, and Yongjie Jessica Zhang

Abstract In this paper, we extend our earlier polycube-based all-hexahedral mesh generation method to hexahedral-dominant mesh generation, and present the HexDom software package. Given the boundary representation of a solid model, HexDom creates a hex-dominant mesh by using a semi-automated polycube-based mesh generation method. The resulting hexahedral dominant mesh includes hexahedra, tetrahedra, and triangular prisms. By adding non-hexahedral elements, we are able to generate better quality hexahedral elements than in all-hexahedral meshes. We explain the underlying algorithms in four modules including segmentation, polycube construction, hex-dominant mesh generation and quality improvement, and use a rockerarm model to explain how to run the software. We also apply our software to a number of other complex models to test their robustness. The software package and all tested models are available in github (<https://github.com/CMU-CBML/HexDom>).

1 Introduction

In finite element analysis (FEA), a 3D domain can be discretized into tetrahedral or hexahedral (hex) meshes. For tetrahedral mesh generation, various strategies have been proposed in the literature [15, 19, 33, 54], such as octree-based [38, 56], Delaunay triangulation [4], and advancing front methods [8, 23, 37]. Because tetrahedral meshes can be created automatically, it has been widely used in industry. However, to achieve the same precision in FEA, a tetrahedral mesh requires more elements than an all-hex mesh does. As a result, many techniques have been developed to generate all-hex meshes [32, 34, 47, 48, 61] or converting imaging data to all-hex meshes [53–55]. Also, hex meshes can serve as multiple-material domains [59, 62] or input control meshes for IGA [16, 17, 29, 43–46, 64]. Some

Y. Yu · J. G. Liu · Y. J. Zhang (✉)

Department of Mechanical Engineering, Carnegie Mellon University, Pittsburgh, PA, USA

e-mail: yuxuany1@andrew.cmu.edu; jialeil@andrew.cmu.edu; jessicaz@andrew.cmu.edu

applications of hex mesh generation in new engineering applications can also be found in [18, 50, 52, 58, 60]. Several literatures develop methods for unstructured hex mesh generation, such as grid-based or octree-based [35, 36], medial surface [30, 31], plastering [2, 39], whisker weaving [7], and vector field-based methods [26]. These methods have been used to create hex meshes for certain geometries, but are not robust and reliable for arbitrary geometries. On the other hand, although an all-hex mesh provides a more accurate solution, a high-quality all-hex mesh is more difficult to create automatically.

Compared with all-hex mesh generation, a hex-dominant mesh generation, which combines advantages of both tetrahedral and hex elements, is more automatic and robust for complex solid models. In the literature, several strategies have been proposed to generate hex-dominant meshes. An indirect method was suggested by [49], the domain is first meshed into tetrahedral elements and then merged into a hex-dominant mesh with the packing technology. Several other hex-dominant meshing techniques were also presented in [24, 25, 28]. Such indirect methods create hex-dominant meshes with too many singularities, and the tetrahedral mesh directly influences the quality of the hex-dominant mesh. Similar to unstructured all-hex mesh generation, the direct method is also more preferable for hex-dominant meshes [27, 41]. The polycube-based method [9, 40] is an attractive direct approach to obtain hex-dominant meshes by using degenerated cubes. The polycube-based method was mainly used for all-hex meshing. A smooth harmonic field [42] was used to generate polycubes for arbitrary genus geometry. Boolean operations [21] were introduced to deal with arbitrary genus geometry. In [22], a polycube structure was generated based on the skeletal branches of a geometric model. Using these methods, the structure of the polycube and the mapping distortion greatly influence the quality of the hex mesh. The calculation of the polycube structure with a low mapping distortion remains an open problem for complex geometry. It is important to improve the quality of the mesh for analysis by using methods such as pillowing, smoothing, and optimization [32, 34, 57, 63]. Pillowing is an insert-sheet technique that eliminates situations where two adjacent hex elements share more than one face. Smoothing and optimizing are used to further improve the quality of the mesh by moving the vertices. In our software, we implement all of the above methods to improve the quality of hex elements.

In this paper, we extend our earlier semi-automatic polycube-based all-hex generation to hex-dominant meshing. The software package includes: (1) polycube based geometric decomposition of a surface triangle mesh; (2) generation of the polycube consisting of non-degenerated and degenerated cubes; (3) creation of a parametric domain for different types of degenerated unit cubes including prisms and tetrahedra; and (4) creation of a hex-dominant mesh. We first go through the entire pipeline and explain the algorithm behind each module of the pipeline. Then, we use a specific example to follow all the steps and run the software. In particular, when user intervention is required, the details of manual work are explained. The paper is organized as follows. In Sect. 2 we provide an overview of the pipeline. In Sect. 3 we present the HexDom software package, with a semi-automatic polycube-based hex-dominant mesh generation of a CAD file. Finally, in Sect. 4 we show various complex models with our software package.

2 Pipeline Design

Our pipeline uses polycube-based method to create a hex-dominant mesh from an input CAD model. As shown in Fig. 1, we first generate a triangle mesh from the CAD model by using the free software LS-PrePost. Then we use centroidal Voronoi tessellation (CVT) segmentation [12–14] to create a polycube structure [40]. The polycube structure consists of multiple non-degenerated cubes and degenerated cubes. The non-degenerated cubes will yield hex elements via parametric mapping [6] and octree subdivision [63]. The degenerated cubes will yield degenerated elements such as prisms and tetrahedra in the final mesh. Here, we implement the subdivision algorithm separately for prism-shape regions and tetrahedral-shape regions. The quality of the hex dominant mesh is evaluated to ensure that the resulting mesh can be used in FEA. In case that a poor quality hex element is generated in hex-dominant meshes, the program has various quality improvement functions, including pillowing [57], smoothing, and optimization [32]. Each quality improvement function can be performed independently and one can

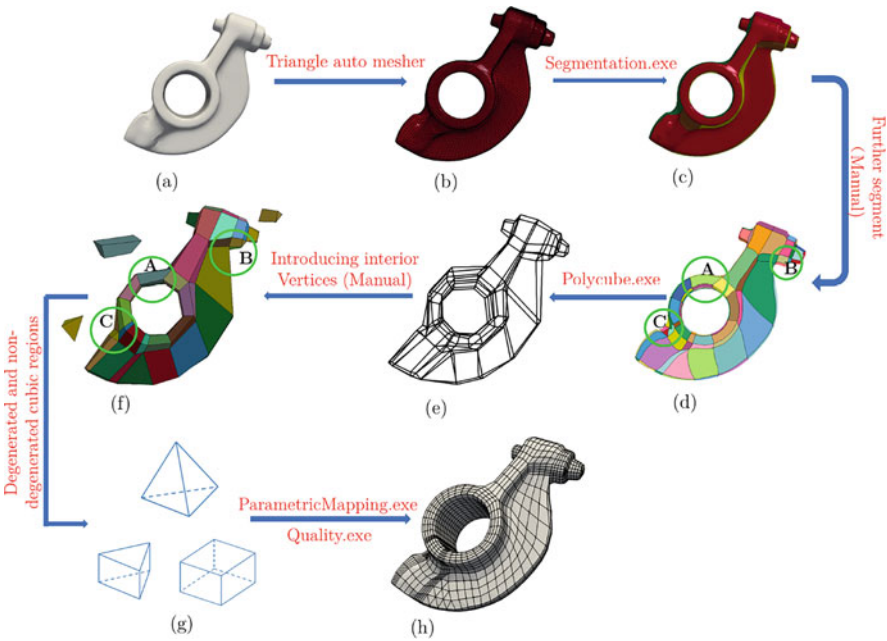


Fig. 1 The HexDom software package. For each process, the black texts describe the object and the red texts show the operation needed to go to the next process. Manual work is involved in further segmentation and introducing interior vertices. Regions A, B and C (green circles) in (d, f) contain a hex, prism and tetrahedral shaped structure, respectively. (a) CAS model. (b) Surface triangle mesh. (c) CVT-based surface segmentation result. (d) Further segmentation result. (e) Surface of the polycube. (f) Polycube structure. (g) Different parametric domain. (h) Hex-dominant mesh

use these functions to improve the mesh quality. Currently, our software only has a command-line interface (CLI). Users need to provide the required options on the command line to run the software. In Sect. 3, we will explain in detail the algorithms implemented in the software as well as how to run the software.

3 HexDom: Polycube-Based Hex-Dominant Mesh Generation

Surface segmentation, polycube construction, parametric mapping, and subdivision are used together in the HexDom software package to generate a hex-dominant mesh from the boundary representation of the input CAD model. Given a triangle mesh generated from the CAD model, we first use surface segmentation to divide the mesh into several surface patches that meet the restrictions of the polycube structure, which will be discussed in Sect. 3.1. The corner vertices, edges, and faces of each surface patch are then extracted from the surface segmentation result to construct a polycube structure. Each component of the polycube structure is topologically equivalent to a cube or a degenerated cube. Finally, we generate the hex-dominant mesh through parametric mapping and subdivision. Quality improvement techniques can be used to further improve the mesh quality.

In this section, we will introduce the main algorithm for each module of the HexDom software package, namely surface segmentation, polycube construction, parametric mapping and subdivision, and quality improvement. We will use a rockerarm model (see Fig. 1) to explain how to run CLI for each module. We will also discuss the user intervention involved in the semi-automatic polycube-based hex-dominant mesh generation.

3.1 Surface Segmentation

The surface segmentation in the pipeline framework is implemented based on CVT segmentation [12–14]. CVT segmentation is used to classify vertices by minimizing an energy function. Each group is called a Voronoi region and has a corresponding center called a generator. The Voronoi region and its corresponding generator are updated iteratively in the minimization process. In [13], each element of the surface triangle mesh is assigned to one of the six Voronoi regions based on the normal vector of the surface. The initial generators of the Voronoi regions are the three principal normal vectors and their opposite normal vectors ($\pm X$, $\pm Y$, $\pm Z$). Two energy functions and their corresponding distance functions are used together in [13]. The classical energy function and its corresponding distance function provide initial Voronoi regions and generators. Then, the harmonic boundary-enhanced (HBE) energy function and its corresponding distance function are applied

to eliminate non-monotone boundaries. The detailed definitions of the energy functions and their corresponding distance functions are described in [13]. The surface segmentation process was also summarized in the **Surface Segmentation Algorithm** in [51].

Once we get the initial segmentation result, we need to further segment each Voronoi region into several patches to satisfy the topological constraints for polycube construction (see Fig. 1d). We use two types of patches. The first type of segmented surface patch corresponds to one boundary surface of the non-degenerated cubes and quadrilateral surface of the prism-shape degenerated cubes. The second type of segmented surface patch corresponds to one triangular boundary surface of the degenerated cubes. The choice of types of patches depends on the following three criteria: (1) geometric features such as sharp corners with small angles and prism/tetrahedral-like features; (2) critical regions based on finite element simulation, such as regions with the maximum stress/strain and regions with a high load; and (3) requirements from user applications which enhance the capability of user interaction. For the first type of segmented surface patch, the following three conditions should be satisfied during the further segmentation: (1) two patches with opposite orientations (e.g., $+X$ and $-X$) cannot share a boundary; (2) each corner vertex must be shared by more than two patches; and (3) each patch must have four boundaries. For the second type of segmented surface patch, we modified the third conditions to that each patch must have three boundaries.

Note that we define the corner vertex as a vertex locating at the corner of the cubic region or degenerated cubic region in the model. The further segmentation is done manually by using the patch ID reassigning function in LS-PrePost. The detailed operation was shown in [51].

3.2 Polycube Construction

In this section, we discuss the detailed algorithm of polycube construction based on the segmented triangle mesh. Several automatic polycube construction algorithms have been proposed in the literature [11, 13, 20], but it is challenging to apply these methods to complex CAD models. The polycube structure does not contain degenerated cubes either. Differently, the polycube in this paper consists of cubes and degenerated cubes and is topologically equivalent to the original geometry. To achieve versatility for real industrial applications, we develop a semi-automatic polycube construction software based on the segmented surface. However, for some complex geometries, the process may be slower due to potentially heavy user intervention.

The most important information we need for a polycube is its corners and the connectivity relationship among them. For the surface of polycube, we can automatically get the corner points and build their connectivity based on the segmentation result by using the algorithm similar to the **Polycube Boundary Surface Construction Algorithm** in [51]. The difference is that we need to adjust

the implementation based on different patch types: finding its three corners for a triangular patch and finding its four corners for a quadrilateral patch. It is difficult to obtain inner vertices and their connectivity because we only have a surface input with no information about the interior volume. In fact, this is where users need to intervene. We use LS-PrePost to manually build the interior connectivity. You can find the detailed operation in Appendix A3 in [51]. As the auxiliary information for this user intervention, the **Polycube Boundary Surface Construction Algorithm** will output corners and connectivity of the segmented surface patches into .k file. Finally, the generated polycube structure is the combination of non-degenerated cubes and degenerated cubes splitting the volumetric domain of the geometry.

3.3 Parametric Mapping and Subdivision

After the polycube is constructed, we need to build a bijective mapping between the input triangle mesh and the boundary surface of the polycube structure. In our software, we implement the same idea as in [22]: using a non-degenerated unit cube or a degenerated unit cube as the parametric domain for the polycube structure. As a result, we can construct a generalized polycube structure that can better align with the given geometry and generate a high quality hex-dominant mesh.

There are three types of elements in the hex-dominant mesh: hex, prism, and tetrahedral. The hex elements form non-degenerated cubic regions. Prism and tetrahedral elements form degenerated cubic regions. We will use octree subdivision to generate hex elements for non-degenerated cubic regions, while using subdivision to generate prism and tetrahedral elements for degenerated cubic regions. Through the pseudocode in the **Parametric Mapping Algorithm** in [51], we describe how to combine the segmented surface mesh, the polycube structure, and the unit cube to create an all-hex mesh. We use this algorithm to create the hex elements in non-degenerated cubic regions. Each non-degenerated cube in the polycube structure represents one volumetric region of the geometry and has a non-degenerated unit cube as its parametric domain. Region A in Fig. 1d, f shows an example of a non-degenerated cube and its corresponding volume domain of the geometry marked in the green circle. For degenerated cubes, there are two types of interface, a triangular face and a quadrilateral face. Region B in Fig. 1d, f shows a prism case, it contains two triangular faces and three quadrilateral faces. For the tetrahedral case shown in Region C in Fig. 1d, f, it contains four triangular faces.

Through the pseudocode in the **Prism Parametric Mapping Algorithm**, we describe how the segmented surface mesh, the polycube structure and the prism-shape degenerated unit cube are combined to generate prism elements. Let $\{\bar{S}_i\}_{i=1}^N$ be the segmented surface patches coming from the segmentation result (see Fig. 2a). Each segmented surface patch corresponds to one boundary surface of the polycube $\bar{P}_i$ ($1 \leq i \leq N$) (see Fig. 2b), where N is the number of the boundary surfaces. There are also interior surfaces, denoted by $\bar{I}_j$ ($1 \leq j \leq M$), where M is the number

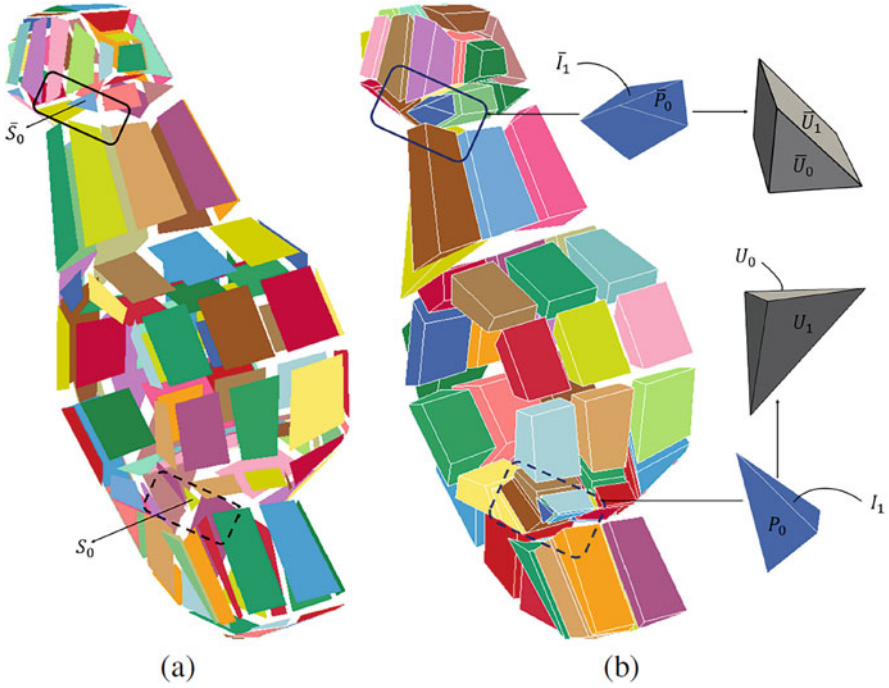


Fig. 2 The polycube construction and the parametric mapping process for prism-shape degenerated cubic regions (see the black boxes) and tetrahedral-shape degenerated cubic regions (see the dashed black boxes). (a) The boundary surface of the polycube generated by Polycube Boundary Surface Construction Algorithm; (b) $\bar{S}_0$, $\bar{P}_0$ and $\bar{U}_0$ are used for parametric mapping to create boundary vertices of the prism-shape degenerated cubic regions. $\bar{I}_1$ and $\bar{U}_1$ are used for linear interpolation to create interior vertices of the prism-shape degenerated cubic regions. S_0 , P_0 and U_0 are used for parametric mapping to create boundary vertices of the tetrahedral-shape degenerated cubic regions. I_1 and U_1 are used for linear interpolation to create interior vertices of the tetrahedral-shape degenerated cubic regions

of the interior surfaces. The union of $\{\bar{P}_i\}_{i=1}^N$ and $\{\bar{I}_j\}_{j=1}^M$ is the set of surfaces of the polycube structure. For the parametric domain, let $\{\bar{U}_k\}_{k=1}^5$ denote the five surface patches of the prism-shape degenerated unit cube (see Fig. 2b).

Each prism-shape degenerated cube in the polycube structure represents one volumetric region of the geometry and has a prism-shape degenerated unit cube as its parametric domain. Figure 2b shows an example of prism-shape degenerated cube and its corresponding volume domain of the geometry marked in the black boxes. Therefore, for each prism-shape degenerated cube in the polycube structure, we can find its boundary surface $\bar{P}_i$ and map the segmented surface patch $\bar{S}_i$ to its corresponding parametric surface $\bar{U}_k$ of the prism-shape degenerated unit cube. To map $\bar{S}_i$ to $\bar{U}_k$, we first map its corresponding boundary edges of $\bar{S}_i$ to the boundary edges of $\bar{U}_k$. Then we get the parameterization of $\bar{S}_i$ by using the cotangent Laplace operator to compute the harmonic function [5, 63]. Compared to non-degenerated

cubic regions algorithm, we introduce three parametric variables in mapping since one face is not axis-aligned. Note that for an interior surface $\tilde{I}_j$ of the polycube structure, we skip the parametric mapping step.

The prism elements can then be obtained from the above surface parameterization combined with subdivision. We generate the prism elements for each prism-shape region in the following process. To obtain vertex coordinates on the segmented patch $\tilde{S}_i$, we first subdivide the prism-shape degenerated unit cube (see Fig. 3a) recursively in order to get their parametric coordinates. The vertex coordinates of triangular faces of the prism-shape degenerated cube are obtained by linear subdivision, while the quadrilateral faces are also obtained by linear subdivision. The physical coordinates can be obtained by using parametric mapping, which has a one-to-one correspondence between the parametric domain $\tilde{U}_k$ and the physical domain $\tilde{S}_i$. To obtain vertices on the interior surface of the prism region, we skip the parametric mapping step and directly use linear interpolation to calculate the physical coordinates. Finally, vertices inside the cubic region are calculated by linear interpolation. The entire prism elements are built by going through all the prism-shape regions.

We perform the similar procedure for the tetrahedra-shape degenerated cubes in the polycube structure. Through the pseudocode in the **Tetrahedral Parametric**

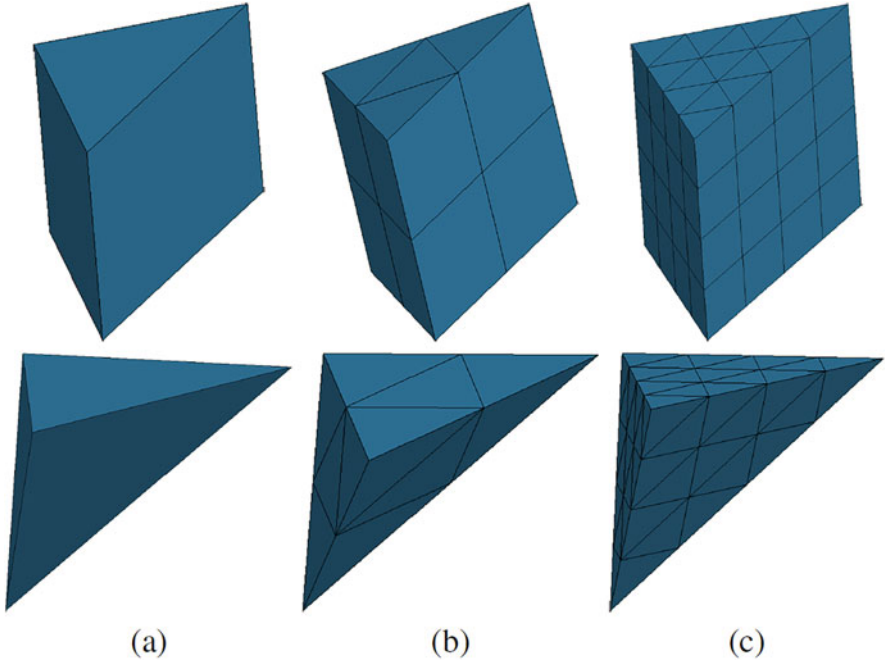


Fig. 3 The subdivision of prism-shape degenerated unit cube (top row) and tetrahedral-shape degenerated unit cube (bottom row). (a) Subdivision level 0; (b) subdivision level 1; and (c) subdivision level 2

Prism Parametric Mapping Algorithm

Input: Segmented triangle mesh $\{\bar{S}_i\}_{i=1}^N$, polycube structure

Output: Prism elements in prism-shape degenerated cubic regions

1: Find boundary surfaces $\{\bar{P}_i\}_{i=1}^N$ and interior surfaces $\{\bar{I}_j\}_{j=1}^M$ in the polycube structure

Surface parameterization step:

2: **for** each prism-shape degenerated cube in the polycube structure **do**

3: Create a prism-shape degenerated cube region $\{\bar{U}_k\}_{k=1}^5$ as the parametric domain

4: **for** each surface in the prism-shape degenerated cube **do**

5: **if** it is a boundary surface $\bar{P}_i$ **then**

6: **if** the surface is not axis-aligned **then**

7: Get the surface parameterization $f : \bar{S}_i \rightarrow \bar{U}_k \subset \mathbb{R}^3$

8: **else**

9: Get the surface parameterization $f : \bar{S}_i \rightarrow \bar{U}_k \subset \mathbb{R}^2$

10: **end if**

11: **end if**

12: **end for**

13: **end for**

Parametric mapping and subdivision step:

14: **for** each prism-shape degenerated cube in the polycube structure **do**

15: Subdivide the prism-shape degenerated unit cube recursively to get parametric coordinates

16: **for** each surface in the prism-shape degenerated cube **do**

17: **if** it is a boundary surface $\bar{P}_i$ **then**

18: Obtain physical coordinates using $f^{-1}(v_{para})$

19: **else if** it is an interior surface $\bar{I}_j$ **then**

20: Obtain physical coordinates using linear interpolation

21: **end if**

22: **end for**

23: Obtain interior vertices in the prism-shape degenerated cubic region using linear interpolation

24: **end for**

Mapping Algorithm. we describe how the segmented surface mesh, the polycube structure and the tetrahedral-shape degenerated unit cube are combined to generate tetrahedral elements. Figure 2b shows an example of tetrahedral-shape degenerated cube and its corresponding volume domain of the geometry marked in the dashed black boxes. The difference is that we use $\{U_k\}_{k=1}^4$ to denote those four surface patches of the tetrahedra-shape degenerated unit cube for the parametric domain. We also introduce three parametric variables in mapping when one of the surfaces is not axis aligned. Then, the tetrahedral elements can be obtained from this surface parameterization combined with linear subdivision. We generate tetrahedral elements for each tetrahedral-shape region in the following process. To obtain vertex coordinates on the segmented patch S_i , we first subdivide the tetrahedral-shape degenerated unit cube (see Fig. 3(bottow row)) recursively in order to get their parametric coordinates by applying linear subdivison. The physical coordinates can be obtained by using the parametric mapping between the parametric domain U_k and the physical domain S_i . I_1 and U_1 are combined for linear interpolation to obtain vertices on the interior surface of the tetrahedra-shape degenerated cubic region.

Finally, vertices inside the tetrahedra-shape degenerated cube region are calculated by linear interpolation. The entire tetrahedral elements are built by going through all the tetrahedral regions.

Tetrahedral Parametric Mapping Algorithm

Input: Segmented triangle mesh $\{S_i\}_{i=1}^N$, polycube structure

Output: Tetrahedral elements in tetrahedral-shape degenerated cubic regions

1: Find boundary surfaces $\{P_i\}_{i=1}^N$ and interior surfaces $\{I_j\}_{j=1}^M$ in the polycube structure

Surface parameterization step:

```

2: for each tetrahedral-shape degenerated cube in the polycube structure do
3:   Create a tetrahedral-shape degenerated cube region  $\{U_k\}_{k=1}^4$  as the parametric domain
4:   for each surface in the tetrahedral-shape degenerated cube do
5:     if it is a boundary surface  $P_i$  then
6:       if the surface is not axis-aligned then
7:         Get the surface parameterization  $f : S_i \rightarrow U_k \subset \mathbb{R}^3$ 
8:       else
9:         Get the surface parameterization  $f : S_i \rightarrow U_k \subset \mathbb{R}^2$ 
10:      end if
11:    end if
12:  end for
13: end for

```

Parametric mapping and subdivision step:

```

14: for each tetrahedral-shape degenerated cube in the polycube structure do
15:   Subdivide the tetrahedral-shape degenerated unit cube recursively to get parametric
      coordinates  $v_{para}$ 
16:   for each surface in the tetrahedral-shape degenerated cube do
17:     if it is a boundary surface  $P_i$  then
18:       Obtain physical coordinates using  $f^{-1}(v_{para})$ 
19:     else if it is an interior surface  $I_j$  then
20:       Obtain physical coordinates using linear interpolation
21:     end if
22:   end for
23:   Obtain interior vertices in the tetrahedral-shape degenerated cubic region using linear
      interpolation
24: end for

```

Based on the **Prism Parametric Mapping Algorithm**, we implemented and organized the code into a CLI program (PrismGen.exe) that can generate prism elements by combining parametric mapping with subdivision. Here, we run the following command to generate the prism elements for the rockerarm model:

```

1 PrismGen.exe -i rockerarm_indexpatch_read.k -p
2 rockerarm_polycube_structure.k -o rockerarm_prism.vtk -s 2

```

There are four options used in the command:

- **-i:** Surface triangle mesh of the input geometry with segmentation information (rockerarm_indexpatch_read.k);
- **-o:** Prism mesh (rockerarm_prism.vtk);

- **-p**: Polycube structure (rockerarm_polycube_structure.k); and
- **-s**: Subdivision level.

We use **-i** to input the segmentation file generated in Sect. 3.1 and use **-p** to input the polycube structure created in Sect. 3.2. Option **-s** is used to set the level of recursive subdivision. There is no subdivision if we set **-s** to be 0. In the rockerarm model, we set **-s** to be 2 to create a level-2 prism elements in the final mesh. The output prism elements are stored in the *vtk* format (see Fig. 4a) and they can be visualized in Paraview [1].

Based on **Tetrahedral Parametric Mapping Algorithm**, we implemented and organized the code into a CLI program (TetGen.exe) that can generate tetrahedral elements by combining parametric mapping with linear subdivision. Here, we run the following command to generate tetrahedral elements for the rockerarm model:

```
1 TetGen.exe -i rockerarm_indexPatch_read.k -p
2 rockerarm_polycube_structure.k -o rockerarm_tet.vtk -s 2
```

There are five options used in the command:

- **-i**: Surface triangle mesh of the input geometry with segmentation information (rockerarm_indexPatch_read.k);
- **-o**: Tet mesh (rockerarm_tet.vtk);
- **-p**: Polycube structure (rockerarm_polycube_structure.k); and
- **-s**: Subdivision level.

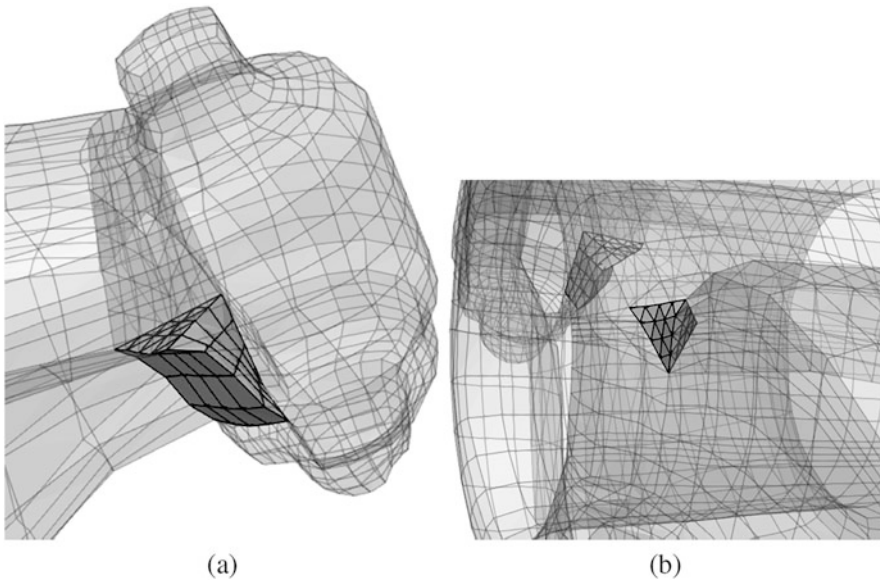


Fig. 4 Prism elements and tetrahedral elements of the rockerarm model (some elements are removed to show the interior of Fig. 1h. (a) Prism elements in a prism-shape region; and (b) tetrahedral elements in a tetrahedral-shape region

We use **-i** to input the segmentation file generated in Sect. 3.1 and use **-p** to input the polycube structure created in Sect. 3.2. Option **-s** is used to set the level of recursive subdivision. There is no subdivision if we set **-s** to be 0. In the rockerarm model, we set **-s** to be 2 to create a level-2 tetrahedral mesh. The output tetrahedral elements are stored in the *vtk* format (see Fig. 4b) and they can be visualized in Paraview.

3.4 Quality Improvement

We integrate three quality improvement techniques in the software package, namely pillowing, smoothing and optimization. Users can improve mesh quality through the command line options. We first use pillowing to insert one layer of elements around the boundary [63] of the hex elements. By using the pillowing technique, we ensure that each hex element has at most one face on the boundary, which can help improve the mesh quality around the boundary. After pillowing, smoothing and optimization [63] are used to further improve the quality of hex elements. For smoothing, different relocation methods are applied to three types of vertices: vertices on sharp edges of the boundary, vertices on the boundary surface, and interior vertices. For each sharp-edge vertex, we first detect its two neighboring vertices on the curve, and then calculate their middle point. For each vertex on the boundary surface, we calculate the area center of its neighboring boundary quadrilaterals (quads). For each interior vertex, we calculate the weighted volume center of its neighboring hex elements as the new position. We relocate the vertex iteratively. Each time the vertex moves only a small step towards the new position and this movement is done only if the new location results in an improved local Jacobian. If there are still poor quality hex elements after smoothing, we run the optimization whose objective function is the Jacobian. Each vertex is then moved toward an optimal position that maximizes the worst Jacobian. We presented the **Quality Improvement Algorithm** in [51] for quality improvement.

4 HexDom Software and Applications

The algorithms discussed in Sect. 3 were implemented in C++. The Eigen library [10] is used for matrix and vector operations. We used a compiler-independent building system (CMake) and a version-control system (Git) to support software development. We have compiled the source code into the following software package:

- HexDom software package:
 - Segmentation module (Segmentation.exe);
 - Polycube construction module (Polycube.exe);

- Hex-dominant mesh generation module (HexGen.exe, PrismGen.exe, TetGen); and
- Quality improvement module (Quality.exe).

The software is open-source and can be found in the following Github link (<https://github.com/CMU-CBML/HexDom>).

We have applied the software package to several models and generated hex-dominant meshes with good quality. For each model, we show the segmentation result, the corresponding polycube structure, and the hex-dominant mesh. These models include: rockerarm (Fig. 1); two types of mount, hepta and a base with four holes (Fig. 5); fertility, ant, bust, igea, and bunny (Fig. 6). Table 1 shows the statistics of all tested models. We use the scaled Jacobian to evaluate the quality of hex elements. The aspect ratio is used as the mesh quality metric for prism and tet elements which is the ratio between the longest and shortest edges of an element. The aspect ratio is computed with the LS-PrePost, which is a pre and post-processor for LS-DYNA [3]. From Table 1, we can observe that the generated hex-dominant meshes have good quality (minimal Jacobian of hex elements >0.1). Figures 5 and 6a show the segmentation results of the testing models. Then, we generate polycubes (Figs. 5 and 6b) based on the surface segmentation. Finally, we generate hex-dominant meshes (Figs. 5 and 6c).

5 Conclusion and Future Work

In this paper, we present a new HexDom software package to generate hex-dominant meshes. The main goal of HexDom is to extend the polycube-based method to hex-dominant mesh generation. The compiled software package makes our pipeline accessible to industrial and academic communities for real-world engineering applications. It consists of six executable files, namely segmentation module (Segmentation.exe), polycube construction module (Polycube.exe), hex-dominant mesh generation module (HexGen.exe, PrismGen.exe, TetGen.exe) and quality improvement module (Quality.exe). These executable files can be easily run in the Command Prompt platform. The rockerarm model was used to explain how to run these programs in detail. We also tested our software package using several other models.

Our software has limitations which we will address in our future work. First, the hex-dominant mesh generation module is semi-automatic and needs user intervention to create polycube structure. Second, the degenerated cubic regions and non-degenerated cubic regions need to be handled separately. We will improve the underneath algorithm and make polycube construction more automatic. In addition, we will also develop spline basis functions for tetrahedral and prism elements to support isogeometric analysis for hybrid meshes.

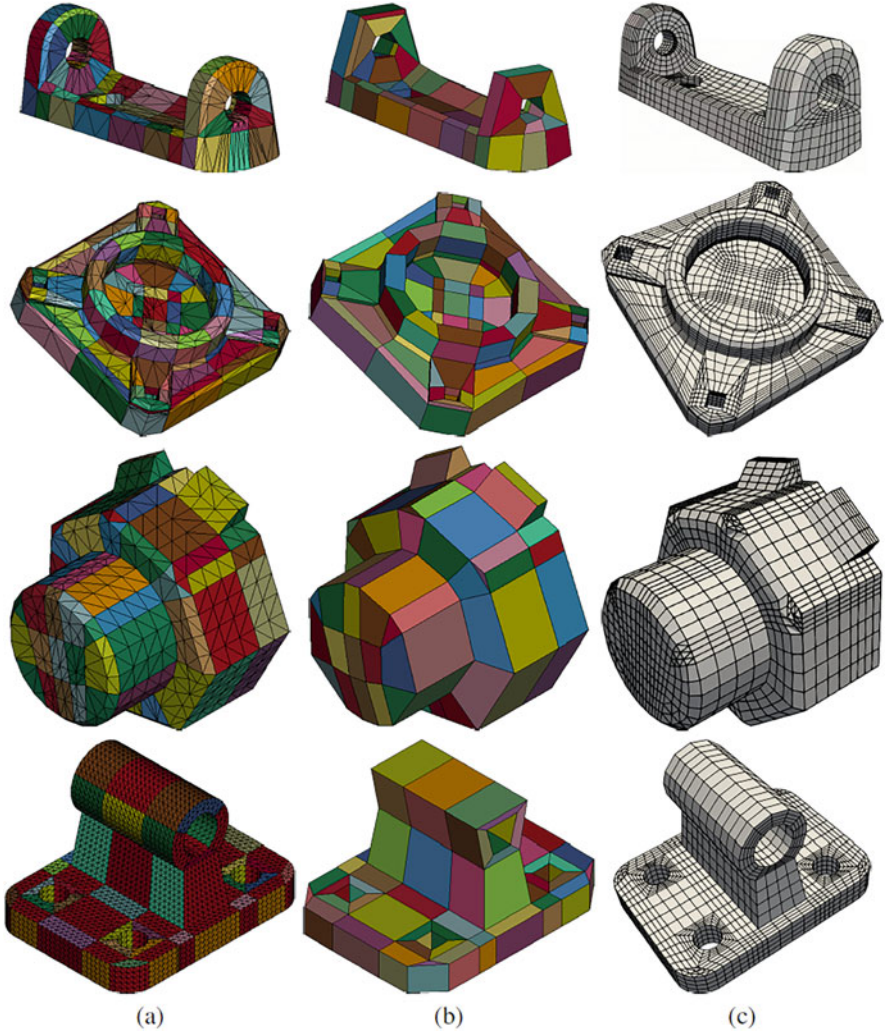


Fig. 5 Results of two types of mount, hepta and a base with four holes. **(a)** Surface triangle meshes and segmentation results; **(b)** polycube structures; and **(c)** hex-dominant meshes

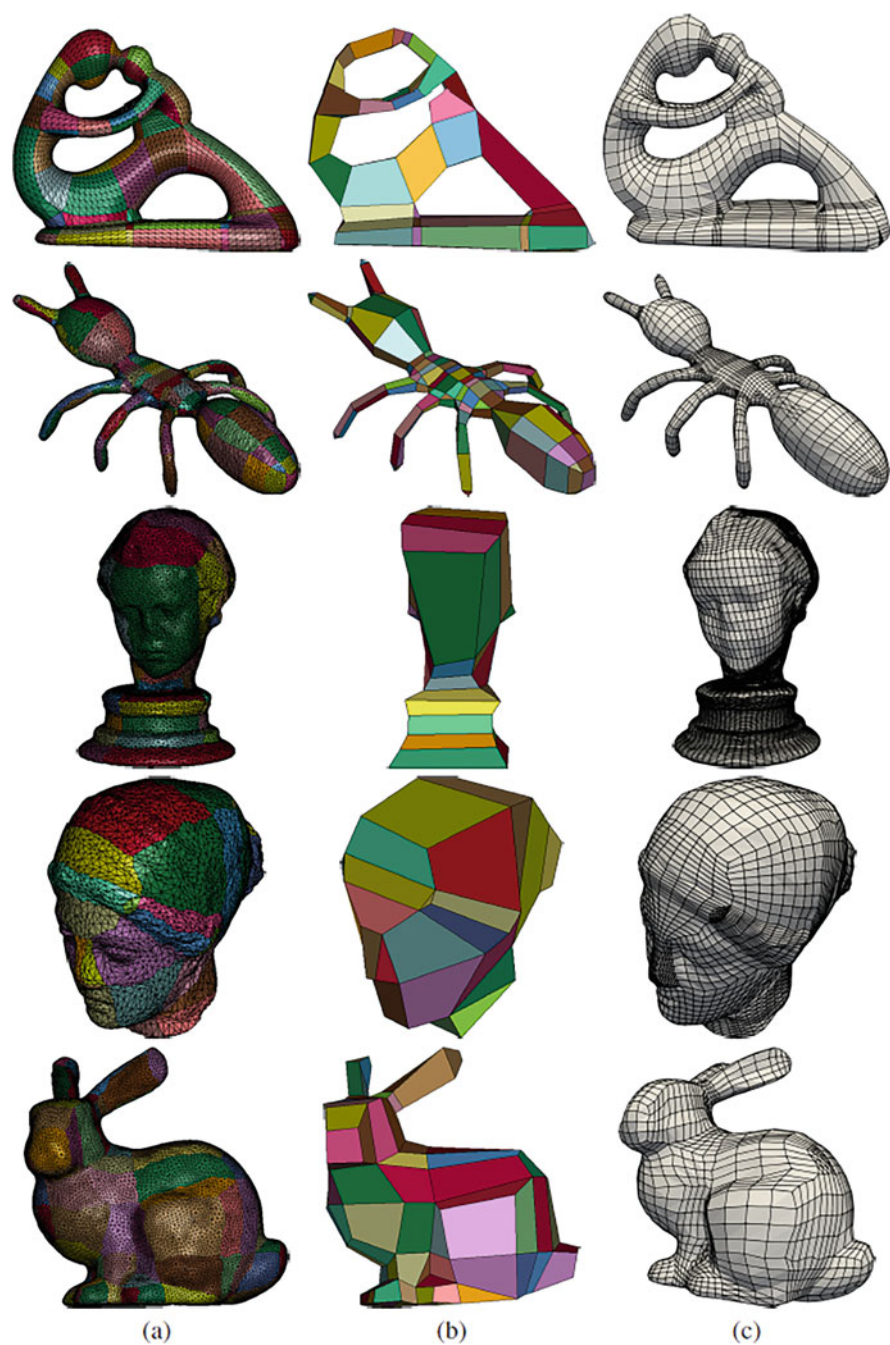


Fig. 6 Results of fertility, ant, bust, igea, and bunny models. (a) Surface triangle meshes and segmentation results; (b) polycube structures; and (c) hex-dominant meshes

Table 1 Statistics of all the tested models

Model	Input triangle mesh (vertices, elements)	Number of elements			Jacobian (worst)		Aspect ratio (min, max)	
		Hex	Prism	Tet	Hex		Prism	Tet
Rockersarm (Fig. 1)	(11, 705, 23, 410)	3840	704	128	0.20		(1.32, 4.93)	(1.62, 2.98)
Mount1 (Fig. 5)	(929, 1868)	4224	640	128	0.20		(1.91, 19.57)	(1.67, 3.69)
Mount2 (Fig. 5)	(1042, 2096)	6720	1024	128	0.28		(2.63, 9.54)	(2.30, 4.03)
Hepta (Fig. 5)	(692, 1380)	3776	1280	128	0.50		(1.51, 4.48)	(1.63, 3.14)
Base (Fig. 5)	(5342, 10, 700)	3712	384	128	0.34		(1.28, 2.33)	(1.70, 8.31)
Fertility (Fig. 6)	(6644, 13, 300)	2752	320	128	0.20		(2.96, 11.40)	(1.69, 2.69)
Ant (Fig. 6)	(7309, 14, 614)	4480	1, 536	128	0.21		(1.16, 6.52)	(1.60, 2.79)
Bust (Fig. 6)	(12, 683, 25, 362)	118, 272	20, 480	8192	0.10		(1.35, 48.66)	(1.56, 4.96)
Igea (Fig. 6)	(4532, 9, 060)	6016	3584	1024	0.21		(1.61, 12.25)	(1.58, 4.69)
Bunny (Fig. 6)	(14, 131, 28, 258)	2752	1472	128	0.20		(1.44, 10.08)	(1.98, 4.18)

Acknowledgments Y. Yu, J. Liu and Y. Zhang were supported in part by Honda funds. We also acknowledge the open source scientific library Eigen and its developers.

References

1. Ahrens, J., Geveci, B., Law, C.: Paraview: an end-user tool for large data visualization. *Visualization Handbook*, vol. 717 (2005)
2. Blacker, T.D., Stephenson, M.B.: Paving: a new approach to automated quadrilateral mesh generation. *Int. J. Numer. Methods Eng.* **32**(4), 811–847 (1991)
3. Corporation, L.S.T.: Ls-dyna keyword user's manual (2007)
4. Delaunay, B.N.: Sur la sphere vide. *Izvestia Akademii Nauk SSSR, Otdelenie Matematicheskikh I Estestvennykh Nauk* **7**(793–800), 1–2 (1934)
5. Eck, M., DeRose, T., Duchamp, T., Hoppe, H., Lounsbery, M., Stuetzle, W.: Multiresolution analysis of arbitrary meshes. In: *Proceedings of the 22nd Annual Conference on Computer Graphics and Interactive Techniques*, pp. 173–182 (1995)
6. Floater, M.S.: Parametrization and smooth approximation of surface triangulations. *Comput. Aided Geom. Design* **14**(3), 231–250 (1997)
7. Folwell, N., Mitchell, S.: Reliable whisker weaving via curve contraction. *Eng. Comput.* **15**(3), 292–302 (1999)
8. Frey, P.J., Borouchaki, H., George, P.L.: Delaunay tetrahedralization using an advancing-front approach. In: *5th International Meshing Roundtable*, pp. 31–48. Citeseer (1996)
9. Gregson, J., Sheffer, A., Zhang, E.: All-Hex mesh generation via volumetric polycube deformation. *Comput. Graph. Forum* **30**(5), 1407–1416 (2011)
10. Guennebaud, G., Jacob, B.: Eigen v3 (2010). <http://eigen.tuxfamily.org>
11. He, Y., Wang, H., Fu, C., Qin, H.: A divide-and-conquer approach for automatic polycube map construction. *Comput. Graph.* **33**(3), 369–380 (2009)
12. Hu, K., Zhang, Y., Liao, T.: Surface segmentation for polycube construction based on generalized centroidal Voronoi tessellation. *Comput. Methods Appl. Mech. Eng.* **316**, 280–296 (2017)
13. Hu, K., Zhang, Y.J.: Centroidal Voronoi tessellation based polycube construction for adaptive all-hexahedral mesh generation. *Comput. Methods Appl. Mech. Eng.* **305**, 405–421 (2016)
14. Hu, K., Zhang, Y.J., Xu, G.: CVT-based 3D image segmentation and quality improvement of tetrahedral/hexahedral meshes using anisotropic Giaquinta-Hildebrandt operator. *Comput. Methods Biomech. Biomed. Eng. Imaging Vis.* **6**(3), 331–342 (2018)
15. Khan, D., Plopski, A., Fujimoto, Y., Kanbara, M., Jabeen, G., Zhang, Y., Zhang, X., Kato, H.: Surface remeshing: a systematic literature review of methods and research directions. *IEEE Trans. Vis. Comput. Graph.* (2020). <https://doi.org/10.1109/TVCG.2020.3016645>
16. Lai, Y., Liu, L., Zhang, Y.J., Chen, J., Fang, E., Lua, J.: Rhino 3D to Abaqus: a T-spline based isogeometric analysis software framework. In: *Advances in Computational Fluid-Structure Interaction and Flow Simulation*, pp. 271–281 (2016)
17. Lai, Y., Zhang, Y.J., Liu, L., Wei, X., Fang, E., Lua, J.: Integrating CAD with Abaqus: a practical isogeometric analysis software platform for industrial applications. *Comput. Math. Appl.* **74**(7), 1648–1660 (2017)
18. Li, A., Chai, X., Yang, G., Zhang, Y.J.: An isogeometric analysis computational platform for material transport simulation in complex neurite networks. *Mol. Cell. Biomech.* **16**(2), 123 (2019)
19. Liang, X., Zhang, Y.: An octree-based dual contouring method for triangular and tetrahedral mesh generation with guaranteed angle range. *Eng. Comput.* **30**(2), 211–222 (2014)
20. Lin, J., Jin, X., Fan, Z., Wang, C.: Automatic polycube-maps. In: *Advances in Geometric Modeling and Processing. Lecture Notes in Computer Science*, vol. 4975, pp. 3–16. Springer Berlin/Heidelberg (2008)
21. Liu, L., Zhang, Y., Hughes, T.J., Scott, M.A., Sederberg, T.W.: Volumetric T-spline construction using Boolean operations. *Eng. Comput.* **30**(4), 425–439 (2014)

22. Liu, L., Zhang, Y., Liu, Y., Wang, W.: Feature-preserving T-mesh construction using skeleton-based polycubes. *Comput. Aided Design* **58**, 162–172 (2015)
23. Lohner, R., Parikh, P.: Three-dimensional grid generation by the advancing front method. *Int. J. Numer. Methods Fluids* **8**, 1135–1149 (1988)
24. Meshkat, S., Talmor, D.: Generating a mixed mesh of hexahedra, pentahedra and tetrahedra from an underlying tetrahedral mesh. *Int. J. Numer. Methods Eng.* **49**(1–2), 17–30 (2000)
25. Meyers, R.J., Tautges, T.J., Tuchinsky, P.M.: The “hex-tet” hex-dominant meshing algorithm as implemented in cubit. In: *International Meshing Roundtable*, pp. 151–158. Citeseer (1998)
26. Nieser, M., Reitebuch, U., Polthier, K.: Cubecover—parameterization of 3D volumes. *Comput. Graph. Forum* **30**(5), 1397–1406 (2011)
27. Owen, S.J.: A survey of unstructured mesh generation technology. In: *International Meshing Roundtable*, Dearborn, MI, vol. 194, pp. 4135–4195 (1998)
28. Owen, S.J., Saigal, S.: H-morph: An indirect approach to advancing front hex meshing. *Int. J. Numer. Methods Eng.* **49**(1–2), 289–312 (2000)
29. Pan, Q., Xu, G., Zhang, Y.: A unified method for hybrid subdivision surface design using geometric partial differential equations. In: *A Special Issue of Solid and Physical Modeling 2013 in Computer Aided Design*, vol. 46, pp. 110–119 (2014)
30. Price, M.A., Armstrong, C.G.: Hexahedral mesh generation by medial surface subdivision: part II. Solids with flat and concave edges. *Int. J. Numer. Methods Eng.* **40**(1), 111–136 (1997)
31. Price, M.A., Armstrong, C.G., Sabin, M.A.: Hexahedral mesh generation by medial surface subdivision: Part I. Solids with convex edges. *Int. J. Numer. Methods Eng.* **38**(19), 3335–3359 (1995)
32. Qian, J., Zhang, Y.: Automatic unstructured all-hexahedral mesh generation from B-Reps for non-manifold CAD assemblies. *Eng. Comput.* **28**(4), 345–359 (2012)
33. Qian, J., Zhang, Y., O’Connor, D.T., Greene, M.S., Liu, W.K.: Intersection-free tetrahedral meshing from volumetric images. *Comput. Methods Biomech. Biomed. Eng. Imaging Vis.* **1**(2), 100–110 (2013)
34. Qian, J., Zhang, Y., Wang, W., Lewis, A.C., Qidwai, M.A.S., Geltmacher, A.B.: Quality improvement of non-manifold hexahedral meshes for critical feature determination of microstructure materials. *Int. J. Numer. Methods Eng.* **82**(11), 1406–1423 (2010)
35. Schneiders, R.: A grid-based algorithm for the generation of hexahedral element meshes. *Eng. Comput.* **12**(3–4), 168–177 (1996)
36. Schneiders, R.: An algorithm for the generation of hexahedral element meshes based on an octree technique. In: *6th International Meshing Roundtable* pp. 195–196 (1997)
37. Seveno, E., et al.: Towards an adaptive advancing front method. In: *6th International Meshing Roundtable*, pp. 349–362 (1997)
38. Shephard, M.S., Georges, M.K.: Automatic three-dimensional mesh generation by the finite octree technique. *Int. J. Numer. Methods Eng.* **32**(4), 709–749 (1991)
39. Staten, M., Kerr, R., Owen, S., Blacker, T.: Unconstrained paving and plastering: progress update. *Proceedings of 15th International Meshing Roundtable* pp. 469–486 (2006)
40. Tarini, M., Hormann, K., Cignoni, P., Montani, C.: Polycube-maps. *ACM Trans. Graph.* **23**(3), 853–860 (2004)
41. Teng, S.H., Wong, C.W.: Unstructured mesh generation: theory, practice, and perspectives. *Int. J. Comput. Geom. Appl.* **10**(3), 227–266 (2000)
42. Wang, W., Zhang, Y., Liu, L., Hughes, T.J.R.: Trivariate solid T-spline construction from boundary triangulations with arbitrary genus topology. *Comput. Aided Design* **45**(2), 351–360 (2013)
43. Wang, W., Zhang, Y., Scott, M.A., Hughes, T.J.R.: Converting an unstructured quadrilateral mesh to a standard T-spline surface. *Comput. Mech.* **48**(4), 477–498 (2011)
44. Wang, W., Zhang, Y., Xu, G., Hughes, T.J.R.: Converting an unstructured quadrilateral/hexahedral mesh to a rational T-spline. *Comput. Mech.* **50**(1), 65–84 (2012)
45. Wei, X., Zhang, Y., Hughes, T.J.R.: Truncated hierarchical tricubic C^0 spline construction on unstructured hexahedral meshes for isogeometric analysis applications. *Comput. Math. Appl.* **74**(9), 2203–2220 (2017)

46. Wei, X., Zhang, Y.J., Toshniwal, D., Speleers, H., Li, X., Manni, C., Evans, J.A., Hughes, T.J.: Blended B-spline construction on unstructured quadrilateral and hexahedral meshes with optimal convergence rates in isogeometric analysis. *Comput. Methods Appl. Mech. Eng.* **341**, 609–639 (2018)
47. Xie, J., Xu, J., Dong, Z., Xu, G., Deng, C., Mourrain, B., Zhang, Y.J.: Interpolatory Catmull-Clark volumetric subdivision over unstructured hexahedral meshes for modeling and simulation applications. *Comput. Aided Geom. Design* **80**, 101867 (2020)
48. Xu, G., Ling, R., Zhang, Y.J., Xiao, Z., Ji, Z., Rabczuk, T.: Singularity structure simplification of hexahedral meshes via weighted ranking. *Comput. Aided Design* **130**, 102946 (2021)
49. Yamakawa, S., Shimada, K.: Fully-automated hex-dominant mesh generation with directionality control via packing rectangular solid cells. *Int. J. Numer. Methods Eng.* **57**(15), 2099–2129 (2003)
50. Yu, Y., Liu, H., Qian, K., Yang, H., McGehee, M., Gu, J., Luo, D., Yao, L., Zhang, Y.J.: Material characterization and precise finite element analysis of fiber reinforced thermoplastic composites for 4D printing. *Comput. Aided Design* **122**, 102817 (2020)
51. Yu, Y., Wei, X., Li, A., Liu, J., He, J., Zhang, Y.J.: HexGen and Hex2Spline: polycube-based hexahedral mesh generation and spline modeling for isogeometric analysis applications in LS-DYNA. In: Springer INdAM Series: Proceedings of INdAM Workshop “Geometric Challenges in Isogeometric Analysis.” (2021)
52. Yu, Y., Zhang, Y.J., Takizawa, K., Tezduyar, T.E., Sasaki, T.: Anatomically realistic lumen motion representation in patient-specific space–time isogeometric flow analysis of coronary arteries with time-dependent medical-image data. *Comput. Mech.* **65**(2), 395–404 (2020)
53. Zhang, Y.: Challenges and advances in image-based geometric modeling and mesh generation. In: *Image-Based Geometric Modeling and Mesh Generation*, pp. 1–10. Springer, Berlin (2013)
54. Zhang, Y.: *Geometric Modeling and Mesh Generation from Scanned Images*. Chapman and Hall/CRC (2016)
55. Zhang, Y., Bajaj, C.L.: Adaptive and quality quadrilateral/hexahedral meshing from volumetric data. *Comput. Methods Appl. Mech. Eng.* **195**(9–12), 942–960 (2006)
56. Zhang, Y., Bajaj, C.L., Sohn, B.S.: 3D finite element meshing from imaging data. *Comput. Methods Appl. Mech. Eng.* **194**(48–49), 5083–5106 (2005)
57. Zhang, Y., Bajaj, C.L., Xu, G.: Surface smoothing and quality improvement of quadrilateral/hexahedral meshes with geometric flow. *Commun. Numer. Methods Eng.* **25**(1), 1–18 (2009)
58. Zhang, Y., Bazilevs, Y., Goswami, S., Bajaj, C.L., Hughes, T.J.R.: Patient-specific vascular NURBS modeling for isogeometric analysis of blood flow. *Comput. Methods Appl. Mech. Eng.* **196**(29–30), 2943–2959 (2007)
59. Zhang, Y., Hughes, T.J.R., Bajaj, C.L.: An automatic 3D mesh generation method for domains with multiple materials. *Comput. Methods Appl. Mech. Eng.* **199**(5–8), 405–415 (2010)
60. Zhang, Y., Liang, X., Ma, J., Jing, Y., Gonzales, M.J., Villongco, C., Krishnamurthy, A., Frank, L.R., Nigam, V., Stark, P., Others: An atlas-based geometry pipeline for cardiac Hermite model construction and diffusion tensor reorientation. *Med. Image Anal.* **16**(6), 1130–1141 (2012)
61. Zhang, Y., Liang, X., Xu, G.: A robust 2-refinement algorithm in octree and rhombic dodecahedral tree based all-hexahedral mesh generation. *Comput. Methods Appl. Mech. Eng.* **256**, 562–576 (2013)
62. Zhang, Y., Qian, J.: Resolving topology ambiguity for multiple-material domains. *Comput. Methods Appl. Mech. Eng.* **247**, 166–178 (2012)
63. Zhang, Y., Wang, W., Hughes, T.J.R.: Solid T-spline construction from boundary representations for genus-zero geometry. *Comput. Methods Appl. Mech. Eng.* **249**, 185–197 (2012)
64. Zhang, Y., Wang, W., Hughes, T.J.R.: Conformal solid T-spline construction from boundary T-spline representations. *Comput. Mech.* **51**(6), 1051–1059 (2013)

Mesh Adaptivity in the Framework of the Cartesian Grid Finite Element Method, cgFEM



Juan José Ródenas, Enrique Nadal, José Albelda, and Manuel Tur

Abstract This work aims at describing the synergistic effect of the use of the Cartesian Grid Finite Element Method (cgFEM), a Fictitious Domain Method, and h -adaptive refinement processes. cgFEM combines the use of a hierarchical data structure with the mesh-geometry independence and the use of Cartesian Grids. The combination of these features allows the simplified and efficient implementation of h -adaptive refinement processes for many relevant applications such as: (i) mesh refinements based on error indicators to control de quality of the solution; (ii) automatic creation of h -adapted meshes for the efficient accuracy control of the FE simulations in structural shape optimization processes; (iii) mesh refinements for image-based simulations in biomechanics in order to adapt the analysis mesh to the features of the image; or (iv) mesh refinements to increase the sharpness of solutions, whose complexity is mesh-independent, obtained with topology optimization algorithms. These applications will be described along this work along with illustrative examples to show the relevance of h -adaptivity in the framework of the cgFEM.

1 Introduction

The Cartesian Grid Finite Element Method (cgFEM) is a methodology based on the Finite Element Method (FEM) where the problem's domain Ω is embedded into an embedding domain Ω^* . Hence, cgFEM can be classified under the umbrella terms of Fictitious Domain Methods (FDM) or Immersed Boundary Methods (IBM) that include methods like the CutFEM [1] or the Finite Cell FEM (FCM) [2, 3]. As opposed to the standard implementations of FEM, where the mesh is geometry-conforming, in these methods, the mesh and the geometry are independent since the

J. J. Ródenas (✉) · E. Nadal · J. Albelda · M. Tur

Instituto de Ingeniería Mecánica y Biomecánica, Universitat Politècnica de València, Valencia, Spain

e-mail: jjrodena@mcm.upv.es; ennaso@upvnet.upv.es; jalbelda@mcm.upv.es; matuva@mcm.upv.es

embedding domain is the domain to be meshed. In particular, for the 3D case, in cgFEM the embedding domain is a rectangular cuboid (a rectangle in the 2D case) that is easily discretized using Cartesian grids. This kind of meshes can be easily refined by element splitting. This guarantees that the meshes are always made up of regular hexahedra (regular quadrilaterals in the 2D case) and allows the definition of very efficient hierarchical data structures based on the Cartesian mesh structure. Therefore, the creation of h -adaptive meshes is simple and efficient, playing a very important role in the cgFEM context. CgFEM combines the hierarchical mesh and data structure of Cartesian grids with the mesh-geometry independence and with h -adaptive refinement process. This combination generates a synergistic effect that allows cgFEM to efficiently perform accurate FE analyses. This contribution precisely aims to describe how the Cartesian mesh and the hierarchical data structure of cgFEM simplify and speed up h -adaptive analyses, which will be of special relevance in structural components optimization processes, and also how the h -adaptive refinements have provided cgFEM with the possibility of carrying out new types of simulations, in particular in the field of biomechanics.

The cgFEM also has number of drawbacks arising from the fact that the boundary of the problem domain $\partial\Omega$ is not conforming with the mesh boundary and, in general, there are no nodes over the domain's boundary. This fact has strong implications in the imposition of the essential boundary conditions that will imply the use of mortar methods [4, 5]. Mortar methods require defining a Lagrange multipliers field whose discretization is crucial to guarantee the stability and well-posedness of the solution. The Lagrange multipliers discretization depends on the discretization mesh and also on the intersection of the mesh and the problem's domain, which is, in general, arbitrary. This has already been the objective of a number of studies that have provided stabilization tools for the Lagrange multipliers space to guarantee the well-posedness of the problem in FEM context, see for instance [1, 4, 5]. On the other hand, due to the arbitrary intersection pattern, also the solution field should be stabilized in order to control the condition number of the problem [6–8].

2 The Cartesian Grid Finite Element Method

This contribution deals with the 2D/3D linear elasticity problem with isotropic material considering the cgFEM methodology. The notation used is presented in this section where the Cauchy stress field is denoted as σ , the displacement field as $\mathbf{u}$, and the strain field as ϵ , all these fields being defined over the domain $\Omega \subset \mathbb{R}^d$, $d = 2, 3$, with boundary denoted by $\partial\Omega$. Prescribed tractions denoted by $\mathbf{t}$ are imposed over the part Γ_N of the boundary, while displacements denoted by $\bar{\mathbf{u}}$ are prescribed over the complementary part Γ_D of the boundary. Body loads are denoted as $\mathbf{b}$.

In the cgFEM context, the problem is solved in the embedding domain Ω^* , $\Omega \subset \Omega^*$, although the energy outside Ω is null. Thus, the variational form of the problem can be written as:

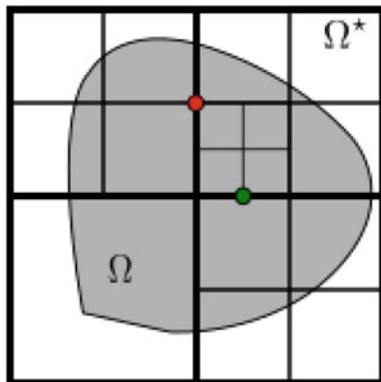
$$\begin{aligned}
 &\text{Find } \mathbf{u} \in V : \forall \mathbf{v} \in V \\
 &a(\mathbf{u}, \mathbf{v}) = l(\mathbf{v}) \quad \text{where} \\
 &a(\mathbf{u}, \mathbf{v}) = \int_{\Omega} \boldsymbol{\varepsilon}(\mathbf{u}) : \boldsymbol{\sigma}(\mathbf{v}) \, d\Omega \\
 &l(\mathbf{v}) = \int_{\Omega} \mathbf{b} \cdot \mathbf{v} \, d\Omega + \int_{\Gamma_N} \mathbf{t} \cdot \mathbf{v} \, d\Gamma,
 \end{aligned} \tag{1}$$

where $V_{\Omega} = \{\mathbf{v} \mid \mathbf{v} \in [H^1(\Omega)]^d\}$, $\boldsymbol{\sigma}(\mathbf{v}) = \mathbf{D} : \boldsymbol{\varepsilon}(\mathbf{v})$, being $\mathbf{D}$ the fourth order tensor relating the stress tensor with the strain tensor, defined as the symmetric gradient of the displacement field. It can be observed that problem (1) is not solvable since Dirichlet boundary conditions are not considered. cgFEM imposes the essential boundary conditions, $\bar{\mathbf{u}}$, via Lagrange multipliers [5]. The choice of the Lagrange multipliers space is critical and not evident in some situations, therefore in the cgFEM context a stabilization technique, also presented in [5], is used. Additionally, the solution space also has to be stabilized because $\partial\Omega$ can arbitrarily cut the elements, leading to ill-conditioning problems [8]. The continuous problem (1) is solved by using a discretization with linear or quadratic elements.

3 Mesh Refinement in cgFEM

The mesh refinement procedure in the cgFEM framework is based on element splitting: each Cartesian element to be refined, also called *parent* element, is split into elements (*child* elements) of a half characteristic size, that is, into 4 elements for the 2D case and into 8 for 3D. The *parent-children* relations have been used to create a hierarchical data structure that allows reusing many calculation. This *parent-children* relations define different mesh levels, where the *Level-0* mesh consist of a single element that defines the embedding domain Ω^* , the *Level-1* mesh is made up of the *child* elements obtained by element splitting of the element of the *Level-0* mesh, and so forth. The implementation only allows a one-level difference between adjacent elements. Figure 1 shows an example of a 2D *h*-adapted mesh of bilinear elements, considering the problem domain Ω and the meshing domain Ω^* . The node highlighted with a red dot is a standard node. However, the node highlighted with a green dot shows that, because of the refinement based on element splitting, the mesh is non-conforming. Therefore, C^0 continuity is enforced using multi-point constraints (MPCs) [9], that is, making the solution at non-conforming element edges dependent on the coarser discretization.

Fig. 1 Sample of a 2D problem configuration considering a h -adapted mesh. The red dot represents a standard node while the green node represents a node with multi-point constraints



The mesh refinement process in cgFEM is guided by different criteria defined according to the objective of the problem to be solved. The main criteria considered in cgFEM, that will be described in detail in the following sections, are:

- *Geometry-based refinement.* Mesh refinement is used to control the geometrical complexity of $\partial\Omega$ into each element cut by the boundary (Sect. 5) and also to facilitate the numerical integration of these elements (Sect. 4).
- *Discretization error.* In this case the mesh refinement is guided by an error indicator that quantifies the quality of the numerical solution and that will allow to generate the optimal mesh configuration for a predefined error level. This mesh refinement criterion is used in structural analysis (Sect. 5), in structural shape optimization (Sect. 6) and in structural topology optimization (Sect. 8).
- *h -adaptivity by projection between geometries.* In the context of structural shape optimization, the discretization error must be taken into account in the analyses of the different geometrical configurations considered during the process. Projection techniques can be used to project information obtained in a geometry previously analysed to the new geometry to be evaluated. This projected information can be then be used to directly obtain the h -adapted mesh required to obtain the solution of the new geometry with the prescribed accuracy, thus eliminating the first steps of the refinement process and speeding up the optimization process (Sect. 6).
- *Image-based refinement.* When the object to be analysed is described by an image, each element of the mesh will have a certain number of voxels, each of them having a different colour and, therefore, different associated material properties. The image-based refinement is used to limit the range of variation of the voxel values into each element (Sect. 7).
- *Boundary sharpening in topology optimization.* Density-based topology optimization produces structures with a blurred zone whose size depends, among other parameters, on the mesh size. A specific h -adaptive mesh refinement procedure has been developed to decrease the size of this zone, thus producing a sharp description of the boundary (Sect. 8).

4 Element Integration

One of the key aspects of IBMs is the procedure for the evaluation of volume integrals in elements cut by $\partial\Omega$. The procedure used in cgFEM, that allows considering volumes defined by NURBS and T-Splines, is described in [10] and essentially consists of the following phases:

1. Intersection of the CAD surfaces with the axes of the Cartesian mesh.
2. Creation of tetrahedra inside the element to generate integration subdomains. An element tetrahedralization, initially defined by tetrahedron with flat faces, is created from the intersections with the mesh axes.
3. Selection of tetrahedron placed into the domain.
4. Integration in the selected integration subdomains. To consider the real surface of the component, in this last step, flat-faced tetrahedra are transformed into curved-faced tetrahedra that describe the CAD surface of the component. This can be done with different degrees of approximation. The linear approximation would directly consider the flat faces determined in the previous step. Polynomial approximations of a higher order can also be considered. However, to avoid geometric modelling errors, cgFEM allows to consider the exact geometry defined by the CAD surface, up to accuracy provided by the numerical integration.

The second step requires the tetrahedralization of the volume of the hexahedron considering how the CAD surface cuts the hexahedron. The procedure for the tetrahedralization is inspired by the Marching Cubes (MC) algorithm [11], which is widely used in graphical applications to represent surfaces. Considering that each of the edges of the hexahedron can be cut at most once, each of the 8 vertices of a hexahedron can have 2 states (to be located inside or outside the surface), so there are $2^8 = 256$ possible types of intersections. Taking symmetry considerations into account, these 256 cases can be reduced to 14 basic cases. The MC algorithm proposes a coding procedure for these cases, so that, once the cut case has been identified, the surface can be quickly represented. If the MC algorithm generates a triangulation of the surface in the element based on the cutting patterns, in cgFEM we have extended this idea to generate a parametrized tetrahedralization of the element's domain based on these patterns. The parametrized tetrahedralization procedure developed for cgFEM allows to significantly speed up step 2 of the integration process since tetrahedralization is eliminated and replaced by a simple process of cutting patterns identification.

Figure 2 shows the 14 configurations considered by the MC algorithm, which correspond to cases in which each edge is cut, at most, once. As shown, only 7 of the 14 configurations correspond to cases in which the hexahedron is cut by a single surface. These are the only cases considered in cgFEM to define tetrahedralization patterns, as shown in the Figure.

If, in an element, we identify a case with any of the other 7 cutting configurations of the MC algorithm not considered in cgFEM, a case in which the edges of the hexahedron are cut more than once, or a case in which the CAD surface does not cut

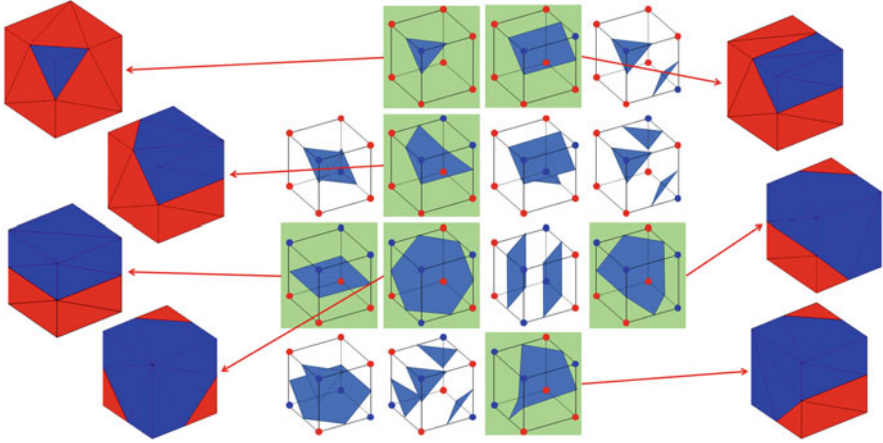


Fig. 2 The 14 cases used by the MC algorithm for surface representation and tetrahedralization patterns of the seven cutting configurations considered in cgFEM

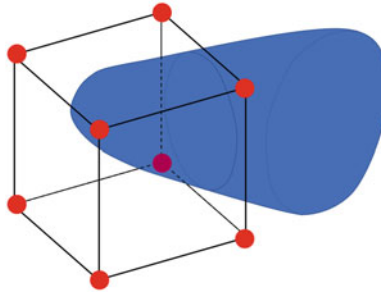


Fig. 3 CAD surface intersecting one of the faces of the hexahedron but not cutting any edge

to any of the edges, but cuts an element face, as in Fig. 3, an alternative procedure is used. In these cases, the element is refined and the procedure is repeated with each of the *children* elements created from the *parent* element. The subdivision procedure is recursively repeated as long as there are cutting patterns other than the 7 tetrahedralization patterns considered in cgFEM, or until the minimum prescribed element size is reached. An exclusive Delaunay's tetrahedralization is performed on elements of this prescribed minimum size that cannot be integrated by any of these 7 parametrized tetrahedralization patterns.

This procedure based on the use of parametrized tetrahedralization patterns and mesh refinement used in cgFEM considerably reduces the time needed to perform numerical integration, while simplifying the process to create geometrically *h*-adapted meshes that will be described in the next section.

5 H-adaptive cgFEM Analysis

The discretization error in the FEM is the error associated to the approximation to the solution based on the polynomial interpolation functions used within each element. This error is associated with the finite size of the elements and decreases as the mesh is refined. The cgFEM uses two criteria to determine which elements have to be refined to improve the quality of the solution. The first of them, the *geometry-based refinement*, focuses on elements cut by $\partial\Omega$ and aims to reduce the geometric complexity of $\partial\Omega$ within these elements. The second criterion, the *solution-based refinement*, aims to make the solution smooth enough within the elements, and will consider elements both, cut by $\partial\Omega$ and internal. The methodology used in cgFEM to run *h*-adaptive analyses is, therefore, a two-steps methodology which considers an initial *geometric refinement* phase that will be followed by a *solution-based refinement* phase, as described below.

5.1 Geometry-Based Mesh Refinement

The analysis starts adapting the dimensions of the embedding domain (Ω^*) to the problem domain Ω . A uniform element size preliminary mesh (not used for FE analysis), defined by the user, is created as the first step of the analysis process. This preliminary mesh is then intersected with the problem domain defined by the CAD model. The first analysis mesh is created following a refinement process based on the geometry of the domain. The procedure consists in recursively refining the elements cut by $\partial\Omega$ where the boundary is too complex. There are basically two geometrical criterion to decide if an element has to be refined:

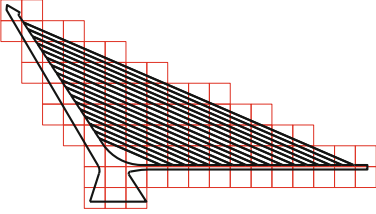
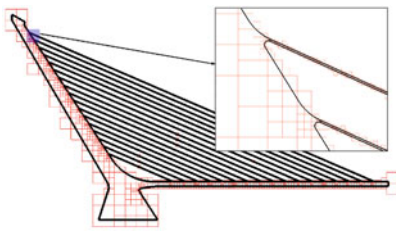
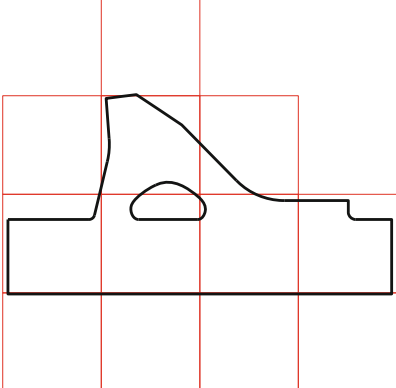
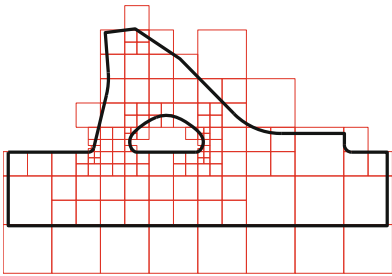
- *Integrability*: As already explained in the previous section, elements that cannot be integrated with any of the 7 cutting configurations of the MC algorithm considered in cgFEM, elements whose edges are cut more than once by $\partial\Omega$, or elements for which the CAD surface cuts an element face but does not cut to any of the edges will be refined.
- *Smoothness*: elements for which $\partial\Omega$ is not smooth enough within the element will also be refined.

The first mesh used for FE analysis is created as a result of this process. Table 1 shows examples of preliminary meshes and the first-analysis meshes for two 2D examples.

5.2 Solution-Based Refinement

After the FE solution of the first analysis mesh has been obtained, new meshes are created following a refinement procedure that takes into account the quality of the

Table 1 Comparison between the preliminary mesh and the geometrically adapted mesh using the curvature criterion

Preliminary mesh	1 st analysis mesh: geometrically adapted mesh
	
	

FE solution. This procedure aims at minimising the error in energy norm of the solution. The exact error in energy norm of the solution is given by:

$$||| \mathbf{e} |||_{\Omega}^2 := \int_{\Omega} (\boldsymbol{\sigma} - \boldsymbol{\sigma}^h)^T \mathbf{D}^{-1} (\boldsymbol{\sigma} - \boldsymbol{\sigma}^h) \, d\Omega \quad (2)$$

where $\boldsymbol{\sigma}^h$ is the FE stress field. Neglecting other error sources, $\mathbf{e} = \mathbf{u} - \mathbf{u}^h$ is the discretization error, being $\mathbf{u}^h$ the FE displacement field. In order to estimate the error in energy norm we use the Zienkiewicz and Zhu (ZZ) error indicator (3), presented in [12], where $\boldsymbol{\sigma}^*$ is an improved stress field obtained from $\boldsymbol{\sigma}^h$ but more accurate than $\boldsymbol{\sigma}^h$:

$$||| \mathbf{e} |||_{\Omega}^2 \approx \mathcal{E}_{ZZ}^2 := \int_{\Omega} (\boldsymbol{\sigma}^* - \boldsymbol{\sigma}^h)^T \mathbf{D}^{-1} (\boldsymbol{\sigma}^* - \boldsymbol{\sigma}^h) \, d\Omega. \quad (3)$$

Obviously, the accuracy of the error estimation is directly related to the accuracy of $\boldsymbol{\sigma}^*$. There are many techniques to obtain the recovered stress field such as the Superconvergent Patch Recovery (SPR) technique [13, 14] which was followed by several works aimed at improving its quality [15–17]. Ródenas et al. proposed, for

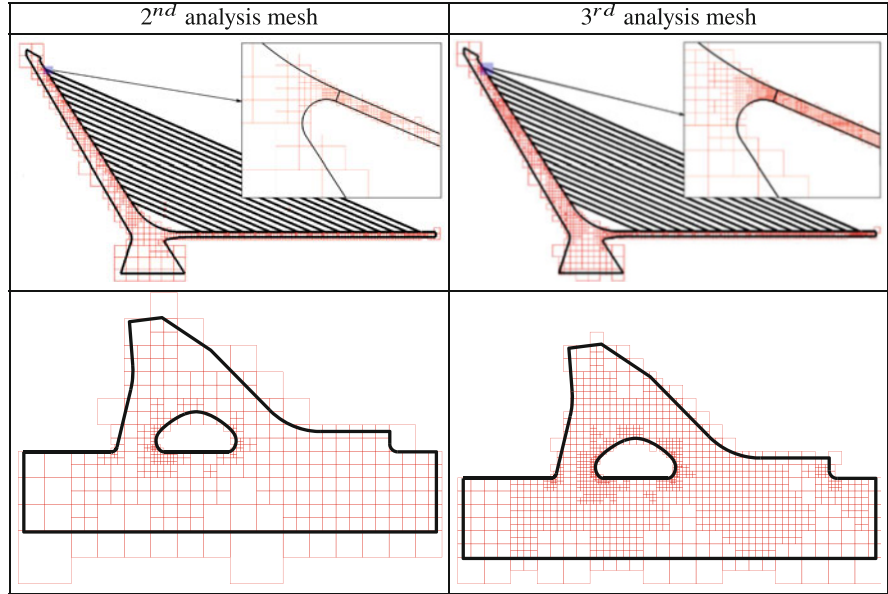
the FEM framework, the SPR-C technique, where equilibrium and compatibility constraints are imposed to the locally recovered solution. The technique, that produces very accurate results, was later adapted to the eXtended Finite Element Method (XFEM) [18] and the cgFEM framework [19].

SPR-based methods do not provide error bounding properties because the recovered solution is not fully equilibrated. However, in the FEM context, Díez et al. [20] proposed a methodology to obtain computable upper bounds of the error in energy norm considering the quasi-equilibrated stress recovered field obtained with the SPR-C technique. This technique was also adapted to the XFEM framework [21]. More recently a new improvement over the SPR-C technique provide a procedure to obtain error bounds for recovery-based error estimators [22].

In this contribution we are not considering error bounding techniques but rather the use of the SPR-C technique as an accurate error indicator to guide the mesh refinement process.

Particularizing (3) at element level, we would obtain the estimation of the error in energy norm at each element. With that information, and using a mesh optimization criterion based on the equidistribution of the error in the elements of the mesh to be created [23], we obtain the new levels (sizes) of the elements in each zone. Examples of analysis meshes obtained by this procedure are represented in Table 2. The refinement process ends when the error indicator is below the threshold defined by the user: $\eta_{est} = \frac{\mathcal{E}_{ZZ}}{\sqrt{|||u^h|||_{\Omega}^2 + \mathcal{E}_{ZZ}}}100 \leq \eta_{obj}$.

Table 2 Second and third analysis mesh obtained using the error estimation information following the first meshes represented in Table 1



6 H-Refinement in Shape Optimization Processes

Since FEM results are used to make design decisions, proper mesh refinement is necessary to obtain results of the desired accuracy. Shape optimization of structural components [24] is an example of this issue: the iterative optimization process evolves trying to find the optimal solution using the results of the structural analysis of previously considered geometric configurations. As a consequence of this, if the results on which the evolution of the optimization process is based on are not of sufficient accuracy, the iterative process will have a slow convergence rate to the solution or will converge toward a result that is not optimal or that is unfeasible [25]. It is therefore necessary to guarantee a certain level of precision in the results of the FE analyses used in these processes. This implies, for instance, the use of adaptive meshes. Even though adaptive meshes can provide computationally efficient finite element models, they involve a very high computational cost in optimization processes due to the high number of geometric configurations to be analysed. The cgFEM methodology can be used in two main ways to reduce the computational cost of the h -adaptive analyses in shape optimization processes: (a) through data-sharing and (b) through data-projection.

6.1 Data-Sharing in Shape Optimization Processes

The use of cgFEM in optimization process considering adaptive mesh refinement can provide computational benefits to the optimization process thanks to the possibility of sharing information between meshes of the same geometry, through a process called vertical data sharing, and between geometries, transversally, through a process called horizontal data sharing:

- **Vertical data sharing.** Analysing a geometry will require creating and analysing meshes that will be successively h -adapted until the prescribed error level is reached. The creation of new meshes in cgFEM is computationally very efficient since the mesh refinement is based on element subdivision. Thus, the generation of the new mesh only requires the substitution of the element to be refined by the ‘child’ elements obtained by subdivision of the ‘parent’ element and, if necessary, the use of MPCs to enforce C^0 continuity. Once the new mesh has been generated, the process of creating the system of equations is very efficient since it is only necessary to eliminate the matrices of the elements to be removed and to include the matrices of the ‘child’ elements that replace them. If the material considered is homogeneous and has linear behaviour, the stiffness matrices of all the interior elements will be proportional to each other, being the scaling factor a function of the size of the elements. Therefore, the only stiffness matrices to be evaluated would be those corresponding to new elements cut by the surface of the domain.

- **Horizontal data sharing** refers to the ability of cgFEM to share information transversally, i.e. between the different geometries considered throughout the optimization process. The highest computational cost associated to the creation of the system of equations to be solved for each geometry, considering linear elastic homogeneous material, is due to the evaluation of the stiffness matrices of elements cut by the surface. The domain's boundary will be made up of several surfaces expressed as a function of a set of parameters $\mathbf{a}$ defined by the designer. Each of the components of $\mathbf{a}$ will be a design parameter, for example, a radius, a distance, a coordinate of a geometrical point, etc. This parametric definition of the domain's boundary, created by the designer, will be used to create different geometries by simply modifying the values of the a_m parameters that define $\mathbf{a}$. The parametric definition of the boundary will modify some of these surfaces while others, the fixed surfaces, will remain unmodified by the parameters used to define the geometry. When using cgFEM, since the same embedding domain is used, the element matrices of elements cut by these fixed surfaces can be shared between different geometries.

Figure 4 shows an example of these data-sharing processes. The images of this figure show the first two geometries analysed in an optimization process of a gravity dam. The external boundary of the dam and the lower horizontal straight line of the inner boundary are considered fixed. The rest of the internal boundary is defined as a function of the parameters of the geometric model; therefore, it is variable. In the images of this figure, the yellow colour represents internal elements whose stiffness matrices are obtained, simply, scaling the stiffness matrix of a reference Cartesian element. The green colour is used to represent elements whose stiffness matrix has to be evaluated in the mesh. Finally, the white colour represents elements whose stiffness matrix has already been evaluated in a previous mesh of the same

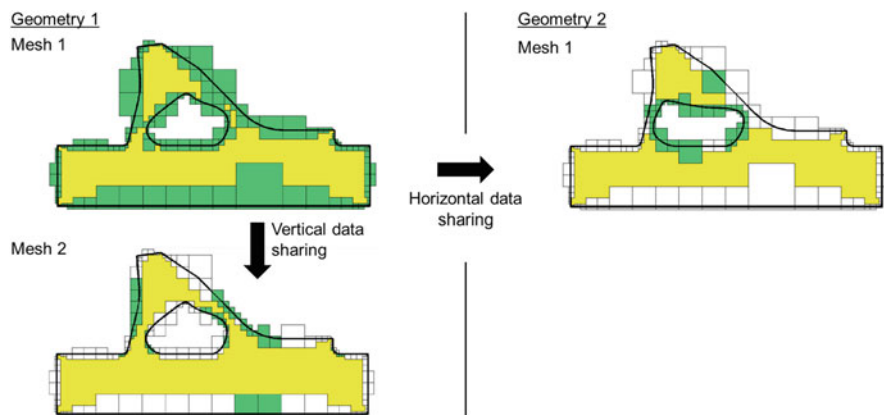


Fig. 4 2D example of data sharing in cgFEM. Green elements: element matrices have to be evaluated in the mesh. Yellow elements: internal elements whose stiffness matrices are obtained by scaling the stiffness matrix of a reference Cartesian element. White elements: element matrices evaluated in a previous mesh or geometry

geometry or in a previously analysed geometry. The figures on the left show the first geometry analysed. Two meshes have been considered for this geometry. To analyse the first mesh, it is necessary to create the stiffness matrices for all the contour elements. In the second mesh we observe that, using the vertical data sharing, it is only necessary to evaluate the stiffness matrices of some elements of the contour. The second geometry is shown on the right. It can be seen that most of the elements cut by fixed curves were already used in the analysis of the first geometry. Therefore, it is only necessary to evaluate the stiffness matrices of a few elements of the fixed curves and of the elements cut by the variable part of the internal boundary.

6.2 Data-Projection in Shape Optimization Processes

Ródenas et al. [25] proposed a methodology to alleviate the computational cost of h -adaptive analyses in shape optimization processes, that can be adapted to both, gradient-based and evolutionary optimization algorithms. This methodology was based on the use of an algorithm to automatically obtain adequate h -adapted meshes by projection from results obtained on previously evaluated geometrical configurations. The methodology is described below.

After a geometric configuration has been analysed with a specific FE mesh, an automatic mesh refinement procedure uses the estimation of the discretization error following the procedure presented in Sect. 5.2 to obtain a new h -adapted mesh of the prescribed accuracy level. Hence, the main idea of the methodology is to project the FE results required by the mesh refinement procedure from a geometry j , already analysed, to a new geometry $j + 1$ to be analysed, and use the mesh refinement procedures to directly obtain the h -adapted mesh required by geometry $j + 1$. Thus, the h -adapted mesh required to analyse the new geometry with the prescribed error level is obtained in one shot, avoiding the initial steps of the iterative refinement process.

Making use of the shape sensibility analysis [34], the following expression is used for the linear projection of data between geometries:

$$M_{j+1} \approx M_j + \sum \left(\frac{\partial M_j}{\partial a_m} \right) \cdot \Delta a_m \quad (4)$$

where M is the magnitude to be projected, a_m is the m -th parameter used to define the geometry and Δa_m is the increment of parameter a_m between geometries j and $j + 1$. In this expression, the value of $\frac{\partial M_j}{\partial a_m}$ is evaluated through shape sensitivity analysis or using finite differences.

Considering that the quality of the FE analysis is evaluated through the estimation of the discretization error in the energy norm, the information to be projected to geometry $j + 1$ is the following:

- Nodal coordinates of the finite element mesh used to analyse geometry j .

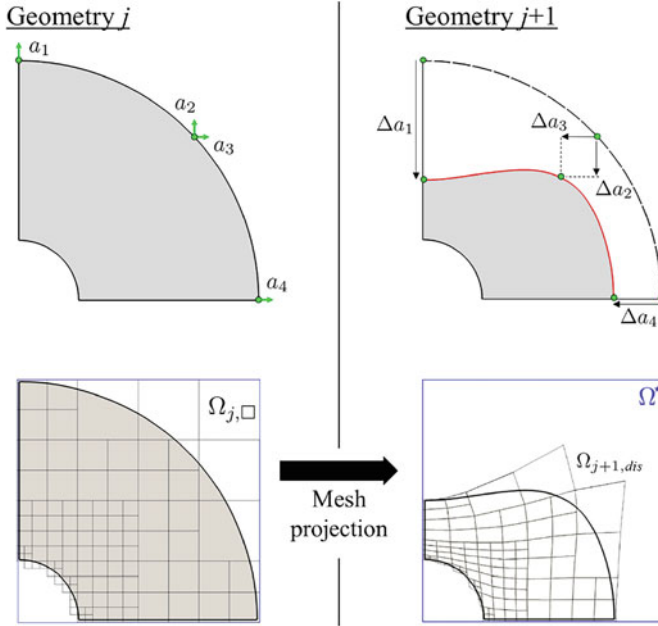


Fig. 5 2D example of mesh projection. Reproduced with permission from [26]

- Estimation of discretization error evaluated at each element of the mesh.
- Energy norm of the solution evaluated at each element of the mesh.

The efficiency of this methodology has been further enhanced using the characteristics of the Cartesian mesh and the hierarchical data structure of the cgFEM framework as described in [26]. The adaptation of the projection procedure to the cgFEM environment is not straightforward. As shown in the example displayed in Fig. 5, when the nodal coordinates of the Cartesian mesh used to analyse an original geometry ($\Omega_{j,\square}$) are projected to another geometry, the mesh obtained ($\Omega_{j+1,dis}$) will be made up of elements that will not belong to the original hierarchical Cartesian grid structure (hence, it will not be possible to use the horizontal data-sharing technique) and that, in general, will be distorted, thus even preventing the use of the most basic benefits provided by cgFEM.

We developed a procedure to solve this problem that allows obtaining a refined Cartesian mesh from the information of the non-Cartesian projected mesh. The procedure, schematically shown in Fig. 6, is based on projecting, into the embedding domain Ω^* , the size information of the elements of the new mesh to be created and, then, creating a Cartesian mesh with this information:

- The new element size evaluated at each element of $\Omega_{j+1,def}$ is assigned to the integration points of these elements. Note that these integration points can be trivially placed in elements of a Cartesian mesh.

- The starting point to create the refined Cartesian mesh is a uniform Cartesian mesh of the desired level (Level-2 in the case of Fig. 6). The integration points with the size information within each element of the Cartesian mesh will be identified.
- The elements of the Cartesian mesh will be recursively refined until the size of each element is smaller than the size defined by the integration points included in the element.

Once the process finishes after having considered all the necessary refinement levels (up to Level-5 in the case of the example of Fig. 6), an h -adapted Cartesian mesh $\Omega_{j+1,\square}$ is obtained, directly from the information obtained in the previous geometry.

Figure 7 shows an example of results obtained with this procedure in a 3D case. In this example, the estimated discretization error in energy norm for the reference geometry was $\eta_{est} = 16.9\%$. The h -adapted meshes for two other geometries, directly created with the information projected from the reference geometry are also shown in the figure. The objective was to create h -adapted meshes with a discretization error in energy norm $\eta_{obj} = 6.0\%$. After creating the h -adapted meshes and running the FE analysis, the error estimation procedure estimated a discretization error in energy norm $\eta_{est} = 5.6\%$ for the first geometry and $\eta_{est} = 6.3\%$ for the second geometry. Taking into account that these two meshes

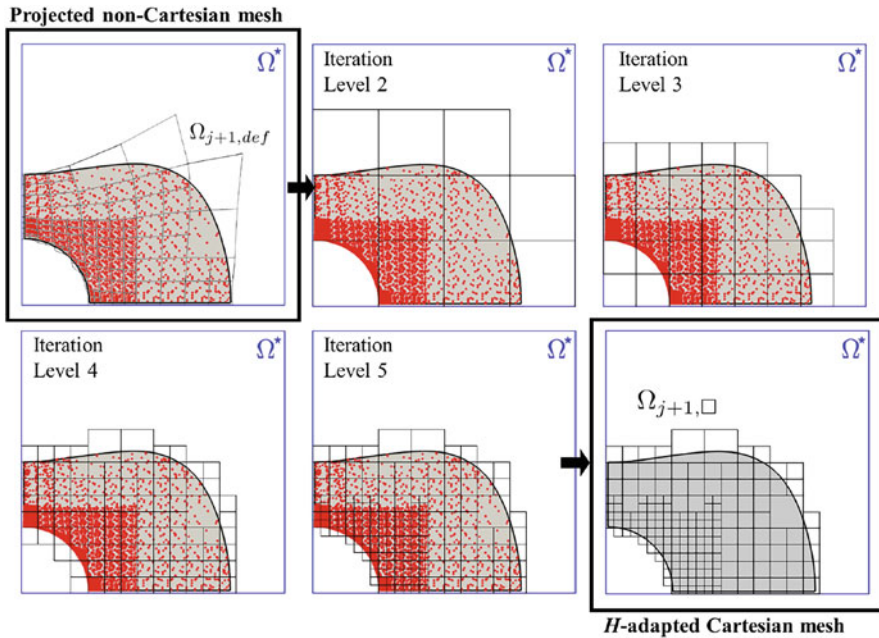


Fig. 6 Creation of Cartesian h -adapted mesh from projected non-Cartesian mesh. Reproduced with permission from [26]

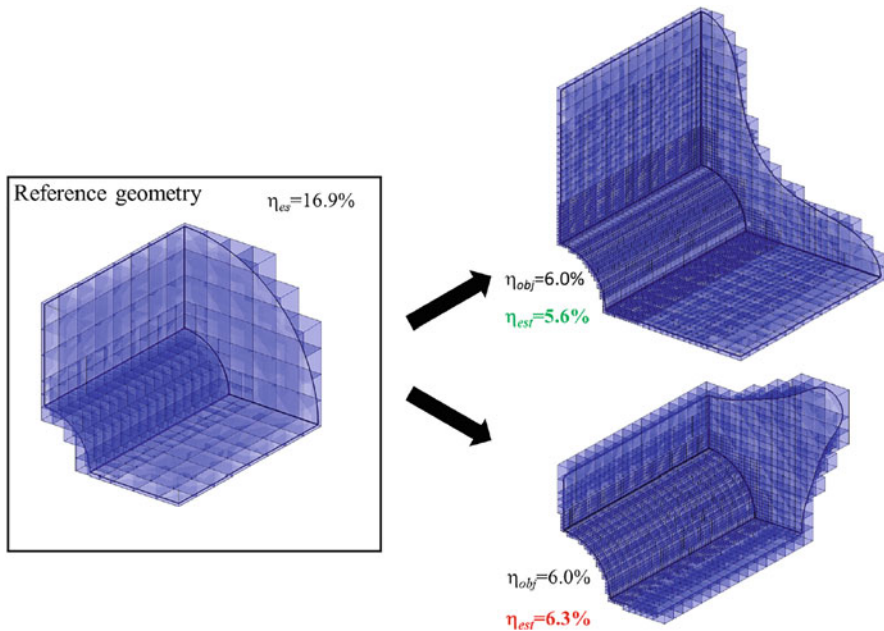


Fig. 7 3D example of h -adapted meshes created using information projected from a reference geometry

were created with information of a relatively different geometry analysed with a coarse mesh ($\eta_{est} = 16.9\%$), the results can be considered as very accurate. As the final estimated error in the first geometry was lower than the objective error, no further mesh refinement was required for this geometry. In the second case, the estimated error, $\eta_{est} = 6.3\%$, was higher than the objective error, $\eta_{obj} = 6.0\%$. Hence, this geometry required further refinement. The number of cases like this last case, where it is necessary to generate a new mesh, can be easily reduced by using data from reference geometries analysed with error levels closer to the objective error than in the case presented in Fig. 7, and specifying objective error levels slightly below the error level required for the FE analyses.

7 H-Refinement in Patient Specific Models

The use of the FEM has spread to many fields, including the field of biomechanics, where one of the first difficulties faced by the analysts is the generation of a finite element model from the available information, which usually comes in the form of a medical image (MI), like a CT-scan or a magnetic resonance.

Matrices are used to store the information of MIs, hence, this information has a Cartesian structure that can be easily handled by the cgFEM. The cgFEM has been

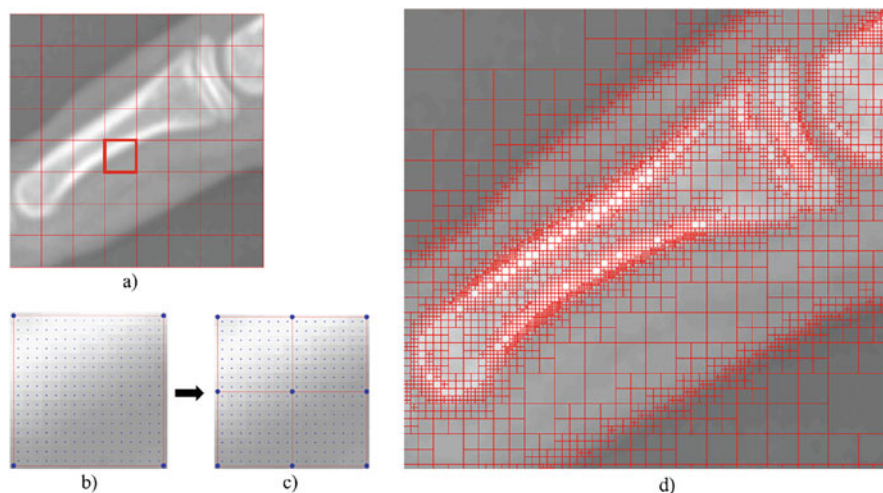


Fig. 8 2D example of automatic creation of h -adapted mesh from a MI. (a) Uniform Cartesian grid over 2D image. (b) Element with 16×16 pixels. (c) Element subdivision to reduce pixel-values range. (d) Final h -adapted mesh

adapted [27] to automate the creation of FE models from MIs to perform patient-specific structural analysis of bones and preoperative implant simulations. In this adaptation, the mesh refinement plays a key role.

If the model of an object is given by a MI (see Fig. 8a), cgFEM will first embed the MI into a uniform mesh with a certain number of voxels falling into each element (see Fig. 8b). In particular, the number of voxels along each Cartesian coordinate will be 2^n ($n \in \mathbb{N}$) so that, if an element is subdivided, it will always contain an integer number of voxels. Each voxel into the image can be considered as an integration sub-domain, where material properties are dictated by the data value of the pixel (grey level, Hounsfield value in CT scans...). This mechanical properties can be extracted from the literature in this field [28] that presents relations between the Hounsfield scale used in CT-scans and the elastic material properties. The numerical integration of the stiffness matrix, considering these integration subdomains, produces a kind of material homogenization that allows cgFEM to consider all the information contained in the MI using a reduced number of degrees of freedom. In elements that contain dissimilar voxel values (see Fig. 8b), this homogenization can negatively affect the quality of the FE model. To reduce the difference between voxel values within an element, it is enough to subdivide the element (see Fig. 8c) recursively until the difference in values of the voxels within each of the elements is less than a predefined value. The result of this simple procedure is a finite element mesh h -adapted to the characteristics of the image, such as the one that can be seen in Fig. 8d. Note that, unlike segmentation techniques, that require to explicitly define the separation between tissues that appears in the image, with the proposed technique, the boundary between tissues is implicitly defined.

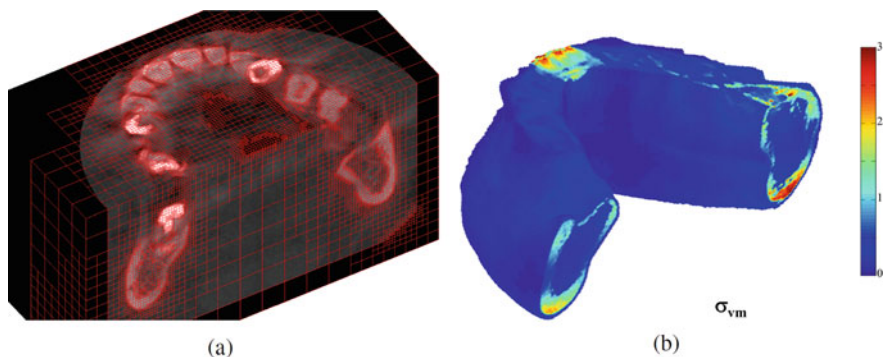


Fig. 9 3D FE model and analysis of human mandible. Reproduced with permission from [27]. (a) FE model of mandible (1/4 shown). (b) Von Mises stress distribution (MPa) due to vertical load on incisor mandibular teeth

Figure 9a represents a FE model of a human mandible (only 1/4 is shown) directly obtained from a CT-scan. The FE model consist of about 2.8 million nodes and was obtained in 63 s in an Intel i7 desktop computer with 16 GB of RAM using a code fully developed in MatLab, which shows the high efficiency of the h -adaptive modelling methodology. The von Mises stress distribution in MPa due to a 10 N vertical load distributed on the incisor mandibular teeth is shown in Fig. 9b.

The integration procedure proposed for the models obtained from IMs is compatible with the procedure used when generating FE models from CAD models. This allows combining both types of models to run bone-implant structural simulations, as shown in references [27, 29].

8 H-Refinement in Topology Optimization

Structural Topology Optimization (TO) is an optimization process that consists on distributing material into a domain in order to get the best topology for supporting certain load configuration. One of the most successful techniques of TO is the SIMP method introduced in [30, 31] due to its performance and simplicity. In minimum compliance TO problems, the SIMP method minimizes the compliance, c , of the structure defined into a design space Ω by modifying the topology (relative material density distribution, $\rho(\mathbf{x})$, $\mathbf{x} \in \Omega$) of a given amount of material $V_f = \int_{\Omega} \rho(\mathbf{x}) \, d\Omega$. SIMP considers the density as a continuous variable and imposes a penalization to the intermediate values in order to sharpen the solution as much as possible. Thus, the TO problem reads as follows: find $\rho \in L^{\infty}(\Omega)$, $\rho_{min} \leq \rho \leq 1$ such that

$$\min_{\rho} : c(\rho) = \frac{1}{2} \int_{\Omega} \boldsymbol{\varepsilon}(\mathbf{u}) \mathbf{D}(\rho) \boldsymbol{\varepsilon}(\mathbf{u}) \, d\Omega, \quad (5)$$

where $\mathbf{D}(\rho) = \rho^p \mathbf{D}_0$, being $\mathbf{D}_0$ for the full material and p the penalization parameter. When $p > 1$, intermediate densities are penalized because they contribute with little stiffness considering their material cost. A value of $p = 3$ is widely used in the literature because it provides results about intermediate material properties that satisfy the Hashin-Shtrikman bound for composite materials [32]. The problem in Eq. (5) is constrained with the volume fraction, v_f , $V(\rho) = \int_{\Omega} \rho \, d\Omega = v_f V_0$ and also with the fulfillment of the elasticity problem defined in Sect. 2. Finally, additional lateral-constraints are added to ρ , whose maximum value is the unity (fully dense material) while the lower bound is commonly set to 10^{-3} in order to avoid ill-conditioning issues when solving the elasticity problem [31].

In order to ensure the well-posedness of the discretized TO problem a stabilization of the raw solution is required [33]. This stabilization is materialized in the SIMP approach by filtering the sensitivity of the compliance. The filtering consist on updating this magnitude at each element taking an average of the sensitivities evaluated at the surrounding elements within a radius r_{min} [31]. This radius can be defined as an absolute magnitude, independent of the mesh size, or relative to the element size.

We will illustrate the use of h -adaptive tools in TO using the reference problem shown in Fig. 10. The design volume, represented by the model shown in Fig. 10a corresponds to a squared section plane-strain hollowed beam subjected to an internal pressure P . The model considers 1/4 of the section and appropriate symmetry constraints to enforce the plain-strain state. The optimization problem consists on minimising the amount of material used while satisfying that the von Mises stress, σ_{vm} , does not exceed the yield limit, S_y . The solution to this problem corresponds to a circular hollowed section, as represented in Fig. 10b.

The maximum von Mises stress in a cylinder under internal pressure can be analytically evaluated as a function of the external radius. Therefore, it is possible

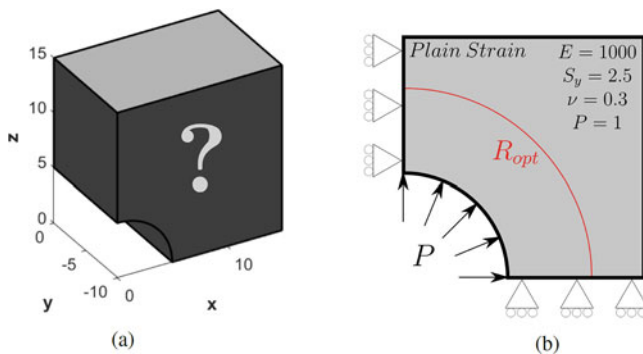


Fig. 10 Reference problem. Reproduced with permission from [35]. (a) Model of the 3D domain used for the optimization problem. Symmetry boundary conditions on planes $x = 0$, $z = 0$, $y = 0$ and $y = -10$. (b) 2D view of the optimization problem, including optimal analytical external boundary (in red) with value $R_{opt} = 9.0468$

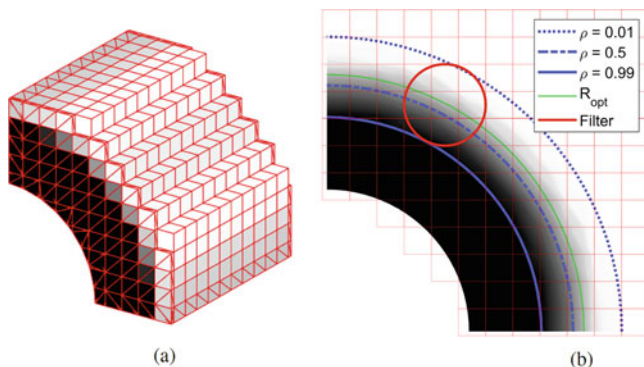


Fig. 11 Reference solution of reference optimization problem. Elements with relative density $\rho \leq 0.01$ have not been represented in (a). The 2D view in (b) includes the optimal analytical radius R_{opt} and the iso-contours $\rho = 0.01$, $\rho = 0.5$ and $\rho = 0.99$ and an example of the area covered by the filter radius. Reproduced with permission from [35]. (a) 3D view. (b) 2D view of middle plane section

to find the value of the minimum external radius that satisfies $\sigma_{vm} \leq S_y$. For the data shown in Fig. 10 (note that, although a coherent system of units has been used, as this example is not representing any real configuration, no units have been specified), this radius is $R_{opt} = 9.0468$, that leads to an optimum volume of $V_{opt} = 446.4545$ in the 3D model, which is used to define the optimum volume fraction of the TO problem, $v_{f,opt}$. The solution to this problem, obtained with TO using a mesh of elements of uniform size and $v_{f,opt}$, represented in Fig. 11, is taken as reference solution for comparison purposes.

Figure 11 shows that the solution recovers the topology of a cylinder, as expected; thus, the external radius would be the parameter of interest. However, the external radius is diffusely represented in a region of thickness approximately equal to the diameter of the region used to filter the sensitivities, represented with a red contour.

In the images shown in Fig. 12 we have consider a cantilever-beam of dimensions $3h \times h$, with constrained displacements on the left-hand side of the model and tangential stresses on the central part of the right-hand side that produce a unit-value vertical force. Using volume fraction $v_f = 0.5$, $E = 1000$, $\nu = 0.3$ and a coherent system of units we solved the TO problem. Figures 12a, b show the effect of using different mesh sizes considering the same absolute value of the global filtering radius r_{min} . Both solutions basically have the same topology and, as observed, the size of the blurred zone in both cases is the same, depending on the filtering radius, as expected. However, if the filtering radius is relative to the element size, when we go from the mesh used in Fig. 12a to the mesh used in Fig. 12c, we find a sharper boundary definition but also a variation on the topology of the solution. The dependence of the topology of the solution with respect to the filtering radius is an undesired effect, but the blurred zone is also desired to be minimal. To solve this problem we propose a modification of the standard SIMP algorithm based on the use of mesh h -adaptivity. We propose to run an iterative loop with the SIMP

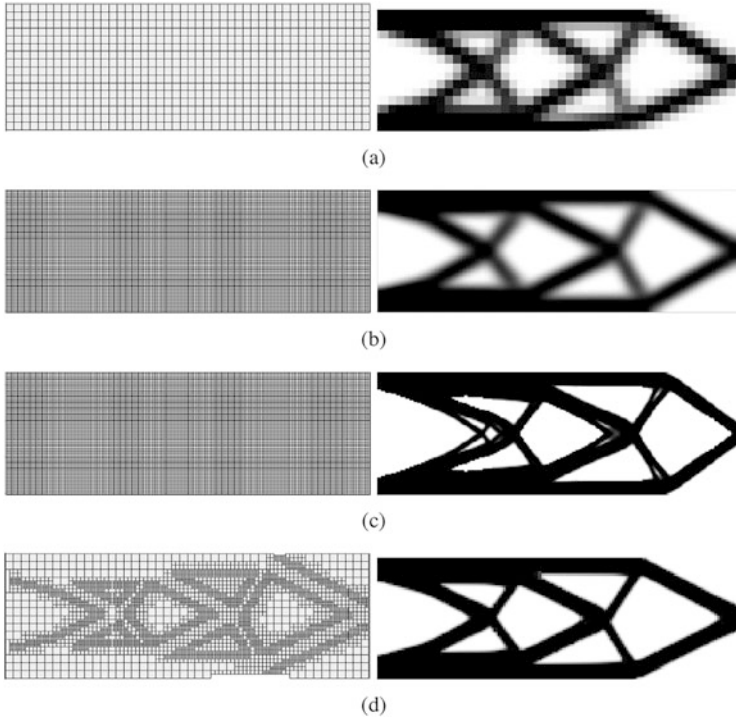


Fig. 12 Filtering Technique. Optimal solution of the cantilever-beam problem considering different types of discretizations. Reproduced with permission from [35]

algorithm with elements of uniform size, to refine the elements with intermediate relative density values and, then, to run the SIMP algorithm again considering an adaptive filtering radius relative to each element's side. This mesh refinement would be repeated until a minimum element size, specified by the analyst is reached. An example of the results obtained with this methodology is shown in Fig. 12d. It can be observed that although the boundary definition is considerably sharper than in the case of Fig. 12a, the topology has not been modified. Details on this technique can be found in [35].

Let us now consider the reference problem and check the accuracy of the proposed h -adaptivity methodology. Figure 13 compares the reference solution (Fig. 13a) obtained problem with a constant global filtering radius, and the proposed method making use of h -adapted meshes and also an adaptive filtering radius (depending on the local mesh size). In both cases, the volume fraction for the SIMP algorithm was $v_f = v_{f,opt}$. As observed, the size of the diffuse region that defines the boundary is considerably reduced, leading to a much sharper definition of the solution.

Stress constraints, like $\sigma_{vm} \leq S_y$, can also be considered by the optimization algorithm. It is possible to find several algorithms in the literature where stress

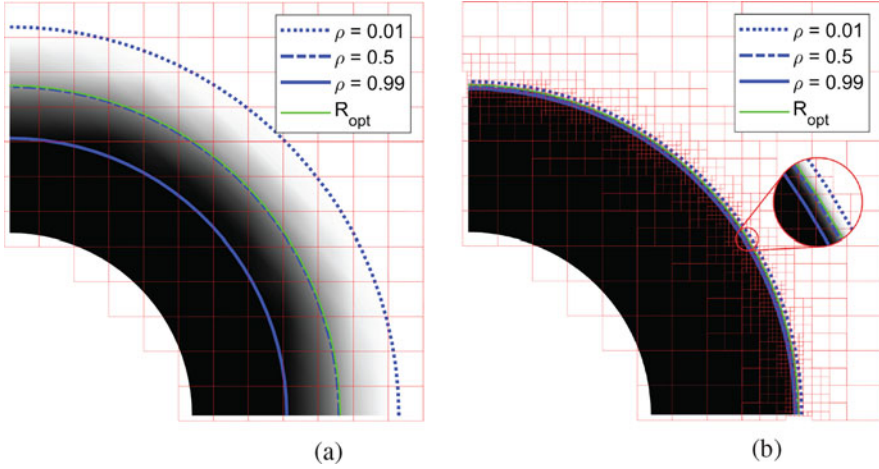


Fig. 13 Reference problem. Density based refinement: comparison of the material distribution obtained with a coarse mesh and with a density-based h -adapted mesh, for $V_f = v_{f,opt}$. Reproduced with permission from [35]. (a) Coarse Mesh. (b) h -adapted Mesh

constraints are considered in the context of TO [36–39], but none of them control the quality of the FE solution. This could lead to unfeasible solutions. Only some recent publications consider the solution quality in TO problems [40, 41], but explicit residual-type error indicators. We propose to use an error-based mesh refinement process for TO based on the recovery type error indicator (3) presented in Sect. 5. In this case, we propose the following modification to take into account the distribution of relative density ρ , that is consistent with the definition of compliance used to define the TO problem:

$$|||e|||_{\Omega}^2 \approx \mathcal{E}_{TO}^2 := \int_{\Omega} \rho^p (\sigma^* - \sigma^h)^T \mathbf{D}_0^{-1} (\sigma^* - \sigma^h) \, d\Omega. \quad (6)$$

Note that, with this modification, elements with $\rho < 1$, will be penalized by the term ρ^p , will have a low value of the error indicator and will not be refined. Hence, this error-based mesh refinement strategy will not help to improve boundary representation. Therefore, the proposed modification decouples the error-based and the density-based refinements: while the error-based refinement will be used to decrease the error of the FE analyses, the density-based refinement will be used to sharpen the representation of the boundary.

The effect of the error-based refinement is shown in Fig. 14 for the reference problem. If the TO problem with stress constraints ($\sigma_{vm} \leq S_y$) is solved considering only the density-based refinement (see Fig. 14a), the definition of the external radius is considerably sharp but the radius obtained is substantially smaller than the optimal analytical radius (represented in green colour). The reason is that the coarse mesh used along the internal radius is unable to provide an accurate evaluation of the

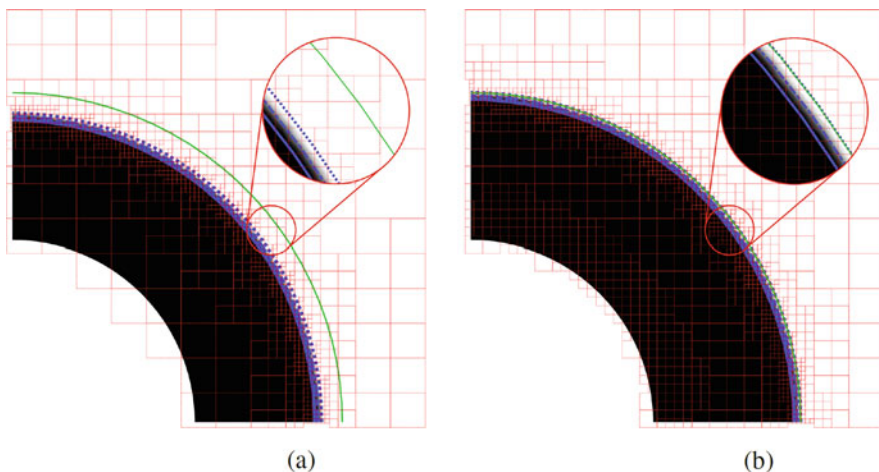


Fig. 14 Reference problem: Effect of the density-based and error-based refinements. The green contour represents the external radius of the optimal analytical solution. Reproduced with permission from [35]. (a) Density-based refinement. (b) Density and error-based refinement

maximum value of σ_{vm} that appears on the internal boundary. On the contrary, when both, density-based and error-based refinement strategies are considered, the definition of the external radius is considerably sharp and accurately located.

The final boundary of the component to be manufactured will lie within the region with intermediate values of ρ ; hence, the size of this region represents the uncertainty on the definition of the boundary. The proposed combination of h -adaptive mesh refinement and adaptive filtering, allows the analyst to decrease the size of the region with intermediate values of ρ , i.e., to limit the uncertainty on the definition of the boundary, by simply decreasing the minimum size of the elements of the FE analysis. Post-processing of the solution will be still needed if a fully sharp boundary definition (fully material/void design) is required.

9 Conclusions

This work has presented different aspects of h -adaptive refinement in the cgFEM context. The applications presented make use of the synergistic effect of the mesh-geometry independence, the hierarchical data structure, the Cartesian meshes and h -adaptivity. Even with the Matlab implementation of cgFEM, this combination allowed us to develop highly efficient algorithms for different applications of interest that could not have been developed without the integration of all these ingredients. The mesh-geometry independence in combination with the hierarchical data structure allows to easily and efficiently transfer information between meshes and even between different geometries (being this process further simplified by

the use of Cartesian meshes), avoiding the cumbersome projection procedures that would be required by standard FE implementations. Cartesian grids produce a perfect match between FEM and objects defined by images, being cgFEM able to avoid the generation of intermediate CAD models of these objects since the geometry is implicitly and accurately defined h -adapted meshes. This is considered specially relevant, in the field of biomechanics, for image-based patient-specific simulations. The techniques presented in this contribution can be combined to create new simulation methodologies. Examples of this are the works in progress of our research group devoted to the development of a hybrid optimization methodology that automatically combines topology optimization and shape optimization or the development of a methodology to design patient-specific optimized implants.

Acknowledgments The authors gratefully acknowledge the financial support of Generalitat Valenciana (Prometeo/2021/046), Ministerio de Economía, Industria y Competitividad (DPI2017-89816-R) and FEDER. The authors also acknowledge the collaboration of O. Marco, J.M. Navarro-Jiménez and D. Muñoz for their collaboration in this work.

References

1. Burman, E., Claus, S., Hansbo, P., Larson, M.G., Massing, A.: CutFEM: Discretizing geometry and partial differential equations. *Int. J. Numer. Methods Eng.* **104**, 472–501 (2015)
2. Düster, A., Sehlhorst, H.G., Rank, E.: Numerical homogenization of heterogeneous and cellular materials utilizing the finite cell method. *Comput. Mech.* **50**(4), 413–431 (2012)
3. Parvizian, J., Düster, A., Rank, E.: Finite cell method. *Comput. Mech.* **41**, 121–133 (2007)
4. Tur, M., Albelda, J., Nadal, E., Ródenas, J.J.: Imposing Dirichlet boundary conditions in hierarchical Cartesian meshes by means of stabilized Lagrange multipliers. *Int. J. Numer. Methods Eng.* **98**, 399–417 (2014)
5. Tur, M., Albelda, J., Marco, O., Ródenas, J.J.: Stabilized method of imposing Dirichlet boundary conditions using a recovered stress field. *Comput. Methods Appl. Mech. Eng.* **296**, 352–375 (2015)
6. Badia, S., Verdugo, F., Martín, A.F.: The Aggregated Unfitted Finite Element Method for Elliptic Problems. *Comput. Methods Appl. Mech. Engrg.* **336**, 533–553 (2018)
7. de Prenter, F., Verhoosel, C., van Zwieten, G., van Brummelen, E.: Condition number analysis and preconditioning of the finite cell method. *Comput. Methods Appl. Mech. Eng.* **316**, 297–327 (2017)
8. Navarro-Jiménez, J.M., Nadal, E., Tur, M., Martínez-Casas, J., Ródenas, J.J.: On the use of stabilization techniques in the Cartesian grid finite element method framework for iterative solvers. *Int. J. Numer. Methods Eng.* **121**, 3004–3020 (2020)
9. Abel, J.F., Shephard, M.S.: An algorithm for multipoint constraints in finite element analysis. *Int. J. Numer. Methods Eng.* **14**(3), 464–467 (1979)
10. Marco, O., Ródenas, J.J., Navarro-Jiménez, J.M., Tur, M.: Robust h -adaptive meshing strategy considering exact arbitrary CAD geometries in a Cartesian grid framework. *Comput. Struct.* **193**, 87–109 (2017)

11. Lorensen, W.E., Cline, H.E.: Marching cubes: A high resolution 3D surface construction algorithm. In: *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques*, vol. 21, pp. 163–169. ACM Press, New York (1987)
12. Zienkiewicz, O.C., Zhu, J.Z.: A simple error estimator and adaptive procedure for practical engineering analysis. *Int. J. Numer. Methods Eng.* **24**(2), 337–357 (1987)
13. Zienkiewicz, O.C., Zhu, J.Z.: The superconvergent patch recovery and a posteriori error estimates. Part 1: The recovery technique. *Int. J. Numer. Methods Eng.* **33**(7), 1331–1364 (1992)
14. Zienkiewicz, O.C., Zhu, J.Z.: The superconvergent patch recovery and a posteriori error estimates. Part 2: Error estimates and adaptivity. *Int. J. Numer. Methods Eng.* **33**(7), 1365–1382 (1992)
15. Blacker, T., Belytschko, T.: Superconvergent patch recovery with equilibrium and conjoint interpolant enhancements. *Int. J. Numer. Methods Eng.* **37**(3), 517–536 (1994)
16. Ramsay, A.C.A., Maunder, E.A.W.: Effective error estimation from continuous, boundary admissible estimated stress fields. *Comput. Struct.* **61**(2), 331–343 (1996)
17. Wiberg, N.E., Abdulwahab, F., Ziukas, S.: Enhanced superconvergent patch recovery incorporating equilibrium and boundary conditions. *Int. J. Numer. Methods Eng.* **37**(20), 3417–3440 (1994)
18. González-Estrada, O.A., Ródenas, J.J., Bordas, S.P.A., Nadal, E., Kerfriden, P., Fuenmayor, F.J.: Locally equilibrated stress recovery for goal oriented error estimation in the extended finite element method. *Comput. Struct.* **152**, 1–10 (2015)
19. Navarro-Jiménez, J.M., Navarro-García, H., Tur, M., Ródenas, J.J.: Superconvergent patch recovery with constraints for three-dimensional contact problems within the Cartesian grid Finite Element Method. *Int. J. Numer. Methods Eng.* **121**(6), 1297–1313 (2020)
20. Díez, P., Ródenas, J.J., Zienkiewicz, O.C.: Equilibrated patch recovery error estimates: simple and accurate upper bounds of the error. *Int. J. Numer. Methods Eng.* **69**, 2075–2098 (2007)
21. Ródenas, J.J., González-Estrada, O.A., Díez, P., Fuenmayor, F.J.: Accurate recovery-based upper error bounds for the extended finite element framework. *Comput. Methods Appl. Mech. Eng.* **199**(37–40), 2607–2621 (2010)
22. Nadal, E., Díez, P., Ródenas, J.J., Tur, M., Fuenmayor, F.J.: A recovery-explicit error estimator in energy norm for linear elasticity. *Comput. Methods Appl. Mech. Eng.* **287**, 172–190 (2015)
23. Fuenmayor, F.J., Oliver, J.L.: Criteria to achieve nearly optimal meshes in the h-adaptive finite element method. *Int. J. Numer. Methods Eng.* **39**(23), 4039–4061 (1996)
24. Upadhyay, B.D., Sonigra, S.S., Daxini, S.D.: Numerical analysis perspective in structural shape optimization: A review post 2000. *Adv. Eng. Softw.* **155**, 102992 (2021)
25. Ródenas, J.J., Bugada, G., Albelda, J., Oñate, E.: On the need for the use of error-controlled finite element analyses in structural shape optimization processes. *Int. J. Numer. Methods Eng.* **87**, 1105–1126 (2011)
26. Marco, O., Ródenas, J.J., Albelda, J., Nadal, E., Tur, M.: Structural shape optimization using Cartesian grids and automatic h-adaptive mesh projection. *Struct. Multidiscip. Optim.* **58**, 61–81 (2017)
27. Giovannelli, L., Ródenas, J., Navarro-Jiménez, J., Tur, M.: Direct medical image-based Finite Element modelling for patient-specific simulation of future implants. *Finite Elem. Anal. Des.* **136**, 37–57 (2017)
28. Dance, D., Christofides, S., Maidment, A., ID, M., Ng, K.: *Diagnostic Radiology Physics: A Handbook for Teachers and Students*. International Atomic Energy Agency, New York (2014)
29. Navarro-Jiménez, J.: *Contact Problem Modelling Using the Cartesian Grid Finite Element Method*. PhD Thesis. Universitat Politècnica de València, València (2019)
30. Bendsoe, M.P.: Optimal shape design as a material distribution problem. *Structural Optimization* **1**(4), 193–202 (1989)
31. Sigmund, O.: A 99 line topology optimization code written in Matlab. *Struct. Multidiscip. Optim.* **21**(2), 120–127 (2001)
32. Ferrer, A.: SIMP-ALL: A generalized SIMP method based on the topological derivative concept. *Int. J. Numer. Methods Eng.* **120**, 361–381 (2019)

33. Dedè, L., Borden, M.J., Hughes, T.J.R.: Isogeometric analysis for topology optimization with a phase field model. In: Archives of Computational Methods in Engineering, **19**(3), 427–465 (2012)
34. Marco, O., Ródenas, J.J., Albelda, J., Nadal, E., Tur, M.: Structural shape optimization using Cartesian grids and automatic h-adaptive mesh projection. Structural and Multidisciplinary Optimization. **58**, 61–81 (2018)
35. Muñoz, D., Albelda, J., Ródenas, J.J., Nadal, E.: Improvement in 3D topology optimization with h-adaptive refinement using the Cartesian grid Finite Element Method. Int. J. Numer. Methods Eng., 1–28, (2021), <https://doi.org/10.1002/nme.6652>
36. Holmberg, E., Torstenfelt, B., Klarbring, A.: Stress constrained topology optimization. Struct. Multidiscip. Optim. **48**(1), 33–47 (2013)
37. Yang, D., Liu, H., Zhang, W., Li, S.: Stress-constrained topology optimization based on maximum stress measures. Comput. Struct. **198**, 23–39 (2018)
38. Ferro, N., Micheletti, S., Perotto, S.: Compliance-stress constrained mass minimization for topology optimization on anisotropic meshes. SN Applied Sciences **2**(7), 1–11 (2020)
39. Ferro, N., Micheletti, S., Perotto, S.: An optimization algorithm for automatic structural design. Comput. Methods Appl. Mech. Eng. **372**, 113335 (2020)
40. Salazar de Troya, M.A., Tortorelli, D.A.: Adaptive mesh refinement in stress-constrained topology optimization. Struct. Multidiscip. Optim. **58**(6), 2369–2386 (2018)
41. Salazar de Troya, M.A., Tortorelli, D.A.: Three-dimensional adaptive mesh refinement in stress-constrained topology optimization. Struct. Multidiscip. Optim. **62**(5), 2467–2479 (2020)

h - and r -Adaptation on Simplicial Meshes Using MMG Tools



Luca Arpaia, H  lo  se Beaugendre, Luca Cirrottola, Algiane Froehly, Marco Lorini, L  o Nouveau, and Mario Ricchiuto

Abstract We review some recent work on the enhancement and application of both r - and h -adaptation techniques, benefitting of the functionalities of the remeshing platform Mmg: www.mmgtools.org. Several contributions revolve around the level-set adaptation capabilities of the platform. These have been used to identify complex surfaces and then to either produce conformal 3D meshes, or to define a metric allowing to perform h -adaptation and control geometrical errors in the context of immersed boundary flow simulations. The performance of the recent distributed memory parallel implementation ParMmg is also discussed. In a similar spirit, we propose some improvements of r -adaptation methods to handle embedded fronts.

1 Introduction

Mesh adaptation has become nowadays a powerful tool to improve the discrete representation of complex solution fields in many applications, and particularly in computational fluid dynamics [47]. Adapting the mesh may lead to a non-negligible computational overhead, as well as to more complex algorithmic and software developments. This motivates the quest for efficient and robust methods.

h -adaptation allows to optimize the discrete representation of a field of interest by inserting and removing mesh entities. This has proven to be a very powerful tool,

L. Arpaia

Coastal Risk and Climate Change Unit, French Geological Survey, Orl  ans, France
e-mail: l.arpaia@brgm.fr

H. Beaugendre · L. Cirrottola · A. Froehly · M. Lorini · M. Ricchiuto (  )

INRIA, Universit   de Bordeaux, CNRS, Bordeaux INP, IMB UMR 5251, Talence, France
e-mail: heloise.beaugendre@inria.fr; luca.cirrottola@inria.fr; algiane.froehly@inria.fr;
marco.lorini@inria.fr; mario.ricchiuto@inria.fr

L. Nouveau

Univ Rennes, INSA Rennes, CNRS, Rennes, France
e-mail: leo.nouveau@insa-rennes.fr

and it is today quite mature and generic (cf. [47] and references therein). Several aspects of this approach are still quite complex in general. This is the case for conservative high order solution transfer between meshes with different topologies, which is a non-trivial step with a non-negligible cost [2, 22, 26, 32, 48]. h -adaptation also entails non foreseeable, non-linear, and dynamic changes in the distribution of the computational workload, which is an inherently weakly parallelizable feature on distributed memory architectures.

Conversely, node relocation without topology change (or r -adaptation) has been the first adaptation approach for structured grids, later extended to unstructured ones. It remains still alluring due to the possibility of a minimally intrusive coupling with existing computational solvers, with no modification of the data structures. Moreover, devising conservative projections is quite natural with r -adaptation, due to the inherently continuous nature of the process. This allows to exploit space-time or Arbitrary-Lagrangian-Eulerian techniques to build high order conservative remapping [33, 54, 55] compliant with the Geometric Conservation Law (GCL).

The results of this paper benefit of the functionalities of the Mmg platform [38] which implements some well known research on metric-based mesh adaptation [16, 21] in a free and open source software application and library for simplicial mesh adaptation [39]. The platform provides adaptive and implicit domain remeshing through three libraries and applications Mmg2d, MmgS and Mmg3d, targeting two-dimensional Cartesian and surfacic triangulations, and three-dimensional tetrahedral meshes. Recent efforts are devoted to the development of its parallel version ParMmg [14, 45] on top of the Mmg3d remesher. A moving mesh library Fmg, which uses the Mmg library for its mesh data structure and geometry representation, is under development. Example of applications in various fields and documentation can be found on the platform website [38], also providing tutorials under the section *Try Mmg!*, and library examples in the respective repositories [39, 45].

The contribution here discussed exploit the functionalities of the platform. We present the work done in the context of the application of h -adaptation to control geometrical errors in immersed boundary flow simulations, and we discuss the performance of the parallel implementation in ParMmg. In a similar spirit, we review some improvements and applications of r -adaptation methods to flows involving immersed solid boundaries, and moving fronts. For immersed boundaries we discuss some ideas used to track embedded fronts and level sets.

The paper is organized in two main parts. Sect. 2 is devoted to the application and implementation of h -adaptation methods, with focus on level-set adaptation, application to immersed boundary methods, and parallel h -adaptation. Sect. 3 presents instead advances and applications of r -adaptation methods with focus on the resolution of embedded fronts. The paper is ended by an overlook on ongoing activities.

2 h -Adaptation: Embedded Geometries, Parallel Implementation

Anisotropic h -adaptation has largely proved useful in the context of numerical simulations to capture the physical behavior of complex phenomena at a reasonable computational cost [24]. Following a now classical approach, we consider adaptation strategies based on a local error estimation, which is converted in a map for the size and orientation of the mesh edges. This is done via an iterative process based on successive flow field evaluations, error estimations, and a-posteriori local mesh modifications. As described in [25], mesh sizes and edge directions are controlled via metric tensors built starting from error estimations involving the Hessian of the target output. The eigenvalues λ_i of these matrices are directly linked to the sizes h_i of the elements edges in the directions i (where $\lambda_i = 1/h_i^2$), with these directions given by the eigenvectors. Different criteria can be used to evaluate metrics, two examples relevant for the applications discussed later are recalled hereafter.

If several metric fields are available, e.g. the ones of Sects. 2.1 and 2.2, one may seek to combine them. To this end, we use here the simultaneous reduction method described in [25]. Denoting by $\mathcal{M}_1$ and $\mathcal{M}_2$ the two metrics defined at the same node, the resulting metric $\mathcal{M}_{1\cap 2}$ respects both sizes prescribed by the two initial metrics. With a geometrical analogy, the ellipsoid $\mathcal{E}_{1\cap 2}$ associated to $\mathcal{M}_{1\cap 2}$ is the biggest ellipsoid included in both $\mathcal{E}_1$ and $\mathcal{E}_2$, associated to $\mathcal{M}_1$ and $\mathcal{M}_2$, respectively.

The metrics for physical and level-set adaptation will be recalled in Sects. 2.1 and 2.2. Then, an application case of level-set adaptation to immersed boundary simulations will be shown in Sect. 2.3 and an example of level-set discretization will be presented in Sect. 2.4 Finally, we will shortly discuss in Sect. 2.5 our extension of h -adaptation techniques to parallel computing environments.

2.1 *Physical Adaptation*

If we aim at controlling the error between a certain output field, u , and its linear interpolation, we can exploit the upper bound on a mesh element K [24]:

$$\|u - \Pi_h u\|_{\infty, K} \leq C_d \max_{\mathbf{e} \in K} \langle \mathbf{e}, \mathcal{M}_1(K) \mathbf{e} \rangle. \quad (1)$$

Here, $\mathbf{e}$ denotes the edges of the mesh element and C_d is a constant depending on the dimension. The metric $\mathcal{M}_1(K)$ is a function of the Hessian of u , $\mathcal{H}(u)$:

$$\mathcal{M}_1 = {}^t \mathcal{R} \begin{pmatrix} \lambda_1 & 0 & 0 \\ 0 & \lambda_2 & 0 \\ 0 & 0 & \lambda_3 \end{pmatrix} \mathcal{R}, \quad (2)$$

with $\mathcal{R}$ the matrix of the eigenvectors of $\mathcal{H}(u)$ and with λ_i defined as:

$$\lambda_i = \min \left(\max \left(|h_i|, \frac{1}{h_{max}^2} \right), \frac{1}{h_{min}^2} \right), \quad (3)$$

where h_i is the i -th eigenvalue of the Hessian, h_{min} (resp. h_{max}) are the minimum (resp. maximum) allowed sizes for the mesh edges.

2.2 Level-Set Adaptation

The principle is here to improve the accuracy in the definition of the zero iso-value of a level-set function, Φ , describing the distance from a given surface. This allows to adapt the volume mesh on which the function is defined. To this end, we can use the metric field proposed in [17], and given by:

$$\mathcal{M}_2 = {}^t \mathcal{R} \begin{pmatrix} \frac{1}{\epsilon^2} & 0 & 0 \\ 0 & \frac{|\lambda_1|}{\epsilon} & 0 \\ 0 & 0 & \frac{|\lambda_2|}{\epsilon} \end{pmatrix} \mathcal{R}, \quad (4)$$

where $\mathcal{R} = (\nabla \Phi \ v_1 \ v_2)$, with $(v_1, \ v_2)$ a basis of the tangent plane to the local iso-value surface of Φ , λ_i the eigenvalues of its Hessian and ϵ a user-defined target approximation error. As proposed in the above reference, this metric field can be imposed in a user-defined layer of thickness w , close to the zero iso-level. Outside this region, the mesh size is set to grow linearly up to h_{max} , unless other constraints are set.

2.3 Level-Sets and IBM with h -Adaptation

Unfitted discretizations are becoming quite popular for the additional geometrical flexibility they offer. In these methods, the accuracy with which boundary conditions are met is directly linked to the accuracy of the implicit description of the solid [37]. Mesh adaptation with respect to the level-set is a useful tool to control this error. We exploit this in the context of a Brinkman-type volumic penalization discretization of the Navier-Stokes equations [29]. This method falls in the category of continuous forcing immersed boundary methods, meaning that the flow equations are solved throughout the whole domain and the enforcement of rigid motion inside the immersed object is done via a continuous volumic

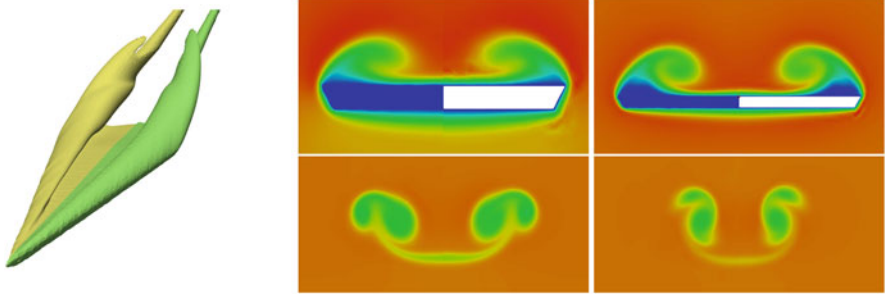


Fig. 1 Laminar delta wing. **left** Roll-up of primary and secondary vortices. **right** $\mathbb{P}^2$ Mach number distribution at $x = 0.5C$, $x = C$, $x = 1.5C$ and $x = 2C$. In each picture on the left half the unfitted computation, on the right the body-fitted one

forcing term. In the case of the Navier-Stokes equations, this leads to a compact form

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla \cdot \mathbf{F}_C(\mathbf{u}) + \nabla \cdot \mathbf{F}_V(\mathbf{u}, \nabla \mathbf{u}) + \mathbf{p}(\chi, \mathbf{u}) = \mathbf{0} \quad , \quad (5)$$

where $\mathbf{u} \in \mathbb{R}^m$ denotes the vector of the m conservative variables, $\mathbf{p} \in \mathbb{R}^m$ the penalization term, d the space dimension, $\mathbf{F}_C, \mathbf{F}_V \in \mathbb{R}^m \otimes \mathbb{R}^d$ the inviscid and viscous flux functions. The localization of the immersed object within the computational Navier-Stokes domain is done through a mask function, χ , which is the Heaviside function of the level-set representation of the immersed solid.

We present in the following results obtained by coupling h -adaptation with a nodal discontinuous Galerkin discretization of the immersed boundary model of Eq. 5. The discretization choices are discussed in [35]. The test case considered is a steady laminar flow at high angle of attack around a delta wing with a sharp leading edge. This is a benchmark test case for adaptive methods for vortex dominated external flows. Even if the geometry is not complex as the one in Sect. 2.4, h -adaptation can be important for the imposition of the immersed boundary condition in order to correctly reproduce the flow topology. See for an example Fig. 1, where the primary and secondary vortices and the DG $\mathbb{P}^2$ Mach number distribution at $x = 0.5C$, $x = C$, $x = 1.5C$ and $x = 2C$ (C being the chord of the wing) are compared between the immersed boundary (left half) and a body-fitted simulation (right half). The mesh for the unfitted simulation consists of 446,294 tetrahedra and is adapted to both the Mach number distribution and the level-set (cf. Sects. 2.1 and 2.2).

Then, as a proof of concept, starting from the DG $\mathbb{P}^2$ simulation on the adapted mesh, the full wing mesh has been adapted to fine level of detail (28 millions tetrahedra) with respect to a metric that takes into account both the level-set and the computed Mach number distribution. In Fig. 2(left) the left half of the wing presents

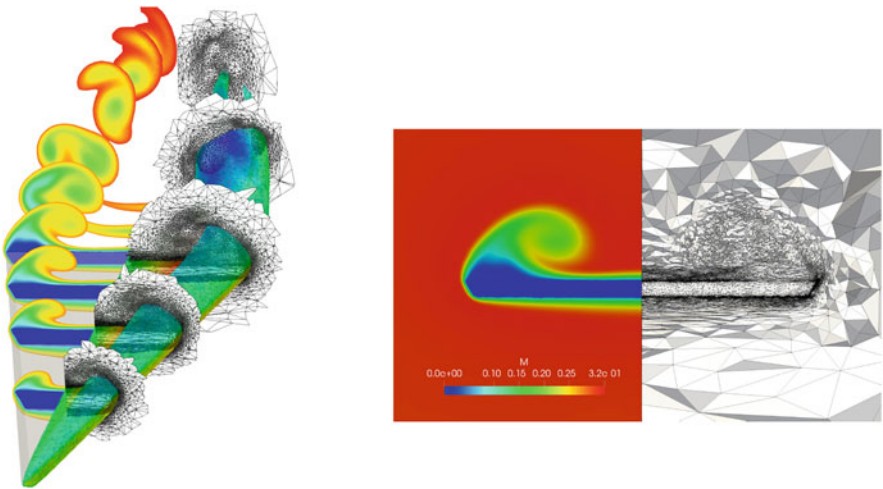


Fig. 2 Laminar delta wing. **left** Slices of the Mach number and adapted mesh. **right** Comparison flow—mesh at the trailing edge

Table 1 Laminar delta wing, mesh statistics for h -adaptation from Original to Final mesh. Percentage of edges $N^{(a,b]}$ whose length in the assigned metrics falls in the interval $l_M \in (a, b]$. Initial mesh: 446,294 tetrahedra. Final mesh: 28,083,948 tetrahedra

Mesh(# tets)	$N^{(0,0.3]}$	$N^{(0.3,0.6]}$	$N^{(0.6,0.7]}$	$N^{(0.71,0.9]}$	$N^{(0.9,1.3]}$	$N^{(1.3,1.41]}$	$N^{(1.41,2]}$	$N^{>2}$
Initial	0.93%	2.81%	1.20%	2.01%	2.99%	0.64%	3.19%	86.23%
Final	0.01%	0.58%	3.20%	24.36%	63.18%	6.22%	2.45%	0%

some slices of the Mach number distribution ($\mathbb{P}^2$ approximation) at different wing locations and in the wake. The surface of the wing is represented only as a reference for the reader. The right half of the wing presents a Mach number isosurface showing the correct representation of the flow topology (with the primary and secondary vortices) and some slices of the adapted mesh. Figure 2(right) shows a slice of the adapted mesh at the trailing edge of the wing compared with the DG solution. The differences between the imposed metrics, that closely follow the Mach distribution and the level-set description of the wing, can be clearly seen in the different regions of the domain. Finally, in Table 1 we present the mesh statistics in terms of quality of the edges with respect to the imposed metrics from the original to the final mesh.

2.4 An Example of Implicit Domain Meshing: La Sagrada Familia

Level-set methods represent the boundary of a closed body as a level-set (typically the zero level-set) of a continuous function. This point of view can be particularly

useful when trying to generate the volume mesh of a body starting only from an approximate representation of its boundary, for example a surface triangulation from an STL file, not necessarily displaying the sufficient regularity necessary for volume mesh generation (for example, being intersecting or non-orientable). In this case, this starting triangulation can be embedded in a larger volume mesh and a signed distance function from the surface triangulation can be defined on mesh nodes. The mesh is adapted according to the metrics defined in the previous section. The procedure can be iterated to increase the geometrical accuracy of the model. A new surface triangulation can also be generated in correspondence of the zero level set, allowing to extract the interior body volume mesh.

An application example is given in Fig. 3, where the implicit domain meshing procedure is applied to the Sagrada Familia starting from the STL files provided by the International Meshing Roundtable for the meshing contest of the 2017 edition,¹ using Mmg [16, 39] for the implicit domain remeshing and Mshdist [17, 40] for the signed distance function computation. Isotropic remeshing is selected for this case, and four remeshing iterations are performed. The initial mesh has 9 million nodes and 51 million tetrahedra, and the final one has 26 million nodes and 153 million tetrahedra (the worst tetrahedron has isotropic quality² equal to 0.2, while 99% of the tetrahedra have quality above 0.5, and 95% of the edges have unit-lengths between 0.7 and 1.4).

2.5 Parallel *h*-Adaptation

Computational mechanics solvers nowadays routinely exploit parallel, distributed memory computer architectures, raising the need for generating and adapting meshes whose size, in terms of computer memory, is larger and larger. Even if the distributed mesh is to be handled by a sequential remesher by gathering it on a single process, it is possible that a single computing node cannot store it in memory due to its memory limitation. Also, sequential remeshing in a parallel simulation represents a significant performance bottleneck [46]. Parallel remeshing is thus becoming increasingly demanded in large scale simulations.

¹ <https://imr.sandia.gov/26imr/MeshingContest.html>.

² The isotropic length of an edge AB is defined as

$$l_{AB} = \frac{||AB||}{h_A - h_B} \log \frac{h_B}{h_A}$$

while the isotropic tetrahedron quality is defined as

$$Q = \alpha \frac{V}{(\sum_{i=1}^6 l_i^2)^{1/3}}$$

with α a normalization factor to get quality equal to 1 for the regular tetrahedron with unit edges.

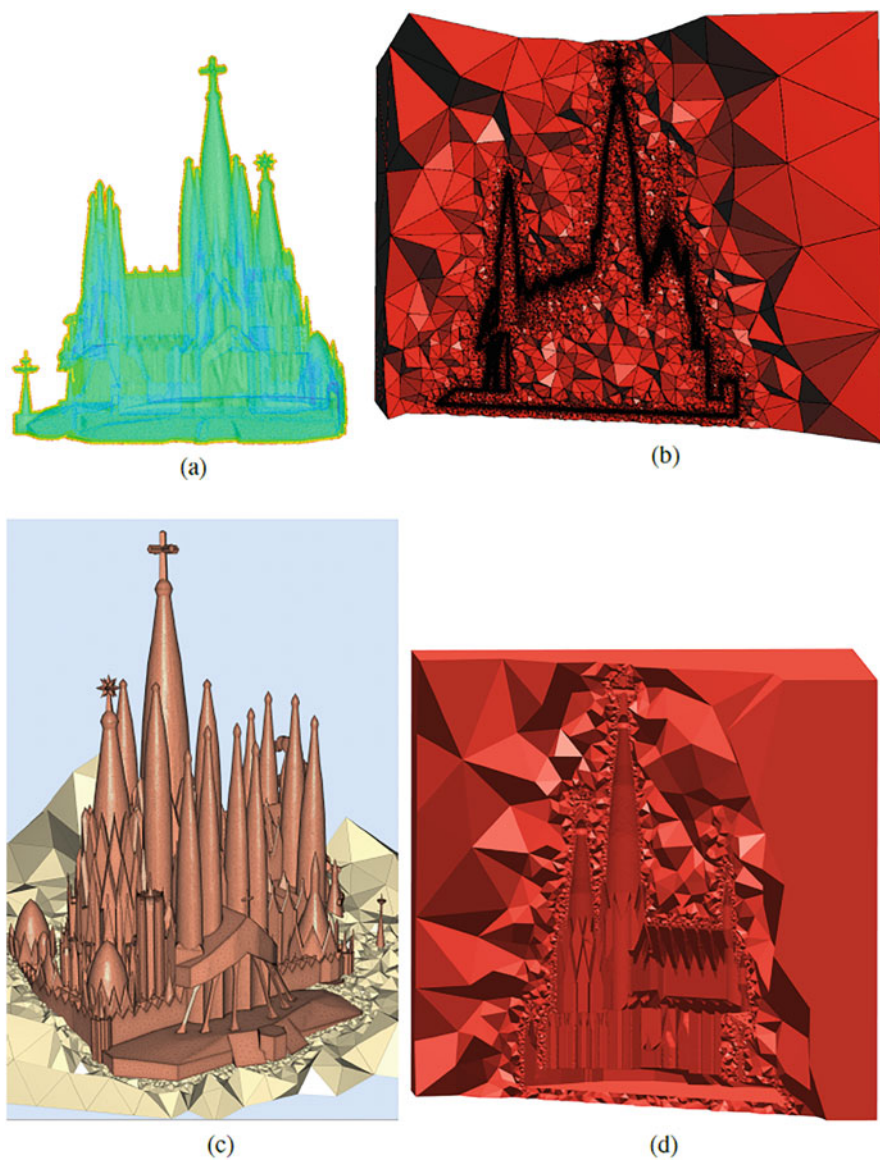


Fig. 3 Implicit domain remeshing and surface discretization of the Sagrada Familia building with Mmg3d. From 50 to 150 million tetrahedra on a desktop computer, with 4 remeshing iterations. (a) Level set function. (b) Volume remeshing. (c) Surface discretization. (d) Detail of the volume remeshing

Table 2 Mesh statistics for the ParMmg weak scaling test. Number of vertices n_v and tetrahedra n_e in input and output using p processors

p	n_v^{in}/p	n_v^{out}/p	n_v^{out}/n_v^{in}	n_v^{out}	n_e^{in}/p	n_e^{out}/p	n_e^{out}/n_e^{in}	n_e^{out}
2	3625	1,293,637	356.81	2, 587, 274	18,876	7,780,974	412.21	15, 561, 948
4	3467	1,341,637	386.88	5, 366, 549	18,798	8,072,081	429.39	32, 288, 325
8	3346	1,380,055	412.38	11, 040, 444	18,666	8,306,084	444.98	66, 448, 675
16	3264	1,412,516	432.66	22, 600, 269	18,599	8,503,695	457.2	136, 059, 129
32	3190	1,437,267	450.46	45, 992, 552	18,557	8,654,569	466.36	276, 946, 210
64	3214	1,431,186	445.2	91, 595, 935	18,625	8,619,098	462.75	551, 622, 317
128	3215	1,444,674	449.31	184, 918, 370	18,878	8,701,524	460.91	1, 113, 795, 077
256	3345	1,468,905	439.01	376, 039, 759	19,705	8,848,464	449.03	2, 265, 206, 835
512	3375	1,446,532	428.52	740, 624, 790	19,998	8,714,450	435.74	4, 461, 798, 709
1024	3335	1,449,215	434.54	1, 483, 996, 788	19,821	8,731,162	440.49	8, 940, 710, 661

The ParMmg application and library [45] is built on top of the Mmg3d remesher to provide parallel tetrahedral mesh adaptation in a free and open source software. Among the many possible remeshing parallelization methods (for example [6, 9, 10, 13, 19, 20, 23, 44]), a modular approach is adopted by selecting an iterative remeshing-repartitioning scheme that does not modify the adaptation kernel [6]. As described in depth in [14], the sequential remeshing kernel is applied at each iteration on the interior partition of each process while maintaining fixed (non-adapted) parallel interfaces. Then the adapted mesh is repartitioned in order to move the non-adapted frontiers to the interior of the partitions at the next iteration, thus progressively eliminating the presence of non-adapted zones as the iterations progress. The repartitioning algorithm currently explicitly displace the parallel interfaces by a given number of element layers, before migrating the mesh parts generated by the displacement.

A weak scaling test is performed by refining a sphere of radius 10 while keeping the workload of each process as constant as possible as the number of processes is increased. The test is performed on the bora nodes of the PlaFRIM cluster,³ and the input and output mesh data are shown in Table 2. The weak scaling performances are shown on the left in Fig. 4. The slow, steady increase in the time spent in the repartitioning and redistribution phase shows that there is still space for optimizing the parallel mesh redistribution scheme.

³ www.plafrim.fr.

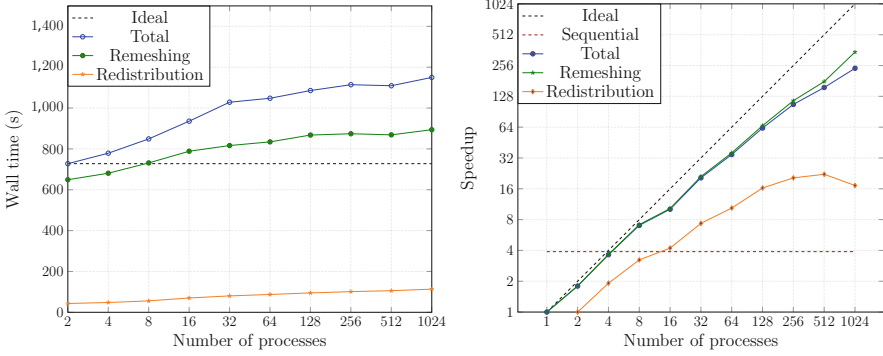


Fig. 4 ParMmg scaling test. **left** Weak scaling for the uniform refinement of a sphere. **right** Strong scaling for the adaptation to the double Archimedean spiral in a sphere

A strong scaling test is then performed with an isotropic sizemap h describing a double Archimedean spiral

$$h(x, y) = \min(1.6 + |\rho - a\theta_1| + 0.005, 1.6 + |\rho + a\theta_2| + 0.0125) \quad (6)$$

with

$$\begin{aligned} \theta_1 &= \phi + \pi \left(1 + \text{floor} \left(\frac{\rho}{2\pi a} \right) \right) \\ \theta_2 &= \phi - \pi \left(1 + \text{floor} \left(\frac{\rho}{2\pi a} \right) \right) \end{aligned} \quad (7)$$

and $\phi = \text{atan2}(y, x)$, $\rho = s\sqrt{x^2 + y^2}$, $a = 0.6$, $s = 0.5$, into a sphere of radius 10 with uniform unit edge length. The surfacic adaptation and a volumic cut of the adapted meshes on 1 and 1024 processes are shown in Fig. 5. The resulting edge length statistics are presented in Table 3, showing the variation in the distribution of the edge lengths as the number of processes is increased. It can be noticed that there is a slight trend for an increase of large edge sizes with more processes. This can be explained by the fact that, when increasing the number of partitions on the same initial mesh, the number of interfaces also increases, requiring more work to the remesher to refine the volume mesh near the coarse parallel interfaces, which are frozen by the algorithm, and making the argument for tuning the number of remeshing iterations (six in this test) with the number of processes. This trend can be also visually appreciated in Fig. 4. The time performances in log scale are shown on the right in Fig. 4. The speedup S_p over p processes is defined as the ratio between the wall time on 1 process and the wall time on p processes

$$S_p = T_1 / T_p \quad (8)$$

except for the speedup of the redistribution part of the program, which is defined with respect to the redistribution time on 2 processes instead of 1 (as there is no

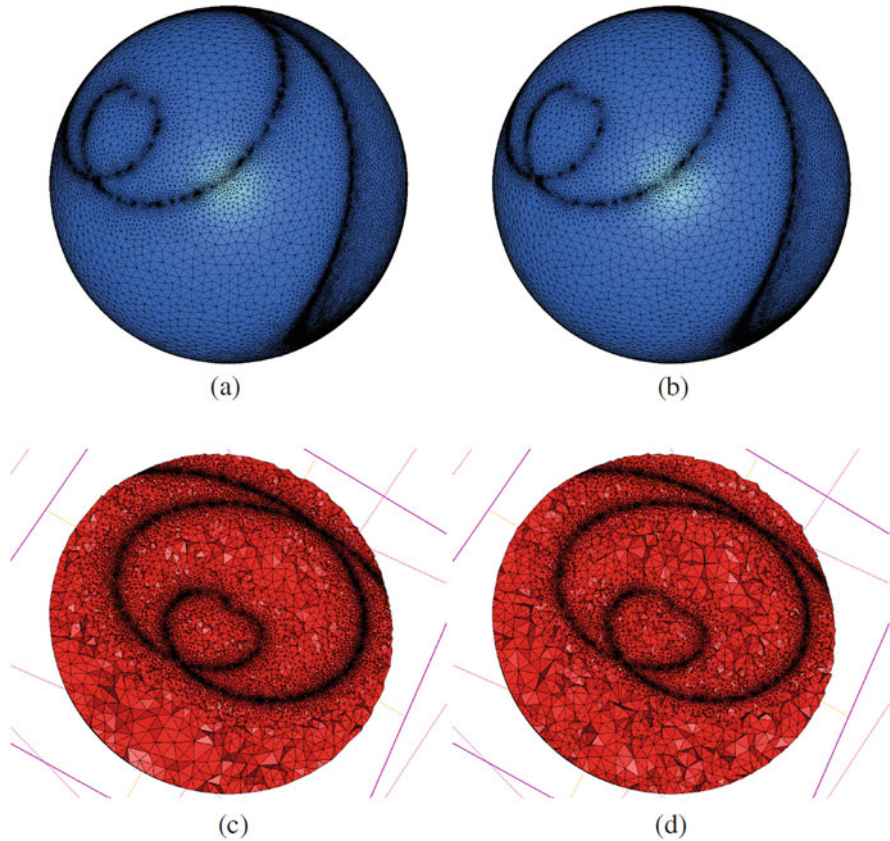


Fig. 5 Adaptation to the double Archimedean spiral on 1 and 1024 processes. (a) Surface adaptation (1 process). (b) Surface adaptation (1024 processes). (c) Volume adaptation (1 process). (d) Volume adaptation (1024 processes)

redistribution on 1 process). A performance reduction in the redistribution phase is visible on the right in Fig. 4, consistently with the weak scaling results on the left. Improved adaptation on many processes and parallel redistribution optimization are the focus of ongoing work. Both the weak and strong scaling tests have been performed with the release v1.3.0 of `ParMmg`[45].

3 r -Adaptation for Embedded Geometries and Fronts

r -adaptation techniques are an appealing approach for unsteady simulations of sharp moving fronts. Compared to h -refinement, one of their attractive characteristics is the relative algorithmic simplicity, and the ease of defining conservative remaps due to the inherent continuous nature of the process.

Table 3 Mesh statistics for the ParMmg strong scaling test. Percentage of edges $N^{(a,b]}$ whose length in the assigned metrics falls in the interval $l_M \in (a, b]$, for each simulation on p processors

p	$N^{(0,0.3]}$	$N^{(0.3,0.6]}$	$N^{(0.6,0.7]}$	$N^{(0.71,0.9]}$	$N^{(0.9,1.3]}$	$N^{(1.3,1.41]}$	$N^{(1.41,2]}$	$N^{(2,5]}$	$N^{>5}$
1	1.34%	35.34%	16.89%	25.49%	20.15%	0.63%	0.16%	0	0
2	1.14%	34.87%	16.97%	25.79%	20.45%	0.63%	0.15%	0	0
4	1.04%	34.20%	17.03%	26.08%	20.85%	0.64%	0.16%	0	0
8	0.98%	33.66%	17.06%	26.30%	21.18%	0.66%	0.16%	0	0
16	1.07%	32.41%	17.06%	26.78%	21.84%	0.68%	0.16%	0	0
32	0.91%	30.78%	17.29%	27.59%	22.57%	0.70%	0.16%	0	0
64	0.93%	30.22%	17.20%	27.76%	23.01%	0.72%	0.17%	0	0
128	0.88%	29.48%	17.20%	28.06%	23.48%	0.73%	0.17%	<0.01%	0
256	0.70%	27.63%	17.20%	28.84%	24.67%	0.77%	0.18%	<0.01%	<0.01%
512	0.66%	27.19%	17.14%	28.96%	25.04%	0.80%	0.20%	<0.01%	<0.01%
1024	0.69%	26.14%	16.91%	29.03%	25.99%	0.92%	0.30%	0.02%	<0.01%

Complete reviews of r -adaptation can be found for example in [7, 31, 53, 56]. In these methods mesh movement is based on parabolic or elliptic differential maps, often referred to as mesh PDEs. Several interesting formulations allowing PDE based mesh movement have also been proposed for fluid structure interaction where formulations in which the mesh is treated as an elastic continuum are often employed (see e.g. [51] and references therein). Although the problem can be formulated as a mapping $\xi : \Omega_{\mathbf{x}} \rightarrow \Omega_{\xi}$ from the physical domain to a reference one, for computational efficiency [11] the PDE problem is often written for the inverse mapping $\mathbf{x} : \Omega_{\xi} \rightarrow \Omega_{\mathbf{x}}$. Thus, a solution is sought directly for the mesh nodes position $\mathbf{x}$, and the mesh PDE is discretized in the (fixed) reference domain Ω_{ξ} . This framework allows a direct analogy between the mesh PDE and Lagrangian or Arbitrary Lagrangian Eulerian continuum mechanics, and its associated computational methods.

Introducing the displacement $\delta = \mathbf{x} - \xi$, the mesh PDE problem can be written as

$$\tau \partial_t \delta - \nabla_{\xi} \cdot \sigma(\delta) = F(\xi, \delta) \quad (9)$$

where $\sigma(\delta)$ can be seen as a Cauchy stress tensor, and the volume force $F(\xi, \delta)$ depends on smoothness estimator. Problem (9) should be supplemented by appropriate boundary conditions. To limit the focus of this contribution we leave this aspect out of the discussion. The interested reader can refer to [15] for a general formulation involving Dirichlet, and slip conditions along curved boundaries.

The parameter/matrix τ is a dimensionless relaxation time allowing to adjust the time scale of the mesh movement w.r.t. the physics. This parabolic regularisation/relaxation is also a good model for elliptic solvers coupled with some truncated iterative process, often used in practice especially when explicit time stepping is used in the flow solver [3, 4, 11, 12, 41, 52]. In particular, in our case the mesh

PDEs are discretized with standard $\mathbb{P}^1$ finite elements on the reference/initial mesh, and new nodal positions are obtained via a small number of nodal Jacobi iterations that can be written as

$$\delta_i^{[k+1]} = \delta_i^{[k]} - (K_{ii}^{[k]})^{-1} \sum_{j \in \mathcal{B}_i} K_{ij}^{[k]} \delta_j^{[k]} + F_i^{[k]} \quad (10)$$

with K denoting the stiffness matrix, and K_{ii} being a block or a diagonal matrix depending on the closure for $\sigma(\delta)$.

3.1 *Embedded Geometries and Distance Function: Laplace vs Elasticity*

The closure laws $\sigma(\delta)$ and $F(\xi, \delta)$ are an essential part of the method. There exist a variety of possibilities that go from simple linear Laplace smoothing to nonlinear gradient flow formulations with embedded non-negativity conditions for the Jacobian of the resulting map [7, 27, 28, 36]. A key role is played by the definition of a monitor function/matrix, which is in general a function of the flow solution in the physical space, and plays the role of the metric field in h -adaptation. In the simplest setting, if adaptation is performed w.r.t. the field u , one starts by defining a scalar function $\omega = \omega(u(\mathbf{x}))$. A somewhat classical definition is

$$\omega = (1 + \alpha u + \beta \|\nabla_{\mathbf{x}} u\| + \gamma \|\mathcal{H}_{\mathbf{x}}(u)\|)^p \quad (11)$$

In this work we consider two cases:

1. the Laplacian mesh movement corresponding to the closure $\sigma(\delta) = \omega(\mathbf{x}) \nabla_{\xi} \delta$
2. the elastic closure $\sigma(\delta) = 2\mu \epsilon(\delta) + \lambda \text{tr}(\epsilon(\delta)) \mathbf{Id}$, with (μ, λ) constant Lamé coefficients, and $\epsilon(\delta)$ the (symmetric) deformation tensor.

Note that for the Laplacian mesh movement, the classical elliptic equation

$$\nabla_{\xi} \cdot (\omega(\mathbf{x}) \nabla_{\xi} \mathbf{x}) = \mathbf{0} \quad \text{in } \Omega_{\xi} \quad (12)$$

is obtained for $F(\xi, \delta) = \nabla_{\xi} \cdot (\omega(\mathbf{x}) \mathbf{Id})$. With elasticity, usually used to propagate an imposed boundary displacement, it is less clear what form the forcing should take. Note that these matrices are constant throughout the iterations when using linear elasticity.

One of our first aims is to enhance as much as possible the resolution of surfaces implicitly defined by a level-set function Φ . We assume to be able to work just with the Dirichlet and natural boundary conditions in Eq. 9. We use this as an exercise to compare the formulations presented in the previous section. Although simple, cost-wise the Laplacian movement presents a considerable non-linearity due to the

dependence on the final configuration both of the stress definition, and in the force. As an alternative, we have tried to use linear elasticity as a mean to relax the strain locally generated by an applied volume force. To this end, we have used the same force definition for all systems, namely $\mathbf{F}(\xi, \delta) = \nabla_{\xi} \cdot (\omega(\mathbf{x})\mathbf{Id})$.

The definition of the monitor function can be set to mimic a mollified characteristic function corresponding to the domains $\Phi > 0$ (or $\Phi < 0$). A possible choice is

$$\omega_{\Phi}^{\text{GB}} = \sqrt{\alpha_0 + \alpha_{\Phi} e^{-\beta_{\Phi} \Phi^2}} \quad (13)$$

with GB standing for gradient based. The constant term α_0 is defined to offset the stiffness in different regions of the domain. A first experiment consists in comparing Laplace and elasticity based deformation with a similar definition of the sensor. In the experiment we present, the reference domain is meshed with a relatively uniform triangulation, and the Newton-Jacobi update of Eq. 10 is repeated for a few thousands iterations or until three orders of magnitude convergence are achieved.

We use as a target the level-set corresponding to an implicit curve resembling a 2D four-lobed flower. The values of the constants in Eq. 13 are slightly tuned to improve the resolution of the curve but have similar orders of magnitude in the two approaches: $(\alpha_{\Phi}, \beta_{\Phi}) = (40, 300)$ for the Laplace deformation, and $(\alpha_{\Phi}, \beta_{\Phi}) = (30, 200)$ for elasticity. The offset is chose as $\alpha_0 = 1$ for the Laplace deformation, while for elasticity we have set $\alpha_0 = 2.5|\kappa|$ with κ an approximation of the curvature of the level-set contours. The latter is introduced in an attempt to improve the capabilities of capturing singularities of the linear elastic deformation. The two approaches are compared in Fig. 6. While allowing more anisotropy in regions with smaller curvature, the elasticity based deformation tends to provide highly stretched meshes in vicinity of singularities as in the corners. The elastic approach tends to produce more anisotropic meshes near the level-set, while keeping good isotropy far from it. The Laplacian approach, instead, tends to spread stretched

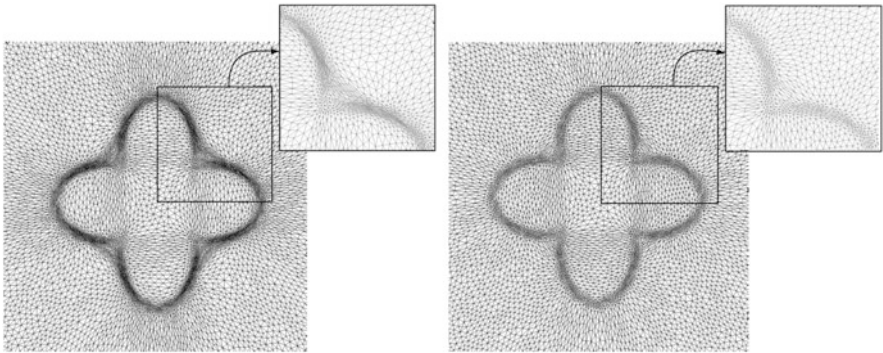


Fig. 6 Embedded geometries. 4-Lobed Flower adaptation. **left** Elasticity based movement. **right** Laplacian based movement

Table 4 Circle adaptation mesh statistics

	# elements	Min. h	Max. h	Av. h	Min. Q_{iso}	Max. Q_{iso}	Av. Q_{iso}
Initial mesh	203	9.19×10^{-3}	2.60×10^{-2}	1.58×10^{-2}	1	1.33	1.05
Laplacian	1758	3.72×10^{-3}	0.15	6.57×10^{-3}	1	1.71	1.12
Elasticity	3379	1.71×10^{-3}	3.79×10^{-2}	6.11×10^{-3}	1	6.50	2.42

Table 5 4-Lobed Flower adaptation mesh statistics

	# elements	Min. h	Max. h	Av. h	Min. Q_{iso}	Max. Q_{iso}	Av. Q_{iso}
Initial mesh	655	9.19×10^{-3}	2.60×10^{-2}	1.58×10^{-2}	1	1.33	1.05
Laplacian	3195	4.67×10^{-3}	0.15	8.74×10^{-3}	1	1.85	1.15
Elasticity	7793	1.49×10^{-3}	4.05×10^{-2}	7.1×10^{-3}	1	7.90	2.52

elements far from the level-set. This behaviour is highlighted in Fig. 7, where the ratio

$$Q_r = \frac{r_{ref}}{r_{adap}} \quad (14)$$

between the radius of the circle inscribed to the elements on the reference and adapted mesh is depicted for the meshes adapted to a flower level-set and a circular level-set. The statistics shown in Tables 5 and 4 also show these trends by reporting, for the elements in a narrow band $|LS| < 10^{-2}$ near the zero level-set, the number of elements, the minimum, maximum and average edge sizes and the 2D isotropic quality Q_{iso} of each element K

$$Q_{iso} = \frac{\beta \sum_{j=1}^3 L_j^2}{S_K} \quad (15)$$

Here the isotropic quality is purposely used to highlight the increased capability of the elastic approach to produce anisotropic elements near the level-set. This may be a limiting factor in more complex situations as e.g. when combining level-set with solution based adaptation.

To obtain a high resolution of the zero level-set without impoverishing too much the surrounding regions, the Laplacian movement could be combined with some way to directly control the mesh density at a given distance from the embedded surface. A possible choice is a piecewise constant definition of the monitor function:

$$\omega_{\Phi}^{PC} = \begin{cases} \omega_1 & \text{if } |\Phi| \leq \Phi_1 \\ \omega_j & \text{if } \Phi_{j-1} < |\Phi| \leq \Phi_j \quad \forall j \in 2, N_{\Phi} - 1 \\ \omega_{N_{\Phi}} & \text{if } \Phi_{N_{\Phi}-1} < |\Phi| \end{cases} \quad (16)$$

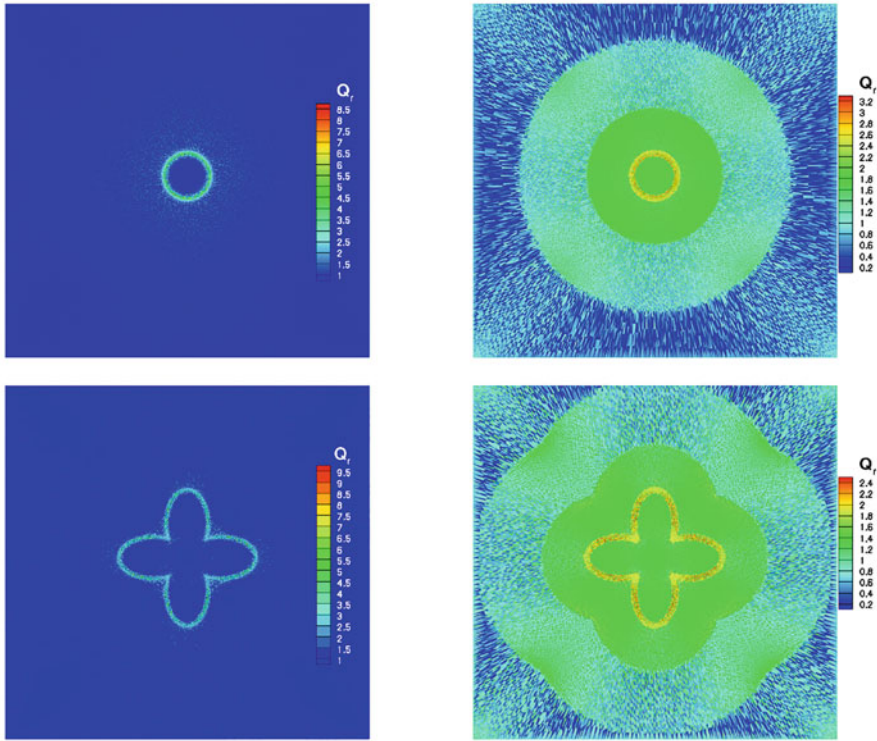


Fig. 7 Embedded geometries. Anisotropic compression, Q_r , of the adapted mesh for a Circle and a 4-Lobed Flower. **left** Elasticity. **right** Laplacian

The number of regions N_Φ is usually taken at least equal to three. This definition allows to set up areas of increasing stiffness, thus with higher density of nodes, as one approaches the zero level-set. The last jump around the usually small value Φ_0 provides the refinement sought around the embedded surface. In this area (13) could also be used, but a higher constant value works fairly well.

To give an example we consider a circular interface, and we compare on the left and central pictures of Fig. 8 the gradient based (or GB) definition with a 4 layers PC with : $(\Phi_1, \Phi_2, \Phi_3) = (0.05, 1, 1.75)$, and $(\omega_1, \omega_2, \omega_3) = (225, 90, 70, 20)$. We see that this definition allows to obtain a denser area of uniform mesh size closer to the zero level-set, thus virtually leaving out nodes for resolving other features, and allows a higher resolution in correspondence of the embedded circle. As a final comparison, in the rightmost picture in Fig. 8 we show the two approaches compared for the 4-lobed flower. This simple piece-wise constant definition of the monitor, combined with Laplacian deformation provides a good degree of control on the node density.

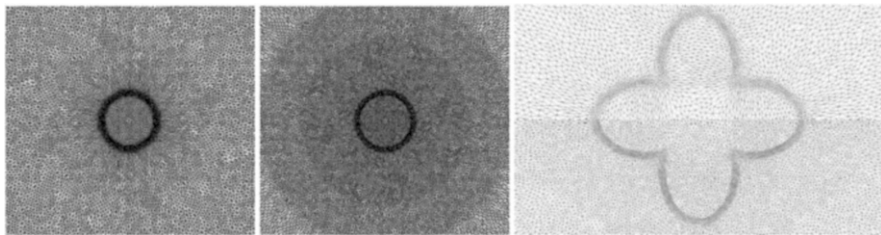


Fig. 8 Embedded geometries. Gradient based (GB) versus piecewise constant (PC) monitor function. **left** Circle: Laplacian GB. **center** Circle: Laplacian PC. **right** 4-Lobed Flower: Laplacian GB (top)—Laplacian PC (bottom)

In the next paragraph we focus on applications based on this approach. The reader can refer to the PhD thesis [41] for more comparisons and applications involving the elastic deformation using the above definitions, and to [7, 27, 56] for the derivation of mesh PDEs which attempt to impose a given metric field without topology change.

3.2 Immersed Boundaries and Moving Bodies

We consider the combination of level-set adaptation, immersed boundary methods, and solution based adaptation. The PDE setting is similar to that of Sect. 2.3, but the numerical results use a flow solver based on a stabilized residual based method in an Arbitrary-Lagrangian-Eulerian (ALE) setting [1, 41, 42]. We focus on forced motion of a solid, whose boundary is implicitly defined via the zero level-set of a signed distance function. To cope with the time dependent nature of the problem in this case we use the explicit knowledge of the exact geometry, and recompute the distance using the method proposed in [18]. As before, we assume to be able to work just with the Dirichlet and natural boundary conditions in Eq. 9.

As in Sect. 2.3, we wish to combine level-set and solution based adaptation. To this end, we introduce the smoothness monitor for the solution

$$\omega_u = \sqrt{1 + \alpha_u \widehat{\nabla} u^2} \quad (17)$$

where $\widehat{\nabla} u$ is a capped gradient norm (see e.g. [12, 52])

$$\widehat{\nabla} u = \min(1, \frac{\|\nabla_{\mathbf{x}} u\|}{\beta_u \|\nabla_{\mathbf{x}} u\|_{\text{ref}}}) \quad (18)$$

with the reference value $\|\nabla u\|_{\text{ref}}$ usually chosen as the maximum over the domain. Note that the gradient is computed w.r.t. the actual coordinates $\mathbf{x}$. The combined

level-set/solution monitor function is finally defined as

$$\omega = \begin{cases} \omega_\Phi & \text{if } |\Phi| \leq \epsilon \\ \max(\omega_\Phi, \omega_u) & \text{if } |\Phi| > \epsilon \end{cases} \quad (19)$$

The initial mesh is adapted to the embedded surface and initial solution, solving the Newton-Jacobi iterations until a relative convergence of a few orders of magnitude, as discussed in the previous section. However, during the ALE time advancement, only a few mesh iterations are performed. For the example shown we use 20 mesh iterations, with the last known mesh as initial guess. This gives a negligible overhead compared to the implicit Crank-Nicolson ALE time stepping used for the flow.

As an example we consider a well known benchmark involving the forced pitching and heaving motion of a NACA 0015 airfoil at Reynolds ≈ 1100 . We refer to [5, 8, 30] for the set up of the computation. Snapshots of the vorticity are reported in Fig. 9, showing the strong separation and vortex shedding taking place. In the same figure we report the corresponding meshes adapted w.r.t. both the level-set, and vorticity. The efficient capturing of the body movement and vorticity fields in the mesh deformation is quite impressive. To validate our approach we compare the lift and torque aerodynamic coefficients to reference literature data [5, 8, 30] in the top pictures of Fig. 10, observing an excellent agreement. We also compare the results obtained on the adaptive mesh, with those obtained on the initial reference triangulation (finest mesh size $h \approx 0.03$), and on a refined one (finest mesh size $h \approx 0.02$). Mesh data, and computational times are reported in Table 6. We can see that the r -adaptive approach allows computational savings of the order of 26.5%,

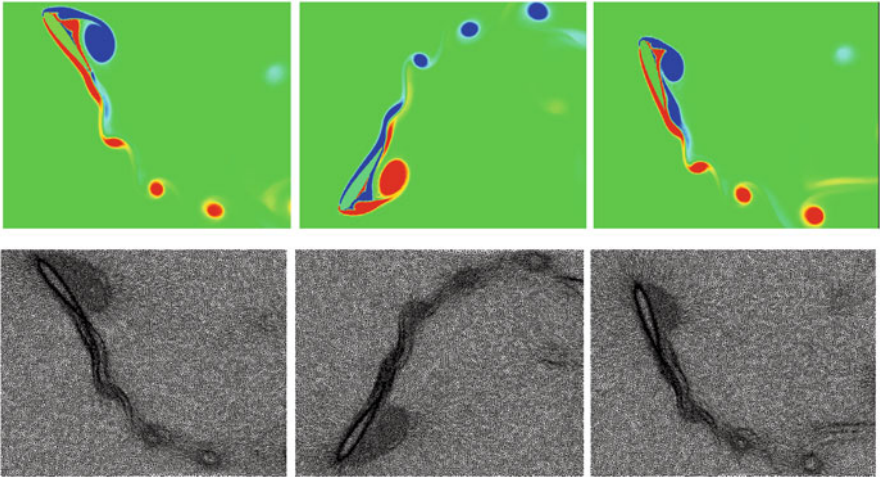


Fig. 9 Oscillation Naca snapshots. **top** Vorticity field. **bottom** Adaptive mesh

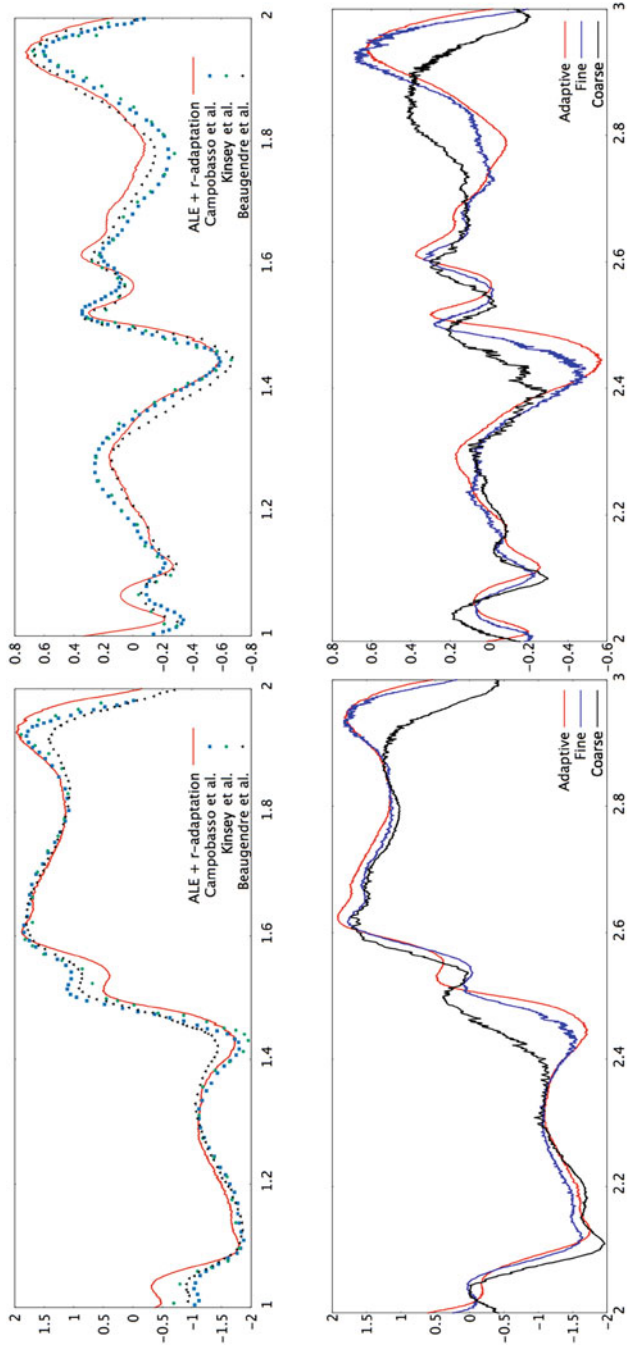


Fig. 10 Oscillating Naca—Aerodynamic coefficients vs time (Left: lift—Right: torque). **top** Comparison with literature. **bottom** Fine/adapted/coarse mesh results

Table 6 Oscillating Naca: computational times for different simulations

Mesh (size)	Number of nodes	MMPDE CPU time [s]	Total CPU time [s]
Fixed $h_{\min} \approx 0.03$	53,495	0	20,515
Fixed $h_{\min} \approx 0.02$	33,139	0	37,716
Adaptive	33,139	3241	27,738

and that without any particular optimization, only 11.7% of the computing time is devoted to the mesh PDE solver.

3.3 Free Surface Flows and Moving Shorelines

Another application with very similar issues is the simulation of free surface waves with moving wet/dry fronts. These can be modelled by the shallow water equations which can be written as a system of balance equations of the form

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla \cdot \mathbf{F}_C(\mathbf{u}) + S(\mathbf{u}; \mathbf{x}) = \mathbf{0} \quad , \quad (20)$$

where $\mathbf{u}$ is the array of the water depth H , and horizontal mass flux vector, $\mathbf{F}_C$ the conservative fluxes, and $S(\mathbf{u}; \mathbf{x})$ a source term modelling several effects such as those of bottom topography, friction, Coriolis forces, and others depending on the application. For applications involving flooding and complex wave propagation ideas similar to those discussed above can be used to adapt the mesh w.r.t shorelines and wave fronts. However, for wave propagation, and in general for hyperbolic problems, a majority of codes are based on an explicit time stepping kernel. This requires limiting somewhat the overhead of the mesh PDE solver for r -adaptation to be interesting. So we do not use the computation of a distance function here, but we proceed as follows:

- generate initial meshes with sizes increasing with the depth, so that a prescribed minimum size is attained at and close to the initial shoreline
- define the regularized Heaviside function

$$\phi_H(\mathbf{x}) = \begin{cases} 1 & \text{if } H(\mathbf{x}) > \epsilon_H \\ \phi(H) & \text{if } 0 < H(\mathbf{x}) \leq \epsilon_H \\ 0 & \text{if } H(\mathbf{x}) = 0 \end{cases}$$

- define a monitor function combining the dry detection function above and a capped gradient (cf. Eq. 18) as

$$\omega = \sqrt{1 + \alpha_\eta \widehat{\nabla \eta}^2 + \alpha_{\text{dry}} \|\nabla_{\mathbf{x}} \phi_H\|^2} \quad (21)$$

where if $b(\mathbf{x})$ denotes the topography, η is the free surface level $H + b$. The regularization $\phi(H) \in [0, 1]$ can be any (at least C^0) function modelling the depth profile in cells cut by the shoreline. Note that, differently than in the case of moving bodies, this curve is in general never known exactly. Even in the initial condition, its definition depends in practice on the resolution of the topographic data. In practice, defining $\phi(H)$ as a linear function is already enough to capture the wet/dry transition.

In the numerical examples shown here, the shallow water equations are solved by means of an explicit stabilized residual based approach in an ad-hoc ALE form. Although the design of numerical schemes is not the focus of this work, we mention that one of the challenges when dealing with the shallow water equations (and in general balance laws) is that the ALE formulation, and the associated remaps, needs to satisfy not only the classical conservation constraints (mass and momentum), but also additional ones related to the preservation of particular steady solutions (well-balanced). As for compressible fluid flows, additional constraints related to the admissibility of the solution (e.g. non-negativity of the depth) should also be embedded. Moreover, in real applications one has to make sure that the remap used for the data of the problem (e.g. topography) is not affected by truncation errors, which is also a constraining requirement. These aspects are covered in detail in [3], in which these constraints are combined within a conservative ALE formulation, with a high order quadrature based projection of the original data on the moving mesh.

Concerning the mesh solver, while for the initial solution Eq. 10 is solved until convergence within a few orders of magnitude, within each time step only 5 Jacobi relaxation iterations are performed to move the mesh, using as initial guess the last available one. As an example, we consider the simulation of the 2011 Tohoku-Honsu tsunami, the interested reader can refer to [3, 4] for thorough validation.

The leftmost picture on Fig. 11 shows an overview of the computational domain, including the position of the GPS buoys whose data has been used for validation,

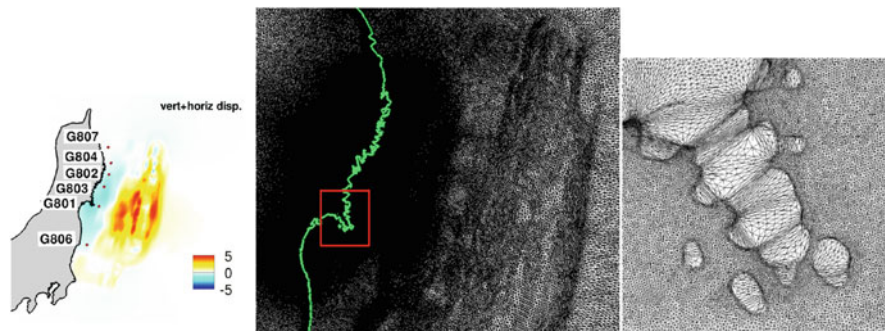


Fig. 11 Tohoku-Honsu tsunami. **left** Problem setup, GPS probes and initial wave height (in meters). **center** Adapted closer to the coastline of the Sendai bay and Myagi prefecture (boxed area). **right** Shoreline of the Myagi prefecture

and the initial free surface perturbation computed based on the seismic data and with approach by Satake [34, 49]. The middle picture in the same figure shows a close up of the Japanese coast in correspondence of the Myagi prefecture (boxed area) and of the Sendai bay, with the adapted mesh in the initial instants of the propagation of the tsunami. The rightmost picture shows a further close up of the Myagi prefecture with the mesh adapted to the coastline. Note that the resolution of the reference mesh goes from 360 m close to the initial shoreline to 15 Km offshore. A fine mesh computation has been run for comparison using resolutions down to 120m close to the shore and of 5 Km offshore.

The left and middle pictures on Fig. 12 show the waves entering the Sendai bay, and the corresponding adapted meshes, while the rightmost pictures report the GPS time series, comparing the adaptive computations to the fine mesh, and to the GPS data showing the potential of the approach used in resolving complex wave propagation on coarser grids. Finally, Fig. 13 shows the distribution of Q_r , defined in Eq. 14, at two different time steps with a close up of the Sendai bay. The compression ratios achieved are of about 2.5–3 across the steepest shoaling waves, and increase above 4 and with peaks of about 6–6.5 in correspondence of the shorelines.

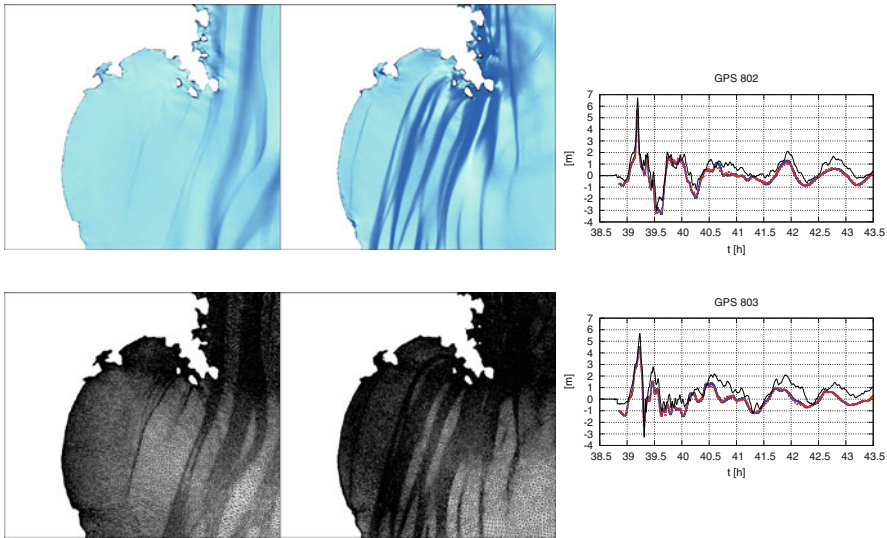


Fig. 12 Tohoku-Honsu tsunami. **left and center** Waves entering the Sendai bay. **right** GPS bouys 802 and 803 (red: adaptive—blue: fine mesh—black: GPS data)

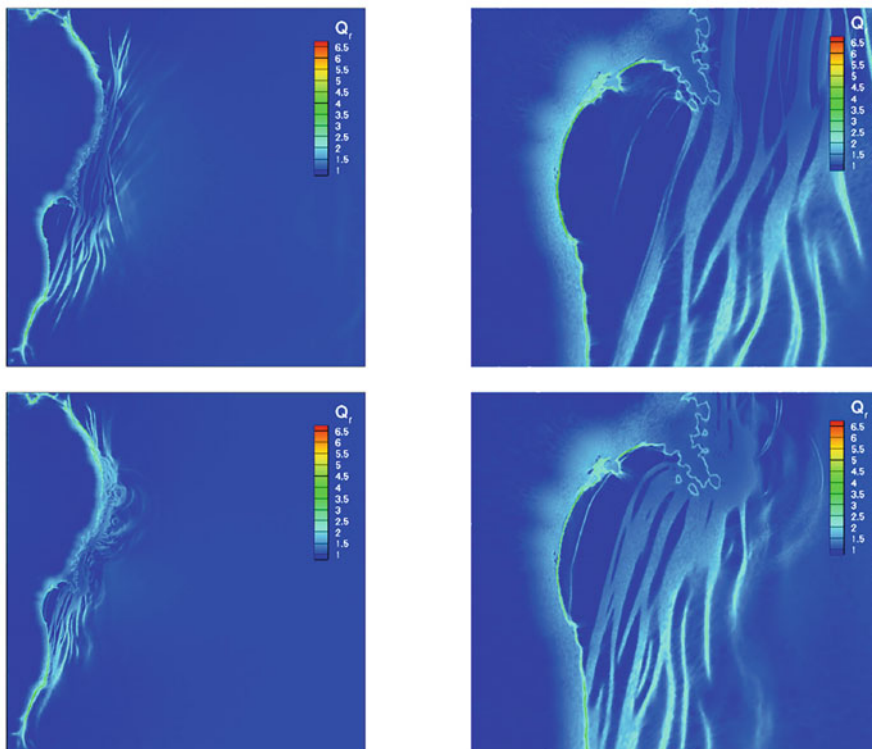


Fig. 13 Tohoku-Honsu tsunami. Anisotropic compression, Q_r , of the adapted mesh. **left** Whole computational domain. **right** Close up of the Sendai bay

4 Summary and Perspectives

This paper reviews recent work performed using on the capabilities of the remeshing software platform *Mmg*. As other open source packages for computational mechanics or mesh generation, this platform allows to apply its existing functionalities to problems of interest, as well as developing new meshing/remeshing methods.

Ongoing work related to the results presented are related to the robustification mesh deformation method in order to account for general curved boundaries [15], to the investigation of the impact of adaptation in the framework of higher order embedded methods [43, 50], to the full support of non-manifold surface adaptation and anisotropic remeshing in parallel, to the exploration of hybrid parallelization strategies.

Acknowledgement Dedicated to our dear friend Cécile Dobrzynski.

References

1. Abgrall, R., Ricchiuto, M.: High order methods for CFD. In: Erwin Stein, R.D.B., Hughes, T.J. (eds.) *Encyclopedia of Computational Mechanics*, 2nd edn. John Wiley and Sons, London (2017)
2. Alauzet, F.: A parallel matrix-free conservative solution interpolation on unstructured tetrahedral meshes. *Comp. Methods Appl. Mech. Eng.* **299**, 116–142 (2016)
3. Arpaia, L., Ricchiuto, M.: r-adaptation for shallow water flows: conservation, well balancedness, efficiency. *Comput. Fluids* **160**, 175–203 (2018)
4. Arpaia, L., Ricchiuto, M.: Well balanced residual distribution for the ALE spherical shallow water equations on moving adaptive meshes. *J. Comput. Phys.* **405**, 109173 (2020)
5. Beaugendre, H., Morency, F.: Innovative Model for Flow Governed Solid Motion Based on Penalization and Aerodynamic Forces and Moments. Inria Research Report RR-8718 (2015)
6. Benard, P., Balarac, G., Moureau, V., Dobrzynski, C., Lartigue, G., D'Angelo, Y.: Mesh adaptation for large-eddy simulations in complex geometries. *Int. J. Numer. Meth. Fluids* **81**, 719–740 (2016)
7. Budd, C.J., Huang, W., Russell, R.D.: Adaptivity with moving grids. *Acta Numer.* **18**, 111–241 (2009)
8. Campobasso, M.S., Drofelnik, J.: Compressible Navier-Stokes analysis of an oscillating wing in a power-extraction regime using efficient low-speed preconditioning. *Comput. Fluids* **67**, 26–40 (2012)
9. Castaños, J.G., Savage, J.E.: The dynamic adaptation of parallel mesh-based computation. In: PPSC (1997)
10. Cavallo, P.A., Sinha, N., Feldman, G.M.: Parallel unstructured mesh adaptation method for moving body applications. *AIAA J.* **43**(9), 1937–1945 (2005)
11. Cenicerros, H.D., Hou, T.Y.: An efficient dynamically adaptive mesh for potentially singular solutions. *J. Comput. Phys.* **172**(2), 609–639 (2001)
12. Chen, G., Tang, H., Zhang, P.: Second-order accurate godunov scheme for multicomponent flows on moving triangular meshes. *J. Sci. Comput.* **34**(1), 64–86 (2008)
13. Chrisochoides, N., Nave, D.: Parallel Delaunay mesh generation kernel. *Int. J. Numer. Methods Eng.* **58**(2), 161–176 (2003)
14. Cirrotola, L., Froehly, A.: Parallel unstructured mesh adaptation using iterative remeshing and repartitioning. Inria Research Report RR-9307 (2019). <https://hal.inria.fr/hal-02386837>
15. Cirrotola, L., Ricchiuto, M., Froehly, A., Re, B., Guardone, A., Quaranta, G.: Adaptive deformation of 3d unstructured meshes with curved body fitted boundaries with application to unsteady compressible flows. *J. Comput. Phys.* **433**, 110177 (2021). <https://doi.org/10.1016/j.jcp.2021.110177>
16. Dapogny, C., Dobrzynski, C., Frey, P.: Three-dimensional adaptive domain remeshing, implicit domain meshing, and applications to free and moving boundary problems. *J. Comput. Phys.* **262**, 358–378 (2014)
17. Dapogny, C., Frey, P.: Computation of the signed distance function to a discrete contour on adapted triangulation. *Rech. Aéropat.* (49), 193–219 (2012)
18. Dapogny, C., Frey, P.: Computation of the signed distance function to a discrete contour on adapted triangulation. *Calcolo* **49**, 193–219 (2012)
19. De Cougny, H.L., Shephard, M.S.: Parallel refinement and coarsening of tetrahedral meshes. *Int. J. Numer. Methods Eng.* **46**(7), 1101–1125 (1999)
20. Dignonnet, H., Coupez, T., Laure, P., Silva, L.: Massively parallel anisotropic mesh adaptation. *Int. J. High Perform. Comput. Appl.* **33**(1), 3–24 (2019)
21. Dobrzynski, C., Frey, P.: Anisotropic delaunay mesh adaptation for unsteady simulations. In: Garimella, R.V. (ed.) *Proceedings of the 17th International Meshing Roundtable*, pp. 177–194. Springer Berlin Heidelberg, Berlin, Heidelberg (2008)
22. Farrell, P., Maddison, J.: Conservative interpolation between volume meshes by local Galerkin projection. *Comput. Methods Appl. Mech. Eng.* **200**(1), 89–100 (2011)

23. Flaherty, J., Loy, R., Özturan, C., Shephard, M., Szymanski, B., Teresco, J., Ziantz, L.: Parallel structures and dynamic load balancing for adaptive finite element computation. *Appl. Numer. Math.* **26**(1), 241–263 (1998)
24. Frey, P., Alauzet, F.: Anisotropic mesh adaptation for CFD computations. *Comput. Methods Appl. Mech. Eng.* (194), 5068–5082 (2005)
25. Frey, P., George, P.: *Mesh Generation. Application to Finite Elements*. Wiley, London (2008)
26. Hermes, D., Persson, P.O.: High-order solution transfer between curved triangular meshes (2018)
27. Huang, W.: Variational mesh adaptation: Isotropy and equidistribution. *J. Comput. Phys.* **174**(2), 903–924 (2001)
28. Huang, W., Kamenski, L.: A geometric discretization and a simple implementation for variational mesh generation and adaptation. *J. Comput. Phys.* **301**, 322–337 (2015)
29. K. Khadra P. Angot, S.P., Caltagirone, J.: Fictitious domain approach for numerical modelling of Navier–Stokes equations. *Int. J. Numer. Methods Fluids* (34), 651–684 (2000)
30. Kinsey, T., Dumas, G.: Parametric study of an oscillating airfoil in a power-extraction regime. *AIAA J.* **46**(6), 543–561 (2008)
31. Knupp, P., Steinberg, S.: *Fundamentals of Grid Generation. The Fundamentals of Grid Generation*. Taylor & Francis, London (1993)
32. Kucharik, M., Shashkov, M.: Extension of efficient, swept-integration-based conservative remapping method for meshes with changing connectivity. *Int. J. Numer. Methods Fluids* **56**(8), 1359–1365 (2008)
33. Kucharik, M., Shashkov, M., Wendroff, B.: An efficient linearity-and-bound-preserving remapping method. *J. Comput. Phys.* **188**(2), 462–471 (2003)
34. LeRoy, S., Lemoine, A., Pedreros, R., Rousseau, M.: Tohoku-oki 2011 Tsunami high-resolution modeling and sensitivity to the rupture complexity: Kamaishi and Sendai areas. In: *French Japanese Week on Disaster Risk Reduction*. Tokyo (2017)
35. Lorini, M., Dobrzynski, C., Perrier, V., Ricchiuto, M.: Preliminary results of a discontinuous galerkin immersed boundary method combining penalization and anisotropic mesh adaptation. In: *Proc. ECCM 6/ECFD 7*, pp. 1875–1885 (2020)
36. Masud, A., Bhanabhagvanwala, M., Khurram, R.A.: An adaptive mesh rezoning scheme for moving boundary flows and fluid–structure interaction. *Comput. Fluids* **36**(1), 77–91 (2007). <https://doi.org/10.1016/j.compfluid.2005.07.013>. Challenges and Advances in Flow Simulation and Modeling
37. Mittal, R., Iaccarino, G.: Immersed boundary methods. *Ann. Rev. Fluids. Mech.* (37), 239–261 (2005)
38. Mmg platform website. <https://www.mmgtools.org/>
39. Mmg version 5.5.2. SWHID: swh:1:rel:fe173a75f45f079d363d5a82204c9737550c5d79; REPOSITORY: <https://github.com/MmgTools/mmg>
40. Mshdist. SWHID: swh:1:dir:ae83c7e306d8a20484e74e5588cdb19ab0ae0d71; REPOSITORY: <https://github.com/ISCDtoolbox/Mshdist>
41. Nouveau, L.: Adaptive residual based schemes for solving the penalized Navier Stokes equations with moving bodies: application to ice shedding trajectories. Theses, Université de Bordeaux (2016). <https://tel.archives-ouvertes.fr/tel-01500093>
42. Nouveau, L., Beaugendre, H., Ricchiuto, M., Dobrzynski, C., Abgrall, R.: An adaptive ALE residual based penalization approach for laminar flows with moving bodies. Inria Research Report RR-8936 (2016)
43. Nouveau, L., Ricchiuto, M., Scovazzi, G.: High-order gradients with the shifted boundary method: an embedded enriched mixed formulation for elliptic PDEs. *J. Comput. Phys.* **398**, 108898 (2019)
44. Olier, L., Biswas, R., Gabow, H.N.: Parallel tetrahedral mesh adaptation with dynamic load balancing. *Parallel Comput.* **26**(12), 1583–1608 (2000)
45. ParMmg version 1.3.0. SWHID:swh:1:rel:bf45d6314a386455d53a6c351be771d6252e0d43; REPOSITORY: <https://github.com/MmgTools/ParMmg>

46. Park, M.A., Krakos, J.J., Michal, T.A., Loseille, A., Alonso, J.J.: Unstructured Grid Adaptation: Status, Potential Impacts, and Recommended Investments Towards CFD 2030. In: AIAA Fluid Dynamics Conference, AIAA AVIATION Forum. Washington DC, United States (2016)
47. Park, M.A., Loseille, A., Krakos, J., Michal, T.R., Alonso, J.J.: Unstructured grid adaptation: status, potential impacts, and recommended investments towards CFD 2030. In: AIAA AVIATION Forum. American Institute of Aeronautics and Astronautics (2016)
48. Re, B., Dobrzynski, C., Guardone, A.: An interpolation-free ale scheme for unsteady inviscid flows computations with large boundary displacements over three-dimensional adaptive grids. *J. Comput. Phys.* **340**, 26–54 (2017)
49. Satake, K., Fujii, Y., Harada, T., Namegaya, Y.: Time and space distribution of coseismic slip of the 2011 Tohoku earthquake as inferred from tsunami waveform data. *Bull. Seismol. Soc. Am.* **103**, 1473–1492 (2013)
50. Song, T., Main, A., Scovazzi, G., Ricchiuto, M.: The shifted boundary method for hyperbolic systems: embedded domain computations of linear waves and shallow water flows. *J. Comput. Phys.* **369**, 45–79 (2018)
51. Stein, K., Tezduyar, T., Benney, R.: Mesh moving techniques for fluid-structure interactions with large displacements. *J. Appl. Mech.* **70**(1), 58–63 (2003)
52. Tang, H., Tang, T.: Adaptive mesh methods for one- and two-dimensional hyperbolic conservation laws. *SIAM J. Numer. Anal.* **41**(2), 487–515 (2003)
53. Tang, T.: Moving mesh methods for computational fluid dynamics. *Contemp. Math.* **383**, 141–174 (2005)
54. Thomas, P., Lombard, C.: Geometric conservation law and its application to flow computations on moving grids. *AIAA J.* **17**(10), 1030–1037 (1979)
55. Trulio, J.G., Trigger, K.R.: Numerical solution of the one-dimensional lagrangian hydrodynamic equations. Tech. rep., California. Univ., Livermore, CA (United States). Lawrence Radiation Lab. (1961)
56. Weizhang, H., Russell, R.: Adaptive Moving Mesh Methods. Applied Mathematical Sciences, vol. 174. Springer, Berlin (2011)

Geometry and Adaptive Mesh Update Procedures for Ballistics Simulations



Saurabh Tendulkar, Fan Yang, Rocco Nastasia, Mark W. Beall,
Assad A. Oberai, Mark S. Shephard, and Onkar Sahni

Abstract Many engineering problems are characterized as having complex geometry that evolves over time combined with complex physical behaviors. In addition to an appropriate combination of physics models, discretization techniques and numerical methods, the effective simulation of such problems requires a set of methods to track the evolution of the simulation domain and the spatial decomposition of that domain into the desired meshes. This paper presents a set of geometry model representation and mesh generation/update procedures developed, and combined, to support the simulation of evolving geometry problems. The geometry representation and tracking procedures presented are designed to effectively interact with complex CAD representations while providing an effective means to account for sliding interfaces for which the base model topology changes. The mesh generation procedures combine automatic boundary layer and unstructured mesh generation with procedures to effectively generate semi-structured meshes in confined regions of geometry that can include step variations. The mesh update procedures combine mesh motion and local cavity mesh modification to provide well controlled, and adapted, meshes throughout an evolving geometry simulation.

S. Tendulkar · R. Nastasia · M. W. Beall
Simmetrix Inc., Clifton Park, NY, USA
e-mail: saurabh@simmetrix.com; nastar@simmetrix.com; mbeall@simmetrix.com

F. Yang
Raysight Intelligent Medical, Shenzhen, China

A. A. Oberai
University of Southern California, Los Angeles, CA, USA
e-mail: aoberai@usc.edu

M. S. Shephard · O. Sahni (✉)
Rensselaer Polytechnic Institute, Troy, NY, USA
e-mail: shephard@rpi.edu; sahni@rpi.edu

1 Introduction

Many engineering problems are characterized by complex geometric domains that evolve over time combined with complex physical behaviors acting over a broad range of length scales. The effective simulation of such problems requires a set of methods to track the evolution of the geometry of the simulation domain and the spatial decomposition of that domain into the desired meshes. This paper presents a set of geometry model representation and mesh generation/update procedures developed and combined to support ballistic simulations of small and large caliber systems [23].

The modeling of a ballistic system requires methods that model the burning of the propellant, the flow of the gases produced, and the motion of the projectile [23, 40]. Central to the development of the needed models, and subsequent model discretizations, is accounting for the evolving interfaces associated with the burning propellant and moving projectile. In the case of the propellant, which is typically a set of pellets or blocks, the ability to model the physics of the burning interfaces requires a careful tracking of the interface geometry. In the case of the projectile, the tight geometric tolerances between the projectile and barrel includes geometric details that must be accurately represented for the proper modeling of the flow of propellant gases within the narrow regions between the projectile and barrel. The methods selected to represent and track the interfaces are critical to the ability to simulate these problems. Procedures for interface modeling employ either implicit representations of the interfaces [4, 10, 24, 32, 41, 42] or explicitly represent them in which case methods in which the mesh must explicitly move with the interface [11, 45, 46]. In this study, an explicit interface tracking approach is developed and used to represent interface physics (e.g., distinct phases, discontinuous physical parameters, etc.), and to allow very tight geometric tolerances associated with the projectile and barrel.

A stabilized finite element based procedure [50] is used to perform the computation. Compressible Navier-Stokes equations in the pressure-primitive form are used [15, 17]. An arbitrary Lagrangian-Eulerian (ALE) [19] frame is employed, where the diffusive flux function stays the same but the advective flux is updated with respect to the moving frame [37]. Streamline upwind Petrov-Galerkin (SUPG) based stabilization is employed [20, 38], where stabilization parameters are available in the Eulerian frame (e.g., see [16, 39]) and in the ALE frame (e.g., see [2]). For problems involving phase change, the conservation laws at the interface must be included. The stabilized finite element formulation of [50] that includes accounting for phase change with discontinuous interpolation at the interface is used. The generalized-alpha method [21] is used for time integration. A discontinuity-capturing operator (e.g., for shocks) is also used which adds diffusion that is proportional to the strong-form residual [18]. In addition, an explicit error estimator is used for mesh adaptation. Error estimation is performed using the variational multiscale (VMS) framework [13, 14] within which the stabilized finite element formulation fits.

The combination of complex evolving geometry, physical behavior happening over varying physical scales and application of explicit interface tracking places a set of requirements on the mesh that can only be addressed by the effective combination of mesh generation and adaptive mesh control techniques that includes:

- Geometric model representations that can effectively track the shape and topological changes of the evolving domain geometry. Such a representation can be built on a set of geometric modeling procedures designed for supporting non-manifold, multi-material, multi-dimensional, analysis geometries [3, 43].
- Fully automatic mesh generation methods that can generate meshes well matched to the mesh-based analysis methods being applied. The complex domains dictate the use of unstructured mesh generation methods while the solution procedures require control of element shapes. For the multiphase flow problems considered here, meshes employing semi-structured boundary layer methods [9, 29, 35, 49] are effective.
- It is a requirement in the application of explicit interface tracking to adapt the mesh to account for the evolving geometry. The minimal capability needed for this is a mesh deformation method focused on controlling the shape of elements. Methods that are limited to deforming elements on a fixed topology mesh can only account for a given limit of geometry change and can not deal with topological changes in the model. To be able to do the complete ballistics simulations desired, methods that introduce topological changes in the mesh are also required. In summary, a combination of mesh motion and regeneration/modification is required in such problems (e.g., see [12, 33, 47]).

Section 2 discusses the geometry operations required to support the procedures discussed in the subsequent sections for the effective tracking of two-phase and sliding interfaces. Section 3 discusses a semi-structured mesh generation procedure developed to create the desired meshes in the thin regions between the projectile and barrel. Section 4 overviews the combined mesh motion and local cavity adaptation procedures that have been developed to maintain element shape control as the domain geometry evolves, and to refine and coarsen the mesh, while maintaining semi-structured meshes in critical regions, to control discretization errors as desired. Section 5 demonstrates the application of the methods developed.

2 Geometry Setup and Updates

The computational domain is defined by a B-rep (boundary representation) geometric model which contains a set of hierarchical topological entities (vertices, edges, faces, regions) that are connected with ancillary entities (uses, loops, shells) [48]. In order to support evolving domain simulations, the geometric model must be constructed appropriately to support motion and deformation in the domain, as well as the requirements of sliding contact between bodies.

Rigid body motion is supported by applying a transformation matrix to the moving body's original position. For example, a simple translation is applied to the projectile body in the simulations shown in Figs. 3 and 8. Any orientation preserving (i.e., positive determinant) affine transformation may be applied, which extends support beyond rigid body motion to, for example, uniform isotropic or anisotropic scaling.

For more general deformations that are non-constant over the body and thus cannot be applied with a transformation matrix, a discrete mesh-based representation of the geometry is used, and the motion is directly applied to the vertices of the computational mesh of the deforming body. This method is used to model burning propellant grains in cannon simulations [50].

2.1 Topology Control

The need to perform continual topology updates in the case of sliding rigid body motion can be attained through the application of a suitable geometry construction. In ballistics, this need arises when the (rigid) projectile moves through the cannon into the ambient air. Three stages are considered for ballistic problems, namely, interior, intermediate and exterior ballistics. In interior ballistics, propellant burn occurs building up the pressure in the breech/back region which pushes the projectile towards the open end/exit of the cannon. During the intermediate stage, the projectile gains speed as it moves down the barrel and often results in a precursor shock (ahead of the projectile into the ambient air). Finally in the exterior stage, the projectile leaves the cannon entirely and moves through the ambient air while resulting in a cannon blast that propagates outwards. To demonstrate the continual topology updates in the sliding rigid body case, we consider a projectile that moves along the cannon including the barrel and muzzle brake, and then leaves the cannon entirely. If the projectile body was treated as fully in contact with the barrel at the CAD level, there would be a face created at the imprint, and the barrel breech side would be a separate region (Fig. 1). As the projectile exits the barrel and moves into the muzzle brake, the circular edge at barrel exit would need to be imprinted and moved along the projectile body and then eliminated when the contact is lost. The imprinted edge would cause the projectile face to be split into multiple faces and, once the projectile has passed the split faces, would need to be recombined. If the projectile also contacted the muzzle brake, more topological operations would be needed. Even for a simple muzzle brake geometry where the contacting face has rotational symmetry, several edges (see Fig. 2) would need to be imprinted and recovered. Without rotational symmetry, for example with holes on the contacting faces, the operations would be even more complex. Also, the association of the computational mesh entities to geometric model entities would have to be updated at every change in topology. All these operations would be not only difficult to implement but also computationally expensive.

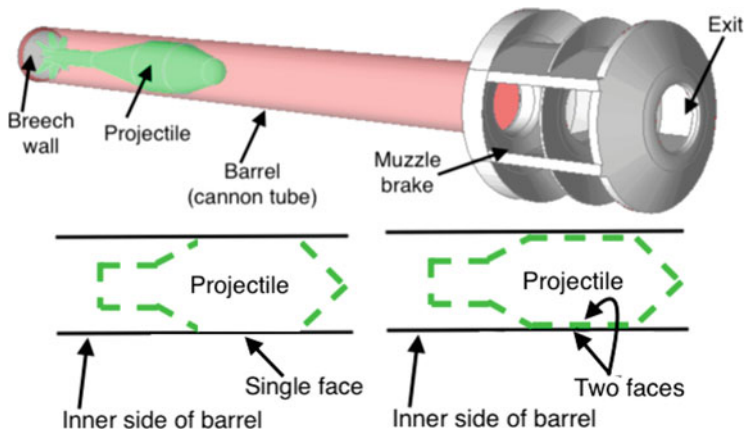


Fig. 1 A cannon with muzzle brake and a finned projectile (top). Topology schematic for true contact in CAD (bottom left) vs special contact for sliding motion (bottom right). Embedding the projectile in the air region with special/sliding contact (bottom right) and preserving the faces in contact prevents the breech (back) portion from being broken off into a separate region, ensures the projectile is in the air region throughout its motion, and renders topological changes unnecessary

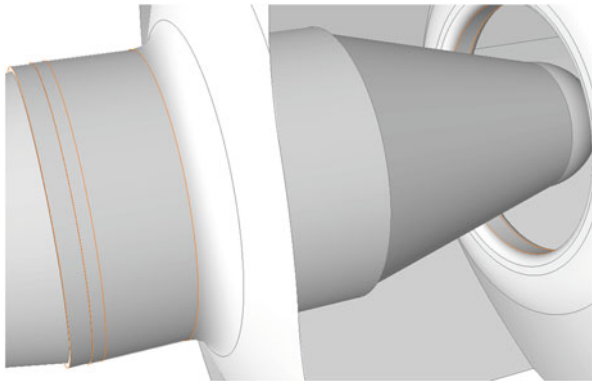


Fig. 2 Projectile contact with muzzle brake at CAD level would result in several edges (shown in orange) needing to be imprinted onto the projectile, its contacting face to be split, and then recovered at different stages as the projectile moves along its path

By considering, and tracking, the contact at the mesh level instead of at the geometric model level, topological changes at the CAD level can be eliminated entirely. Section 3 describes the meshing specializations developed for the purposes of maintaining and evolving this contact. At the CAD level, a “gap”, which can be of zero thickness, is specified between the contacting projectile and barrel faces, so that they are not treated as being in contact in the CAD model for topological considerations. The model region remains the same behind and in front of the projectile, which means the projectile carries out its entire motion while

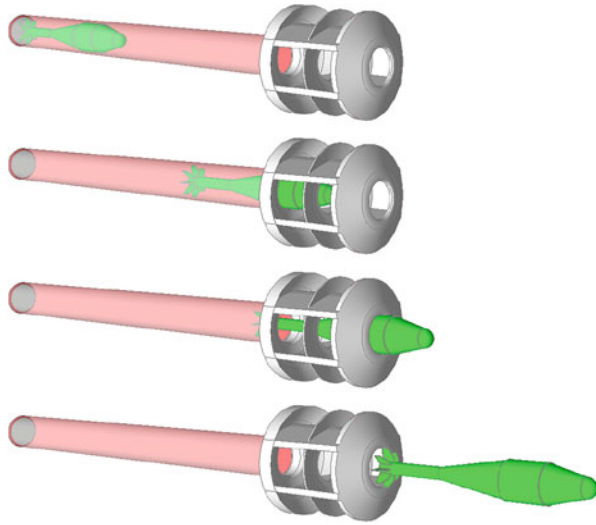


Fig. 3 Evolution of geometric model during projectile motion. The model is constructed with a zero thickness “gap” between the projectile and the barrel. This obviates the need for topology changes as the projectile moves along the barrel and muzzle brake and then exits the cannon. The outer “air” region is not shown

still embedded in this air region, with no CAD topology changes necessary at any time. The sliding bodies are kept as separate CAD parts in the native model. The topology of a moving body is connected to the model region in which it is embedded by ensuring that the outer shell of the moving (projectile) body is a shell in the embedded (air) region. The successful application of this approach to tracking the evolution of the model places specific requirements on the mesh generation and tracking procedures that are discussed in Sect. 3. Figure 3 shows the projectile successfully moving through the barrel and muzzle brake and exiting the cannon entirely without topological changes.

3 Specialized Meshing Extensions

Ballistics simulations exhibit strong gradients normal to surfaces due to high speed viscous boundary layer flow. They also contain areas that are thin compared to the desired mesh size in a particular direction, e.g., between a projectile rotating band and a rifled barrel. In these cases, highly anisotropic layered semistructured elements are desired at the interfaces to achieve solution accuracy. An aligned mesh of zero thickness may also be used to model sliding contact. Specialized extensions

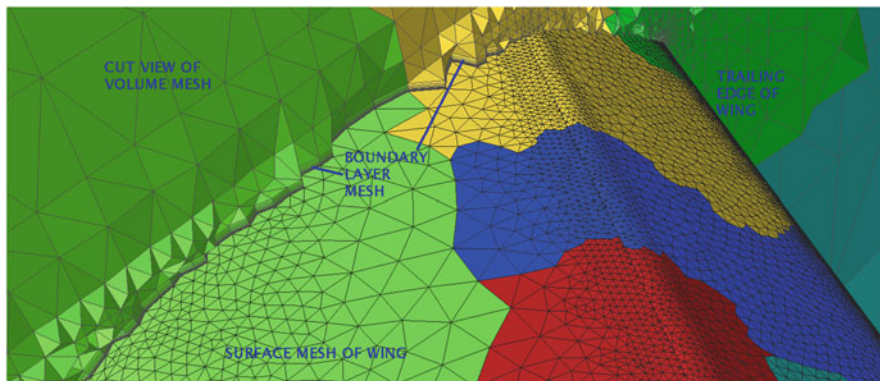


Fig. 4 Parallel mesh of an airplane wing with cutplane showing boundary layer and unstructured volume elements. Colors denote mesh parts of the partitioned mesh

to fully automatic mesh generation procedures have been developed to create such meshes in boundary layers and thin (including zero gap) sections.

3.1 *Boundary Layer Meshing*

Boundary layer meshes are semistructured anisotropic meshes with layered and graded elements typically used to model high gradients near no-slip surfaces. To construct these meshes, the unstructured surface mesh is inflated into the volume in a graded fashion along the local surface normals in stacks of layers up to a specified distance [9, 35]. The interior domain is then filled with unstructured elements. These meshes may be generated in a parallel distributed as well as multithreaded environment [44], see Fig. 4. In ballistics problems, boundary layer meshes are useful not only to resolve high speed viscous flows but also around propellant grains to capture steep gradients on each side of solid-gas interfaces [50] (Fig. 10).

3.2 *Automatic Thin Section Meshing*

Many simulation domains have areas that are geometrically thin compared to the mesh size in those areas. It is useful to detect these areas and automatically mesh them with aligned anisotropic meshes so that mesh quality and solution accuracy are retained. A specialized thin section meshing algorithm automatically detects these areas by projecting points on geometric entities between potential sources and destinations and determining whether they are within a specified thickness related to mesh size, and then automatically meshes the detected areas with extrusion-like

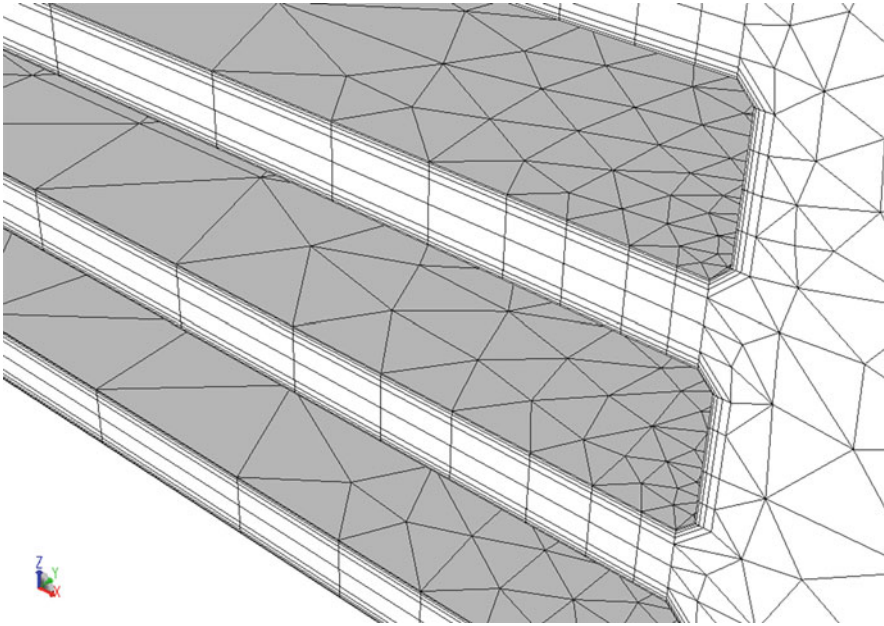


Fig. 5 Heat exchanger model with a graded anisotropic partial thin section mesh in the fins that connects to boundary layer mesh on the sides

aligned meshes [31]. While the typical use case of this algorithm is indeed sections that are thin, the main requirement is that sources and destinations project to each other so that an aligned mesh may be created. It is also not necessary for an entire model region to be thin: partial thin sections are supported as shown in the example of Fig. 5. The mesh in the thin section may have graded heights and connect to a boundary layer mesh as shown.

In cannon simulations, a thin section mesh is ideal to study the effect of high speed bleed flow in narrow gaps between the projectile and the barrel, especially in the case of a rifled barrel where the projectile band engages with the rifling. Here the gaps have “steps” due to the rifling [6] and it is necessary to consider these steps while creating thin section meshes. The thin section meshing procedure had to be extended for the projection and meshing algorithms to account for these steps. The enhancements include:

- Projections need to be carried out in both directions between the source and destination sides of a thin section, otherwise a step may “hide” one side from the other. Thus in the case of Fig. 6 (middle), a unidirectional projection from source to destination would exclude the areas next to the smaller holes. These are processed by projections in the other direction shown by the smaller arrows.
- The calculation of positions and connectivity during meshing is no longer regular and needs to be adjusted to align with the steps, as shown in Fig. 6 at bottom.

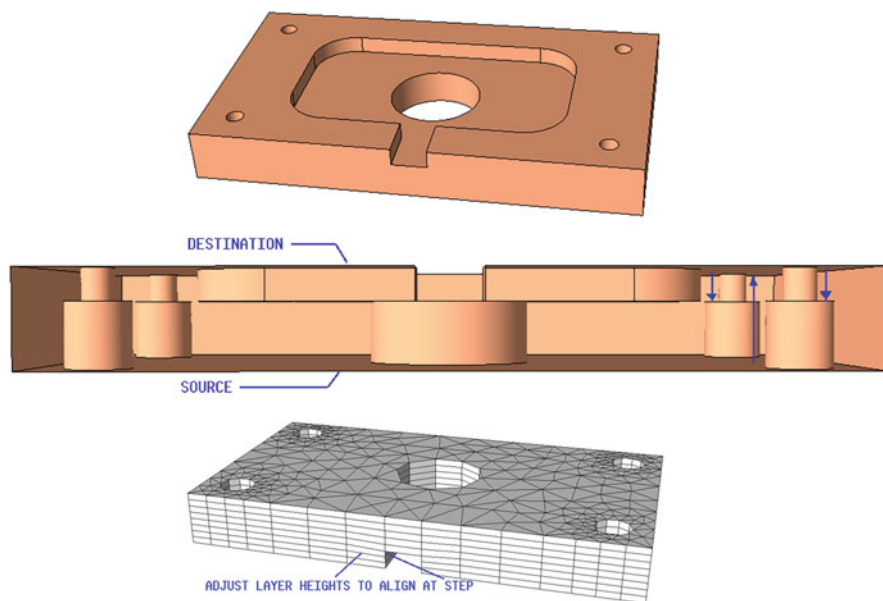


Fig. 6 Stepped thin section meshing. Top: Geometric model. Middle: Cut view showing need to project from destination to source. Bottom: Final mesh showing alignment at step corner

After successfully implementing these changes, an aligned, extruded, possibly anisotropic mesh is created that has a varying number of layers to account for the steps while still maintaining the required structure.

A thin section mesh is also used to simulate sliding motion between the projectile and the barrel. This is done to avoid the complexities and inefficiencies associated with topological changes at the CAD level (see Sect. 2). A zero gap thin section mesh in which pairs of aligned vertices across the section are coincident is instead used to represent this contact. In this case the mesh within the thin section as defined by the contacting bodies is degenerate. At the ends of the contact where the zero gap mesh connects to the surrounding mesh, constraints are applied to ensure the degeneracy is restricted to one element layer next to the contact. The zero gap thin section mesh is moved along with the projectile in an automatic manner. Information about the contact and the zero gap mesh that represents it is provided to the physics solver so that it can properly analyze this contact and simulate sliding motion.

In Fig. 7, the highlighted stationary barrel face is coincident with eight faces at the fins and one face at the central body of the projectile. Zero volume structured wedge elements are created to bridge the contacting surfaces. The barrel has a small step approaching the muzzle brake, and the zero gap mesh connects to a finite thin

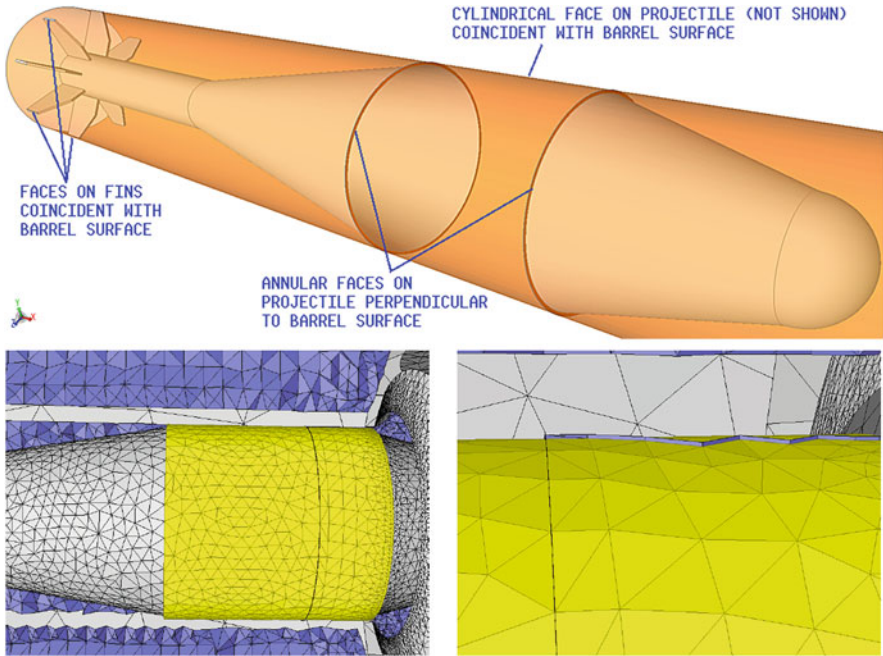


Fig. 7 Zero gap thin section mesh is created between coincident projectile and barrel faces. This connects to the mesh around it, including to a finite thin section mesh (a zoomed view is shown at bottom right) when the barrel steps up toward the muzzle brake

section mesh that is created in this narrow gap when the projectile reaches this step. The evolving mesh of a simulation with this configuration is seen in Fig. 8.

Thin section meshing is currently supported in serial only. Distributed parallel support is under development.

4 Mesh Update Methods

The mesh must be updated to remain consistent with the geometric model as the simulation evolves. To support general geometry changes both mesh motion and local mesh modification are used. At every timestep, the analysis computes the motion and deformation of moving bodies as well as the motion of mesh nodes on stationary bodies. The mesh motion is based on a mesh elasticity analogy [50]. Mesh entities on bodies undergoing rigid body motion or supported transformations (see Sect. 2) move with the geometry. Mesh entities on geometric boundaries are constrained to move along the boundary, and thus, reside on the boundary, by projecting their analysis specified moved positions on the model entity and using the mesh elasticity analogy again on these geometry/boundary conformant positions.

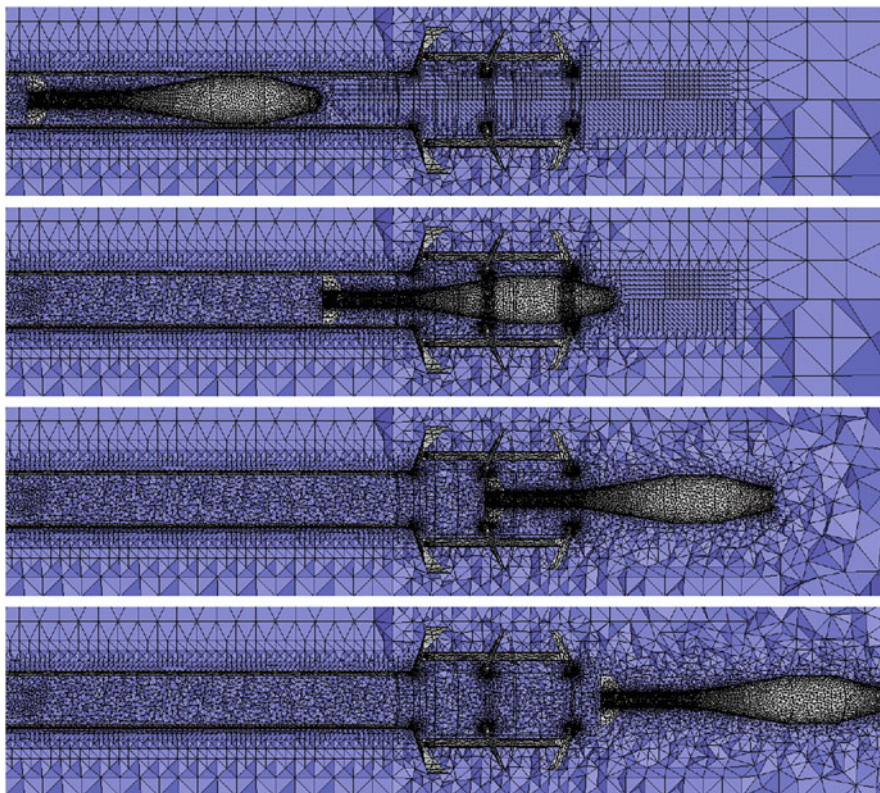


Fig. 8 Cross section of the volume mesh at various instances of projectile motion. The finer mesh on the projectile tail is graded to the surrounding domain. The finer mesh at the muzzle brake is graded onto the projectile body and tail fins. As the projectile moves and exits the cannon, the mesh in the barrel behind it and the mesh on the projectile graded from the muzzle brake are coarsened

The projection to the geometric entity is done by means of a local search for efficiency. In summary, this is achieved in two steps. In the first step, mesh elasticity analogy is used together with the geometric constrain to displace mesh entities on geometric boundaries in a conformant fashion and subsequently, mesh elasticity analogy is used again, where geometry/boundary conformant positions from the first step are employed, to move the rest of the (interior) mesh entities.

The analysis has a trigger to enable mesh modifications based on needing to maintain mesh validity, quality and desired resolution. When the moved mesh does not satisfy any of the specified criteria, mesh modification operations are applied to fulfill the criteria, so that subsequent steps can again be carried out using the mesh elasticity method, until modification is deemed necessary at a further step. The evolving domain analysis thus involves a loop consisting of several mesh motion steps followed by a mesh modification step when needed. Mesh modifications are based on local mesh cavity operations such as splits, swaps, collapses, local

remeshing [5, 7, 26–28]. They are supported in a distributed MPI-based as well as multithreaded environment [1, 30, 36]. Conformance of the mesh to the geometry is always maintained, thus any mesh vertices on boundaries that result from mesh modification operations are constrained to lie on the boundary [25]. Whenever a mesh modification operation is applied, the solution is transferred at the level of the mesh cavity through the use of a callback mechanism. For example, when edges of an element are split/refined, a list of the old elements as well as new elements are passed to the callback function which then accesses the solution values on the old elements and transfers them onto the new elements according to procedures encoded into the callback function. By providing the users with the cavity mesh before and after the mesh modification, users can develop callback functions that meet the needs of their analysis code. In the case of mesh refinement it is easy to develop callbacks that maintain the fields exactly, thus satisfying any constraints. More care is required in the case of collapse or swap operations. Linear interpolations over the mesh before mesh modification is currently being used to set nodal values.

As the mesh is modified, it is also adapted to satisfy the desired mesh resolution. As the domain evolves, the mesh size field [26] that defines the desired mesh resolution must also evolve. The moving body may have a finer or a coarser resolution than its surrounding, and sometimes both at different locations on the body. Any desired fine resolution must not only be retained during the motion but also graded to the adjacent mesh, to ensure mesh quality and solution accuracy. The grading may occur away from or toward the moving body depending on whether the body has a finer or coarser resolution than the surrounding. Further, any fine resolution that occurs solely due to this grading is reversed by appropriately coarsening the mesh for efficiency purposes when the source that caused this grading is no longer adjacent due to the motion.

The grading from fine to coarse mesh portions requires keeping track of distances from finer sources and adjusting surrounding sizes per a specified gradation rate. All information needed for this is available in the current instance of the evolved geometry/domain. However, the coarsening needed behind the motion relies on an initial mesh size field in the domain which is no longer present in the current mesh instance. This original mesh size field is accounted for at every mesh modification stage as an additional control. Note that the original mesh size field is defined on geometric entities and thus, it is available at each instance as the geometry/domain evolves.

Figure 8 illustrates such an evolving mesh adaptation. The mesh on the projectile tail is finer than that in the cannon interior, but the mesh on the muzzle brake entrances is finer than that on the projectile main body and tail fins. The projectile tail mesh is graded to the surrounding mesh in the interior while the finer muzzle brake mesh is graded onto the projectile body and tail fins. As the projectile moves further along, the mesh behind its tail and the mesh on its body and fins are no longer near the muzzle brake entrances and thus, coarsened.

4.1 *Update of Specialized Meshes*

As geometries evolve, the boundary layer and thin section meshes described in Sect. 3 must also be evolved/moved and adapted. The structure of these layered meshes as well as the desired resolution and anisotropy must be retained during these evolution and adaptation procedures.

We have developed an explicit repositioning method for the layered elements. The mesh motion within boundary layers employs boundary layer growth curve information. A growth curve is a collection or stack of mesh edges defined along the local normal direction to the boundary/interface. At an interface, growth curves on each side are treated independently based on the growth curve rules for the region on that side of the interface. Currently, a growth curve is geometrically graded as defined by four parameters: first layer thickness, growth ratio, total number of layers and growth direction or local unit normal vector. The first three parameters are either prescribed or computed adaptively [7]. When prescribed, the values are allowed to vary in space and time, and the same applies in the case of an adaptive determination, however, the values are computed automatically. The fourth parameter of growth direction is calculated based on the current geometry shape which evolves with time.

For meshes in the boundary layer, adaptation may occur in the in-plane as well as thickness directions [7, 44]. In the in-plane direction, the boundary layer mesh is refined or coarsened using edge splits or collapses to conform to a specified mesh size field typically obtained from error estimation procedures. Edge swaps may be also performed to improve mesh quality. Boundary layer structure is retained during adaptation by consistently propagating these mesh modification operations through individual boundary layer element stacks. For thickness adaptation, new layer thicknesses and number of layers may be specified. The unstructured mesh connected to the boundary layer is adjusted to allow thicknesses to change. Adaptive meshes for a Delery bump simulation [7], where transonic flow goes over a bump and becomes supersonic, are shown in Fig. 9.

Boundary layer adaptation is necessary to capture the high speed viscous flow in cannons as well as at the solid-gas interfaces during propellant grain burn. Parallel adapted meshes for a grain burn simulation are shown in Fig. 10.

For thin section meshes, in-plane adaptation is supported. Similar to boundary layer meshes, the structure of these meshes is maintained so that any mesh modification operations such as edge splits and swaps are carried out through the entire stack of layered elements. See Fig. 11.

4.2 *Trigger for Mesh Modification*

Mesh motion alone can lead to severely distorted meshes that can become invalid under a large motion/deformation in the geometry. To deal with this, we apply a combination of mesh motion and mesh modification. However, mesh modification

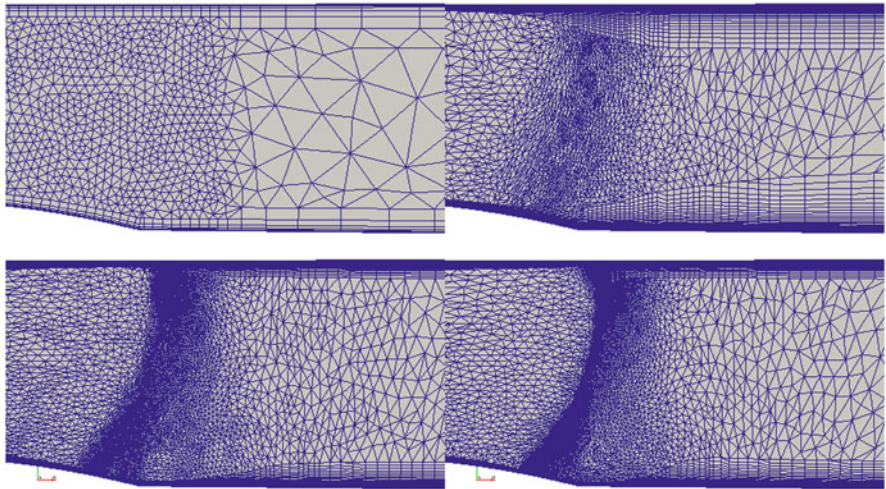


Fig. 9 Adaptive meshes for Delery bump simulation. The boundary layer mesh has been adapted in both in-plane and thickness directions

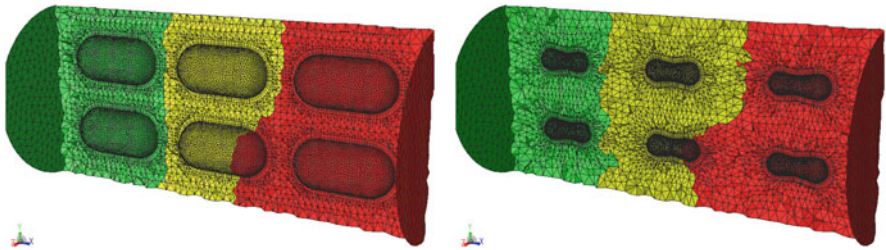


Fig. 10 Geometry deformation and parallel mesh adaptation for propellant burn simulation. A layered mesh is used at the grain boundary solid-gas interface and adapted as the grains shrink and deform. Colors denote mesh parts of the partitioned mesh

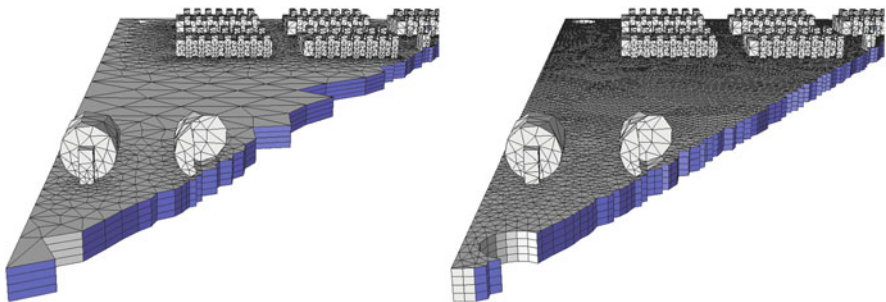


Fig. 11 Thin section mesh adaptation. Cut view through initial (left) and adapted (right) meshes shows refinement maintaining conformance to geometry

leads to connectivity changes that are time consuming to perform and update into the solver. Thus, it is desirable to apply mesh modification at a lower frequency (i.e., not at every time step) and a mechanism is needed that can trigger mesh modification only when necessary.

Two criteria are used to trigger mesh modification. If either is met then mesh modification is executed. Both criteria rely on the mesh size field which is determined at each time step, which is relatively inexpensive. It is either prescribed based on the current state of the geometry (i.e., it evolves in time with the geometry), computed based on an error estimator, or a combination of the two.

The first criterion is based on mesh resolution and a mesh modification is triggered when the current mesh resolution is not satisfactory at any given time step. To do this, edge lengths are checked with respect to the mesh size field and (if applicable) the current/estimated element error is checked against the target local error [8, 34].

The second criterion targets mesh quality and requires element shape quality to be above a threshold. Generally speaking, mesh quality refers to geometric properties of the mesh elements such as local volume, orientation, aspect ratio, etc. Several element shape quality measures are available in the literature, such as mean ratio, skewness and so on [22]. Note that it is necessary to use a signed quality measure such that any inverted/invalid element is detected with a negative value. In this work, we use a normalized mean ratio in the transformed/metric space defined by the given mesh size field (e.g., see [26]). For a tetrahedron it is given as:

$$\gamma_e^M = \frac{(\sqrt{72}V_e^M)^{\frac{2}{3}}}{\frac{1}{6} \sum_{i=1}^6 (l_i^M)^2} \quad (1)$$

where V_e^M is the element volume in the metric space and l_i^M is the length of the i th element edge in the metric space. The normalized mean ratio in the metric space can attain a maximum value of 1 for an element with the “best” shape (e.g., a regular tetrahedron). Any other suitable measure of the element shape quality can be used. Mesh modification is triggered when the shape quality measure for any element in the mesh is below the threshold value. For the current choice of element shape quality measure, 0.3 is used as the threshold value.

5 Results

Two problems are considered to demonstrate the utility of the current parallel interface tracking approach.

5.1 Droplet Case with Phase Change

The first problem involves 6 droplets undergoing phase change with arbitrary deformations. In this case, we employ ALE-based stabilized finite element formulation with phase change and discontinuous interpolation at the interface [50]. The six droplets undergo phase change from denser liquid to lighter gas inside a chamber. The simulation goal is to predict the rise of pressure and temperature in the chamber. A fully 3D transient phase change simulation is performed, see [50] for more details. Figure 12 shows the center plane of the problem with 6 droplets in an adiabatic cylindrical chamber with no-slip walls. At the beginning, each droplet is a 2 mm long circular cylinder with hemispherical end caps of 0.5 mm radius. The phase change rate is governed by the Vieille's law:

$$u_p = a(p^+)^n \quad (2)$$

where p^+ is the pressure of the surrounding gas, while the exponent is set to be $n = 0.7$ and the pre-factor to be $a = 7.9\text{e-}5 \text{ m/(sPa}^n\text{)}$. The initial pressure of the closed chamber is 1 atm. As the droplets undergo phase change from denser phase to lighter phase, the pressure increases and speeds up the phase change rate in return. Layered elements are used on both sides of the interface.

Figure 13 shows mesh and solution fields around one of the droplets at three time instances (i.e., at $t = t_0, t_1$ and t_2). A cut through the mesh is shown, where the change in droplet size/volume is clear between the first and last instances. Solution fields of velocity magnitude and temperature are shown, where at each instance the former is shown in the left half of the droplet and the latter in the right half. $t = t_0$ is near the beginning while at $t = t_1$ and t_2 mesh modification is automatically triggered as discussed above. Thus, there are two mesh and solution states at $t = t_1$ and t_2 , one after mesh motion (only) and the other after mesh modification. In this case, the mesh size field is prescribed to be finer near the interface and is based on the current state of the geometry. The structure and resolution of the anisotropic

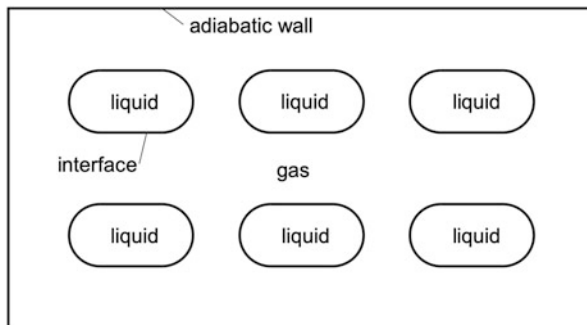


Fig. 12 Setup of the 6-droplet case

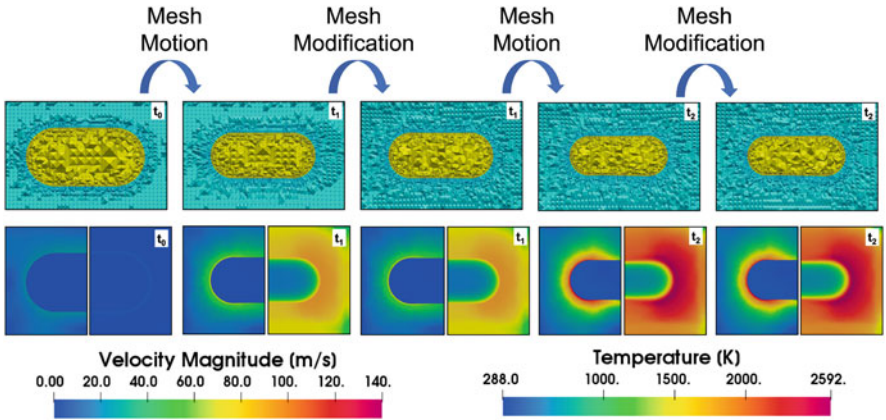


Fig. 13 Mesh and solution fields around one of the 6 droplets at three time instances, $t = t_0, t_1$ and t_2 , where $t = t_0$ is near the beginning while at $t = t_1$ and t_2 mesh modification is triggered

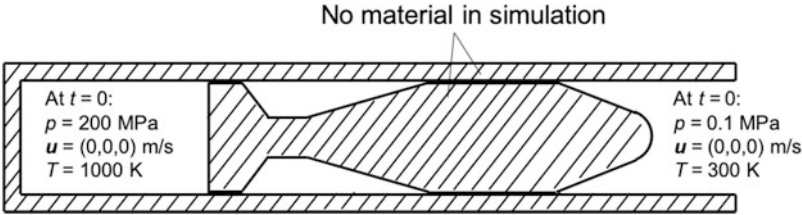


Fig. 14 Setup of the projectile case

layered elements are maintained during mesh motion and modification. The overall mesh quality is maintained to be above the threshold value of 0.3. The solution fields are well resolved, especially at the interface including discontinuity in the normal velocity component and steep normal gradient in the velocity and temperature fields.

5.2 Projectile Case with Rigid Motion

A finned projectile inside a pressurized cannon is considered. In this case, the projectile velocity and flow field at the exit of the cannon are of interest. Figure 14 shows the problem setup, where high pressure and temperature is set at the left/closed end of the cannon. The high pressure pushes the projectile towards the open end. The projectile is considered to be a rigid object. It translates axially along the tube. It is 711 mm in length and has 8 fins. The inner length of the cannon is 2000 mm, while its inner diameter is 120 mm. The outer boundary is set as outflow and placed at a sufficiently long distance away from the cannon. Currently, no-slip and zero heat flux conditions are set on all the walls, where no-slip condition due

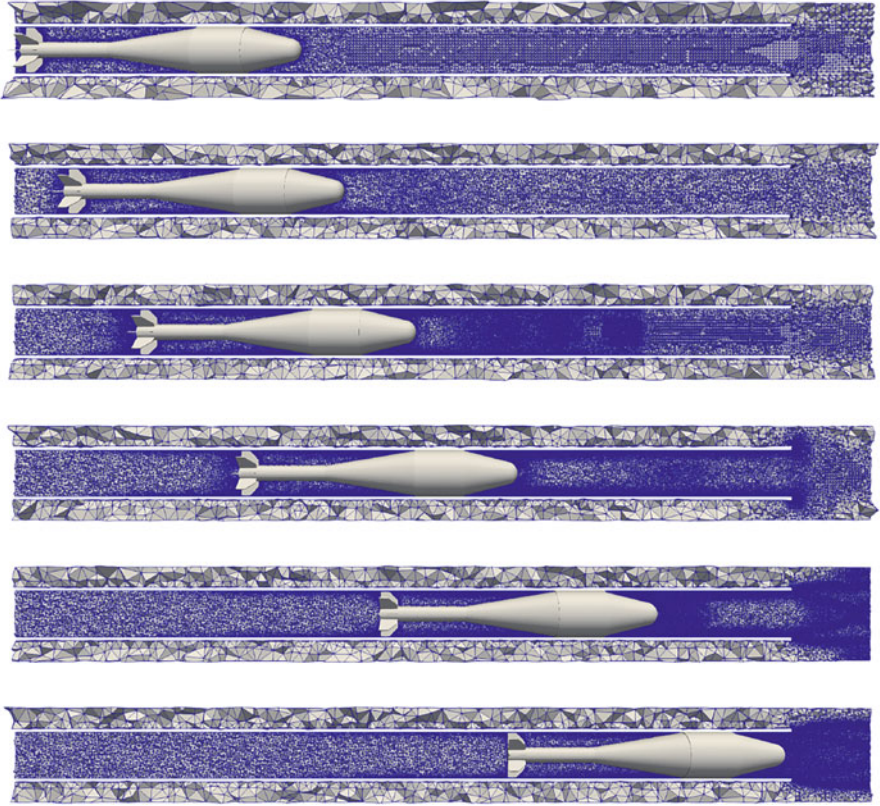


Fig. 15 Mesh at six different instances for the adapted projectile case

to the projectile is given precedence at the contact between projectile and inner side of the cannon. The projectile starts at 20 mm from the left inner side of the cannon and moves about 1300 mm in 6 ms such that the projectile reaches near the exit of the cannon. Error-based mesh adaptation is used for this case. VMS-based error estimator for the velocity in the L_2 norm is used to drive mesh adaptation.

Evolution of the geometry and mesh is shown in Fig. 15 at six instances as the projectile moves from the closed to open end of the cannon. A cut through the mesh is shown. A finer mesh is seen in regions of interest, especially in regions with shock, i.e., mesh adaptation focuses resolution in the shock regions. The minimum mesh quality is maintained to be 0.3 throughout the span of this simulation.

Figure 16 shows the Mach number together with the adapted mesh at four instances (only ambient air region is shown). In the first two instances, the adapted

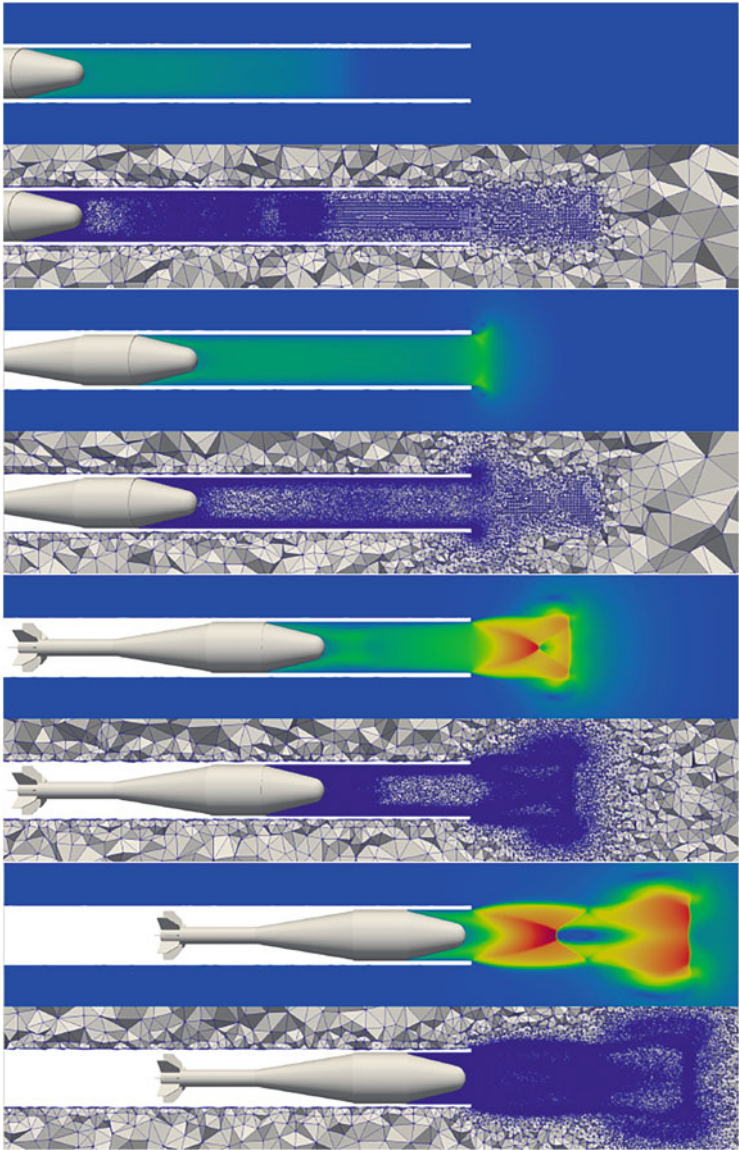


Fig. 16 Mach number and mesh for the projectile case at four different instances of $t = 2.5, 3.5, 4.5$ and 5.5 ms

mesh refines near the precursor wave and near the exit of the cannon. In the later two instances, the complex shock structure, which mesh adaptation has nicely resolved, is formed. To amplify on this aspect, a zoomed view near the exit of the cannon is shown in Fig. 17 at the two later instances.

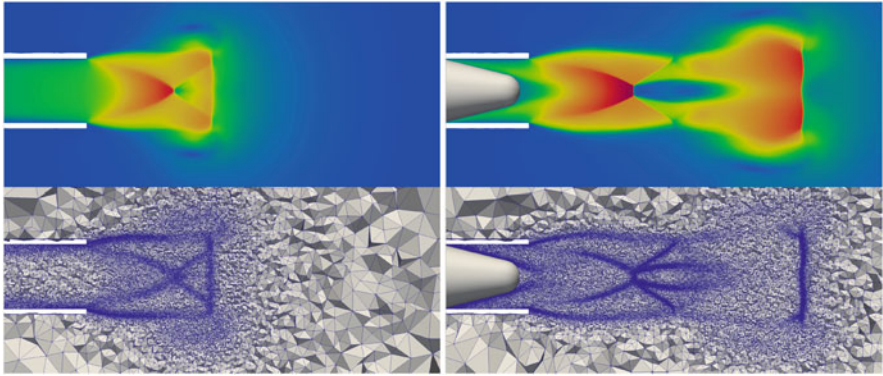


Fig. 17 Zoomed view of Mach number and mesh for the projectile case at two later instances of $t = 4.5$ and 5.5 ms

6 Closing Remarks

A set of geometric tracking, mesh generation and mesh update procedures have been developed that can support a wide range of flow simulations that include the need to track evolving geometry. A new approach for tracking the geometric changes caused by sliding interfaces that avoids the need for repeated local topological updates has been presented. New methods to address the creation of well controlled semi-structured layered meshes in the presence of heavily constrained geometry caused by thin sections that can contain steps were presented. To account for the evolution of the geometry and control of mesh discretization errors, a combined mesh motion and local mesh modification procedure to perform mesh updates at the appropriate time steps has been developed.

To demonstrate the capabilities of the developed procedures, example problems relevant to ballistics simulation were presented. Ongoing developments are targeting full support for a more complete set of topological changes caused by evolving geometry, in particular being able to handle phase change cases where regions are completely eliminated or are split into multiple regions. Efforts are ongoing to continue to improve the mesh adaptation procedures to more effectively control the mesh in the complex thin sections as the simulation proceeds.

Acknowledgments This material is based upon work supported by the US Army grants W911NF1410301, W911NF16C0117 and W911NF19C0094.

References

1. Alauzet, F., Li, X., Seol, E., Shephard, M.: Parallel anisotropic 3D mesh adaptation by mesh modification. *Eng. Comput.* **21**(3), 247–258 (2006)
2. Bazilevs, Y., Calo, V.M., Hughes, T.J.R., Zhang, Y.: Isogeometric fluid–structure interaction: theory, algorithms, and computations. *Comput. Mech.* **43**(1), 3–37 (2008)
3. Beall, M., Walsh, J., Shephard, M.: A comparison of techniques for geometry access related to mesh generation. *Eng. Comput.* **20**(3), 210–221 (2004)
4. Boettinger, W.J., Warren, J.A., Beckermann, C., Karma, A.: Phase-field simulation of solidification. *Ann. Rev. Mater. Res.* **32**(1), 163–194 (2002)
5. Caplan, P.C., Haimes, R., Darmofal, D.L., Galbraith, M.C.: Four-dimensional anisotropic mesh adaptation. *Comput. Aided Design* **129**, 102915 (2020)
6. Carson, R., Sahni, O.: Numerical investigation of propellant leak methods in large-caliber cannons for blast overpressure attenuation. *Shock Waves* **24**(6), 625–638 (2014)
7. Chitale, K., Sahni, O., Shephard, M., Tendulkar, S., Jansen, K.: Anisotropic adaptation for transonic flows with turbulent boundary layers. *AIAA J.* **53**(2), 367–378 (2015)
8. Galimov, A., Sahni, O., Lahey, R., Shephard, M., Drew, D., Jansen, K.: Parallel adaptive simulation of a plunging liquid jet. *Acta Math. Sci.* **30**(2), 522–538 (2010)
9. Garimella, R., Shephard, M.: Boundary layer mesh generation for viscous flow simulations. *Int. J. Numer. Methods Eng.* **49**(1–2), 193–218 (2000)
10. Gil, A.J., Carreno, A.A., Bonet, J., Hassan, O.: An enhanced immersed structural potential method for fluid–structure interaction. *J. Comput. Phys.* **250**, 178–205 (2013)
11. Glimm, J., Grove, J.W., Li, X.L., Shyue, K.M., Zeng, Y., Zhang, Q.: Three-dimensional front tracking. *SIAM J. Sci. Comput.* **19**(3), 703–727 (1998)
12. Hassan, O., Sørensen, K.A., Morgan, K., Weatherill, N.P.: A method for time accurate turbulent compressible fluid flow simulation with moving boundary components employing local remeshing. *Int. J. Numer. Methods Fluids* **53**(8), 1243–1266 (2007)
13. Hauke, G., Doweidar, M., Miana, M.: Proper intrinsic scales for a-posteriori multiscale error estimation. *Comput. Methods Appl. Mech. Eng.* **195**(33), 3983–4001 (2006)
14. Hauke, G., Fuster, D., Lizarraga, F.: Variational multiscale a posteriori error estimation for systems: the Euler and Navier–Stokes equations. *Comput. Methods Appl. Mech. Eng.* **283**, 1493–1524 (2015)
15. Hauke, G., Hughes, T.J.R.: A unified approach to compressible and incompressible flows. *Comput. Methods Appl. Mech. Eng.* **113**(3–4), 389–395 (1994)
16. Hauke, G.: Simple stabilizing matrices for the computation of compressible flows in primitive variables. *Comput. Methods Appl. Mech. Eng.* **190**(51–52), 6881–6893 (2001)
17. Hauke, G., Hughes, T.J.R.: A comparative study of different sets of variables for solving compressible and incompressible flows. *Comput. Methods Appl. Mech. Eng.* **153**(1–2), 1–44 (1998)
18. Hughes, T., Mallet, M.: A new finite element formulation for computational fluid dynamics: IV. A discontinuity-capturing operator for multidimensional advective-diffusive systems. *Comput. Methods Appl. Mech. Eng.* **58**(3), 329–336 (1986)
19. Hughes, T.J., Liu, W.K., Zimmermann, T.K.: Lagrangian–Eulerian finite element formulation for incompressible viscous flows. *Comput. Methods Appl. Mech. Eng.* **29**(3), 329–349 (1981)
20. Hughes, T.J.R., Scovazzi, G., Tezduyar, T.E.: Stabilized methods for compressible flows. *J. Sci. Comput.* **43**(3), 343–368 (2010)
21. Jansen, K.E., Whiting, C.H., Hulbert, G.M.: A generalized- α method for integrating the filtered Navier–Stokes equations with a stabilized finite element method. *Comput. Methods Appl. Mech. Eng.* **190**(3–4), 305–319 (2000)
22. Knupp, P.: Algebraic mesh quality metrics for unstructured initial meshes. *Finite Elements Anal. Design* **39**(3), 217–241 (2003)
23. Krier, H., Summerfield, M.: Interior Ballistics of Guns (Progress in Astronautics and Aeronautics), vol. 66. American Institute of Aeronautics & Astronautics (1979)

24. Lafaurie, B., Nardone, C., Scardovelli, R., Zaleski, S., Zanetti, G.: Modelling merging and fragmentation in multiphase flows with surfer. *J. Comput. Phys.* **113**(1), 134–147 (1994)
25. Li, X., Shephard, M., Beall, M.: Accounting for curved domains in mesh adaptation. *Int. J. Numer. Methods Eng.* **58**(2), 247–276 (2003)
26. Li, X., Shephard, M., Beall, M.: 3D anisotropic mesh adaptation by mesh modification. *Comput. Methods Appl. Mech. Eng.* **194**(48–49), 4915–4950 (2005)
27. Loseille, A., Alauzet, F., Menier, V.: Unique cavity-based operator and hierarchical domain partitioning for fast parallel generation of anisotropic meshes. *Comput. Aided Design* **85**, 53–67 (2017)
28. Loseille, A., Lohner, R.: Cavity-based operators for mesh adaptation. In: 51st AIAA Aerospace Sciences Meeting Including the New Horizons Forum and Aerospace Exposition, p. 152 (2013)
29. Loseille, A., Löhner, R.: Robust boundary layer mesh generation. In: *Proceedings of the 21st International Meshing Roundtable*, pp. 493–511. Springer, Berlin (2013)
30. Lu, Q., Shephard, M., Tendulkar, S., Beall, M.: Parallel mesh adaptation for high-order finite element methods with curved element geometry. *Eng. Comput.* **30**(2), 271–286 (2014)
31. Luo, X., Shephard, M., Yin, L., O'Bara, R., Nastasia, R., Beall, M.: Construction of near optimal meshes for 3D curved domains with thin sections and singularities for p-version method. *Eng. Comput.* **26**(3), 215–229 (2010)
32. Nagrath, S., Jansen, K.E., Lahey Jr. R.T.: Computation of incompressible bubble dynamics with a stabilized finite element level set method. *Comput. Methods Appl. Mech. Eng.* **194**(42–44), 4565–4587 (2005)
33. Quan, S., Schmidt, D.P.: A moving mesh interface tracking method for 3D incompressible two-phase flows. *J. Comput. Phys.* **221**(2), 761–780 (2007)
34. Rodriguez, J., Sahni, O., Lahey, R., Jansen, K.: A parallel adaptive mesh method for the numerical simulation of multiphase flows. *Comput. Fluids* **87**, 115–131 (2013)
35. Sahni, O., Jansen, K., Shephard, M., Taylor, C., Beall, M.: Adaptive boundary layer meshing for viscous flow simulations. *Eng. Comput.* **24**(3), 267–285 (2008)
36. Sahni, O., Ovcharenko, A., Chitale, K., Jansen, K., Shephard, M.: Parallel anisotropic mesh adaptation with boundary layers for automated viscous flow simulations. *Eng. Comput.* **33**(4), 767–795 (2017)
37. Scovazzi, G.: Galilean invariance and stabilized methods for compressible flows. *Int. J. Numer. Methods Fluids* **54**(6–8), 757–778 (2007)
38. Sevilla, R., Hassan, O., Morgan, K.: An analysis of the performance of a high-order stabilised finite element method for simulating compressible flows. *Comput. Methods Appl. Mech. Eng.* **253**, 15–27 (2013)
39. Shakib, F., Hughes, T.J.R., Johan, Z.: A new finite element formulation for computational fluid dynamics: X. The compressible Euler and Navier–Stokes equations. *Comput. Methods Appl. Mech. Eng.* **89**(1–3), 141–219 (1991)
40. Stiefel, L.: *Gun Propulsion Technology (Progress in Astronautics and Aeronautics)*, vol. 109. American Institute of Aeronautics & Astronautics (1988)
41. Sussman, M., Puckett, E.G.: A coupled level set and volume-of-fluid method for computing 3d and axisymmetric incompressible two-phase flows. *J. Comput. Phys.* **162**(2), 301–337 (2000)
42. Sussman, M., Smereka, P., Osher, S., et al.: A level set approach for computing solutions to incompressible two-phase flow. Technical report, University of California, Los Angeles, 1994
43. Tendulkar, S., Beall, M., Nastasia, R., Downie, B., Klaas, O., Shephard, M., Sahni, O.: Analysis-driven geometry and meshing for large scale simulations. In: *NAFEMS World Congress (NWC)*, Quebec, Canada, June 2019
44. Tendulkar, S., Beall, M., Nastasia, R., Sahni, O., Tran, S., Shephard, M.: Advanced meshing and mesh adaptation for complex flow problems. In: *NAFEMS World Congress (NWC)*, San Diego, CA, June 2015
45. Tryggvason, G., Bunner, B., Esmaeeli, A., Juric, D., Al-Rawahi, N., Tauber, W., Han, J., Nas, S., Jan, Y.-J.: A front-tracking method for the computations of multiphase flow. *J. Comput. Phys.* **169**(2), 708–759 (2001)

46. Unverdi, S.O., Tryggvason, G.: A front-tracking method for viscous, incompressible, multi-fluid flows. *J. Comput. Phys.* **100**(1), 25–37 (1992)
47. Wan, J., Kocak, S., Shephard, M.: Automated adaptive 3D forming simulation processes. *Eng. Comput.* **21**(1), 47–75 (2005)
48. Weiler, K.: The radial edge structure: a topological representation for non-manifold geometric boundary modeling. In: *Geometric Modeling for CAD Applications*, pp. 3–36. Elsevier, Amsterdam (1988)
49. Xie, Z.Q., Sevilla, R., Hassan, O., Morgan, K.: The generation of arbitrary order curved meshes for 3D finite element analysis. *Comput. Mech.* **51**(3), 361–374 (2013)
50. Zhang, Y., Chandra, A., Yang, F., Shams, E., Sahni, O., Shephard, M., Oberai, A.: A locally discontinuous ALE finite element formulation for compressible phase change problems. *J. Comput. Phys.* **393**, 438–464 (2019)

High-Order Implicit Shock Tracking (HOIST)

Andrew Shi, Per-Olof Persson, and Matthew J. Zahr

Abstract We present a framework for resolving discontinuous solutions of conservation laws using *implicit tracking* and a high-order discontinuous Galerkin (DG) discretization. Central to the framework is an optimization problem and associated sequential quadratic programming solver which simultaneously solves for a discontinuity-aligned mesh and the corresponding high-order approximation to the flow that does not require explicit meshing of the *a priori* unknown discontinuity surface. We utilize an error-based objective function that penalizes violation of the DG residual in an enriched test space, which endows the method with r -adaptive behavior: mesh nodes move to track discontinuities with element faces and improve the conservation law approximation in smooth regions of the flow. This method is shown to deliver highly accurate solutions on coarse, high-order discretizations without nonlinear stabilization and recover optimal convergence rates $O(h^{p+1})$ for problems with discontinuous solutions. We demonstrate this framework on a series of inviscid steady and unsteady conservation laws, the latter of which using both a space-time and method of lines discretization.

1 Introduction and Background

It is widely believed that higher fidelity is required for problems with propagating waves, turbulent fluid flow, nonlinear interactions, and multiple scales [34]. This has resulted in a significant interest in high-order accurate methods, such as discontinuous Galerkin (DG) methods [5, 17], which have the potential to produce accurate solutions on coarse meshes. Among the most significant challenges associated

A. Shi · P.-O. Persson (✉)

Department of Mathematics, University of California, Berkeley, CA, USA

e-mail: andrewshi94@berkeley.edu; persson@berkeley.edu

M. J. Zahr

Department of Aerospace and Mechanical Engineering, University of Notre Dame, Notre Dame, IN, USA

e-mail: mzahr@nd.edu

with high-order methods is their sensitivity to under-resolved features, in particular for nonlinear problems where the spurious oscillations often cause a breakdown of the numerical solvers. This is exacerbated for problems with shocks where the low dissipation associated with high-order methods is insufficient to stabilize the solution. Since shocks are present in many important problems in fields such as aerospace, astrophysics, and combustion, this poses a fundamental barrier to widespread adoption of these methods.

Most of the techniques for addressing shocks are based on *shock capturing*, that is, the numerical discretization somehow incorporates the discontinuities independently of the computational grid. One simple method is to use a sensor that identifies the mesh elements that contain shocks, and reduce their polynomial degrees [2, 4]. Related, more sophisticated approaches include limiting, such as the weighted essentially non-oscillatory (WENO) schemes [15, 18, 21]. For high-order methods, artificial viscosity has also proven to be competitive, since it can smoothly resolve the jumps in the solution without introducing additional discontinuities between the elements [25]. The main problem with all these approaches is they lead to globally first-order accurate schemes. This can be remedied by local mesh refinement around the shock (*h*-adaptivity) [9], although the anisotropic high-order mesh adaptation is challenging and requires highly refined elements near the shock.

An alternative approach is *shock tracking* or *shock fitting*, where the computational mesh is moved such that its faces are aligned with the discontinuities in the solution. This is natural in the setting of a DG method since the numerical scheme already incorporates jumps between the elements and the approximate Riemann solvers employed on the element faces handle the discontinuities correctly. However, it is a difficult meshing problem since it essentially requires generating a fitted mesh to the (unknown) shock surface. Many previous approaches employ specialized formulations and solvers which are dimension-dependent and do not easily generalize [3, 14, 16] and/or are limited to relatively simple problems [29, 30, 33]. In addition, early approaches to shock fitting have been applied to low-order schemes where the relative advantage over shock capturing is smaller than for high-order methods [1, 31]. Explicit shock tracking strategies [23, 26] have been proposed which are able to attain high-order accuracy in the presence of shocks, but require a specialized strategy to explicitly track the shock separately from the remainder of the flow. These methods are not easily applicable to discontinuities whose topologies not known *a priori*. For these reasons, shock tracking is largely not used in practical CFD today.

In [35, 37], we introduced a novel approach to shock tracking for steady conservation laws (including space-time formulations of time-dependent problems) that does not require explicitly generating a mesh of the unknown discontinuity surface. Rather, the conservation law is discretized on a mesh without knowledge of the discontinuity surface and an optimization problem is formulated such that its solution is the pair $(\mathbf{u}, \mathbf{x})$, where $\mathbf{x}$ is the positions of the mesh nodes that cause element faces to align with discontinuities in the flow and $\mathbf{u}$ is the solution of discretized conservation law on the mesh defined by $\mathbf{x}$. That is, discontinuity tracking is implicitly achieved through the solution of an optimization problem and

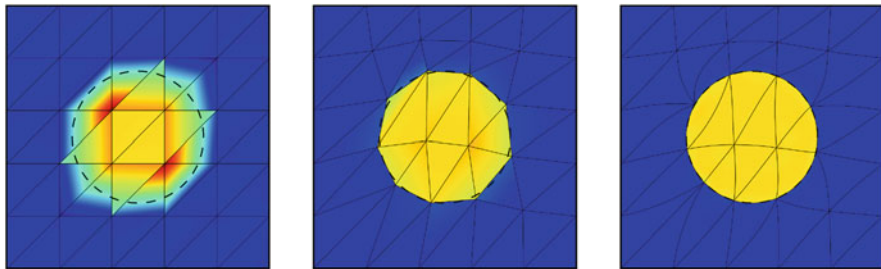


Fig. 1 L^2 projection of a piecewise constant function onto a non-aligned (*left*) vs. discontinuity-aligned mesh with linear (*middle*) and cubic (*right*) elements and corresponding solution. See Example 4.1 of [35] for details

will be referred to as *implicit shock tracking*. While this approach works with any discretization that allows for inter-element discontinuities, we focus on high-order DG methods due to the high degree of accuracy attainable on coarse meshes, proper treatment of discontinuities with approximate Riemann solvers, and the ability to use curved elements to track discontinuities with curvature (Fig. 1). The implicit tracking optimization problem proposed in [37] minimizes the violation of the DG residual in an enriched test space while enforcing that the standard DG (same test and trial space) equations are satisfied. This objective function is a surrogate for violation of the infinite-dimensional weak formulation of the conservation law, which endows the method with r -adaptive behavior: it promotes alignment of the mesh with discontinuities and adjusts nodes in smooth region to improve approximation of the conservation law. The optimization problem is solved using a sequential quadratic programming method with a Levenberg-Marquardt Hessian approximation that simultaneously converges the mesh and solution to their optimal values, which never requires the fully converged DG solution on a non-aligned mesh and does not require nonlinear stabilization. The combination of implicit tracking with a DG discretization leads to a high-order accurate numerical method that has been shown to provide accurate approximations to high-speed inert [35, 37] and reacting flows [36].

In [27], we further extend the framework developed in [35, 37] for steady conservation laws and space-time formulations of unsteady conservation laws to a method of lines discretization approach. While space-time methods are attractive for a number of reasons, the method of lines approach tends to be more practical for complex problems. The key ingredients of the method of lines approach are: (1) an Arbitrary Lagrangian-Eulerian formulation of the conservation law to handle the deforming mesh (which deforms to track the shock through the domain), (2) semi-discretization with DG to obtain a system of ordinary differential equations, (3) high-order temporal discretization with a diagonally implicit Runge Kutta (DIRK) method, and (4) implicit shock tracking at each time step following the approach in [35, 37]. We utilize a Rankine-Hugoniot based procedure to construct

an initial guess for each timestep which significantly accelerates the performance of the solver and allows to us to achieve Newton-like convergence.

To our knowledge, the only other approach to implicit shock tracking was proposed in [6–8], where the authors enforce a DG discretization with unconventional numerical fluxes and the Rankine-Hugoniot interface conditions in a minimum-residual sense. Interestingly, enforcement of the interface condition circumvented traditional stability requirements for the DG numerical fluxes, allowing them to solely rely on fluxes interior to an element. Their method was shown to successfully track even complex discontinuity surfaces and provide high-order approximations to the conservation law on traditionally coarse, high-order meshes. Their initial formulation was extended to viscous problems in [20] and recast as a least-square discontinuous Galerkin method in [19]. While notably different from this and our approach, a related early optimization-based approach to solving for meshes that are optimal for a given problem in the context of FEM appears in [32], though for simpler problems that do not contain discontinuities.

The remainder of this chapter is organized as follows. Section 2 introduces the governing system of inviscid conservation laws and its discretization using a DG method. Section 3 presents the error-based objective function and the constrained optimization framework in both the steady and unsteady case. Section 4 discusses practical details required for the proposed tracking framework such as initialization of the SQP solver and topological mesh operations to remove small elements. Finally, Sect. 5 presents a number of numerical experiments that demonstrate the method on a variety of steady and unsteady flows using coarse, high-order meshes and demonstrates high-order convergence of the method. For unsteady flows, we compare the space-time and method of lines approaches and discuss cases when one is more suitable.

2 Governing Equations and High-Order Discretization

Consider a general system of m inviscid conservation laws, defined on the fixed domain $\Omega \subset \mathbb{R}^d$ and subject to appropriate boundary conditions,

$$\frac{\partial U}{\partial t} + \nabla \cdot F(U) = 0 \quad \text{in } \Omega \times [0, T] \quad (1)$$

where $U : \Omega \times [0, T] \rightarrow \mathbb{R}^m$ is the solution of the system of conservation laws, $F : \mathbb{R}^m \rightarrow \mathbb{R}^{m \times d}$ is the flux function, $\nabla := (\partial_{x_1}, \dots, \partial_{x_d})$ is the gradient operator in the physical domain such that $\nabla W(x, t) = [\partial_{x_1} W(x, t) \cdots \partial_{x_d} W(x, t)] \in \mathbb{R}^{N \times d}$ for any $W : \Omega \times [0, T] \rightarrow \mathbb{R}^N$ and $x \in \Omega$, $t \in [0, T]$, and the boundary of the domain $\partial\Omega$ has outward unit normal $n : \partial\Omega \rightarrow \mathbb{R}^d$. The conservation law in (1) is supplemented with the initial condition $U(x, 0) = \bar{U}(x)$ for all $x \in \Omega$, where $\bar{U} : \Omega \rightarrow \mathbb{R}^m$. In general, the solution $U(x)$ may contain discontinuities, in which

case, the conservation law (1) holds away from the discontinuities and the Rankine-Hugoniot conditions [22] hold at discontinuities.

We will construct a high-order numerical method that tracks discontinuities with the computational grid as they evolve through the domain, which places three requirements on the discretization: (1) a high-order, stable, and convergent discretization of the conservation law in (1), (2) employs a solution basis that supports discontinuities between computational cells or elements, and (3) allows for deformation of the computational domain. As such, our method is based on a standard high-order DG-DIRK discretization of an Arbitrary Lagrangian-Eulerian (ALE) formulation of the governing equations. We insist on a high-order discretization given their proven ability [6, 35] to deliver accurate solutions on coarse discretizations provided discontinuities are tracked.

2.1 Arbitrary Lagrangian-Eulerian Formulation

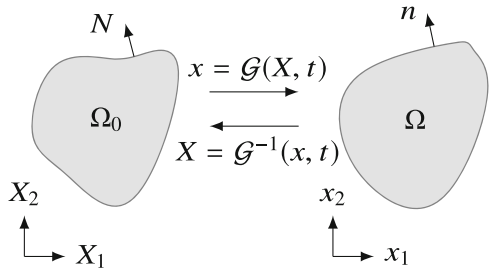
We use an ALE formulation of the governing equations to account for the time-dependent domain deformations required to track discontinuities as they evolve. To this end, we introduce a time-dependent domain mapping (Fig. 2)

$$\mathcal{G} : \Omega_0 \times [0, T] \rightarrow \Omega; \quad \mathcal{G} : (X, t) \mapsto \mathcal{G}(X, t), \quad (2)$$

where $\Omega_0 \subset \mathbb{R}^d$ is a fixed reference domain, T is the final time, and at each time $t \in [0, T]$, $\mathcal{G}(\cdot, t) : \Omega_0 \rightarrow \Omega$ is a diffeomorphism. We note that the domain Ω is fixed, i.e., Ω occupies the same region of $\mathbb{R}^d$ at any time $t \in [0, T]$; the time-dependent diffeomorphism is introduced as an integral part of the proposed numerical method to track discontinuities as they evolve. Following the approach in [24], the conservation law on the physical domain Ω is transformed to a conservation law on the reference domain Ω_0

$$\frac{\partial U_X}{\partial t} + \nabla_X \cdot F_X(U_X; G, v) = 0 \quad \text{in } \Omega_0 \quad (3)$$

Fig. 2 Mapping between reference and physical domains



where $U_X : \Omega_0 \times [0, T] \rightarrow \mathbb{R}^m$ is the solution of the transformed conservation law, $F_X : \mathbb{R}^m \times \mathbb{R}^{d \times d} \times \mathbb{R}^d \rightarrow \mathbb{R}^{m \times d}$ is the transformed flux function, $\nabla_X := (\partial_{X_1}, \dots, \partial_{X_d})$ is the gradient operator on the reference domain, and the deformation gradient $G : \Omega_0 \times [0, T] \rightarrow \mathbb{R}^{d \times d}$, mapping Jacobian $g : \Omega_0 \times [0, T] \rightarrow \mathbb{R}$, and mapping velocity $v : \Omega_0 \times [0, T] \rightarrow \mathbb{R}^d$ are defined as

$$G = \nabla_X \mathcal{G}, \quad g = \det G, \quad v = \frac{\partial \mathcal{G}}{\partial t}. \quad (4)$$

The transformed and physical solutions are related, for any $X \in \Omega_0$ and $t \in [0, T]$, as

$$U_X(X, t) = g(X, t)U(\mathcal{G}(X, t), t) \quad (5)$$

and the transformed flux is defined as

$$F_X : (W_X; \Theta, \xi) \mapsto [(\det \Theta)F((\det \Theta)^{-1}W_X) - W_X \otimes \xi]\Theta^{-T}. \quad (6)$$

The transformed conservation law is supplemented with the initial condition $U_X(X, 0) = \tilde{U}_X(X)$ for all $X \in \Omega_0$, where $\tilde{U}_X : \Omega_0 \rightarrow \mathbb{R}^m$ is $\tilde{U}_X(X) = g(X, 0)\tilde{U}(\mathcal{G}(X, 0))$.

2.2 Discontinuous Galerkin Discretization

We use a nodal discontinuous Galerkin method [5, 17] to discretize the transformed conservation law (3). Let $\mathcal{E}_h$ represent a discretization of the reference domain Ω_0 into non-overlapping, potentially curved, computational elements. Additionally, we introduce the DG approximation (trial) space of discontinuous piecewise polynomials associated with the mesh $\mathcal{E}_h$

$$\mathcal{V}_h^p = \left\{ v \in [L^2(\Omega_0 \times [0, T])]^m \mid v(\cdot, t)|_K \in [\mathcal{P}_p(K)]^m, \forall K \in \mathcal{E}_h, t \in [0, T] \right\} \quad (7)$$

where $\mathcal{P}_p(K)$ is the space of polynomial functions of degree at most $p \geq 1$ on the element K , and we take the DG test space to be $\mathcal{V}_h^{p'}$, where $p' \geq p$. Finally, we define the space of admissible domain mappings as the space of continuous piecewise polynomials of degree q associated with the mesh $\mathcal{E}_h$

$$\mathcal{W}_h = \left\{ v \in [C^0(\Omega_0 \times [0, T])]^d \mid v(\cdot, t)|_K \in [\mathcal{P}_q(K)]^d, \forall K \in \mathcal{E}_h, t \in [0, T] \right\} \quad (8)$$

Then, the DG residual form $r : \mathcal{V}_h^p \times \mathcal{V}_h^{p'} \times \mathcal{W}_h \rightarrow \mathbb{R}$ is given by

$$r_h^{p,p'} : (W_X, \psi_X, Q) \mapsto \sum_{K \in \mathcal{E}_{h,q}} r_K^{p,p'}(W_X, \psi_X, Q), \quad (9)$$

where the elemental DG form $r_K^{p,p'} : \mathcal{V}_h^p \times \mathcal{V}_h^{p'} \times \mathcal{W}_h \rightarrow \mathbb{R}$ is given by

$$\begin{aligned} r_K^{p,p'} : (W_X, \psi_X, Q) \mapsto & \int_K \psi_X \cdot \dot{W}_X \, dV \\ & + \int_{\partial K} \psi_X^+ \cdot \mathcal{H}_X(W_X^+, W_X^-, N; \nabla_X Q, \dot{Q}) \, dS \\ & - \int_K F_X(W_X; \nabla_X Q, \dot{Q}) : \nabla_X \psi_X \, dV, \end{aligned} \quad (10)$$

where N is the outward normal to the surface ∂K , $\mathcal{H}_X : \mathbb{R}^m \times \mathbb{R}^m \times \mathbb{R}^d \times \mathbb{R}^{d \times d} \times \mathbb{R}^d \rightarrow \mathbb{R}^m$ is the numerical flux function associated with the reference inviscid flux F_X , and W_X^+ (W_X^-) is the interior (exterior) trace of W_X to element K ; for points $X \in \partial K \cap \partial \Omega_0$, W_X^- is a boundary state constructed to enforce the appropriate boundary condition. In this work, we take the numerical flux to be a smoothed version of the Roe flux (Section 2.3 of [37]).

Next, we introduce a (nodal) basis over each element for the trial space ($\mathcal{V}_h^p$), test space ($\mathcal{V}_h^{p'}$), and mapping space ($\mathcal{W}_h$) to reduce (10) to a system of ordinary differential equations (ODEs) in residual form. In the case where $p' = p$, we denote the residual $\mathbf{r}^{\text{uns}} : \mathbb{R}^{N_u} \times \mathbb{R}^{N_u} \times \mathbb{R}^{N_x} \times \mathbb{R}^{N_x} \rightarrow \mathbb{R}^{N_u}$, which is defined as

$$\mathbf{r}^{\text{uns}} : (\dot{\mathbf{w}}, \mathbf{w}, \dot{\mathbf{y}}, \mathbf{y}) \mapsto \mathbf{m} \dot{\mathbf{w}} + \mathbf{f}(\mathbf{w}, \mathbf{y}, \dot{\mathbf{y}}), \quad (11)$$

where $N_u = \dim \mathcal{V}_h^p$, $N_x = \dim \mathcal{W}_h$, $\mathbf{m} \in \mathbb{R}^{N_u \times N_u}$ is the mass matrix associated with the test/trial space $\mathcal{V}_h^p$, and $\mathbf{f} : \mathbb{R}^{N_u} \times \mathbb{R}^{N_x} \times \mathbb{R}^{N_x} \rightarrow \mathbb{R}^{N_u}$ is the algebraic form of the second and third terms in (10). In this notation, the standard DG discretization reads: given $\mathbf{x} : [0, T] \rightarrow \mathbb{R}^{N_x}$, find $\mathbf{u} : [0, T] \rightarrow \mathbb{R}^{N_u}$ such that

$$\mathbf{r}^{\text{uns}}(\dot{\mathbf{u}}(t), \mathbf{u}(t), \dot{\mathbf{x}}(t), \mathbf{x}(t)) = \mathbf{0}, \quad \mathbf{u}(0) = \bar{\mathbf{u}}, \quad \mathbf{x}(0) = \bar{\mathbf{x}}, \quad (12)$$

for all $t \in [0, T]$, where $\mathbf{u}$ is the time-dependent coefficients of the DG solution, $\mathbf{x}$ is the time-dependent coefficients of the domain mapping (nodal coordinates of the mesh), $\bar{\mathbf{u}} \in \mathbb{R}^{N_u}$ is the algebraic representation of the initial condition $\bar{U}_X$, and $\bar{\mathbf{x}} \in \mathbb{R}^{N_x}$ is an initial condition for the nodal coordinates. Typically, the evolution of the mesh coordinates $\mathbf{x}(t)$ is known analytically or governed by a dynamical system (e.g., fluid-structure interaction); however, in this work, it will be determined as the solution of an optimization problem (after temporal discretization) such that discontinuities are tracked over time.

Finally, we use the expansions in the nodal bases to define the *enriched residual* $\mathbf{R}^{\text{uns}} : \mathbb{R}^{N_u} \times \mathbb{R}^{N_u} \times \mathbb{R}^{N_x} \times \mathbb{R}^{N_x} \rightarrow \mathbb{R}^{N'_u}$ associated with a trial space of degree p' as

$$\mathbf{R}^{\text{uns}} : (\mathring{\mathbf{w}}, \mathbf{w}, \mathring{\mathbf{y}}, \mathbf{y}) \mapsto \mathbf{M} \mathring{\mathbf{w}} + \mathbf{F}(\mathbf{w}, \mathbf{y}, \mathring{\mathbf{y}}), \quad (13)$$

where $N'_u = \dim \mathcal{V}_h^{p'}$, $\mathbf{M} \in \mathbb{R}^{N'_u \times N_u}$ is the mass matrix associated with the test space $\mathcal{V}_h^{p'}$ and trial space $\mathcal{V}_h^p$, and $\mathbf{F} : \mathbb{R}^{N_u} \times \mathbb{R}^{N_x} \times \mathbb{R}^{N_x} \rightarrow \mathbb{R}^{N'_u}$ is the algebraic form of the second and third terms in (10). In this work, we take $p' = p+1$, but other choices are possible as well. The enriched residual will be used in the subsequent sections to define the implicit tracking objective function.

2.3 High-Order Temporal Discretization

Proceeding with the method of lines, we discretize the dynamical system in (12) using a diagonally implicit Runge-Kutta method (DIRK) to yield a sequence of algebraic systems of equations. Unlike fully implicit Runge-Kutta methods, an s -stage DIRK scheme has a Butcher tableau $(A, b, c) \in \mathbb{R}^{s \times s} \times \mathbb{R}^s \times \mathbb{R}^s$ where A is lower triangular (Table 1). As a result, the i th stage only depends on the solution for stages $1, \dots, i$, allowing the stages to be solved sequentially. We partition the time interval $[0, T]$ into N_T intervals of equal size $\Delta t = T/N_T$ with endpoints $\{t_n\}_{n=0}^{N_T}$, where $t_0 = 0$ and $t_n = t_{n-1} + \Delta t$ for $n = 1, \dots, N_T$. An s -stage DIRK discretization of (12) with Butcher tableau (A, b, c) reads: for $n = 1, \dots, N_T$ and $i = 1, \dots, s$,

$$\begin{aligned} \mathbf{u}_0 &= \bar{\mathbf{u}}, \quad \mathbf{u}_{n+1} = \mathbf{u}_n + \sum_{j=1}^s b_i \mathbf{k}_{n,j}^u, \quad \mathbf{u}_{n,i} = \mathbf{u}_n + \sum_{j=1}^i a_{ij} \mathbf{k}_{n,j}^u \\ \mathbf{x}_0 &= \bar{\mathbf{x}}, \quad \mathbf{x}_{n+1} = \mathbf{x}_n + \sum_{j=1}^s b_i \mathbf{k}_{n,j}^x, \quad \mathbf{x}_{n,i} = \mathbf{x}_n + \sum_{j=1}^i a_{ij} \mathbf{k}_{n,j}^x \\ m \mathbf{k}_{n,i}^u &= -\Delta t f(\mathbf{u}_{n,i}, \mathbf{x}_{n,i}, \mathbf{v}_{n,i}), \quad \mathbf{k}_i^x = \Delta t \mathbf{v}_{n,i} \end{aligned} \quad (14)$$

where $\mathbf{u}_0, \mathbf{u}_n, \mathbf{u}_{n,i}, \mathbf{k}_{n,i}^u \in \mathbb{R}^{N_u}$ and $\mathbf{x}_0, \mathbf{x}_n, \mathbf{x}_{n,i}, \mathbf{k}_{n,i}^x \in \mathbb{R}^{N_x}$ are implicitly defined as the solution of (14); $\mathbf{u}_n \approx \mathbf{u}(t_n)$ and $\mathbf{x}_n \approx \mathbf{x}(t_n)$ are the state and mesh

Table 1 Butcher tableau for DIRK schemes

$$\begin{array}{c|c} c & A \\ \hline & b^T \end{array} = \begin{array}{ccc|ccc} c_1 & a_{11} & & & & \\ \vdots & \vdots & & \ddots & & \\ c_s & a_{s1} & \dots & a_{ss} & & \\ \hline & b_1 & \dots & b_s & & \end{array}$$

approximation at each time step $n = 0, \dots, T$; $\mathbf{u}_{n,i} \approx \mathbf{u}(t_n + c_i \Delta t)$ and $\mathbf{x}_{n,i} \approx \mathbf{x}(t_n + c_i \Delta t)$ are the state and mesh approximations at each stage $i = 1, \dots, s$ of each time interval $n = 1, \dots, T$; $\mathbf{k}_{n,i}^u$ and $\mathbf{k}_{n,i}^x$ are the solution and mesh stage updates, and $\mathbf{v}_{n,i} \in \mathbb{R}^{N_x}$ is a stage-consistent mesh velocity [11, 27] defined as

$$\mathbf{v}_{n,i} = \sum_{j=1}^i (\mathbf{A}^{-1})_{ij} \frac{\mathbf{x}_{n,j} - \mathbf{x}_n}{\Delta t}, \quad (15)$$

i.e., the mesh velocity is consistent with the DIRK scheme and the mesh position at the stages.

We convert the modified DIRK system in (14) to residual form $\mathbf{r}_{n,i} : \mathbb{R}^{N_u} \times \mathbb{R}^{N_x} \rightarrow \mathbb{R}^{N_u}$ at a fixed step $n \in \{1, \dots, N_T\}$ and stage $i \in \{1, \dots, s\}$ as

$$\mathbf{r}_{n,i}^{\text{uns}} : (\mathbf{w}, \mathbf{y}) \mapsto \mathbf{m} \xi_{n,i}(\mathbf{w}) + \Delta t \mathbf{f}(\mathbf{w}, \mathbf{y}, \zeta_{n,i}(\mathbf{y})), \quad (16)$$

where $\xi_{n,i} : \mathbb{R}^{N_u} \rightarrow \mathbb{R}^{N_u}$ maps the state stage $(\mathbf{u}_{n,i})$ to the corresponding stage update $(\mathbf{k}_{n,i}^u)$

$$\xi_{n,i} : \mathbf{w} \mapsto (\mathbf{A}^{-1})_{ii}(\mathbf{w} - \mathbf{u}_n) + \sum_{j=1}^{i-1} (\mathbf{A}^{-1})_{ij}(\mathbf{u}_{n,j} - \mathbf{u}_n) \quad (17)$$

and $\zeta_{n,i} : \mathbb{R}^{N_x} \rightarrow \mathbb{R}^{N_x}$ maps the mesh stage $(\mathbf{x}_{n,i})$ to the corresponding stage-consistent velocity $(\mathbf{v}_{n,i})$ as

$$\zeta_{n,i} : \mathbf{y} \mapsto (\mathbf{A}^{-1})_{ii} \frac{\mathbf{y} - \mathbf{x}_n}{\Delta t} + \sum_{j=1}^{i-1} (\mathbf{A}^{-1})_{ij} \frac{\mathbf{x}_{n,j} - \mathbf{x}_n}{\Delta t}. \quad (18)$$

Similarly, we define the corresponding fully discrete enriched residual function $\mathbf{R}_{n,i} : \mathbb{R}^{N_u} \times \mathbb{R}^{N_x} \rightarrow \mathbb{R}^{N'_u}$ as

$$\mathbf{R}_{n,i}^{\text{uns}} : (\mathbf{w}, \mathbf{y}) \mapsto \mathbf{M} \xi_{n,i}(\mathbf{w}) + \Delta t \mathbf{F}(\mathbf{w}, \mathbf{y}, \zeta_{n,i}(\mathbf{y})), \quad (19)$$

which we use in next section to define the objective function of the implicit tracking optimization problem. Notice that only the *spatial* test space is enriched in the definition of $\mathbf{R}_{n,i}^{\text{uns}}$ relative to $\mathbf{r}_{n,i}^{\text{uns}}$; the temporal discretization in $\mathbf{r}_{n,i}^{\text{uns}}$ and $\mathbf{R}_{n,i}^{\text{uns}}$ is identical.

In this work, we consider a third-order L -stable DIRK scheme with three stages, which we denote by DIRK3. The Butcher tableau is given in Table 2.

Table 2 Butcher tableau for DIRK3, where $\beta = 0.435866521508459$, $\gamma = -\frac{6\beta^2-16\beta+1}{4}$, and $\omega = \frac{6\beta^2-20\beta+5}{4}$

β	β		
$\frac{1+\beta}{2}$	$\frac{1+\beta}{2} - \beta$	β	
$\gamma + \omega + \beta$	γ	ω	β
	γ	ω	β

2.4 Special Case: Reduction to Steady Conservation Law

We close this section by noting that this discretization procedure simplifies considerably for the special case of the steady conservation law, which is obtained by dropping the time derivative term in (1). In this setting, the mapping (2) is independent of time so all terms depending on the mapping velocity vanish. Furthermore, the transformed state U_X does not need to account for the mapping Jacobian, which further simplifies the definition of the transformed flux. In the steady setting, the DG discretization leads to an algebraic residual form $\mathbf{r}^{\text{ste}} : \mathbb{R}^{N_u} \times \mathbb{R}^{N_x} \rightarrow \mathbb{R}^{N_u}$ and enriched residual form $\mathbf{R}^{\text{ste}} : \mathbb{R}^{N_u} \times \mathbb{R}^{N_x} \rightarrow \mathbb{R}^{N'_u}$. A standard DG discretization reads: given $\mathbf{x} \in \mathbb{R}^{N_x}$, find $\mathbf{u} \in \mathbb{R}^{N_u}$ such that $\mathbf{r}(\mathbf{u}, \mathbf{x}) = \mathbf{0}$. See Section 2 of [37] for details.

3 Implicit Shock Tracking

In this section, we present the implicit tracking framework that poses an optimization problem over the discrete solution and mesh that aims to align element faces with discontinuities in the solution. First, we introduce the optimization formulation for steady conservation laws and extend it to the unsteady case (method of lines discretization) by “solving a steady problem at each timestep”.

3.1 Steady Case

We formulate the problem of tracking discontinuities as a constrained optimization problem over the PDE state and coordinates of the mesh nodes that minimizes an objective function, $f : \mathbb{R}^{N_u} \times \mathbb{R}^{N_x} \rightarrow \mathbb{R}$, while enforcing the DG discretization of the conservation law. That is, we define $\mathbf{u} \in \mathbb{R}^{N_u}$ and $\mathbf{x} \in \mathbb{R}^{N_x}$ as

$$(\mathbf{u}, \mathbf{x}) := \arg \min_{\mathbf{w} \in \mathbb{R}^{N_u}, \mathbf{y} \in \mathbb{R}^{N_x}} f(\mathbf{w}, \mathbf{y}) \quad \text{subject to: } \mathbf{r}^{\text{ste}}(\mathbf{w}, \mathbf{y}) = \mathbf{0}. \quad (20)$$

The objective function is constructed such that the solution of the optimization problem $(\mathbf{u}, \mathbf{x})$ is a mesh $(\mathbf{x})$ that aligns element faces with discontinuities in the DG solution $(\mathbf{u})$. The optimization-based tracking method directly inherits the benefits of standard DG methods, i.e., high-order accuracy and conservation, due to the constraint that exactly enforces the DG discretization. Finally, the proposed optimization-based tracking method is nonlinearly stable if all discontinuities are successfully tracked, i.e., additional stabilization (e.g., limiting, artificial viscosity) is not required, which we demonstrate in our numerical experiments in Sect. 5.

We propose an objective function that consists of two terms: one term penalizes a measure of the DG solution error $f_{\text{err}} : \mathbb{R}^{N_u} \times \mathbb{R}^{N_x} \rightarrow \mathbb{R}$ and the other term penalizes distortion of the mesh $f_{\text{msh}} : \mathbb{R}^{N_x} \rightarrow \mathbb{R}$, i.e.,

$$f : (\mathbf{w}, \mathbf{y}) \mapsto f_{\text{err}}(\mathbf{w}, \mathbf{y}) + \kappa^2 f_{\text{msh}}(\mathbf{y}), \quad (21)$$

where $\kappa \in \mathbb{R}_+$ is a parameter that weights the contribution of the two terms. Since a piecewise polynomial solution on an aligned mesh will have much lower error than on a non-aligned mesh, f_{err} promotes alignment of the mesh with discontinuities while f_{msh} prevents the mesh from entangling or becoming unacceptably skewed.

For the error-like tracking term, we use the norm of the DG residual corresponding to an enriched *test space*, i.e.,

$$f_{\text{err}} : (\mathbf{w}, \mathbf{y}) \mapsto \frac{1}{2} \mathbf{R}^{\text{ste}}(\mathbf{w}, \mathbf{y})^T \mathbf{R}^{\text{ste}}(\mathbf{w}, \mathbf{y}), \quad (22)$$

where we enrich the test space using polynomials of one degree higher than the trial space. This follows on a large body of work that uses residual-based error indicators to drive h -, p -, and r -adaptivity [10]. Furthermore, it was shown to lead to a robust and reliable tracking framework [37].

To maintain a high-quality mesh, we define f_{msh} as the distortion of the physical mesh relative to the distortion of the reference mesh

$$f_{\text{msh}} : \mathbf{y} \mapsto \frac{1}{2} (\mathbf{R}_{\text{msh}}(\mathbf{y}) - \mathbf{R}_{\text{msh}}(\mathbf{X}))^T (\mathbf{R}_{\text{msh}}(\mathbf{y}) - \mathbf{R}_{\text{msh}}(\mathbf{X})), \quad (23)$$

where $\mathbf{R}_{\text{msh}} : \mathbb{R}^{N_x} \rightarrow \mathbb{R}^{|\mathcal{E}_{h,q}|}$ is the algebraic system corresponding to the elemental mesh distortion $r_K^{\text{msh}} : \mathcal{W}_h \rightarrow \mathbb{R}$, commonly used for high-order mesh generation [13], defined as

$$r_{\text{msh}}^K : \mathbf{Q} \mapsto \int_K \left(\frac{\|\nabla_X \mathbf{Q}\|_F^2}{d(\det \nabla_X \mathbf{Q})_+^{2/d}} \right)^2 dv \quad (24)$$

where d is the dimension and $\mathbf{X} \in \mathbb{R}^{N_x}$ is the nodal coordinates of the reference mesh.

The optimization problem in (20) is solved using the sequential quadratic programming (SQP) method proposed in [37] that defines the sequences $\{\mathbf{u}^{(k)}\}_{k=0}^{\infty} \subset \mathbb{R}^{N_u}$ and $\{\mathbf{x}^{(k)}\}_{k=0}^{\infty} \subset \mathbb{R}^{N_x}$ as

$$\mathbf{u}^{(k+1)} = \mathbf{u}^{(k)} + \alpha_{k+1} \Delta \mathbf{u}^{(k+1)}, \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_{k+1} \Delta \mathbf{x}^{(k+1)}, \quad (25)$$

for $k = 0, 1, \dots$, where $\alpha_{k+1} \in (0, 1]$ is a step length determined via a line search applied to the ℓ_1 merit function, and $\Delta \mathbf{u}^{(k+1)} \in \mathbb{R}^{N_u}$ and $\Delta \mathbf{x}^{(k+1)} \in \mathbb{R}^{N_x}$ are search directions. At a given iteration k , the search directions $\Delta \mathbf{u}^{(k)}$ and $\Delta \mathbf{x}^{(k)}$ are computed simultaneously as the solution of a quadratic approximation to the optimization problem in (20) with a regularized Levenberg-Marquardt approximation of the Hessian; for details, see section 4 of [37].

3.2 Unsteady Case (Method of Lines)

We extend the steady implicit shock tracking framework in Sect. 3.2 to time-dependent problems discretized using a method of lines approach (in contrast to the space-time approach in [6, 37]). To begin, we formulate the problem of tracking discontinuities at a given stage i and timestep n as a constrained optimization problem over the PDE state and mesh stage that minimizes an objective function, $f_{n,i} : \mathbb{R}^{N_u} \times \mathbb{R}^{N_x} \rightarrow \mathbb{R}$, while enforcing the DG-DIRK discretization. That is, we define $\mathbf{u}_{n,i} \in \mathbb{R}^{N_u}$ and $\mathbf{x}_{n,i} \in \mathbb{R}^{N_x}$ as

$$(\mathbf{u}_{n,i}, \mathbf{x}_{n,i}) := \arg \min_{\mathbf{w} \in \mathbb{R}^{N_u}, \mathbf{y} \in \mathbb{R}^{N_x}} f_{n,i}(\mathbf{w}, \mathbf{y}) \quad \text{subject to: } \mathbf{r}_{n,i}^{\text{uns}}(\mathbf{w}, \mathbf{y}) = \mathbf{0}. \quad (26)$$

for $n = 1, \dots, N_T$ and $i = 1, \dots, s$. As in the steady case, the objective function is constructed such that the solution of the PDE-constrained optimization problem is a feature-aligned mesh by using the norm of the enriched DG-DIRK residual

$$f_{n,i} : (\mathbf{w}, \mathbf{y}) \mapsto \frac{1}{2} \|\mathbf{R}_{n,i}^{\text{uns}}(\mathbf{w}, \mathbf{y})\|_2^2. \quad (27)$$

This choice of optimization formulation *exactly matches the form* of the optimization problem in (20), and therefore, we use the same SQP solver to compute the solution of (26). For a fixed time step n , once the stage states $\{\mathbf{u}_{n,i}\}_{i=1}^s$ and meshes $\{\mathbf{x}_{n,i}\}_{i=1}^s$ are computed, the state and mesh can be advanced to the next time step $(\mathbf{u}_{n+1}$ and $\mathbf{x}_{n+1})$ using the relationships in (14). The DIRK schemes considered in this work (Table 1) satisfy the property that $A_{si} = b_i$ for $i = 1, \dots, s$, which implies the state and mesh at the final stage of time step n are identical to the state and mesh at time step $n + 1$, i.e., $\mathbf{u}_{n+1} = \mathbf{u}_{n,s}$ and $\mathbf{x}_{n+1} = \mathbf{x}_{n,s}$.

Unlike the steady case (21), we do not include the mesh quality term in the objective function for the unsteady (method of lines) case. In the steady case,

there is no information about the shock location *a priori*, which usually requires significant deformation of the initial mesh to align with shocks, necessitating the use of f_{msh} (23). However, in the context of timestepping, we have useful information from the previous timestep to use as an initial guess for both the state and mesh. We can combine this information along with the Rankine-Hugoniot conditions and a high-order mesh smoothing procedure to obtain an excellent initial guess for the mesh at each Runge-Kutta stage (Sect. 4.1.2). In a sense, the additional time dimension allows the mesh regularization to be decoupled from the implicit shock tracking procedure, which is one advantage of the method of lines discretization over the space-time formulation.

4 Practical Considerations

4.1 Solution and Mesh Initialization

The implicit shock tracking optimization problems in (20) and (26) are non-convex and therefore the initial guess for the SQP solver is critical to obtain a good solution. In the present context, this means we must provide a reasonable initial guess for the mesh coordinates $\mathbf{x}^{(0)}$ and DG solution $\mathbf{u}^{(0)}$.

4.1.1 Steady Case

The mesh is always initialized from the reference mesh which comes from mesh generation without knowledge of the discontinuity surface. To avoid local minima in the optimization problem (20) that arise when high-order meshes are used, we usually initialize the tracking problem for $p \geq 0, q > 1$ (solution space of degree p and mesh of degree q) from the solution of the tracking problem for $p' \leq p, q' = 1$ because we have observed that $q = 1$ tracking is quite robust (Section 5.1.1 of [37]) and convergence of the $q > 1$ solution from a straight-sided tracking mesh is rapid.

4.1.2 Unsteady Case (Method of Lines)

In the method of lines setting where implicit shock tracking is performed at each time step, an obvious idea for the initial guess of the optimization problem (26) is the converged mesh and solution from the previous time t_n . In practice, this is too far off to attain good convergence properties except for prohibitively small choices of timestep Δt . Instead, we employ an initial guess where we advect each node on the discontinuity surface from the previous time by the instantaneous shock speed determined by the Rankine-Hugoniot

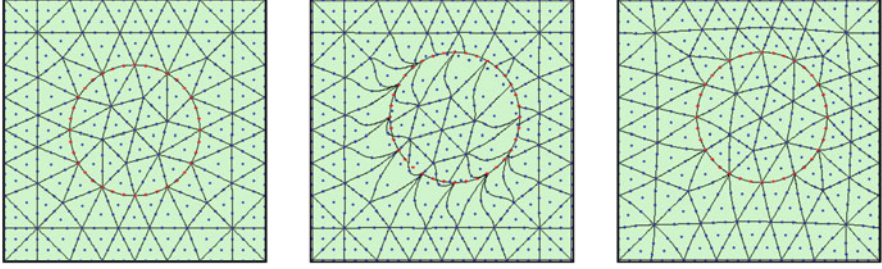


Fig. 3 Shock nodes (.....) (left), advected by instantaneous shock speed (middle) and smoothed mesh (right)

conditions. The remaining nodes are updated using standard optimization-based mesh smoothing using the high-order mesh distortion metric (24). This provides a good initial guess for the shock-aligned mesh for each stage of the optimization problem (Fig. 3). In general, we will need a regularized distortion metric following a procedure described in [12] to handle tangled meshes.

For the initial guess for the solution, we use the converged physical solution (U) from the previous timestep. However, since we applied our DG discretization to the transformed conservation law (3), we are solving for the reference solution (U_X). Therefore, to use the physical solution at time t_n as the initial guess for time t_{n+1} , we multiply it by the ratio of the Jacobian of the initial guess for the mapping at time t_{n+1} (advection based on Rankine-Hugoniot conditions and smoothing) to the Jacobian of the converged mapping at time t_n . Future work to enhance the robustness of these initial guesses might consider more advanced methods to approximate the shock velocity [26] and extrapolation based estimates for the solution.

4.2 Edge Collapses and Solution Transfer

As we will show in our numerical experiments in Sect. 5, for the steady case we typically start with an initial mesh that is far from alignment with the shock. In general, this requires large deformations of the initial mesh, which can result in severely ill-conditioned elements and drastically degrade the quality of our solution. To address this, we follow the approach in [6, 37] and collapse elements once they become problematic. In particular, after each SQP iteration, we tag elements for removal if their volume is smaller than some fraction (20% in this work) of their initial volume and remove them by collapsing their shortest edge and updating the connectivity of the mesh (Fig. 4). Further details of this edge collapse procedure and corresponding solution transfer are described in Section 5.2 of [37].

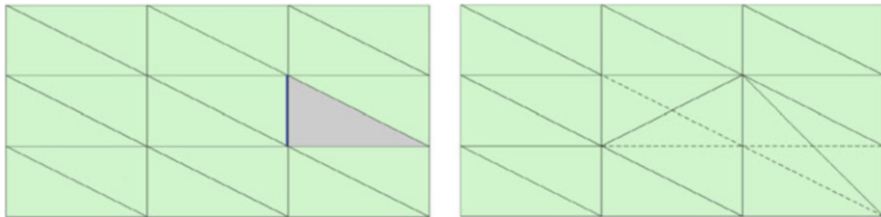


Fig. 4 Demonstration of edge collapse algorithm: the element identified in the original mesh (*left*) is collapsed along the highlighted edge to produce the new mesh (*right*) with the original elements shown in dashed lines for reference

5 Numerical Experiments

In this section, we demonstrate the implicit shock tracking method for a number of nonlinear conservation laws. For the unsteady problems, we apply implicit shock tracking in the space-time and method of lines settings, and compare the two approaches.

5.1 1D Time-Dependent, Inviscid Burgers' Equation

We consider the time-dependent, inviscid Burgers' equation in one spatial dimension that governs nonlinear advection of a scalar quantity through the one-dimensional spatial domain $\Omega = [-1, 1]$

$$\begin{aligned} \frac{\partial}{\partial t} U(x, t) + \frac{\partial}{\partial x} \left(\frac{1}{2} U(x, t)^2 \right) &= 0 & \text{for } x \in \Omega, \, t \in [0, T] \\ U(-1, t) &= 0 & \text{for } t \in [0, T] \\ U(x, 0) &= \bar{U}(x) & \text{for } x \in \Omega \end{aligned} \tag{28}$$

where $U : \Omega \times [0, T] \rightarrow \mathbb{R}$ is the conserved quantity implicitly defined as the solution of (28), $T = 1$ is the final time, $\bar{U} : \Omega \rightarrow \mathbb{R}$ is the initial condition, defined as

$$\bar{U} : x \mapsto 2(x+1)^2(1-H(x)), \tag{29}$$

and $H : \mathbb{R} \rightarrow \{0, 1\}$ is the Heaviside function.

5.1.1 Space-Time Approach

First, we consider a space-time formulation of (28) that takes the form of a steady conservation law (Eq. (1) without the time derivative term) over the space-time domain $\tilde{\Omega} := \Omega \times [0, T]$ with flux function $F : \mathbb{R} \rightarrow \mathbb{R}^{1 \times 2}$, defined as

$$F : W \mapsto \left[\frac{1}{2} W^2 \quad W \right]. \quad (30)$$

The implicit tracking procedure is initialized from a structured mesh of 64 space-time simplex elements; the polynomial degree of the solution (p) and mesh (q) are varied in $p = q \in \{1, 2, 3\}$. Even though the initial mesh is far from tracking the discontinuity (some faces are nearly orthogonal to the discontinuity, rather than parallel to it), our method tracks a faceted approximation ($p = q = 1$) to the discontinuity in only 40 iterations, requiring 7 element collapses (Fig. 5). The mesh and solution for the high-order elements ($p = q > 1$) are initialized from the $p = q = 1$ tracking mesh and solution. These high-order approximations provide high-quality approximations of the discontinuous space-time solution on the coarse mesh (Fig. 6).

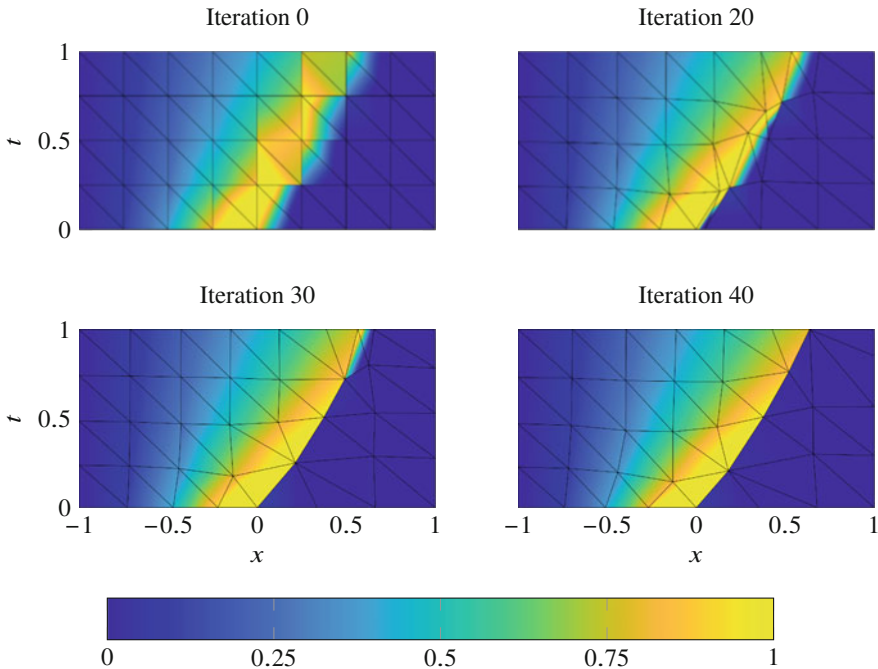


Fig. 5 Space-time solution of one-dimensional, inviscid Burgers' equation using the tracking method at various iterations throughout the solution procedure using a $p = q = 1$ basis for the solution and mesh

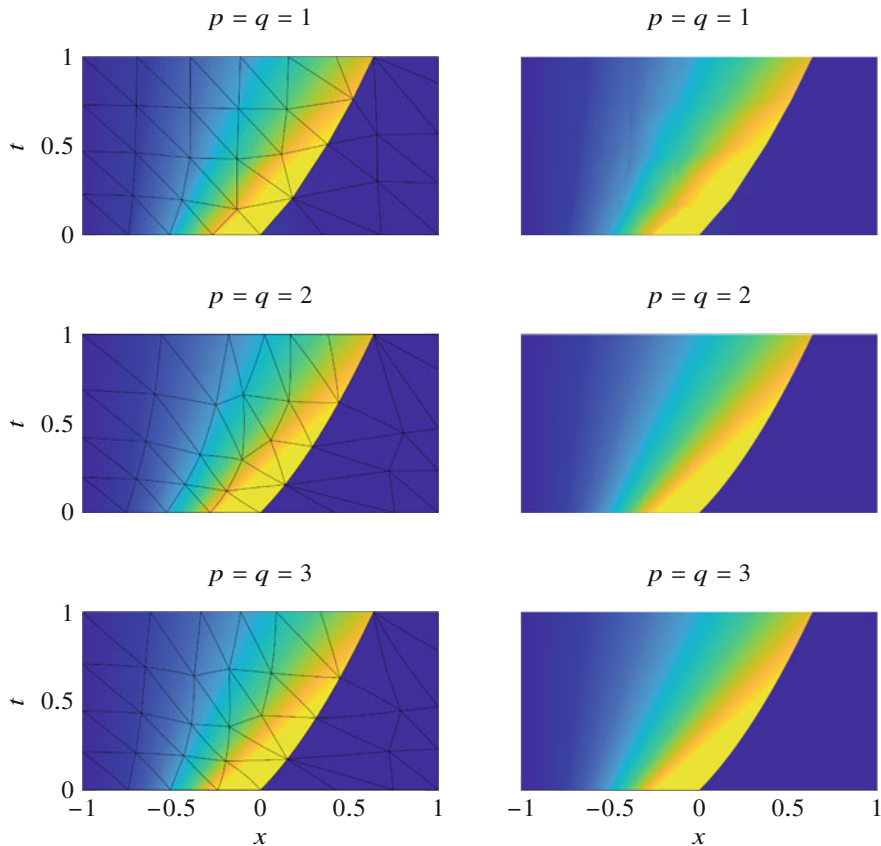


Fig. 6 Space-time solution of one-dimensional, inviscid Burgers' equation using the implicit tracking method with a $p = q = 1$ (top), $p = q = 2$ (middle), and $p = q = 3$ (bottom) basis for the solution and mesh with (left) and without (right) element boundaries

5.1.2 Method of Lines Approach

Next, we consider (28) in a method of lines setting where we apply the DG discretization in space (Sect. 2.2), DIRK discretization in time (Sect. 2.3) and solve a steady implicit tracking problem at each time step (Sect. 3.2). That is, instead of solving a tracking problem over the two-dimensional space-time domain $\bar{\Omega}$ (all of space and time coupled), we solve a sequence of one-dimensional tracking problems, each one corresponding to an instant in time. This approach only requires a mesh of the reference domain $\Omega_0 = [-1, 1]$, which we construct such that an element interface lies at the initial shock location ($x = 0$), i.e., the shock in the initial condition is tracked. The shock tracking solution is computed using a DG discretization on this mesh with 20 elements of degree $p = 3$, $q = 1$ and a DIRK3 temporal discretization with 20 time steps (Fig. 7).

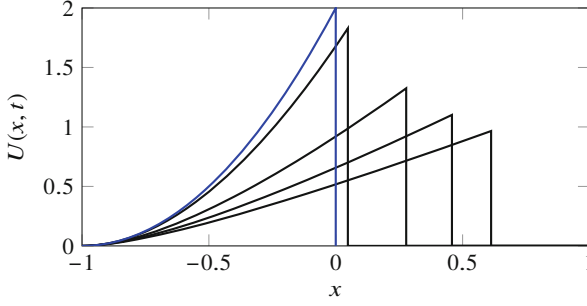


Fig. 7 Method of lines solution of the one-dimensional, inviscid Burgers' equation with $p = 4$, $q = 1$. Initial condition $\bar{U}(x)$ (—) and tracking solution at times $t = 0.05, 0.35, 0.65, 0.95$ (—)

5.2 2D Time-Dependent, Inviscid Burgers' Equation

Next, we consider the time-dependent, inviscid Burgers' equation in two spatial dimensions that governs nonlinear advection of a scalar quantity through the two-dimensional spatial domain $\Omega = [-1, 1]^2$

$$\begin{aligned} \frac{\partial}{\partial t} U(x, t) + \frac{\partial}{\partial x_j} \left(\frac{1}{2} U(x, t)^2 \beta_j \right) &= 0 & \text{for } x \in \Omega, t \in [0, T] \\ U(x, t) &= 0 & \text{for } x \in \partial\Omega, t \in [0, T] \\ U(x, 0) &= \bar{U}(x) & \text{for } x \in \Omega \end{aligned} \quad (31)$$

where $U : \Omega \times [0, T] \rightarrow \mathbb{R}$ is the conserved quantity implicitly defined as the solution of (31), $\beta = (1, 0)$ is the flow direction, $T = 2$ is the final time, $\bar{U} : \Omega \rightarrow \mathbb{R}$ is the initial condition, defined as

$$\bar{U} : (x_1, x_2) \mapsto \begin{cases} (0.5 - 2(x_2^2 - 0.25)) \left(\frac{4}{3}(x_1 + 0.75) \right) & x \in \Omega_{\square} \\ 0 & \text{elsewhere,} \end{cases} \quad (32)$$

where $\Omega_{\square} := [-0.75, 0] \times [-0.5, 0.5]$. The initial condition is constructed such that the initially straight shock curves over time, which is tracked by the high-order mesh. The shock tracking solution is computed using a DG discretization on a mesh with 128 simplex elements of degree $p = q = 2$ and a DIRK3 temporal discretization with 40 time steps (Fig. 8). The mesh smoothing procedure described in Sect. 4.1.2 is important to maintain high-quality elements as the shock moves across the domain.

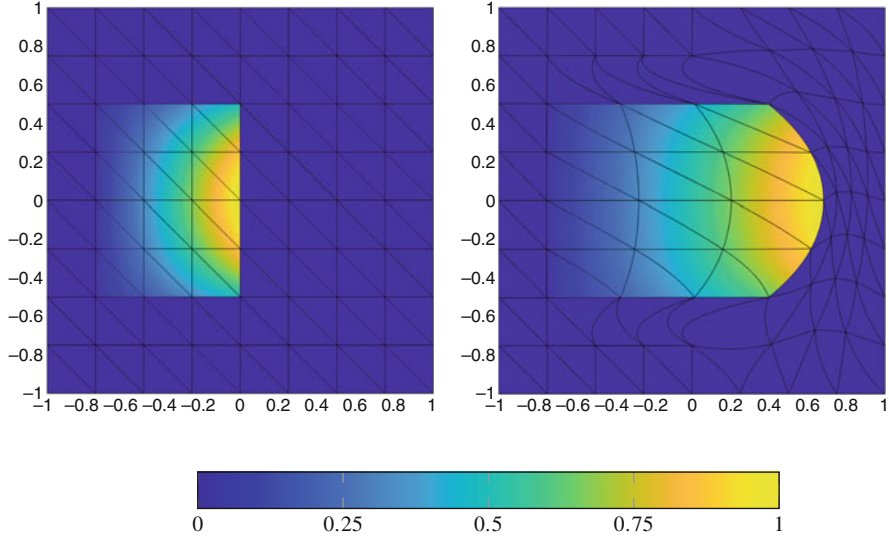


Fig. 8 Method of lines solution of two-dimensional, inviscid Burgers' equation with $p = q = 2$. Initial condition $\bar{U}(x)$ (left) and solution at $T = 2$ (right)

5.3 Euler Equations

The Euler equations govern the flow of an inviscid, compressible fluid through a domain $\Omega \subset \mathbb{R}^d$

$$\begin{aligned} \frac{\partial}{\partial t} \rho(x, t) + \frac{\partial}{\partial x_j} (\rho(x, t) v_j(x, t)) &= 0 \\ \frac{\partial}{\partial t} (\rho(x, t) v_i(x, t)) + \frac{\partial}{\partial x_j} (\rho(x, t) v_i(x, t) v_j(x, t) + P(x, t) \delta_{ij}) &= 0 \\ \frac{\partial}{\partial t} (\rho(x, t) E(x, t)) + \frac{\partial}{\partial x_j} ([\rho(x, t) E(x, t) + P(x, t)] v_j(x, t)) &= 0 \end{aligned} \quad (33)$$

for all $x \in \Omega$, $t \in [0, T]$, $i = 1, \dots, d$ and summation is implied over the repeated index $j = 1, \dots, d$, where $\rho : \Omega \times (0, T) \rightarrow \mathbb{R}_+$ is the density of the fluid, $v_i : \Omega \times (0, T) \rightarrow \mathbb{R}$ for $i = 1, \dots, d$ is the velocity of the fluid in the x_i direction, and $E : \Omega \times (0, T) \rightarrow \mathbb{R}_{>0}$ is the total energy of the fluid, implicitly defined as the solution of (33). For a calorically ideal fluid, the pressure of the fluid, $P : \Omega \times (0, T) \rightarrow \mathbb{R}_{>0}$, is related to the energy via the ideal gas law

$$P = (\gamma - 1) \left(\rho E - \frac{\rho v_i v_i}{2} \right), \quad (34)$$

where $\gamma \in \mathbb{R}_{>0}$ is the ratio of specific heats. By combining the density, momentum, and energy into a vector of conservative variables $U : \Omega \times [0, T] \rightarrow \mathbb{R}^{d+2}$, defined as

$$U : (x, t) \mapsto \begin{bmatrix} \rho(x, t) \\ \rho(x, t)v(x, t) \\ \rho(x, t)E(x, t) \end{bmatrix} \quad (35)$$

the Euler equations are a conservation law of the form (1). Now, we investigate the shock tracking framework on three benchmark examples governed by these equations: Sod's shock tube (a Riemann problem), the Shu-Osher problem, and supersonic flow over a NACA0012 airfoil.

5.3.1 Sod's Shock Tube

Sod's shock tube is a Riemann problem for the Euler equations that models an idealized shock tube where the membrane separating a high pressure region from a low pressure one is instantaneously removed. This is a commonly used validation problem since it has an analytical solution and features a shock wave, a rarefaction wave, and contact discontinuity. The flow domain is $\Omega = [0, 1]$, the final time is $T = 0.2$, the initial condition is given in terms of the density, velocity, and pressure as

$$\rho(x, 0) = \begin{cases} 1 & x < 0.5 \\ 0.125 & x \geq 0.5 \end{cases}, \quad v(x, 0) = 0, \quad P(x, 0) = \begin{cases} 1 & x < 0.5 \\ 0.1 & x \geq 0.5, \end{cases} \quad (36)$$

and the density, velocity, and pressure are prescribed at $x = 0$ and the velocity is prescribed at $x = 1$ (values can be read from the initial condition). The solution of this problem contains three waves (shock, contact, rarefaction) that emanate from $x = 0.5$ and move at different speeds, which is a generalized triple point in space-time. The method of lines approach cannot handle this case because a single node lies at $(x, t) = (0.5, 0)$, which cannot track all three waves at time $t > 0$. Thus, we use the space-time implicit shock tracking approach to solve this problem. The method is initialized with an unstructured mesh of the space-time domain $\bar{\Omega} = [0, 1] \times [0, 0.2]$ consisting of 173 simplex elements ($p = 3, q = 1$) with additional refinement near $(0.5, 0)$ to resolve the geometric complexity of the triple point for a total of 5190 spatiotemporal degrees of freedom. The DG solution is initialized with the $p = 0$ solution on the reference mesh. The final space-time tracking solution is shown in Fig. 9 where all features (head and tail of rarefaction, shock, and contact) are tracked. A total of 47 element collapses are required, mostly near the triple point to obtain elements that do not cross between the five distinct regions (left state, rarefaction, between rarefaction and contact, between contact and shock, and right

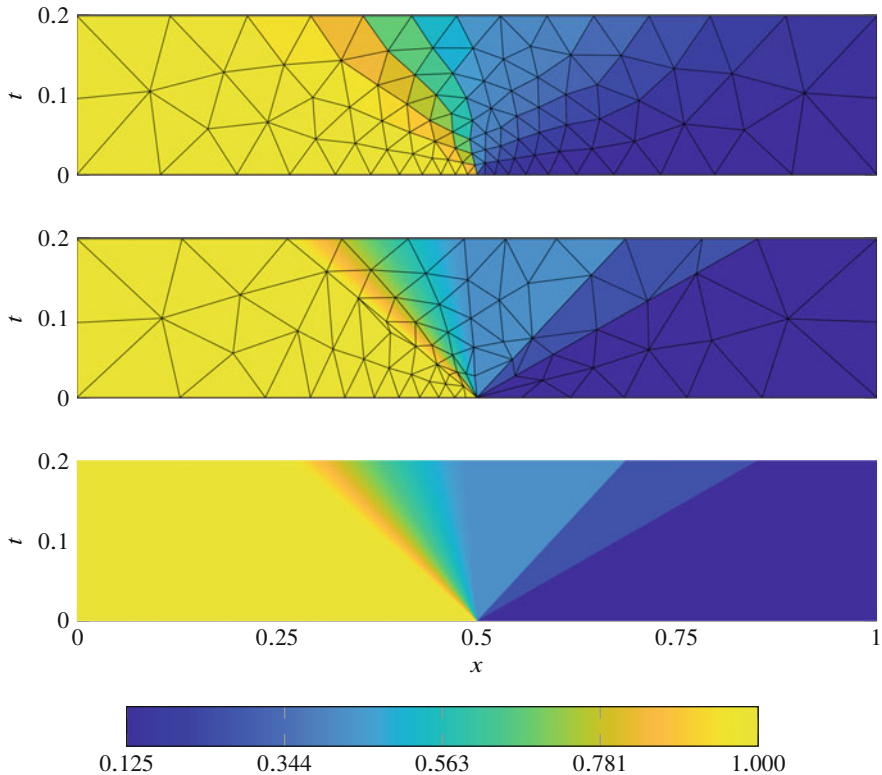


Fig. 9 Space-time solution of Sod's shock tube (density) using implicit shock tracking using a $p = 3$ DG discretization (*center* with element boundaries, *bottom* without element boundaries), initialized from an unstructured mesh without knowledge of the discontinuity surfaces and a $p = 0$ DG solution (*top*)

state); the final mesh contains 126 elements (3780 degrees of freedom). Despite the reduction in the number of degrees of freedom, the solution at the final time T agrees well with the exact solution (Fig. 10) because the cubic basis functions do not cross discontinuities or kinks.

5.3.2 Shu-Osher Problem

The Shu-Osher problem [28] is a one-dimensional idealization of shock-turbulence interaction where a Mach 3 shock moves into a field with a small sinusoidal density disturbance. The flow domain is $\Omega = [-4.5, 4.5]$, the final time is $T = 1.1$, the

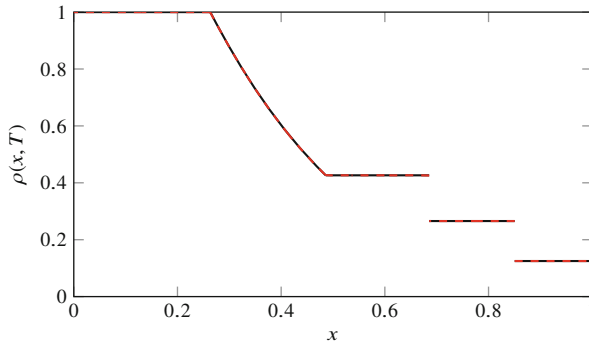


Fig. 10 Slice of space-time implicit shock tracking solution (—) at time $T = 0.2$ relative to the exact solution (---)

initial condition is given in terms of the density, velocity, and pressure as

$$\begin{aligned} \rho(x, 0) &= \begin{cases} 3.857143 & x < -4 \\ 1 + 0.2 \sin(5x) & x \geq -4 \end{cases} \\ v_1(x, 0) &= \begin{cases} 2.629369 & x < -4 \\ 0 & x \geq -4 \end{cases} \\ P(x, 0) &= \begin{cases} 10.3333 & x < -4 \\ 1 & x \geq -4, \end{cases} \end{aligned} \quad (37)$$

and the density, velocity, and pressure are prescribed at $x = -4.5$ and the velocity is prescribed at $x = 4.5$ (values can be read from the initial condition). The final time is chosen such that waves trailing behind the primary shock do not steepen into shock waves; shock formation will be the subject of future work. The shock tracking solution is computed using a DG discretization on a mesh with 288 elements of degree $p = 4$, $q = 1$, and a DIRK3 temporal discretization with 110 time steps (Fig. 11) along with a reference solution computed using a fifth-order WENO method with 200 elements and temporal integration via RK4 with 110 timesteps [28]. The shock tracking solution actually overshoots the reference solution at the formation of the trailing waves, which suggests the reference solution is being overly dissipated by the WENO scheme (*left inset*). The shock is perfectly represented by the aligned mesh in the shock tracking solution compared to the reference (*right inset*).

Unlike Sod's shock tube, the Shu-Osher problem is well-suited for the method of lines approach because there is no space-time triple point, e.g., from multiple waves emanating from a point or intersecting discontinuities.

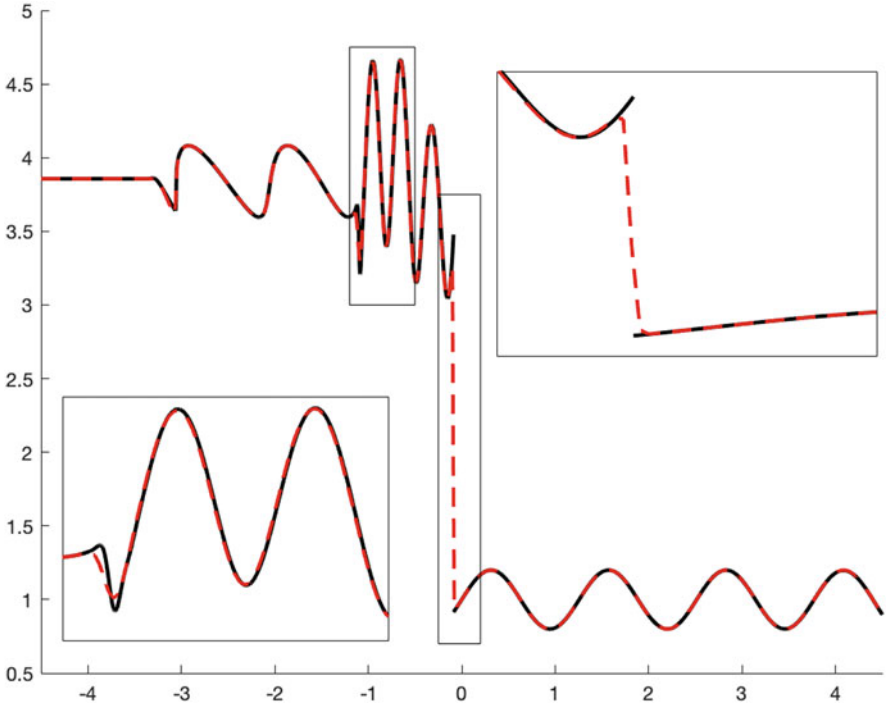


Fig. 11 Density at $T = 1.1$ of Shu-Osher problem for the reference (---) and shock tracking (—) solutions

In this case, the method of lines approach is preferred because computations are only required on a d -dimensional mesh as opposed to a $(d + 1)$ -dimensional space-time mesh (all time coupled), which is more practical for large problems.

5.3.3 Supersonic Flow Over Airfoil

Finally, we apply the implicit tracking method to solve for supersonic flow over a NACA0012 airfoil (Fig. 12), which is governed by the 2D steady, compressible Euler equations (see [37] for a complete description of the problem). This is a difficult problem because there are two distinct shocks that must be resolved: a bow shock ahead of the leading edge and an oblique shock off the tail. To demonstrate the implicit shock tracking method, we use a coarse mesh consisting of 160 simplex elements with a second-order ($p = q = 1$) and third-order ($p = q = 2$) solution and mesh discretization; the initial mesh is generated without knowledge of the shock location (Fig. 12, left). In both cases, the tracking procedure tracks the shocks given the resolution in the finite element space, despite the initial mesh and

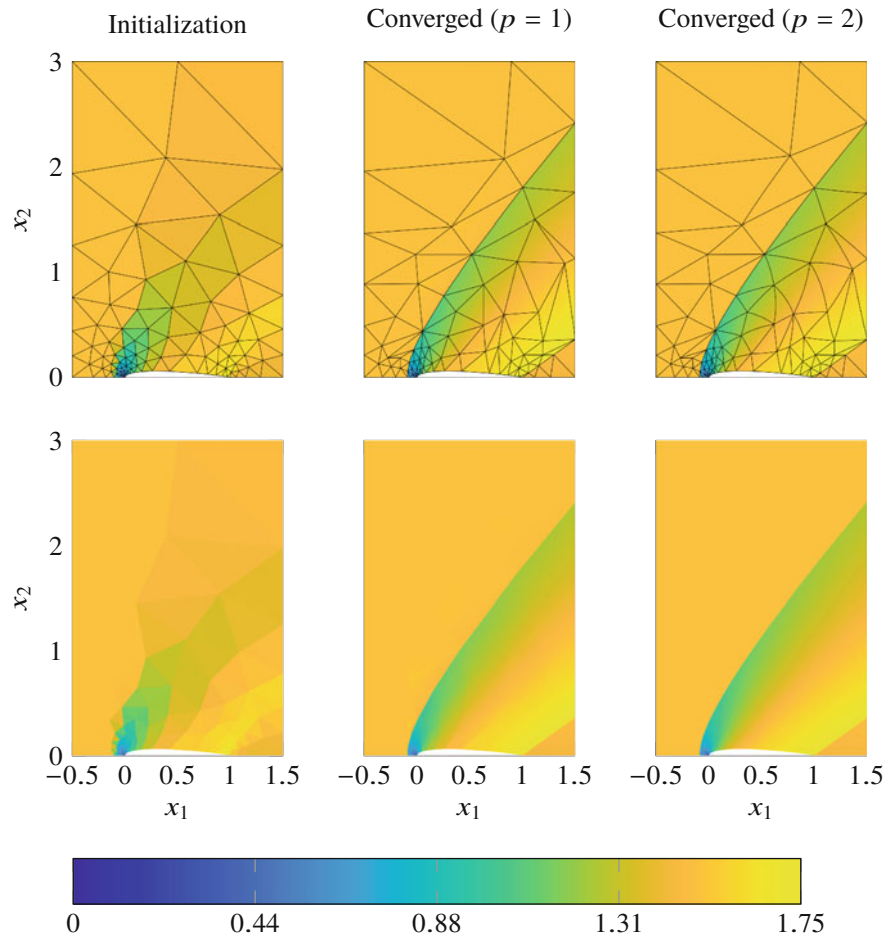


Fig. 12 Solution (Mach) of Euler equations over the NACA0012 airfoil ($M_\infty = 1.5$) using the implicit tracking method with a $p = q = 1$ (*center*) and $p = q = 2$ (*right*) basis for the solution and mesh with (*top*) and without (*bottom*) element boundaries. The implicit tracking procedure is initialized from a mesh generated without knowledge of the shock surface and a $p = 0$ DG solution (*left*)

solution being far from aligned with the shock. The second-order approximation is somewhat underresolved as seen by the faceted shock approximation and solution near the airfoil; however, the third-order solution is well-resolved: the high-order elements curve to the shock and the flow solution is well-resolved throughout the domain.

6 Conclusion

This work provides an overview of the implicit shock tracking method developed in [27, 35, 37] for steady and unsteady conservation laws with discontinuous solutions. For unsteady problems, both space-time and method of lines discretization approaches are considered. The key ingredient of the method is an optimization formulation that imposes the standard DG discretization as a constraint and minimizes the magnitude of the DG residual corresponding to an enriched test space (and a mesh quality term). The optimization variables are taken to be the DG solution and nodal coordinates, which are computed simultaneously using a sequential quadratic programming method. In the method of lines setting, the tracking procedure is applied at each stage of the high-order DIRK temporal discretization. We demonstrate the implicit shock tracking procedure using a number of standard steady and unsteady flow problems; in all cases, the method is capable of tracking discontinuities and providing high-quality flow approximations using coarse, high-order meshes. For unsteady problems, the method of lines approach is more practical, particularly as the size and difficulty of the problem increases; however, it is limited in that it cannot handle colliding shocks (triple points in space-time) without complex mesh operations and solution reinitialization. In these cases, the space-time approach is preferred due to its generality of tracking discontinuities in space-time, which naturally handles triple points. However, this may only hold for 1D and 2D problems, as the practicality of space-time approach for complex problems in 3D remains under investigation. Finally, for unsteady problems where the mesh can deform significantly as the shock moves across the domain, a curved mesh adaptation procedure consisting of a complete set of local mesh topology modification operators (edge collapses, splits, and flips) is required.

Acknowledgments This material is based upon work supported by the Air Force Office of Scientific Research (AFOSR) under award number FA9550-20-1-0236. The content of this publication does not necessarily reflect the position or policy of any of these supporters, and no official endorsement should be inferred.

References

1. Baines, M.J., Leary, S.J., Hubbard, M.E.: Multidimensional least squares fluctuation distribution schemes with adaptive mesh movement for steady hyperbolic equations. *SIAM J. Sci. Comput.* **23**(5), 1485–1502 (2002)
2. Baumann, C.E., Oden, J.T.: A discontinuous *hp* finite element method for the Euler and Navier-Stokes equations. *Int. J. Numer. Methods Fluids* **31**(1), 79–95 (1999). Tenth International Conference on Finite Elements in Fluids (Tucson, AZ, 1998)
3. Bell, J.B., Shubin, G.R., Solomon, J.M.: Fully implicit shock tracking. *J. Comput. Phys.* **48**(2), 223–245 (1982)
4. Burbeau, A., Sagaut, P., Bruneau, C.-H.: A problem-independent limiter for high-order Runge-Kutta discontinuous Galerkin methods. *J. Comput. Phys.* **169**(1), 111–150 (2001)

5. Cockburn, B., Shu, C.-W.: Runge-Kutta discontinuous Galerkin methods for convection-dominated problems. *J. Sci. Comput.* **16**(3), 173–261 (2001)
6. Corrigan, A., Kercher, A., Kessler, D.: A moving discontinuous Galerkin finite element method for flows with interfaces. *Int. J. Numer. Methods Fluids* **89**(9), 362–406 (2019)
7. Corrigan, A., Kercher, A., Kessler, D.: The moving discontinuous Galerkin method with interface condition enforcement for unsteady three-dimensional flows. In: *AIAA Scitech 2019 Forum* (2019)
8. Corrigan, A., Kercher, A., Kessler, D., Wood-Thomas, D.: Convergence of the moving discontinuous galerkin method with interface condition enforcement in the presence of an attached curved shock. In: *AIAA Aviation 2019 Forum*, pp. 3207 (2019)
9. Dervieux, A., Leservoisier, D., George, P.-L., Coudière, Y.: About theoretical and practical impact of mesh adaptation on approximation of functions and PDE solutions. *Int. J. Numer. Methods Fluids* **43**(5), 507–516 (2003). ECCOMAS Computational Fluid Dynamics Conference, Part I (Swansea, 2001)
10. Fidkowski, K.J.: Output error estimation strategies for discontinuous Galerkin discretizations of unsteady convection-dominated flows. *Int. J. Numer. Methods Eng.* **88**(12), 1297–1322 (2011)
11. Froehle, B., Persson, P.-O.: Nonlinear elasticity for mesh deformation with high-order discontinuous Galerkin methods for the Navier-Stokes equations on deforming domains. In: *Spectral and High Order Methods for Partial Differential Equations ICOSAHOM 2014*, pp. 73–85. Springer, Berlin (2015)
12. Gargallo-Peiró, A., Roca, X., Peraire, J., Sarrate, J.: Optimization of a regularized distortion measure to generate curved high-order unstructured tetrahedral meshes. *Int. J. Numer. Methods Eng.* **103**(5), 342–363 (2015)
13. Gargallo-Peiró, A., Roca, X., Peraire, J., Sarrate, J.: A distortion measure to validate and generate curved high-order meshes on CAD surfaces with independence of parameterization. *Int. J. Numer. Methods Eng.* **106**(13), 1100–1130 (2016)
14. Glimm, J., Li, X.-L., Liu, Y.-J., Xu, Z.-L., Zhao, N.: Conservative front tracking with improved accuracy. *SIAM J. Numer. Anal.* **41**(5), 1926–1947 (2003)
15. Harten, A., Engquist, B., Osher, S.J., Chakravarthy, S.R.: Uniformly high-order accurate essentially nonoscillatory schemes. III. *J. Comput. Phys.* **71**(2), 231–303 (1987)
16. Harten, A., Hyman, J.M.: Self adjusting grid methods for one-dimensional hyperbolic conservation laws. *J. Comput. Phys.* **50**(2), 235–269 (1983)
17. Hesthaven, J., Warburton, T.: *Nodal Discontinuous Galerkin Methods: Algorithms, Analysis, and Applications*. Springer, Berlin (2007)
18. Jiang, G.-S., Shu, C.-W.: Efficient implementation of weighted ENO schemes. *J. Comput. Phys.* **126**(1), 202–228 (1996)
19. Kercher, A.D., Corrigan, A.: A least-squares formulation of the moving discontinuous Galerkin finite element method with interface condition enforcement. *Comput. Math. Appl.* **95**, 143–171 (2021)
20. Kercher, A.D., Corrigan, A., Kessler, D.A.: The moving discontinuous Galerkin finite element method with interface condition enforcement for compressible viscous flows. *Int. J. Numer. Methods Fluids* **93**(5), 1490–1519 (2021)
21. Liu, X.-D., Osher, S.J., Chan, T.: Weighted essentially non-oscillatory schemes. *J. Comput. Phys.* **115**(1), 200–212 (1994)
22. Majda, A.: *Compressible Fluid Flow and Systems of Conservation Laws in Several Space Variables*, vol. 53. Springer, Berlin (2012)
23. Moretti, G.: Thirty-six years of shock fitting. *Comput. Fluids* **31**(4–7), 719–723 (2002)
24. Persson, P.-O., Bonet, J., Peraire, J.: Discontinuous Galerkin solution of the Navier–Stokes equations on deformable domains. *Comput. Methods Appl. Mech. Eng.* **198**(17), 1585–1595 (2009)
25. Persson, P.-O., Peraire, J.: Sub-cell shock capturing for discontinuous Galerkin methods. In: *44th AIAA Aerospace Sciences Meeting and Exhibit*, Reno, Nevada, 2006. AIAA-2006-0112

26. Rawat, P.S., Zhong, X.: On high-order shock-fitting and front-tracking schemes for numerical simulation of shock–disturbance interactions. *J. Comput. Phys.* **229**(19), 6744–6780 (2010)
27. Shi, A., Zahr, M.J., Persson, P.-O.: Implicit shock tracking for unsteady flows by the method of lines (2021). [arXiv:2101.08913](https://arxiv.org/abs/2101.08913)
28. Shu, C.-W., Osher, S.: Efficient implementation of essentially non-oscillatory shock-capturing schemes, II. In: *Upwind and High-Resolution Schemes*, pp. 328–374. Springer, Berlin (1989)
29. Shubin, G.R., Stephens, A.B., Glaz, H.M.: Steady shock tracking and Newton’s method applied to one-dimensional duct flow. *J. Comput. Phys.* **39**(2), 364–374 (1981)
30. Shubin, G.R., Stephens, A.B., Glaz, H.M., Wardlaw, A.B., Hackerman, L.B.: Steady shock tracking, Newton’s method, and the supersonic blunt body problem. *SIAM J. Sci. Stat. Comput.* **3**(2), 127–144 (1982)
31. Trepanier, J.-Y., Paraschivoiu, M., Reggio, M., Camarero, R.: A conservative shock fitting method on unstructured grids. *J. Comput. Phys.* **126**(2), 421–433 (1996)
32. Turcke, D.: On optimum finite element grid configurations. *AIAA J.* **14**(2), 264–265 (1976)
33. Van Rosendale, J.: Floating shock fitting via lagrangian adaptive meshes. Technical Report ICASE Report No. 94–89, Institute for Computer Applications in Science and Engineering, 1994
34. Wang, Z.J., Fidkowski, K., Abgrall, R., Bassi, F., Caraeni, D., Cary, A., Deconinck, H., Hartmann, R., Hillewaert, K., Huynh, H.T., et al.: High-order CFD methods: current status and perspective. *Int. J. Numer. Methods Fluids* **72**(8), 811–845 (2013)
35. Zahr, M.J., Persson, P.-O.: An optimization-based approach for high-order accurate discretization of conservation laws with discontinuous solutions. *J. Comput. Phys.* **365**, 105–134 (2018)
36. Zahr, M.J., Powers, J.M.: High-order resolution of multidimensional compressible reactive flow using implicit shock tracking. *AIAA J.* **59**(1), 150–164 (2021)
37. Zahr, M.J., Shi, A., Persson, P.-O.: Implicit shock tracking using an optimization-based high-order discontinuous Galerkin method. *J. Comput. Phys.* **410**, 109385 (2020)

Breakthrough ‘Workarounds’ in Unstructured Mesh Generation



Rainald Löhner

Abstract After a brief historical review of unstructured grid generation methods the two ‘breakthrough workarounds’ that made these methods reliable industrial tools are discussed. In many previous publications these important ‘workarounds’ were never mentioned. Yet without them computational science would not have become the third pillar of the empirical sciences (besides theory and experiments).

1 Introduction

All numerical methods that solve partial differential equations are based on the spatial subdivision of a domain into non-overlapping polyhedra (also denoted as [finite] elements or [finite] volumes) or points ([finite] points) with a defined vicinity. Common to all of these methods is the need to fill a computational domain (or surface) with a so-called mesh. For many decades the task of mesh generation took a secondary position to field solvers. It was considered handyman work as compared to the high nuances of field solvers, which could be embedded in Sobolev spaces and for which countless theorems in numerical analysis could be proven. And yet: no mesh, no run !

As field solvers matured and computers became more powerful, analysts attempted the simulation of ever increasing geometrical and physical complexity. At some point (probably around 1985), the main bottleneck in the analysis process became the grid generation itself.

For complex geometries, only tetrahedra hold the promise of complete automation and directional adaptivity, two important building blocks required in order to reduce the onerous man-hours grid generation has witnessed in the past, and opening the way to optimal grids, automatic design and optimization. For this reason, only tetrahedral grid generation is considered in the sequel.

R. Löhner (✉)

Center for Computational Fluid Dynamics, George Mason University, Fairfax, VA, USA
e-mail: rlohner@gmu.edu

The late 1980s and 1990s saw a considerable amount of effort devoted to automatic grid generation, as evidenced by the many references (e.g. [3–6, 10, 15–17, 19, 20, 22–37, 40–48, 51–53, 55, 58–60, 62, 65]) books (e.g. [7, 8, 14, 18, 21]) and Conferences devoted to the subject (e.g. the bi-annual International Conference on Numerical Grid Generation in Computational Fluid Dynamics and Related Fields ([1, 50, 61]) and the yearly Meshing Roundtable organized by Sandia Laboratories [49] (1992–present) resulting in a number of powerful and by now mature techniques. It was in this period that the two main ‘breakthrough workarounds’ in mesh generation occurred. Work in grid generation has not stopped (see, e.g. [2, 11, 38, 39, 56, 57, 63, 64] for a non-exhaustive list more recent work), but without these workarounds the reliability of mesh generators required for industrial production runs would not have been achieved.

Incidentally, the development of 3-D grid generation codes coincided with the advent of the 3-D graphics board (the Silicon Graphics workstations of the late 1980s). This was a key necessity in order to debug visually what can be at time very involved steps in the algorithms developed.

It is fair to state that for computational mechanics, the overwhelming amount of man-hours is still devoted to pre-processing. However, because of the availability of automatic grid generators, importing and cleaning geometry, specifying physics, boundary conditions, and run-time options are now the most tedious tasks.

2 Unstructured Grid Generation

There appear to be only two automatic ways to fill space with a general, possibly adaptively stretched, unstructured mesh of tetrahedra:

M.1 Fill Empty, i.e. Not Yet Gridded Space The idea here is to proceed from the boundaries into as yet ungridded space until the complete computational domain is filled with elements. The ‘front’ denotes the boundary between the region in space that has been filled with elements and that which is still empty. The key step is the addition of a new volume or **element** to the ungridded space. Methods falling under this category are called **advancing front techniques (AFTs)**.

M.2 Modify and Improve an Existing Grid In this case, an existing grid is modified by the introduction of new points. After the introduction of each point, the grid is reconnected or reconstructed locally in order to improve the mesh quality. The key step is the addition of a new **point** to an existing grid. The elements whose circumsphere contain the point are removed, and the resulting void faces reconnected to the point, thus forming new elements. The methods falling under this category are called **Delaunay triangulation techniques (DTTs)**.

It is interesting to note that in both cases the step from an algorithm or concept to a reliable code required ‘breakthrough workarounds’.

If only isotropic or near-isotropic elements are required, then ‘volume-to-surface’ techniques such as cut-cell or octree [52, 65] methods can also be employed for general geometries.

3 Advancing Front Techniques

The advancing front technique consists algorithmically of the following steps (see Fig. 1):

- F1. Define the boundaries of the domain to be gridded.
- F2. Define the spatial variation of element size, stretchings, and stretching directions for the elements to be created.
- F3. Using the information given for the distribution of element size and shape in space and the line-definitions, generate sides along the lines that connect surface patches. These sides form an initial front for the triangulation of the surface patches.
- F4. Using the information given for the distribution of element size and shape in space, the sides already generated, and the surface definition, triangulate the surfaces. This yields the initial front of faces.

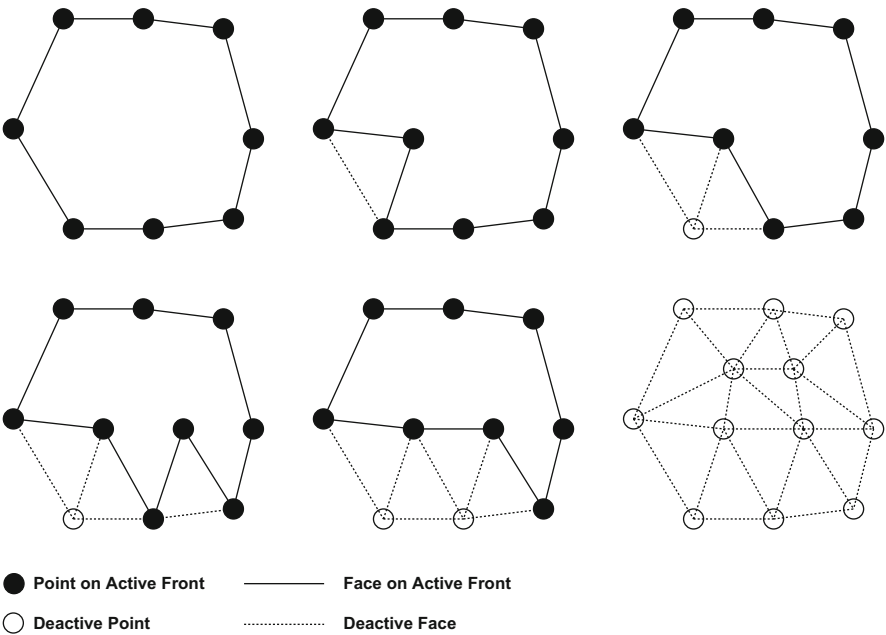


Fig. 1 Advancing front technique

- F5. Find the generation parameters (element size, element stretchings and stretching directions) for these faces.
- F6. Select the next face to be deleted from the front; in order to avoid large elements crossing over regions of small elements, the face forming the smallest new element is selected as the next face to be deleted from the list of faces.
- F7. For the face to be deleted:
 - F7.1 Select a ‘best point’ position for the introduction of a new point `ipnew`.
 - F7.2 Determine whether a point exists in the already generated grid that should be used in lieu of the new point. If there is such a point, set this point to `ipnew` and continue searching (`goto` F.7.2).
 - F7.3 Determine whether the element formed with the selected point `ipnew` crosses any given faces. If it does, select a new point as `ipnew` and try again (`goto` F.7.3).
- F8. Add the new element, point, and faces to their respective lists.
- F9. Find the generation parameters for the new faces from the background grid and the sources.
- F10. Delete the known faces from the list of faces.
- F11. If there are any faces left in the front, `goto` F6.

Algorithmically, the following tasks need to be accomplished reliably:

- Checking the intersection of faces (trivial for the eye, complicated to code);
- Fast data structures to minimize search overheads (steps F.5, F.6, F.7.2, F.7.3, F.9);
- Filtering and ordering of data to minimize CPU overheads.

Unlike the DTTS, where a rather complete set of theorems exist to prove optimality of discretization, the AFTs represent more of a ‘construction as you go’ or ‘handyman approach’ to mesh generation. As far as the author is aware, there is no ‘hard’ or ‘provable’ guarantee that the fronts will eventually close and a mesh can be obtained. And yet, AFTs are used daily in production environments where they work dependably and produce grids that are at least as good if not better than those from DDTs. Furthermore, they can be extended to cases where one needs to generate separated objects of arbitrary shape that are closely packed yet separated [38, 39].

3.1 The Workaround: Sweep and Retry

As teams developing 3-D AFTs soon discovered, the merging of fronts is fraught with problems: in some cases elements of the proper shape can not be introduced. This does not happen in 2-D, and for this reason developers tried to obtain a ‘clean solution’ for a while. However, as 3-D grids became commonplace, the high percentage of cases that could not be gridded had to be addressed. To see why 3-D is so much more difficult than 2-D, consider just the so-called Steiner prism shown in

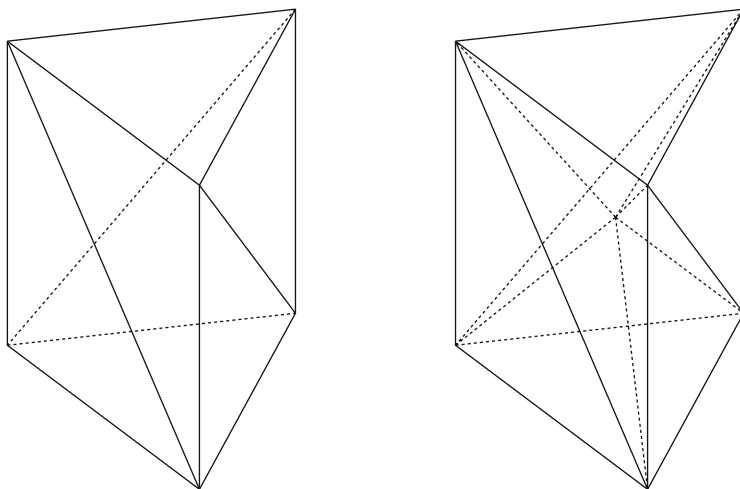


Fig. 2 Steiner prism; solution via extra point

Fig. 2a. There is no way to form a tetrahedron whose size is commensurate with the size of the prism. And this is just one of many possible cases that can be conceived. The immediate solution is to allow points closer to the face being removed—at the expense of smaller elements (see Fig. 2b). This solved a large percentage of the problems. Still, in some cases the front could not be closed.

So: how could one arrive at a reliable, robust advancing front mesher? The answer was a ‘workaround’: sweep and retry. The key idea is to mesh as much as possible, leaving the faces that could not introduce new elements in the front. Then, should any faces be left, several layers of elements attached to them are removed. These cavities are then remeshed again. This ‘sweep and retry’ technique, which has no sound theoretical basis and which could, in principle, fail, has proven extremely robust and reliable. It made the key difference in going from an algorithm that failed 10% of the time to a grid generation technique that only fails if either the surface triangulation is topologically incorrect, or the change in element size is extremely abrupt—and both of these cases are undesirable as far as accuracy and cost for computational mechanics codes.

The availability of the sweep and retry technique has also had two important side benefits:

- Mesh quality during grid generation: It is important (both for grid generation and field solvers) not to allow any bad elements. Therefore, if a well-shaped tetrahedron can not be introduced, the face is skipped.
- Smoothing: If elements with negative or small Jacobians appear during smoothing (as is the case with most spring-analogy smoothers), these elements are removed. The unmeshed regions of space are then enlarged and regridded. Smoothing improves the mesh quality substantially, leading to better results for field solvers.

4 Delaunay Triangulation

The Delaunay triangulation technique has a long history in mathematics, geophysics and engineering [18, 21]. Given a set of points $\mathcal{P} := \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$, one may define a set of regions or volumes $\mathcal{V} := v_1, v_2, \dots, v_n$ assigned to each of the points, that satisfy the following property: any location within v_i is closer to $\mathbf{x}_i$ than to any other of the points:

$$v_i := \mathcal{P} : \| \mathbf{x} - \mathbf{x}_i \| < \| \mathbf{x} - \mathbf{x}_j \| \quad \forall j \neq i . \quad (1)$$

This set of volumes $\mathcal{V}$, which covers the domain completely, is known as the **Dirichlet tessellation**. The volumes v_i are convex polyhedra and can assume fascinating shapes. They are referred to as Voronoi regions. Joining all the pairs of points $\mathbf{x}_i, \mathbf{x}_j$ across polyhedral boundaries results in a triangulation of the convex hull of $\mathcal{P}$. It is this triangulation that is commonly known as the **Delaunay triangulation**. The set of triangles (tetrahedra) that form the Delaunay triangulation satisfy the property that no other point is contained within the circumcircle (circumsphere) formed by the nodes of the triangle (tetrahedron). Although this and other properties of Delaunay triangulations have been studied for over a century time, practical 3-D triangulation procedures have only appeared in the mid-1980s, reaching maturity in the 1990s [3–6, 9, 10, 12, 17–20, 25, 28–30, 40–43, 51, 53, 54, 58–60, 62]. Most of these are based on the Bowyer-Watson algorithm [6, 58], which is summarized here for 3-D applications:

- B1. Define the convex hull within which all points will lie. This can either be a single large tetrahedron (4 points), or a brick (8 points) subdivided into five tetrahedra.
- B2. Introduce a new point $\mathbf{x}_{n+1}$.
- B3. Find all tetrahedra `LEDEL(1:NEDEL)` whose circumsphere contains $\mathbf{x}_{n+1}$; these are the tetrahedra that will be deleted. The resulting cavity is called star-shaped domain.
- B4. Find all the points belonging to these tetrahedra.
- B5. Find all the external faces `LFEXT(1:NFEXT)` of the void that results due to the deletion of these tetrahedra.
- B6. Form new tetrahedra by connecting the `NFEXT` external faces to the new point $\mathbf{x}_{n+1}$.
- B7. Add the new elements and the point to their respective lists.
- B8. Update all data structures.
- B9. If more points are to be introduced, goto B.2.

The algorithm described above assumes a given point distribution. The specification of a point distribution (in itself a tedious task) may be replaced by a general specification of desired element size and shape in space. The key idea, apparently proposed independently by Frey [13] and Holmes [25], is to check the discrepancy between the desired and the actual element shape and size of the current mesh.

Points are then introduced in those regions where the discrepancy exceeds a user-defined tolerance. An automatic Delaunay mesh generator of this kind proceeds as follows:

- D1. Assume given a boundary point distribution.
- D2. Generate a Delaunay triangulation of the boundary points.
- D3. Using the information stored on the background grid and the sources, compute the desired element size and shape for the points of the current mesh.
- D5. `nnewp=0` ! Initialize new point counter
- D6. `do ielem=1,nelem` ! Loop over the elements
 - Define a new point `inewp` at the centroid of `ielem`;
 - Compute the distances `dispc(1:4)` from `inewp` to the four nodes of `ielem`;
 - Compare `dispc(1:4)` to the desired element size and shape;
 - If any of the `dispc(1:4)` is smaller than a fraction α of the desired element length: skip the element (`goto D6.`);
 - Compute the distances `dispn(1:nneip)` from `inewp` to the new points in the neighbourhood;
 - If any of the `dispn(1:nneip)` is smaller than a fraction β of the desired element length: skip the element (`goto D6.`);
 - `nnewp=nnewp+1` ! Update new point list
 - Store the desired element size and shape for the new point;
 - `enddo`
- D7. `if (nnewp.gt.0) then`
 - Perform a Delaunay Triangulation for the new points;
 - `goto D.5`
- `endif`

The procedure outlined above introduces new points in the elements. One can also introduce them at edges [21].

Algorithmically, the following tasks need to be accomplished reliably:

- Checking whether the circumsphere of a tetrahedron contains a point (far from trivial when the point is close to the radius);
- Fast data structures to minimize search overheads (steps B.3, B.5, D.3, D.6);
- Filtering and ordering of data to minimize CPU overheads;
- Recovering the original surface.

4.1 The Workaround: ‘Almost Delaunay’

As teams developing Delaunay triangulation techniques soon discovered, the pure Delaunay circumsphere criterion can lead to a high percentage of degenerate elements called ‘slivers’. In 2-D the probability of bad elements is much lower than

in 3-D, and for this reason this shortcoming was ignored for a while. However, as 3-D grids became commonplace, the high number of slivers present in typical Delaunay grids had to be addressed. The best way to avoid slivers is by relaxing the ‘sound, mathematically elegant’ Delaunay criterion by a ‘workaround’: The star-shaped domain is modified by adding back elements whose faces would lead to bad elements. This fundamental departure from the traditional Delaunay criterion, first proposed by George [17] to the chagrin of many mathematicians and computational geometers, has allowed this class of unstructured grid generation algorithms to produce reliably quality grids. It is a simple change, but has made the difference between a theoretical exercise and a practical tool.

One should remark that before this workaround, mesh improvement techniques such as edge flip, face swap and node smoothing had already been developed, and were used extensively to remove the high number of slivers. However, in some cases, if a mesh is too bad to start with, these methods can only do so much. It is much better to avoid the bad elements from the very beginning.

5 Examples

No grid generation paper would be complete without some examples. Three are included here, taken from solid and fluid mechanics. In all cases, the grids were generated using advancing front techniques, either for elements [33, 37] or for objects defined by surface triangulations [38, 39].

5.1 *Mechanical Parts*

The surface mesh of this rather complicated assembly of mechanical parts is shown in Fig. 3a–d. The final generated mesh had about 400 Ktets.

5.2 *Ignition of Pellets*

This case considers the ignition of pellets in a tube. Figure 4a shows the tube, indicating the ignition point. The pellets are shown in Fig. 4b, c. Note that in this case the pellets are generated by first defining the surface triangulation for one pellet, and then filling the available space using an advancing grid generator for objects [38, 39]. This simulation was performed by coupling flow and thermal codes. A plane cut of the resulting fields is shown in Fig. 5.

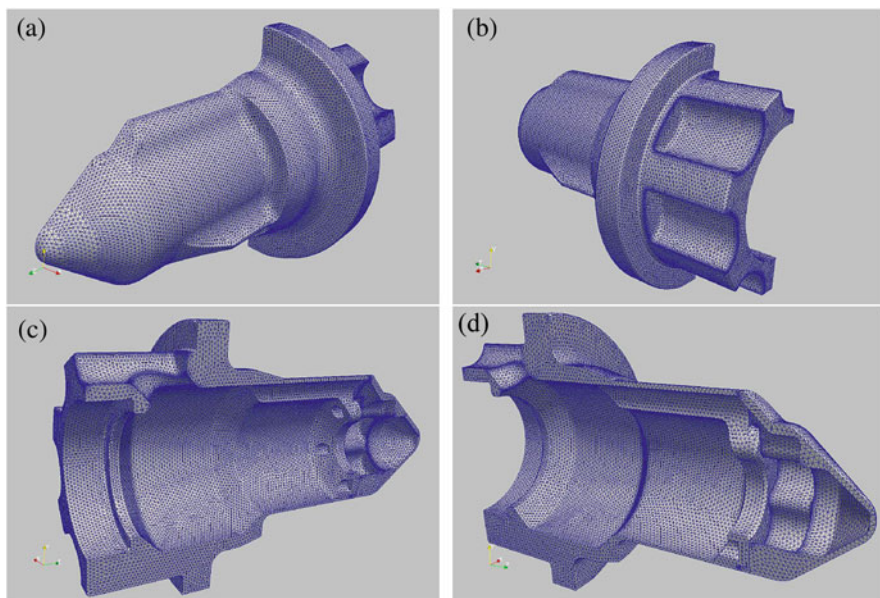


Fig. 3 (a–d) Mechanical parts

5.3 *Toldo for Holy Mosque in Makkah*

This case considers an environmental quality simulation in the area inside and around the holy mosque in Makkah, Saudi Arabia. The CAD definition is shown in Fig. 6. The simplified surface definition used for the run, as well as three views of the mesh employed are shown in Figs. 7, 8, 9, and 10. The CO₂ distribution at the level of the pilgrims under the toldo is shown in Fig. 11. Even though the domain is rather large, the total number of elements was kept to a modest 250 Mels.

6 Conclusions and Outlook

Many textbooks, publications and reports have recorded the emergence of automatic unstructured grid generators. In most of these, the reader is left with the impression that all is well and it simply takes a bit of coding to obtain a mesh to whatever the problem at hand.

What is very often not mentioned are the very things that make successful or widely used codes work. We have decided to label these as ‘workarounds’. The reason is that they deviate from the strict implementation of the algorithms exposed in these publications, and in many cases are rather primitive heuristic shortcuts

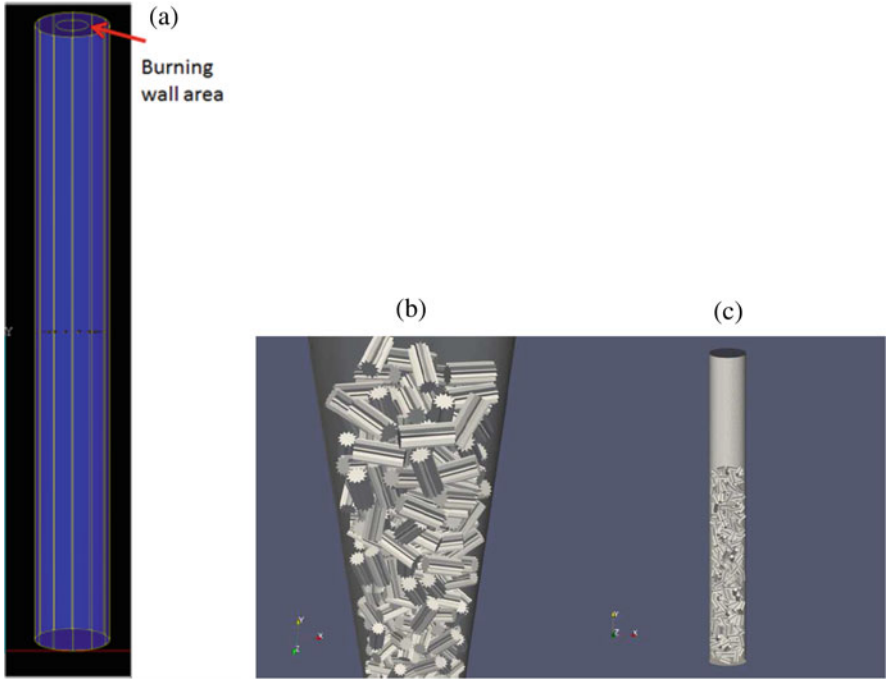


Fig. 4 (a–c) Ignition of pellets

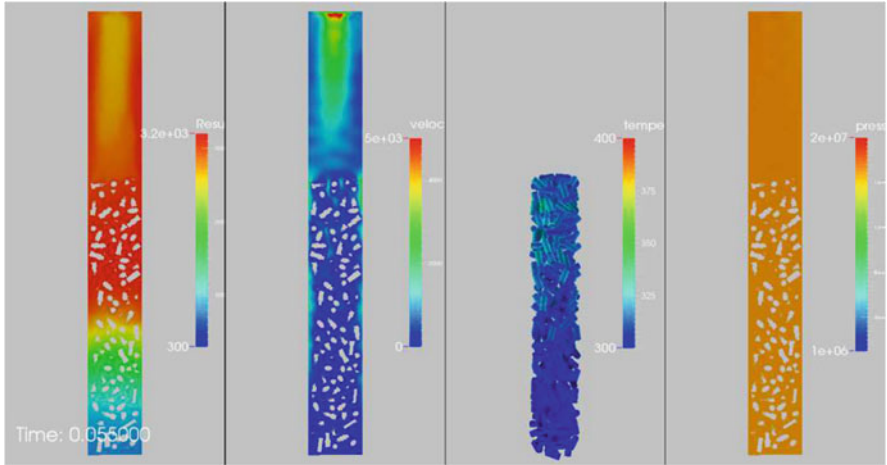


Fig. 5 Ignition of pellets: flow and thermal fields

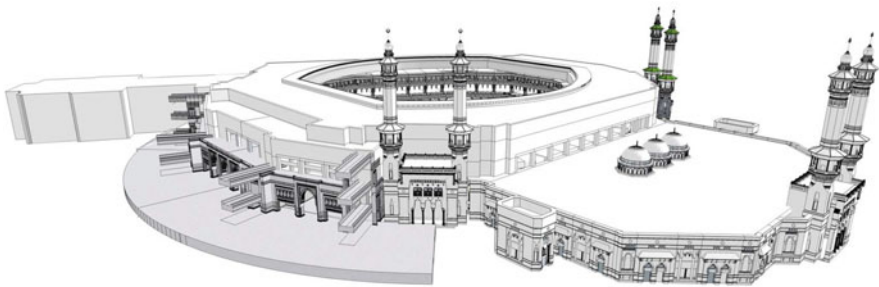


Fig. 6 Holy Mosque at Makkah: CAD definition

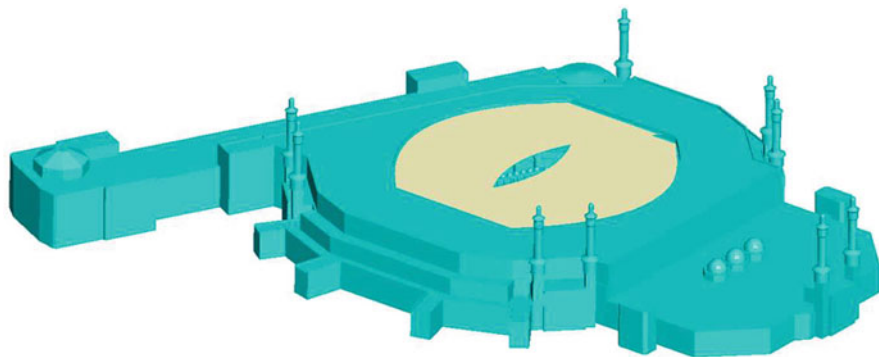


Fig. 7 Holy Mosque at Makkah: simplified surface definition

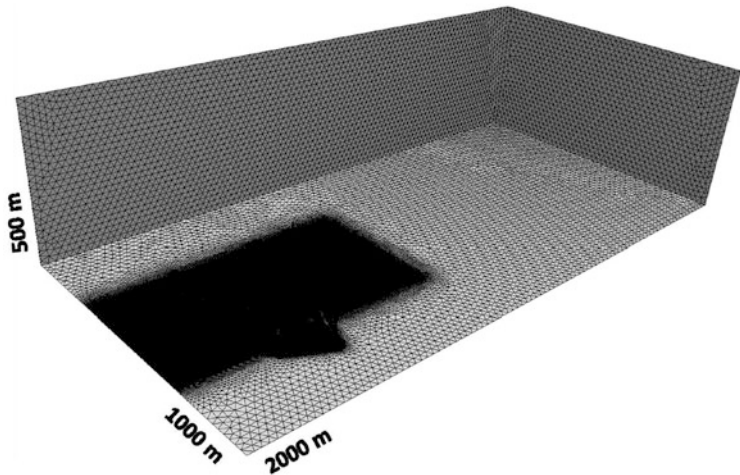


Fig. 8 Holy Mosque at Makkah: surface mesh

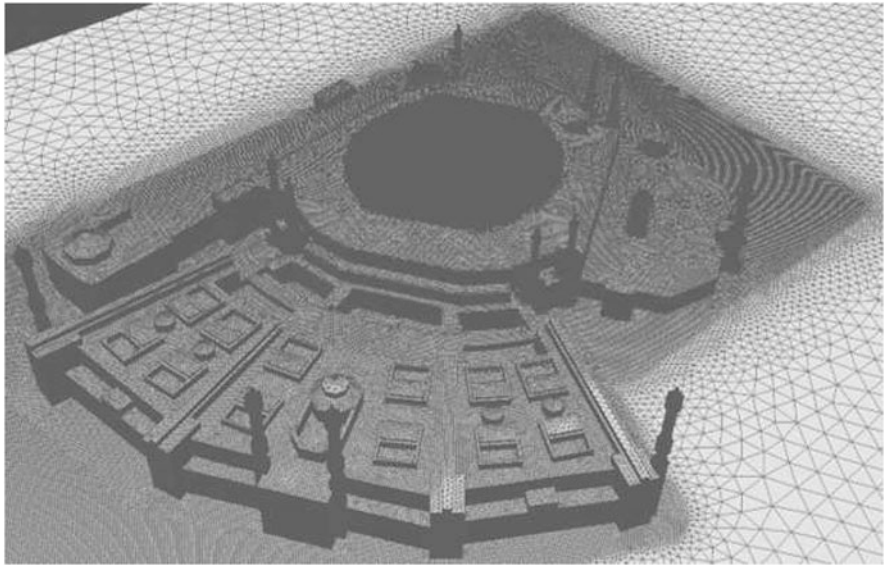


Fig. 9 Holy Mosque at Makkah: detail of surface mesh

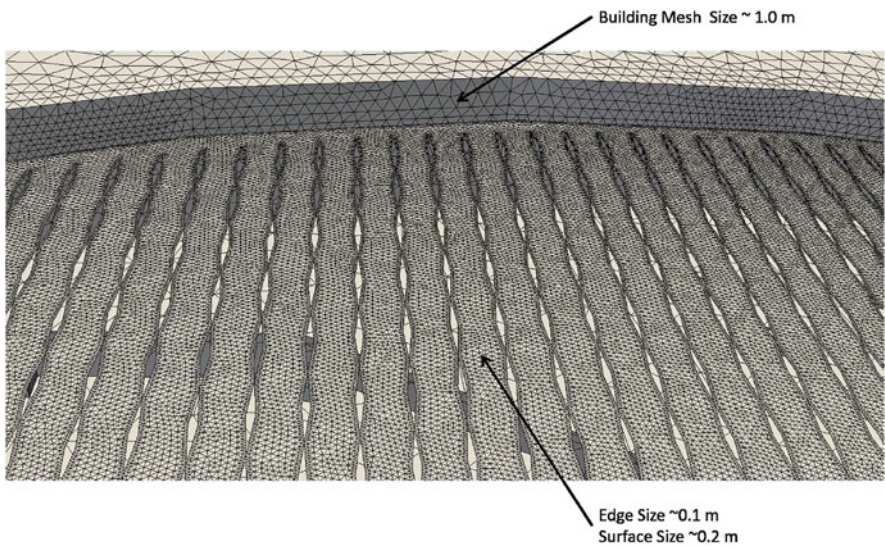


Fig. 10 Holy Mosque at Makkah: detail of surface mesh

without a firm theoretical basis. Yet these ‘workarounds’ are essential in order to get reliable, working codes.

We felt that in a time when these codes are seeing widespread use as ‘black boxes’ or ‘open source libraries’, without users or developers ever going into the

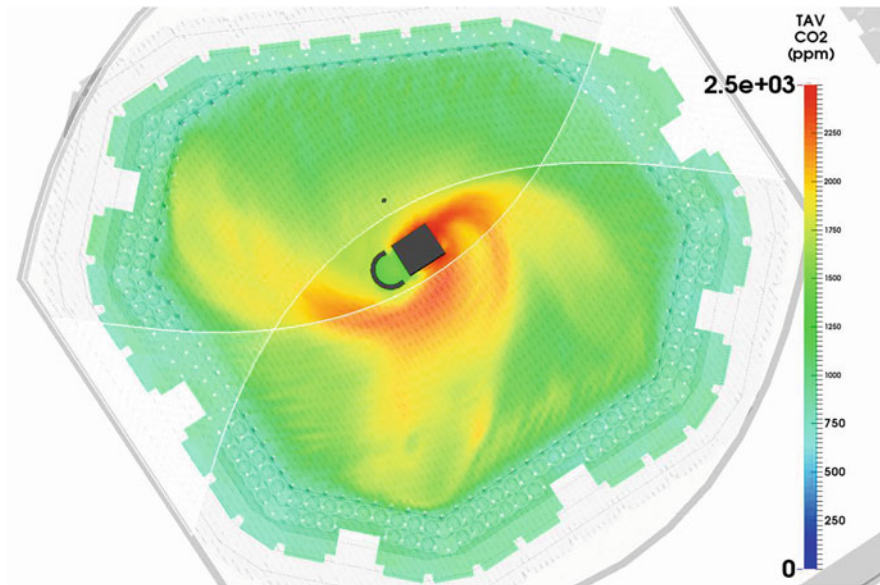


Fig. 11 Holy Mosque at Makkah: CO₂ levels at pilgrim height

details of the actual coding, it would be prudent to record these workarounds for posterity.

Acknowledgments It is a great pleasure to acknowledge insightful discussions with Oubey Hassan on a whole range of topics in computational mechanics. He has distinguished himself in an astounding number of fields, among them grid generation, flow and electromagnetic solvers, parallel computing, visualization, optimization and control, and has contributed in each of these in a significant way.

The collaboration with SLRasch, Stuttgart for the toldo case and ASI Inc., Potomac, MD, for the pellet ignition simulation is greatly acknowledged.

References

1. Arcilla, A.S., Häuser, J., Eiseman, P.R., Thompson, J.F. (eds.): Proc. 3rd Int. Conf. Numerical Grid Generation in Computational Fluid Dynamics and Related Fields. North-Holland (1991)
2. Aubry, R., Löhner, R.: Generation of viscous grids at ridges and corners. *Int. J. Numer. Methods Eng.* **77**, 1247–1289 (2009)
3. Baker, T.J.: Three-Dimensional Mesh Generation by Triangulation of Arbitrary Point Sets. AIAA-CP-87-1124, 8th CFD Conf., Hawaii (1987)
4. Baker, T.J.: Developments and trends in three-dimensional mesh generation. *Appl. Numer. Methods* **5**, 275–304 (1989)
5. Boender, E.: Reliable Delaunay-based mesh generation and mesh improvement. *Commun. Appl. Numer. Methods Eng.* **10**, 773–783 (1994)
6. Bowyer, A.: Computing Dirichlet tessellations. *Comput. J.* **24**(2), 162–167 (1981)

7. Carey, G.F.: Grid Generation, Refinement, and Redistribution. Wiley, London (1993)
8. Carey, G.F.: Computational Grids: Generation, Adaption, and Solution. Taylor & Francis, London (1997)
9. Cavendish, J.C.: Automatic triangulation of arbitrary planar domains for the finite element method. *Int. J. Numer. Methods Eng.* **8**, 679–696 (1974)
10. Cavendish, J.C., Field, D.A., Frey, W.H.: An approach to automatic three-dimensional finite element generation. *Int. J. Numer. Methods Eng.* **21**, 329–347 (1985)
11. Chen, J., Zheng, J., Zheng, Y., Si, H., Hassan, O., Morgan, K.: Improved boundary constrained tetrahedral mesh generation by mesh transformation. *Appl. Math. Model.* **51**, 764–790 (2017)
12. Choi, B.K., Chin, H.Y., Loon, Y.I., Lee, J.W.: Triangulation of scattered data in 3D space. *Comput. Aided Geom. Design* **20**, 239–248 (1988)
13. Frey, W.H.: Selective refinement: a new strategy for automatic node placement in graded triangular meshes. *Int. J. Numer. Methods Eng.* **24**, 2183–2200 (1987)
14. Frey, P.J., George, P.L.: Mesh Generation Application to Finite Elements. Hermes Science Publishing, Oxford, Paris (2000)
15. Frykestig, J.: Advancing Front Mesh Generation Techniques with Application to the Finite Element Method. Pub. 94:10, Chalmers University of Technology, Göteborg, Sweden (1994)
16. Fuchs, A.: Automatic grid generation with almost regular Delaunay tetrahedra. In: *Proc. 7th Int. Meshing Roundtable*, pp. 133–148 (1998)
17. George, P.L., Hecht, F., Saltel, E.: Fully automatic mesh generation for 3D domains of any shape. *Impact Comput. Sci. Eng.* **2**(3), 187–218 (1990)
18. George, P.L.: Automatic Mesh Generation. Wiley, London (1991)
19. George, P.L., Hecht, F., Saltel, E.: Automatic mesh generator with specified boundary. *Comput. Methods Appl. Mech. Eng.* **92**, 269–288 (1991)
20. George, P.L., Hermeline, F.: Delaunay's mesh of a convex polyhedron in dimension d . Application to arbitrary Polyhedra. *Int. J. Numer. Methods Eng.* **33**, 975–995 (1992)
21. George, P.L., Borouchaki, H.: Delaunay Triangulation and Meshing. Editions Hermes, Paris (1998)
22. Hassan, O., Probert, E.J., Morgan, K., Peraire, J.: Mesh generation and adaptivity for the solution of compressible viscous high speed flows. *Int. J. Numer. Methods. Eng.* **38**(7), 1123–1148 (1995)
23. Hassan, O., Morgan, K., Probert, E.J., Peraire, J.: Unstructured tetrahedral mesh generation for three-dimensional viscous flows. *Int. J. Numer. Methods Eng.* **39**(4), 549–567 (1996)
24. Holmes, D.G., Lamson, S.C.: Compressible flow solutions on adaptive triangular meshes. In: *Open Forum AIAA- Reno'86—Meeting* (1986)
25. Holmes, D.G., Snyder, D.D.: The generation of unstructured triangular meshes using Delaunay triangulation. In: Sengupta et al. (eds.), *Numerical Grid Generation in Computational Fluid Dynamics*, pp. 643–652. Pineridge Press, Swansea, Wales (1988)
26. Huet, F.: Generation de Maillage Automatique dans les Configurations Tridimensionnelles Complexes Utilisation d'une Methode de 'Front'; AGARD-CP-464, 17 (1990)
27. Jin, H., Tanner, R.I.: Generation of unstructured tetrahedral meshes by the advancing front technique. *Int. J. Numer. Methods Eng.* **36**, 1805–1823 (1993)
28. Joe, B.: Construction of three-dimensional Delaunay triangulations using local transformations. *Comput. Aided Geom. Design* **8**, 123–142 (1991)
29. Joe, B.: Delaunay versus max-min solid angle triangulations for three-dimensional mesh generation. *Int. J. Numer. Methods Eng.* **31**, 987–997 (1991)
30. Lee, D.T., Schachter, B.J.: Two algorithms for constructing a Delaunay triangulation. *Int. J. Comput. Inf. Sc.* **9**(3), 219–242 (1980)
31. Lo, S.H.: A new mesh generation scheme for arbitrary planar domains. *Int. J. Numer. Methods Eng.* **21**, 1403–1426 (1985)
32. Lo, S.H.: Finite element mesh generation over curved surfaces. *Comput. Struct.* **29**, 731–742 (1988)
33. Löhner, R., Parikh, P.: Three-dimensional grid generation by the advancing front method. *Int. J. Numer. Methods Fluids* **8**, 1135–1149 (1988)

34. Löhner, R.: Three-dimensional fluid-structure interaction using a finite element solver and adaptive remeshing. *Comput. Syst. Eng.* **1**(2–4), 257–272 (1990)
35. Löhner, R., Baum, J.D.: Adaptive H-refinement on 3-D unstructured grids for transient problems. *Int. J. Numer. Methods Fluids* **14**, 1407–1419 (1992)
36. Löhner, R., Camberos, J., Merriam, M.: Parallel unstructured grid generation. *Comput. Methods Appl. Mech. Eng.* **95**, 343–357 (1992)
37. Löhner, R.: Automatic unstructured grid generators. *Finite Elements Anal. Design* **25**, 111–134 (1997)
38. Löhner, R., Oñate, E.: An advancing front technique for filling space with arbitrary objects. *Int. J. Numer. Methods Eng.* **78**, 1618–1630 (2009)
39. Löhner, R., Oñate, E.: Advancing front techniques for filling space with arbitrary separated objects. *Finite Elements Anal. Design* **46**, 140–151 (2010)
40. Marcum, D.L., Weatherill, N.P.: Unstructured grid generation using iterative point insertion and local reconnection. *AIAA J.* **33**(9), 1619–1625 (1995)
41. Mavriplis, D.J.: An advancing front Delaunay triangulation algorithm designed for robustness. In: AIAA-93-0671 (1993)
42. Merriam, M.: An efficient advancing front algorithm for Delaunay triangulation. In: AIAA-91-0792 (1991)
43. Merriam, M., Barth, T.: 3D CFD in a day: the laser digitizer project. In: AIAA-91-1654 (1991)
44. Peraire, J., Vahdati, M., Morgan, K., Zienkiewicz, O.C.: Adaptive remeshing for compressible flow computations. *J. Comput. Phys.* **72**, 449–466 (1987)
45. Peraire, J., Peiro, J., Formaggia, L., Morgan, K., Zienkiewicz, O.C.: Finite element Euler calculations in three dimensions. *Int. J. Numer. Methods Eng.* **26**, 2135–2159 (1988)
46. Peraire, J., Morgan, K., Peiro, J.: Unstructured finite element mesh generation and adaptive procedures for CFD. In: AGARD-CP-464, 18 (1990)
47. Peraire, J., Peiro, J., Morgan, K.: Adaptive remeshing for three-dimensional compressible flow computations. *J. Comput. Phys.* **103**, 269–285 (1992)
48. Rebay, S.: Efficient unstructured mesh generation by means of Delaunay triangulation and Bowyer-Watson algorithm. *J. Comput. Phys.* **106**(1), 125–138 (1993)
49. Sandia National Laboratories: Proc. International Meshing Roundtable (1992–Present)
50. Sengupta, S., Häuser, J., Eisman, P.R., Thompson, J.F. (eds.): Proc. 2nd Int. Conf. Numerical Grid Generation in Computational Fluid Dynamics. Pineridge Press, Swansea, Wales (1988)
51. Shenton, D.N., Cendes, Z.J.: Three-dimensional finite element mesh generation using Delaunay tessellation. In: IEEE Trans. on Magnetics, MAG-21, pp. 2535–2538 (1985)
52. Shepard, M.S., Georges, M.K.: Automatic three-dimensional mesh generation by the finite octree technique. *Int. J. Numer. Methods Eng.* **32**, 709–749 (1991)
53. Sloan, S.W., Houlsby, G.T.: An implementation of Watson's algorithm for computing 2-dimensional Delaunay triangulations. *Adv. Eng. Softw.* **6**(4), 192–197 (1984)
54. Tanemura, M., Ogawa, T., Ogita, N.: A new algorithm for three-dimensional Voronoi tessellation. *J. Comput. Phys.* **51**, 191–207 (1983)
55. van Phai, N.: Automatic mesh generation with tetrahedron elements. *Int. J. Numer. Methods Eng.* **18**, 237–289 (1982)
56. Walton, S., Hassan, O., Morgan, K.: Advances in co-volume mesh Generation and mesh optimization techniques. *Comput. Struct.* **181**, 70–88 (2017)
57. Wang, D., Hassan, O., Morgan, K., Weatherill, N.: Enhanced remeshing from STL files with applications to surface grid generation. *Commun. Numer. Methods Eng.* **23**(3), 227–139 (2007)
58. Watson, D.F.: Computing the N-dimensional Delaunay tessellation with application to Voronoi polytopes. *Comput. J.* **24**(2), 167–172 (1981)
59. Weatherill, N.P.: Delaunay triangulation in computational fluid dynamics. *Comput. Methods Appl.* **24**(5/6), 129–150 (1992)
60. Weatherill, N.P., Hassan, O., Marchant, M.J., Marcum, D.L.: Adaptive inviscid flow solutions for aerospace geometries on efficiently generated unstructured tetrahedral meshes. In: AIAA-93-3390 (1993)

61. Weatherill, N.P., Eiseman, P.R., Häuser, J., Thompson, J.F. (eds.): Proc. 4th Int. Conf. Numerical Grid Generation in Computational Fluid Dynamics and Related Fields. Pineridge Press, Swansea, Wales (1993)
62. Weatherill, N.P., Hassan, O.: Efficient three-dimensional Delaunay triangulation with automatic point creation and imposed boundary constraints. *Int. J. Numer. Methods Eng.* **37**, 2005–2039 (1994)
63. Xie, Z.Q., Hassan, O., Morgan, K.: Tailoring unstructured meshes for use with a 3-D time domain co-volume algorithm in computational electromagnetics. *Int. J. Numer. Methods Eng.* **87**, 48–65 (2011)
64. Xie, Z.Q., Sevilla, R., Hassan, O., Morgan, K.: The generation of arbitrary order curved meshes for 3D finite element analysis. *Comput. Mech.* **51**, 361–374 (2013)
65. Yerry, M.A., Shepard, M.S.: Automatic three-dimensional mesh generation by the modified-octree technique. *Int. J. Numer. Methods Eng.* **20**, 1965–1990 (1984)

An Adaptive Conservative Moving Mesh Method



Simone Appella, Chris Budd, and Tristan Pryer

Abstract We present an adaptive moving mesh strategy endowed with a mass conservation property for the numerical solution of the two-dimensional linear transport equation. We achieve this through the solution of an auxiliary moving mesh problem driven by a gradient based monitor function. The method we propose is of staggered type, hence requires an appropriate data transfer operator over different meshes. We make use of an L^2 projection operator that requires the construction of a *supermesh* for evaluation. We summarise extensive numerical experiments aimed at testing the robustness of the method and its ability to approximate challenging numerical features.

1 Introduction

Partial Differential Equations (PDEs) arise from a variety of areas [1], including meteorological, such as the semi-geostrophic equations [2], oceanographical, such as the Korteweg-de Vries (KdV) and optical, such as the nonlinear Schrödinger equations [3]. These equations are examples of *conservation laws*, which often develop very localised transient features, such as shocks and moving fronts. These features are difficult to approximate using standard numerical schemes posed over uniform meshes. One potential solution to this, well used in finite element techniques, is to employ different refinement strategies to increase the spatial accuracy in the region of the propagating shock. These include local mesh refinement [4], local polynomial enrichment [5] and moving the underlying mesh nodes [6], h -, p - and r -refinement, respectively. One of the common features of these refinement strategies is that they all alter the number or the location of the degrees of freedom in the mesh.

S. Appella (✉) · C. Budd · T. Pryer
Department of Mathematical Sciences, University of Bath, Bath, UK
e-mail: sa2407@bath.ac.uk; mascjb@bath.ac.uk; tmp38@bath.ac.uk

Moving mesh methods are, by now, well studied in the numerical solution of PDEs [7, 8]. The main advantage of these methods is that they do not alter the topology of the initial mesh, preserving the number of nodes, connectivity and data structures. This allows for an efficient implementation and coupling to existing CFD solvers [9]. The main ingredients, in addition to the underlying PDE discretisation, are the mesh equations and the monitor function. The mesh adaptation is driven by the monitor function. This controls the relative density of the mesh points in the physical domain and is commonly based on the gradient or curvature of the field [7, 10].

Fundamentally, moving mesh strategies can be split into two categories [11]:

- Interpolation free (*quasi-Lagrange*) methods—The mesh equation and PDE are approximated simultaneously. This is computationally more complex as the position of the nodes of the mesh are additional degrees of freedom in the system. There is, however, no requirement to transfer data from an old mesh to the new one.
- Interpolation based (*rezoning*) methods—The mesh equation and PDE are staggered, the PDE is solved first and mesh equations subsequently based on that solution. Computationally, this is cheaper than interpolation free methods. However a *data transfer operator* is required to interpret the discrete solution over the new mesh.

In this work, we are concerned with interpolation based methods and pay particular attention to properties of the *data transfer operator*, a necessity allowing the evolution of the discrete solution as the underlying mesh changes. Its definition does introduce various challenges as it is a common source of diffusion. Embedding conservative properties into numerical schemes is often in conflict with the design of stable schemes in CFD. The long term dynamics of solutions can be destroyed by the addition of *artificial numerical diffusion*. The reason for inclusion of the proposed transfer operator is the desirable stability properties this endows on the discrete scheme.

Our focus in this paper is illustrating how the data transfer step can be done in a conservative fashion through a *local Galerkin projection*, removing one of the inherent disadvantages of interpolation based methods over those that are interpolation free [12]. More specifically, the conservative property is the total mass of the scalar field on which this procedure is applied. This requires the construction of a *supermesh*, a conforming triangulation obtained by the intersection between the old and the new mesh.

The novelty of this work consists of coupling the conservative Galerkin projection with Winslow's *variable diffusion method* for mesh movement [13]. To showcase the properties of our approach, we employ the adaptive strategy with a well used upwinding *discontinuous Galerkin* (dG) discretisation of the transport problem. We use this as our test problem as it is arguably the most challenging conservation law to approximate despite its linearity. The reason for this is that any numerical errors are propagated in time, there is no chance to rectify this through

entropic loss as is common in other (systems of) conservation laws that develop shocks, shallow water equations or Euler's equations, for example.

We conduct numerical experiments to assess the robustness and approximability of the r -adaptive procedure and the adaptive-conservative dG scheme. We are able to show that:

1. The moving mesh method is able to generate meshes locally refined around areas of interest. Families of these meshes are able to optimally approximate even discontinuous functions.
2. The conservative r -adaptive approximation significantly outperforms the non-conservative variant as well as the equivalent simulation over a uniform grid.
3. The moving mesh problem is sensitive to parametric choices, especially for irregular functions.
4. With good choices of these parameters, the r -adaptive approximation of the transport equation outperforms the uniform counterpart.

The remainder of the paper is structured as follows: In Sect. 2, we introduce the problem, relevant notation and the discretisation scheme. Section 3 contains an overview of moving mesh methods used for solving time-dependent PDEs as well as our proposed moving mesh scheme using Winslow's method. In Sect. 4, we describe the local conservative Galerkin projection and the algorithm that is used to construct and process the supermesh. Section 5 illustrate numerical experiments designed to test the discretisation of the moving mesh PDE. Finally, we conclude in Sect. 6.

2 Problem Setup and Discretisation

Throughout this work we denote the standard Lebesgue spaces by $L^p(\omega)$ for $\omega \subseteq \mathbb{R}$, $p \in [1, \infty]$, equipped with corresponding norms $\|u\|_{L^p(\omega)}$. In addition, we define $H^k(\omega)$ as the Hilbert space of order $k \in \mathbb{N}$ of functions u defined over $\omega \subseteq \mathbb{R}$:

$$H^k(\omega) := \{u \in L^2(\omega) : \|u\|_{H^k(\omega)} < \infty\}, \quad (1)$$

where $\|u\|_{H^k(\omega)}^2 = \sum_{\alpha \leq k} |D^\alpha u|^2$ and D^α are understood in weak sense.

We also make use of the following notation for time-dependent Bochner spaces:

$$L^p(0, T; H^k(\omega)) := \left\{ u : [0, T] \rightarrow H^k(\omega) : \int_0^T \|u(t)\|_{H^k(\omega)}^p dt < \infty \right\}. \quad (2)$$

Furthermore, for a portion of the boundary $\Gamma \subset \partial\omega$, we define

$$H^1_{0,\Gamma}(\omega) := \left\{ u \in H^1(\omega) : u|_\Gamma = 0 \right\}. \quad (3)$$

Let $\Omega \subset \mathbb{R}^2$ be a convex, simply connected domain of interest with outward pointing normal $\mathbf{n}$ and consider the problem

$$\begin{aligned}\partial_t u + \mathbf{v} \cdot \nabla u &= 0 \text{ in } \Omega \times [0, T] \\ u(\mathbf{x}, 0) &= u_0(\mathbf{x}) \text{ on } \Omega \times \{0\} \\ u &= g \text{ on } \partial\Omega_- \times [0, T],\end{aligned}\tag{4}$$

where

$$\partial\Omega_- = \{x \in \partial\Omega : \mathbf{v} \cdot \mathbf{n} < 0\},\tag{5}$$

$\mathbf{v}$ denotes a prescribed velocity, and the inflow boundary condition $g \in L^2(\partial\Omega_-)$ for $t \in [0, T]$.

Let $\mathcal{T}$ be a conforming mesh of Ω into simplicial and/or box-type elements. Note that we believe it is possible to extend the method quite naturally to polytopic meshes [14]. For exposition, though, we consider $\mathcal{T}$ as a finite family of sets such that:

1. $K \in \mathcal{T}$ implies K is an open triangle or quadrilateral,
2. for any $K, J \in \mathcal{T}$ we have that $\overline{K} \cap \overline{J}$ is either empty or a complete $(2 - r)$ -dimensional simplex/box (i.e., it is either a vertex for $r = 2$, or an edge for $r = 1$, or the whole of $\overline{K}$ and $\overline{J}$) of both $\overline{K}$ and $\overline{J}$ and
3. $\bigcup_{K \in \mathcal{T}} \overline{K} = \overline{\Omega}$.

We consider the *finite element space*

$$\mathbb{V} := \{\Phi \in L^2(\Omega) : \Phi|_K \in \mathbb{P}^k(K)\},\tag{6}$$

where $\mathbb{P}^k(K)$ is the space of polynomials of total degree k for $k \geq 0$. Alternatively, when $K \in \mathcal{T}$ is a box-type element, we can also consider polynomials of degree k in each variable, typically mapped from a reference hypercube.

For $w \in \mathbb{V}$ and any $K \in \mathcal{T}$, we define the *upwind jump* across the inflow boundary ∂K_- by

$$[w](\mathbf{x}) := \lim_{\epsilon \rightarrow 0^+} (w(\mathbf{x} + \epsilon \mathbf{v}) - w(\mathbf{x} - \epsilon \mathbf{v})).\tag{7}$$

Let us denote $\mathcal{G} := \{w \in L^2(\Omega) : \mathbf{v} \cdot \nabla w \in L^2(\Omega)\}$ as the graph space of the PDE. For $w \in \mathcal{G}$, we let w_K^+ and w_K^- denote the interior and exterior trace of w on K , respectively. The exterior trace on K can be referred to as the interior trace on K' , where K' shares an edge contained in $\partial K_- / \partial\Omega_-$. Then

$$[w]|_K := w|_K^+ - w|_K^-.\tag{8}$$

Let

$$\begin{aligned} \mathcal{A}(u, \Phi) &:= \sum_{K \in \mathcal{T}} \left[\int_K \partial_t u \Phi + \mathbf{v} \cdot \nabla u \Phi \, dx - \int_{\partial K_- / \partial \Omega_-} \mathbf{v} \cdot \mathbf{n} [u] \Phi^+ \, ds \right. \\ &\quad \left. - \int_{\partial K_- \cap \partial \Omega_-} \mathbf{v} \cdot \mathbf{n} u^+ \Phi^+ \, ds \right] \\ l(\Phi) &:= \sum_{K \in \mathcal{T}} \left[\int_{\partial K_- \cap \partial \Omega_-} \mathbf{v} \cdot \mathbf{n} g \Phi^+ \, ds \right]. \end{aligned} \quad (9)$$

The semidiscrete dG method we consider to approximate (4) is to seek $U \in C^1([0, T]; \mathbb{V})$ such that

$$\mathcal{A}(U, \Phi) = l(\Phi) \quad \forall \Phi \in \mathbb{V}. \quad (10)$$

Fully Discrete Scheme Subdivide the time domain $[0, T]$ into a partition of N_T consecutive adjacent subintervals with endpoints denoted $0 = t_0 < t_1 < \dots < t_{N_T} = T$. We denote the n -th timestep as $\tau_n = t_n - t_{n-1}$ and consistently use the shorthand $F^n(\cdot) = F(\cdot, t_n)$ for a time-dependent function or function space.

We discretise (10) using a three-stage Strong-Stability-Preserving Runge-Kutta (SSPRK) scheme [15]. Let $\Phi^n \in \mathbb{R}^M$ denote the vector of basis functions such that $\mathbb{V}^n = \text{span}(\Phi^n)$. The SSPRK method can be realised by defining mass and advection matrices $\mathbf{M}^n$ and $\mathbf{A}^n$ respectively componentwise as

$$\begin{aligned} \mathbf{M}_{i,j}^n &= \int_{\Omega} \Phi_j^n \Phi_i^n \, dx \\ \mathbf{A}_{i,j}^n &= - \sum_{K \in \mathcal{T}} \left[\int_K (\mathbf{v} \cdot \nabla \Phi_j^n) \Phi_i^n \, dx - \int_{\partial K_- / \partial \Omega_-} \mathbf{v} \cdot \mathbf{n} [\Phi_j^n] \Phi_i^{n,+} \, ds \right. \\ &\quad \left. - \int_{\partial K_- \cap \partial \Omega_-} \mathbf{v} \cdot \mathbf{n} \Phi_j^{n,+} \Phi_i^{n,+} \, ds \right] \\ \mathbf{b}_i^n &= \sum_{K \in \mathcal{T}} \int_{\partial K_- \cap \partial \Omega_-} \mathbf{v} \cdot \mathbf{n} \Phi_i^{n,+} g \, ds. \end{aligned} \quad (11)$$

Note that each of the matrices are time-dependent. This is since $\mathbb{V}^n$ changes in time through movement of degrees of freedom.

The SSPRK3 solution $U^{n+1} \in \mathbb{V}^n$ is computed by the following multi-stage iterations for $n = 0, \dots, N-1$:

$$\begin{aligned} \mathbf{M}^n W^1 &= U^n + \tau_n (\mathbf{A}^n U^n - \mathbf{b}^n) \\ \mathbf{M}^n W^2 &= \frac{3}{4} U^n + \frac{1}{4} (W^1 + \tau_n (\mathbf{A}^n W^1 - \mathbf{b}^n)) \\ \mathbf{M}^n U^{n+1} &= \frac{1}{3} U^n + \frac{2}{3} (W^2 + \tau_n (\mathbf{A}^n W^2 - \mathbf{b}^n)), \end{aligned} \quad (12)$$

where $W^{1,2} \in \mathbb{R}^M$ and, by an abuse of notation, we denote U^i as the array of finite element coefficients at time t_i . The $\mathbf{M}^n$ is a block diagonal matrix, thus can be inverted elementwise leading to very little computational overhead, as expected from an explicit timestepping method. Note that the finite element space $\mathbb{V}^n$ does not change from one RK stage to another. See Algorithm 1 for further details.

3 Moving Mesh Methods

Moving mesh strategies are characterised by three major components: the mesh adaptation strategy, the method used to discretize the physical PDE, and the approach used to couple the moving mesh to the evolving solution of the PDE.

The effect of the mesh movement in the time discretization of the physical PDE can be treated with either the *quasi-Lagrange* approach or the *rezoning* approach [7, 10]. Quasi-Lagrange methods ensure the mesh points move continuously in time and physical time derivatives are transformed into time derivatives along mesh trajectories with an additional convection term [10]. If an explicit time integrator is used for solving the physical PDE, this extra convection term can pose a critical constraint on the choice of the time-step size due to the Courant-Friedrichs-Lewy (CFL) condition. Indeed such methods are often unstable and very stiff [16].

Rezoning approaches require that the mesh is updated at each time step by solving a MMPDE. This procedure treats the MMPDE and the physical PDE separately, offering flexibility for coding each component with any preferred numerical scheme. The physical solution must then be transferred from the old to the new mesh. This step is critical for the success of this approach, as a poor choice can lead to loss of both accuracy and stability. A conservative interpolation procedure [12, 17] is desirable to preserve those properties and ensure that conservation laws are inherited by the numerical scheme.

Given $U^n \in \mathbb{V}^n$, the solution of (12) at the time t_n , Algorithm 1 describes the *adaptive-conservative dG scheme* that will be studied in the numerical experiments of Sect. 5. Note that it inherently calls another algorithm defined by Algorithm 2.

3.1 Winslow's Variational-Based Diffusion Method

In this section, we briefly review Winslow's adaptive diffusion method for generating the new adapted mesh $\mathcal{T}$, interpreted as the image from a computational domain Ω_c , having a fixed mesh $\mathcal{T}_c$, to a physical domain Ω , with variable mesh $\mathcal{T}_i$. With a variational method, this mapping is determined as the minimizer of an adaptation functional [18, 19].

Algorithm 1 Adaptive-conservative dG scheme**Require:** $u_0, \mathcal{T}^0, T$ **Ensure:** U^{N_T}

- 1: Set $n := 0, t_0 := 0$
- 2: Let $U^0 = \Pi u_0$ be the L^2 projection of the initial data
- 3: Adapt the mesh $\mathcal{T}^0$ by solving the MMPDE (Algorithm 2)
- 4: **while** $t_n \leq T$ **do**:
- 5: Step forward the SSPRK-dG scheme (12) to compute $U^{n+1} \in \mathbb{V}^n$
- 6: Adapt the mesh $\mathcal{T}^{n+1}$ by solving the MMPDE (Algorithm 2)
- 7: Construct an appropriate projector $\Pi : \mathbb{V}^n \rightarrow \mathbb{V}^{n+1}$ and let $\widehat{U}^{n+1} = \Pi U^n$
- 8: Update the timestep τ_{n+1} to ensure the CFL condition is satisfied
- 9: Set $t_{n+1} = t_n + \tau_{n+1}, U^n := \widehat{U}^{n+1}, n += 1$
- 10: **end while**

The MMPDE associated with the minimizer will be solved by the Euler-Lagrange equation, that will be discretized in space and time in the following subsections. Further details on MMPDE based methods can be found in [10].

3.2 Derivation of the MMPDE

A moving mesh method is defined by the smooth mapping $\mathbf{x} = \mathbf{x}(\xi) : \Omega_c \rightarrow \Omega$ or the inverse coordinate transformation $\xi = \xi(\mathbf{x}) : \Omega \rightarrow \Omega_c$. Practically, most of the MMPDEs were developed in terms of the inverse coordinate transformation, as this guarantees existence and uniqueness of the solution [13, 20]. Thus, we will first solve the MMPDE associated with $\xi(\mathbf{x})$ and then determine the transformed coordinate $\mathbf{x}(\xi)$ by change of variable.

The functional for $\xi = \xi(\mathbf{x})$ has the form

$$I[\xi] = \int_{\Omega} F(\nabla \xi, \xi, \mathbf{x}) \, d\mathbf{x}, \quad (13)$$

where the adaptation functional F accounts for the concept of *equidistribution* and *alignment*.

The first condition controls the size of each element of the mesh through a positive monitor function $w(\mathbf{x})$ such that

$$\int_K w(\mathbf{x}) \, d\mathbf{x} = \frac{\sigma}{M} \quad \forall K \in \mathcal{T}, \quad (14)$$

where M is the number of the elements of $\mathcal{T}$ and $\sigma = \int_{\Omega} w(\mathbf{x}) \, d\mathbf{x}$. The equidistribution condition uniquely defines the transformation in one dimension. A further *alignment* condition must be imposed for higher dimensions. This specifies a preferred direction for the edges of $\mathcal{T}$ to follow. Specifically, letting

$\gamma_1, \dots, \gamma_{d(d+1)/2}$ be the edges of an element $K \subset \mathbb{R}^d$, the condition requires that

$$|\gamma_1|_M = \dots = |\gamma_{d(d+1)/2}|_M, \quad (15)$$

where $|\gamma_i|_M$ is the length of γ_i in the metric M . In the case of Winslow's adaptation method, the metric M does not enforce any preferred direction and can be written as $M = w(\mathbf{x})I_d$, where I_d is the identity matrix in $\mathbb{R}^{d \times d}$.

Setting the first variation of (13) to zero leads to the Euler-Lagrange equation. After rearranging some terms, the equation can be written in terms of the elliptic PDE

$$\sum_{i,j} A_{i,j} \frac{\partial^2 \mathbf{x}}{\partial \xi_i \partial \xi_j} + \sum_i B_i \frac{\partial \mathbf{x}}{\partial \xi_i} = 0, \quad (16)$$

where A, B are matrices involving the quantities F and the jacobian matrix $\mathbf{J} = \partial \mathbf{x} / \partial \boldsymbol{\xi}$ with determinant J .

In the case $d = 2$, the computational domain is referenced by the coordinates $\boldsymbol{\xi} = (\xi, \eta)$, while the physical domain is expressed in terms of the continuous coordinates $\mathbf{x} = (x, y)$. The matrices A, B are defined as

$$\begin{aligned} A_{1,1} &= \frac{1}{J^2 w} (x_\eta^2 + y_\eta^2) I_2, \quad A_{1,2} = -\frac{1}{J^2 w} (x_\xi x_\eta + y_\xi y_\eta) I_2 = A_{2,1}, \quad A_{2,2} = \frac{1}{J^2 w} (x_\xi^2 + y_\xi^2) I_2, \\ B_1 &= \frac{1}{J^2 w^2} \left[(x_\eta^2 + y_\eta^2) w_\xi - (x_\xi x_\eta + y_\xi y_\eta) w_\eta \right], \\ B_2 &= \frac{1}{J^2 w^2} \left[-(x_\xi x_\eta + y_\xi y_\eta) w_\xi + (x_\xi^2 + y_\xi^2) w_\eta \right]. \end{aligned}$$

The solution to (16) can be computed efficiently by solving the gradient flow equation for the adaptation functional (13). This transforms (16) into the parabolic PDE

$$\partial_s \mathbf{x}(\boldsymbol{\xi}, s) = \frac{1}{b(\mathbf{x}, s)} \left[\sum_{i,j} A_{i,j} \frac{\partial^2 \mathbf{x}}{\partial \xi_i \partial \xi_j} + \sum_i B_i \frac{\partial \mathbf{x}}{\partial \xi_i} \right], \quad (17)$$

where we are using s to denote the temporal variable, not to be confused with t , used in the moving mesh algorithm. The balancing function $b(\mathbf{x}, s)$ is chosen such that all the mesh points move with a uniform time scale. We will always set $b(\mathbf{x}, s) \equiv 1$ and drop it in future discussion.

3.2.1 Winslow's Adaptation Functional

The adaptation functional that we use was introduced by Winslow [13]. The *Winslow' functional* takes the form

$$I_{Win}[\xi] = \frac{1}{2} \int_{\Omega} \frac{1}{w(\mathbf{x})} \sum_i (\nabla \xi_i)^T (\nabla \xi_i) d\mathbf{x}, \quad (18)$$

where $w(\mathbf{x}) > 0$ is a given monitor function. This function can be chosen to depend on the solution $u(\mathbf{x})$ of the physical PDE. A natural choice is given by the arc-length monitor function

$$w = \sqrt{1 + \frac{1}{\delta} \|\nabla u\|_{L^2}^2}, \quad (19)$$

where δ is a user-specified intensity parameter for $\mathcal{T}$ [7, 10], as this places more points in regions of high gradient. A more in-depth analysis on the choice of the monitor function can be found in Sect. 3.4. The MMPDE associated with the functional (18) becomes

$$\partial_s \mathbf{x}(\xi, s) = - \sum_{i=1} \mathbf{a}_i \nabla \cdot (\mathbf{M}^{-1} \mathbf{a}^i), \quad (20)$$

where $\mathbf{a}^i = \nabla \xi_i$, $\mathbf{a}_i = \partial \mathbf{x} / \partial \xi_i$, and $\mathbf{M} = w(\mathbf{x}) I_2$ is the metric function. The replacement of the previous terms in the two-dimensional space gives

$$\begin{aligned} \partial_s \mathbf{x}(\xi, s) = \frac{1}{J^2 w^2} \left[(x_\eta^2 + y_\eta^2) \frac{\partial}{\partial \xi} \left(w \frac{\partial \mathbf{x}}{\partial \xi} \right) - (x_\xi x_\eta + y_\xi y_\eta) \frac{\partial}{\partial \eta} \left(w \frac{\partial \mathbf{x}}{\partial \xi} \right) \right. \\ \left. - (x_\xi x_\eta + y_\xi y_\eta) \frac{\partial}{\partial \xi} \left(w \frac{\partial \mathbf{x}}{\partial \eta} \right) + (x_\xi^2 + y_\xi^2) \frac{\partial}{\partial \eta} \left(w \frac{\partial \mathbf{x}}{\partial \eta} \right) \right]. \end{aligned} \quad (21)$$

This PDE allows us to compute $\mathbf{x}(\xi, s_{i+1})$ and hence $\mathcal{T}_{i+1}$ given the knowledge of $w(\mathbf{x})$ and $\mathcal{T}_c$.

3.3 Numerical Solution of Winslow's MMPDE

In this section, we illustrate the numerical procedure to solve numerically the MMPDE (21) by using a finite element discretisation. After obtaining the weak formulation of the MMPDE, we discretize the equation in space and time using finite element space comprised of linear elements and a semi-implicit Euler integrator. The computational domain is still referred by the coordinates $\xi = (\xi, \eta)$, while the continuous coordinates of Ω are expressed by $\mathbf{x} = (x, y)$.

3.3.1 Weak Formulation

We begin by splitting the boundary of Ω_c into two components, Γ_D and Γ_N . The weak formulation of Winslow' MMPDE is obtained by multiplying both sides of (21) by an appropriate test function and integrate by parts over Ω_c . We seek $\mathbf{x} \in H_{0,\Gamma_D}^1(\Omega_c)^2$

$$\begin{aligned} \int_{\Omega_c} A(\mathbf{x}) \partial_s \mathbf{x} \Phi \, d\xi &= \int_{\Omega_c} w(\mathbf{x}) \left\{ -\frac{\partial}{\partial \xi} (\alpha_1(\mathbf{x}) \Phi) \frac{\partial \mathbf{x}}{\partial \xi} + \left[\frac{\partial}{\partial \eta} (\alpha_2(\mathbf{x}) \Phi) \frac{\partial \mathbf{x}}{\partial \xi} + \right. \right. \\ &\quad \left. \left. \frac{\partial}{\partial \xi} (\alpha_2(\mathbf{x}) \Phi) \frac{\partial \mathbf{x}}{\partial \eta} \right] - \frac{\partial}{\partial \eta} (\alpha_3(\mathbf{x}) \Phi) \frac{\partial \mathbf{x}}{\partial \eta} \right\} d\xi \quad \forall \Phi \in H_{0,\Gamma_D}^1(\Omega_c)^2, \end{aligned} \quad (22)$$

where $\alpha_1(\mathbf{x}) = x_\eta^2 + y_\eta^2$, $\alpha_2(\mathbf{x}) = x_\xi x_\eta + y_\xi y_\eta$, $\alpha_3(\mathbf{x}) = x_\xi^2 + y_\xi^2$ and $A(\mathbf{x}) = J^2 w^2(\mathbf{x})$.

Note that the weak form (22) requires the solution $\mathbf{x}(\xi, t)$ to be twice weakly differentiable. This condition is delicate to interpret in a primal form, so we circumvent the difficulties by computing $\alpha_{1,2,3}$ in the following fashion:

$$\begin{aligned} \tilde{\alpha}_1(\mathbf{x}) &= \Pi(x_\eta^2 + y_\eta^2) \\ \tilde{\alpha}_2(\mathbf{x}) &= \Pi(x_\xi x_\eta + y_\xi y_\eta) \\ \tilde{\alpha}_3(\mathbf{x}) &= \Pi(x_\xi^2 + y_\xi^2). \end{aligned} \quad (23)$$

Time Discretisation The discretisation of (22) in the temporal variable is realised with a *semi-implicit Euler scheme*. To that end, we divide the domain $[0, S]$ into a partition of N_S consecutive subintervals $0 = s_0 < s_1 < \dots < s_{N_S} = S$, with $s_i - s_{i-1} = k$ for all i . We treat the non-linear terms through evaluation at the current time step s_i , while the linear terms are evaluated at the next time step s_{i+1} . We denote $\mathbf{x}_{i+1}$ to be the solution of (22) at s_{i+1} . Specifically, we seek $\mathbf{x}_{i+1} \in H_{0,\Gamma_D}^1(\Omega_c)^2$ such that

$$\begin{aligned} &\int_{\Omega_c} A(\mathbf{x}_i) \left(\frac{\mathbf{x}_{i+1} - \mathbf{x}_i}{k} \right) \Phi \, d\xi \\ &= \int_{\Omega_c} w(\mathbf{x}_i) \left\{ -\frac{\partial}{\partial \xi} (\tilde{\alpha}_1(\mathbf{x}_i) \Phi) \frac{\partial \mathbf{x}_{i+1}}{\partial \xi} + \left[\frac{\partial}{\partial \eta} (\tilde{\alpha}_2(\mathbf{x}_i) \Phi) \frac{\partial \mathbf{x}_{i+1}}{\partial \xi} + \frac{\partial}{\partial \xi} (\tilde{\alpha}_2(\mathbf{x}_i) \Phi) \frac{\partial \mathbf{x}_{i+1}}{\partial \eta} \right] \right. \\ &\quad \left. - \frac{\partial}{\partial \eta} (\tilde{\alpha}_3(\mathbf{x}_i) \Phi) \frac{\partial \mathbf{x}_{i+1}}{\partial \eta} \right\} d\xi \quad \forall \Phi \in H_{0,\Gamma_D}^1(\Omega_c)^2. \end{aligned} \quad (24)$$

Fully Discrete Scheme Let Ω_c be tessellated with a conforming, shape regular mesh $\mathcal{T}_c$ that is fixed in time. We denote by $\Phi \in \mathbb{R}^P$ the vector of basis functions such that $\mathbb{P}^1(\mathcal{T}_c) = \text{span}(\Phi)$. The algebraic system for the numerical solution $\mathbf{X}_i = (X_i, Y_i)$ of (24) can be decoupled and for the first component takes the form

$$(\mathbf{M}_i - k\mathbf{S}_i)\mathbf{X}_{i+1} = \mathbf{M}_i\mathbf{X}_i, \quad (25)$$

where $\mathbf{X}_i$ denotes the array of coefficients at time s_i and the weighted mass and stiffness matrices $\mathbf{M}_i, \mathbf{S}_i$ are given by

$$(\mathbf{M}_i)_{a,b} = \int_{\Omega_c} A(\mathbf{X}_i) \Phi_a(\xi) \Phi_b(\xi) d\xi = \int_{\Omega_c} J(\xi)^2 W(\mathbf{X}_i)^2 \Phi_a(\xi) \Phi_b(\xi) d\xi, \quad (26)$$

$$(\mathbf{S}_i)_{a,b} = \int_{\Omega_c} W(\mathbf{X}_i) \left(-\frac{\partial}{\partial \xi} (\tilde{\alpha}_1(\mathbf{X}_i) \Phi_a) \frac{\partial \Phi_b}{\partial \xi} + \frac{\partial}{\partial \eta} (\tilde{\alpha}_2(\mathbf{X}_i) \Phi_a) \frac{\partial \Phi_b}{\partial \xi} + \right. \\ \left. \frac{\partial}{\partial \xi} (\tilde{\alpha}_2(\mathbf{X}_i) \Phi_a) \frac{\partial \Phi_b}{\partial \eta} - \frac{\partial}{\partial \eta} (\tilde{\alpha}_3(\mathbf{X}_i) \Phi_a) \frac{\partial \Phi_b}{\partial \eta} \right) d\xi. \quad (27)$$

Practically, the matrices (26) and (27) are evaluated by a three point Gaussian quadrature rule and the non-symmetric system (25) is solved by using sparse LU decomposition [21]. For large problems resulting from, for example a high-resolution mesh, iterative methods can be used as faster and less memory-demanding options, e.g. GMRES and MINRES.

3.3.2 Boundary Conditions of the MMPDE

For the sake of clarity, the numerical solution of (22) computes the shift of the new mesh coordinates rather than the coordinates themselves. Homogeneous Dirichlet BCs fix the mesh points over Γ_D . However, this condition generally does not ensure good mesh quality, especially close to the boundary, and a combination of Dirichlet and Neumann BCs result in less skewed meshes [7]. For $\Omega = [0, 1]^2$ we enforce essential Dirichlet conditions on X_i on the east-west sides and natural Neumann conditions on the top-bottom sides. Concerning Y_i , Dirichlet conditions are applied on the top-bottom sides and Neumann conditions on the east-west sides. Importantly, this allows the mesh nodes to move along the boundaries.

3.4 The Monitor Function

To assemble the systems in (25), the discrete monitor function $W(\mathbf{X}_i) \in \mathbb{P}^1(\mathcal{T}_i)$ must be defined over Ω_c . We choose to represent $\widehat{W}_i \in \mathbb{P}^1(\mathcal{T}_c)$. The discrete map $\mathbf{X}_i(\xi)$ allows us to compute the coefficients $\widehat{W}_i$ as

$$\widehat{W}_i = \widehat{W}_i(\xi) = W(\mathbf{X}_i(\xi)) = \sqrt{1 + \frac{1}{\delta} \|G[U_i](\mathbf{X}_i)\|_{L^2}^2}, \quad (28)$$

where the optimal intensity parameter, δ [10], is chosen as

$$\delta = \frac{1}{|\Omega|} \int_{\Omega} \|G[U_i]\|_{L^2} d\mathbf{x}. \quad (29)$$

The term $G[V] \in \mathbb{V} \times \mathbb{V}$ is a *discrete gradient operator* of a generic finite element object $V \in \mathbb{V}$, defined through

$$\int_{\Omega} G[V] \Phi d\mathbf{x} = \int_{\Omega} \nabla V \cdot \Phi d\mathbf{x} - \int_{\mathcal{E}} \llbracket V \rrbracket \{\Phi\} dS \quad \forall \Phi \in \mathbb{V}. \quad (30)$$

The monitor function can be directly defined with respect to the vector U_i and denoted by $\widehat{W}(U_i)$.

A common practice that we utilise in our numerical experiments to reduce the skewness in a mesh is to smooth $\widehat{W}$ by solving a diffusion PDE [22]. This procedure can be realised by considering the equation to find $\widetilde{W} \in \mathbb{P}^1(\mathcal{T}_c)$ such that

$$\int_{\Omega_c} \Phi \widetilde{W} d\xi + \int_{\Omega_c} \beta \nabla \Phi \cdot \nabla \widetilde{W} d\xi = \int_{\Omega_c} \Phi \widehat{W} d\xi \quad \forall \Phi \in \mathbb{P}^1(\mathcal{T}_c), \quad (31)$$

where $\beta \ll 1$ is a diffusion parameter and homogeneous Neumann boundary conditions are applied on $\partial\Omega_c$.

3.5 The Resulting Mesh

The resulting MMPDE scheme provides an adapted mesh $\mathcal{T}_{N_S}$ to the solution U^{n+1} of (12). This represents a subfunction called by Algorithm 1 and is composed by the following procedure:

Algorithm 2 MMPDE

Require: $N_S, \text{TOL}, k, \beta, b$

Ensure: $\mathcal{T}_{N_S}$

- 1: Set $i := 0$ and $\varepsilon := 2\text{TOL}$
 - 2: **while** $i < N_S$ or $\varepsilon > \text{TOL}$ **do**:
 - 3: Solve the fully discretised MMPDE (25) to obtain $\mathbf{X}_{i+1}$ and generate the mesh $\mathcal{T}_{i+1}$
 - 4: Compute $\widehat{W}(U^{n+1})$ using (28)
 - 5: Smooth $\widehat{W}(U^{n+1})$ to obtain $\widetilde{W}(U^{n+1})$ by (31)
 - 6: Set $\varepsilon = \|\mathbf{X}_{i+1} - \mathbf{X}_i\|_{L^2(\Omega_c)} / \|\mathbf{X}_i\|_{L^2(\Omega_c)}$
 - 7: Set $\mathbf{X}_i := \mathbf{X}_{i+1}$ and $i += 1$
 - 8: **end while**
-

The errors in the solution of (25) are critically dependent on the nature on the resulting mesh. Essentially, the error is a combination of the *skewness*, or lack of

shape regularity, and the *scale* of the mesh. The skewness indicates how far mesh elements are from being equilateral, while the scale checks how well the mesh is adapted to the solution. For triangular elements, the skewness can be computed as the ratio between the minimum and maximum vertex angle. A more general indicator has been proposed in [7] and employs the local map $Q_K : \hat{K} \rightarrow K$, with $\hat{K} \in \mathcal{T}_c$ and $K \in \mathcal{T}$. Given the corresponding Jacobian $\mathbf{J}_K$ with eigenvalues λ_1 and λ_2 , the quality measure for the element K is defined as

$$Q_K = \frac{1}{2} \left(\frac{\lambda_1}{\lambda_2} + \frac{\lambda_2}{\lambda_1} \right). \quad (32)$$

We define the *global quality measure* as

$$Q = \max_{K \in \mathcal{T}} Q_K \quad (33)$$

and analyse a variety of adaptive meshes in the numerical experiments of Sect. 5.

4 Data Transfer Over Timesteps

Data transfer of the field U^{n+1} from the mesh $\mathcal{T}^n$ to the new adapted mesh $\mathcal{T}^{n+1} := \mathcal{T}_{N_S}$ is an important step for the accuracy and stability of the solution of a physical PDE. A Lagrange interpolator in this role suffers from several drawbacks. In particular, the integral of the interpolated function is not conserved, monotonicity is preserved only for linear interpolation, and is not suitable for discontinuous fields.

The *local Galerkin projection* does not suffer from the same drawbacks and we will use this to represent the scalar field onto the new mesh [12]. This process involves the construction of a conforming triangulation that captures the features of $\mathcal{T}^{n+1}$ and $\mathcal{T}^n$. In the literature this is referred to as a *supermesh*.

4.1 Local Galerkin Projection

Consider the projection of the scalar field $U^{n+1} \in \mathbb{V}^n$ to $\mathbb{V}^{n+1}$. Let $\mathbb{V}^n = \text{span}(\Phi^n)$ and $\mathbb{V}^{n+1} = \text{span}(\Phi^{n+1})$. We make use of an L^2 projection; to that end we seek $\hat{U} \in \mathbb{V}^{n+1}$ such that

$$\int_{\Omega} \hat{U} \Phi_i^{n+1} \, dx = \int_{\Omega} U^{n+1} \Phi_i^{n+1} \, dx \quad \forall i \in \{1, \dots, M\}, \quad (34)$$

or algebraically, if we abuse notation slightly and let $U^{n+1}, \hat{U}$ denote the array of the finite element coefficients and

$$(\mathbf{M}_1)_{i,j} = \int_{\Omega} \Phi_i^{n+1} \Phi_j^{n+1} \, d\mathbf{x}, \quad (35)$$

$$(\mathbf{M}_2)_{i,j} = \int_{\Omega} \Phi_i^{n+1} \Phi_j^n \, d\mathbf{x} \quad \forall i, j \in \{1, \dots, M\}. \quad (36)$$

then $\widehat{U}$ solves

$$\mathbf{M}_1 \widehat{U} = \mathbf{M}_2 U^{n+1}. \quad (37)$$

Note that the computation of $\widehat{U}$ is not trivial as it involves the computation of products between basis functions of $\mathbb{V}^n$ and $\mathbb{V}^{n+1}$. Over each element $\hat{K} \in \mathcal{T}^{n+1}$, the basis functions of $\mathbb{V}^n$ are in general discontinuous piecewise polynomials. Since the integrals over $\hat{K}$ are evaluated exactly at the quadrature points only for polynomials, each entry of $\mathbf{M}_2$ must be computed at the region of intersection between $\hat{K}$ and $K \in \mathcal{T}^n$. The collection of those meshed regions defines a conforming triangulation called *supermesh*.

For a given element $\hat{K} \in \mathcal{T}^{n+1}$, we must ensure all possible intersecting elements in $\mathcal{T}^n$ are found efficiently. The greedy algorithm performs $O(|\mathcal{T}^{n+1}| |\mathcal{T}^n|)$ intersection tests. These can be first filtered by an Axis Aligned Bounding Box tree algorithm [23], which only consider pairs with intersecting bounding boxes. The construction takes $O(|\mathcal{T}^n| \log |\mathcal{T}^n|)$, and the query for each element takes $O(\log |\mathcal{T}^n|)$. This is the strategy we utilise.

The vertices of the intersecting polygon can be computed with the *Sutherland-Hodgman clipping algorithm* [24]. The meshing procedure is performed connecting with an edge a generic vertex with all the other vertices except the adjacent ones. If $K \cap \hat{K}$ is formed by n vertices, with $4 \leq n \leq 6$, the intersecting region can be decomposed in $n - 2$ triangles. For each triangle K the integral $\int_{K \cap \hat{K}} \Phi_i^{n+1} \Phi_j^n \, d\mathbf{x}$ can be computed exactly using the Gaussian quadrature rule. An illustration of the procedure is given in Fig. 1. Note that, for the lowest order dG space, only the area $|K \cap \hat{K}|$, which is computed with the *shoelace method* [25], is needed to assemble $\mathbf{M}_2$.

5 Numerical Experiments

In this section we test the methods described in Sects 2–4. The implementation has been carried out using the DOLFIN interface of FEniCS [26]. The software Paraview has been used as visualization tool. For each test, we used a Gaussian quadrature rule of order 6 and took $k = 0$ in $\mathbb{V}$ for simplicity.

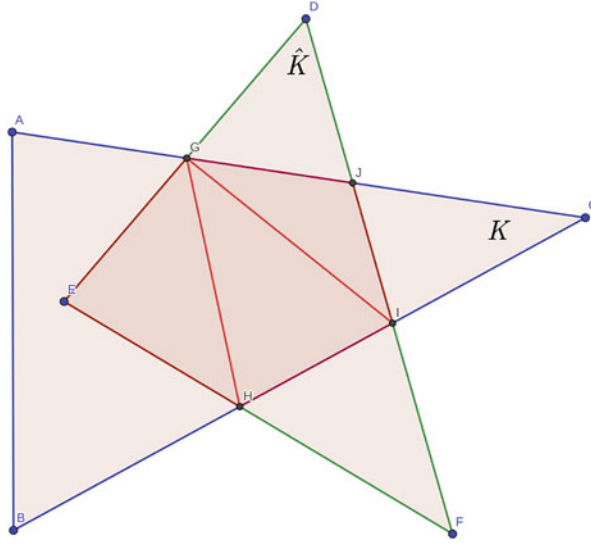


Fig. 1 An illustration of the intersection between two triangles. The resulting polygon is subdivided into a shape regular triangulation which represents the supermesh over the intersection

5.1 Test 1: Mesh Adaptation to a Scalar Function

In this first experiment, we test the effectivity of Algorithm 2 to construct meshes that resolve certain prescribed functions with differing properties and regularity. With that in mind, let $\mathcal{T}_i$ denote the sequence of meshes generated by Algorithm 2 from an initial uniform criss-cross mesh. Let $\Pi_i u \in \mathbb{V}_i := \mathbb{V}(\mathcal{T}_i)$ be the L^2 projection of a function u .

Consider the following three functions:

$$u(x, y) = \exp \left(-50 \left((x - 0.5)^2 + (y - 0.5)^2 \right) \right), \quad (38)$$

$$u(x, y) = 3 - \tanh \left(30 \left((x - 0.3) + (y - 0.2) \right) \right), \quad (39)$$

$$u(x, y) = \begin{cases} 3.0 & \text{if } y + 0.8(x - 0.2) \leq 0, \\ 1.0 & \text{otherwise.} \end{cases} \quad (40)$$

In Fig. 2a we show the behaviour of $\|u - \Pi_i u\|_{L^2(\Omega)}$ and in Fig. 2b the quality measure Q for each of the meshes produced for the functions (38), (39) and (40) using Algorithm 2.

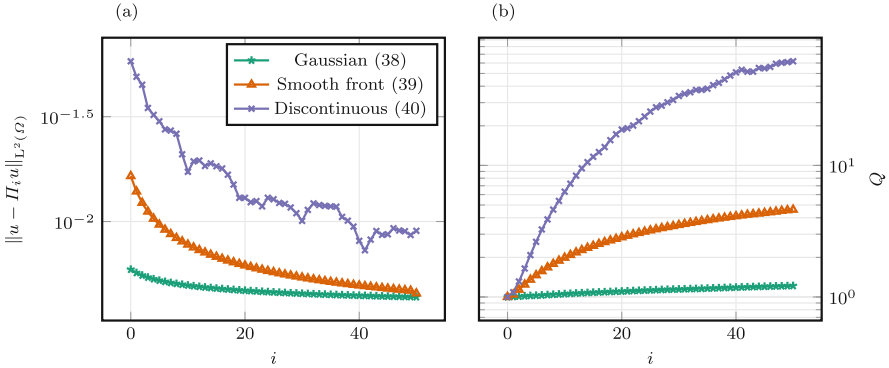


Fig. 2 Test 1—Here we show the projection error and the mesh quality measure Q for (38), (39) and (40) using Algorithm 2 for 50 iterations. The initial criss-cross computational mesh is fixed and composed of 10^4 elements, whereas the physical mesh is then iteratively adapted. In this experiment we took the time step $k = 10^{-3}$ (25) and the smoothing parameter $\beta = 7 \times 10^{-4}$ (31). Note that the discontinuous function is approximated almost as well as the other functions despite the discontinuity not being aligned with the mesh. To compensate, as expected, the mesh quality measure increases dramatically. This results in quite anisotropic elements. (a) The projection error as a function of iterations of the discretisation of the MMPDE. (b) The mesh quality as a function of iterations of the discretisation of the MMPDE

5.2 Test 2: Convergence on the Adaptively Generated Grids

In this test, we examine the approximability of the finite element spaces over the meshes generated by the moving mesh procedure in Algorithm 2. We consider a family of uniform criss-cross computational meshes, each one a uniform refinement of the previous mesh. For each of these initial meshes, we use Algorithm 2 to generate a mesh $\mathcal{T}_{N_S}$, which is able to resolve the features of the functions (38)–(40). For each realisation of $\mathcal{T}_{N_S}$, we let Πu denote the L^2 projection onto the finite element space associated to this mesh and we test the asymptotic convergence rates, measured in terms of the degrees of freedom.

Figure 3 shows the functions together with a visualisation of the result of applying Algorithm 2. Figure 4 shows the convergence of the best approximation over the uniform and warped meshes. Notice that the functions are better approximated over the warped meshes, but also that the discontinuous solution is approximated optimally despite the lack of regularity. This is consistent with other studies of irregular functions on warped meshes [27].

5.2.1 Adaptive-Conservative dG Transport Equation with Upwinding

This last set of experiments detail the validation of the fully coupled adaptive conservative moving mesh dG scheme for the numerical solution of the transport

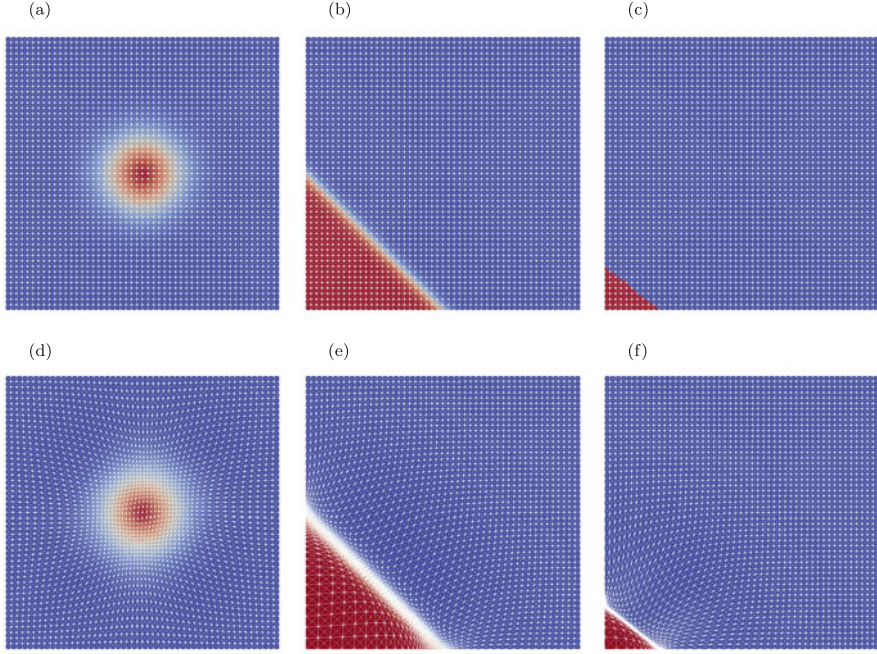


Fig. 3 Test 1—Visualisations of the functions (38)–(40) discretised in $\mathbb{V}_i$ at iteration $i = 0$ and $i = 50$ on the domain $\Omega = [0, 1]^2$. The physical mesh $\mathcal{T}$ is comprised by 10^4 triangular elements and for $i = 0$ equal to the criss-cross computational mesh. Note that the mesh distorts locally to ensure good approximation of the hard-to-approximate features. (a) A Gaussian function on the initial criss cross mesh, $i = 0$. (b) A smooth front on the initial criss cross mesh, $i = 0$. (c) A discontinuous front on the initial criss cross mesh, $i = 0$. (d) A Gaussian function on the final mesh, $i = 50$. (e) A smooth front on the final mesh, $i = 50$. (f) A discontinuous front on the final mesh, $i = 50$

equation (4) via Algorithm 1. It employs both the MMPDE (Algorithm 2) and the conservative Galerkin projection as a data transfer operator.

5.2.2 Test Problems

We prescribe two different initial conditions

$$u_0(\mathbf{x}) = 3 - \tanh(30((x - 0.2) + (y - 0.2))), \quad (41)$$

$$u_0(\mathbf{x}) = \begin{cases} 3.0 & \text{if } y + 0.8(x - 0.2) \leq 0, \\ 1.0 & \text{otherwise,} \end{cases} \quad (42)$$

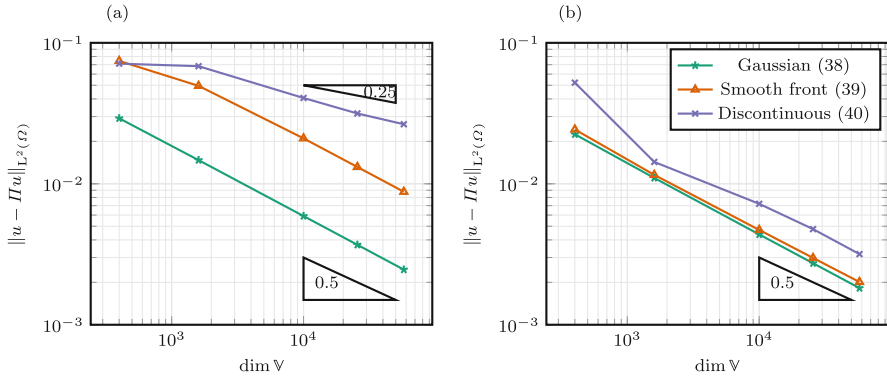


Fig. 4 Test 2—Asymptotic convergence rates for the best approximation on the finite element space over the meshes generated by Algorithm 2. Note that for uniform meshes the approximation of the discontinuous function is slower due to the lack of regularity. The adapted meshes are able to resolve this to ensure optimal rates for the piecewise constant approximations [27]. (a) A family of uniform meshes. (b) A family of adaptive meshes

for $\Omega := [0, 1]^2$ and set a constant velocity field

$$\mathbf{v} = \left(\frac{1}{2}, \frac{\sqrt{2}}{2} \right), \quad (43)$$

as not to align with the mesh. We impose an inflow boundary condition consistent with the exact solution

$$u(\mathbf{x}, t) = u_0(\mathbf{x} - \mathbf{v}t) \quad (44)$$

over $\partial\Omega_-$, which corresponds to the west and south sides of Ω . The solutions we consider are a smooth travelling front and a discontinuous travelling front.

5.3 Test : Error Accumulation in Time

In this test, for the smooth front initial condition given by (39), we compare the performance of the proposed adaptive scheme with and without the conservative data transfer projection to that one of the uniform scheme. As such, we are able to show the importance of the conservative data transfer operator for the success of the scheme. To that end, we take an initial uniform triangulation formed of 10^4 criss-cross elements. The mesh is then adapted to the initial condition using Algorithm 2. The transport equation is then solved using the three-stage SSPRK-dG scheme (12).

The results are shown in Fig. 5, where it is apparent that the L^2 error over time grows at a smaller rate for the conservative adaptive scheme than its uniform and nonconservative counterparts.

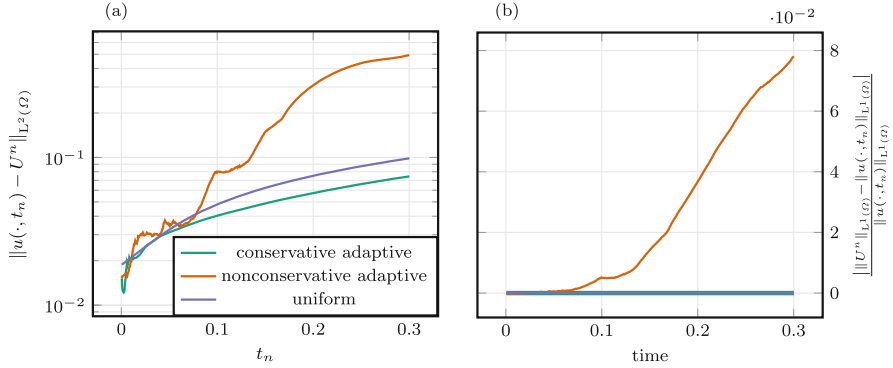


Fig. 5 Test 3—Here we show the error accumulation in time for the smooth initial condition (41). The parameters used for the mesh adaptation are: $N_S = 15$, $k = 10^{-3}$ and $\beta = 10^{-5}$. In each case the initial mesh was chosen such that $\dim \mathbb{V} = 10^4$. The uniform scheme outperforms the nonconservative adaptive one as the error introduced by Lagrange interpolation from $\mathcal{T}^n$ to $\mathcal{T}^{n+1}$ is not counterbalanced by the gain in accuracy on the adapted mesh. (a) Error accumulation in time for the uniform, adaptive nonconservative and adaptive conservative schemes. (b) Relative mass error for the uniform, adaptive nonconservative and adaptive conservative schemes

5.4 Test 4: Sensitivity of the MMPDE Algorithm

In this test, we explore the sensitivity of the MMPDE described in Algorithm 2. We fix the initial triangulation to be of criss-cross type with 10^4 elements. We fix the parameters $k = 10^{-3}$, $\beta = 10^{-4}$ and vary the parameter S within Algorithm 2.

Figure 6 shows results by varying the parameter S for both initial conditions. The SSPRK-dG scheme is kept the same. The key point is that the MMPDE must be solved in the time interval $[0, S]$ for S large enough to ensure that (17) is close enough to reach a steady state.

5.5 Test 5: Asymptotic Convergence Rates

In this test, we compare the adaptive-conservative scheme to an equivalent SSPRK-dG scheme running on the initial uniform mesh. In both cases, the time step has been chosen adaptively to ensure a CFL condition is satisfied. For the uniform scheme, we take a family of consecutively refined initial meshes where $\dim \mathbb{V} \in [100, 6400]$ and run the SSPRK-dG scheme (12). For the adaptive scheme, we take the same initial meshes; however this time apply Algorithm 2. In both cases we track the L^2 error evaluated as a function of time as well as the time step.

We test two exact solutions, a smooth moving front and a discontinuous moving front. The results are shown in Figs. 7 and 8, where we are able to see an improved

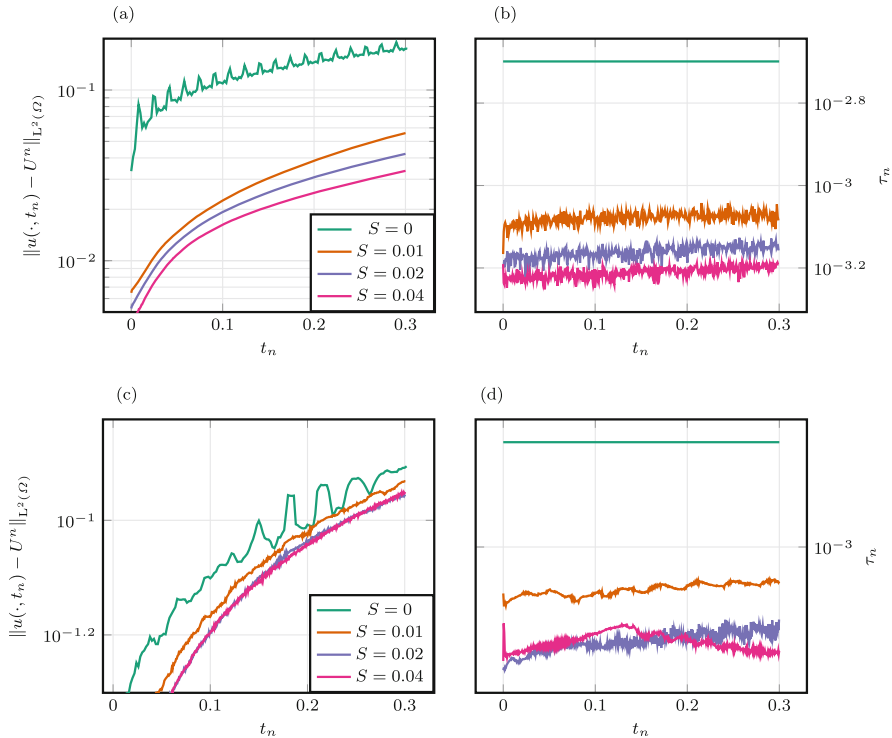


Fig. 6 Test 4—Here we show the effect of varying S , the end time of the moving mesh algorithm, within Algorithm 2. We fix $k = 10^{-3}$ and $\beta = 10^{-4}$. We examine the effect of modifying S for the solutions given by (44) with initial conditions (41) and (42). We show that the error stagnates as S gets large, meaning that the MMPDE is close to a steady state for that particular S . (a) The L^2 error as a function of time for the adaptive approximation of (44) with smooth initial conditions (41). (b) The timestep as a function of time for the adaptive approximation of (44) with smooth initial conditions (41). (c) The L^2 error as a function of time for the adaptive approximation of (44) with discontinuous initial conditions (42). (d) The timestep as a function of time for the adaptive approximation of (44) with discontinuous initial conditions (42)

convergence rate for the adaptive scheme applied to both test problems over the uniform one.

6 Conclusions

In this paper, we have proposed and studied an r -adaptive algorithm for the solution of a linear hyperbolic PDE. The moving mesh method is able to adapt the mesh to increase the spatial accuracy of the solution in the L^2 norm. We achieved this by making use of a conservative Galerkin projection as a data transfer operator via the construction of a *supermesh*.

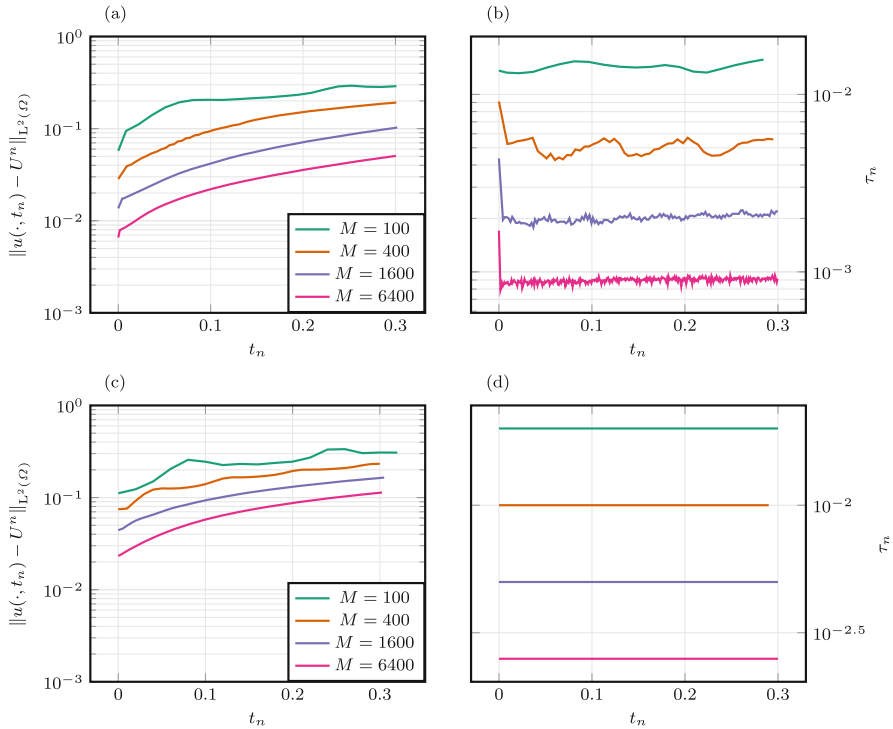


Fig. 7 Test 5—A comparison of the adaptive scheme given by Algorithm 1 and an equivalent uniform scheme, both initialised with the same uniform meshes. The family of uniformly refined initial grids ensures $\dim \mathbb{V} \in [100, 6400]$. We are examining the approximation of the exact solution (44) with the smooth initial condition (41). Notice that the adaptive scheme is able to achieve an approximation with smaller L^2 error than the uniform scheme as well as a higher rate of convergence. (a) The L^2 error as a function of time for the adaptive approximation. (b) The time step as a function of time for the adaptive approximation. (c) The L^2 error as a function of time for the uniform approximation. (d) The time step as a function of time for the uniform approximation

While the scheme itself shows good error reduction properties, it has a number of shortcomings. In particular, for each time step the solution of a quasilinear parabolic PDE is required to compute the solution to the Winslow' mesh adaptation step. This dramatically increases computational cost over, say an h - or p -adaptive method, to achieve the same error tolerance. That being said, the method has the potential for massive parallelisation since at any time step, the mesh is just an affine transformation of a uniform grid.

Future research directions on this topic will examine:

1. The reduction in computational time of the 'inner loop' moving mesh strategy to achieve comparable accuracy;
2. Scalability of the algorithm through parallelisation;

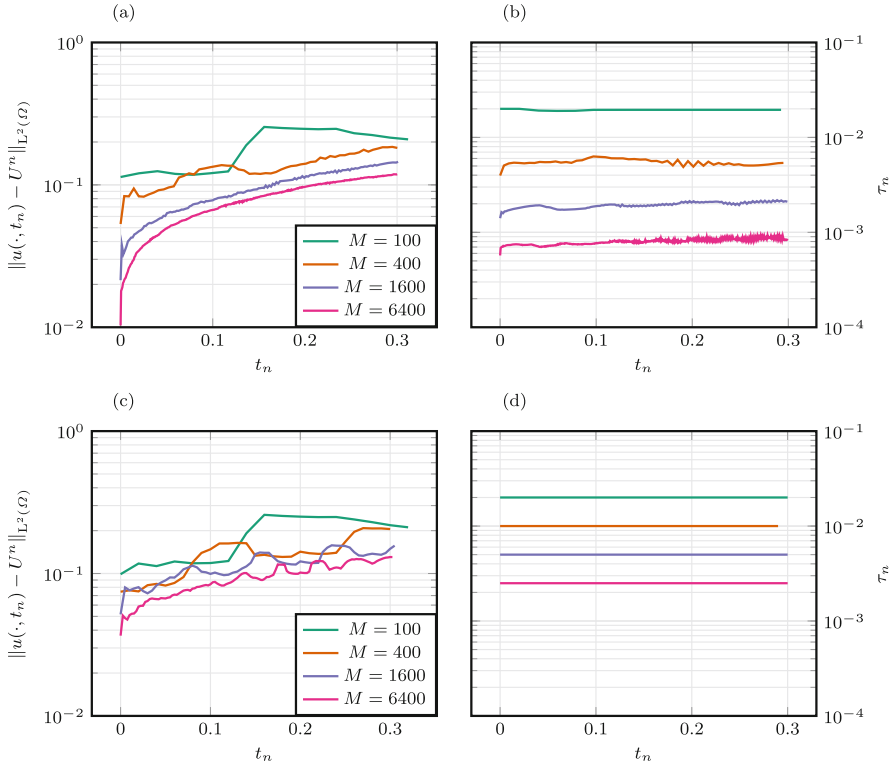


Fig. 8 Test 5—A comparison of the adaptive scheme given by Algorithm 1 and an equivalent uniform scheme, both initialised with the same uniform meshes. The family of uniformly refined initial grids ensures $\dim \mathbb{V} \in [100, 6400]$. We are examining the approximation of the exact solution (44) with the discontinuous initial condition (42). Notice that the adaptive scheme is marginally more accurate than the uniform scheme. (a) The L^2 error as a function of time for the adaptive approximation. (b) The time step as a function of time for the adaptive approximation. (c) The L^2 error as a function of time for the uniform approximation. (d) The time step as a function of time for the uniform approximation

3. The applicability of the method to more physical relevant problems, such as the shallow water equations on the sphere [28]. These require tailored methods with good conservative properties for long time simulations. In particular, for global atmospheric flows, the ability of massive parallelisation is crucial.

Acknowledgments This research has been funded in part by the EPSRC through the CDT program in Statistical and Applied Mathematics at the University of Bath (EPSRC Reference: EP/L015684/1). We thank the MET Office staff and the reviewer of this manuscript for their valuable advice and comments.

References

1. Shepherd, T.G.: Symmetries, conservation laws, and Hamiltonian structure in geophysical fluid dynamics. In: *Advances in Geophysics*, vol. 32, pp. 287–338. Elsevier, Amsterdam (1990)
2. Roulstone, I., Norbury, J.: A Hamiltonian structure with contact geometry for the semi-geostrophic equations. *J. Fluid Mech.* **272**, 211–234 (1994)
3. Müller, P., Garrett, C., Osborne, A.: Rogue waves. *Oceanography* **18**(3), 66 (2005)
4. Plewa, T., Linde, T., Weirs, V.G.: Adaptive Mesh Refinement—Theory and Applications. In: *Proceedings of the Chicago Workshop on Adaptive Mesh Refinement Methods*, Sept. 3–5, 2003. *Lecture Notes in Computational Science and Engineering*. Springer, Berlin (2005)
5. Ainsworth, M., Senior, B.: An adaptive refinement strategy for hp-finite element computations. *Appl. Numer. Math.* **26**(1), 165–178 (1998)
6. Eiseinan, P.R.: Adaptive grid generation. *Comput. Methods Appl. Mech. Eng.* **64**(1), 321–376 (1987)
7. Budd, C., Huang, W., Russell, R.: Adaptivity with moving grids. *Acta Numer.* **18**, 111–241 (2009)
8. Budd, C., Huang, W., Russell, R.: Moving mesh methods for problems with blow-up. *SIAM J. Sci. Comput.* **17**, 305–327 (1996)
9. Hertel, C., et al.: Using a moving mesh PDE for cell centres to adapt a finite volume grid. *Flow Turbul. Combust.* **90**(4), 785–812 (2013)
10. Huang, W., Russell, R.: *Adaptive Moving Mesh Methods*, vol. 174 (2011)
11. Tang, T.: Moving mesh methods for computational fluid dynamics. *Contemp. Math.* **383**(8), 141–173 (2005)
12. Farrell, P., Maddison, J.: Conservative interpolation between volume meshes by local Galerkin projection. *Comput. Methods Appl. Mech. Eng.* **200**, 89–100 (2010)
13. Winslow, A.M.: Numerical solution of the quasilinear poisson equation in a nonuniform triangle mesh. *J. Comput. Phys.* **1**(2), 149–172 (1966)
14. Cangiani, A., Georgoulis, E., Houston, P.: hp-version discontinuous Galerkin methods on polygonal and polyhedral meshes. *Math. Mod. Methods Appl. Sci.* **24**(10), 2009–2041 (2014)
15. Gottlieb, S., Grant, Z.J., Isherwood, L.: Strong stability preserving integrating factor Runge-Kutta methods. *SIAM J. Numer. Anal.* **56**(6), 3276–3307 (2018)
16. Shengtai, L., Linda, R., Petzold, Yuhe, R.: Stability of moving mesh systems of partial differential equations. *SIAM J. Sci. Comput.* **20**, 719–738 (1998)
17. Zhang, Z.: Moving mesh method with conservative interpolation based on L^2 -projection. *Commun. Comput. Phys.* **1**, 930–944 (2006)
18. Huang, W.: A comparative numerical study of meshing functionals for variational mesh adaptation. *J. Math. Study* **48**(2), 168–186 (2015)
19. Huang, W., Sun, W.: Variational mesh adaptation II: error estimates and monitor functions. *J. Comput. Phys.* **184**(2), 619–648 (2003)
20. Dvinsky, A.S.: Adaptive grid generation from harmonic maps on Riemannian manifolds. *J. Comput. Phys.* **95**(2), 450–476 (1991)
21. Bunch, J.R., Hopcroft, J.E.: Triangular factorization and inversion by fast matrix multiplication. *Math. Comput.* **28**, 231–236 (1974)
22. Huang, W., Kamenski, L.: On the mesh nonsingularity of the moving mesh PDE method. *Math. Comput.* **87**, 1887–911 (2015)
23. Hart, K.: ABBTree (2016). <https://github.com/kip-hart/ABBTree>
24. Sutherland, I.E., Hodgman, G.W.: Reentrant polygon clipping. *Commun. ACM* **17**(1), 32–42 (1974)
25. Meister, A.L.F.: *Generalia de genesi figurarum planarum et inde pendentibus earum affectionibus* (1769)
26. Logg, A., Wells, G.N.: DOLFIN. *ACM Trans. Math. Softw.* **37**(2), 1–28 (2010)
27. Georgoulis, E., Makridakis, C., Pryer, T.: Bab̆uska-Osborn techniques in discontinuous Galerkin methods: L^2 -norm error estimates for unstructured meshes (2017). arXiv:1704.05238
28. Weller, H., et al.: Mesh adaptation on the sphere using optimal transport and the numerical solution of a Monge—Ampère type equation. *J. Comput. Phys.* **308**, 102–123 (2016)

A Global Optimization and Adaptivity-Based Algorithm for Automated Edge Grid Generation



Suzanne M. Shontz and David McLaurin

Abstract Meshes are commonly generated as a successor to geometric models in computational design and analysis. Computational analyses are performed in industry and academia for use in research, development, and manufacturing. The accuracy of these analyses depends on the fidelity of the geometric models and meshes. Element quality also plays an important role. Whereas most mesh generation strategies focus on element quality, this should be secondary to accurate representation of the real-world object. In McLaurin and Shontz (Automated edge grid generation based on arc-length optimization. In: Proceedings of the 22nd International Meshing Roundtable, pp. 385–403, Springer, Berlin, 2014), we recently proposed an automated method for generation of optimal edge grids via minimization of the arc-length deficit via a local optimization technique. In this paper, we explore the use of several different global optimization and adaptivity strategies for generation of optimal edge grids. Our results demonstrate the robustness and accuracy of our global optimization and adaptivity method for edge grid generation. We also discuss potential extensions of our method which incorporate a computational budget, achieve greater efficiency through development of a hybrid method based on local and global optimization and adaptivity, and incorporate symmetries.

1 Introduction

Meshes have become an important part of geometric design and computational analyses in industry and academia. For example, meshes have been employed in car crash simulations [1], computer-animated movies [2], and virtual simulations

S. M. Shontz (✉)

Department of Electrical Engineering and Computer Science, Institute for Information Sciences, Bioengineering Program, University of Kansas, Lawrence, KS, USA

e-mail: shontz@ku.edu

D. McLaurin

Siemens, Austin, TX, USA

of patient-specific medical treatments [3], among many other fields. The accuracy of a computational analysis depends significantly as to the digital representation of the real-world object and to its fidelity. However, it remains a real challenge to produce high-fidelity computational representations of real-world objects. In fact, up to 75% of the time to solution can be consumed in the generation of high-fidelity computational representations [4]. This work seeks to improve the process of creating a computational model of an object through acceleration of the mesh generation process.

Many mesh generation algorithms follow a traditional hierarchical procedure when generating a volume mesh in three dimensions [5–7]. That is, they first generate nodes on corners of the geometric object (in 0D), followed by edge grids (in 1D), then surface meshes (in 2D) and finally volume meshes (in 3D). Such an approach to mesh generation makes good sense given that the meshes on shared surfaces must be conformal. (For example, the surface meshes which bound the volume meshes must be conformal.)

Edge grids are typically generated using the one-dimensional version of the mesh generation algorithm. Examples of methods used to generate edge grids include the advancing front algorithm [8] and algorithms which use iterative refinement or enrichment to generate edge grids from an initial coarse configuration [9, 10]. Methods typically specify the point spacing at both ends of the curve. This ensures that the node distribution and growth ratios are compatible with the quality of the final edge grid. While *a posteriori* mesh quality improvement methods [11–14] could be applied, the goal is to generate edge grids that are of good quality.

The computation and time associated with edge grid generation are trivial when compared to that required for surface or volume mesh generation. However, the point-spacing values at the endpoints of curves typically have to be set manually to satisfy a desired length-scale for surface and volume mesh generation. Given that the fidelity of the edge grids are propagated to the surface meshes and then to the volume meshes, time spent generating an optimal edge grid is well worth the computational effort which is required.

Various researchers have generated optimal edge grids using either local or global optimization. For example, the interface between curves was first linearized in [15] in order to simplify the process of generating edge grids and surface meshes on topologically adjacent patches. Other researchers have developed geometry aware or curvature-based approaches. Examples of such approaches include the discretization of curves for use in level set methods [16], energy minimization [17], curvature minimization [18], and angle minimization [19]. Dynamic programming methods have also been used to generate optimal edge grids [20]. Several of the existing methods for optimal edge grid generation require the use of topologically adjacent geometry [21–23] or are applied to a certain class of curves (e.g., [24]).

The work here seeks to build on that in [25] and [26] to develop optimal edge grid generation algorithms based on global optimization and adaptivity. The four optimal edge grid generation algorithms developed in this paper are general-purpose algorithms, i.e., they can be applied to any “digital curve” independent

of its representation since they do not rely on the use of derivatives. In contrast, most of the algorithms for edge grid generation operate solely on a specific type of curve, e.g., NURBS, B-splines, cubic splines, etcetera. Depending on the context, various types of digital curves are most popular. For example, NURBS curves are the standard in computer-aided design, whereas parametric curves are more common in pattern recognition [27]. Within the isogeometric analysis community, T-splines have grown in popularity [28, 29]. The focus of the methods developed in this paper is the fidelity of the optimal edge grids generated via global optimization and adaptivity. Element quality is also incorporated into one of these methods. In addition, robustness and efficiency of the methods are also discussed.

2 Refinement Via Arc-Length Deficit

We are interested in the development of global optimization and adaptivity methods for generation of optimal edge grids. An edge grid is a piecewise linear representation (i.e., a *discretization*) of an analytical curve (*curve*) in $\mathbb{R}^3$. The *discretization error* refers to the accuracy of the discretization in its representation of a curve.

As noted in [25], there are several different ways in which the discretization error of an analytical curve can be defined. Similarly, there are many possible definitions for the *optimality* of an edge grid.

As in [25], we define the discretization error to be the *arclength deficit*, i.e., the difference between the arclength of the curve minus the arclength of the discretization. The latter is calculated as the sum of the edge lengths of the edge grid. An *optimal edge grid* is hence one that minimizes the arclength deficit.

We now formalize the statement of our optimization problem. As in [25], let $C(u)$ be a parameterized curve and D be a discretization of the curve with n_t points given by $P_i : i \in \{1, \dots, n_t\}$. The edge grid is comprised of $n_t - 1$ segments, $S_j : j \in \{1, \dots, (n_t - 1)\}$. Segment S_j is given by two successive parameter values, i.e., u_j and u_{j+1} . Let $L(S)$ be a function that calculates the length of a segment in physical coordinates. Then the optimization problem to be solved can be stated as follows:

$$\begin{aligned} & \underset{u_i}{\text{minimize}} \quad O = - \sum_{j=1}^{n_t-1} L_j \\ & \text{subject to} \quad u_1 = a \\ & \quad \quad \quad u_1 < u_2, \\ & \quad \quad \quad u_2 < u_3, \\ & \quad \quad \quad \vdots \\ & \quad \quad \quad u_{n_t-1} < u_{n_t}, \\ & \quad \quad \quad u_{n_t} = b. \end{aligned}$$

The above optimization problem is a mixed integer linear programming problem if the number of points and their parameter values corresponding to their positions are both unknown [25]. The problem could likely be solved by one of the standard mixed integer linear programming solvers in existence (e.g., [30–32]). However, since mixed integer linear programming problems are, in general, NP-hard, we focus instead on the development of practical algorithms that control the number of edge grid nodes through the incorporation of additional constraints.

Given that the desired number of nodes in an edge grid is not known and is difficult to compute, we add bounds on the segments' lengths in the edge grid. To this end, let e and m be lower and upper bounds on the edge lengths of the segments in physical coordinates. (Here we set $e = 10^{-10}$ and $m = \text{arclength}/2$.) Then, we arrive at the following set of constraints:

$$\begin{aligned} P_1 &= \alpha, \\ e &\leq L_1 \leq m \\ e &\leq L_2 \leq m \\ e &\leq \vdots \leq m \\ e &\leq L_{n_t-1} \leq m \\ P_{n_t} &= \beta. \end{aligned}$$

Together with these additional constraints, the optimization problem is now a linear programming problem.

This suggests the following approach for the generation of optimal edge grids. We start with an edge grid represented by a single segment which connects two nodes at the endpoints of the curve. We then follow a two-step iterative approach in order to refine the edge grid and optimize the node placement. First, the edge grid is refined in an adaptive manner in order to increase the arclength of the discretization. Then a globally optimal placement of the nodes in the edge grid is determined. This refine-then-optimize procedure is repeated until convergence is obtained, i.e., until the arclength deficit falls below the specified tolerance.

We now describe the building blocks of our refine-then-optimize algorithms. First, we describe the algorithm which we use for global optimization of the node placement in the edge grids. For ease of description, we describe the optimization problem for a given segment with $n_t = 3$ points. That is, we consider the case when an edge segment is being refined through the addition of a node.

As several edge segments may be refined within a single iteration of the adaptive refinement procedure, a single, unified global optimization procedure is used to optimally place all of the interior nodes in the edge grid at once. Note this may also result in an adjustment of the positions of the previously existing interior nodes in the edge grid.

We use the `patternsearch` command [33] within the Matlab programming environment to globally optimize the placement of nodes in the edge grid. The

Algorithm 1 Optimization algorithm with $n_t = 3$

```

1:  $n_t = 3$ 
2:  $u_i : i \in \{1, 2, 3\}$ 
3:  $u_1$  and  $u_3$  define the segment  $S_{1,3}$ 
4: procedure GLOBAL OPTIMIZATION( $S_{1,3}$ )
5:    $L(S_{1,3}) = \text{length of segment}$ 
6:   Place interior node  $u_2$  to maximize  $L(S_{1,2}) + L(S_{2,3})$  within tolerance
7: end procedure

```

specific type of direct search method employed is the generalized pattern search algorithm [34]. It is important to note that all algorithms contained in Matlab's Global Optimization Toolbox, including the generalized pattern search method, search extensively for a global minimum. However, there is no guarantee that such a minimum is returned. Instead, it may be the case that a local minimum is outputted from the method. It is important to keep this in mind when interpreting the experimental results in Sect. 3.

Next, we describe our adaptive refinement procedure.

Algorithm 2 Refinement and optimization algorithm for discretization

```

 $D(S_j) : j \in \{1, \dots, n_t\}$ 
for  $j = 1 : n_t - 1$  do
  if  $S_j$  is optimal then
    do nothing
  else
    mark  $S_j$  for refinement
  end if
end for
refine all of the marked segments (i.e., if  $S_j$  is marked for refinement, then replace  $S_j$  with  $S_{j, j+\frac{1}{2}}$  and  $S_{j+\frac{1}{2}, j+1}$ ).
optimize the node placement for  $\{S_j\}$  using Algorithm 1.

```

There are a couple of important differences to note between this approach and the one we took in [25]. First, we perform global optimization of the node placement in an all-at-once manner (as opposed to local optimization on a segment-by-segment basis). Second, our approach is iterative (instead of recursive). This is mainly due to the ease of implementing an iterative approach in the Matlab programming environment, whereas our previous approach was developed in C/C++. However, it should be noted that there are some differences between recursive and iterative approaches. Recursive solutions are typically more elegant, but can be harder to understand. In addition, recursive methods can run more slowly than iterative methods (depending on the computing environment), although this can be improved with

caching. Such methods also can run out of stack space (in extreme cases), whereas iterative methods are typically more efficient in limited memory environments. Here, memory would only be expected to play a role in larger-scale settings.

We are now ready to describe the four global optimization and adaptivity algorithms we have developed for edge grid generation. The algorithms are based on the use of Algorithm 1 for global optimization of the node placement and Algorithm 2 for the adaptive refinement. Each algorithm employs different optimality criteria within Algorithm 2 which results in different implications for the adaptivity procedure.

2.1 Edge Grid Generation Algorithm 1: Global Optimization Only

Our first algorithm for edge grid generation performs global optimization of the node placement on each iteration using Algorithm 1. However, it does not perform adaptivity in the usual sense. Instead, the algorithm adds a single node to the edge grid and adjusts the starting point of the global optimization procedure to be uniformly-spaced parameter values. The refine-then-optimize procedure is then repeated until the arclength deficit drops below a prescribed tolerance. Clearly this algorithm is not a practical one based on how the adaptivity is performed. However, it gives a baseline as to what the global optimization procedure can achieve in an “ideal” setting with the minimal number of nodes.

2.2 Edge Grid Generation Algorithm 2: Global Optimization and Adaptivity Based on Local Arclength Deficit

Our second algorithm for edge grid generation performs the standard global optimization of the node placement using Algorithm 1. Refinement in Algorithm 2 is performed whenever a local segment possesses more than its share of the error of the arclength deficit (assuming a uniform error distribution). If all segments share the error in a balanced way, then every other segment is refined. The refine-then-optimize procedure is then repeated until the arclength deficit drops below a prescribed tolerance.

2.3 *Edge Grid Generation Algorithm 3: Global Optimization and Adaptivity Based on Delta-Segment Length*

Our third algorithm for edge grid generation performs global optimization of the node placement in the standard manner using Algorithm 1. Refinement is then performed whenever the subdivision of a segment in the edge grid into two segments results in a sufficient increase in the *delta-segment length*. That is, the segment is subdivided whenever the new segments' combined length is at least a specified fraction longer than the existing segment length. (Here we set the fraction to $(1 + 0.00005 * (0.01 * X))$, where $0.01 * X$ is the convergence tolerance converted to a percentage.) Alternatively, the segment is also subdivided into two segments when the length of the new edge grid is at least a specified fraction longer than the existing edge grid. (Here the same fraction is used to control refinement based on both the local and global delta-segment lengths.) The refine-then-optimize procedure is then repeated until the arclength deficit falls below a specified tolerance.

2.4 *Edge Grid Generation Algorithm 4: Global Optimization and Adaptivity Based on Delta-Segment Length and Quality Control*

Our fourth algorithm for edge grid generation performs the same refine-then-optimize procedure on each iteration as does Edge Grid Generation Algorithm 3. However, before the start of the next iteration, the method performs an additional step for quality control. That is, it subdivides a segment into two segments whenever the growth ratio is too small or too large. (Here the lower and upper bounds for the growth ratios are set to $1 - 0.01 * X$ and $1 + 0.01 * X$.) The *growth ratio* is defined as the ratio of the length of the current segment to the length of an adjacent segment. The refine-then-optimize procedure is then repeated until the arclength deficit falls below a specified tolerance.

3 Numerical Experiments

In this section, we show the results of performing several numerical experiments to generate edge grids on various curves. Four numerical experiments were designed to test the performance of each of our methods on a set of curves. Three curves were selected in order to demonstrate various aspects of each of our methods. The first is the *butterfly* curve (Fig. 1). This curve was chosen due to its changes in curvature as well as its self-intersecting nature. The second one, i.e., the *cochleoid* curve (Fig. 2) was chosen since it is a non-self-intersecting smooth curve. The final curve we selected is the *dove2* curve (Fig. 3). This one was selected due to its sharp features

Fig. 1 Experiment 1 Results:
Butterfly Curves: 10% deficit (top),
1% deficit (middle),
0.1% deficit (bottom);
 $x(t) = \sin(t) * (\exp(\cos(t)) - 2 \cos(4t) + (\sin(t/12)^5))$,
 $y(t) = \cos(t) * (\exp(\cos(t)) - 2 \cos(4t) + (\sin(t/12)^5))$,
 $0 < t < 2\pi$. The true length of the butterfly curve is 35.6181

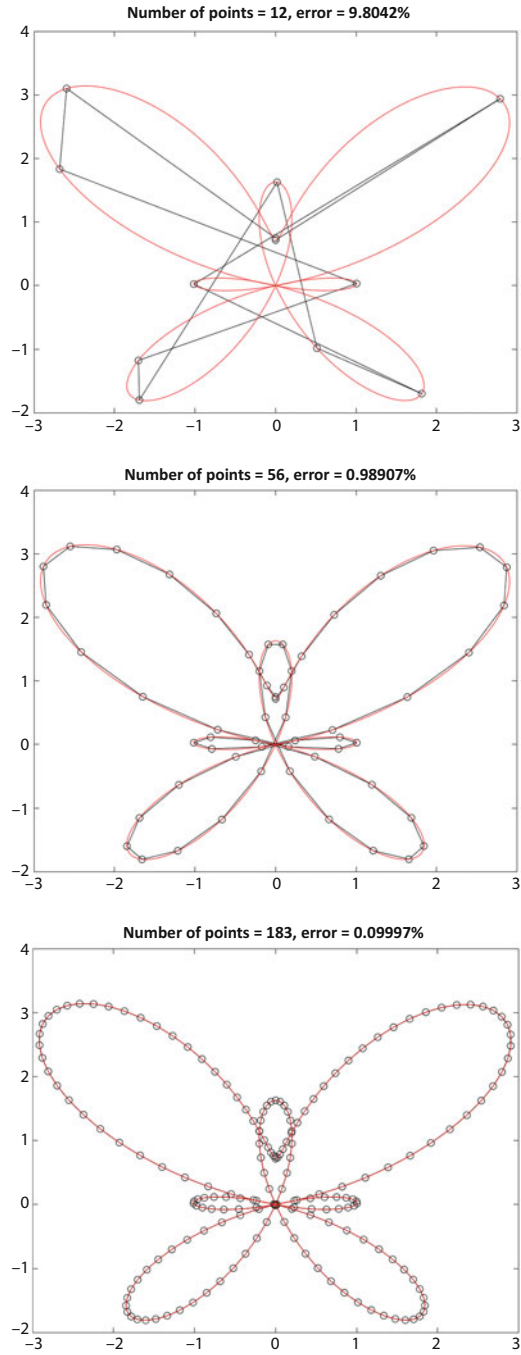


Fig. 2 Experiment 1 Results:
Cochleoid Curves: 10% deficit (top), 1% deficit (middle), 0.1% deficit (bottom);
 $x(t) = (a \sin(t) \cos(t))/t$,
 $y(t) = (a \sin(t) \sin(t))/t$,
 $a = 1, 0 < t < 2\pi$. The true length of the cochleoid curve is 2.9470

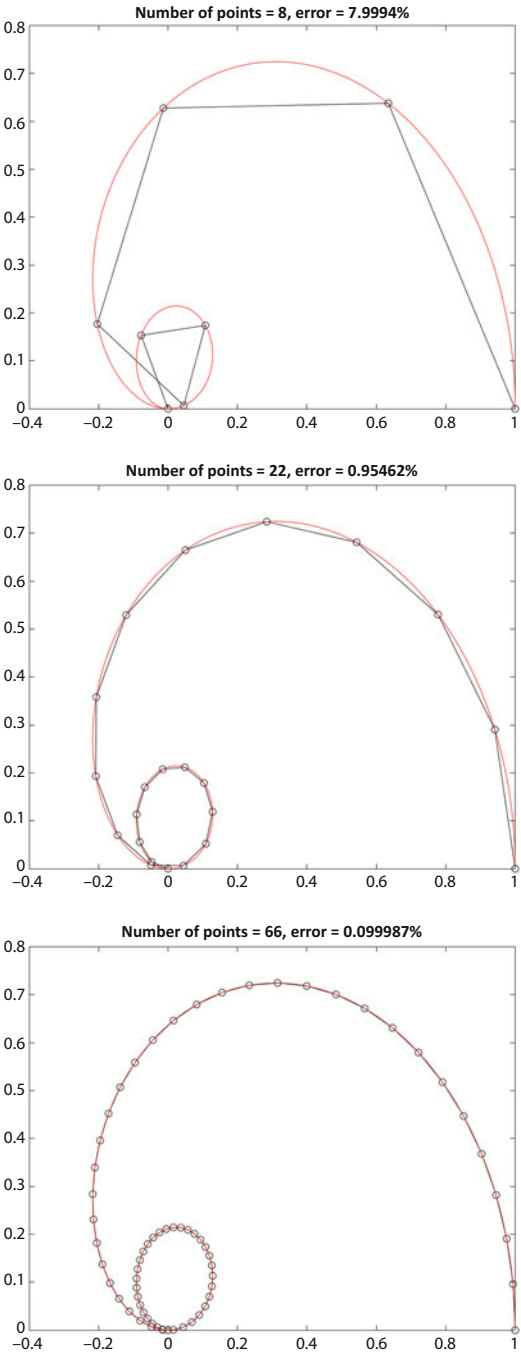


Fig. 3 Experiment 1 Results:
Dove2 Curves: 10% deficit (top), 1% deficit (middle),
0.1% deficit (bottom);
 $x(t) = \cos(at) - \cos(bt)^j$,
 $y(t) = \sin(ct) - \sin(dt)^k$,
 $a = 1, b = 100, j = 3, c = 1, d = 50, k = 3, 0 < t < \frac{\pi}{5}$.
The true length of the dove2 curve is 49.0848

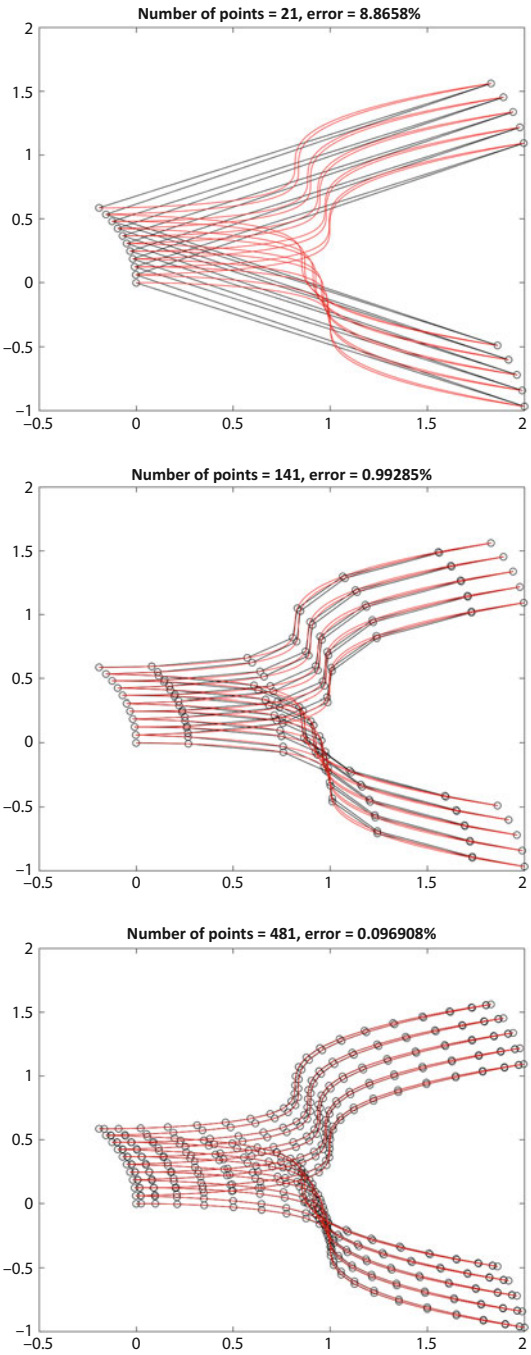


Table 1 Parameters used by the generalized pattern search method

Parameter	Value
‘MeshAccelerator’	‘on’
TolMesh	1e-3
TolFun	1e-3
MaxFunEvals	200*npoints
TolX	1e-3
MaxIter	200*npoints

and sharp changes in curvature. In the subsequent figures, the curve is shown in red, and the nodes of the edge grid are indicated with black circles. In addition, the number of nodes in the optimal edge grid and the arclength deficit are reported in the title of each plot. The true length of each curve is specified in the captions for Figs. 1, 2, and 3. While the three curves represent academic test cases, all of these features are present in real-world applications.

There were several parameters that were employed by the various edge grid generation algorithms. Table 1 summarizes the parameters used by the generalized pattern search algorithm in Matlab. The tolerances for the stopping criteria were set fairly loosely to allow for quicker convergence. (For more details, see the Mathworks documentation on the `patternsearch` command.)

Experiment 1: Global Optimization Only. The purpose of the first experiment was to test the performance of Edge Grid Generation Algorithm 1. The specified refine-then-optimize procedure was repeated until the arclength deficit fell below a prescribed tolerance (i.e., 10%, 1%, or 0.1%).

As seen in Fig. 1, Edge Grid Generation Algorithm 1 initially does a poor job of generating an edge grid which is representative of the butterfly shape. This is because it takes several shortcuts when generating the edge grid as opposed to following the longer path traced by its shape. This is, however, corrected when the arclength deficit is decreased further.

An interesting feature arises when Edge Grid Generation Algorithm 1 is used to generate an edge grid for the cochleoid curve. As observed in Fig. 2, the initial edge grid is self-intersecting despite the curve itself being non-self-intersecting. As the arclength deficit is decreased further, the edge grids that are generated are non-self-intersecting.

Edge Grid Generation Algorithm 1 does a reasonable job of generating edge grids which represent the dove2 geometry as shown in Fig. 3. It does the best job initially of resolving the geometry where the sharp features are present. However, the method eventually reproduces both the sharp and non-sharp features as the arclength deficit is decreased further.

Since this is not a practical algorithm for edge grid generation, we do not concern ourselves with the number of function evaluations taken by the generalized pattern search method.

Experiment 2: Global Optimization and Adaptivity Based on Local Arclength Deficit. The purpose of this experiment is to test the performance

of Edge Grid Generation Algorithm 2. The refine-then-optimize procedure was repeated until the arclength deficit drops below the prescribed tolerance (i.e., 10%, 1%, or 0.1%).

As seen in Fig. 4, Edge Grid Generation Algorithm 2 does a better job of distributing the edge grid over the entire butterfly shape initially. The edge grids do a better job of representing the shape of the butterfly as the representation deficit is decreased. However, compared to the edge grids generated by Edge Grid Generation Algorithm 1, more nodes are placed in regions of higher curvature, and fewer nodes are placed in regions of lower curvature.

The generated edge grids for the cochleoid curve are shown in Fig. 5. The results for this curve are similar to those for Edge Grid Generation Algorithm 1. The one difference that is observed is that the adaptivity portion of Edge Grid Generation Algorithm 2 incorporated additional nodes into the edge grid with the 1% arclength deficit.

The generated edge grids for the dove2 curve are shown in Fig. 6. They incorporated numerous nodes in the adaptivity portion of the method. Interestingly, the highest percentage of nodes are concentrated on the outermost curve. The pattern search method converged to rather different solutions compared to those reported in Experiment 1.

Table 2 shows the number of function evaluations required by the pattern search method during the global optimization procedure in Experiment 2. Typically, a greater number of function evaluations are required to determine optimal node placement via the pattern search method for the edge grids with a smaller arclength deficit. However, this is not always the case. For example, the results demonstrate that for the dove2 curve, fewer function evaluations were needed to determine optimal node placement for the edge grids with the 1% and 0.1% deficit than for the one with the 10% deficit. This is despite there being greater than three times (and approximately thirteen times) as many nodes in the edge grid with the 1% deficit (and the 0.1% deficit). This was only observed with the use of Edge Grid Generation Algorithm 2 and is suggestive of different convergence behavior.

Experiment 3: Global Optimization and Adaptivity Based on Delta-Segment Length. The purpose of this experiment is to test the performance of Edge Grid Generation Algorithm 3. The refine-then-optimize procedure was repeated until the arclength deficit fell below a specified tolerance (i.e., 10%, 1%, or 0.1%).

Edge Grid Generation Algorithm 3 does the best job of generating edge grids which most accurately represent the butterfly shape as seen in Fig. 7. However, rather irregular spacing can also be observed between adjacent nodes. This suggests the need to incorporate a mechanism for controlling the grid quality.

The edge grids generated by Edge Grid Generation Algorithm 3 for the cochleoid curve (Fig. 8) are similar to those generated by the previous methods. Good quality edge grids have been generated.

Fig. 4 Experiment 2 Results:
Butterfly Curves: 10% deficit
(top), 1% deficit (middle),
0.1% deficit (bottom)

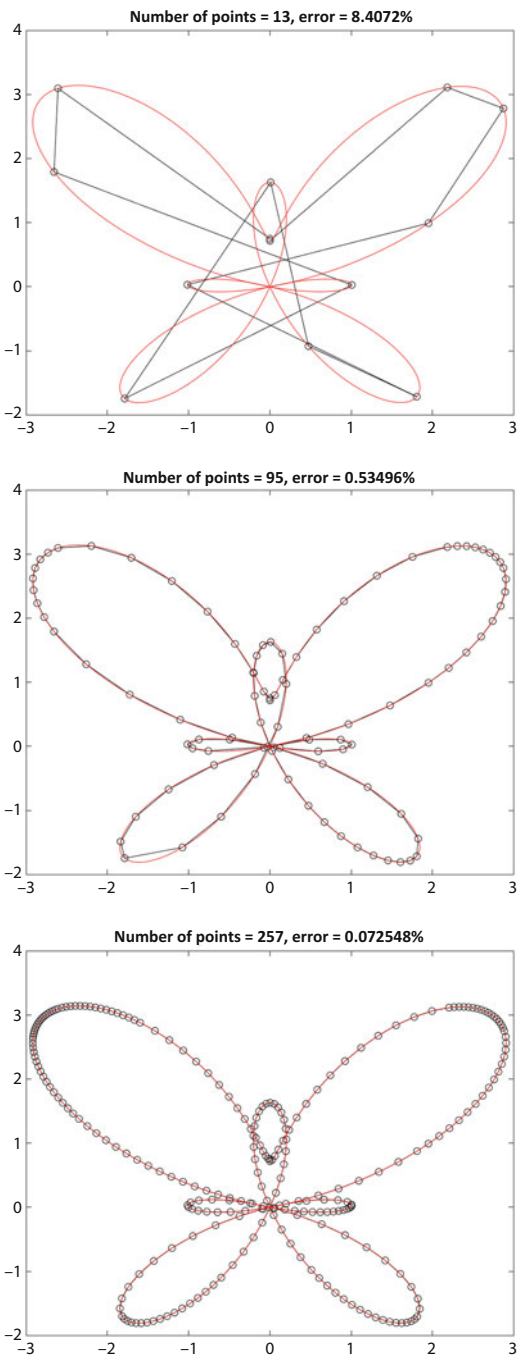


Fig. 5 Experiment 2 Results:
Cochleoid Curves: 10%
deficit (top), 1% deficit
(middle), 0.1% deficit
(bottom)

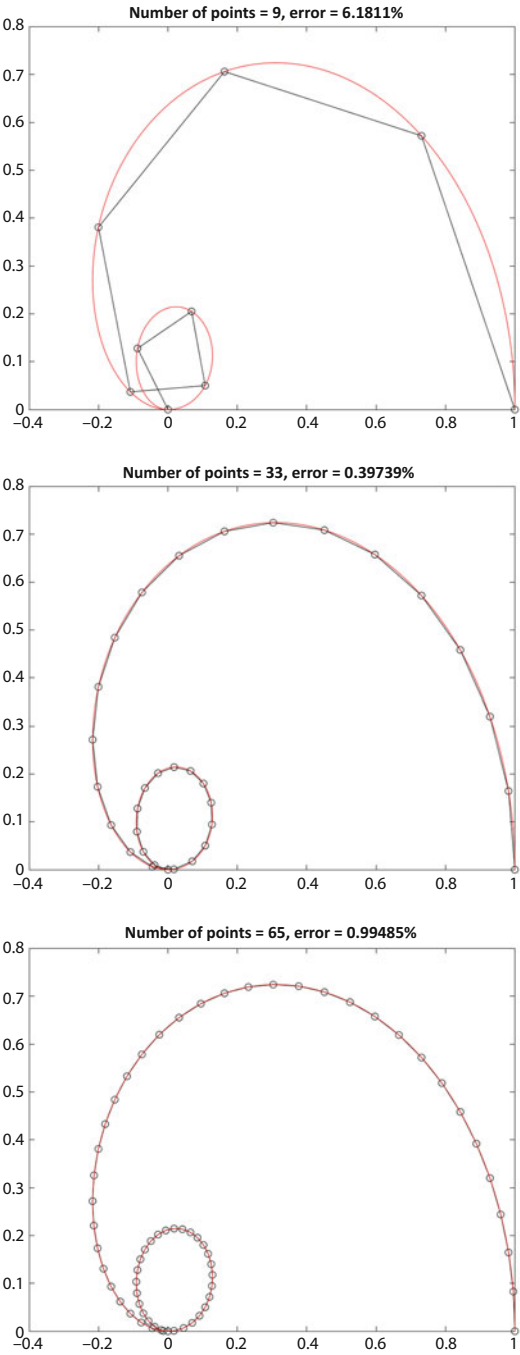


Fig. 6 Experiment 2 Results:
Dove2 Curves: 10% deficit (top),
1% deficit (middle),
0.1% deficit (bottom)

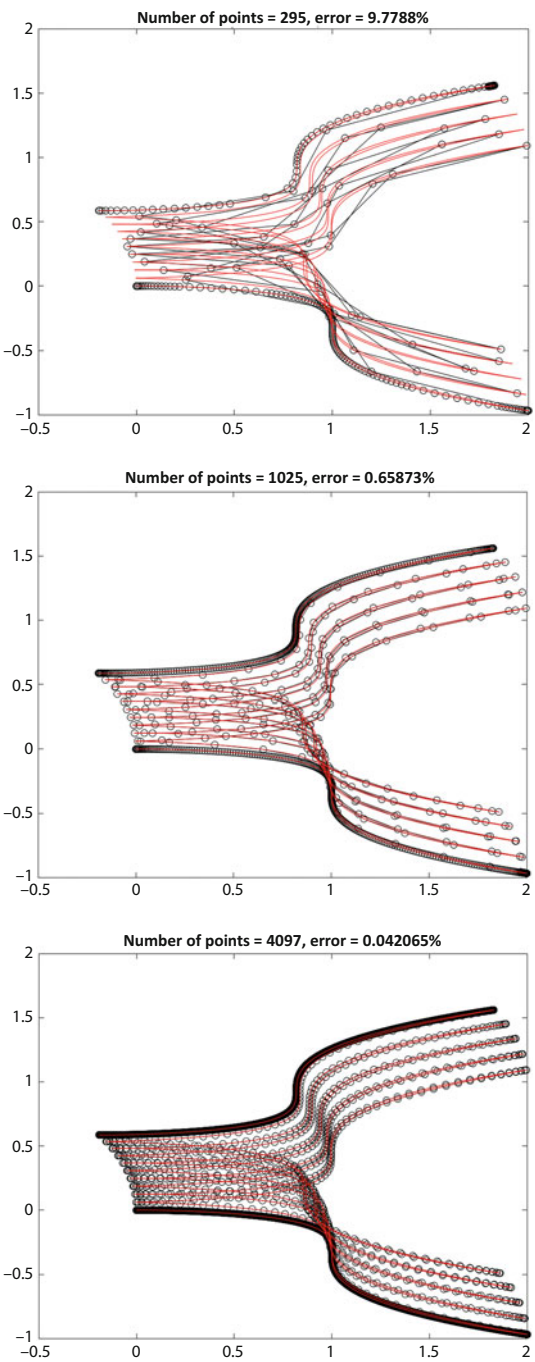


Table 2 Experiment 2
Results: Number of function evaluations

	Butterfly	Cochleoid	dove2
10% deficit	1511	526	100
1% deficit	1517	723	83
0.1% deficit	2797	725	87

The edge grids for the dove2 curve which were generated by Edge Grid Generation Algorithm 3 (Fig. 9) appear similar to those generated by Edge Grid Generation Algorithm 2. However, the edge grids for Edge Grid Generation Algorithm 3 are more efficient in that they use many fewer nodes.

Table 3 shows the number of function evaluations required by the pattern search method during the global optimization procedure for Experiment 3. In this case, the number of function evaluations required by the pattern search algorithm for node placement increased as the generated edge grids decreased their arclength deficit. However, for the butterfly and dove2 curves, the additional number of function evaluations required for node placement was rather minimal despite the increased number of nodes present in the more accurate edge grids. This was also true for the cochleoid curves with 1% and 0.1% arclength deficit. This suggests that the accuracy gained is well worth the additional computational effort for Edge Grid Generation Algorithm 3.

Experiment 4: Global Optimization and Adaptivity Based on Delta-Segment Length and Quality Control. The purpose of this experiment is to test the performance of Edge Grid Generation Algorithm 4. The refine-then-optimize procedure was repeated until the arclength deficit fell below a specified tolerance (i.e., 10%, 1%, or 0.1%).

As seen in Fig. 10, the 10% and 1% arclength deficit butterfly edge grids generated by Edge Grid Generation Algorithm 4 required significantly more nodes for a fixed arclength deficit than did those generated by Edge Grid Generation Algorithm 3. This is not surprising given the demonstrated need to control the edge grid quality through the growth ratio. (Note this was not the case for the 0.1% arclength deficit edge grid generated by this algorithm, as it already demonstrated good quality and appropriate growth ratios.) The same behavior was exhibited by the cochleoid edge grids that were generated by this algorithm (see Fig. 11). While this is fine for these curves, a more difficult or noisy curve may require the addition of a very high number of nodes, which would cause inefficiency. This was the case for the dove2 edge grids (see Fig. 12). Hence, proper selection of the parameters used to perform the refinement based on the growth ratios will be key. The resulting edge grids also demonstrate that the quality control should also be applied to the final iteration (instead of simply exiting once the prescribed arclength deficit has been met).

Fig. 7 Experiment 3 Results:
Butterfly Curves: 10% deficit
(top), 1% deficit (middle),
0.1% deficit (bottom)

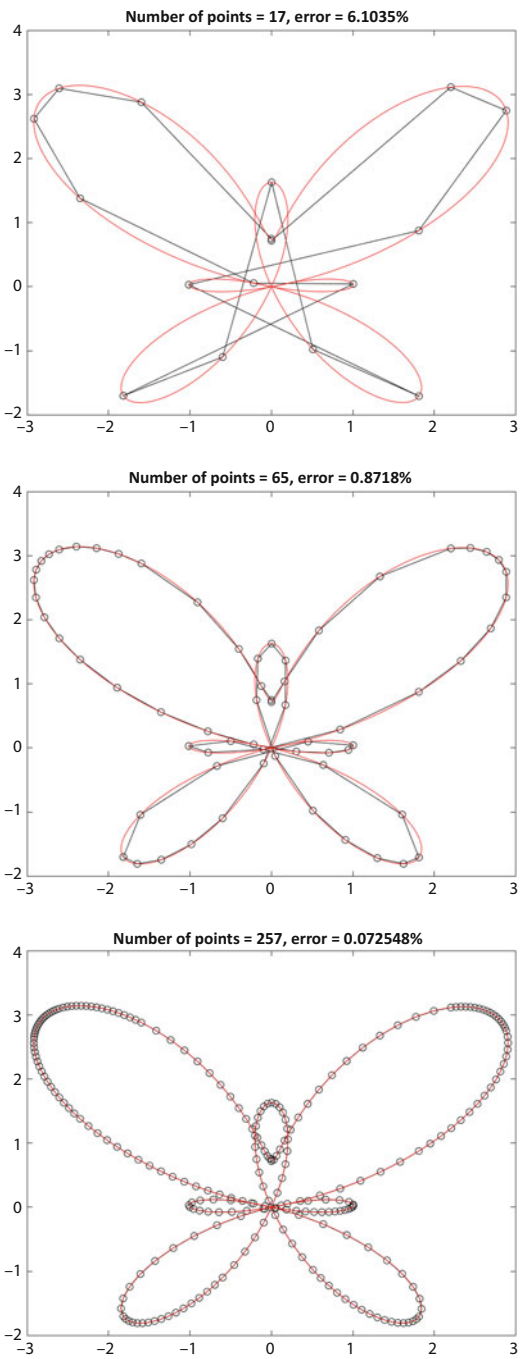


Fig. 8 Experiment 3 Results:
Cochleoid Curves: 10%
deficit (top), 1% deficit
(middle), 0.1% deficit
(bottom)

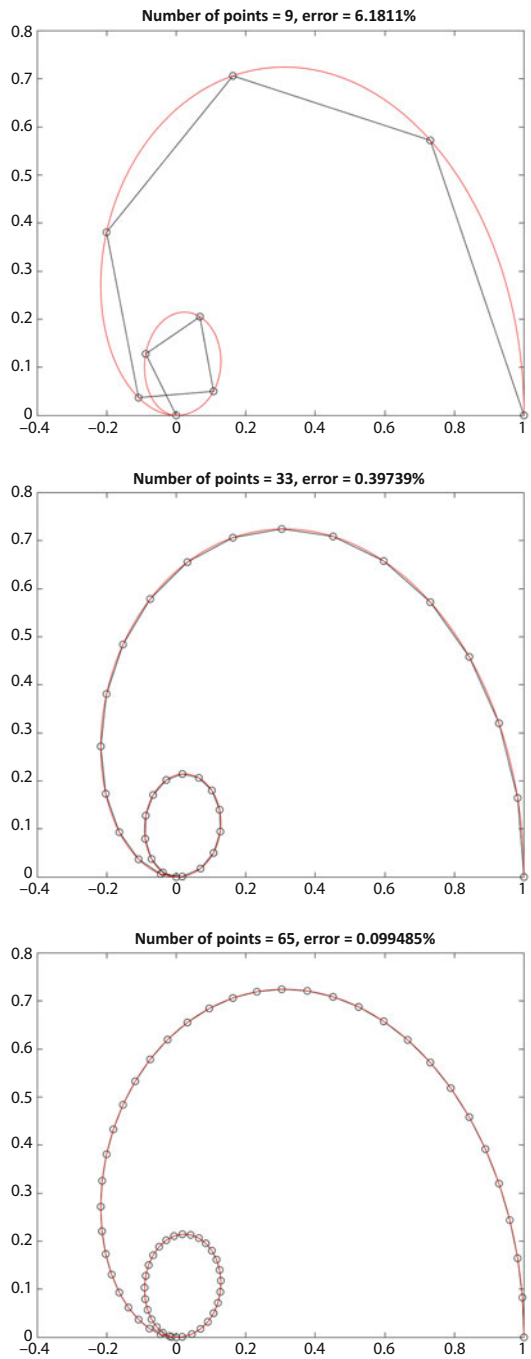


Fig. 9 Experiment 3 Results:
Dove2 Curves: 10% deficit (top),
1% deficit (middle),
0.1% deficit (bottom)

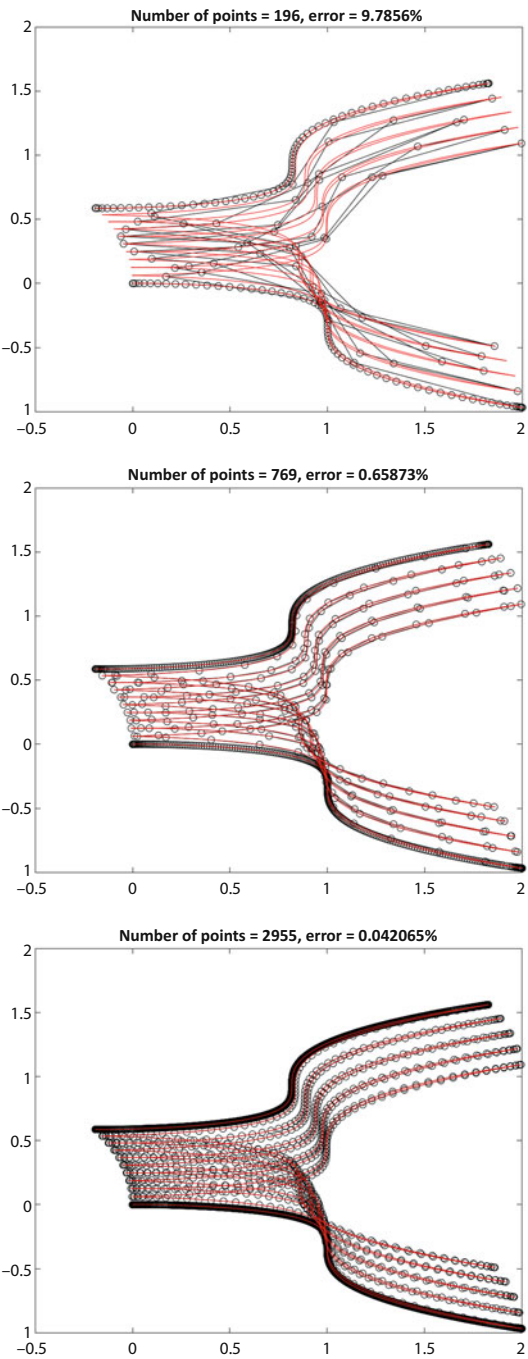


Table 3 Experiment 3
Results: Number of function
evaluations

	Butterfly	Cochleoid	dove2
10% deficit	2789	526	79
1% deficit	2793	723	83
0.1% deficit	2797	725	87

Table 4 shows the number of function evaluations required by the pattern search method during the global optimization procedure for Experiment 4. Compared to the results shown in Table 3, the algorithm sometimes required fewer and sometimes required more function evaluations for the optimization procedure to generate a particular edge grid. This suggests that the optimization procedure is rather sensitive to starting point.

The convergence plots of all of the edge grid generation algorithms are shown in Fig. 13. Note that in each case, the convergence plot for Edge Grid Generation Algorithm 4 is for the experiment with the 0.1% arclength deficit. As observed in Fig. 13, the convergence of the various methods for the butterfly and cochleoid test cases is similar. However, very different paths to convergence are demonstrated by the various edge grid generation methods for the dove2 test case. The pattern search method is sensitive to the choice of parameters, and different solutions have been obtained by the various methods. This suggests that there is still room for designing improved edge grid generation algorithms, particularly with respect to how the adaptation is performed and its interaction with the optimization method.

4 Conclusions and Future Work

In this paper, we presented several different algorithms for edge grid generation based on global optimization and adaptivity. All of the methods performed the global optimization step using a generalized pattern search method in Matlab. Adaptivity was then performed in one of several different ways which attempted to control the amount of error in the discretized curve either locally or globally. Edge grid quality was also controlled by the incorporation of a refinement step based on the growth ratios of each segment. The results show that in each case, the generated edge grid is optimal with respect to arclength deficit. In addition, the algorithms have been shown to be robust with respect to self-intersections, sharp features, and abrupt changes in curvature. Which edge grid is preferred for a specific geometry depends on the computational budget (e.g., in regards to the number of nodes in the edge grid or the number of function evaluations required by the pattern search method), the desired nodal spacing, and other

Fig. 10 Experiment 4
Results: Butterfly Curves:
10% deficit (top), 1% deficit
(middle), 0.1% deficit
(bottom)

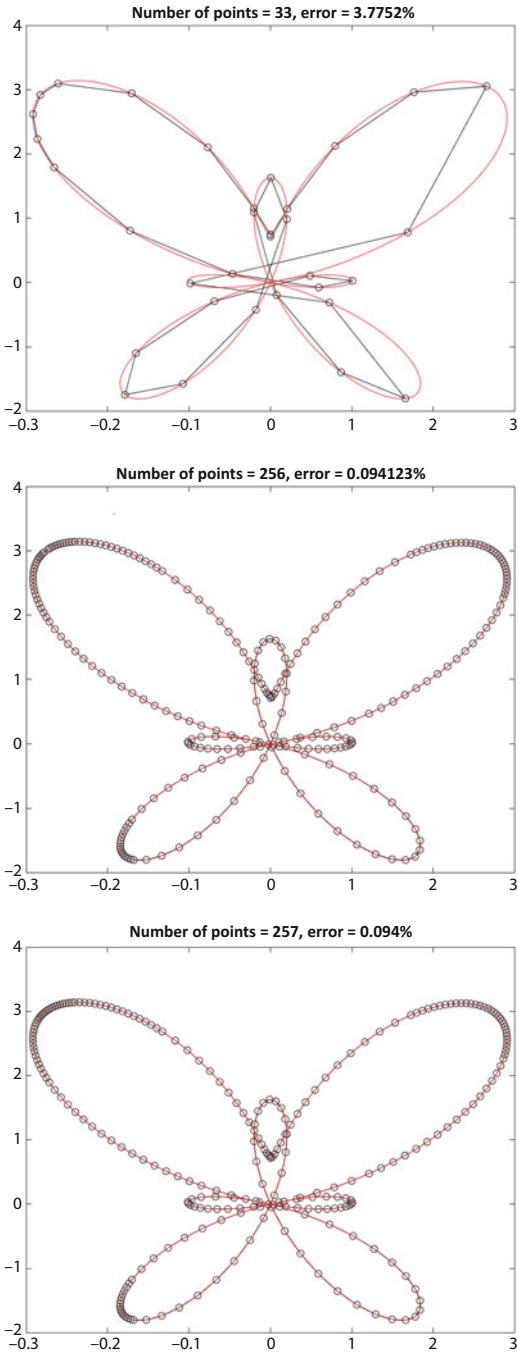


Fig. 11 Experiment 4
Results: Cochleoid Curves:
10% deficit (top), 1% deficit
(middle), 0.1% deficit
(bottom)

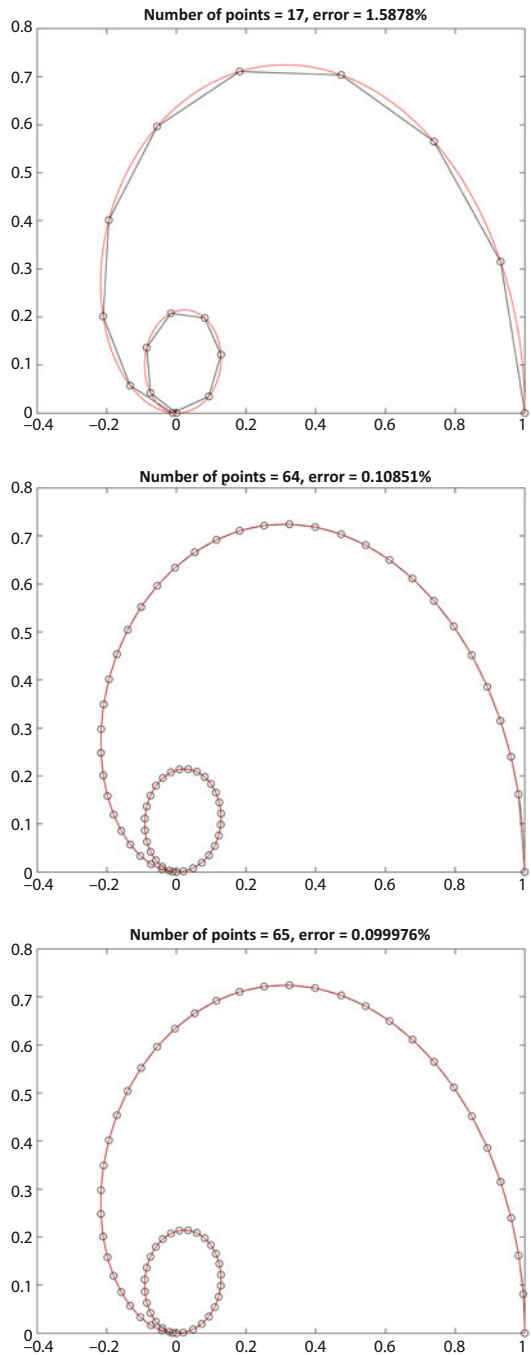


Fig. 12 Experiment 4
Results: Dove2 Curves: 10%
deficit (top), 1% deficit
(middle), 0.1% deficit
(bottom)

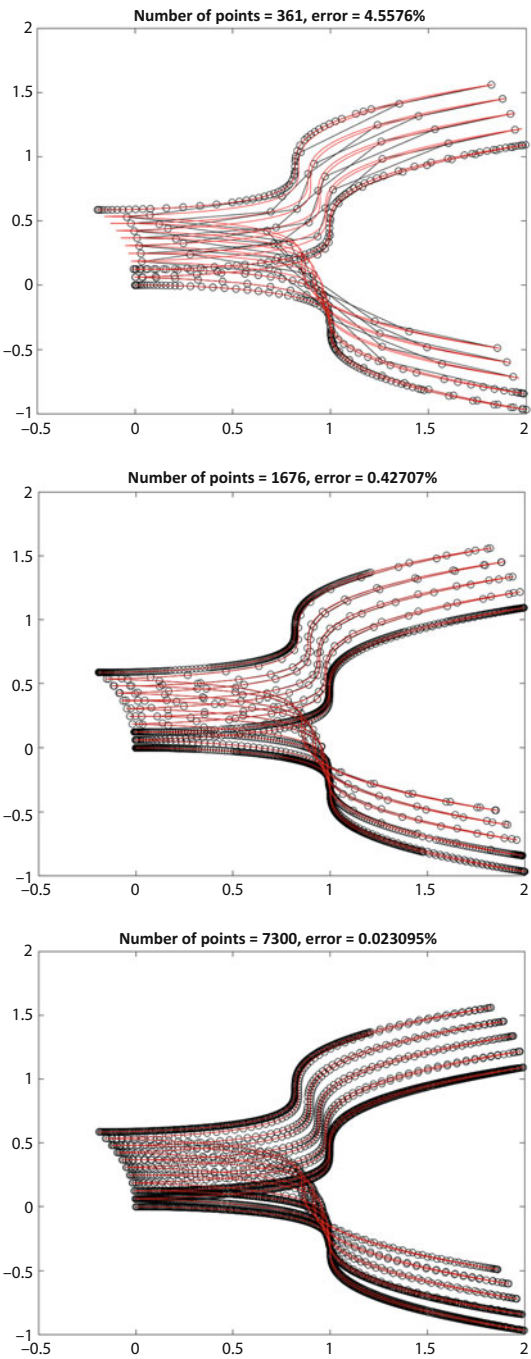


Table 4 Experiment 4
Results: Number of function
evaluations

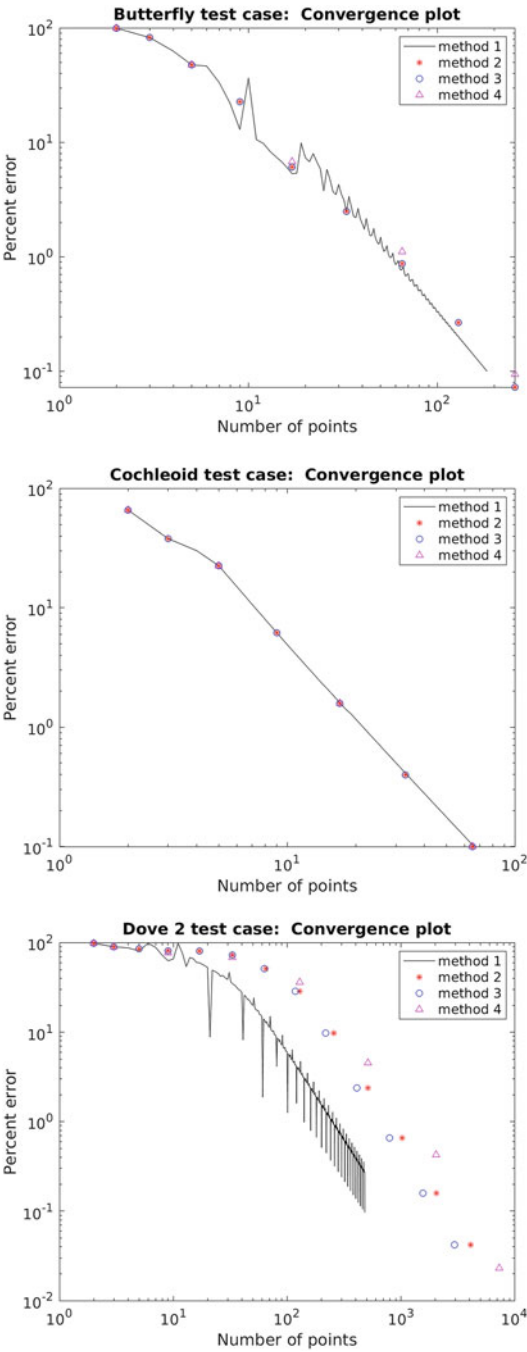
	Butterfly	Cochleoid	dove2
10% deficit	645	709	86
1% deficit	2549	711	87
0.1% deficit	2549	711	89

factors that may be present when working with a real-world application. However, absent any additional information, the desired optimal edge grid is one that balances the grid quality with the computational effort invested in its generation. Additional investigation is needed in order to determine improved values of the parameters which control the global optimization and adaptivity portions of each algorithm. In particular, autotuning of the parameters may be an interesting avenue for further exploration. While the results presented here were shown in two dimensions, the methods can also be used to generate edge grids in three dimensions.

There are a few possibilities for extensions of this work. First, it would be interesting to incorporate the concept of a computational budget into the methods. For example, how would one go about generating an optimal edge grid given a maximum number of nodes in the edge grid or a maximum number of function evaluations that the pattern search method could perform? Second, it is likely possible to generate a more efficient automated edge grid generation method by combining our previous method which was based on local optimization and adaptivity [25] with one of the current methods which is based on global optimization and adaptivity. For example, the global method could handle aspects such as the overall shape of the curve, whereas the local method could focus on the smaller features. Alternatively, the local method could be used when farther from an optimal solution, and the global method could then be used when closer to an optimal edge grid. Finally, it would be interesting to investigate methods designed to incorporate symmetries into the generation of the optimal edge grids. This would potentially benefit applications with symmetry and reduce the computational expense.

Acknowledgments The work of the first author was funded in part by NSF CAREER Award ACI-1500487 (formerly ACI-1330056 and OCI-1054459) and NSF grants OAC-1808553 and CCF-1717894. The authors would like to thank the anonymous referee for his/her careful reading of the paper and for his/her helpful suggestions which strengthened it.

Fig. 13 Convergence Plots for the Butterfly, Cochleoid, and Dove2 Test Cases



References

1. Hickey, A., Xiao, S.: Finite element modeling and simulation of car crash. *Int. J. Mod. Stud. Mech. Eng.* **3**, 1–5 (2017)
2. Villemin, R., Herv, C., Konishi, S., Tejima, T., Villemin, R., Yu, D.: Art and technology at Pixar, from toy story to today. In: *SA '15: SIGGRAPH Asia 2015 Courses*, pp. 1–89 (2015)
3. Aycock, K., Campbell, R., Manning, K., Sastry, S., Shontz, S., Lynch, F., Craven, B.: A computational model for predicting inferior vena cava filter performance on a patient-specific basis. *J. Biomech. Eng.* **136**, 081003 (2014)
4. Bischoff, S., Kobbelt, L.: Structure preserving CAD model repair. *Comput. Graph. Forum* **24**(3), 527–536 (2005)
5. Cheng, S., Dey, T., Shewchuk, J.: *Delaunay Mesh Generation*. CRC Press, New York (2012)
6. Marcum, D.: Advancing-front/local-reconnection (AFLR) unstructured grid generation. In: *Computational Fluid Dynamics Review*, p. 140. World Scientific, Singapore (1998)
7. CUBIT: Geometry and Mesh Generation Toolkit. <http://cubit.sandia.gov>
8. Löhner, R., Parikh, P.: Generation of three-dimensional unstructured grids by the advancing-front method. *Int. J. Numer. Meth. Fl.* **8**(10), 1135–1149 (1988)
9. Shewchuk, J.: Tetrahedral mesh generation by Delaunay refinement. In: *Proceedings of the 14th Annual Symposium on Computational Geometry*, pp. 86–95 (1998)
10. Shewchuk, J.: Delaunay refinement algorithms for triangular mesh generation. *Comput. Geom. Theory Appl.* **22**, 21–74 (2002)
11. Freitag, L., Knupp, P.: Tetrahedral mesh improvement via optimization of the element condition number. *Int. J. Numer. Meth. Eng.* **53**, 1377–1391 (2001)
12. Munson, T.: Mesh shape-quality optimization using the inverse mean-ratio metric. *Math. Program.* **110**, 561–690 (2007)
13. Kim, J., Panitanarak, T., Shontz, S.: A multiobjective mesh optimization framework for mesh quality improvement and mesh untangling. *Int. J. Numer. Meth. Eng.* **94**, 20–42 (2013)
14. Sastry, S., Shontz, S., Vavasis, S.: A log-barrier method for mesh quality improvement and untangling. *Eng. Comput.* **30**, 315–329 (2014)
15. Laug, P., Borouchaki, H.: Curve linearization and discretization for meshing composite surfaces. *Commun. Numer. Meth. Eng.* **20**, 869–876 (2004)
16. Macklin, P., Lowengrub, J.: An improved geometry-aware curvature discretization for level set method: Applications to tumor growth. *J. Comput. Phys.* **215**, 392–401 (2006)
17. Hofer, M., Pottmann, H.: Energy-minimizing splines in manifold. In: *ACM Transactions on Graphics (TOG)—Proceedings of the ACM SIGGRAPH 2004*, vol. 23, pp. 284–293 (2004)
18. El-Zehiry, N., Grady, L.: Fast global optimization of curvature. In: *Computer Vision and Pattern Recognition (Princeton, NJ)*, pp. 3257–3264. IEEE, New York (2010)
19. Ebeida, M., Davis, R., Freund, R.: A new fast hybrid adaptive grid generation technique for arbitrary dimensional domains. *Int. J. Numer. Meth. Eng.* **84**, 305–329 (2010)
20. Horng, J., Li, J.: An automatic and efficient dynamic programming algorithm for polygonal approximation of digital curves. *Pattern Recogn. Lett* **23**, 171–182 (2002)
21. Quadros, W., Owen, S., Brewer, M., Shimada, K.: Finite element mesh sizing for surfaces using skeleton. In: *Proceedings of the 13th International Meshing Roundtable* (2004)
22. Cabello, J.: Towards quality surface meshing. In: *Proceedings of the 12th International Meshing Roundtable* (2003)
23. Cunha, A., Canann, S., Saigal, S.: Automatic boundary sizing for 2D and 3D meshes. In: *Proceedings of the 6th International Meshing Roundtable* (1997)
24. Cuilliere, J.: A direct method for the automatic discretization of 3D parametric curves. *Comput.-Aided Des.* **29**, 639–647 (1997)
25. McLaurin, D., Shontz, S.: Automated edge grid generation based on arc-length optimization. In: *Proceedings of the 22nd International Meshing Roundtable*, pp. 385–403. Springer, Berlin (2014)

26. McLaurin, D.: Automated, curvature based edge-grid generation. In: Proceedings of AlaSim 2012 (Huntsville, AL), Alabama Modeling and Simulation Council (2012)
27. Sarfraz, M.: Interactive Curve Modeling: With Applications to Computer Graphics, Vision, and Image Processing. Springer, Berlin (2008)
28. Hughes, T., Cottrell, J., Bazilevs, Y.: Isogeometric analysis: CAD, finite elements, NURBS, exact geometry, and mesh refinement. *Comput. Methods Appl. Mech. Eng.* **194**, 4135–4195 (2005)
29. Bazilevs, Y., Calo, V., Cottrell, J., Evans, J., Hughes, T., Lipton, S., Scott, M., Sederberg, T.: Isogeometric analysis using T-splines. *Comput. Methods Appl. Mech. Eng.* **199**, 229–263 (2010)
30. Nemhauser, G., Savelsbergh, M., Sigismondi, G.: MINTO, a Mixed INTEger Optimizer. *Oper. Res. Letters* **15**, 47–58 (1994)
31. MOSEK homepage. www.mosek.com/index.php
32. Ralphs, T., Güzelsoy, M.: The Symphony callable library for mixed integer programming. In: The Next Wave in Computing, Optimization, and Decision Technologies, Operations Research/Computer Science Interfaces Series, vol. 29, pp. 61–76 (2005)
33. Hooke, R., Jeeves, T.: “Direct search” solution of numerical and statistical problems. *J. ACM* **8**, 212–229 (1961)
34. Audet, C., Dennis, J.: Analysis of generalized pattern searches. *SIAM J. Optim.* **13**, 889–903 (2003)